DSA27 · CS2101 · IS122

Data Structures and Algorithms

Lectures, question bank and worked answers

The students' book

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Fall 2026

Contents

How to use this book 9

The course 10

Course guide 11

Study plan 17

Fifteen weeks 21

Week 1 — Why This Course, and Why Python 22
- Before We Start . . . 22
- Programming Languages Are Not the Same . . . 23
- Comparing Languages Honestly . . . 29
- Why Python for This Course . . . 31
- From a Variable to a Data Structure . . . 32
- Cost Is the Whole Point . . . 34
- The Course Itself . . . 36
- Homework 1 . . . 40
- Summary . . . 40
- Questions for Week 1 . . . 41

Week 2 — Complexity, and the Array 48
- Where We Are . . . 48
- What Do We Count? . . . 48
- Big-O, Big-Ω, Big-Θ . . . 51
- Best, Worst, Average . . . 53
- Reading the Cost Off the Code . . . 55
- The Array . . . 56
- The Course Array . . . 59
- The Rule, From Today . . . 63
- This Week . . . 65
- Summary . . . 66
- Questions for Week 2 . . . 67

Week 3 — Recursion 78
- Where We Are . . . 78
- The Idea . . . 78
- The Call Stack . . . 80

Recursion in Practice . . . 82
What Recursion Costs . . . 84
Branching Recursion . . . 87
Recursion or a Loop? . . . 89
This Week . . . 90
Summary . . . 91
Questions for Week 3 . . . 92

Week 4 — Dynamic Arrays **102**
Where We Are . . . 102
Capacity and Size . . . 102
Growing . . . 104
How Much to Grow? . . . 105
Amortised Analysis . . . 106
Shrinking . . . 109
Python's `list` . . . 110
All the Operations . . . 111
This Week . . . 112
Summary . . . 113
Questions for Week 4 . . . 114

Week 5 — Linked Lists **120**
Where We Are . . . 120
Nodes and References . . . 120
Walking the List . . . 122
Inserting . . . 123
Removing . . . 125
Reversing in Place . . . 126
Array Against Linked List . . . 127
Variants . . . 128
This Week . . . 130
Summary . . . 131
Questions for Week 5 . . . 132

Week 6 — Stacks **139**
Where We Are . . . 139
The Stack ADT . . . 139
Two Honest Implementations . . . 140
Application 1: Balanced Brackets . . . 142
Application 2: Postfix . . . 142
Application 3: Infix to Postfix . . . 143
Application 4: The Call Stack . . . 145
This Week . . . 146
Summary . . . 146
Questions for Week 6 . . . 147

Week 7 — Queues **154**
Where We Are . . . 154
The Queue ADT . . . 154
The Obvious Design . . . 155
The Ring Buffer . . . 157
Two More Honest Queues . . . 160
Measured . . . 161
Where Queues Are Used . . . 163
This Week . . . 164
Summary . . . 165
Questions for Week 7 . . . 165

Week 8 — Searching **174**
Where We Are . . . 174
Linear Search . . . 174
Binary Search . . . 175
Bounds: Searching for Positions . . . 178
Three More Searches . . . 179
Measured . . . 181
This Week . . . 183
Summary . . . 184
Questions for Week 8 . . . 185

Week 9 — Basic Sorting **192**
Where We Are . . . 192
Two Words Before Any Algorithm . . . 193
Three $O(n^2)$ Sorts . . . 194
Watching an Algorithm . . . 198
Best, Worst, Average . . . 200
Beyond Comparisons . . . 202
This Week . . . 204
Summary . . . 205
Questions for Week 9 . . . 206

Week 10 — Advanced Sorting **214**
Where We Are . . . 214
Divide and Conquer . . . 215
Merge Sort . . . 215
Quicksort . . . 219
Heap Sort . . . 223
Measured . . . 226
What Python Actually Uses . . . 227
Stability . . . 228
This Week . . . 229
Summary . . . 230
Questions for Week 10 . . . 232

Week 11 — Trees **239**
Where We Are . . . 239
The Vocabulary . . . 240
Binary Trees . . . 241
Binary Search Trees . . . 243
Delete: Three Cases . . . 245
The Four Traversals . . . 248
When the Tree Is a Linked List . . . 253
This Week . . . 255
Summary . . . 256
Questions for Week 11 . . . 257

Week 12 — Heaps and Priority Queues **265**
Where We Are . . . 265
The Priority Queue ADT . . . 266
The Heap . . . 267
Sift Up: `push` . . . 269
Sift Down: `pop` . . . 271
Build-Heap in $O(n)$ . . . 273
The Priority Queue on Top . . . 275
Heap Sort, Revisited . . . 278
This Week . . . 279
Summary . . . 280
Questions for Week 12 . . . 281

Week 13 — Hash Tables **289**
Where We Are . . . 289
The Dream . . . 290
Hash Functions . . . 291
Collisions . . . 293
Chaining . . . 294
Open Addressing . . . 296
Load Factor and Resizing . . . 300
Measured . . . 301
The Real Thing . . . 304
This Week . . . 305
Summary . . . 306
Questions for Week 13 . . . 307

Week 14 — Graphs and Graph Searches **315**
Where We Are . . . 315
The Vocabulary . . . 316
Two Representations . . . 318
Breadth-First Search . . . 320
Depth-First Search . . . 323
Using the Searches . . . 326

Two Real Graphs . . . 330
Beyond the Bylaw . . . 331
This Week . . . 331
Summary . . . 332
Questions for Week 14 . . . 333

Week 15 — The Principles of Language Translation **341**
Where We Are . . . 341
What a Translator Does . . . 342
Stage 1: The Lexer . . . 343
Grammars . . . 344
Stage 2: Recursive Descent . . . 346
Stage 3: Walking the Tree . . . 351
Two Routes, One Answer . . . 352
Measured . . . 353
Where This Grows Up . . . 354
The Course, Looking Back . . . 355
This Week . . . 356
Summary . . . 358
Questions for Week 15 . . . 359

Answers **366**

Answers — Week 1 **367**
Part A — Multiple choice . . . 367
Part B — Short answer and essay . . . 369
Part C — Trace the code . . . 370
Part D — Find and fix the bug . . . 371
Part E — Write the code . . . 371

Answers — Week 2 **373**
Part A — Multiple choice . . . 373
Part B — Short answer and essay . . . 374
Part C — Complexity analysis . . . 376
Part D — Proofs . . . 376
Part E — Array state . . . 377
Part F — Trace the code . . . 378
Part G — Find and fix the bug . . . 378
Part H — Write the code . . . 379

Answers — Week 3 **381**
Part A — Multiple choice . . . 381
Part B — Short answer and essay . . . 382
Part C — Recurrences . . . 384
Part D — Trace the code and the call stack . . . 385
Part E — Find and fix the bug . . . 386

Part F — Write the code . . . 386

Answers — Week 4 **388**
Part A — Multiple choice . . . 388
Part B — Short answer and essay . . . 389
Part C — Trace the growth . . . 390
Part D — Proofs . . . 391
Part E — Find and fix the bug . . . 391
Part F — Write the code . . . 392

Answers — Week 5 **394**
Part A — Multiple choice . . . 394
Part B — Short answer and essay . . . 395
Part C — Trace the code . . . 396
Part D — List state . . . 397
Part E — Complexity analysis . . . 398
Part F — Find and fix the bug . . . 398
Part G — Write the code . . . 399

Answers — Week 6 **401**
Part A — Multiple choice . . . 401
Part B — Short answer and essay . . . 402
Part C — Trace the code and the algorithms . . . 403
Part D — Find and fix the bug . . . 404
Part E — Write the code . . . 405

Answers — Week 7 **408**
Part A — Multiple choice . . . 408
Part B — Short answer and essay . . . 409
Part C — Trace the code . . . 410
Part D — Ring state . . . 412
Part E — Complexity analysis . . . 413
Part F — Find and fix the bug . . . 413
Part G — Write the code . . . 414

Answers — Week 8 **417**
Part A — Multiple choice . . . 417
Part B — Short answer and essay . . . 418
Part C — Trace the algorithms . . . 419
Part D — Complexity analysis . . . 421
Part E — Find and fix the bug . . . 421
Part F — Write the code . . . 422

Answers — Week 9 **425**
Part A — Multiple choice . . . 425
Part B — Short answer and essay . . . 426
Part C — Trace the algorithms . . . 428

Part D — Array state . . . 429
Part E — Complexity analysis . . . 430
Part F — Find and fix the bug . . . 431
Part G — Write the code . . . 432

Answers — Week 10 **435**
Part A — Multiple choice . . . 435
Part B — Short answer and essay . . . 436
Part C — Trace the algorithms . . . 438
Part D — Array state . . . 440
Part E — Complexity analysis . . . 441
Part F — Find and fix the bug . . . 442
Part G — Write the code . . . 443

Answers — Week 11 **447**
Part A — Multiple choice . . . 447
Part B — Short answer and essay . . . 449
Part C — Trace the code . . . 450
Part D — Tree state — draw every step . . . 452
Part E — Complexity analysis . . . 454
Part F — Find and fix the bug . . . 455
Part G — Write the code . . . 456

Answers — Week 12 **459**
Part A — Multiple choice . . . 459
Part B — Short answer and essay . . . 460
Part C — Trace the code . . . 462
Part D — Heap state — draw every step . . . 464
Part E — Complexity analysis . . . 465
Part F — Find and fix the bug . . . 465
Part G — Write the code . . . 466

Answers — Week 13 **469**
Part A — Multiple choice . . . 469
Part B — Short answer and essay . . . 470
Part C — Trace the code . . . 472
Part D — Table state: draw every step . . . 474
Part E — Complexity analysis . . . 475
Part F — Find and fix the bug . . . 476
Part G — Write the code . . . 477

Answers — Week 14 **480**
Part A — Multiple choice . . . 480
Part B — Short answer and essay . . . 481
Part C — Trace the algorithms . . . 483
Part D — Graph state . . . 485
Part E — Complexity analysis . . . 486

Part F — Find and fix the bug . . . 487
Part G — Write the code . . . 488

Answers — Week 15 **493**
Part A — Multiple choice . . . 493
Part B — Short answer and essay . . . 494
Part C — Trace the code . . . 496
Part D — Trees and stacks: draw every step . . . 497
Part E — Complexity analysis . . . 499
Part F — Find and fix the bug . . . 499
Part G — Write the code . . . 500

Mock Exam — Weeks 1 to 3 — practice paper **505**
Section A — Multiple choice (30 marks) . . . 505
Section B — Written answers (30 marks) . . . 505
Marking yourself . . . 506

Mock Exam — Weeks 1 to 7 — practice paper **507**
Section A — Multiple choice (30 marks) . . . 507
Section B — Written answers (30 marks) . . . 507
Marking yourself . . . 508

Mock Exam — Weeks 1 to 15 — practice paper **509**
Section A — Multiple choice (30 marks) . . . 509
Section B — Written answers (30 marks) . . . 509
Marking yourself . . . 510

How to use this book

This book is the whole of DSA27 on paper: the fifteen lecture handouts, and after each one the question bank for that week. The answers to every question are together at the back, in Part III, with the practice papers.

The order to use it in, each week.

1. **Read the week's chapter before the lecture**, or straight after it. The handout holds everything on the slides plus what is said out loud.
2. **Do the lab.** The lab manual walks you through building that week's structure, one method at a time. The lab is where the course is learned.
3. **Answer the week's questions on paper, without a computer.** The final exam is written, on paper, with no computer: practise the way you will be tested.
4. **Only then turn to the answers** at the back. Mark yourself honestly, and look at which *kind* of question you lost marks on. Every question is tagged **[what]** (recall), **[how]** (apply) or **[why]** (explain and justify), and the exam leans on **[why]**.

What the question codes mean. `W8-M03` is week 8, multiple-choice question 3. The letters are: **M** multiple choice · **E** short answer and essay · **T** trace the code · **K** complexity analysis · **P** proofs · **R** recurrences · **S** draw the state after every step · **B** find and fix the bug · **C** write the code. The **C** problems are real exercises in the course repository: `practice/weekNN.py`, checked by `pytest`.

The storage rule. From Week 2 on, every structure you build keeps its data only in the course `Array`, in node objects, or in another structure you built yourself — never in a Python `list`, `dict` or `set`. Lecture 02 explains why. You will see the rule at work in every chapter.

The repository. Everything in this book comes from https://github.com/helghareeb/DSA27, where you also find the code skeletons, the tests, the notebooks, the slides and the worked solutions. When the book names a file such as `dsa/stack.py`, it is a file in that repository.

Announcements are on the course WhatsApp channel; video is on https://www.youtube.com/@0xHGH.

Honesty about the answers. Every trace, output and number in the answers was produced by running the code, not written from memory. If you find an answer you believe is wrong, say so in the lecture or on the channel: you will be right sometimes, and the book will be corrected.

The course

Course guide

> *Every rule on this page comes from **the لائح** **that governs your own program**, with the page number printed beside it. All four bylaws are in this repository under `regulations/`. Where this guide and your bylaw disagree, **the bylaw is right** — tell me and I will fix the guide.*

---

1. Which course are you registered in?

Three programs, three bylaws, **one course**. Find your row.

Your program	Code	Bylaw	Spec page
Artificial Intelligence	**CS2101**	AI, 2020	p. 44
Bioinformatics / Medical Informatics	**IS122**	Medical Informatics, 2014	p. 35
Software Engineering	**IS122**	Software Engineering, 2013	p. 38

> ***On the name "Bioinformatics".*** *There is no Bioinformatics* program *at the faculty — the one usually meant is **Medical Informatics (المعلوماتية الطبي)**, and its bylaw is the 2014 file above. Bioinformatics is a* course *inside it (MED121, p. 45). If you were told you are in Bioinformatics, the 2014 bylaw is yours.*

Side by side

	AI (2020)	**Bio / Med. Inf. (2014)**	**SWE (2013)**
Code	**CS2101**	**IS122**	**IS122**
Arabic name	هياكل البيانات و الخوارزميات	هياكل البيانات وتحليل الخوارزميات	هياكل البيانات وتحليل الخوارزميات
Credit hours	**3**	**3**	**3**
Weekly hours	2 lecture + 2 lab	2 lecture + 2 lab	2 lecture + 2 lab
Prerequisite	**CS1002** Object Oriented Programming	**CS012**, **MATH012**	**CS012**, **MATH012**
Level / semester	Level 2, Semester 3	Level 2 *(semester not stated)*	Sophomore, Semester 1

	AI (2020)	Bio / Med. Inf. (2014)	SWE (2013)
Credits to graduate	**138**	**135**	**135**

Different codes. Same subject, same hours, same room, same exam paper.

---

2. What the ﷺ says this course contains

The **2013 and 2014** texts are identical, word for word (p. 38 / p. 35):

> *"Introduce the fundamental concepts of data structures and the algorithms that proceed from them. Topics include **recursion**, the underlying philosophy of **object-oriented programming**, fundamental data structures (including **stacks, queues, linked lists, hash tables, trees, and graphs**), the basics of **algorithmic analysis**, and an introduction to the **principles of language translation**."*

The **2020 AI** text (p. 44) is longer and names more structures:

> *"This course provides fundamental data structures, algorithms, and **abstract data types** using mix of programming and theory. Main topics include data structures such as **arrays, lists, linked lists, stacks, queues, hash tables, heaps, priority queues, graphs, and trees**. It introduces algorithms such as those that are used for **list manipulation, graph searches, sorting, searching, and tree traversals**. Also, it introduces **analyzing and managing the complexity** associated with data structures and their operations."*

This course covers the union of both, so that every student in the room gets everything their own bylaw promises and a little more. `02-coverage.md` maps each declared topic to the module that implements it and the test that grades it.

Two things worth knowing about where this course sits

It assumes less than you might fear. The bylaw says this course *teaches* recursion and the philosophy of object orientation — it does not assume them. If you are in Bio, nothing before this course taught you either: CS012 is structured programming only, and MATH012 *Discrete Structures* has no recursion, induction or asymptotics in it (2014, p. 30). That is exactly why Week 3 is recursion and why nothing here assumes you have written a class before.

For two of the three programs, this is the only algorithms course you will ever take. The AI bylaw has a follow-on — **AI3001 Analysis and Design of AI Algorithms** (2020, p. 48). The 2013 and 2014 bylaws have **none at all**. So if you are in SWE or Bio, what you do not learn here, you will not be taught anywhere else in your degree. Plan accordingly.

What this course unlocks

Course	AI	Bio	SWE
Database Systems	IS2102	IS123	IS123
Computer Networks	—	IT131	IT131
Computer Vision	—	IT132	IT132
Pattern Recognition	AI3301	IT137	—
Computer Graphics	AI2102	—	—
Software Construction	—	—	**SWE141**

SWE141 (2013, p. 42) is worth singling out: "BNF and basic theory of grammars and parsing… formal languages". That is where this course's "principles of language translation" grows up.

---

3. How you are assessed

All three bylaws agree on the three rules that matter.

Rule	Value
Final written exam	**60%** of the course mark
Coursework (أعمال فصلي)	the other **40%**
To pass	**≥ 60% overall and ≥ 30% of the final exam**
Attendance	**≥ 75%** of lectures and labs, or محروم

Sources: SWE 2013 pp. 11–13, p. 18 · Med. Inf. 2014 pp. 11–13, p. 18 · AI 2020 pp. 14–16, p. 19.

The two rules that fail students

1 — The double threshold. You need **both**. A student who collects 35 of the 40 coursework marks and then scores 25% on the final has **failed**, however survivable the total looks. Coursework cannot rescue a final exam below 30%.

2 — Attendance. 75% of a 15-week course is roughly three sessions you can miss. Not five. Below that you are recorded as محروم and are not allowed into the final exam at all — it is not a penalty applied to your mark, it is a locked door.

How the 40% is split

The bylaws set bounds, not a fixed split:

- **SWE 2013 (p. 13) and Bio 2014 (p. 12):** midterm **≥ 20%**
- **AI 2020 (p. 16):** midterm **≥ 10%**, final **≥ 50%**
- All three: no single component may **exceed 60%**

> ***Not yet fixed.*** *The exact split for this course will be announced in the lecture and on the WhatsApp channel, and written here once it is. A split of* ***Final 60 · Midterm 20 · Coursework and practical 20*** *satisfies all three bylaws and is what I intend to propose. Until it is announced,*

> *treat only the four rules in the table above as binding.*

Grade scale

Grade	Points	Percentage	Grade	Points	Percentage
A+	4.0	≥ 97%	C+	2.3	73 – < 76%
A	4.0	93 – < 97%	C	2.0	70 – < 73%
A–	3.7	89 – < 93%	C–	1.7	67 – < 70%
B+	3.3	84 – < 89%	D+	1.3	64 – < 67%
B	3.0	80 – < 84%	D	1.0	60 – < 64%
B–	2.7	76 – < 80%	**F**	0.0	**< 60%**

Minimum pass in any course is **D**. Graduation needs a cumulative GPA of at least **2.00 / 4.00**, across **135** credit hours (SWE, Bio) or **138** (AI).

One conflict, documented rather than hidden: the 2013 bylaw values A+ at 4.33 on p. 14 and at 4.00 on p. 57. The other three documents all say 4.00, so 4.00 is used here. See `regulations/dsa-in-your-program.md`.

---

4. What you need before week 2

```
git clone https://github.com/helghareeb/DSA27.git
cd DSA27

py -3.13 -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt

pytest -m "not challenge"       # every one of these must pass
```

You also need **Graphviz** installed as a program — the `dot` binary, not only the pip package. The environment tests check for it.

Full setup notes, including the Windows *"python opens the Microsoft Store"* trap and the VS Code debugging setup, are in the repository `README.md`.

---

5. How the course runs

Lecture (2 h/week)	The idea, its cost, and why it exists.
Lab (2 h/week)	You implement it. The skeletons in `dsa/` raise `NotImplementedError`.

Tests	`tests/` defines what "correct" means. `pytest -m challenge`.
Solutions	`solutions/` solves every exercise taught so far — for **after** you have tried. `pytest --solutions` runs the tests on them.
Notebooks	One per week, in `notebooks/`, wired to the drawing helpers.
Visualisation	`viz/` draws, animates and measures. Already written — use it.

The rule that shapes everything, from `dsa/__init__.py`:

> ***No module may use the built-in it is reimplementing.***

`dsa/dynamic_array.py` may not be backed by a `list`. `dsa/hashmap.py` may not be backed by a `dict`. Routing around this may make tests pass; it will not make you a programmer.

The fifteen weeks are laid out in `01-study-plan.md`.

Asking for help

1. Read the failing test. It states the contract.
2. Set a breakpoint and step through it — **F5**, *pytest: current test file*.
3. Then ask, on the WhatsApp channel, with the failing test name and the error text.

"It does not work" is not a question. "`test_pop_from_empty_raises` expects `IndexError` and I am getting `None`" is.

---

6. From next year: the 2026 bylaw

The faculty has a new bylaw — `FCIS-Bylaw-2026.pdf`, 328 pages, covering all seven programs. It replaces all three of the above with a single **CS201**: 3 credit hours, prerequisite CS101, Sophomore Fall, assessed 20/15/5/60, a major requirement in every program, and 142 credit hours to graduate (pp. 36, 138).

It does not govern you. It is in the repository so you can see where the faculty is going, and so this material is ready for the students who arrive under it.

---

7. Links

Repository	https://github.com/helghareeb/DSA27
WhatsApp channel	https://whatsapp.com/channel/0029Vb8XynEFy72KWbH6vS2V
YouTube	https://www.youtube.com/@0xHGH

Study plan	`01-study-plan.md`
Coverage matrix	`02-coverage.md`
Lecture 01	`../lectures/01-why-this-course/lecture.md`
Your program's regulations	`regulations/`

Study plan

Fifteen teaching weeks, because that is what the لائح sets: "خمسة عشر أسبوعا" — fifteen weeks of teaching per main semester (SWE 2013, p. 8; Medical Informatics 2014, p. 7). The **midterm** falls in the window set by the faculty calendar; the date is announced on the WhatsApp channel, not fixed here.

The plan covers the **union** of what all three bylaws declare, so every student gets everything their own program promises. Which bylaw asks for what is in `02-coverage.md`.

Every week names the module you implement and the test file that grades it, so this plan and the repository cannot quietly drift apart.

Labs, weeks 1–3. Alongside the first three lectures, the labs bring everyone to the same Python: Lab 01 (interpreter, numbers, strings, lists), Lab 02 (control flow, functions, errors, modules) and Lab 03 (data structures, classes, generators). Exercises in `labs/`, graded by `tests/test_lab01.py` to `tests/test_lab03.py`. See the lab manual.

The plan

Wk	Topic	Implement	Graded by
1	Why this course · ADTs · OOP philosophy · Know-How	—	Homework 1
2	**Complexity, and the Array** — Big-O, measuring, the course `Array`	`dsa/array_ops.py`	`tests/test_array_ops.py`
3	**Recursion** — base case, recursive case, the call stack	`dsa/recursion.py`	`tests/test_recursion.py`
4	Dynamic arrays · amortised cost	`dsa/dynamic_array.py`	`tests/test_dynamic_array.py`
5	Linked lists — insert, remove, reverse	`dsa/linked_list.py`	`tests/test_linked_list.py`
6	Stacks — balanced brackets, shunting-yard	`dsa/stack.py`	`tests/test_stack_queue.py`

Wk	Topic	Implement	Graded by
7	Queues — the O(n) vs O(1) demonstration	`dsa/queue.py`	`tests/test_stack_queue.py`
8	Searching — 7 variants · *midterm window*	`dsa/searching.py`	`tests/test_searching.py`
9	Basic sorting · counting sort	`dsa/sorting.py`	`tests/test_sorting.py`
10	Advanced sorting — merge, quick, heap	`dsa/sorting.py`	`tests/test_sorting.py`
11	**Trees** and the four traversals	`dsa/tree.py`	`tests/test_tree.py`
12	**Heaps** and priority queues	`dsa/heap.py`	`tests/test_heap.py`
13	Hash tables — chaining and open addressing	`dsa/hashmap.py`	`tests/test_hashmap.py`
14	**Graphs** and graph searches	`dsa/graph.py`	`tests/test_graph.py`
15	**Principles of language translation**	`dsa/translation.py`	`tests/test_translation.py`

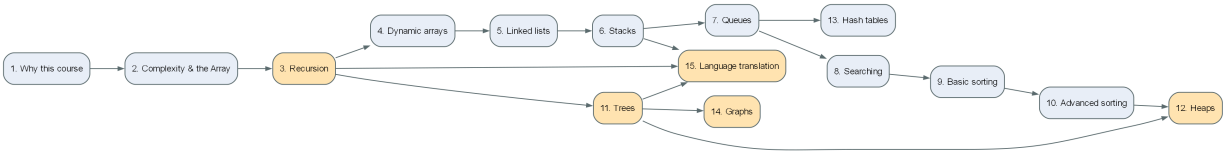


Figure 1: The dependency graph. An arrow means the later week genuinely needs the earlier one.

Week by week

1 — Why this course, and why Python. Languages are sets of decisions. Where C came from and the book to own. Paradigms, type systems, memory models. The ladder from variable to data structure. ADT vs implementation. *Know-how.* → Lecture 01

2 — Complexity, and the Array. Big-O, Θ and Ω; best, average and worst case; space against time; the rules for reading cost off code — including the O(n) hidden inside one line of Python. Then the part most courses skip: `measure()` and `plot_growth()` from `viz/complexity.py`, so you *see* what Big-O throws away. The second half is the **array**: contiguous memory, O(1) indexing, O(n) insertion, and the course `Array` (`dsa/array.py`) — the one storage primitive every structure from here on is built on. **From this week, no structure may store its data in a `list`, `dict` or `set`.** → Lecture 02 *Declared as "the basics of algorithmic analysis" (2013/2014), "analyzing and managing the complexity" and "arrays" (2020).*

3 — Recursion. Base case, recursive case, the call stack, and why Python stops you at about a thousand frames. Fast exponentiation to show that *how* you recurse decides the complexity. Towers of Hanoi for branching recursion. `merge_sorted` here is half of merge sort in week 10. → Lecture 03 *Declared first in the 2013/2014 list of topics — and taught nowhere before this course in any of the three programs.*

4 — Dynamic arrays. An array that grows. You build `DynamicArray` on the course `Array` from Week 2 — not on a `list`. Doubling on resize, and why that makes append **amortised** O(1). Plot `resize_count` and watch the argument become a picture. → Lecture 04

5 — Linked lists. Nodes and references. Why insertion at the head is O(1) and indexing is O(n) — the exact mirror of an array. Reversal in place. This is the week to use the debugger and watch `node.next` move. → Lecture 05

6 — Stacks. LIFO, built on your own `DynamicArray` with the top at the end. Then the applications that justify it: balanced-bracket checking, and infix → postfix by the shunting-yard algorithm — which is the first half of week 15. → Lecture 06

7 — Queues. FIFO. `SlowQueue` (`DynamicArray.pop(0)`, O(n)) and `CircularQueue` (a ring on an `Array`, O(1)) are deliberately paired in `dsa/queue.py` so you measure the difference rather than be told about it. You need a queue again in week 11 for level-order traversal. → Lecture 07

8 — Searching. *(midterm window)* Linear, binary, binary recursive, `lower_bound`, `upper_bound`, jump, exponential, interpolation. Eight functions, one idea: what a sorted invariant buys you. → Lecture 08 · mock exam, weeks 1–7

9 — Basic sorting. Bubble, selection, insertion — each written twice, once plain and once as a generator yielding snapshots, so `viz/animate.py` can animate *your* code. Then counting sort, which beats the O(n log n) bound by not comparing at all. → Lecture 09 · Lab 09

10 — Advanced sorting. Merge sort (divide and conquer, stable, O(n) extra space), quicksort (in place, and why the pivot strategy decides between O(n log n) and $O(n^2)$), heap sort — which is week 12 arriving early. → Lecture 10 · Lab 10

11 — Trees. Terminology, binary trees, binary search trees. Insert, search, and the delete with three cases. The four traversals: in-order (which comes out **sorted**), pre-order (which **rebuilds** the tree), post-order (which **frees** it), and level-order (which needs a **queue**, not the call stack). Why an unbalanced BST degrades into a linked list. → Lecture 11 · Lab 11

12 — Heaps and priority queues. The heap property, and the fact that the array *is* the tree — `viz.draw.draw_array_as_tree` shows this with no conversion at all. `sift_up`, `sift_down`, and build-heap in O(n) rather than O(n log n). Then a priority queue on top, and the link back to heap sort. → Lecture 12 · Lab 12

13 — Hash tables. Hashing, collisions, load factor, resizing. Chaining against open addressing with tombstones — both are in `dsa/hashmap.py`. Why "O(1) average" is a promise about the *average*, and what breaks it. → Lecture 13 · Lab 13

14 — Graphs and graph searches. Adjacency list against adjacency matrix, paired so you measure the trade-off. BFS, DFS recursive and DFS with your own stack. Shortest path by edge count, connected components, cycle detection — where the directed and undirected cases genuinely

differ — and topological sort. → Lecture 14 · Lab 14

15 — The principles of language translation. The capstone, and the one week that needs everything in the course: a **stack** to evaluate postfix, **recursion** to parse, and a **tree** to hold the result. `tokenize` → `Parser` → an AST → `evaluate`. Precedence turns out not to be a table of numbers but the *shape* of the grammar. *Declared in the 2013/2014 bylaws, and the foundation of SWE141 Software Construction (2013, p. 42).* → Lecture 15 · Lab 15

Two notes on scope

Dynamic programming is enrichment, not examinable. `dsa/dynamic_programming.py` is scaffolded and ready — `fib_naive` / `fib_memo` / `fib_table`, coin change, LCS, edit distance, knapsack, LIS. **No bylaw declares it for this course**, so it is not examined here.

It is still worth your weekend, and for a different reason depending on who you are. If you are in **AI**, you meet this material again in **AI3001 Analysis and Design of AI Algorithms**, and arriving having already seen it is a real advantage. If you are in **SWE or Bio**, there is no later algorithms course in your program at all — so this file is your only structured route into divide-and-conquer, greedy methods and dynamic programming. I would rather tell you that than let you find out in an interview.

Where this stops. Balanced trees (AVL, red-black), weighted shortest paths (Dijkstra), minimum spanning trees and NP-completeness are beyond what any of the three bylaws declare for this course. They are named here so you know the words, and know they exist, and know where to look next.

Grading, in one line

Final **60%** · coursework **40%** — and you need **≥ 60% overall**, **≥ 30% of the final**, and **≥ 75% attendance**. Full detail and sources: `00-course-guide.md`.

Fifteen weeks

Week 1 — Why This Course, and Why Python

> ***How to read this document.*** *This is the handout for Lecture 01. It contains everything that was on the slides, plus the parts I said out loud and the sources for every claim. If a statement comes from the faculty bylaw, the page number is printed next to it so you can check it yourself. You should check it yourself.*
>
> *Slides:* `DSA27-L01-slides.pdf` *· Repository:* *https://github.com/helghareeb/DSA27*

Before We Start

Where everything lives

What	Where
Code, exercises, these notes	https://github.com/helghareeb/DSA27
Course updates, announcements	WhatsApp channel
Some — not all — video lectures	https://www.youtube.com/@0xHGH

Announcements go to the WhatsApp channel first. The repository is the source of truth for anything technical: if the channel and the repository disagree, the repository is right and I have made a mistake in the channel.

The QR code for the channel is in `docs/assets/whatsapp-channel-qr.jpg`, and on the links page, `docs/links.md`.

The one thing to do today

```
git clone https://github.com/helghareeb/DSA27.git
cd DSA27
py -3.13 -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
pytest -m "not challenge"
```

Every one of those tests must pass **before** you write a line of course code.

Those tests do not check your algorithms. They check your machine: your interpreter version, that you are inside a virtual environment, the plotting stack, the Graphviz `dot` binary, and the

drawing helpers in `viz/`. When one fails, the message names exactly what is missing. Fix that first. Debugging a linked list on a broken environment is a waste of your evening.

Full setup instructions, including the Windows "Microsoft Store opens instead of Python" trap, are in the repository `README.md`.

Programming Languages Are Not the Same

The argument that sounds right

Every language can print `Hello, World`.

Every language can add two numbers.

Every language is, in the end, reduced to instructions the same processor executes.

So all programming languages are the same. Only the syntax differs.

This is the most common thing an intelligent second-year student believes, and I want to spend real time on it, because the rest of the course depends on taking it apart.

The argument is not stupid. It has a true premise: in terms of *what can be computed*, the languages you will meet are equivalent — they are all Turing-complete, so any program you can write in one, you can write in another. That is a genuine theorem and it is worth knowing.

The error is in the word "same". Turing-equivalence tells you the set of computable functions is identical. It tells you **nothing** about what the code costs to write, what it costs to read six months later, what it costs to run, which mistakes it lets you make silently, or how many people can maintain it.

Why it is wrong

Turing-equivalence says: **the same problems are solvable.**

It does not say they are solvable:

- at the same **cost**
- by the same number of **people**
- in the same amount of **time**
- with the same number of **mistakes surviving to production**

A language is not a neutral pipe you pour logic through. A language is a set of **decisions** — about what is easy, what is hard, what is forbidden, and what fails loudly versus what fails silently. Those decisions were made by people, for reasons, and the reasons are usually written down.

A language is a set of decisions

$$\text{Programming Language} \longrightarrow \text{Design} \longrightarrow \text{Philosophy}$$

Ask of any language: **what did its designers refuse to do, and why?**

That chain — language, design, philosophy — is the one I drew on the board. Read it right to left and it explains the language. Read it left to right and it explains the code you are looking at.

The decisions have owners

Person	Language	The decision, in one line
Dennis Ritchie	**C**	A portable assembler. Trust the programmer.
Bjarne Stroustrup	**C++**	Leave no room for a lower-level language.
Guido van Rossum	**Python**	Readability is worth paying for.
Graydon Hoare	**Rust**	Memory safety without a garbage collector.
Pike, Thompson, Griesemer	**Go**	Fewer features, faster builds, at scale.

Fifty years of decisions

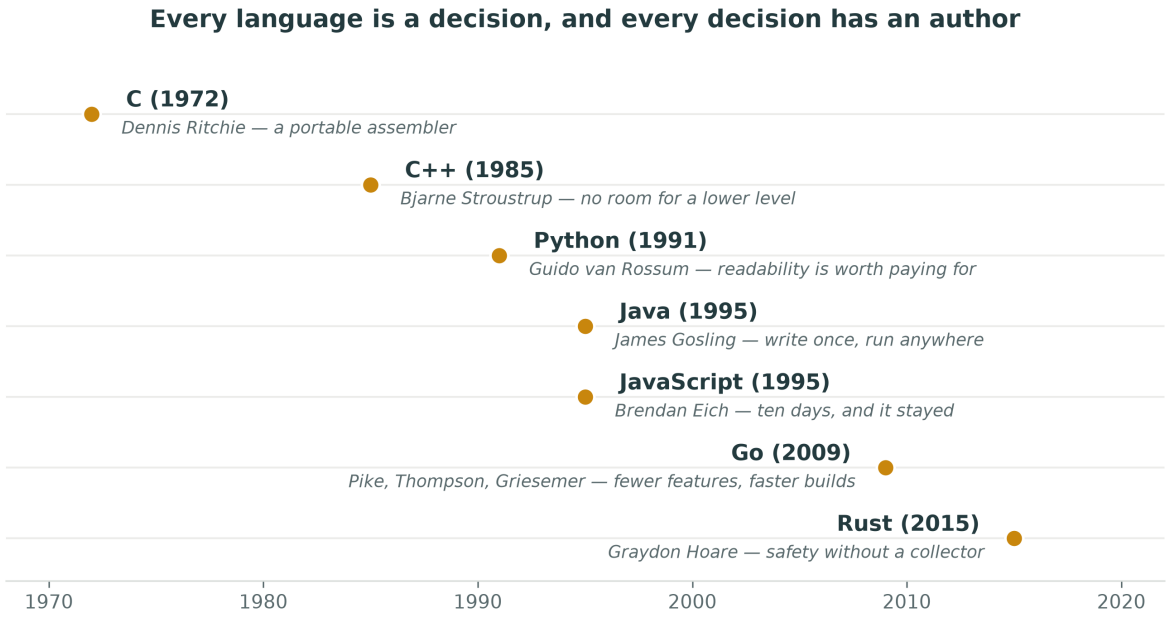


Figure 2: Each language answers a different question, and the answer has an author.

C. Its own section follows — it is the one underneath all the others.

Python. Van Rossum's priority was that code is read far more often than it is written. The consequences are everywhere: significant indentation (you cannot write badly-indented Python, because badly-indented Python is a *different* program), one obvious way to do things, a small keyword set. Run `import this` in a Python prompt and read the nineteen lines that come out — that is *PEP 20, The Zen of Python*, the language's philosophy stated by its own community.

C++. Stroustrup's rule was that C++ should not leave a gap underneath it that forces you down to assembly or C. Hence: you pay for what you use and nothing else, you get direct control of memory, and the language will let you do almost anything — including destroy yourself. The philosophy produces both the performance and the danger.

Rust. Hoare's question was whether memory safety requires a garbage collector. Rust's answer — ownership and borrowing, checked at compile time — is a genuinely new point in the design space, and it is why Rust has been the "most admired" language in the Stack Overflow survey for years running.

Go. Deliberately *small*. Its designers left out features other languages consider essential, on the grounds that a large team compiling a large codebase benefits more from simplicity and build speed than from expressiveness.

Four different answers. None of them is wrong. They are answers to **different questions**.

Where C came from

- **Dennis Ritchie**, at **Bell Labs**, in the early 1970s.
- Built out of Ken Thompson's language **B**, for one purpose: to write **Unix**.
- Unix was rewritten in C in 1973 — the first major operating system not written in assembly.
- Ritchie and Thompson shared the **1983 Turing Award**. Ritchie died in 2011.

That origin explains everything about how C looks. It was not designed to be pleasant, or safe, or modern. It was designed to *be a portable assembler* — something you could write an operating system in, and then move that operating system to a different machine. So the language stays small, stays close to the hardware, and trusts you completely. Every sharp edge in C is the price of that one decision.

C is still underneath you. The CPython interpreter running your Python is written in C. So are Linux, Windows' kernel, SQLite, git and ffmpeg. When your Python `list` grows and quietly doubles its storage — the thing you will implement by hand in Week 4 — that doubling is happening in C, a few layers below your code.

The book to own

> ***Brian W. Kernighan and Dennis M. Ritchie***
> The C Programming Language*, 2nd edition*
> *Prentice Hall, 1988 · ISBN 0-13-110362-8*

Everyone calls it **K&R**. Get the **2nd edition** — it covers ANSI C.

https://cs.princeton.edu/~bwk/cbook.html

The first edition is from 1978; the second, from 1988, is the one to buy. It is under 300 pages, and it is still the best book written about the language — partly because one of its two authors *created the language*. There is no more primary source than that.

Buy it, or read it from the link above, which is Kernighan's own page for the book. Do not download the scanned copies that turn up in search results; they are pirated, and you can see from the page above where the real thing lives.

And here is the joke paying off. This lecture opened with "every language can print `Hello, World`". Why *that* program, in every tutorial, in every language, for fifty years? Because it is the first program in K&R. One book set the opening line for the entire profession.

Why C matters for this course specifically. C makes memory visible. `malloc`, `free`, pointers, `struct` — nothing is hidden and nothing is automatic. Every structure we are about to build is transparent in C and invisible in Python, which is exactly the honest drawback of Python that Part 3 gets to. If you ever want to *really* understand a linked list, write one in C. You will never forget what a pointer is again.

A correction I owe you

On the board I put **Donald Knuth** next to Stroustrup and van Rossum. That was loose, and I want to fix it.

- Knuth did not design a language you will write production code in.
- He wrote *The Art of Computer Programming*, created **TeX**, and invented **literate programming**.
- His subject is **algorithms** — their analysis, their correctness, their cost.

He is not the bridge to languages. He is the bridge to this course.

I am correcting this deliberately and in public, because it is a habit I want you to copy. When you find out that something you said was wrong, you say so plainly, you fix it, and you move on. You do not quietly delete it and hope nobody noticed. In engineering, a person who corrects themselves is more trustworthy than a person who is never wrong, because the second person does not exist.

And the correction is useful here. Knuth's life's work is the question this course asks: *given a problem, what is the best way to organise the data, what does the resulting algorithm cost, and how do you prove it?* The famous line is his:

> *"Premature optimization is the root of all evil (or at least most of it) in programming."* — *Donald Knuth,* Structured Programming with go to Statements*, 1974*

Note the word **premature**. It is usually quoted by people who want permission never to think about performance. Knuth's actual point, in context, is that you should not micro-optimise the 97% of code that does not matter — *so that* you can spend real effort on the 3% that does. Finding that 3% is what complexity analysis is for. That is Week 2.

Paradigms: the same job, three ways

Sum the squares of the even numbers in a list.

```
# Procedural — describe the steps
total = 0
for n in numbers:
    if n % 2 == 0:
        total += n * n
```

```python
# Functional — describe the transformation
total = sum(map(lambda n: n * n, filter(lambda n: n % 2 == 0, numbers)))

# Declarative (comprehension) — describe the result
total = sum(n * n for n in numbers if n % 2 == 0)
```

Same output. Same language. **Three different ways of thinking.**

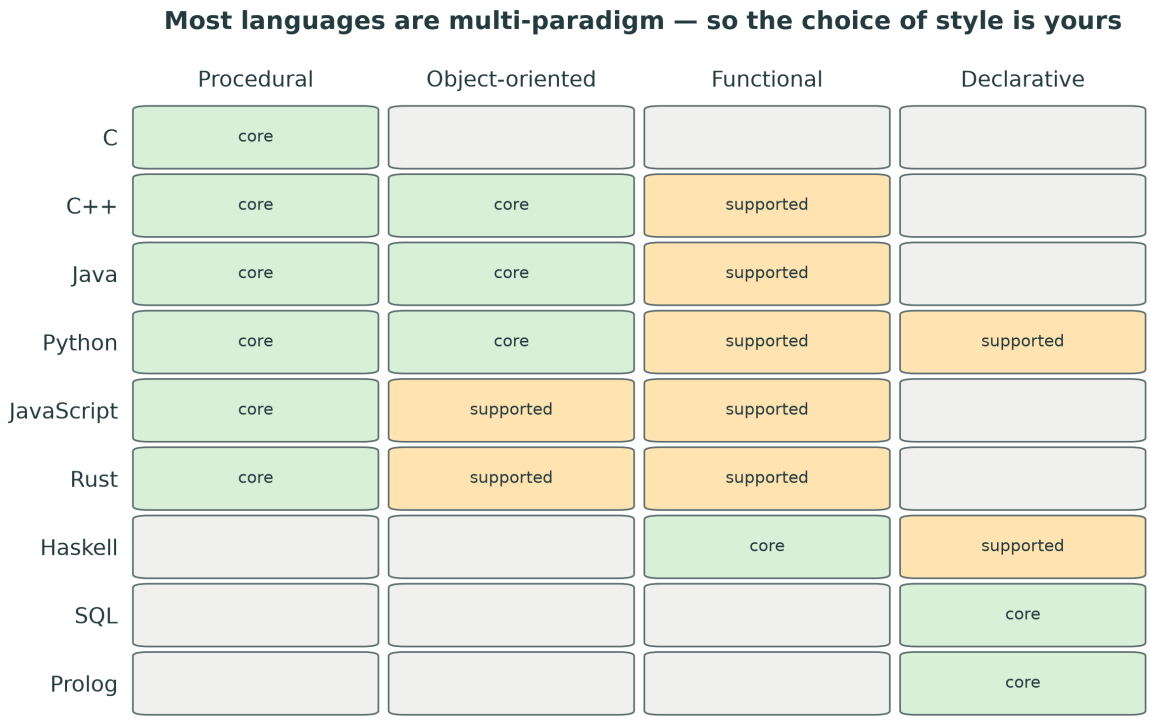


Figure 3: Almost every language you will use supports more than one style, which is why the choice is yours to make.

A **paradigm** is not a feature list, it is a way of decomposing a problem. The four you will meet:

- **Procedural / imperative** — the program is a sequence of steps that change state. C, Pascal, and the inside of almost every Python function.
- **Object-oriented** — the program is objects that own their data and expose operations on it. Java, C#, C++, and Python when you write a `class`. *This is how `dsa/` is built: a `Stack` owns its storage and exposes `push`, `pop` and `peek`. You should not be able to reach inside it, and neither should its users.*
- **Functional** — the program is the composition of functions, and you avoid changing state. Haskell, Elixir, Lisp; and `map`, `filter` and closures in Python.
- **Declarative / logic** — you state *what* you want and the system works out *how*. SQL, regular expressions, CSS, Prolog.

Most languages you will use are **multi-paradigm**, and Python aggressively so. That is a strength and a trap: Python will let you write all three styles above, so *you* have to decide which one makes the code clearest. The third version is the one I would accept in a review. It is shortest, it says

what it means, and it builds no intermediate list.

Type systems: two axes, not one

Students collapse these into one question. They are independent.

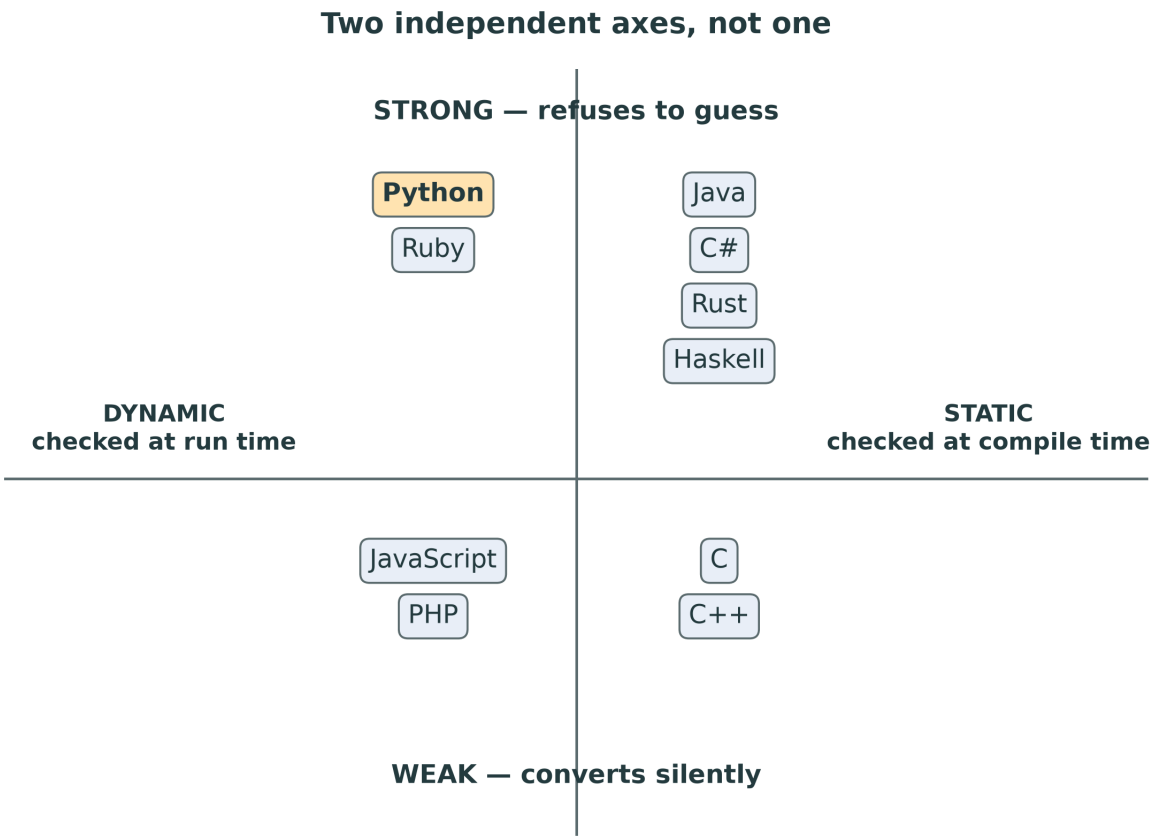


Figure 4: Static versus dynamic is *when*; strong versus weak is *how strictly*. Python sits top-left: dynamic, and strict.

Static vs dynamic is about **when** types are checked. **Strong vs weak** is about **how strictly** types are kept apart.

Python is **dynamically** typed — a name can be bound to an `int` now and a `str` later — but **strongly** typed, which is why this raises an error instead of guessing:

```
>>> "3" + 5
TypeError: can only concatenate str (not "int") to str
```

JavaScript, which is dynamic and weak, answers `"35"`. Neither behaviour is a bug. They are different decisions about whether a surprising conversion is more useful than a loud failure. Python's answer — fail loudly — is the one that will save you in this course, because a silent type coercion inside a sorting routine is genuinely hard to find.

One more thing worth knowing: Python has **optional type hints** (`def push(self, value: int) -> None:`). They are not enforced at run time — the interpreter ignores them — but tools

read them, and so do humans. The skeletons in `dsa/` use them. Treat them as documentation your editor can check.

Memory: who cleans up?

- **Manual** — you allocate, you free. C, C++. Fastest, and the source of decades of security vulnerabilities.
- **Garbage collected** — the runtime frees what you can no longer reach. Python, Java, C#, Go. Safe, at the cost of control and some pauses.
- **Ownership** — the compiler proves at build time that memory is freed exactly once. Rust. Safe *and* fast, at the cost of a harder compiler to satisfy.

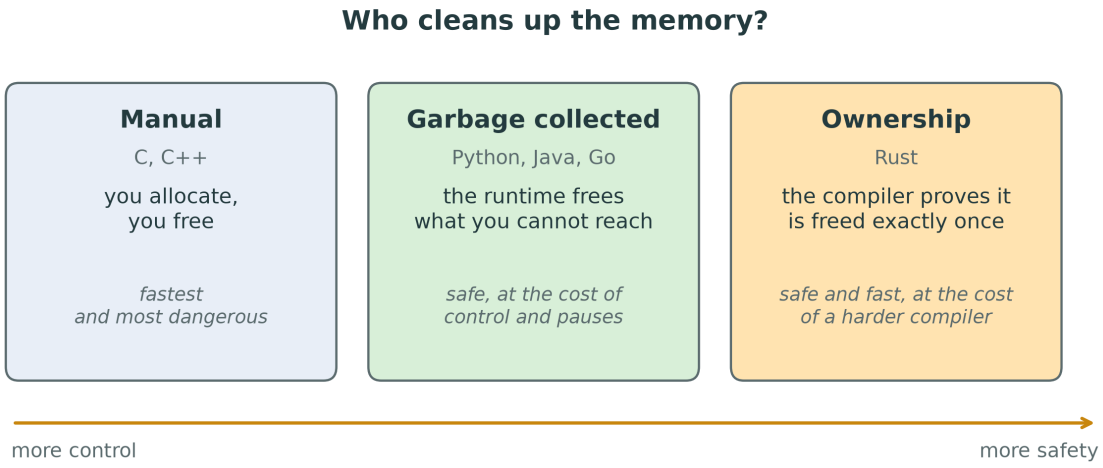


Figure 5: Control at one end, safety at the other. C sits on the left; Python sits in the middle and hides the question.

This matters to us more than it looks. When you implement `dsa/linked_list.py` and you "remove" a node, in C you would call `free()`, and a mistake there is a crash or a security hole. In Python you simply stop referring to the node and the garbage collector deals with it.

That convenience is exactly what makes the structure hard to *see*. You will build a linked list without ever touching a pointer. So use the debugger — set a breakpoint in `dsa/linked_list.py`, press **F5**, choose *pytest: current test file*, and watch `node.next` change in the Variables panel as you step with **F10**. The repository README has the setup.

Seeing a reference move beats any diagram, including the ones I drew.

Comparing Languages Honestly

What are you actually asking?

"Which language is best?" is not a question. These are questions:

- Best **for what task**? (a web service, embedded firmware, data analysis, a game)
- Best **for whom**? (you, alone, learning — or forty engineers for ten years)

- Best by **which measure**? (speed to write, speed to run, hiring pool, safety)
- Best **when**? (this project, or the one you maintain until 2035)

Change any one of those and the answer changes. A language that is perfect for a weekend data-analysis script is a poor choice for firmware on a device with 64 KB of RAM, and the reverse is equally true. This is not diplomacy, it is engineering.

The image I put on the board is the point: a hammer, surrounded by bent nails. A hammer is an excellent tool. The nails are bent because the hammer was the only tool the person owned. Learn more than one language — and the reason is not the second language itself. It is that you cannot see the decisions your first language made until you have seen a different set.

The rankings disagree — and that is the lesson

Index	What it actually counts	Crowns
TIOBE	Search-engine hits for "X programming"	Python
Stack Overflow Survey	Self-reported usage by respondents	JavaScript
GitHub / Octoverse	Commits and repositories	TypeScript / Python
PYPL	Google searches for "X tutorial"	Python

Different instruments. Different answers. **None of them measures quality.**

TIOBE, September 2026 — Python 17.76%, C 10.28%, C++ 8.67%, Java 7.54%, C# 4.22%, JavaScript 2.76%, Visual Basic 2.55%. Rust sits at number 10.

Read those numbers carefully. TIOBE counts *search-engine results*, so it is a proxy for how much people write and ask about a language — not how much code is running in production, and certainly not how good the language is. Visual Basic ranking above Rust should tell you everything about what the index is and is not.

Compare: the Stack Overflow Developer Survey has put **JavaScript** first in self-reported usage nearly every year since 2011, while **Rust** has been the *most admired* language — roughly 72% of the developers who use it want to keep using it. Usage and admiration are different questions, measured differently, with different answers.

So what do you do with the rankings? Two things, and only two:

1. Use them to see **trends over years**, not positions in a month. The interesting fact is not that Python is first this September — it is that it climbed there and stayed.
2. Use them to check that a language is **not dying**, because a language with no community has no libraries, no answers and no jobs.

Never use them to pick a language for a task. For that, you read about the task.

All figures above are dated **September 2026** and will be wrong soon. Any slide that quotes a ranking without a date is lying to you by omission — including, I will admit, some of my own

older slides.

Ecosystem beats syntax

What you actually adopt when you adopt a language:

- **Libraries** — has someone already solved your problem, well?
- **Tooling** — debugger, profiler, package manager, formatter, test runner
- **Community** — is your question already answered?
- **Interoperability** — can it call the fast thing written in something else?
- **Longevity** — will it still be maintained in ten years?

Syntax is the part of a language you stop noticing after two weeks. The ecosystem is the part you live inside for years. When a team argues about which language to use, this list is what they are really arguing about, even when they think they are arguing about semicolons.

Why Python for This Course

The honest case for Python

- **Where the field is.** Research, scientific computing, data analysis, and essentially all AI/ML/DL tooling.
- **Bindings.** The heavy numerical work is C, C++, CUDA or Fortran underneath; Python is the layer you steer it from. NumPy is not slow Python — it is fast C with a Python handle on it.
- **Reading cost.** Python is close enough to pseudocode that an algorithm survives the translation onto the page. That matters in a course where the algorithm *is* the content.
- **You will meet it again.** In every one of your programs — AI, Medical Informatics, Software Engineering.

That "bindings" point is the arrow I drew from the Python cloud across to the production box, and it is the reason Python occupies the position it does. A language that is easy to write but slow to run would be a toy. A language that is easy to write and can *call* the fast thing is infrastructure.

The honest case against Python — for us

Python already contains, as built-ins, most of what this course studies.

You will build	Python already has
Dynamic array	`list`
Hash map	`dict`
Stack / queue	`list`, `collections.deque`
Sorting	`sorted()`, `list.sort()` — Timsort
Priority queue	`heapq`

The tool hides exactly the thing we are here to look at.

This is a real drawback and I will not pretend otherwise. It is also precisely why the rule in `dsa/__init__.py` exists:

> *No module may use the built-in it is reimplementing.*

So `dsa/dynamic_array.py` may not be backed by a `list` — it uses a raw `ctypes` block, and `make_block()` is given to you because allocating raw memory is not the lesson. `dsa/hashmap.py` may not be backed by a `dict`. If you route around this rule the tests may even pass, and you will have learned nothing. The tests are not the point. They are the *evidence* for the point.

Python is the vehicle of this course, not its subject. Everything you build here — the cost of an insertion, why a hash map degrades, why quicksort's pivot matters — transfers unchanged to C++, Java, Rust or Go. The syntax is the only part that does not transfer, and the syntax is the cheap part.

From a Variable to a Data Structure

The ladder

`variable` → `object` → `class` → `collection` → **data structure**

```
i = 42                    # a name bound to an object
s = "hello"               # an object of class str
xs = [42, "hello", 3.14]  # a collection of references
```

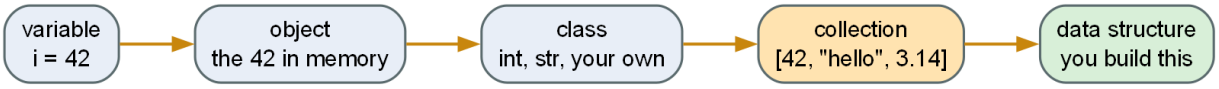


Figure 6: Five rungs. This course lives on the last one, but it only makes sense if the first four are solid.

In Python a variable is **not a box holding a value**. It is a **name bound to an object**.

This is the ladder I drew, and it is worth climbing slowly.

In C, `int i = 42;` reserves four bytes and puts the number in them. The name *is* the storage.

In Python, `i = 42` creates (or finds) an integer object somewhere in memory and binds the name `i` to it. The name is a label, not a container. Two consequences you will trip over this term:

```
a = [1, 2, 3]
b = a             # b is not a copy. Both names point at ONE list.
b.append(4)
print(a)          # [1, 2, 3, 4]
```

and:

```
xs = [42, "hello", 3.14]
```

A Python `list` does not store the objects; it stores **references** to them, which is how one list holds three different types. When you build `dsa/dynamic_array.py` on a raw `ctypes` block you will be storing references too — and you will see the machinery that `list` normally hides.

Abstract Data Type vs implementation

Abstract Data Type (ADT) — النوع المجرد للبيانات The **contract**: what operations exist, what they mean, and what they cost. *Nothing* about how the data is stored.

Data structure — هيكل البيانات A concrete **arrangement of data in memory** that fulfils the contract.

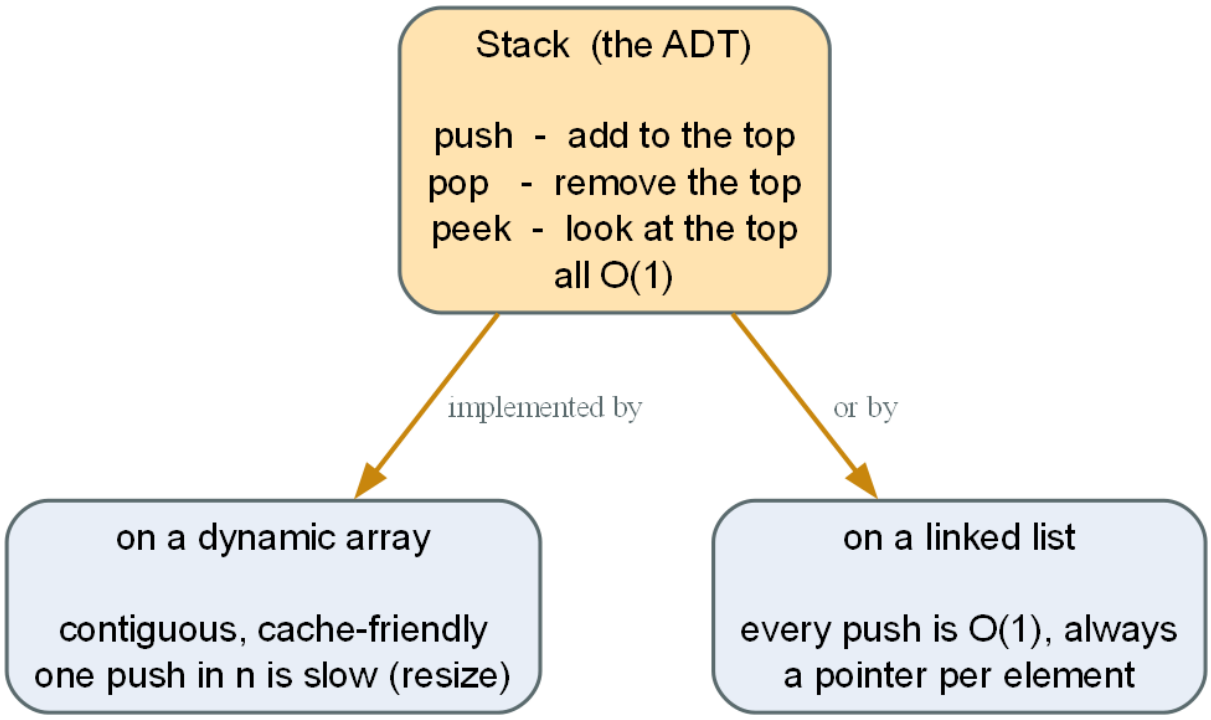


Figure 7: One contract, two honest implementations. Choosing between them is the engineering.

One ADT, many structures. The ADT **Stack** says only: push adds, pop removes the most recent, peek looks, all in O(1). That contract can be fulfilled by a dynamic array or by a linked list. Both are honest stacks. They differ in cache behaviour, in memory overhead per element, and in whether a single operation can occasionally be slow.

Choosing between them **is the engineering**, and you cannot choose without knowing the cost of each — which is why complexity comes before the structures, in Week 2.

A short glossary, since I will use both languages all term:

English	العربي
Data structure	هيكل البيانات
Algorithm	الخوارزمي
Abstract Data Type	النوع المجرد للبيانات
Complexity	التعقي
Array	المصفوف

English	العربي
Linked list	القائمة المتصلة
Stack	المكدس
Queue	الطابور
Tree	الشجرة
Graph	الرسم البياني
Hash table	جدول التجزئة
Sorting	الترتيب
Searching	البحث
Recursion	الاستدعاء الذاتي

Know-How

Three different things, and only the third is worth much:

1. **Know-what** — you can define a stack. *An hour's reading.*
2. **Know-how** — you can build one that works. *This course's exercises.*
3. **Know-when and know-why** — you can say why a stack and not a queue, and what it costs. *This course's exams, and your career.*

Know-How is the word I wrote on the board and circled, and it is the spine of this course.

The reason `dsa/` is a set of skeletons that all raise `NotImplementedError` is level 2. The reason the exam will ask you *which structure and why* rather than *define a heap* is level 3.

A student who has memorised that a hash map is "O(1) average" has level 1. A student who can implement one has level 2. A student who can explain why their hash map degraded to O(n) on real data, and then fix it, has level 3 — and is employable.

Cost Is the Whole Point

What an algorithm is

An **algorithm** is a finite, unambiguous procedure that turns an input into the correct output.

We care about three properties, in this order:

1. **Correctness** — does it produce the right answer, always?
2. **Cost in time** — how does the work grow as the input grows?
3. **Cost in space** — how does the memory grow as the input grows?

Correctness first. A fast wrong answer is worthless, and "it worked on my three test cases" is not correctness. This is why the exercises in this course are delivered as **tests**: `tests/` defines what "correct" means, in a form a machine can check, before you write a line of the implementation.

Measure, do not assert

Do not take my word that binary search beats linear search. **Measure it.**

```
from viz.complexity import measure, plot_growth
import random

sizes = [1000, 2000, 4000, 8000, 16000]
make = lambda n: (sorted(random.random() for _ in range(n)), 0.5)

plot_growth(
    {"linear": measure(lambda a: linear_search(*a), sizes, make),
     "binary": measure(lambda a: binary_search(*a), sizes, make)},
    reference=["n", "log n"],
)
```

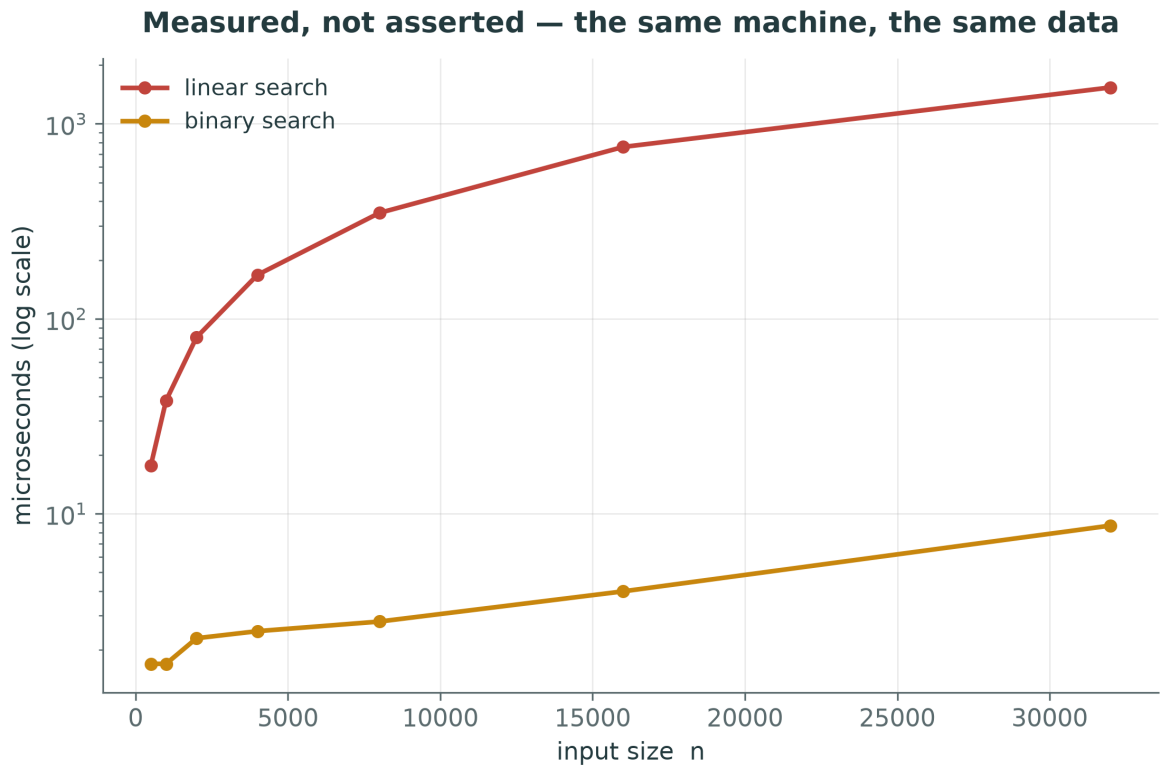


Figure 8: Real timings on one machine. Linear search climbs; binary search barely moves from 500 items to 32,000.

`viz/complexity.py` is already written for you — it is infrastructure, not an exercise. `measure()` times a function over a range of input sizes and returns the best time at each size; `plot_growth()` draws your measurements against scaled reference curves for 1, log n, n, n log n, n^2, n^3 and 2^n.

Notice what the picture gives you that a table of Big-O does not: the **crossover point**. Linear search often beats binary search on small inputs, because binary search does more work per step and the constant factors dominate. Big-O deliberately throws those constants away. That is what makes it useful at scale and misleading at n = 20.

This is the habit I want. When you are unsure about cost, do not argue and do not guess. Measure it, plot it, and look.

The gap is not small

n	O(log n)	O(n)	O(n log n)	$O(n^2)$	$O(2^n)$
10	3	10	33	100	1,024
100	7	100	664	10,000	$\sim 1.3 \times 10^{30}$
1,000	10	1,000	9,966	1,000,000	—
1,000,000	20	10^6	$\sim 2 \times 10^7$	10^{12}	—

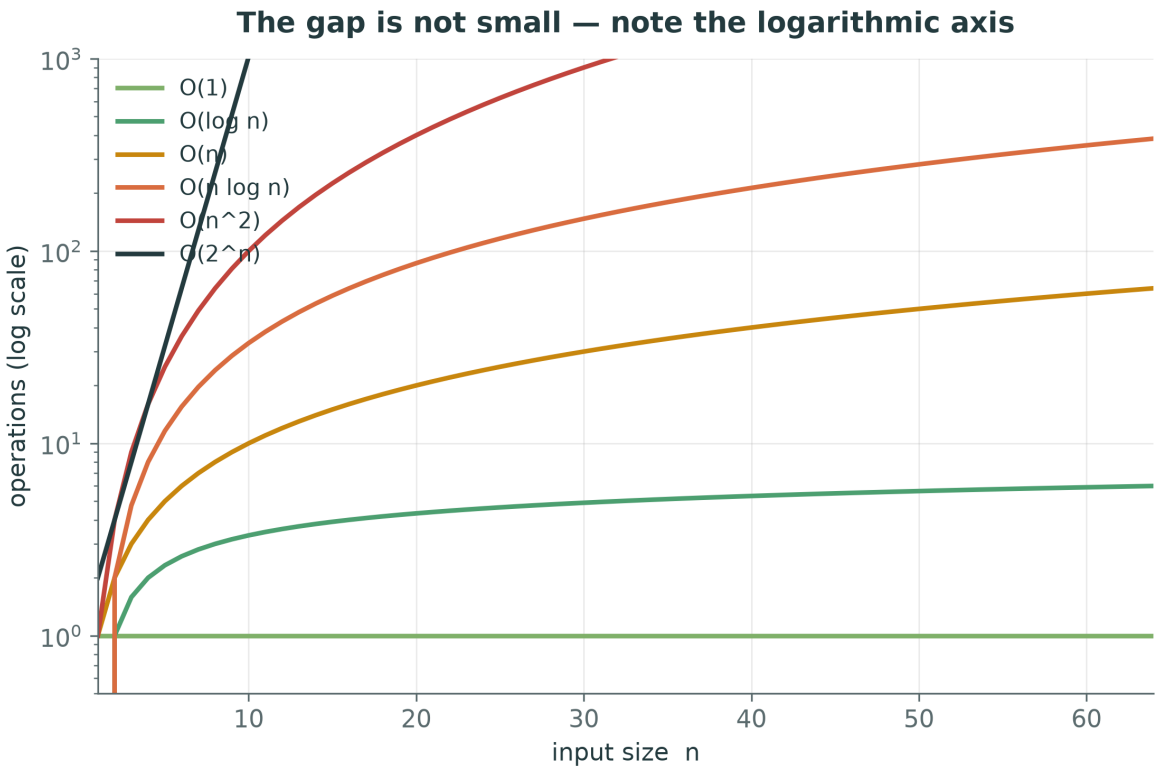


Figure 9: The same six curves, drawn. Note the logarithmic vertical axis — without it, everything but 2^n would be flat against the floor.

At n = 1,000,000 the O(log n) algorithm does **20** steps. The $O(n^2)$ one does a **trillion**.

Read the last row until it is uncomfortable. This is not an academic distinction and it is not something a faster machine fixes. If your algorithm is $O(n^2)$ and your data grows tenfold, your runtime grows a hundredfold; buying a computer twice as fast buys back a factor of two.

Choosing the right structure is how you move between rows of that table. That is the entire course, stated in one table.

The Course Itself

Your course, officially

Three programs, three bylaws, **one course**.

	AI	Bio / Medical Informatics	Software Engineering
Code	**CS2101**	**IS122**	**IS122**
Hours	3 (2 lec + 2 lab)	3 (2 lec + 2 lab)	3 (2 lec + 2 lab)
Prerequisite	CS1002 OOP	CS012, MATH012	CS012, MATH012
Level	Level 2, Sem. 3	Level 2	Sophomore, Sem. 1
Bylaw	2020, p. 44	2014, p. 35	2013, p. 38

Different codes. Same subject, same hours, same room.

The Arabic name differs slightly too: **هياكل البيانات و الخوارزميات** in the AI bylaw, **هياكل البيانات وتحليل الخوارزميات** in the other two. All four bylaw PDFs are in the repository at `docs/course/regulations/`, and every claim on this page carries its page number so you can check it.

What the bائ�ج says this course contains. The 2013 and 2014 texts are identical, word for word:

> *"Introduce the fundamental concepts of data structures and the algorithms that proceed from them. Topics include **recursion**, the underlying philosophy of **object-oriented programming**, fundamental data structures (including **stacks, queues, linked lists, hash tables, trees, and graphs**), the basics of **algorithmic analysis**, and an introduction to the **principles of language translation**."*

The 2020 AI text adds **arrays, heaps, priority queues, sorting, searching, graph searches and tree traversals** by name. The 15-week plan covers the union of both, and `docs/course/02-coverage.md` maps every declared topic to the module and the test that grades it.

Two things worth knowing about where this course sits.

It assumes less than you think. The bائ�ج says this course teaches recursion and the philosophy of object orientation — it does not assume them. If you are in Bio, nothing before this course taught you either. That is expected, and it is why Week 3 is recursion.

For two of the three programs, this is the only algorithms course you will ever take. The AI bائ�ج has a follow-on, AI3001 *Analysis and Design of AI Algorithms*. The 2013 and 2014 bائ�ج have none at all. So if you are in SWE or Bio, what you do not learn here, you will not be taught anywhere.

What it unlocks: Database Systems, Computer Networks and Computer Vision all take this course as a prerequisite — and for SWE, so does **SWE141 Software Construction**, which is where "principles of language translation" grows into grammars and parsers. In Week 15 you will build the small version of it: text in, a tree out, an answer at the end.

And from next year: the 2026 faculty bائ�ج replaces all three with a single **CS201**, 3 credit hours, prerequisite CS101, Sophomore Fall, a major requirement in all seven programs. It does not govern you. It is in the repository if you want to see where the faculty is going.

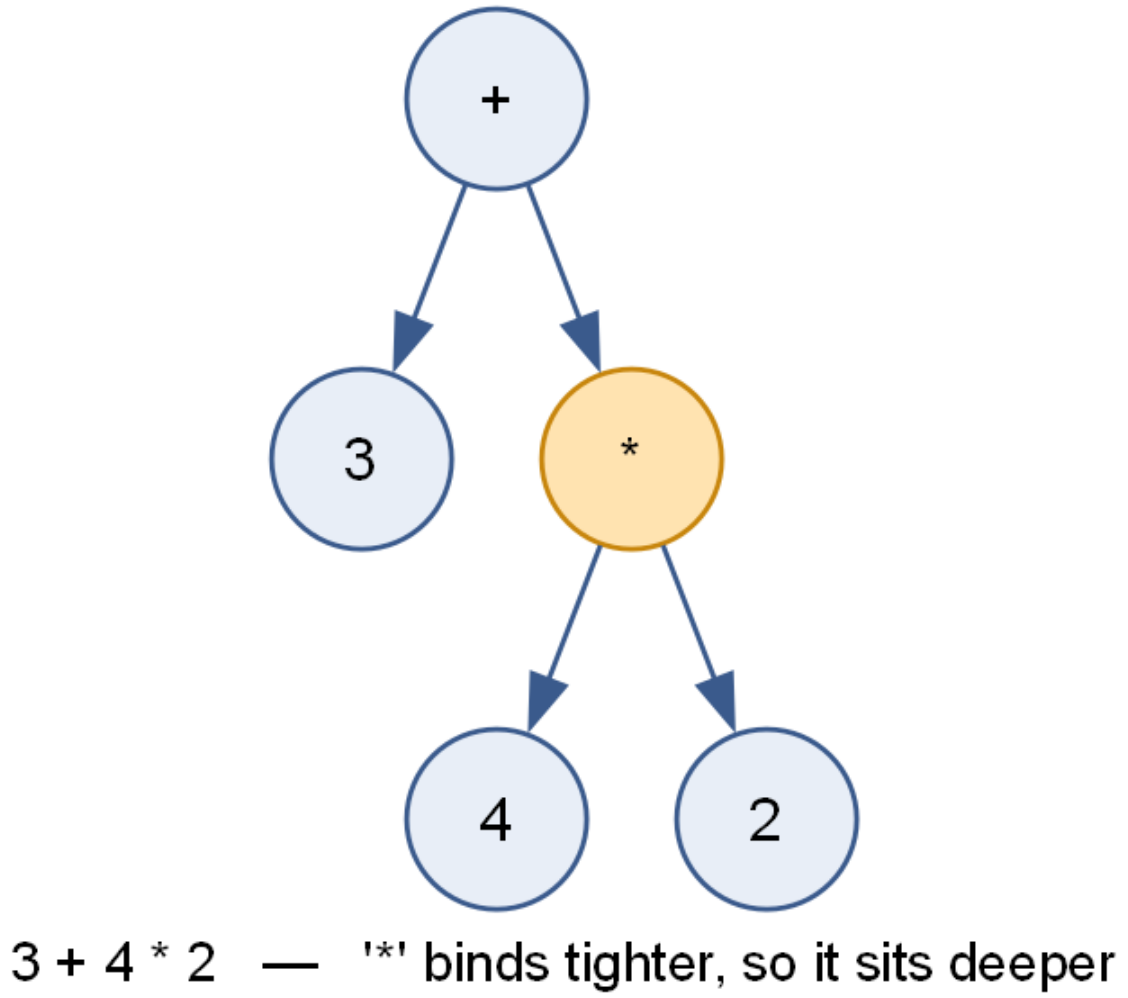


Figure 10: Week 15. `3 + 4 * 2` after parsing — * binds tighter, so it sits deeper in the tree and is evaluated first. Precedence is not a table of numbers; it is a shape.

How you pass

Same three rules in all three ﷺ:

- Final written exam is **60%**; coursework is the other **40%**.
- Pass needs **≥ 60% overall and ≥ 30% of the final** — both.
- **≥ 75% attendance**, or you are ﷺ and cannot sit the final at all.

Sources: SWE 2013 pp. 11–13 and p. 18; Medical Informatics 2014 pp. 11–13 and p. 18; AI 2020 pp. 14–16 and p. 19.

The three ﷺ differ only in how the 40% is split — SWE and Bio require the midterm to be at least 20%, AI at least 10%, and none of them may let any single component exceed 60%. **The exact split for this course will be announced in the lecture and on the WhatsApp channel**, and it will be written into `docs/course/00-course-guide.md` once it is fixed. Until then, treat the split as not yet decided, and the three rules above as binding.

Read the 30% rule again, because students lose years to it: someone who collects 35 of the 40 coursework marks and then scores 25% on the final has **failed**, regardless of the total. Coursework cannot rescue a final exam below 30%.

The attendance rule is not a threat, it is arithmetic. 75% of a 15-week course means you can miss roughly three sessions. Not five.

Grade scale — A+ (≥97%) through F (<60%), with **D** the minimum pass in any course. The three ﷺ disagree on whether A+ is worth 4.00 or 4.33 points; three of the four documents say **4.00**. That conflict, and two others, are documented rather than hidden in `docs/course/regulations/dsa-in-your-program.md`.

How we will work

- **Lecture** — the idea, the cost, and why it exists.
- **Lab** — you implement it. The skeletons in `dsa/` all raise `NotImplementedError` on day one. That is the assignment.
- **Tests** — `tests/` defines "correct". `pytest -m challenge` runs every exercise.
- **Visualisation** — `viz/` draws arrays, lists, trees and graphs, animates your sorts, and plots your measured complexity. It never imports `dsa/`, so it works with whatever you write.

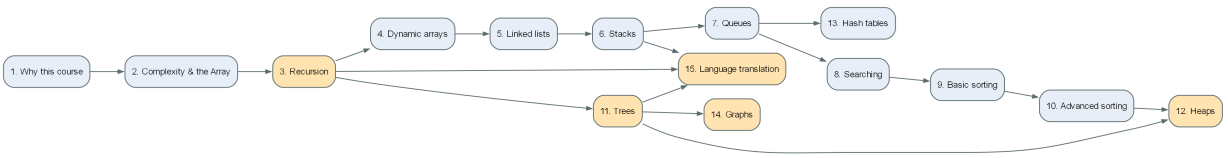


Figure 11: Fifteen weeks. An arrow means the later week genuinely needs the earlier one — which is why recursion comes before trees, and trees before the parser.

They all fail on day one. That is the point, and it is the honest version of a course: you are not asked to reproduce my solution, you are asked to satisfy a specification.

The full 15-week plan, with the module and test file for each week, is in `docs/course/01-study-plan.md`, and the course map above is generated from it.

Homework 1

Due before the next lecture

1. **Set up your environment.** Clone the repository, create the virtual environment, install the requirements, and make `pytest -m "not challenge"` pass. Bring the output.
2. **Read** Peter Norvig, *Teach Yourself Programming in Ten Years* — https://norvig.com/21-days.html. It is short.
3. **Run** `import this` in a Python prompt and read *The Zen of Python*. Pick the line you disagree with most, and be ready to say why.
4. **Write — one page, no more.** Choose two languages you have used or heard about. Compare them on **three criteria you choose yourself**, say which you would pick for a **specific named task**, and state what you are giving up by picking it.

On item 2: Norvig's essay is the antidote to "Learn X in 21 Days". His argument is that expertise takes roughly ten years of deliberate practice, and that the books promising otherwise are selling something. I put this on the last slide of a talk seven years ago and I am putting it in the first lecture of this course, because the sooner you stop looking for the shortcut, the sooner you start walking the road.

On item 4: there is no correct answer and I am not looking for one. I am looking for whether you can state a criterion, apply it, and be honest about the cost. A paragraph ending "…so I would choose Go, and I am giving up the library ecosystem I would have had in Python" earns full marks. A paragraph ending "…so Python is the best language" earns very few.

Summary

Six things to keep

1. Languages are **not** the same. Each is a set of decisions with an author and a reason.
2. "Which is best?" is unanswerable until you say **best for what, for whom, by which measure**.
3. Popularity indices measure **different things** and disagree. None measures quality. Always check the date.
4. We use Python because it reads like the algorithm — and we **reimplement its built-ins** because it otherwise hides the subject.
5. An **ADT** is a contract; a **data structure** is one way to keep it. Choosing between them is the engineering.
6. **Know-how** beats know-what; **know-why** beats both.

Next

Week 2 — Complexity. How to say what an algorithm costs, and how to *measure* whether you were right.

Before then: make `pytest -m "not challenge"` pass.

Sources and further reading

Official — the ﷺ that governs you

- *Software Engineering, 2013* — `Program-SoftwareEngineering-2013.pdf`. IS122 specification p. 38; study plan p. 29; grades p. 14; assessment p. 13; attendance p. 18.
- *Medical Informatics, 2014* — `Program-MedicalInformatics-2014.pdf`. IS122 specification p. 35; grades p. 14; assessment p. 12; attendance p. 18.
- *Artificial Intelligence, 2020* — `Program-ArtificialIntelligence-2020.pdf`. CS2101 specification p. 44; study plan p. 35; grades pp. 16–17; attendance p. 19.
- *FCIS Faculty Bylaw, 2026* — `FCIS-Bylaw-2026.pdf`. **Does not govern you**; it replaces all three above from next year with CS201 (p. 138).

All four are in `docs/course/regulations/`, and `dsa-in-your-program.md` beside them extracts everything they say about this course, with page numbers.

Read this term

- **Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, 2nd ed., Prentice Hall, 1988. ISBN 0-13-110362-8** — "K&R", the book written by the man who made C. https://cs.princeton.edu/~bwk/cbook.html
- Peter Norvig, *Teach Yourself Programming in Ten Years* — https://norvig.com/21-days.html
- Tim Peters, *PEP 20 — The Zen of Python* — https://peps.python.org/pep-0020/
- Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms* (CLRS), 4th ed., MIT Press, 2022 — the reference.
- Sedgewick & Wayne, *Algorithms*, 4th ed. — gentler, excellent figures.
- Goodrich, Tamassia & Goldwasser, *Data Structures and Algorithms in Python* — closest to what we do here.

Data quoted in this lecture

- TIOBE Index, September 2026 — https://www.tiobe.com/tiobe-index/
- Stack Overflow Developer Survey — https://survey.stackoverflow.co/

All figures are as of **22 September 2026** and will need rechecking.

This document replaces "Programming Languages are Not the Same" (22 September 2019). The argument survives; the data, the links and one attribution did not.

Questions for Week 1

> ***The answers are at the back of the book,*** *in* Answers — Week 1. *Attempt every question on paper first. Each question is tagged with its level:* ***[what]*** *— recall,* ***[how]*** *— apply,* ***[why]*** *— explain and justify. Marks shown for written questions are a guide to the length of answer expected.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W1-M01 – W1-M24
B	Short answer and essay	W1-E1 – W1-E6
C	Trace the code — what is printed?	W1-T1 – W1-T6
D	Find and fix the bug	W1-B1 – W1-B3
E	Write the code — checked by `pytest`	W1-C1 – W1-C4

Part A — Multiple choice

W1-M01 [what] "Every general-purpose programming language is Turing-complete." What does that tell you?

- **a)** All languages run equally fast
- **b)** Any problem solvable in one of them is solvable in any other
- **c)** All languages differ only in syntax
- **d)** A program written in one language runs unchanged in any other

W1-M02 [what] Python's type system is best described as:

- **a)** static and strong
- **b)** static and weak
- **c)** dynamic and weak
- **d)** dynamic and strong

W1-M03 [how] What does `"3" + 5` do in Python?

- **a)** evaluates to `"35"`
- **b)** evaluates to `8`
- **c)** raises `TypeError`
- **d)** raises `SyntaxError`

W1-M04 [what] How does Rust manage memory?

- **a)** The programmer calls `free` for every allocation
- **b)** A garbage collector frees unreachable objects at run time
- **c)** Ownership rules, checked by the compiler, guarantee each allocation is freed exactly once
- **d)** It never frees memory until the program ends

W1-M05 [what] Where and why was C created?

- **a)** By Dennis Ritchie at Bell Labs, to write the Unix operating system
- **b)** By Bjarne Stroustrup at Bell Labs, to add classes to Simula
- **c)** By Donald Knuth, to typeset *The Art of Computer Programming*
- **d)** By Guido van Rossum, as a teaching language

W1-M06 [what] An **Abstract Data Type** specifies:

- **a)** how the data is laid out in memory

- **b)** which programming language the structure must be written in
- **c)** the class hierarchy of the implementation
- **d)** the operations, what they mean and what they cost — nothing about storage

W1-M07 [how] Which of these is the *declarative (comprehension)* style from Lecture 01?

- **a)** `for n in numbers: if n % 2 == 0: total += n * n`
- **b)** `total = sum(n * n for n in numbers if n % 2 == 0)`
- **c)** `total = sum(map(lambda n: n * n, filter(lambda n: n % 2 == 0, numbers)))`
- **d)** `while i < len(numbers): ...`

W1-M08 [what] What does the TIOBE index actually count?

- **a)** search-engine results for "X programming"
- **b)** the quality of each language
- **c)** commits on GitHub
- **d)** self-reported usage in a developer survey

W1-M09 [why] In the course's three levels of knowledge, which is **know-why**?

- **a)** You can define a stack
- **b)** You can implement a stack that passes its tests
- **c)** You can say why a stack and not a queue, and what it costs
- **d)** You can recite the Big-O of every stack operation

W1-M10 [what] To pass this course, a student needs:

- **a)** at least 50% overall
- **b)** at least 60% overall and at least 30% of the final exam
- **c)** at least 60% overall, whatever the final exam mark
- **d)** at least 30% of the final exam, whatever the overall mark

W1-M11 [how] What is `-7 // 2`?

- **a)** `-3`
- **b)** `-3.5`
- **c)** `3`
- **d)** `-4`

W1-M12 [how] What is `-7 % 3`?

- **a)** `2`
- **b)** `-1`
- **c)** `1`
- **d)** `-2`

W1-M13 [how] What is `2 ** 3 ** 2`?

- **a)** `64`
- **b)** `512`
- **c)** `36`
- **d)** `12`

W1-M14 [how] What is `type(8 / 4)`?

- **a)** `<class 'int'>`
- **b)** `<class 'str'>`
- **c)** `<class 'float'>`
- **d)** it raises `TypeError`

W1-M15 [how] With `word = "Python"`, what is `word[1:4]`?

- **a)** `"Pyt"`
- **b)** `"ytho"`
- **c)** `"ython"`
- **d)** `"yth"`

W1-M16 [how] What is `"Mansoura"[::-2]`?

- **a)** `"ausa"`
- **b)** `"aruosnaM"`
- **c)** `"Mnor"`
- **d)** `"arsa"`

W1-M17 [how] After `a = [1, 2, 3]`, `b = a`, `b.append(4)`, what is `a`?

- **a)** `[1, 2, 3]`
- **b)** `[4, 1, 2, 3]`
- **c)** `[1, 2, 3, 4]`
- **d)** `[[1, 2, 3], 4]`

W1-M18 [how] After `a = [1, 2]` and `c = a[:]`, what are `c is a` and `c == a`?

- **a)** `True`, `True`
- **b)** `False`, `True`
- **c)** `True`, `False`
- **d)** `False`, `False`

W1-M19 [how] `grid = [[0] * 2] * 2`, then `grid[0][0] = 1`. What is `grid`?

- **a)** `[[1, 0], [0, 0]]`
- **b)** `[[1, 1], [0, 0]]`
- **c)** `[[1, 0], [1, 0]]`
- **d)** `[[1, 1], [1, 1]]`

W1-M20 [what] What is `round(2.5) + round(3.5)`?

- **a)** 6
- **b)** 7
- **c)** 5
- **d)** 8

W1-M21 [why] Why is `0.1 + 0.2 == 0.3` `False` in Python?

- **a)** Python has a bug in float addition
- **b)** `0.1` is stored as an `int`
- **c)** == compares identity, not value, for floats
- **d)** `0.1`, `0.2` and `0.3` cannot be stored exactly in binary floating point

W1-M22 [how] What happens when you run `s = "abc"` then `s[0] = "z"`?

- **a)** `s` becomes `"zbc"`
- **b)** a `TypeError` is raised, because strings are immutable
- **c)** a new string `"zbc"` is created and `s` refers to it
- **d)** an `IndexError` is raised

W1-M23 [how] A function's body is `print(width * height)` and nothing else. What does `x = area(3, 4)` store in `x`?

- **a)** `12`
- **b)** `"12"`
- **c)** `None`
- **d)** nothing — it is a `NameError`

W1-M24 [how] What does `int("4.5")` do?

- **a)** returns `4`
- **b)** returns `5`
- **c)** returns `4.5`
- **d)** raises `ValueError`

Part B — Short answer and essay

W1-E1 [why] *(4 marks)* "All programming languages are the same; only the syntax differs." Explain what is true in this statement and what is wrong with it. Give one concrete example of a design decision that makes two languages genuinely different.

W1-E2 [why] *(4 marks)* Explain the two independent axes of type systems. Place Python and JavaScript on both axes, and show with one expression how their behaviour differs.

W1-E3 [what] *(3 marks)* Describe the three approaches to memory management from Lecture 01, naming one language that uses each. Which one does Python use, and why does that make data structures harder to *see* in Python?

W1-E4 [why] *(4 marks)* Distinguish an **Abstract Data Type** from a **data structure**. Illustrate with the Stack ADT and two different structures that implement it.

W1-E5 [why] *(3 marks)* Give two reasons the course uses Python, and the honest drawback of Python *for this course*. How does the course deal with that drawback?

W1-E6 [why] *(3 marks)* "In Python a variable is not a box holding a value; it is a name bound to an object." Explain this with a short code example and a diagram, and state one consequence for lists.

Part C — Trace the code

For each program, write **exactly** what is printed.

W1-T1 [how]

```
print(17 // 5, 17 % 5, -17 // 5, -17 % 5)
print(divmod(23, 4), 2 ** 10, 7 / 2)
```

W1-T2 [how]

```
s = "Data Structures"
print(s[0], s[-1], s[5:9], s[:4].upper())
print(s.split(), len(s), s.count("t"))
```

W1-T3 [how]

```
a = [1, 2, 3]
b = a
c = a[:]
a.append(4)
b[0] = 99
c = c + [5]
print(a, b, c)
a += [6]
print(b)
```

W1-T4 [how]

```
n, count = 1000, 0
while n > 1:
    n = n // 3
    count += 1
print(n, count)
```

W1-T5 [how]

```
name, mark = "Omar", 87.456
print(f"{name:>6}|{mark:.1f}|{mark:8.2f}|{len(name):03d}")
```

W1-T6 [how]

```
a, b = 0, 1
result = []
while len(result) < 8:
    result.append(a)
    a, b = b, a + b
print(result)
print(result[-3:], result[::3])
```

Part D — Find and fix the bug

Each function has **one** bug. Say what goes wrong, for which input, and fix it.

W1-B1 [how] Should return the initials of a name, e.g. `"H.E."`.

```
def initials(full_name):
    result = ""
    for word in full_name.split(" "):
        result += word[0].upper() + "."
    return result
```

W1-B2 [how] Should return the average of a non-empty list.

```
def average(values):
    total = 0
    i = 0
    while i < len(values):
        total += values[i]
    return total / len(values)
```

W1-B3 [how] Should return a copy of `values` with `item` added at the end, **without changing** `values`.

```
def with_item(values, item):
    result = values
    result.append(item)
    return result
```

Part E — Write the code

Write these in `practice/week01.py` and check with `pytest tests/test_practice_week01.py -v`. The docstrings in that file are the full specification.

W1-C1 [how] `count_vowels(text)` — how many of a, e, i, o, u appear in `text`, in either case. `count_vowels("Data Structures")` is 5.

W1-C2 [how] `second_largest(values)` — the largest value strictly smaller than the maximum, in **one pass** and **without sorting**. Raise `ValueError` if there is no such value. `second_largest([4, 9, 2, 9, 7])` is 7.

W1-C3 [how] `compress(text)` — run-length encoding: `compress("aaabcc")` is `"a3b1c2"`.

W1-C4 [how] `is_anagram(first, second)` — same letters, same counts, ignoring case and spaces. `is_anagram("Dormitory", "dirty room")` is `True`.

Week 2 — Complexity, and the Array

> ***How to read this document.*** *This is the handout for Lecture 02. It holds everything on the slides, plus what I said out loud and the reasoning behind each claim. Two things are new this week and both matter for the rest of the term: the* ***language of cost*** *(Big-O and its relatives), and the* ***Array****, the one storage primitive every structure in this course will be built on.*
>
> *Slides:* `DSA27-L02-slides.pdf` *· Code:* `dsa/array.py`, `dsa/array_ops.py` *· Tests:* `tests/test_array.py`, `tests/test_array_ops.py` *· Interactive demo: Array Lab*

Where We Are

Last week, in one line

An **ADT** is a contract. A **data structure** keeps the contract. Choosing between structures **is the engineering** — and you cannot choose without knowing what each one **costs**.

Today: **how to state a cost**, and **the first structure whose costs we state**.

Lecture 01 ended with a promise: complexity comes before the structures, because every choice of structure is a choice about cost. This lecture keeps that promise twice over. The first half gives you the vocabulary — Big-O, Big-Ω, Big-Θ, best, worst and average case, time and space. The second half applies it to the simplest structure there is, the **array**, and introduces the rule that governs every structure you write from now on.

Today

1. What we count, and why we throw most of it away
2. Big-O, Big-Ω, Big-Θ — the definitions, and one proof
3. Best, worst and average case; time and space
4. Rules for reading the cost off code
5. **The array**: what it is in memory, and what it costs
6. Arrays in Python — and **the course `Array`**
7. **The rule** from today on: no `list`, `dict` or `set` inside a structure

What Do We Count?

Two ways to answer "how fast?"

Measure	Count
time it with a clock	count the basic steps
true for this machine, this input, today	true for every machine

Measure	Count
gives you the constants	gives you the **shape**
`viz.complexity.measure`	pen and paper

You need both. This lecture is mostly about counting — and ends by measuring.

A stopwatch answers "how long did it take?", which depends on the processor, the Python version, what else was running, and the input you happened to pick. A count answers "how does the work **grow** as the input grows?", which depends only on the algorithm.

The count is what lets you predict. If an algorithm does n^2 steps and takes one second at n = 10,000, it will take about a hundred seconds at n = 100,000 — on any machine. The stopwatch cannot tell you that; the count can.

The model: basic steps

A **basic step** takes a constant amount of time, whatever the input size:

- arithmetic and comparison on small numbers: `a + b`, `x < y`
- assigning a variable: `x = y`
- reading or writing **one** array slot: `a[i]`, `a[i] = v`
- calling a function, returning from one (not what the function then does)

The cost of an algorithm is **the number of basic steps**, as a function of the input size **n**.

This is the *RAM model* (random-access machine): memory is a long row of slots, and reaching any slot costs the same. It is a simplification — real machines have caches, and Lecture 01 warned that Python integers are not fixed-size — but it is the right simplification, because it keeps what matters for large n and throws away what does not.

Note the third bullet. **Reading one array slot is a basic step.** That is not an assumption we get for free; it is a property of how arrays are laid out in memory, and the second half of this lecture explains why it is true.

What is n? Whatever measures the size of the input: the number of elements in an array, the number of characters in a string, the number of nodes in a graph. Sometimes there are two: an n × m matrix, or a graph with V nodes and E edges.

Counting a loop

```python
def total(arr):
    result = 0                  # 1 step
    for i in range(len(arr)):   # the body runs n times
        result += arr[i]        #   2 steps: read, add-and-assign
    return result               # 1 step
```

$$T(n) = 2n + 2$$

Count it honestly and you get something like 2n + 2. Count it more pedantically — the loop itself compares and increments a counter — and you get 4n + 3, or 5n + 2. Nobody agrees on the exact constants, and **it does not matter**, for a reason the next slide makes precise: every one of those formulas is "a constant times n, plus a constant", and they all grow the same way.

Counting a nested loop

```
def count_pairs(arr):
    n, pairs = len(arr), 0
    for i in range(n):
        for j in range(i + 1, n):      # runs n-1, n-2, ..., 1, 0 times
            if arr[i] == arr[j]:
                pairs += 1
    return pairs
```

$$(n-1) + (n-2) + \cdots + 1 + 0 = \frac{n(n-1)}{2} = \tfrac{1}{2}n^2 - \tfrac{1}{2}n$$

This is exercise 7 from Lab 02, `count_pairs`, and now you can see where the formula comes from. The inner loop runs n − 1 times for i = 0, n − 2 for i = 1, and so on down to 0. Adding 1 + 2 + … + (n − 1) is the sum Gauss is said to have done as a schoolboy: pair the first with the last, and you have (n − 1)/2 pairs each summing to n.

The leading term is ½n². Double n and the work goes up about four times.

Throw away what does not matter

n	$\frac{1}{2}n^2$	$\frac{1}{2}n$	share of the total from $\frac{1}{2}n^2$
10	50	5	91%
1,000	500,000	500	99.9%
1,000,000	5×10^{11}	500,000	99.9999%

For large n, **the fastest-growing term is everything**. And the constant in front of it (½) depends on the machine and on how you count. So we drop both:

$$\tfrac{1}{2}n^2 - \tfrac{1}{2}n \quad \longrightarrow \quad n^2$$

Two things are being thrown away, for two different reasons.

Lower-order terms go because they stop mattering. By n = 1,000 the ½n term is a rounding error.

Constant factors go because they are not a property of the algorithm. The same algorithm has a different constant in C and in Python, on a laptop and on a server, counted by me and counted by you. What stays the same everywhere is the *shape*: n², n, log n.

This is the deal Big-O offers. You give up the constants, and in return you get a statement that is true on every machine and that tells you what happens when n grows. It is a very good deal for large n and a misleading one for small n — which is why we still measure.

Big-O, Big-Ω, Big-Θ

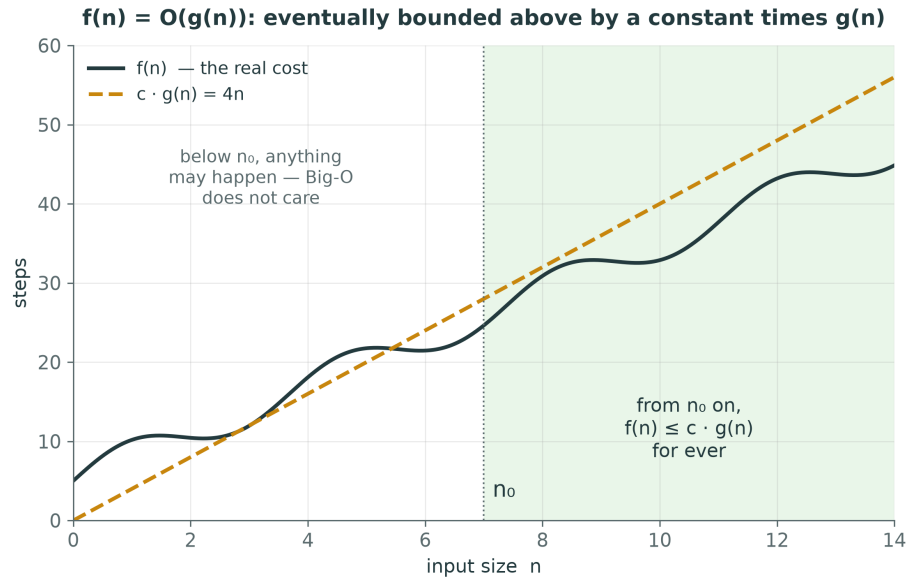
Big-O: an upper bound

$$f(n) = O(g(n))$$

means: there are constants $c > 0$ and n_0 such that

$$f(n) \leq c \cdot g(n) \quad \text{for every } n \geq n_0.$$

"From some point on, f never grows faster than a constant times g."



Read the definition slowly, because every part of it is doing a job.

- **"There are constants c and n_0"** — you get to choose them. Proving f(n) = O(g(n)) means finding *one* pair that works.
- **"c · g(n)"** — the constant factor is allowed. That is how the definition throws constants away: 1000n is O(n), because c = 1000 works.
- **"for every n ≥ n_0"** — only large n counts. Below n_0, f may be as badly behaved as it likes. That is how lower-order terms disappear.
- **"≤"** — it is an **upper bound**. It says f grows *no faster* than g. It does not say f grows *as fast as* g.

That last point surprises people: n = O(n^2) is **true**. It is just not very informative, like saying "the lecture room holds fewer than a million people". When we say "linear search is O(n)" we mean, informally, the *tightest* upper bound we can prove — but the notation itself only promises "no worse than".

In Arabic you will see رتبة التعقيد (order of complexity) or simply *O* الكبير.

A proof, once

Claim. $3n + 5 = O(n)$.

Proof. Choose $c = 4$ and $n_0 = 5$. For every $n \geq 5$:

$$3n + 5 \;\leq\; 3n + n \;=\; 4n.$$

So $3n + 5 \leq 4 \cdot n$ for all $n \geq 5$. ■

The only step that needs thought is "5 ≤ n", which is true exactly because we chose $n_0 = 5$. Other choices work too: c = 8 and $n_0 = 1$ ($3n + 5 \leq 3n + 5n = 8n$ for $n \geq 1$). The definition asks for *some* c and n_0, not the best ones.

The same idea shows that **any polynomial is Big-O of its highest power**: $a_k n^k + \ldots + a_1 n + a_0 = O(n^k)$. Replace every lower power by n^k (for $n \geq 1$ each is smaller), add up the coefficients, and that sum is your c.

And to show something is **not** O(g), show that no c can work: n^2 is not O(n), because $n^2 \leq c \cdot n$ would need $n \leq c$ for *every* large n, and no constant is bigger than every n.

You will be asked to do one proof like this in the exam. You will be asked, far more often, to *read off* the Big-O of a piece of code — which the rules later in this lecture make mechanical.

Big-Ω and Big-Θ

Notation	Means	Like
$f = O(g)$	$f \leq c \cdot g$ eventually	$\leq$ — an **upper** bound
$f = \Omega(g)$	$f \geq c \cdot g$ eventually	$\geq$ — a **lower** bound
$f = \Theta(g)$	both: $c_1 g \leq f \leq c_2 g$	$=$ — a **tight** bound

$\frac{1}{2}n^2 - \frac{1}{2}n$ is $O(n^2)$, $\Omega(n^2)$ and therefore $\Theta(n^2)$. It is also $O(n^3)$ — true, and useless.

Ω (Omega) is the mirror image of O: there are c > 0 and n_0 such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$. "f grows *at least* as fast as g."

Θ (Theta) is both at once. f = Θ(g) means f is sandwiched between two constant multiples of g: it grows *exactly* like g, up to constants. That is usually what people mean when they say "Big-O" in conversation.

In this course, and in most interviews, "the complexity is O(n)" is understood as "Θ(n), and that is tight". In a proof or an exam answer, use the symbol you can actually prove.

One more use of Ω that matters later: **a lower bound on a problem**, not an algorithm. Any sorting algorithm that works only by comparing elements needs Ω(n log n) comparisons in the worst case. No cleverness gets under it. In Week 9, counting sort gets under it by *not comparing* — which is exactly the loophole the bound leaves open.

The classes you will meet

Big-O	Name	Example in this course	n = 1,000,000
$O(1)$	constant	`a[i]`; push onto a stack	1
$O(\log n)$	logarithmic	binary search (Week 8)	20
$O(n)$	linear	linear search; insert into an array	10^6
$O(n \log n)$	linearithmic	merge sort, heap sort (Week 10)	2×10^7
$O(n^2)$	quadratic	bubble, selection, insertion sort	10^{12}
$O(2^n)$	exponential	all subsets (Week 3)	—

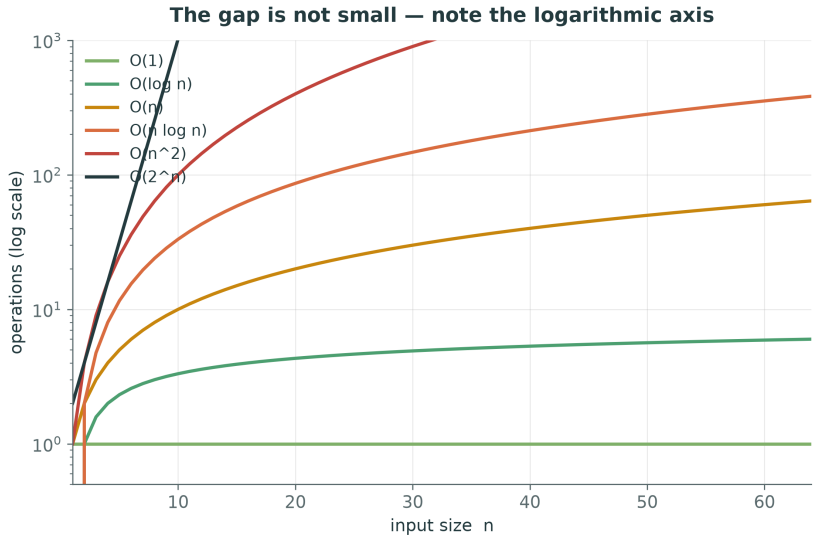


Figure 12: The same curves as Lecture 01. Note the logarithmic vertical axis.

Learn this table. The right-hand column is the one to feel: at a billion simple steps a second, 10^6 steps is a millisecond, 2×10^7 is a fiftieth of a second, and 10^{12} is **more than a quarter of an hour**. For 2^n at n = 1,000,000 there is no useful number — the universe is younger.

Where does log n come from? From **halving**. Lab 01's Checkpoint 4 halved 100 down to 1 in six steps; halving n down to 1 takes about $\log_2 n$ steps. Any algorithm that throws away half of what is left at each step is O(log n).

Best, Worst, Average

One algorithm, many costs

```
def find(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

Case	When	Comparisons
Best	target is `arr[0]`	1 — $O(1)$
Worst	target is last, or absent	n — $O(n)$
Average	present, every position equally likely	$\frac{n+1}{2}$ — $O(n)$

The cost of an algorithm is not one function of n; it depends on *which* input of size n you give it. So we name three:

Worst case — the maximum over all inputs of size n. This is the default: when someone says "linear search is O(n)" they mean the worst case. It is a **guarantee**: no input can do worse.

Best case — the minimum. Rarely useful on its own. "My sort is O(n) in the best case" usually means "on input that was already sorted", which is not a reason to choose it.

Average case — the expected cost over some *distribution* of inputs. It needs an assumption, and you must state it. For linear search, if the target is present and equally likely to be in any of the n positions, the expected number of comparisons is (1 + 2 + … + n)/n = (n + 1)/2. Still O(n): on average you search half the array, and half is a constant.

Do not confuse these with O, Ω and Θ. "Worst case" says *which input*; "Big-O" says *what kind of bound*. "The worst case of linear search is Θ(n)" is a perfectly sensible sentence that uses both.

Measured: same n, different cost

This is the cost of inserting one element into an array of 20,000, depending on *where*. At index 0 every element must shift; at the end, none do. Same operation, same n, a factor of thousands between best and worst. The straight line is the cost being exactly proportional to the number of elements after the insertion point, n − i. Averaged over every position that is n/2 — O(n).

You will produce the points on that line yourself, with your own `insert_at` from `dsa/array_ops.py`.

Time and space

Space complexity: how the *extra* memory an algorithm needs grows with n.

Function	Extra space	Why
`find(arr, x)`	$O(1)$	two variables, whatever n is
`reverse_in_place(arr)`	$O(1)$	swaps within the array

Function	Extra space	Why
`resized(arr, 2n)`	$O(n)$	a whole new array
`merge_sorted(a, b)`	$O(n+m)$	the output array
recursive `total(values)`	$O(n)$	one stack frame per element

We usually count **auxiliary** space — memory beyond the input itself. Reversing an array in place needs two indices and one temporary: O(1). Copying it needs n new slots: O(n).

The last row is the one people forget. **Recursion uses space.** Every call that has not yet returned holds a frame on the call stack (Lab 03, Part 10). A recursive function n calls deep uses O(n) stack space even if it creates no data at all — and Python stops it at about 1,000 frames.

Time and space often trade against each other. Merge sort (Week 10) buys its O(n log n) time with O(n) extra space; heap sort gets the same time in O(1) extra space and pays in other ways. Knowing which you are paying for is know-why.

Reading the Cost Off the Code

Five rules

1. **Sequence — add.** $O(f)$ then $O(g)$ is $O(f+g)$ = the bigger one.
2. **Loop — multiply.** A loop of n iterations with an $O(g)$ body is $O(n \cdot g)$.
3. **Nested loops — multiply again.** Two full nested loops over n: $O(n^2)$.
4. **Halving — logarithm.** A loop that halves what is left is $O(\log n)$.
5. **Function calls cost what the function costs** — including built-ins.

Rule 1: an O(n) loop followed by an O(n^2) loop is O(n + n^2) = O(n^2). The bigger term swallows the smaller.

Rule 2 needs care about *what the body costs*. A body of three basic steps is O(1), so n iterations cost O(n). But a body that itself does O(n) work — see rule 5 — makes the loop O(n^2).

Rule 3 is rule 2 applied twice. The triangular loop (`j` from `i + 1`) is still O(n^2): it does half the work of the square loop, and half is a constant.

Rule 4 is where log n comes from:

```
steps = 0
while n > 1:
    n //= 2
    steps += 1        # runs about log2(n) times
```

Rule 5 is the one that catches Python programmers, and it is why the second half of this lecture exists.

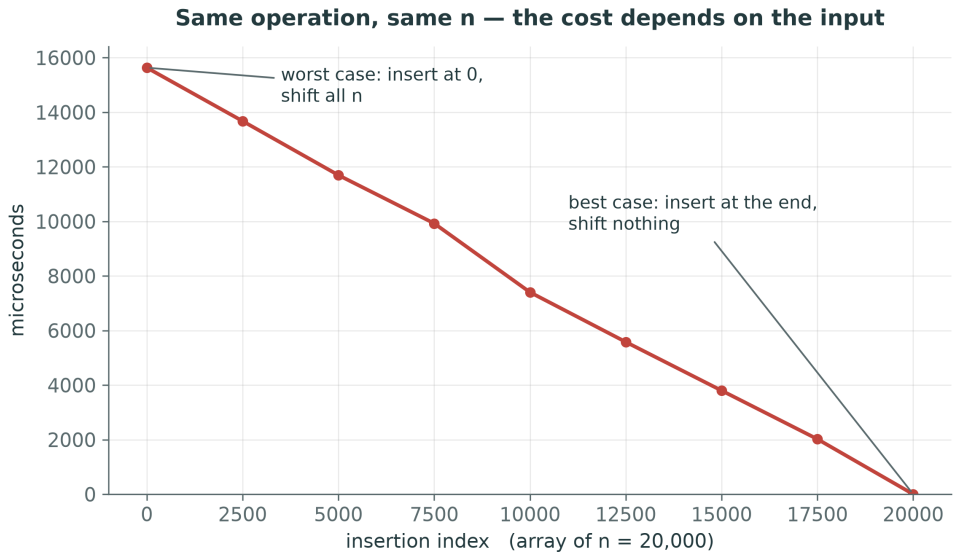


Figure 13: Insertion into an array of 20,000 elements, at nine different positions. Real timings on the course `Array`.

Rule 5: the hidden loop

```
def has_duplicate(values):
    for i in range(len(values)):
        if values[i] in values[i + 1:]:     # <- how much does this line cost?
            return True
    return False
```

One line, **two hidden O(n) loops**: the slice **copies** up to n elements, and `in` **searches** them one by one. Inside an n-iteration loop: $O(n^2)$.

A call is only O(1) if you **know** it is O(1).

This is the most important slide for Python programmers, and the reason this course is built the way it is.

`values[i + 1:]` looks like one step. It builds a new list and copies every element after i into it — O(n). `x in some_list` looks like one step. It walks the list comparing — O(n). `list.insert(0, x)` looks like one step. It shifts every element — O(n). `list.pop(0)`, `list.remove(x)`, `list.index(x)`, `sorted(xs)`: all look like one step, none is.

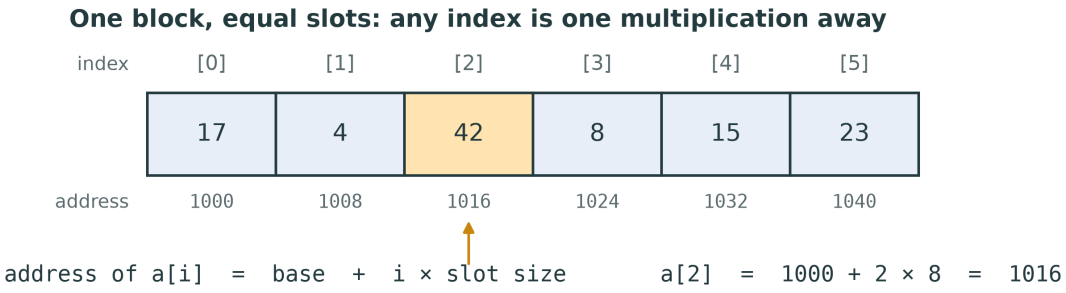
Python's built-ins are superbly engineered, and their costs are documented. But they are *hidden* inside one short line, and a student who has never had to write them does not see them. You cannot count the cost of code you cannot see.

So, from today, you are going to see it.

The Array

What an array is

A **fixed number** of **equal-sized slots**, stored **side by side** in memory.



$$\text{address of } a[i] = \text{base} + i \times \text{slot size}$$

One multiplication and one addition — **whatever i is, whatever n is.** That is why `a[i]` is $O(1)$.

Everything about the array follows from that picture.

O(1) indexing comes from two properties together: the slots are **contiguous** (no gaps, one block), and they are **equal-sized**. Then the address of slot i is arithmetic, not a search. Take away either property and indexing stops being O(1) — a linked list (Week 5) keeps its elements anywhere in memory, so reaching element i means following i links.

Fixed size is the price. The block was reserved at a particular size, and the memory right after it may already belong to something else. The array cannot grow in place. To get a bigger one you must reserve a new block and copy — O(n). That is the whole of Week 4.

Why indices start at 0. Because i is an *offset* from the base: element 0 is zero slots from the start. Languages that start at 1 must subtract 1 somewhere.

Array in Arabic is المصفوف. You will also hear *contiguous memory* as ذاكرة متجاورة.

In C, the array is the machine

```c
int a[6] = {17, 4, 42, 8, 15, 23};   // 6 × 4 bytes, reserved once
a[2] = 99;                           // base + 2 × 4: one instruction
a[6] = 0;                          // no check — writes over whatever is next
```

- The size is part of the type, fixed at compile time
- Every slot holds an `int`, so every slot is 4 bytes
- **No bounds check**: an out-of-range index is not an error, it is a bug

C is the language that most directly exposes what an array is — which is why it is worth two minutes, even though we write Python. (Kernighan and Ritchie, the book Lecture 01 recommended, spends a chapter on it.)

The last line is famous. C does not check the index. `a[6]` computes base + 6 × 4 and writes there, over whatever variable happens to live next. This is the root of a whole family of security

holes, *buffer overflows*. Every array in this course **does** check, and raises `IndexError` — the extra comparison is O(1) and worth it.

Arrays in Python: four candidates

	Grows?	Holds	Hides the cost?
`list`	yes — a dynamic array	references to any objects	**yes**: `append`, `insert`, `pop(0)`, `in`, slicing
`array.array`	yes	numbers only, one type, packed	yes: same methods as `list`
`numpy.ndarray`	no	numbers, one type, packed	yes: whole-array operations in C
`ctypes` array	**no**	C values, or references	**no** — index, assign, nothing else

Python does not have a plain, C-style array as a built-in type. What it has:

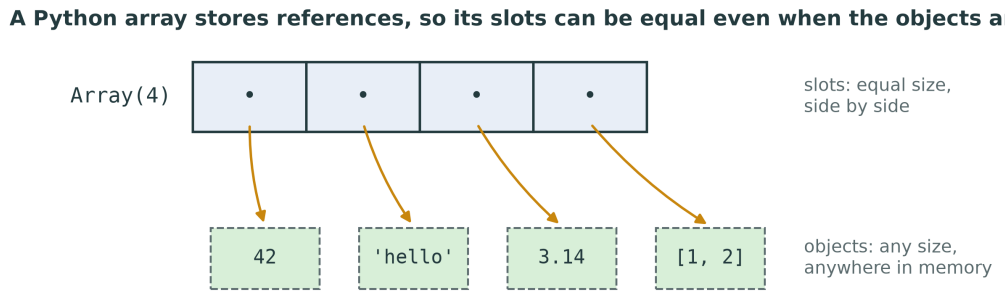
`list` is itself a **dynamic array** — the structure you will build in Week 4. Under the hood it is exactly a block of references plus a size and a capacity. Every convenient method on it is an algorithm with a cost, already written for you.

`array.array` (the standard-library `array` module) stores numbers packed tightly — `array('i', [1, 2, 3])` uses 4 bytes per element instead of a reference to an int object. It is compact, and useful for large numeric data. But it can `append`, `insert` and `pop` just like a list: it is a dynamic array too.

NumPy arrays are fixed-size and packed, and the backbone of data science and of most of what the AI program does with data. But their power is whole-array operations — `a + b` adds a million numbers in one line, in C. Wonderful for computation; useless for learning what one insertion costs.

`ctypes` arrays are real C arrays, reached from Python. They have a fixed size, and you can index them and assign to them — nothing else. That is exactly the primitive we want.

Python stores references



A Python array slot does not hold 42; it holds **a reference to** the object 42. Slots are equal-sized — one reference each — even when the objects are not. That is how one array holds an int, a string and a list, and still indexes in O(1).

This is the ladder from Lecture 01 again: a name is bound to an object, and a collection is a collection of references. The slot size is the size of one reference (8 bytes on a 64-bit machine), so the address arithmetic works.

The cost is a second hop: `a[i]` finds the reference in O(1), then follows it to the object, which lives wherever Python put it. Packed arrays (`array.array`, NumPy) avoid that hop by storing the numbers themselves, which is why they are faster for numerical work — and why they can hold only one type.

The Course Array

`dsa/array.py` — given to you

```
from dsa.array import Array

a = Array(5)                    # 5 slots, all None. Fixed for ever.
a[0] = 42                       # O(1)
a[0]                            # O(1) -> 42
len(a)                          # 5: the capacity, not "how many are used"
list(a)                         # iterate: [42, None, None, None, None]

Array.from_values([3, 1, 2])  # a full array, for tests and the REPL
Array(3, fill=0)              # every slot starts as 0
```

A real C array of references (`ctypes`), wrapped so that it **checks its bounds**.

`dsa/array.py` is infrastructure, like `viz/`: already written, read it, do not change it. It is thirty lines, and every one is worth reading.

Three details:

- **`len(a)` is the capacity.** An `Array(5)` always has length 5, whether you have put anything in it or not. Tracking how many slots are *in use* — the **size** — is your job, in a separate variable. That split, capacity against size, is the heart of the dynamic array.
- **Every slot starts as `None`** (or the `fill` you give). A raw ctypes slot starts out NULL — reading it raises a confusing `ValueError` — so the class fills them.
- **`from_values`** copies an iterable into a new, exactly-full Array. It is for building test data and experimenting at the REPL. Do not use it inside a structure to dodge the rules — converting to a tuple and back is exactly the kind of hidden O(n) this lecture is about.

What it will not do

```
>>> a = Array(3)
```

```
>>> a[3]
IndexError: Array index 3 out of range for length 3 ...
>>> a[-1]
IndexError: ...                    # no negative indices, as in C
>>> a[0:2]
TypeError: Array does not support slicing; copy with a loop
>>> a.append(1)
AttributeError: 'Array' object has no attribute 'append'
```

No growing. No shrinking. No inserting. No slicing. No searching methods. **Every one of those is an algorithm — and you are going to write them.**

Each refusal is deliberate.

No negative indices, because an array index is an offset from the base, and there is nothing before the base. When you build `DynamicArray` in Week 4 you will *implement* negative indexing (`arr[-1]` is the last element), and you will see that it is a translation, `i + size`, on top of an array that has none.

No slicing, because a slice is a copy — O(k) — dressed as one step. If you want a copy, write the loop and see the cost.

No append, because an array cannot grow. That is the definition.

The operations, and what they cost

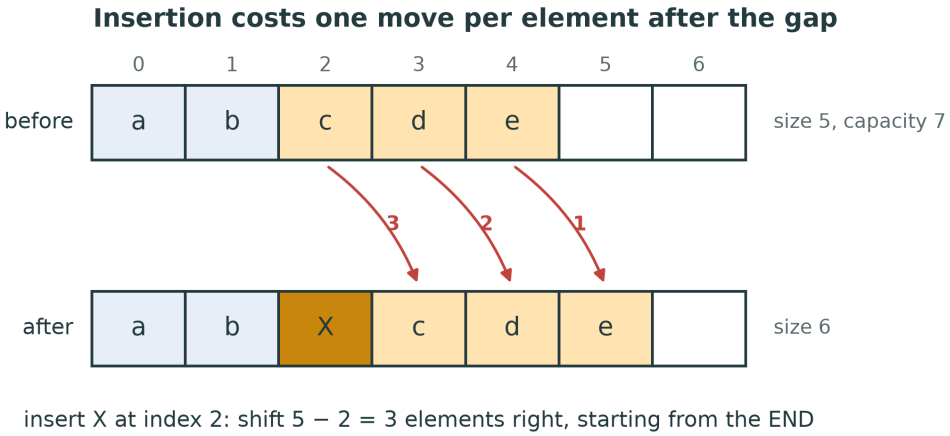
Operation	How	Cost
read / write slot `i`	address arithmetic	$O(1)$
find a value	check slots one by one	$O(n)$
insert at `i`	shift `i .. size-1` **right**, then write	$O(n)$
remove at `i`	shift `i+1 .. size-1` **left**	$O(n)$
insert / remove at the **end**	nothing to shift	$O(1)$
grow	new array, copy everything	$O(n)$

This table is the array's **contract** — its ADT, with costs. Every row is a function in `dsa/array_ops.py` or a consequence of one, and every structure you build this term inherits these costs from the array underneath it.

Look at the two rows about the end. Inserting and removing at the *end* is O(1), because nothing has to move. That single fact decides:

- that a **stack** should keep its top at the *end* of an array (Week 6);
- that a **queue** cannot simply remove from the front of an array — which is why Week 7 needs a ring buffer;
- that a **heap** adds at the end and repairs upward (Week 12).

Insertion, slot by slot




```
def insert_at(arr, size, index, value):
    for i in range(size, index, -1):   # from the END, backwards
        arr[i] = arr[i - 1]
    arr[index] = value
    return size + 1
```

This is the core of `insert_at` from this week's exercise, without the error checks (the full contract is in the docstring: `OverflowError` when full, `IndexError` for a bad index).

Why backwards? Shift forwards — copy `arr[2]` into `arr[3]` first — and you have overwritten d before moving it; every slot ends up holding c. Moving right, you must start at the right. Moving left (removal), you start at the left. Trace both on paper once; it is the most common bug in this week's exercises.

The loop runs size − index times. Index 0: size moves, the worst case. Index size: zero moves, which is just writing at the end.

Watch it run: the Array Lab

Array Lab — open it in any browser.

- The code, the variables and the **slots**, one step at a time
- Every **shift** animated; a counter of reads and writes
- `arr[i]`, `find`, `insert_at`, `remove_at`, `reverse_in_place`, `rotate_left`
- *What happens next?* — answer before you step
- Scenarios that **go wrong**: shifting from the front, a stale last slot, one step too far, `arr[-1]`

The Array Lab is the demo used in the lecture. It runs the code of this week's exercises on the course `Array` and shows each step: the line that runs, the variables, and the slots, with in-use slots, free slots and the `size` boundary drawn differently. Use it to check your hand traces.

- **Online:** https://helghareeb.github.io/DSA27/lectures/02-complexity-and-arrays/array-lab.html

- **Offline:** open `docs/lectures/02-complexity-and-arrays/array-lab.html` from your copy of the repository. It is a single file and needs no installation.

How to use it:

- Pick a scenario at the top. Change the inputs in the >>> line and press **Run**.
- **Step** (→) moves one step and **Back** (←) undoes it. **Step over** (O) runs a whole call (an append, a `_resize`, a reversal) in one step; **Step out** (U) finishes the current call. **Play** (P) runs on its own.
- At the key moments the demo stops with a question. Answer it (A–D) **before** you look; that is where the learning happens.

What to try, in order: `arr[i]` (one slot looked at, whatever n is), `find` best case against worst case, `insert_at` at the front against at the end (count the shifts), and then every scenario under *Goes wrong*. For each broken one, find the bug in the code **before** you step.

The scenarios show code that solves this week's exercises in `dsa/array_ops.py`. Write your own version first. The *Dynamic array* scenarios belong to Lecture 04.

Measured, not asserted

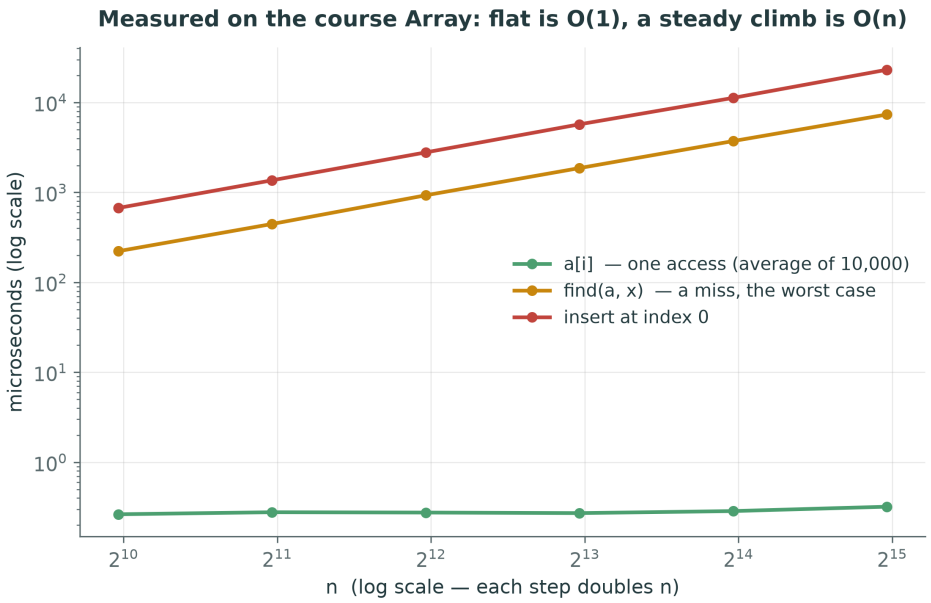


Figure 14: Real timings on the course `Array`. Each step right doubles n.

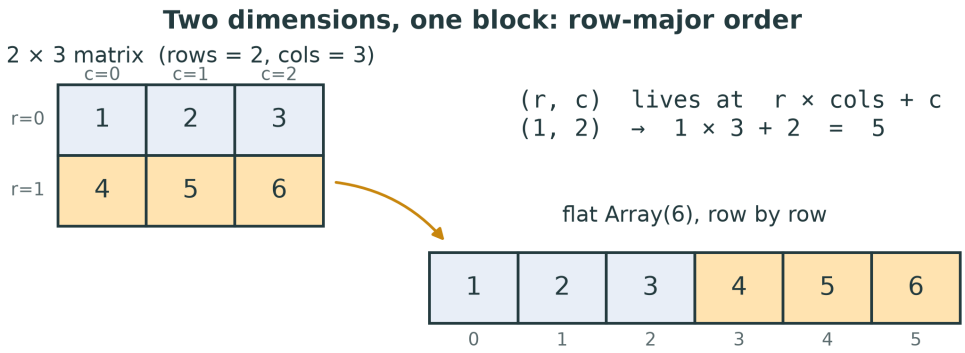
Three operations on the course `Array`, timed with `viz.complexity.measure` as n doubles from 1,024 to 32,768. Both axes are logarithmic, so a straight line climbing one grid step per doubling is O(n), and a flat line is O(1).

- **Indexing** stays at about a quarter of a microsecond whatever n is. (One access is too fast to time on its own, so the figure times 10,000 and divides — an honest measurement has to know its own limits.)
- **Linear search for a missing value** doubles when n doubles: O(n).

- **Inserting at index 0** also doubles: O(n) — and it sits *above* the search, because each step does a read *and* a write where search does a read and a comparison. Same Big-O, different constant: exactly the thing Big-O throws away, and exactly the thing a measurement shows you.

At n = 32,768, one index is roughly 25,000 times cheaper than one front insertion. That is the difference between O(1) and O(n), in microseconds.

Two dimensions, one block



A matrix is stored as **one** array, row after row: element (r, c) lives at index $r \times \text{cols} + c$.

Memory is one-dimensional, so a 2-D array has to be flattened. **Row-major** order — row 0, then row 1, … — is what C and NumPy use by default; Fortran and MATLAB use column-major. The formula is address arithmetic again, so (r, c) is still O(1).

The alternative is an array of arrays: an `Array` of `rows` slots, each holding an `Array` of `cols`. It works, costs an extra hop per access, and lets rows have different lengths. `transpose_flat` in this week's exercises uses the flat layout, so that you work the index arithmetic by hand.

The Rule, From Today

No `list`, `dict` or `set` inside a structure

A data structure in `dsa/` keeps its data **only** in:

1. the course **`Array`** (`dsa/array.py`), or
2. **node objects** — `Node`, `TreeNode`, `Entry`, …, or
3. **another structure you have already built** in `dsa/`.

Never in a Python `list`, `dict`, `set` or `collections.deque`.

Lists remain the **interface**: functions may take a list and return a list, and the tests use them freely.

This is the rule that governs the rest of the course. Here is exactly how the repository applies it, so that you can see the structures stacking up week by week:

Week	Structure	Stores its data in
4	`DynamicArray`	an `Array`, replaced by a bigger one when full
5	`LinkedList`	`Node` objects
6	`Stack`	your `DynamicArray` — top at the end
7	`SlowQueue` / `CircularQueue`	your `DynamicArray` / an `Array` as a ring
11	`BinarySearchTree`	`TreeNode` objects
12	`MinHeap`, `PriorityQueue`	your `DynamicArray` / your `MinHeap`
13	`ChainingHashMap` / `OpenAddressingHashMap`	an `Array` of `Entry` chains / two `Arrays`
14	`Graph` / `MatrixGraph`	your hash map and `DynamicArrays`

Read down the right-hand column. By Week 14 your graph is built on your hash map, which is built on your array; your breadth-first search runs on your queue, which runs on your array. Nothing in the stack is borrowed. When something is slow, you will know which line of *your* code made it slow.

Why not lists? Not because lists are bad — they are excellent, and you will use them for the rest of your career. Because a list hides, behind one convenient method, exactly the cost this course exists to teach. A stack built on `list.append` teaches you nothing about why a stack is O(1). A stack built on an array you had to grow yourself does.

What is still allowed. Python lists as *function inputs and return values* — `bfs` returns a list of nodes, `sorting` functions take one — because that is the convenient interface for tests and for you. Plain local variables. Tuples to return two values. What is not allowed is a `list`, `dict` or `set` as the *storage* of a structure, or as the working storage of an algorithm that has a structure of yours available (use your `CircularQueue` for BFS, not a list).

The TAs will check this when they mark. A structure that passes every test by hiding a `list` inside scores as not done.

The same idea, all term

```
Array                          (given, O(1) index, fixed size)
 └── DynamicArray              (Week 4: grow by copying — amortised O(1)
↪  append)
      ├── Stack                (Week 6)
      ├── SlowQueue            (Week 7 — pop(0) is O(n), measure it)
      └── MinHeap              (Week 12)
            └── PriorityQueue
 ├── CircularQueue             (Week 7 — O(1) both ends, on a ring)
```

```
└── hash maps              (Week 13)
    └── Graph              (Week 14, with DynamicArray)
```

Every arrow is "is built on". Linked lists and trees sit beside this tree, built from nodes rather than arrays — and Week 5 exists to compare the two families directly.

This Week

Exercises: `dsa/array_ops.py`

Function	Target
`find(arr, target, size=None)`	$O(n)$ worst, $O(1)$ best
`is_sorted(arr, size=None)`	$O(n)$, stop early
`insert_at(arr, size, index, value)`	$O(n)$ — shift right, from the end
`remove_at(arr, size, index)`	$O(n)$ — shift left
`reverse_in_place(arr)`	$O(n)$ time, $O(1)$ space
`rotate_left(arr, k)`	$O(n)$ time, $O(1)$ space — three reversals
`resized(arr, new_length)`	$O(n)$ — the dynamic array's one move
`merge_sorted(a, b)`	$O(n+m)$ — half of merge sort
`transpose_flat(arr, rows, cols)`	$O(rows \times cols)$

```
pytest tests/test_array.py -v          # the Array itself — passes already
pytest tests/test_array_ops.py -v      # your exercises
```

Every function takes an `Array` and works with indices only: no slicing, no converting to a list, no `list` scratch space. Where a function builds a new array, it builds an `Array`.

Several of them take a `size`: the array is only partly filled, and the slots from `size` on are free. That is how `insert_at` and `remove_at` can change "how many elements there are" without the array changing length.

`rotate_left` is the one that needs an idea. Rotating [1 2 3 4 5] left by 2: reverse the first 2 → [2 1 3 4 5]; reverse the rest → [2 1 5 4 3]; reverse the whole → [3 4 5 1 2]. Three reversals, O(n) time, no second array. Convince yourself on paper *why* it works before you write it.

Homework 2 — before Lecture 03

1. **Implement** `dsa/array_ops.py` until `pytest tests/test_array_ops.py` passes.
2. **Measure** your `insert_at` at index 0 and at the end, for n = 1,000 … 32,000, with `viz.complexity.measure`, and plot both with `plot_growth(..., reference=["1", "n"])`. One paragraph: do the curves match the table?
3. **Prove** $5n^2 + 3n + 7 = O(n^2)$: give c and n_0 and show the inequality.
4. **Read off** the Big-O of the three snippets in the handout, and say which rule you used for each.
5. **Lab 02** exercises, if not finished in the lab.

For item 2, the Lecture 01 example shows the shape of the call. A starting point:

```
from dsa.array import Array
from dsa.array_ops import insert_at
from viz.complexity import measure, plot_growth

sizes = [1000, 2000, 4000, 8000, 16000, 32000]

def make(n):
    arr = Array(n + 1)
    for i in range(n):
        arr[i] = i
    return arr, n

front = measure(lambda a: insert_at(a[0], a[1], 0, -1), sizes, make)
end   = measure(lambda a: insert_at(a[0], a[1], a[1], -1), sizes, make)
plot_growth({"insert at 0": front, "insert at end": end}, reference=["1",
 ↪  "n"])
```

For item 4, the three snippets:

```
# (a)
for i in range(n):
    for j in range(10):
        total += i * j

# (b)
i = n
while i > 0:
    for j in range(n):
        count += 1
    i //= 2

# (c)
for i in range(n):
    if find(arr, i) != -1:          # arr has n elements
        hits += 1
```

Hand in items 2–4 as one page (a PDF, or a notebook with the plot) through the channel the TAs announce.

Summary

Seven things to keep

1. Count **basic steps** as a function of n; keep the **shape**, drop the constants and the lower-order terms.

2. $f = O(g)$: **eventually** $f \leq c \cdot g$. Ω is ≥; Θ is both.
3. **Worst case** is the guarantee. Say which case you mean.
4. **Sequence adds, loops multiply, halving is log n** — and a call costs what the function costs.
5. An array is **contiguous, equal-sized slots**: $O(1)$ index, **fixed size**, $O(n)$ insert and search.
6. The course **`Array`**: index, assign, length. Nothing else.
7. From today: structures store data in **`Array`, nodes, or your own structures** — never `list`, `dict` or `set`.

Next

Week 3 — Recursion. Base case, recursive case, and the call stack — the first place where *how* you solve a problem decides its complexity.

Lab 03 this week ends with a bridge into it.

Sources and further reading

- **Python Software Foundation.** *TimeComplexity* — the documented costs of `list`, `collections.deque`, `set` and `dict` operations in CPython. The page to check before you believe any one-line Python operation is O(1).
- **Python Software Foundation.** *ctypes — A foreign function library for Python*, section "Arrays". The mechanism under `dsa/array.py`.
- **B. W. Kernighan and D. M. Ritchie.** *The C Programming Language*, 2nd ed., Prentice Hall, 1988, chapter 5, "Pointers and Arrays" — the address arithmetic on the array slides, from the people who designed it.
- **T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein.** *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, chapter 3, "Growth of Functions" — the formal definitions of O, Ω and Θ used here.
- **M. T. Goodrich, R. Tamassia and M. H. Goldwasser.** *Data Structures and Algorithms in Python*, Wiley, 2013, chapters 3 and 5 — algorithm analysis, and array-based sequences built on `ctypes`, which is the approach this course takes.

Every figure in this lecture is generated by `tools/figures_l02.py`; the two timing figures are real measurements and will differ slightly on your machine. Run the script and compare.

Questions for Week 2

> ***The answers are at the back of the book,*** *in* Answers — Week 2. *Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify. Unless a question says otherwise, n is the input size and `Array` is the course array from `dsa/array.py`.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W2-M01 – W2-M26

Part	Type	Questions
B	Short answer and essay	W2-E1 – W2-E6
C	Complexity analysis — give Θ and justify	W2-K1 – W2-K6
D	Proofs	W2-P1 – W2-P3
E	Array state — show every step	W2-S1 – W2-S4
F	Trace the code	W2-T1 – W2-T4
G	Find and fix the bug	W2-B1 – W2-B3
H	Write the code — checked by `pytest`	W2-C1 – W2-C4

Part A — Multiple choice

W2-M01 [how] $f(n) = 3n^2 + 5n + 2$ is:

- **a)** $\Theta(n)$
- **b)** $\Theta(n^2)$
- **c)** $\Theta(n^3)$
- **d)** $\Theta(n \log n)$

W2-M02 [why] Which statement is **true**?

- **a)** $n^2 = O(n)$
- **b)** $n = \Omega(n^2)$
- **c)** $n = O(n^2)$
- **d)** $2^n = O(n^3)$

W2-M03 [what] $f(n) = O(g(n))$ means:

- **a)** there are constants $c > 0$ and n_0 with $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$
- **b)** $f(n) \leq g(n)$ for every n
- **c)** $f(n) = g(n)$ for all large n
- **d)** there are constants $c > 0$ and n_0 with $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$

W2-M04 [how] In the **worst case**, how many comparisons does linear search make on an array of n elements?

- **a)** 1
- **b)** n / 2
- **c)** $\log_2 n$
- **d)** n

W2-M05 [how] The target is present and equally likely to be at any of the n positions. On **average**, how many comparisons does linear search make?

- **a)** $(n + 1)/2$
- **b)** n
- **c)** $\log_2 n$
- **d)** $n^2/2$

W2-M06 [how] What is the running time of this loop?

```
i = n
while i > 1:
    i = i // 2
```

- **a)** $\Theta(1)$
- **b)** $\Theta(n)$
- **c)** $\Theta(\log n)$
- **d)** $\Theta(n/2)$

W2-M07 [how] What is the running time?

```
for i in range(n):
    for j in range(i):
        count += 1
```

- **a)** $\Theta(n)$
- **b)** $\Theta(n \log n)$
- **c)** $\Theta(i \cdot n)$
- **d)** $\Theta(n^2)$

W2-M08 [how] What is the running time?

```
for i in range(n):
    for j in range(5):
        count += 1
```

- **a)** $\Theta(n)$
- **b)** $\Theta(n^2)$
- **c)** $\Theta(5^n)$
- **d)** $\Theta(\log n)$

W2-M09 [what] For a Python `list` of n items, `x in values` costs:

- **a)** $O(1)$, because lists are hash tables
- **b)** $O(n)$
- **c)** $O(\log n)$
- **d)** $O(n^2)$

W2-M10 [what] Which Python `list` operation is $O(1)$ (amortised)?

- **a)** `values.insert(0, x)`
- **b)** `values.pop(0)`
- **c)** `values.append(x)`
- **d)** `values.remove(x)`

W2-M11 [why] Why is reading `a[i]` from an array $O(1)$?

- **a)** The array is sorted
- **b)** Python caches the last element read

- **c)** The array searches from both ends at once
- **d)** The slots are contiguous and equal-sized, so the address is base + i × slot size

W2-M12 [how] A partly filled array has **size 6**. How many elements must move to insert a new value at **index 2**?

- **a)** 4
- **b)** 2
- **c)** 6
- **d)** 3

W2-M13 [how] When inserting into an array, the elements after the gap must be shifted right. In which order?

- **a)** From the insertion index towards the end
- **b)** From the last used slot back towards the insertion index
- **c)** Any order — the result is the same
- **d)** Swap each element with its neighbour, from the front

W2-M14 [how] `a = Array(4)`, then `a[-1]`. What happens?

- **a)** returns the last slot, `None`
- **b)** returns `a[3]`
- **c)** raises `IndexError`
- **d)** raises `TypeError`

W2-M15 [how] `a = Array(5)`, then `a[0] = 7`. What is `len(a)`?

- **a)** 1
- **b)** 0
- **c)** 6
- **d)** 5

W2-M16 [how] A 3 × 4 matrix (3 rows, 4 columns) is stored flat in row-major order. At which index is element (2, 1)?

- **a)** 7
- **b)** 9
- **c)** 6
- **d)** 11

W2-M17 [how] What extra space does `reverse_in_place(arr)` need (swap from both ends, no second array)?

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n^2)$

W2-M18 [why] Under the course storage rule (Lecture 02), which of these is **not** allowed?

- **a)** a `Stack` storing its items in the student's own `DynamicArray`
- **b)** a `Graph` storing adjacency in the student's own `ChainingHashMap`

- **c)** a queue class storing its items in a Python `list`
- **d)** `bfs` returning the visit order as a Python `list`

W2-M19 [what] Python's built-in `list` is best described as:

- **a)** a linked list of values
- **b)** a fixed-size C array of integers
- **c)** a hash table keyed by position
- **d)** a dynamic array of references to objects

W2-M20 [how] An $O(n)$ loop is followed by an $O(n^2)$ loop. The whole is:

- **a)** $O(n^2)$
- **b)** $O(n^3)$
- **c)** $O(n)$
- **d)** $O(n + n)$

W2-M21 [how] `values` is a Python list of n items. The cost of:

```
for i in range(len(values)):
    if values[i] in values[i + 1:]:
        return True
```

- **a)** $O(n)$
- **b)** $O(n^2)$
- **c)** $O(n \log n)$
- **d)** $O(1)$

W2-M22 [what] When does the `else` block of a `for` loop run?

- **a)** Whenever the loop body raises an exception
- **b)** Only if the loop runs zero times
- **c)** When the loop finishes without executing `break`
- **d)** After every iteration

W2-M23 [how] Given

```
def f(a, L=[]):
    L.append(a)
    return L
```

what does `f(1)` then `print(f(2))` print?

- **a)** `[2]`
- **b)** `[1]`
- **c)** `[2, 1]`
- **d)** `[1, 2]`

W2-M24 [how] What is the value of `0 or 'x' and ''`?

- **a)** `''`
- **b)** `'x'`
- **c)** `0`

- **d)** `False`

W2-M25 [how] `list(range(2, 11, 3))` is:

- **a)** `[2, 5, 8, 11]`
- **b)** `[2, 5, 8]`
- **c)** `[3, 6, 9]`
- **d)** `[2, 3, 4, 5, 6, 7, 8, 9, 10]`

W2-M26 [how] What does `print(g())` print?

```
def g():
    try:
        return 1
    finally:
        print("bye")
```

- **a)** 1
- **b)** bye
- **c)** bye then 1
- **d)** 1 then bye

Part B — Short answer and essay

W2-E1 [why] *(4 marks)* Big-O throws away constant factors and lower-order terms. Explain *why* each is thrown away, give one situation where doing so is misleading, and explain how measuring complements counting.

W2-E2 [what] *(4 marks)* Define O, Ω and Θ precisely. For $f(n) = 3n^2 + 2n$, give one true statement using each, and one *true but uninformative* O statement.

W2-E3 [why] *(3 marks)* Describe the best, worst and average case of linear search, with the number of comparisons for each. Which one does "linear search is $O(n)$" usually refer to, and why is that the default?

W2-E4 [why] *(4 marks)* Explain why indexing an array is $O(1)$ but inserting into the middle is $O(n)$. What does the fact that an array has a *fixed size* force you to do when it fills up, and what does that cost?

W2-E5 [why] *(4 marks)* State the course storage rule from Lecture 02. Give the reason for it, list what remains allowed, and show how two structures from later in the course follow it.

W2-E6 [what] *(3 marks)* Compare Python's `list`, `array.array`, `numpy.ndarray` and a `ctypes` array on: can it grow, what it holds, and whether it hides costs. Why is the course `Array` built on `ctypes`?

Part C — Complexity analysis

Give the running time in Θ notation as a function of n, and **justify** it in one or two sentences (which rule, which sum). Where best and worst differ, give both.

W2-K1 [how]

```
total = 0
for i in range(n):
    for j in range(n):
        total += i * j
for k in range(n):
    total += k
```

W2-K2 [how]

```
i = 1
while i < n:
    i = i * 2
```

W2-K3 [how]

```
for i in range(n):
    j = n
    while j > 1:
        j = j // 2
```

W2-K4 [how] (`arr` is an `Array` of n elements.)

```
def has_duplicate(arr):
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] == arr[j]:
                return True
    return False
```

W2-K5 [how] (`values` is a Python list of n items.)

```
for i in range(n):
    rest = values[i:]
```

W2-K6 [how]

```
count = 0
for i in range(n):
    for j in range(100):
        for k in range(i):
            count += 1
```

Part D — Proofs

W2-P1 [how] *(3 marks)* Prove that $2n + 10 = O(n)$ by giving c and n_0 and showing the inequality.

W2-P2 [why] *(3 marks)* Prove that n^2 is **not** $O(n)$.

W2-P3 [how] *(4 marks)* Prove that $4n^2 + n = \Theta(n^2)$.

Part E — Array state

Draw the array after each step (use _ for an unused slot) and count the elements moved.

W2-S1 [how] An `Array` of capacity 7 holds `[10, 20, 30, 40, 50, _, _]`, size 5.

(a) `size = insert_at(arr, size, 1, 15)`
(b) then `remove_at(arr, size, 3)` (and `size` decreases by one)

W2-S2 [why] A student writes the shifting loop in insertion **forwards**:

```
for i in range(index, size):
    arr[i + 1] = arr[i]
arr[index] = value
```

Show the array after inserting `"X"` at index 1 into `["a", "b", "c", "d", _]`, size 4. What went wrong, and what is the fix?

W2-S3 [how] `rotate_left` by three reversals. Show the array after each reversal when rotating `[1, 2, 3, 4, 5, 6]` left by 2.

W2-S4 [how] The 3 × 4 matrix holding the values 0 to 11 row by row is stored flat in row-major order.

(a) Which value is at (1, 3)? At which flat index?
(b) Which (row, column) is flat index 10?
(c) After `transpose_flat` (now 4 × 3, row-major), at which flat index is the value that was at (1, 3)?

Part F — Trace the code

Write exactly what is printed.

W2-T1 [how]

```
for n in range(10, 16):
    for d in range(2, n):
        if n % d == 0:
            break
    else:
```

```
        print(n, end=" ")
print("done")
```

W2-T2 [how]

```
def run(command):
    match command.split():
        case ["add", a, b]:
            return int(a) + int(b)
        case ["neg", a]:
            return -int(a)
        case ["sum", *rest]:
            return len(rest)
        case _:
            return "?"

print(run("add 2 3"), run("neg 4"), run("sum"), run("sum 1 2 3"), run("add
 ↪  1"))
```

W2-T3 [how]

```
def f(x):
    try:
        print("A", end=" ")
        y = 10 // x
        print("B", end=" ")
    except ZeroDivisionError:
        print("C", end=" ")
        return 0
    else:
        print("D", end=" ")
        return y
    finally:
        print("E", end=" ")

print(f(2))
print(f(0))
```

W2-T4 [how]

```
def change(values, n):
    values.append(n)
    n = n * 10
    values = [0]
    return n

data, k = [1], 2
```

```
result = change(data, k)
print(data, k, result)
```

Part G — Find and fix the bug

W2-B1 [how] Should return `True` exactly when n is prime.

```
def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5)):
        if n % d == 0:
            return False
    return True
```

W2-B2 [how] Should return the index of `target` in `arr`, or −1.

```
def find(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
        else:
            return -1
```

W2-B3 [how] Should remove and return `arr[index]` from a partly filled array, shifting the rest left.

```
def remove_at(arr, size, index):
    value = arr[index]
    for i in range(index, size):
        arr[i] = arr[i + 1]
    return value
```

Part H — Write the code

In `practice/week02.py`; check with `pytest tests/test_practice_week02.py -v`. Work on the `Array` with indices only, and meet the stated complexity.

W2-C1 [how] `count_occurrences(arr, target, size=None)` — O(n).

W2-C2 [why] `remove_all(arr, size, target)` — remove every occurrence in **one pass**, O(n) time and O(1) extra space; return the new size. Why is calling `remove_at` once per occurrence $O(n^2)$?

W2-C3 [why] `two_sum_sorted(arr, target)` — in a **sorted** array, find i < j with `arr[i] + arr[j] == target` in O(n). Why is it correct to move only one index at each step?

W2-C4 [how] `prefix_sums(arr)` — a new `Array` with running totals, O(n). How does it let you answer "sum of `arr[i..j]`" in O(1)?

Week 3 — Recursion

> ***How to read this document.*** *This is the handout for Lecture 03. It holds everything on the slides, plus what I said out loud. Recursion is the first topic in this course where the* technique *is the lesson rather than a structure — and every later week leans on it: trees, merge sort and quicksort, depth-first search, and the parser in the final week.*
>
> *Slides:* `DSA27-L03-slides.pdf` *· Code:* `dsa/recursion.py` *· Tests:* `tests/test_recursion.py` *· Lab: Lab 03, Part 10 · Interactive demo: Recursion Lab*

Where We Are

Last week, in one line

Cost is counted as a function of n, and the **array** is the first structure we priced: $O(1)$ to index, $O(n)$ to insert or search.

Today: a way of **writing** algorithms — and a new way of **counting** what they cost.

Lecture 02 gave you the vocabulary of cost and applied it to loops. Loops are easy to count: multiply the iterations by the cost of the body. Today's algorithms have no loops at all, or hardly any. They call themselves. Counting them needs a new tool — the **recurrence** — and a new picture — the **recursion tree**.

Recursion is declared first in the topic list of the 2013 and 2014 bylaws ("Topics include recursion…", SWE 2013 p. 38; Medical Informatics 2014 p. 35), and none of the three programs teaches it before this course. So we start from nothing.

Today

1. A function defined in terms of itself — the two rules
2. What really happens: **the call stack**
3. Recursion over numbers, over arrays, over nested data
4. The cost of recursion: **recurrences** and **recursion trees**
5. The same answer at very different costs: Fibonacci, and power
6. **Branching** recursion: Hanoi, and every choice
7. Recursion or a loop?

The Idea

A definition that uses itself

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1)! & \text{if } n > 0 \end{cases}$$

```python
def factorial(n):
    if n == 0:                          # base case: answer directly
        return 1
    return n * factorial(n - 1)         # recursive case: a SMALLER problem
```

A **recursive** function solves a problem by solving a **smaller instance of the same problem**, and building its answer from that.

The mathematical definition and the Python are the same thing written twice. That is the appeal of recursion: when a problem is naturally defined in terms of a smaller version of itself, the code can say so directly.

Recursion in Arabic is الاستدعاء الذاتي — literally "self-calling".

(This `factorial` is the first function in `dsa/recursion.py`. The exercise also asks it to reject negative n with a `ValueError` — which the version above does not do. What *would* the version above do with `factorial(-1)`? Think about it before the next slide.)

The two rules

Every recursive function needs:

1. **A base case** — an input small enough to answer **without** recursing.
2. **A recursive case that makes progress** — every call must move **closer to the base case**.

Break rule 1, or rule 2, and the calls never stop:

```
>>> factorial(-1)          # -1, -2, -3, ... never reaches 0
RecursionError: maximum recursion depth exceeded
```

`factorial(-1)` calls `factorial(-2)`, which calls `factorial(-3)`, and so on for ever — moving *away* from the base case, not towards it. Rule 2 is broken for negative inputs. Python gives up after about a thousand calls with a `RecursionError`.

"Closer" needs a measure: a number that gets smaller with every call and cannot go below the base case. For `factorial` it is n itself. For a list it is the length. For a tree it will be the height. When you write a recursive function, **name the measure** — if you cannot, you probably have an infinite recursion.

Trust the recursion

To write the recursive case, **assume the recursive call already works** — for the smaller input — and ask only:

> Given the answer for the smaller problem, how do I build the answer for mine?

For `factorial(n)`: "if I had `factorial(n - 1)`, I would multiply it by n."

Do **not** trace every call in your head. That is the computer's job.

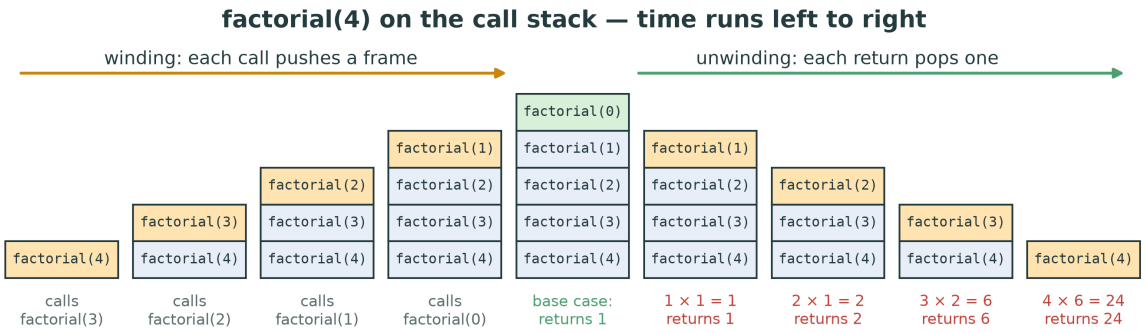
This is the hardest habit and the most important one. Beginners try to follow `factorial(5)` all the way down to `factorial(0)` and back, and get lost. You do not need to. If the base case is right, and the recursive case is right *assuming* the smaller call is right, then the whole function is right.

That argument has a name: **mathematical induction**. The base case is the induction base; the recursive case is the inductive step. You met induction in discrete mathematics (or you will); recursion is induction that runs. It is why people call this the "recursive leap of faith" — except it is not faith, it is a proof.

Trace by hand only to *debug*, or to answer an exam question that asks you to. Which brings us to what actually happens.

The Call Stack

Every call gets a frame



Each call pushes a **frame** holding its own n and where to return. The base case stops the growth; then each `return` pops a frame and hands its value down.

There is not "one n". Each call to `factorial` has **its own** local variables in its own **frame** on the **call stack**. When `factorial(4)` calls `factorial(3)`, the frame for `factorial(4)` is paused — it is waiting to do `4 * (...)` — and a new frame is pushed on top.

Read the figure left to right:

- **Winding.** Five calls, five frames. Nothing has been multiplied yet; every frame is waiting.
- **The base case.** `factorial(0)` returns 1 without calling anything. The stack stops growing.
- **Unwinding.** Each return pops the top frame and gives its value to the frame below, which can now finish its multiplication: 1 × 1, then 2 × 1, 3 × 2, 4 × 6. The last frame returns 24 to whoever called it.

This is exactly a **stack** — last in, first out — the ADT you build in Week 6. The most recent call is the first to finish. The same machinery runs every function call in every program you write; recursion only makes it visible.

See it yourself. Put `factorial` in a file, set a breakpoint on its first line in VS Code, press F5 and step with F11. The *Call Stack* panel shows one `factorial` frame per call, each with its own `n`. Lab 03, Part 10.4 walks you through it.

Watch it run: the Recursion Lab

Recursion Lab — open it in any browser.

- The code, the **call stack** and the **recursion tree**, one step at a time
- **Step over** a call: trust the recursion
- *What happens next?* — answer before you step
- Nine scenarios that **go wrong**: no base case, no progress, a missing `return`, a wrong base value, hidden $O(n^2)$...

The Recursion Lab is the demo used in the lecture. It runs every example in this handout, plus nine broken versions, and shows each step: the line that runs, every frame on the stack with its own variables, and the tree of calls growing. Use it to check your hand traces.

- **Online:** https://helghareeb.github.io/DSA27/lectures/03-recursion/recursion-lab.html
- **Offline:** open `docs/lectures/03-recursion/recursion-lab.html` from your copy of the repository. It is a single file and needs no installation.

How to use it:

- Pick a scenario at the top. Change the input in the `>>>` line and press **Run**.
- **Step** (→) moves one step and **Back** (←) undoes it. **Step over** (O) runs a whole call in one step: *Trust the recursion*, made visible. **Step out** (U) finishes the current call. Click any node of the tree to jump to the moment that call started.
- At the key moments the demo stops with a question. Answer it (A–D) **before** you look; that is where the learning happens.

What to try, in order: `factorial` (winding and unwinding), `fib` against `fib_memo` (watch the calls counter), and then every scenario under *Goes wrong*. For each broken one, find the bug in the code **before** you step.

Two scenarios, fast `power` and Hanoi, show code that solves an exercise in `dsa/recursion.py`. Write your own version first.

Depth is space

The call stack is memory. A recursion that goes **d** calls deep holds **d** frames at once:

Function	Deepest point	Extra space
`factorial(n)`	n + 1 frames	$O(n)$
`power(x, n)` by halving	about $\log_2 n$ frames	$O(\log n)$
`hanoi(n)`	n frames	$O(n)$ — though it makes 2^n calls

```
>>> import sys
>>> sys.getrecursionlimit()
1000
```

Space is decided by the **deepest** point, not the total number of calls: frames are popped as calls return, so they do not accumulate. `hanoi(20)` makes over a million calls, but never more than 20 are on the stack at once.

Python's limit. CPython refuses to go deeper than about 1,000 frames, and raises `RecursionError`. `sys.setrecursionlimit` can raise it, but the real stack underneath is finite, and a big enough recursion then crashes the interpreter outright. If a recursion needs to be 100,000 deep, the answer is not a bigger limit; it is a loop, or a recursion that is only log n deep. That is exactly what `tests/test_recursion.py::test_power_is_logarithmic` checks: an O(n)-deep `power(2, 4096)` hits the limit; an O(log n)-deep one uses about a dozen frames (13 or 14, depending on whether your base case is n = 0 or n = 1).

Many other languages eliminate the frame of a call made in *tail position* (`return f(n - 1)` as the very last action) — *tail-call optimisation*. Python deliberately does not, so in Python every recursive call costs a frame.

Recursion in Practice

Over numbers: shrink the number

```
def count_digits(n):
    """How many decimal digits in a non-negative int."""
    if n < 10:                        # one digit
        return 1
    return 1 + count_digits(n // 10) # drop the last digit
```

```
def to_binary(n):
    """'101' for 5. n >= 0."""
    if n < 2:
        return str(n)
    return to_binary(n // 2) + str(n % 2)
```

The measure: **n itself**, divided by 10 or 2 each call — so these are $O(\log n)$ deep.

Two patterns you will reuse: `n // 10` and `n % 10` take a decimal number apart (the exercise `sum_digits` is a close relative of `count_digits`); `n // 2` and `n % 2` take a binary number apart.

Notice where the work happens in `to_binary`: the recursive call builds the *leading* digits, and the current call appends the *last* one. The order of the digits comes out right because the concatenation happens on the way *back up* — during unwinding.

`gcd` in the exercises is Euclid's algorithm: gcd(a, b) = gcd(b, a mod b), with gcd(a, 0) = a. It is the oldest algorithm still in daily use, about 2,300 years old, and it is naturally recursive. Its measure

is b, which at least halves every two steps — so it is O(log min(a, b)).

Over an array: shrink the range, not the array

```
def array_max(arr, i=0):
    """Largest value in a non-empty Array, from index i on."""
    if i == len(arr) - 1:                    # one element left
        return arr[i]
    rest = array_max(arr, i + 1)             # the answer for the rest
    return arr[i] if arr[i] > rest else rest
```

The recursion moves an **index**; the array never changes. $O(n)$ time, $O(n)$ stack.

This works on the course `Array` from Lecture 02 — which has no slicing — and that is not a limitation, it is the right design. The tempting version on a Python list is

```
def list_max(values):
    if len(values) == 1:
        return values[0]
    rest = list_max(values[1:])              # a slice COPIES the rest: O(n)
    return values[0] if values[0] > rest else rest
```

and it is **O(n²)**, not O(n): each call copies everything after it, so the copies add up to (n − 1) + (n − 2) + … + 1. It is Lecture 02's hidden-loop rule again. Recursion over a sequence should pass **indices** (`i`, or `lo` and `hi`) and leave the data where it is.

The exercises `reverse_string` and `is_palindrome` do allow slicing, because there the recursion is the lesson; `test_power_is_logarithmic` is the one test that punishes a careless design. Knowing that slicing costs O(n) is still part of the answer when the exam asks for the complexity.

Over nested data: the data is recursive

```
def depth(nested):
    """How deeply lists are nested. depth([1, [2, [3]]]) == 3."""
    if not isinstance(nested, list):
        return 0                             # a plain value: no list at all
    deepest = 0
    for item in nested:
        d = depth(item)
        if d > deepest:
            deepest = d
    return 1 + deepest
```

A list of lists of lists… is defined in terms of itself — so the function is too.

Some data is recursive by nature: a list whose items may be lists; a folder whose entries may be folders; an expression whose operands may be expressions (Week 15); a tree whose children are

trees (Week 11). For such data, recursion is not a trick, it is the obvious shape — one call per nested piece.

Note that this function has a **loop and** recursion: it loops over the items at one level and recurses into each. `flatten` in the exercises has exactly this shape. The exercise's test `test_flatten_treats_a_string_as_a_value` exists because a string is iterable too: "ab" contains "a", which contains "a", which contains "a"... Check the type you mean to descend into — `list` — not "anything iterable".

What Recursion Costs

Recurrences

Write the cost **T(n)** in terms of the cost of the smaller calls:

Code shape	Recurrence	Solution
one call on n – 1, O(1) work	$T(n) = T(n-1) + c$	$O(n)$
one call on n / 2, O(1) work	$T(n) = T(n/2) + c$	$O(\log n)$
one call on n – 1, **O(n) work** (a slice)	$T(n) = T(n-1) + cn$	$O(n^2)$
two calls on n – 1	$T(n) = 2T(n-1) + c$	$O(2^n)$
two calls on n – 1 and n – 2	$T(n) = T(n-1) + T(n-2) + c$	$O(1.62^n)$
two calls on n / 2, O(n) work	$T(n) = 2T(n/2) + cn$	$O(n \log n)$

A **recurrence** is an equation that defines a function in terms of itself on smaller inputs — recursion, applied to the cost. You read it straight off the code: *how many* recursive calls, *on what size*, and *how much work* outside them.

You should be able to write the recurrence for any function in `dsa/recursion.py`, and know the six solutions in this table. The last row is merge sort, in Week 10 — it is there so you recognise it when it arrives.

Solving one by unrolling

$$\begin{aligned} T(n) &= T(n-1) + c \\ &= T(n-2) + 2c \\ &= T(n-3) + 3c \\ &\;\;\vdots \\ &= T(0) + nc \;=\; O(n) \end{aligned}$$

$$\begin{aligned} T(n) &= T(n/2) + c = T(n/4) + 2c = T(n/8) + 3c = \dots \\ &= T(1) + c \log_2 n \;=\; O(\log n) \end{aligned}$$

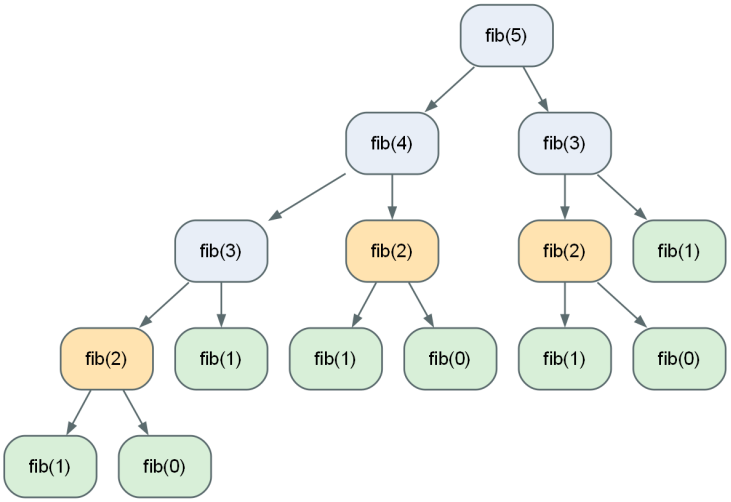
Unrolling (or *substitution*): replace T of the smaller input by its own definition, again and again, until you see the pattern, then jump to the base case.

In the first, each step peels off one c, and it takes n steps to reach T(0).

In the second, the input halves each step. Halving n down to 1 takes $\log_2 n$ steps — the same count as Lab 01's Checkpoint 4 and Lecture 02's rule 4 — and each step adds one c.

The $O(n^2)$ row unrolls the same way: $T(n) = cn + c(n-1) + \ldots + c \cdot 1 = c \cdot n(n+1)/2$.

Solving one with a tree



fib(5): 15 calls. fib(2) alone is computed 3 times (highlighted). Leaves are base cases.

`fib(n) = fib(n-1) + fib(n-2)` — **two** calls, on inputs almost as big.

```
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

When a function makes **more than one** recursive call, draw the **recursion tree**: one node per call, children for the calls it makes. The total cost is the sum of the work in every node; with O(1) work per node, it is the **number of nodes**.

`fib(5)` makes 15 calls. And look at the highlighted nodes: `fib(2)` is computed **three separate times**, `fib(3)` twice. The tree roughly doubles in size each time n grows by one — more precisely, it grows by a factor of about 1.618, the golden ratio — because each node spawns two subtrees that are *almost* as big as itself. `fib(n)` makes $2 \cdot \text{fib}(n+1) - 1$ calls: exponential.

The code is correct. It is also **useless** for n above about 35. That is the lesson: a correct recursion can be exponentially wasteful, and the recursion tree is how you see it.

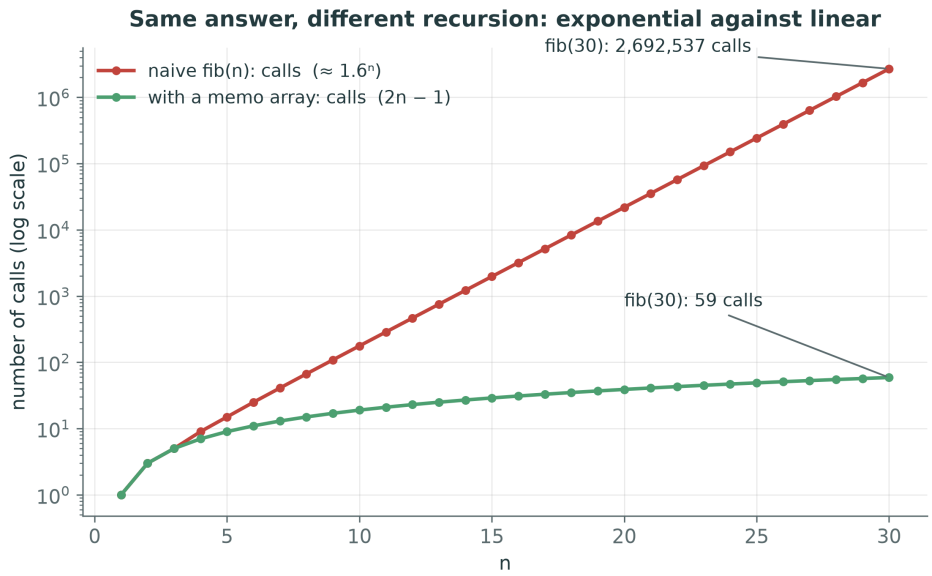
Same answer, one array: memoisation

```
from dsa.array import Array
```

```
def fib_memo(n, memo=None):
    if memo is None:
        memo = Array(n + 1)                # memo[k] will hold fib(k)
    if n < 2:
        return n
    if memo[n] is None:                    # not computed yet
        memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]
```

Compute each `fib(k)` **once**, store it, look it up after that.

Exponential against linear



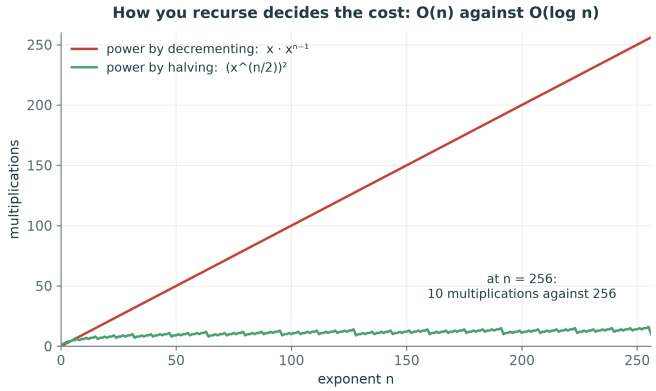
The waste in naive `fib` is recomputing the same subproblem. The cure is to **remember** each answer the first time you compute it — *memoisation* — and look it up after that. Each `fib(k)` is now computed once; the tree collapses to a path, and the calls drop from 2,692,537 to 59 for n = 30. Exponential became linear, at the price of O(n) extra space for the memo.

The memo is an `Array`, not a `dict` — the course rule from Lecture 02 — and an `Array` is the right choice anyway: the keys are the integers 0..n, which is exactly what array indices are.

This one idea — *overlapping subproblems, solved once and stored* — is the heart of **dynamic programming**, which is enrichment in this course (`dsa/dynamic_programming.py`). If you are in AI, you meet it properly in AI3001. If you are in SWE or Bio, that file is your only structured route into it.

How you recurse decides the cost

$$x^n = x \cdot x^{n-1} \qquad \text{versus} \qquad x^n = \left(x^{n/2}\right)^2 \quad (\times\, x \text{ if n is odd})$$



$T(n) = T(n-1) + c = O(n)$ against $T(n) = T(n/2) + c = O(\log n)$.

Both definitions of x^n are correct. The first recurses on n – 1: n calls, n multiplications, n frames — and `power(2, 4096)` hits Python's recursion limit. The second recurses on n // 2, **once** — compute the half power one time and square it: about $\log_2 n$ calls, a couple of multiplications each. At n = 256, ten multiplications instead of 256; at $n = 10^9$, about 60 instead of a billion.

The trap in the fast version: writing `power(x, n // 2) * power(x, n // 2)`. That makes **two** calls on n/2 — T(n) = 2T(n/2) + c — which is O(n) again, and you have gained nothing. Call once, keep the result in a variable, square the variable.

`power` is exercise 4 in `dsa/recursion.py`. The docstring gives the identity; the code is yours.

Branching Recursion

Towers of Hanoi: the puzzle

Three pegs, n disks of different sizes, all stacked on the first peg, largest at the bottom. **Goal:** move the whole tower to another peg.

- Move **one disk** at a time: the top disk of a peg
- **Never** put a larger disk on a smaller one
- The third peg is your **spare**

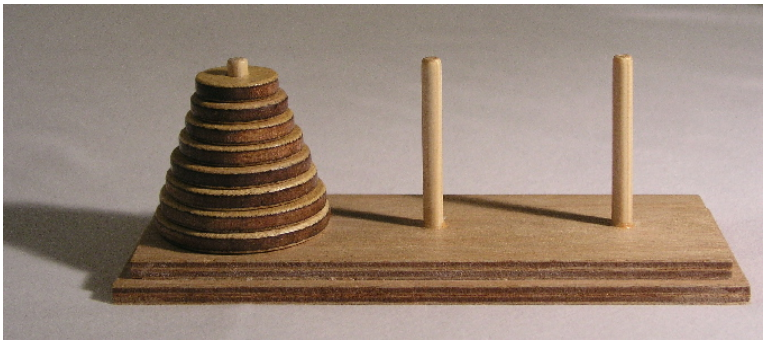


Figure 15: A wooden Tower of Hanoi with 8 disks (photo: Evanherk, Wikimedia Commons, CC BY-SA 3.0)

Watch the solution. The animation solves the puzzle for 4 disks in 15 moves (open the animation; André Karwath, Wikimedia Commons, CC BY-SA 2.5). Watch the largest disk: it moves exactly

once, in the middle (move 8). Before that move, the 3 smaller disks have all gone to the spare peg; after it, they come back on top of it. That is the whole algorithm on the next slide.

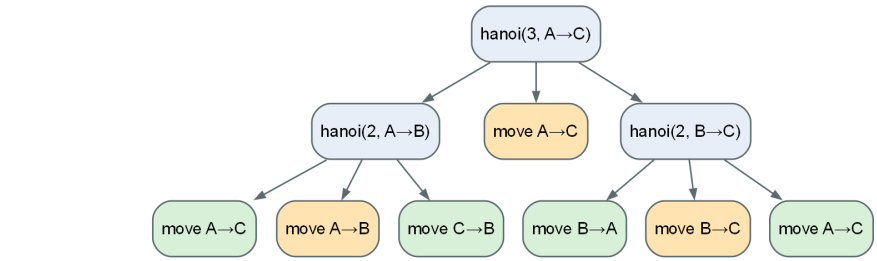
The puzzle was published by the French mathematician Édouard Lucas in 1883. Try n = 3 by hand before you read on: can you do it in 7 moves?

Towers of Hanoi

Move n disks from peg A to peg C, one at a time, never a larger disk on a smaller one.

1. Move the top **n − 1** disks from A to **B** (out of the way) — *recursively*
2. Move the **largest** disk from A to C — *one move*
3. Move the **n − 1** disks from B to **C**, on top of it — *recursively*

$$T(n) = 2T(n-1) + 1 \quad \Rightarrow \quad T(n) = 2^n - 1 \text{ moves}$$



hanoi(3): move 2 out of the way, move the largest, move 2 back on top — 7 moves = $2^3 - 1$. Read the moves left to right along the bottom.

Hanoi is the classic example of a problem that is hard to solve directly and almost trivial to solve recursively — *if* you trust the recursion. You never think about moving 7 small disks; you think "move the pile of 7 somewhere else, then move the big one". The recursive call handles the pile.

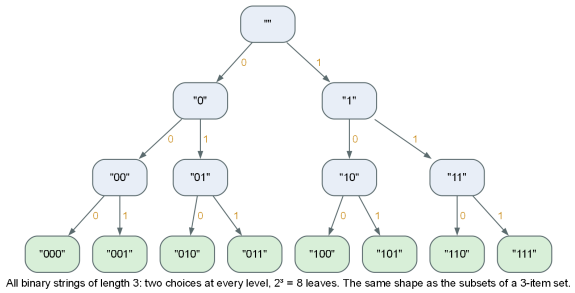
The move count. Unroll it: T(1) = 1, T(2) = 3, T(3) = 7, T(4) = 15 — each is one less than a power of two, $T(n) = 2^n - 1$, which you can prove by induction: $2(2^{n-1} - 1) + 1 = 2^n - 1$. The legend says monks move 64 golden disks one per second: $2^{64} - 1$ seconds is about 585 billion years.

And $2^n - 1$ is not the fault of the algorithm. It is the **minimum** possible — the largest disk must move at least once, and before it can, all n − 1 others must be on the spare peg. Some problems are exponential by nature, and that is a lower bound on the problem, not a defect of your code (Lecture 02's Ω).

In the tree, the moves appear in order along the bottom: A→C, A→B, C→B, A→C, B→A, B→C, A→C. The exercise `hanoi` returns exactly that list for n = 3. The four lines of code are yours to write.

Every choice: two branches per decision

```
def binary_strings(n):
    """Every string of n bits, e.g. n=2 -> ['00', '01', '10', '11']."""
    if n == 0:
        return [""]                          # one way to choose nothing
    shorter = binary_strings(n - 1)
    return ["0" + s for s in shorter] + ["1" + s for s in shorter]
```



All binary strings of length 3: two choices at every level, $2^3 = 8$ leaves. The same shape as the subsets of a 3-item set.

n decisions, 2 choices each: 2^n results, and at least $O(2^n)$ time just to write them down.

Many problems ask for **every** combination: every subset, every ordering, every path. The recursive pattern is always the same: make **one** decision, recurse on the rest, combine.

For binary strings the decision is "is the first bit 0 or 1?". For **subsets** (exercise `subsets`) it is "is the first item **out** or **in**?" — exactly the same tree with "out/in" instead of "0/1", so there are 2^n subsets. For **permutations** (exercise `permutations`) it is "which item goes **first**?" — n choices, then n − 1, then n − 2: n! results.

Look at the base case: there is exactly **one** binary string of length 0 — the empty string — so the answer is `[""]`, not `[]`. Likewise `subsets([])` is `[[]]` (one subset: the empty one) and `permutations([])` is `[[]]`. Getting that base case wrong is the most common bug in these two exercises: return `[]` and every result built from it is empty too.

These outputs are exponential or factorial in size, so no algorithm can be faster than the output is long. When a problem needs all 2^n or n! answers, the cost is in the problem, not in the recursion. When it only needs *one good* answer, exploring all of them is how programs become unusably slow — which is what much of algorithm design is about.

Merging: recursion on two inputs

To merge two sorted lists:

- If either is empty, the answer is the other one.
- Otherwise, the **smaller head** comes first, followed by the merge of everything else.

Measure: `len(left) + len(right)`, one less each call — $O(n + m)$ calls.

`merge_sorted` in `dsa/recursion.py` is this idea. You wrote the iterative version last week, on `Arrays`, in `dsa/array_ops.py`; now write it recursively. Two details the tests check:

- **Stability.** When the two heads are equal, take from `left` first. The test uses pairs like `(1, "L")` and `(1, "R")` to see which went first. Stability matters in Week 10, where merge sort's stability is one of its selling points.
- **Recursion depth.** The depth is n + m, so with Python's limit this version handles only short lists. That is fine for the exercise — and a good reason why merge sort's real merge step, in Week 10, is a loop.

Recursion or a Loop?

Anything recursive can be a loop

- **Tail-shaped** recursion (one call, nothing done after it) is a loop in disguise:

```
def gcd_loop(a, b):
    while b != 0:
        a, b = b, a % b
    return a
```

- **Branching** recursion becomes a loop **plus your own stack** — which is what the call stack was doing for you. (Week 14: `dfs` and `dfs_iterative`.)

Every recursive algorithm can be rewritten without recursion, because the computer runs recursion using a stack, and you can keep that stack yourself.

For recursion with a single call as the very last action — gcd, or the recursive `find` — the rewrite is a simple `while` loop, and in Python it is usually the better choice: no frame per step, no recursion limit.

For recursion that branches — Hanoi, trees, graph search — the rewrite needs an explicit stack, and the code gets longer and harder to read. Week 14 has you write depth-first search both ways precisely so that you see what the call stack was doing.

When to choose which

Recursion, when…	A loop, when…
the data is recursive: trees, nested lists, grammars	the data is flat: arrays, ranges
the problem splits in halves: binary search, merge sort	the recursion would be one call deep per element
it branches: all subsets, Hanoi, backtracking	depth could exceed ~1,000
clarity matters more than the last 20% of speed	you are in a hot loop in Python

Recursion is a way of thinking first, and a way of writing code second.

Most algorithms in the second half of this course are recursive *in idea* — split the problem, solve the parts, combine — whether or not the final code calls itself. That idea, **divide and conquer**, is binary search (Week 8), merge sort and quicksort (Week 10), and every tree operation (Week 11). Learning to think recursively this week is what makes those weeks easy.

This Week

Exercises: `dsa/recursion.py`

Function	Idea	Target
`factorial(n)`	n · (n − 1)!; reject n < 0	$O(n)$
`sum_digits(n)`	last digit + the rest	$O(\log n)$
`gcd(a, b)`	Euclid: gcd(b, a mod b)	$O(\log \min(a, b))$
`power(x, n)`	halve the exponent — **once**	$O(\log n)$
`reverse_string(s)`	last + reverse(rest)	—
`is_palindrome(s)`	ends match, recurse on the middle	—
`flatten(nested)`	recurse into lists only	—
`merge_sorted(l, r)`	smaller head first; stable	$O(n + m)$
`hanoi(n, …)`	n − 1 away, largest, n − 1 back	$O(2^n)$
`subsets(items)`	out, or in	$O(2^n)$ results
`permutations(items)`	which goes first?	$O(n!)$ results

```
pytest tests/test_recursion.py -v
```

Do them in order. The first four are one base case and one step; the next four recurse over sequences; the last three branch.

The functions return Python lists — lists are the *interface* (Lecture 02), and none of these functions is a data structure. What the rule still asks: no `itertools`, no built-in that does the exercise for you, and think about what each slice costs.

Before you write each one, write down: the base case, the measure that shrinks, and the recurrence. If you cannot write the recurrence, you do not yet know what your function does.

Homework 3 — before Lecture 04

1. **Implement** `dsa/recursion.py` until `pytest tests/test_recursion.py` passes.
2. **Trace** `hanoi(3, "A", "C", "B")` by hand: draw the call tree and list the moves in order. Check against your code.
3. **Recurrences.** For each of `sum_digits`, `power`, `hanoi` and `permutations`, write the recurrence and solve it.
4. **Measure** `fib` (naive) and `fib_memo` from the handout with `viz.complexity.measure` for n = 5, 10, 15, 20, 25, and plot both. At what n does the naive one take longer than a second on your machine?
5. **Lab 03** exercises, if not finished in the lab.

Items 2–4 go on one page, handed in the way the TAs announce. For item 4, `fib` grows so fast that you should start small — n = 35 may already take several seconds.

Summary

Seven things to keep

1. Recursion = a **base case** + a recursive case that **makes progress**.
2. **Trust the recursion**: assume the smaller call works; build on it. That is induction.

3. Each call gets a **frame** on the **call stack**; stack space = the **deepest** point. Python stops at ~1,000.
4. The cost is a **recurrence**: how many calls, on what size, how much work outside them.
5. $T(n-1)+c \to O(n) \cdot T(n/2)+c \to O(\log n) \cdot 2T(n-1)+c \to O(2^n)$.
6. **How** you recurse decides the cost: fast power, memoised Fibonacci. Slicing hides $O(n)$.
7. **Branching** recursion explores every choice — 2^n subsets, $n!$ orderings.

Next

Week 4 — Dynamic arrays. An array that grows: a fixed `Array`, replaced by a bigger one when it fills — and why doubling makes append **amortised** $O(1)$.

Sources and further reading

- **H. Abelson and G. J. Sussman.** *Structure and Interpretation of Computer Programs*, 2nd ed., MIT Press, 1996, section 1.2, "Procedures and the Processes They Generate" — linear recursion, iteration, and tree recursion, with Fibonacci as the example of wasted work.
- **T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein.** *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, chapter 4, "Divide-and-Conquer" — the substitution and recursion-tree methods for solving recurrences.
- **M. T. Goodrich, R. Tamassia and M. H. Goldwasser.** *Data Structures and Algorithms in Python*, Wiley, 2013, chapter 4, "Recursion" — including Python's recursion limit and how to eliminate tail recursion.
- **D. E. Knuth.** *The Art of Computer Programming*, Vol. 2, 3rd ed., Addison-Wesley, 1997, section 4.5.2 — Euclid's algorithm, its history, and why it takes O(log n) steps.
- **E. Lucas.** The Tower of Hanoi puzzle and its legend were published by the French mathematician Édouard Lucas in 1883.

Every figure in this lecture is generated by `tools/figures_l03.py`. The two charts are counted, not timed, so they are identical on every machine.

Questions for Week 3

> ***The answers are at the back of the book,*** *in* Answers — Week 3. *Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W3-M01 – W3-M26
B	Short answer and essay	W3-E1 – W3-E6
C	Recurrences — write and solve	W3-R1 – W3-R5
D	Trace the code and the call stack	W3-T1 – W3-T8
E	Find and fix the bug	W3-B1 – W3-B4
F	Write the code — checked by `pytest`	W3-C1 – W3-C5

Part A — Multiple choice

W3-M01 [what] Every correct recursive function needs:

- **a)** a base case, and a recursive case that moves closer to the base case
- **b)** a loop and a return statement
- **c)** at least two recursive calls
- **d)** a global variable to count the calls

W3-M02 [how] With `def factorial(n): return 1 if n == 0 else n * factorial(n - 1)`, what does `factorial(-1)` do?

- **a)** returns 1
- **b)** returns –1
- **c)** raises `ValueError`
- **d)** raises `RecursionError`

W3-M03 [what] By default, CPython stops a recursion at a depth of about:

- **a)** 100
- **b)** 1,000
- **c)** 100,000
- **d)** there is no limit

W3-M04 [how] How much stack space does `factorial(n)` use?

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n!)$

W3-M05 [how] $T(n) = T(n/2) + c$ solves to:

- **a)** $O(\log n)$
- **b)** $O(n)$
- **c)** $O(n \log n)$
- **d)** $O(1)$

W3-M06 [how] $T(n) = 2T(n-1) + c$ solves to:

- **a)** $O(n)$
- **b)** $O(n^2)$
- **c)** $O(n \log n)$
- **d)** $O(2^n)$

W3-M07 [how] $T(n) = T(n-1) + cn$ solves to:

- **a)** $O(n)$
- **b)** $O(n^2)$
- **c)** $O(2^n)$
- **d)** $O(n \log n)$

W3-M08 [how] How many moves does Towers of Hanoi need for 5 disks?

- **a)** 25
- **b)** 32
- **c)** 31
- **d)** 10

W3-M09 [how] How many subsets does a set of 4 items have (including the empty set and the whole set)?

- **a)** 8
- **b)** 16
- **c)** 24
- **d)** 15

W3-M10 [how] How many orderings (permutations) do 4 distinct items have?

- **a)** 24
- **b)** 16
- **c)** 12
- **d)** 4

W3-M11 [how] How many calls in total does the naive `fib(5)` make (counting the first)?

- **a)** 5
- **b)** 8
- **c)** 9
- **d)** 15

W3-M12 [why] Fast power written as `half = power(x, n // 2) * power(x, n // 2)` (then × x if n is odd). Its cost is:

- **a)** $O(\log n)$ — it halves n
- **b)** $O(1)$
- **c)** $O(n)$ — two calls on n/2 undo the saving
- **d)** $O(n^2)$

W3-M13 [why] `list_max(values)` recurses on `values[1:]`. Its running time is:

- **a)** $O(n)$
- **b)** $O(n^2)$ — every slice copies the rest
- **c)** $O(\log n)$
- **d)** $O(n \log n)$

W3-M14 [what] Memoisation means:

- **a)** storing the answer to each subproblem the first time and looking it up after
- **b)** replacing recursion with a loop
- **c)** raising the recursion limit
- **d)** printing each call to debug it

W3-M15 [why] `hanoi(20)` makes over a million calls. At most how many frames are on the call stack at the same time?

- **a)** about 1,000,000
- **b)** 2^{20}
- **c)** 1,000 — the recursion limit
- **d)** about 20

W3-M16 [how] What does `f(3)` print?

```
def f(n):
    if n == 0:
        return
    print(n, end=" ")
    f(n - 1)
    print(n, end=" ")
```

- **a)** `3 2 1`
- **b)** `1 2 3 3 2 1`
- **c)** `3 2 1 1 2 3`
- **d)** `3 3 2 2 1 1`

W3-M17 [why] What should `subsets([])` return?

- **a)** `[[]]`
- **b)** `[]`
- **c)** None
- **d)** `[None]`

W3-M18 [what] Tail-call optimisation in Python:

- **a)** is done automatically for every tail call
- **b)** is not done: every recursive call costs a stack frame
- **c)** is done only when `sys.setrecursionlimit` is called
- **d)** turns recursion into a loop at compile time

W3-M19 [how] `marks = [3, 1, 2]; result = marks.sort()`. What is `result`?

- **a)** `[1, 2, 3]`
- **b)** `[3, 1, 2]`
- **c)** an error
- **d)** None

W3-M20 [what] What is `type({})`?

- **a)** `<class 'set'>`
- **b)** `<class 'list'>`
- **c)** `<class 'dict'>`
- **d)** `<class 'tuple'>`

W3-M21 [what] The average cost of `x in s` for a Python `set` of n items is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$

- **d)** $O(n^2)$

W3-M22 [why] What is wrong with this class?

```
class Dog:
    tricks = []
    def __init__(self, name):
        self.name = name
    def add_trick(self, trick):
        self.tricks.append(trick)
```

- **a)** `__init__` must return `self`
- **b)** `add_trick` needs a `return`
- **c)** `name` should be a class variable
- **d)** every `Dog` shares one `tricks` list

W3-M23 [what] Calling a function that contains `yield`:

- **a)** runs the body and returns the first yielded value
- **b)** returns a generator object without running the body yet
- **c)** runs the whole body and returns a list
- **d)** raises `StopIteration`

W3-M24 [why] Why should a queue not be a Python list used with `pop(0)`?

- **a)** `pop(0)` raises an error on lists
- **b)** lists cannot hold more than 1,000 items
- **c)** `pop(0)` shifts every remaining item, $O(n)$ per dequeue
- **d)** `pop(0)` removes the last item, not the first

W3-M25 [how] What is `counts` after this?

```
counts = {}
for w in "a b a".split():
    counts[w] = counts.get(w, 0) + 1
```

- **a)** `{'a': 1, 'b': 1}`
- **b)** `{'a': 2, 'b': 1}`
- **c)** a `KeyError`
- **d)** `{'a': 2}`

W3-M26 [how] The recurrence of naive Fibonacci, $T(n) = T(n-1) + T(n-2) + c$, grows like:

- **a)** $O(n)$
- **b)** $O(n^2)$
- **c)** $O(n \log n)$
- **d)** $O(\varphi^n)$ with $\varphi \approx 1.618$

Part B — Short answer and essay

W3-E1 [how] *(4 marks)* Draw the call stack at every step of `factorial(3)`, from the first call until the final value is returned. Label winding, the base case, and unwinding, and show the value each frame returns.

W3-E2 [why] *(4 marks)* Explain why the naive recursive Fibonacci takes exponential time, using a recursion tree. Show how memoisation fixes it, and state the new time and space costs.

W3-E3 [why] *(3 marks)* When should you prefer recursion, and when a loop? Include two concerns that are specific to Python.

W3-E4 [why] *(4 marks)* Describe the recursive strategy for Towers of Hanoi. Write its recurrence, solve it, and explain why no algorithm can use fewer moves.

W3-E5 [why] *(3 marks)* Explain "trust the recursion" and how it relates to mathematical induction.

W3-E6 [why] *(4 marks)* For each job, choose `list`, `tuple`, `set` or `dict` and justify by cost: (a) the marks of a class, in the order entered; (b) checking whether a student ID has already been registered; (c) looking up a student's name by ID; (d) one student's (ID, name, mark), used as a key elsewhere.

Part C — Recurrences

For each function write the recurrence for its running time $T(n)$ (and the base case), then solve it.

W3-R1 [how] n is the number of digits of the input.

```
def sum_digits(x):
    if x < 10:
        return x
    return x % 10 + sum_digits(x // 10)
```

W3-R2 [why]

```
def mystery(n):
    if n <= 1:
        return 1
    return mystery(n - 1) + mystery(n - 1)
```

What does `mystery(n)` return? Rewrite it to make it $O(n)$ — and then $O(1)$.

W3-R3 [why] `n = hi - lo`.

```
def range_sum(arr, lo, hi):          # sum of arr[lo .. hi-1]
    if hi - lo == 0:
        return 0
    if hi - lo == 1:
        return arr[lo]
```

```
    mid = (lo + hi) // 2
    return range_sum(arr, lo, mid) + range_sum(arr, mid, hi)
```

What is its stack depth?

W3-R4 [how] `n = len(text)`.

```
def reverse(text):
    if text == "":
        return ""
    return reverse(text[1:]) + text[0]
```

W3-R5 [why] Prove by induction that the Hanoi recurrence $T(1) = 1, T(n) = 2T(n-1) + 1$ has the solution $T(n) = 2^n - 1$.

Part D — Trace the code and the call stack

W3-T1 [how] What is printed?

```
def g(n):
    if n < 10:
        return n
    return g(n // 10) + n % 10

print(g(4096), g(7))
```

W3-T2 [how] What is printed?

```
def h(n):
    if n < 2:
        return str(n)
    return h(n // 2) + str(n % 2)

print(h(13), h(1), h(8))
```

W3-T3 [how] List every call made by `gcd(48, 18)`, in order, with the value each returns. What is the maximum stack depth?

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)
```

W3-T4 [how] What is printed?

```
calls = 0
```

```
def fib(n):
    global calls
    calls += 1
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(6), calls)
```

W3-T5 [how] `hanoi(n, source, target, spare)` moves n − 1 disks from source to spare, moves one disk from source to target, then moves n − 1 disks from spare to target. List the moves of `hanoi(2, "A", "C", "B")`, and give the **4th** move of `hanoi(3, "A", "C", "B")`.

W3-T6 [how] What is printed?

```
def count_down_up(n):
    if n == 0:
        return [0]
    return [n] + count_down_up(n - 1) + [n]

print(count_down_up(2), len(count_down_up(5)))
```

W3-T7 [how] What is printed?

```
words = "the cat the hat".split()
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1
print(counts)
print(sorted(counts, key=lambda w: -counts[w])[0], {len(w) for w in words})
```

W3-T8 [how] What is printed?

```
class Countdown:
    def __init__(self, n):
        self.n = n

    def __iter__(self):
        k = self.n
        while k > 0:
            yield k
            k -= 1

c = Countdown(3)
print(list(c), sum(c), len(list(c)))
```

Part E — Find and fix the bug

W3-B1 [why] Correct results, but far too slow. Why, and how do you fix it?

```
def power(x, n):
    if n == 0:
        return 1
    if n % 2 == 0:
        return power(x, n // 2) * power(x, n // 2)
    return x * power(x, n - 1)
```

W3-B2 [how] Should return every subset of `items`.

```
def subsets(items):
    if not items:
        return []
    rest = subsets(items[1:])
    return rest + [[items[0]] + s for s in rest]
```

W3-B3 [how] Should flatten nested lists: `flatten([1, ["ab", [2]]])` is `[1, "ab", 2]`.

```
def flatten(nested):
    result = []
    for item in nested:
        if hasattr(item, "__iter__"):
            result += flatten(item)
        else:
            result.append(item)
    return result
```

W3-B4 [how] Should return the sum of a list.

```
def total(values, i=0):
    if i == len(values):
        return 0
    values[i] + total(values, i + 1)
```

Part F — Write the code

In `practice/week03.py`; check with `pytest tests/test_practice_week03.py -v`. Every function must be **recursive**.

W3-C1 [how] `array_sum(arr, i=0)` — sum of a course `Array` from index i, without slicing.

W3-C2 [how] `count_char(text, ch)` — occurrences of one character.

W3-C3 [how] `is_sorted_rec(arr, i=0)` — is the `Array` non-decreasing from i?

W3-C4 [why] `no_consecutive_ones(n)` — every binary string of length n with no `"11"`. How many are there for n = 1, 2, 3, 4, 5, and why?

W3-C5 [how] `pascal_row(n)` — row n of Pascal's triangle, built from row n − 1.

Week 4 — Dynamic Arrays

> ***How to read this document.*** *This is the handout for Lecture 04. It holds everything on the slides, plus what I said out loud. This week you build the first real data structure of the course — the one Python's `list` already is — and meet the idea that makes it work:* ***amortised*** *cost.*
>
> *Slides: `DSA27-L04-slides.pdf` · Code: `dsa/dynamic_array.py` · Tests: `tests/test_dynamic_array.py` · Interactive demo: Array Lab*

Where We Are

Two weeks ago, one problem

The course `Array` gives $O(1)$ indexing — and a **fixed size**.

```
a = Array(4)
a[4] = "e"          # IndexError: there is no slot 4, and there never will be
```

Python's `list` grows for ever, and append is fast. **How?**

Lecture 02 introduced the array and its one great limitation: the size is decided when it is created. The memory right after the block may belong to something else, so it cannot grow in place.

Yet you have used Python lists since your first week, appending millions of items, and append never seemed slow. Today we build the structure that explains it — the **dynamic array** — on nothing but the course `Array`, and we prove why append is fast *on average* even though it is occasionally very slow.

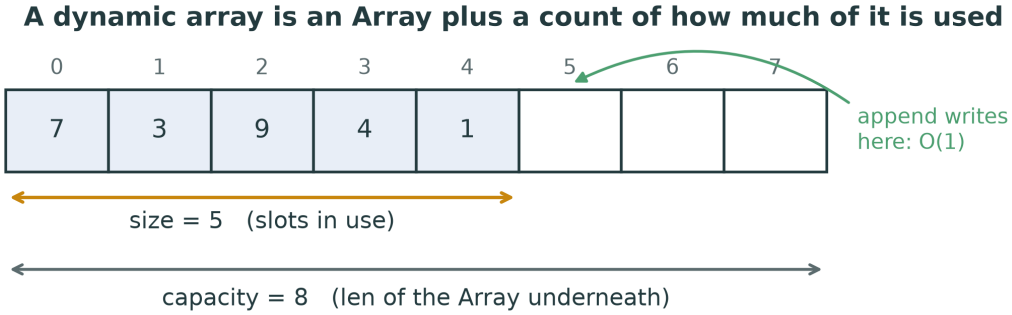
Lecture 03's recursion is not needed today; complexity from Lecture 02 is needed throughout.

Today

1. Capacity and size: an Array plus a count
2. Growing: allocate, copy, switch
3. **How much** to grow — by 1, by 100, or × 2?
4. **Amortised** analysis: why append is $O(1)$ on average
5. Shrinking, and how not to thrash
6. Python's own `list`, measured
7. The rest of the operations, and their costs

Capacity and Size

An Array plus a count



- **capacity** — how many slots the Array underneath has (`len(self._block)`)
- **size** — how many of them hold data (`self._size`, and what `len()` returns)
- Always $0 \leq$ size $\leq$ capacity

This is the split you already met in Lecture 02's `insert_at(arr, size, …)`: the array has a length, and separately you track how much of it is in use. A dynamic array simply keeps both inside one object.

```
class DynamicArray:
    def __init__(self, values=(), growth=2):
        self._size = 0                    # slots in use
        self._capacity = 1                # slots reserved
        self._block = make_block(1)       # an Array(1)
        ...
```

That is the skeleton you are given in `dsa/dynamic_array.py`. `make_block` is just `Array(capacity)`.

While there is room, appending is one write into the next free slot and one increment: $O(1)$. The whole question of this lecture is what happens when there is no room.

Dynamic array in Arabic: المصفوفة الديناميكية — the array whose size changes.

Reading and writing are still O(1)

```
def __getitem__(self, index):
    if index < 0:
        index += self._size               # -1 means the last USED slot
    if not 0 <= index < self._size:
        raise IndexError(index)
    return self._block[index]
```

Negative indices are **translated**: the course `Array` has none, so the dynamic array provides them — in $O(1)$.

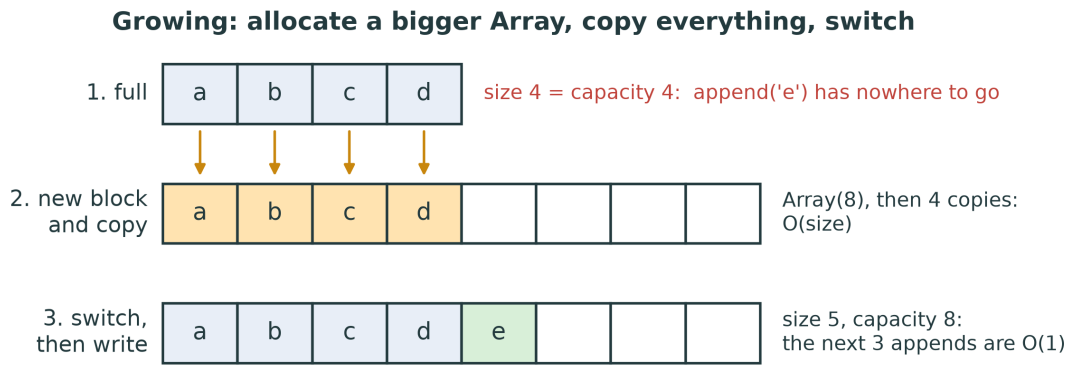
Two details that the tests check:

- The bounds are **0 to size − 1**, not 0 to capacity − 1. Slots beyond `size` exist in the Array but hold nothing the user put there; reading one must be an `IndexError`, exactly as with a Python list.
- `arr[-1]` is the last *used* element, so the translation adds `size`, not `capacity`.

That is essentially the whole of `__getitem__` — it is here because the translation is the idea worth seeing. `__setitem__` follows the same shape and is yours to write, along with everything that follows.

Growing

Allocate, copy, switch



A full array cannot grow in place. So:

1. allocate a **new, bigger** `Array`
2. **copy** every element across — $O(\text{size})$
3. **switch** `self._block` to the new Array (the old one is garbage-collected)

That is `_resize(capacity)` in the exercise, and it is the one genuinely expensive thing a dynamic array does:

```
def _resize(self, capacity):
    block = make_block(capacity)          # 1. allocate
    for i in range(self._size):           # 2. copy — O(size)
        block[i] = self._block[i]
    self._block = block                   # 3. switch
    self._capacity = capacity
    self.resize_count += 1                # the tests and the notebook read
    ↪ this
```

(This one is shown in full because it is mechanical; deciding *when* to call it and *with what capacity* is the exercise.)

Step 3 is where Python's garbage collection, from Lecture 01, quietly does work: nothing refers to the old Array any more, so its memory is reclaimed. In C you would call `free` on it yourself — and forgetting would leak memory.

append now reads: *if full, resize; then write at* `size` *and increment.* A single append that triggers a resize costs $O(n)$. So is append $O(n)$? That depends entirely on **how much bigger** the new Array is.

How Much to Grow?

Option 1: one slot at a time

Capacity 1, 2, 3, 4, … — **every** append is a resize.

Copies for n appends:

$$0 + 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2} = \Theta(n^2)$$

Every append is $O(n)$. 100,000 appends ≈ 5 **billion** copies.

The most economical in memory — never a wasted slot — and hopeless in time. The k-th append copies the k – 1 elements already there. It is the triangular sum from Lecture 02 again.

Option 2: a constant step — say, 100 at a time

Resize only every 100 appends — surely much better?

Copies for n appends: $100 + 200 + 300 + \cdots \approx \frac{n^2}{2\times 100}$

Still $\Theta(n^2)$ — just 100 times smaller. 100,000 appends ≈ 50 million copies.

This is the trap that looks like a fix. Growing by a constant c divides the number of resizes by c, but each resize still copies everything, and the amounts copied still grow linearly: c, 2c, 3c, … A sum of n/c terms growing linearly is quadratic. The constant c only changes the constant in front of the n^2 — the part Big-O throws away.

Counted exactly for n = 100,000 appends starting from capacity 1: growing by 1 copies 4,999,950,000 elements; growing by 10 copies 499,960,000. The second is ten times smaller, and still hopeless.

Option 3: double

Capacity 1, 2, 4, 8, 16, … — resize only when size reaches a power of two.

Copies for n appends:

$$1 + 2 + 4 + \cdots + 2^k \;<\; 2n \qquad (2^k < n)$$

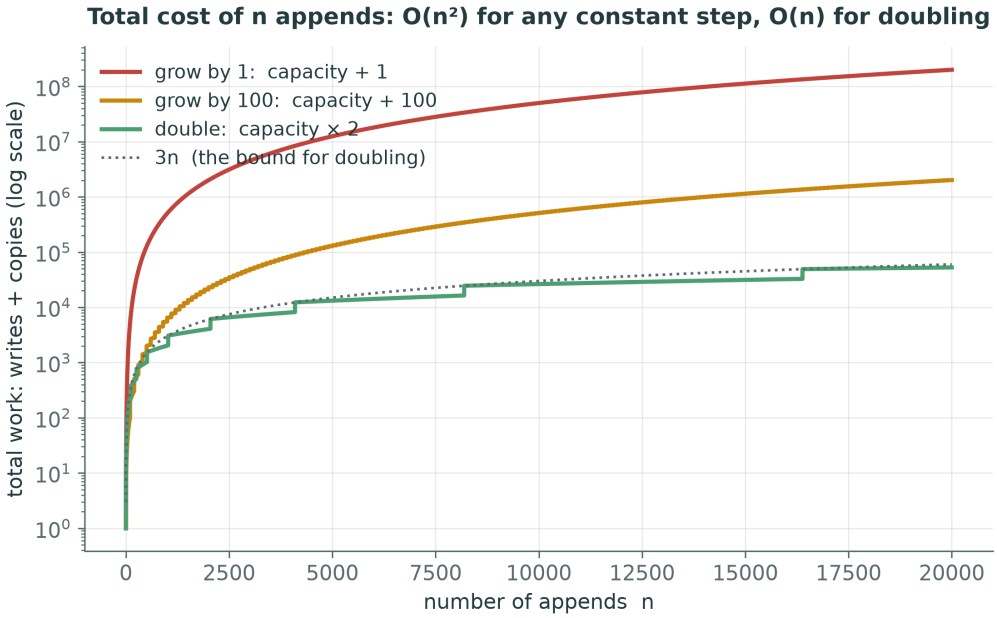
$\Theta(n)$ in total. 100,000 appends: 131,071 copies.

Resizes happen at sizes 1, 2, 4, 8, …, and each copies that many elements. The copies form a **geometric** series, and a geometric series is dominated by its last term: $1 + 2 + 4 + \cdots + 2^k =$

$2^{k+1} - 1 < 2 \cdot 2^k$. The last resize copied fewer than n elements, so all the copying together is fewer than 2n.

Counted exactly for n = 100,000: 131,071 copies — against 5 billion for growing by one. That is the whole of this lecture in two numbers.

The three side by side

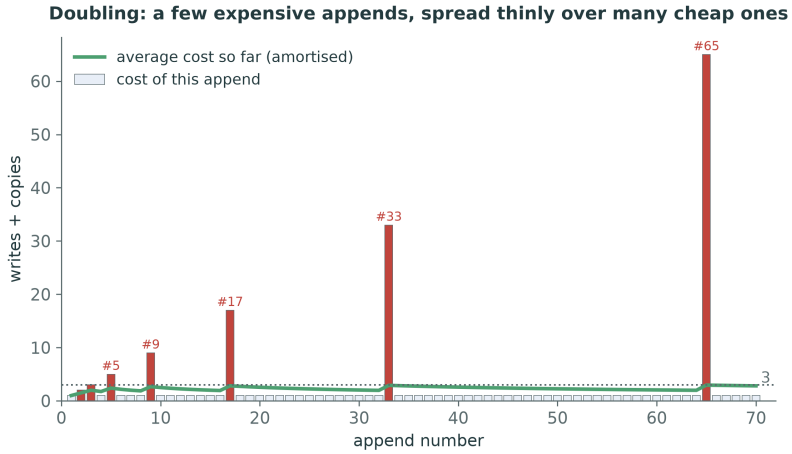


Counted, not timed: the total number of writes and copies for n appends, for each growth rule, on a logarithmic axis. Growing by 1 and growing by 100 are the same shape — quadratic — shifted apart by a constant. Doubling is a different shape: it stays under the dotted line 3n, for every n.

This is the plot your `resize_count` lets you draw in the notebook `notebooks/04-dynamic-array.ipynb`.

Amortised Analysis

One expensive append, many cheap ones



The red bars are resizes: expensive, but **exponentially rare**. The green line — the average so far — never rises above **3**.

Each bar is the cost of one append with doubling, counting one unit per write and one per copied element. Most appends cost 1. Append number 2, 3, 5, 9, 17, 33, 65, … — one more than a power of two — pays for copying everything first.

The expensive appends are far apart, and they get further apart exactly as fast as they get more expensive. That balance is the whole trick.

Watch it run: the Array Lab, again

Array Lab — the *Dynamic array* scenarios.

- append from empty: watch the block **double** and every element copied
- A bar per append and the running average — the last slide's chart, built live
- `insert_at` on a full block, `pop()` against `pop(0)`, `da[-1]`
- **Goes wrong:** `_resize` that forgets to copy, or forgets to switch; growing by **one** slot (120 copies for 16 appends)

The same demo as in Lecture 02, now with `DynamicArray`. During a resize it draws the old block and the new one together, animates each copy across, and then lets the old block go — step 3 of *allocate, copy, switch*, with nothing referring to it any more.

- **Online:** https://helghareeb.github.io/DSA27/lectures/02-complexity-and-arrays/array-lab.html
- **Offline:** open `docs/lectures/02-complexity-and-arrays/array-lab.html` from your copy of the repository.

How to use it:

- Pick a scenario at the top. Change the inputs in the `>>>` line and press **Run**.
- **Step** (→) moves one step and **Back** (←) undoes it. **Step over** (O) runs a whole call (an append, a `_resize`, a reversal) in one step; **Step out** (U) finishes the current call. **Play** (P) runs on its own.

- At the key moments the demo stops with a question. Answer it (A–D) **before** you look; that is where the learning happens.

What to try, in order: appends with doubling (when do the resizes happen?), the same with `growth` set to 3, then *growing by one slot* — count the copies against doubling — and the two broken `_resize` scenarios. For each broken one, find the bug in the code **before** you step.

The scenarios show code that solves this week's exercise in `dsa/dynamic_array.py`. Write your own version first.

Amortised cost: the definition

The **amortised cost** of an operation is the **total cost of any sequence** of n operations, **divided by n**.

For append with doubling: total $< 3n$, so the amortised cost is $< 3 = O(1)$.

- One individual append can still cost $O(n)$ — the **worst case** is unchanged.
- But **no sequence** of n appends can cost more than $3n$ — a **guarantee**.

Amortise comes from finance: paying off a large cost in small instalments over time. (In Arabic, الإطفاء or التكلفة المستهلك.) The large cost here is a resize; the instalments are the cheap appends that come before it.

The proof — the aggregate method. Take any n appends starting from an empty array of capacity 1.

- Every append writes one element: **n** writes.
- Resizes happen when size reaches 1, 2, 4, ..., 2^k, with $2^k < n$, and copy that many elements: $1 + 2 + \cdots + 2^k = 2^{k+1} - 1 < 2n$ copies.
- Total: fewer than $n + 2n = 3n$. Divided by n: fewer than 3 per append. ■

The same proof, as a bank account. Charge every append 3 coins. One pays for its own write. The other two are saved *on the element just written*. When the array doubles from capacity c to 2c, the c/2 elements written since the last resize each hold 2 saved coins — c coins in all, exactly enough to copy the c elements. The account never goes negative, so 3 coins per append always suffice.

Amortised is not "average case"

	Average case (Lecture 02)	Amortised
Averages over	possible **inputs**	the operations in **one sequence**
Needs	an assumption about the inputs	nothing
Guarantee	"usually"	**always**, for every sequence

append is **worst-case** $O(n)$, **amortised** $O(1)$. Both statements are true and both are useful.

Average-case analysis says: if the inputs are random in a certain way, the expected cost is this. It can be wrong for the inputs you actually get.

Amortised analysis makes no assumption at all. It says: whatever you do, any n appends in a row cost at most 3n. There is no unlucky input that breaks it.

When would the worst case still matter? When *one* slow operation is a problem in itself — a game that must draw a frame every 16 milliseconds, a controller that must respond in real time. Then an occasional $O(n)$ pause is a bug even though the average is excellent, and real-time systems reserve capacity in advance. For almost everything else, amortised $O(1)$ is as good as $O(1)$.

Any constant factor works

Growth rule	Total copies for n appends	Amortised append	Worst unused space
+ 1	$\approx n^2/2$	$O(n)$	none
+ c (constant)	$\approx n^2/2c$	$O(n)$	c slots
× 2	$< 2n$	$O(1)$	about half
× 1.5	$< 3n$ (roughly)	$O(1)$	about a third

What makes it $O(1)$ is growing by a **factor** — a percentage of the current size — not the particular factor.

For any factor r > 1 the copies form a geometric series with ratio r, whose sum is at most about $n \cdot \frac{r}{r-1}$ — a constant times n. A smaller factor copies a little more often but wastes less memory; a larger one wastes more memory but copies less. That is a time–space trade-off with no single right answer, which is why real implementations differ: many use 2, some 1.5, and CPython uses about 1.125 (next slide).

(In the exercise the factor is `self.growth`, 2 by default. `Array` needs a whole number of slots, so a factor like 1.5 needs rounding up — `int(capacity * 1.5) + 1` — if you experiment with it.)

Shrinking

Give memory back — carefully

Pop until the array is nearly empty, and most of the Array is wasted. Shrink when size falls to **one quarter** of capacity — and then **halve**.

Why a quarter, not a half?

Start: capacity 8, size 8 (full)	halve at ½ full	halve at ¼ full
append → size 9	double to 16: copy 8	double to 16: copy 8
pop → size 8	**halve** to 8: copy 8	nothing (8 > 16/4)
append → size 9	**double** to 16: copy 8	nothing

Start: capacity 8, size 8 (full)	halve at ½ full	halve at ¼ full
pop → size 8 repeat…	**halve** to 8: copy 8 **every operation** $O(n)$	nothing $O(1)$

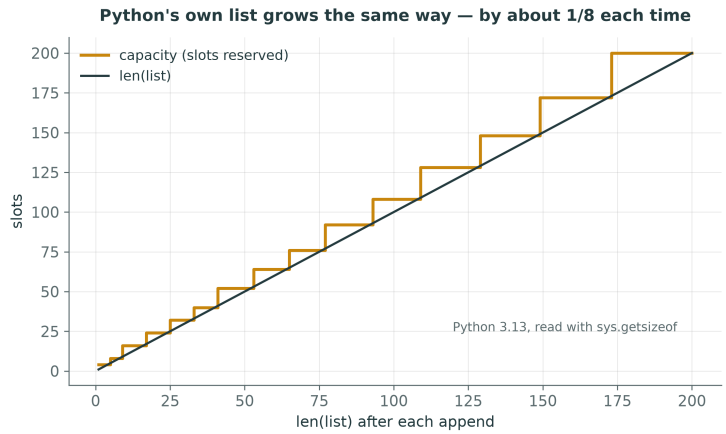
Growing at "full" and shrinking at "half full" leaves no gap between the two thresholds. An append that just crossed the growth boundary is immediately followed by a pop that crosses the shrink boundary, and the array copies itself back and forth on every single operation. That is **thrashing**, and it turns an amortised $O(1)$ structure into an $O(n)$ one on a perfectly ordinary sequence.

Shrinking at a quarter leaves slack on both sides. After a halving, the array is half full, so it needs about n/4 more pops, or n/4 more appends, before it resizes again — and those operations have paid for the copy. Amortised $O(1)$ for any mix of appends and pops.

The `DynamicArray` exercise does **not** require shrinking; the tests never check capacity after pop. It is an extension for Homework 4 — and the question bank has a practice problem on it.

Python's `list`

Measured: CPython does exactly this



Read straight from the interpreter with `sys.getsizeof`: capacity 4, 8, 16, 24, 32, 40, 52, 64, 76, 92, … — grow by about **one eighth**, plus a little.

A Python list's memory is a fixed header plus one 8-byte reference per slot of capacity, so `sys.getsizeof` reveals the capacity:

```
import sys
empty = sys.getsizeof([])
values = []
for i in range(20):
    values.append(i)
    print(len(values), (sys.getsizeof(values) - empty) // 8)
```

Run it on your machine. On Python 3.13 the capacity steps through 4, 8, 16, 24, 32, 40, 52, 64, … —

slightly more than one eighth extra each time, rounded, with bigger jumps while the list is small. CPython chooses a small factor to waste little memory, because Python programs hold enormous numbers of lists. It is still a *factor*, so `append` is still amortised $O(1)$ — the same argument as this lecture, with a different constant.

That is Lecture 01's promise kept: the doubling that "happens in C, a few layers below your code", now visible, and now yours.

All the Operations

`DynamicArray`: the contract with costs

Operation	Cost	Why
`a[i]`, `a[i] = x`, `len(a)`	$O(1)$	address arithmetic; a stored count
`append(x)`	**amortised** $O(1)$, worst $O(n)$	occasional resize, paid for by the rest
`pop()` from the end	$O(1)$	nothing moves
`insert_at(i, x)`	$O(n)$	shift `i .. size-1` right (and maybe resize)
`pop(i)` from the middle	$O(n)$	shift `i+1 .. size-1` left
`x in a`, search	$O(n)$	one by one
iterate	$O(n)$	

Exactly the costs of Python's `list` — because it **is** one.

Compare with Lab 03's table of list methods: every row matches. You now know *why* each one costs what it does, because you have written the code that pays it.

`insert_at` and `pop(i)` are Lecture 02's `insert_at` and `remove_at` from `dsa/array_ops.py`, moved inside the class — with one addition: `insert_at` must resize first if the array is full.

Remember the storage rule: `_block` is an `Array` and nothing else. No list anywhere in the class, not even as scratch space during a resize.

What it is built for

From here on, the dynamic array is **storage** for other structures:

Week	Structure	Uses the `DynamicArray` as
6	`Stack`	the items — top at the **end**, so push and pop are $O(1)$
7	`SlowQueue`	the items — `pop(0)` from the front, $O(n)$: measure it
12	`MinHeap`	the complete binary tree, stored level by level

Week	Structure	Uses the `DynamicArray` as
14	`Graph`	each node's list of neighbours

A bug in your `DynamicArray` becomes a bug in all four. **Test it well now.**

This is the storage rule from Lecture 02 paying off. Because the stack, the slow queue, the heap and the graph are all built on the structure you write this week, their costs come straight from this lecture's table: a stack is $O(1)$ because append and pop at the end are; the slow queue is slow because `pop(0)` shifts; a heap can add a leaf in amortised $O(1)$ before sifting it up.

This Week

Exercises: `dsa/dynamic_array.py`

Method	Target	Watch out for
`append(value)`	amortised $O(1)$	resize **before** writing; grow by `self.growth`
`_resize(capacity)`	$O(n)$	copy only `size` elements; count `resize_count`
`__getitem__`, `__setitem__`	$O(1)$	bounds are `0 .. size-1`; translate negatives
`insert_at(index, value)`	$O(n)$	resize if full; shift from the **end**
`pop(index=-1)`	$O(1)$ at the end, else $O(n)$	`IndexError` when empty; clear the freed slot

```
pytest tests/test_dynamic_array.py -v
```

`test_doubling_keeps_resizes_logarithmic`: 1,000 appends must resize fewer than 20 times.

The constructor, `__len__`, `__iter__`, `__repr__` and `make_block` are given. The resize test is the one that separates doubling from a constant step: with doubling, 1,000 appends from capacity 1 resize exactly 10 times ($2^{10} = 1024$); with "+ 1" they resize 999 times.

After this week, `pytest tests/test_stack_queue.py` will start to make sense: the stack and the slow queue already use your `DynamicArray`, so they depend on it being right.

Homework 4 — before Lecture 05

1. **Implement** `dsa/dynamic_array.py` until `pytest tests/test_dynamic_array.py` passes.
2. **Measure.** Make a copy of your class that grows by `+ 10` instead of `× growth`. Time 1,000 … 32,000 appends for both with `viz.complexity.measure`, and plot with `reference=["n", "n^2"]`. Plot `resize_count` against n for both as well.

3. **Prove.** With growth factor 3 (capacity 1, 3, 9, 27, ...), show that n appends cost $O(n)$ in total.
4. **Extend.** Add shrinking to your class: halve the capacity when pop leaves the array one quarter full. Write a test that alternates append and pop at the boundary, and check with `resize_count` that it does not thrash.

For item 2, the starting point is the `measure` example in the Lecture 02 handout; `make` should return an empty array and the function should append n items to it. Keep both classes in a scratch file or the notebook, not in `dsa/dynamic_array.py`, so the tests keep checking the real one.

For item 3, follow the aggregate proof: where do the resizes happen, how much does each one copy, and what does the geometric series sum to?

Summary

Seven things to keep

1. A dynamic array is an **`Array` plus a size**; capacity ≥ size.
2. Full? **Allocate** a bigger Array, **copy** everything, **switch** — $O(n)$.
3. Grow by a **constant** → $\Theta(n^2)$ for n appends. Grow by a **factor** → $\Theta(n)$.
4. **Amortised** $O(1)$: any n appends cost < 3n. The worst single append is still $O(n)$.
5. Amortised is a **guarantee over a sequence**, not an average over inputs.
6. Shrink at **one quarter** full, not one half — or it **thrashes**.
7. Python's `list` is exactly this, growing by about **1/8**.

Next

Week 5 — Linked lists. The opposite trade: no copying, ever, and $O(1)$ insertion at the front — paid for with $O(n)$ indexing.

Sources and further reading

- **T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein.** *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, chapter 17, "Amortized Analysis" — the aggregate, accounting and potential methods, with dynamic tables (section 17.4) as the worked example, including the one-quarter shrinking rule.
- **M. T. Goodrich, R. Tamassia and M. H. Goldwasser.** *Data Structures and Algorithms in Python*, Wiley, 2013, section 5.3, "Dynamic Arrays and Amortization" — a dynamic array built on a `ctypes` array, the approach this course takes.
- **CPython source.** `Objects/listobject.c`, function `list_resize` — the over-allocation rule you measured with `sys.getsizeof`.
- **R. E. Tarjan.** "Amortized Computational Complexity", *SIAM Journal on Algebraic and Discrete Methods* 6(2), 1985 — the paper that named and formalised amortised analysis.

Every figure in this lecture is generated by `tools/figures_l04.py`. The cost charts are counted; the CPython chart is read from your interpreter, so run the script to see your own Python's growth

pattern.

Questions for Week 4

> ***The answers are at the back of the book,*** *in* Answers — Week 4. *Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify. Unless a question says otherwise, a dynamic array starts* ***empty with capacity 1****, and "doubling" means that an append which finds it full first doubles the capacity.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W4-M01 – W4-M22
B	Short answer and essay	W4-E1 – W4-E5
C	Trace the growth — show every step	W4-T1 – W4-T5
D	Proofs	W4-P1 – W4-P2
E	Find and fix the bug	W4-B1 – W4-B4
F	Write the code — checked by `pytest`	W4-C1 – W4-C3

Part A — Multiple choice

W4-M01 [what] For a `DynamicArray`, `len(a)` returns:

- **a)** the size — the number of elements stored
- **b)** the capacity of the `Array` underneath
- **c)** capacity minus size
- **d)** the number of resizes so far

W4-M02 [how] With doubling, what is the capacity after 5 appends?

- **a)** 5
- **b)** 6
- **c)** 8
- **d)** 16

W4-M03 [how] With doubling, how many resizes do 1,000 appends cause?

- **a)** 9
- **b)** 10
- **c)** 500
- **d)** 999

W4-M04 [how] If the capacity grows by **one** slot at every resize, the total number of copies for n appends is:

- **a)** $\Theta(n)$
- **b)** $\Theta(\log n)$
- **c)** $\Theta(n \log n)$

- **d)** $\Theta(n^2)$

W4-M05 [why] If the capacity grows by a constant **100** slots at every resize, the amortised cost of append is:

- **a)** $O(n)$ — the constant only divides the copying by 100
- **b)** $O(1)$ — 100 is large enough
- **c)** $O(\log n)$
- **d)** $O(100)$, which is $O(1)$

W4-M06 [what] With doubling, the amortised cost of append is:

- **a)** $O(\log n)$
- **b)** $O(1)$
- **c)** $O(n)$
- **d)** $O(n \log n)$

W4-M07 [why] With doubling, the **worst-case** cost of a single append is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n^2)$

W4-M08 [why] How does amortised cost differ from average-case cost?

- **a)** They are the same thing
- **b)** Amortised cost is always larger
- **c)** Amortised cost averages over random inputs
- **d)** Amortised cost averages over a sequence of operations and needs no assumption about the inputs

W4-M09 [how] With doubling, the total number of element **copies** for n appends is:

- **a)** less than 2n
- **b)** exactly $n \log_2 n$
- **c)** exactly $n^2/2$
- **d)** exactly n

W4-M10 [how] With doubling, which of these appends triggers a resize?

- **a)** the 16th
- **b)** the 17th
- **c)** the 18th
- **d)** the 32nd

W4-M11 [why] To shrink without thrashing, a dynamic array should halve its capacity when the size falls to:

- **a)** half the capacity
- **b)** capacity – 1
- **c)** one quarter of the capacity
- **d)** zero

W4-M12 [what] "Thrashing" in a dynamic array means:

- **a)** the array is full
- **b)** an append causes a resize
- **c)** the garbage collector runs
- **d)** alternating operations near one boundary make the array resize on almost every operation

W4-M13 [what] CPython's `list` grows its capacity by about:

- **a)** one eighth, plus a little
- **b)** one slot
- **c)** a factor of 10
- **d)** exactly double

W4-M14 [how] How can you see a Python list's capacity from Python?

- **a)** `len(values)`
- **b)** from `sys.getsizeof(values)`
- **c)** `values.capacity`
- **d)** it cannot be observed

W4-M15 [how] In a `DynamicArray` of size 5 and capacity 8, `a[-1]` reads `self._block` at index:

- **a)** 7
- **b)** −1
- **c)** 4
- **d)** it raises `IndexError`

W4-M16 [what] `insert_at(0, x)` on a dynamic array of size n costs:

- **a)** $O(1)$
- **b)** amortised $O(1)$
- **c)** $O(\log n)$
- **d)** $O(n)$

W4-M17 [what] `pop()` — removing the **last** element — costs:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n^2)$

W4-M18 [why] Why does growing by a factor give amortised $O(1)$ append?

- **a)** Because the array never needs to copy
- **b)** Because the copies form a geometric series, dominated by its last term, which is less than n
- **c)** Because Python optimises it
- **d)** Because resizes happen at random

W4-M19 [why] Growing by a factor of **1.5** instead of 2:

- **a)** makes append amortised $O(n)$

- **b)** makes append amortised $O(\log n)$
- **c)** keeps append amortised $O(1)$, with more copies but less wasted space
- **d)** removes the need for resizing

W4-M20 [how] With doubling and no shrinking, just after a resize, about how much of the capacity is unused?

- **a)** none
- **b)** a quarter
- **c)** three quarters
- **d)** about half

W4-M21 [what] Which structure in `dsa/` stores its items in the student's own `DynamicArray`?

- **a)** `Stack`
- **b)** `CircularQueue`
- **c)** `OpenAddressingHashMap`
- **d)** `BinarySearchTree`

W4-M22 [how] In the accounting ("bank") proof for doubling, how many coins must each append be charged?

- **a)** 1
- **b)** 3
- **c)** $\log_2 n$
- **d)** n

Part B — Short answer and essay

W4-E1 [why] *(5 marks)* Define amortised cost. Prove that n appends to a dynamic array that doubles cost $O(n)$ in total, and conclude the amortised cost of one append.

W4-E2 [why] *(4 marks)* Explain why growing the capacity by a constant number of slots gives $\Theta(n^2)$ total cost for n appends, however large the constant, while growing by a constant factor gives $\Theta(n)$.

W4-E3 [why] *(3 marks)* `append` is "worst-case $O(n)$ and amortised $O(1)$". Explain both statements, why both are true, and when the worst case still matters.

W4-E4 [why] *(4 marks)* Explain thrashing with a concrete sequence of operations, and the shrinking rule that prevents it.

W4-E5 [why] *(3 marks)* Growth factor is a time–space trade-off. Explain the trade-off, and why CPython's `list` uses a small factor (about 1.125).

Part C — Trace the growth

Doubling, starting empty with capacity 1, unless stated otherwise. Count one unit of work per element **written** by append and one per element **copied** by a resize.

W4-T1 [how] After 10 appends: what is the capacity, how many resizes have happened, and how many elements have been copied in total?

W4-T2 [how] For 20 appends, list which appends trigger a resize, and compute the total work (writes + copies) and the average work per append.

W4-T3 [how] Now the capacity grows by **+3** at each resize (1, 4, 7, 10, ...). For 12 appends: which appends trigger a resize, what is the final capacity, and how many elements are copied?

W4-T4 [how] A shrinking dynamic array doubles when full and, **after** a pop, halves when size ≤ capacity // 4 (and capacity > 1). Starting empty, perform `append(1)` ... `append(5)`, then `pop()` four times. Give the size and capacity after every operation.

W4-T5 [how] Among the first 40 appends with doubling, which ones are "expensive" (trigger a resize)? What pattern do their numbers follow?

Part D — Proofs

W4-P1 [why] *(4 marks)* A dynamic array grows by a factor of **4** (capacity 1, 4, 16, 64, ...). Prove that n appends cost $O(n)$ in total.

W4-P2 [why] *(4 marks)* A dynamic array grows by a constant c slots. Prove that n appends copy at least $\frac{n^2}{2c} - n$ elements, and hence that the total cost is $\Omega(n^2)$ for any fixed c.

Part E — Find and fix the bug

Each method belongs to a `DynamicArray` with `_block` (an `Array`), `_size` and `_capacity`.

W4-B1 [how]

```
def append(self, value):
    if self._size > self._capacity:
        self._resize(self._capacity * 2)
    self._block[self._size] = value
    self._size += 1
```

W4-B2 [how]

```
def __getitem__(self, index):
    if index < 0:
        index += self._size
    if not 0 <= index < self._capacity:
        raise IndexError(index)
    return self._block[index]
```

W4-B3 [why] Passes every test except `test_doubling_keeps_resizes_logarithmic`.

```
def append(self, value):
    if self._size == self._capacity:
        self._resize(self._capacity + self.growth)
    self._block[self._size] = value
    self._size += 1
```

W4-B4 [why] Passes every test, yet holds on to memory it should not.

```
def pop(self, index=-1):
    if self._size == 0:
        raise IndexError("pop from empty")
    if index < 0:
        index += self._size
    value = self._block[index]
    for i in range(index, self._size - 1):
        self._block[i] = self._block[i + 1]
    self._size -= 1
    return value
```

Part F — Write the code

In `practice/week04.py`; check with `pytest tests/test_practice_week04.py -v`.

W4-C1 [how] `capacity_after(n, factor=2)` — the capacity after n appends.

W4-C2 [how] `copies_for(n, grow)` — how many elements n appends copy, when `grow(capacity)` gives the new capacity. Use it to check the numbers in Lecture 04.

W4-C3 [why] `ShrinkingArray` — a dynamic array on an `Array` that doubles when full and halves when a pop leaves it one quarter full. Why does `test_shrinking_array_does_not_thrash` pass with the quarter rule and fail with a half rule?

Week 5 — Linked Lists

> ***How to read this document.*** *This is the handout for Lecture 05. It holds everything on the slides, plus what I said out loud. The linked list is the exact mirror of the dynamic array: every operation that was cheap there is expensive here, and the other way round. It is also the first structure built from* ***nodes and references*** *rather than from an array — the building blocks of trees and graphs later.*
>
> *Slides:* `DSA27-L05-slides.pdf` *· Code:* `dsa/linked_list.py` *· Tests:* `tests/test_linked_list.py` *· Interactive demo: Linked List Lab*

Where We Are

Last week, one weakness

The dynamic array: $O(1)$ to index, amortised $O(1)$ to append —

and $O(n)$ to insert at the **front**, because **everything shifts**.

Today: a structure where nothing ever shifts, and nothing is ever copied.

Every array-based structure pays for contiguity. Keeping elements side by side is what makes `a[i]` a single address calculation — and it is also why inserting at position 0 moves every other element, and why growing means copying the whole array.

A linked list gives up contiguity entirely. Each element lives in its own small object, anywhere in memory, and knows only where the next one is. Insertion becomes a matter of changing two references. Indexing becomes a walk.

Linked list in Arabic: القائمة المتصل.

Today

1. Nodes and references
2. Walking a list — and why indexing is $O(n)$
3. Inserting: at the front, in the middle, at the end
4. Removing — and who frees the node
5. Reversing in place, with three references
6. Array against linked list: costs, memory, measured
7. Variants: tail pointer, doubly linked, sentinel

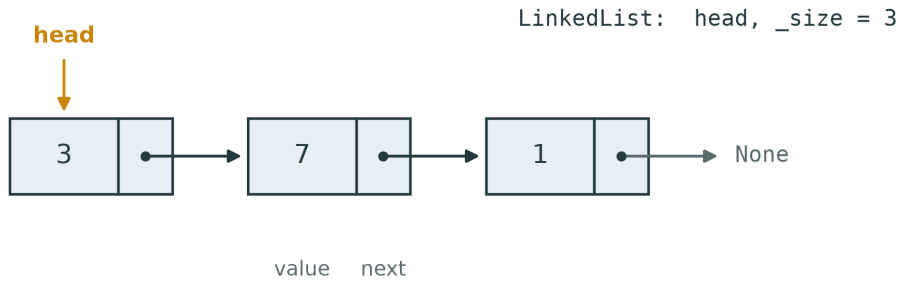
Nodes and References

A node: a value and a link

```
class Node:
    __slots__ = ("value", "next")

    def __init__(self, value, next=None):
        self.value = value
        self.next = next          # another Node, or None at the end
```

A linked list: nodes, each holding a value and a reference to the next



The list object keeps only **head** (the first node) and **_size**.

`Node` is given to you in `dsa/linked_list.py`, exactly as above. `__slots__` tells Python that a node has only these two attributes, which saves memory — it matters when a list has a million nodes.

The picture is the whole structure. There is no block, no capacity, no index arithmetic: just a chain of references ending in `None`. The `LinkedList` object holds the entry point, `head`, and a count, `_size`.

This is also the course storage rule from Lecture 02 at work: the linked list stores its data in **node objects** — one of the three things a structure is allowed to use. No `list` anywhere.

Build one by hand

```
>>> from dsa.linked_list import Node
>>> c = Node(1)
>>> b = Node(7, c)
>>> a = Node(3, b)
>>> a.value, a.next.value, a.next.next.value
(3, 7, 1)
>>> a.next.next.next is None
True
```

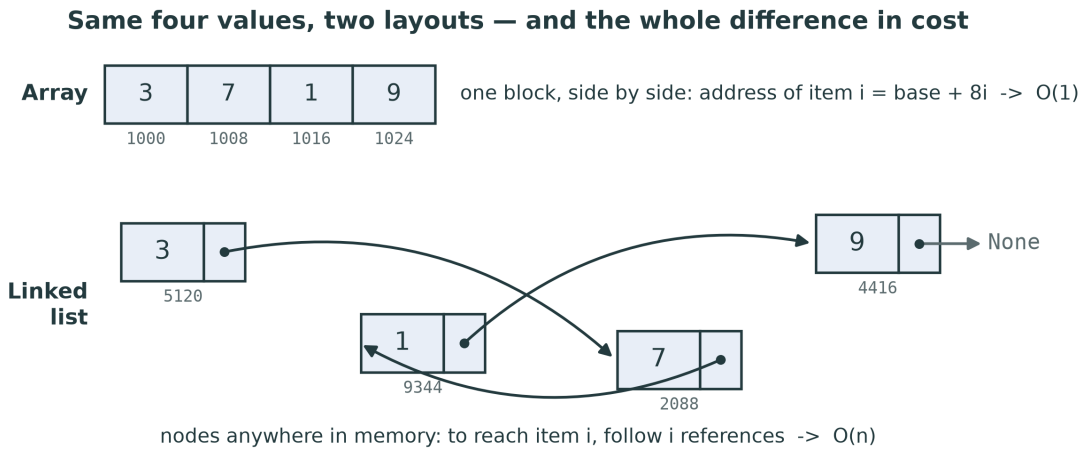
A reference **is** the arrow in the picture. `a.next` *is* `b`.

Do this at the REPL before you write any of the exercise. Everything that follows is only a matter

of which reference you change, and in what order.

Lecture 01 said a Python variable is a name bound to an object, and Lab 01 made you draw the arrows for `b = a`. A linked list is that idea turned into a data structure: `node.next` is just another name bound to another object.

Two layouts, one difference



The array's slots are contiguous, so the address of item i is arithmetic. The nodes can be anywhere — they are created one at a time, wherever Python has room — so the only way to find item i is to start at `head` and follow i references.

The addresses in the figure are illustrative, but the point is real: this is why indexing a linked list is $O(n)$, and it is also why a linked list is slower in practice than its Big-O suggests. Modern processors fetch memory in blocks and are fast at walking contiguous data (the *cache*); jumping from node to node defeats that.

Walking the List

The traversal loop

```
node = self.head
while node is not None:
    # ... use node.value ...
    node = node.next
```

Every linked-list algorithm is this loop, plus a little bookkeeping.

- `node is not None` — stop **after** the last node
- `node = node.next` — the only way to move

Learn this loop until you write it without thinking. `__iter__` in the given code is exactly this loop with `yield node.value` in the middle. `find(value)` is this loop with an index counter and an early return. `__getitem__(index)` is this loop stopped after `index` steps.

Two classic mistakes:

- `while node.next is not None:` stops **on** the last node instead of after it — right when you need the last node (appending), wrong when you need to visit every node.
- Forgetting `node = node.next` — an infinite loop. Press Ctrl+C.

Watch it move. Set a breakpoint inside this loop in `dsa/linked_list.py`, press F5, choose *pytest: current test file*, and step with F10. In the Variables panel, `node` jumps from object to object. `viz.draw.draw_linked_list(list(ll), highlight=i)` draws the same thing.

Indexing is $O(n)$ — that is the lesson

	`DynamicArray[i]`	`LinkedList[i]`
How	base + i × slot size	follow `next` i times from `head`
Cost	$O(1)$	$O(i)$ — $O(n)$ worst case

Same interface, `a[i]`. **Completely different cost.**

```
for i in range(len(ll)):
    print(ll[i])          # O(n) each time: O(n²) for the whole loop!
```

Iterate with `for value in ll:` — one walk, $O(n)$ in total.

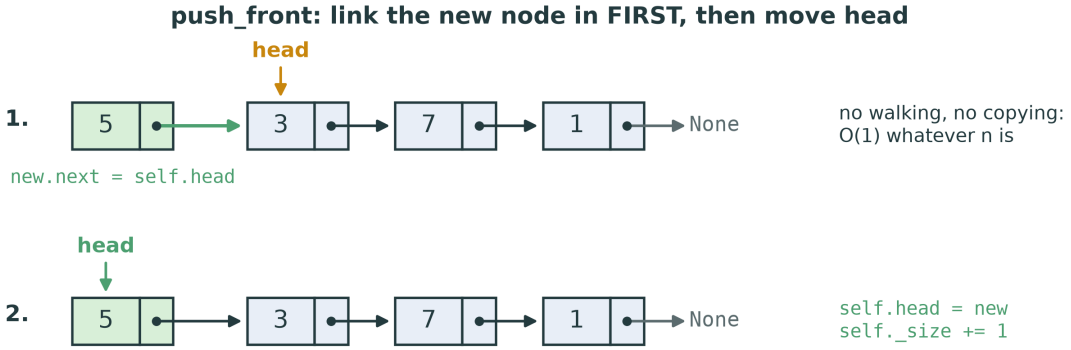
This is the most important slide for your own code. Python lets `LinkedList` support `ll[i]` through `__getitem__`, and it looks exactly like array indexing. But each `ll[i]` starts again from `head`. A loop that indexes 0, 1, 2, … does $0 + 1 + 2 + \cdots + (n-1)$ steps — the triangular sum again, $O(n^2)$.

An interface does not tell you the cost. The ADT does — and for a linked list, positional access is $O(n)$ by contract.

`len(ll)` on the other hand is $O(1)$, because the list keeps `_size` up to date. Without it, `len` would be a walk too. That is a design decision with a price: every method that adds or removes a node must remember to update `_size`.

Inserting

At the front: $O(1)$



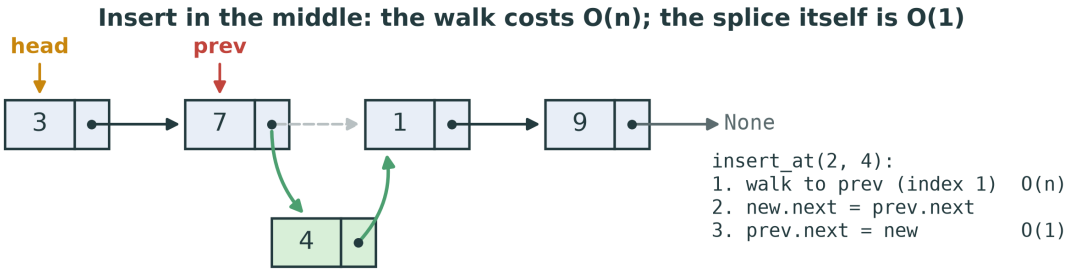

```
new = Node(value)
new.next = self.head      # 1. link the new node to the old first node
self.head = new           # 2. then make it the first node
```

Two references change, whatever the length of the list. No shifting: compare Lecture 02's array insertion, which moved every element.

Order matters. Do step 2 first and `self.head` already points at the new node, so step 1 sets `new.next = new` — a node pointing at itself, and the rest of the list is lost (unreachable, then garbage-collected). With references, the rule is: **link the new node in before you let go of the old one.**

Both steps also fit in one line — `self.head = Node(value, self.head)` — because Python evaluates the right-hand side, including the old `self.head`, before assigning. Then `self._size += 1`.

In the middle: walk, then splice



To put a new node at index i, stop at the node **before** it (index i − 1), then:

```
new.next = prev.next
prev.next = new
```

A singly linked list can only be changed from the node **before** the change, because only that node holds the reference that must be redirected. So insertion at index i walks i − 1 steps to find

`prev`: $O(n)$ in the worst case. The splice itself is two assignments, $O(1)$.

This is the honest comparison with an array: the array *finds* position i in $O(1)$ and pays $O(n)$ to *make room*; the list pays $O(n)$ to *find* position i and $O(1)$ to make room. If you already hold a reference to the right node — which happens often, for example while walking — the list's insertion is $O(1)$ and the array's is still $O(n)$.

`insert_at(0, value)` has no `prev`; it is `push_front`. Handle it first, then walk for index ≥ 1. The exercise must accept `index == len(self)` (insert at the end) and raise `IndexError` otherwise out of range.

At the end: $O(n)$ — unless you keep a tail

`append` must find the last node: walk from `head` until `node.next is None` — $O(n)$.

Keep a second reference, **`_tail`**, to the last node, and `append` becomes $O(1)$:

```
self._tail.next = new
self._tail = new
```

The price: **every** method that can change the last node must keep `_tail` correct.

The exercise asks for `append` in $O(n)$, with the tail pointer as a challenge in the docstring. If you add it, audit every method: `push_front` on an empty list must set the tail too; `pop_front` of the only node must clear it; `remove` of the last node must move it back — and in a **singly** linked list, "back" means walking to find the new last node, $O(n)$. `reverse` swaps head and tail.

That is the general lesson of this lecture: every extra reference buys speed somewhere and costs care everywhere.

Removing

From the front: $O(1)$

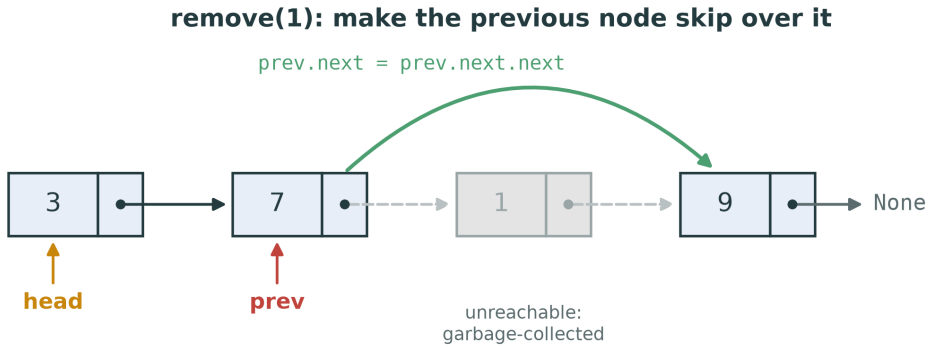
```
if self.head is None:
    raise IndexError("pop from empty list")
value = self.head.value
self.head = self.head.next
self._size -= 1
return value
```

The old first node is no longer referenced by anything — so it is **gone**.

Who frees the node? In C, you would call `free(old)`, and forgetting is a memory leak while freeing too early is a crash. In Python, the garbage collector reclaims any object nothing refers to — Lecture 01's memory models. You simply stop pointing at it.

That is convenient and it hides the structure, which is why the debugger is worth using this week: watch `self.head` change and the old node disappear from view.

By value: skip over it



Find the node **before** the one to remove, then `prev.next = prev.next.next`.

The **head** is the special case: it has no `prev`.

`remove(value)` removes the **first** node holding `value` and returns `True`, or returns `False` if there is none. The shape:

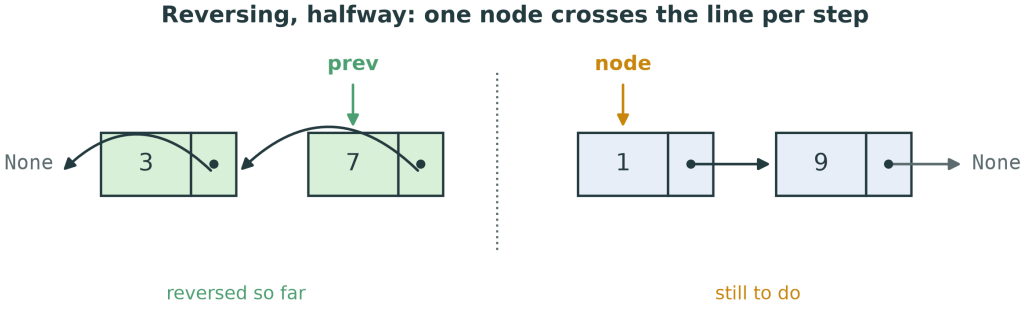
1. If the list is empty, return `False`.
2. If the head holds `value`, move `head` on, as in `pop_front`.
3. Otherwise walk with `prev`, checking `prev.next.value`, and when it matches, `prev.next = prev.next.next`.

Do not forget `_size`. And compare the cost with the array: finding the value is $O(n)$ in both; removing it is $O(1)$ here and $O(n)$ there, because the array must close the gap.

The special case for the head is annoying enough that there is a standard trick to remove it — the sentinel node, at the end of this lecture.

Reversing in Place

Three references



Keep **prev** (the reversed part) and **node** (the rest). One step moves one node across the line:

1. remember `node.next` — you are about to overwrite it

2. point `node.next` back at `prev`
3. advance: `prev` becomes `node`, `node` becomes the remembered next

At the end, `prev` is the new head. $O(n)$ time, $O(1)$ extra space.

This is the classic linked-list problem, and it appears in interviews for a reason: it tests whether you can hold an **invariant** in your head. Before every step, `prev` is the head of a correctly reversed list of the nodes already visited, and `node` is the head of the part still to do. The step moves exactly one node from the second to the first, and the invariant still holds.

Start: `prev = None` (nothing reversed yet), `node = self.head`. Stop when `node` is `None`. Then `self.head = prev`.

Step 1 is the one people forget. Once you have set `node.next = prev`, the only reference to the rest of the list is gone — unless you saved it first.

The test `test_reverse_relinks_rather_than_rebuilding` checks that you reversed the **same** node objects. Building a new list of new nodes is $O(n)$ extra space and fails the test — the point is to re-link, not to copy.

Draw three or four nodes on paper, and do the three moves by hand, before you write a line.

Array Against Linked List

The costs, side by side

Operation	`DynamicArray`	`LinkedList`
read / write position i	$O(1)$	$O(n)$
insert or remove at the **front**	$O(n)$	$O(1)$
append at the **end**	amortised $O(1)$	$O(n)$ — $O(1)$ with a tail
remove from the end	$O(1)$	$O(n)$ singly linked
insert / remove **given the position's neighbour**	$O(n)$	$O(1)$
search for a value	$O(n)$	$O(n)$
`len`	$O(1)$	$O(1)$ — because of `_size`
memory per element	one reference	a whole node

Read the table as a set of trade-offs, not a ranking. Neither structure is better. Each is $O(1)$ exactly where the other is $O(n)$ — which is why this course builds both, and why Week 6 (stacks) and Week 7 (queues) will ask you to choose between them.

"Remove from the end" is $O(n)$ for a singly linked list even with a tail pointer: after removing the last node, you need the new last node, and only the node before it knows where it is. A **doubly** linked list fixes that.

Measured, not asserted

Both panels are log–log. At n = 32,768:

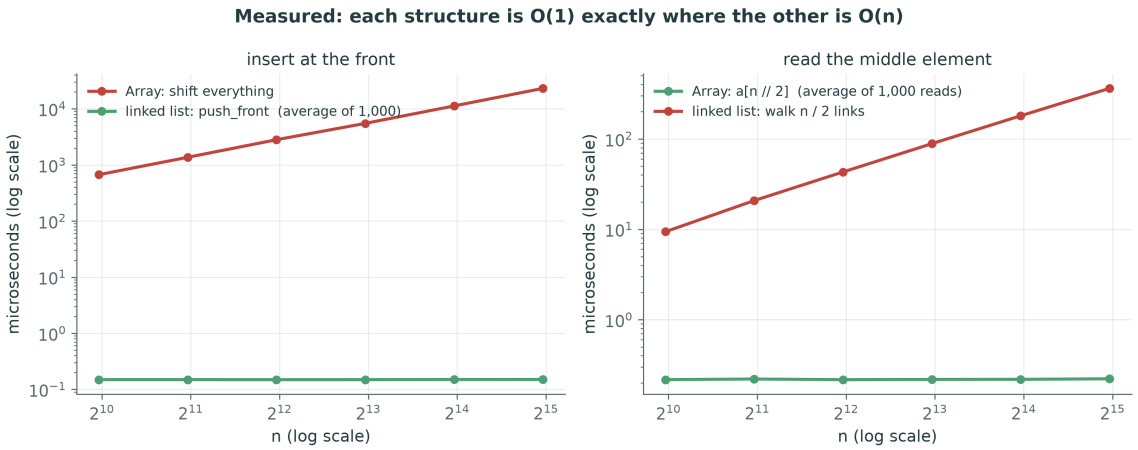


Figure 16: Real timings, reference implementations on the course `Array` and on nodes.

- inserting at the front of an Array took about 23,000 µs (every element shifted); `push_front` took about 0.15 µs — independent of n;
- reading the middle element of an Array took about 0.2 µs; walking to the middle of a linked list took about 370 µs.

The flat lines are averaged over 1,000 operations each, because a single $O(1)$ operation is too fast for the clock — the same caution as in Lecture 02.

Memory: a node is not free

```
>>> import sys
>>> sys.getsizeof(Node(1))     # with __slots__, on 64-bit CPython 3.13
48
```

An array slot is **8 bytes** (one reference). A node is **48** — six times as much, per element, **plus** the value object both of them point to.

Every node is a full Python object with a header, a type pointer and two references. A dynamic array stores only the references, side by side, with some unused capacity. For a million small values, that difference is tens of megabytes.

Memory layout also matters for speed: an array's references are contiguous and read efficiently in blocks; nodes are scattered, and every `node.next` may be a jump to a different place in memory. That is why, in practice, arrays often beat linked lists even for operations where the Big-O is equal.

When is a linked list the right choice? When you insert and remove at the front a lot (a stack, Week 6); when you splice in and out at positions you already hold (an LRU cache, a text editor's buffer); when you must never pay for a big copy at an unlucky moment; and as the **chains** inside a hash table (Week 13), where each bucket is a short linked list of `Entry` nodes.

Variants

Doubly linked, and sentinels

Doubly linked: each node also holds `prev`.

- remove a node you hold in $O(1)$ — no need to find its predecessor
- walk both ways; remove from the end in $O(1)$
- costs one more reference per node, and twice the bookkeeping

Sentinel (dummy) node: a permanent, valueless node before the first real one.

- every real node now has a predecessor — **no special case for the head**

Circular: the last node points back to the first — useful for round-robin.

Python's `collections.deque` — the queue from Lab 03 — is a doubly linked list of fixed-size blocks: $O(1)$ at both ends like a linked list, with less memory overhead per element like an array. Real systems often combine the two ideas.

With a sentinel, `remove` has no special case:

```
prev = self.sentinel                # sentinel.next is the first real node
while prev.next is not None:
    if prev.next.value == value:
        prev.next = prev.next.next
        self._size -= 1
        return True
    prev = prev.next
return False
```

The exercise does **not** use a sentinel — `self.head` is the first real node, as the tests check — so you will write the head case yourself. The trick is worth knowing for the exam and for later structures.

Watch it run: the Linked List Lab

Linked List Lab — open it in any browser.

- The code, the references and the **nodes and links**, one step at a time
- Insert at the front, in the middle, at the end; pop, remove, find, reverse
- **Doubly linked**: four links per insertion; pop from both ends in $O(1)$
- A tail pointer and a sentinel; counters of **hops** and **links changed**
- *What happens next?* — answer before you step
- Eight scenarios that **go wrong**: links in the wrong order, off by one, no head case, a forgotten `prev`…

The Linked List Lab is the demo used in the lecture. It runs every operation in this handout, a doubly linked list, and eight broken versions, and shows each step: the line that runs, every reference (`head`, `prev`, `current`, the new node…) attached to the node it points to, and each link as it changes. The link about to change is drawn dashed one step **before** it changes, so you can predict it. A node nothing refers to any more fades out, marked *unreachable*. Use it to check your drawings.

- **Online:** https://helghareeb.github.io/DSA27/lectures/05-linked-lists/linked-list-lab.html
- **Offline:** open `docs/lectures/05-linked-lists/linked-list-lab.html` from your copy of the repository. It is a single file and needs no installation.

How to use it:

- Pick a scenario at the top. Change the values, the index or the value in the `>>>` line and press **Run**.
- **Step** (→) moves one step and **Back** (←) undoes it. **Skip loop** (O) runs the rest of a walk in one step; **Step out** (U) finishes the current method. **Play** (P) runs on its own.
- At the key moments the demo stops with a question. Answer it (A–D) **before** you look; that is where the learning happens.

What to try, in order: `push_front` against `append` (watch the hops counter), `insert_at` in the middle (why stop at the node *before* the gap?), `remove` at the head and in the middle, `reverse`, then the doubly linked scenarios, and then every scenario under *Goes wrong*. For each broken one, find the bug in the code **before** you step.

The singly linked scenarios show code that solves this week's exercise in `dsa/linked_list.py`. Write your own version first. The doubly linked list was written for the demo; there is no exercise for it.

This Week

Exercises: `dsa/linked_list.py`

Method	Target	The trap
`push_front(value)`	$O(1)$	link first, then move `head`
`append(value)`	$O(n)$ (tail: $O(1)$ challenge)	the empty list has no last node
`insert_at(index, value)`	$O(n)$	index 0 is `push_front`; stop at `index - 1`
`pop_front()`	$O(1)$	`IndexError` when empty
`remove(value)`	$O(n)$	the head has no `prev`; return `True`/`False`
`find(value)`	$O(n)$	`-1` when absent
`reverse()`	$O(n)$, $O(1)$ space	save `next` before you overwrite it
`__getitem__(index)`	$O(n)$	`IndexError` out of range

Every method that adds or removes a node: **update `_size`**.

```
pytest tests/test_linked_list.py -v
```

`Node`, the constructor, `__len__`, `__iter__` and `__repr__` are given. Draw before you code: for each method, sketch the nodes and references before and after, and write the reference assignments in the order that never loses the rest of the list.

Test the edge cases by hand: the empty list, a one-node list, the first node, the last node. Almost every linked-list bug lives in one of those four.

Homework 5 — before Lecture 06

1. **Implement** `dsa/linked_list.py` until `pytest tests/test_linked_list.py` passes.
2. **Tail pointer.** Add `_tail` and make append $O(1)$. List every method you had to change, and why.
3. **Measure.** Time `push_front` against `DynamicArray.insert_at(0, x)`, and `ll[n // 2]` against `arr[n // 2]`, for n = 1,000 … 32,000. Do your curves match the figure in this handout?
4. **On paper.** Trace `reverse()` on `[1, 2, 3, 4]`: draw `prev`, `node` and all the links after each step.

For item 3, remember to average fast operations over many repetitions, as the figure's legend says — a single $O(1)$ call is below the clock's resolution.

Summary

Seven things to keep

1. A linked list is **nodes** — a value and a `next` reference — plus a `head`.
2. Every algorithm is the **traversal loop**: `while node is not None: … node = node.next`.
3. **Indexing is** $O(n)$; iterate instead of indexing in a loop.
4. Front insert and remove: $O(1)$. Link the new node in **before** moving `head`.
5. Changes happen **from the node before**; the head is the special case (or use a sentinel).
6. Reverse with **three references**; save `next` first.
7. Array and linked list are **mirror images**: each is $O(1)$ where the other is $O(n)$ — choose by the operations you need.

Next

Week 6 — Stacks. One ADT, and the question this lecture prepares: array or linked list underneath? Then the applications that make stacks essential.

Sources and further reading

- **M. T. Goodrich, R. Tamassia and M. H. Goldwasser.** *Data Structures and Algorithms in Python*, Wiley, 2013, chapter 7, "Linked Lists" — singly, circular and doubly linked lists, with sentinels, and the comparison with array-based sequences.
- **T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein.** *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, section 10.2, "Linked lists" — including sentinels.
- **B. W. Kernighan and D. M. Ritchie.** *The C Programming Language*, 2nd ed., Prentice Hall, 1988, chapter 6, "Structures" — self-referential structures: the same node, with `malloc` and `free` visible.

- **CPython source.** `Modules/_collectionsmodule.c` — deque as a doubly linked list of fixed-size blocks.

Every figure in this lecture is generated by `tools/figures_l05.py`. The timing figure is real measurement and will differ slightly on your machine.

Questions for Week 5

> ***The answers are at the back of the book,*** *in* Answers — Week 5. *Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify.* `Node(value, next=None)` *and* `LinkedList` *are the classes of* `dsa/linked_list.py`*: a singly linked list with* `head` *and* `_size`*, and no tail pointer unless a question says so.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W5-M01 – W5-M22
B	Short answer and essay	W5-E1 – W5-E5
C	Trace the code	W5-T1 – W5-T5
D	List state — draw every step	W5-S1 – W5-S3
E	Complexity analysis	W5-K1 – W5-K3
F	Find and fix the bug	W5-B1 – W5-B4
G	Write the code — checked by `pytest`	W5-C1 – W5-C5

Part A — Multiple choice

W5-M01 [what] A node of a singly linked list holds:

- **a)** a value and a reference to the next node
- **b)** a value and its index in the list
- **c)** a value and references to every later node
- **d)** only a value; the list stores the order separately

W5-M02 [what] Reading `ll[i]` on a linked list of n nodes costs:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** amortised $O(1)$

W5-M03 [what] `push_front` costs:

- **a)** $O(n)$
- **b)** $O(1)$
- **c)** amortised $O(1)$
- **d)** $O(\log n)$

W5-M04 [what] append on a singly linked list **without** a tail reference costs:

- **a)** $O(1)$
- **b)** amortised $O(1)$
- **c)** $O(\log n)$
- **d)** $O(n)$

W5-M05 [why] Which operation is $O(1)$ on a linked list but $O(n)$ on a dynamic array?

- **a)** inserting at the front
- **b)** reading element i
- **c)** appending at the end
- **d)** searching for a value

W5-M06 [why] Which operation is $O(1)$ on a dynamic array but $O(n)$ on a linked list?

- **a)** inserting at the front
- **b)** removing the first element
- **c)** reading element i
- **d)** `len`

W5-M07 [why] In `push_front`, why must `new.next = self.head` come **before** `self.head = new`?

- **a)** Python evaluates assignments right to left
- **b)** Done the other way round, `new.next` would point to `new` itself and the rest of the list would be lost
- **c)** It is only a style convention
- **d)** Otherwise `_size` would be wrong

W5-M08 [why] To remove a node from a singly linked list, you need a reference to:

- **a)** the head only
- **b)** the node after it
- **c)** the last node
- **d)** the node before it

W5-M09 [what] Which node is the special case in `remove(value)` without a sentinel?

- **a)** the head
- **b)** the last node
- **c)** the middle node
- **d)** none — there is no special case

W5-M10 [how] What is the running time of this loop over a linked list of n nodes?

```
for i in range(len(ll)):
    total += ll[i]
```

- **a)** $O(n)$
- **b)** $O(n \log n)$
- **c)** $O(n^2)$
- **d)** $O(1)$

W5-M11 [why] Why is `len(ll)` $O(1)$ in `dsa/linked_list.py`?

- **a)** Python caches the length of every object
- **b)** The list keeps `_size` up to date in every method that adds or removes
- **c)** Linked lists have a fixed size
- **d)** It is not — it walks the list

W5-M12 [how] Reversing a linked list in place needs how much extra space?

- **a)** $O(n)$
- **b)** $O(\log n)$
- **c)** $O(n^2)$
- **d)** $O(1)$

W5-M13 [what] On 64-bit CPython, a `Node` with `__slots__` takes about 48 bytes. One slot of an array of references takes:

- **a)** 8 bytes
- **b)** 48 bytes
- **c)** 1 byte
- **d)** it depends on the value stored

W5-M14 [why] A **doubly** linked list can do in $O(1)$ what a singly linked one cannot:

- **a)** read element i
- **b)** remove a node you already hold a reference to
- **c)** search for a value
- **d)** sort the list

W5-M15 [why] What is a sentinel (dummy) node for?

- **a)** to mark the end of the list instead of `None`
- **b)** to store the length
- **c)** to give every real node a predecessor, removing the special case for the head
- **d)** to make indexing $O(1)$

W5-M16 [why] A singly linked list **with** a tail reference. Removing the **last** node costs:

- **a)** $O(1)$ — the tail points at it
- **b)** amortised $O(1)$
- **c)** $O(\log n)$
- **d)** $O(n)$ — the new last node must be found from the head

W5-M17 [what] Python's `collections.deque` is implemented as:

- **a)** a doubly linked list of fixed-size blocks
- **b)** a dynamic array
- **c)** a hash table
- **d)** a singly linked list of single values

W5-M18 [why] Why does the traversal loop test `while node is not None:` rather than `while node.next is not None:`?

- **a)** It is faster

- **b)** The second version stops **on** the last node, so it never visits it — and crashes on an empty list
- **c)** `node.next` is not allowed in a condition
- **d)** There is no difference

W5-M19 [how] After `a = Node(1)`, `b = Node(2, a)`, `c = Node(3, b)`, what is `c.next.next.value`?

- **a)** 3
- **b)** 2
- **c)** 1
- **d)** it raises `AttributeError`

W5-M20 [what] In Python, what happens to a node that nothing refers to any more?

- **a)** it stays in memory until the program ends
- **b)** you must call `del` on it
- **c)** it becomes the new head
- **d)** the garbage collector reclaims it

W5-M21 [how] To insert a new node at index i ≥ 1, you walk to the node at index:

- **a)** i − 1
- **b)** i
- **c)** i + 1
- **d)** 0 only

W5-M22 [what] Where in this course do short linked lists appear inside another structure?

- **a)** in the dynamic array
- **b)** as the chains of a hash table
- **c)** in the heap
- **d)** in the course `Array`

Part B — Short answer and essay

W5-E1 [why] *(4 marks)* Compare a dynamic array and a singly linked list on four operations, with their costs. For each structure, give one situation where you would choose it, and say why.

W5-E2 [how] *(4 marks)* Explain `push_front` step by step with a diagram. Show what goes wrong if the two assignments are done in the other order.

W5-E3 [why] *(3 marks)* `LinkedList` supports `ll[i]`, just like an array. Explain why the cost is different, and why `for i in range(len(ll)): ll[i]` is a bad loop. What should be written instead?

W5-E4 [why] *(4 marks)* A tail reference makes `append` $O(1)$. What does it cost? Name the methods that must change, and explain why removing the last node is still $O(n)$.

W5-E5 [why] *(3 marks)* Explain what a doubly linked list and a sentinel node each fix, and what each costs.

Part C — Trace the code

W5-T1 [how] What is printed?

```
head = None
for v in [1, 2, 3]:
    head = Node(v, head)
node = head
while node is not None:
    print(node.value, end=" ")
    node = node.next
```

W5-T2 [how] What is printed?

```
a = Node(1, Node(2, Node(3, Node(4))))
a.next = a.next.next
print(a.value, a.next.value, a.next.next.value, a.next.next.next)
```

W5-T3 [how] What does `mystery` return for chains of 5 nodes, 6 nodes, 1 node, and for `None`? What does it compute?

```
def mystery(head):
    count = 0
    node = head
    while node is not None and node.next is not None:
        node = node.next.next
        count += 1
    return count
```

W5-T4 [how] Trace `reverse()` on the list 1 → 2 → 3. Give `prev`, `node` and the links after each step.

W5-T5 [how] What is printed? What kind of structure has been made?

```
x = Node("a")
y = Node("b", x)
x.next = y
print(y.next.next.value, x.next.next is x)
```

Part D — List state

W5-S1 [how] Draw the list after each line, and give `len(ll)` and `ll.find(3)` at the end.

```
ll = LinkedList([1, 2, 3])
ll.push_front(0)
```

```
ll.insert_at(2, 9)
ll.pop_front()
ll.remove(2)
ll.append(5)
```

W5-S2 [why] A student writes `push_front` as

```
new = Node(value)
self.head = new
new.next = self.head
```

Draw the list after `ll = LinkedList([3, 7]); ll.push_front(5)`. What does `list(ll)` do?

W5-S3 [how] Draw the three references and all the links, step by step, while `remove(7)` runs on 3 → 7 → 1 → 9.

Part E — Complexity analysis

W5-K1 [how] Give Θ and justify:

```
ll = LinkedList()
for i in range(n):
    ll.append(i)            # no tail reference
```

How would you build the same list in $\Theta(n)$ without a tail reference?

W5-K2 [how] Give Θ and justify, for a linked list `ll` of n nodes:

```
count = 0
for value in ll:
    if ll.find(value) != -1:
        count += 1
```

W5-K3 [why] A queue must support "add at one end" and "remove from the other". With a singly linked list and **both** head and tail references, which end should be the front of the queue, and why? What are the costs?

Part F — Find and fix the bug

W5-B1 [how]

```
def find(self, value):
    node, index = self.head, 0
    while node.next is not None:
        if node.value == value:
```

```
            return index
        node, index = node.next, index + 1
    return -1
```

W5-B2 [how]

```
def remove(self, value):
    prev = self.head
    while prev is not None and prev.next is not None:
        if prev.next.value == value:
            prev.next = prev.next.next
            self._size -= 1
            return True
        prev = prev.next
    return False
```

W5-B3 [how]

```
def pop_front(self):
    value = self.head.value
    self.head = self.head.next
    return value
```

W5-B4 [why]

```
def reverse(self):
    prev, node = None, self.head
    while node is not None:
        node.next = prev
        prev = node
        node = node.next
    self.head = prev
```

Part G — Write the code

In `practice/week05.py`; check with `pytest tests/test_practice_week05.py -v`. Each function takes the first `Node` of a chain; walk and re-link — no copying into a Python list.

W5-C1 [how] `middle_value(head)` — the middle value in **one** pass.

W5-C2 [why] `has_cycle(head)` — does the chain loop back on itself? $O(1)$ extra space. Why must the fast reference eventually meet the slow one?

W5-C3 [how] `merge_sorted_chains(first, second)` — merge by re-linking, stable.

W5-C4 [how] `remove_duplicates_sorted(head)` — in place.

W5-C5 [how] `kth_from_end(head, k)` — k = 1 is the last node.

Week 6 — Stacks

> ***How to read this document.*** *This is the handout for Lecture 06. It holds everything on the slides, plus what I said out loud. The stack is the simplest ADT in the course, and one of the most useful: the structure is almost trivial, and the interesting part is what you* do *with it. It is also the first time you choose between the two structures of Weeks 4 and 5.*
>
> *Slides:* `DSA27-L06-slides.pdf` *· Code:* `dsa/stack.py` *· Tests:* `tests/test_stack_queue.py` *(the stack half)*

Where We Are

Two structures, one question

Week 4: the **dynamic array** — cheap at the **end**. Week 5: the **linked list** — cheap at the **front**.

Today: an ADT that only ever touches **one end** — so either structure can keep its contract in $O(1)$. Which, and how?

Lecture 01 separated the **ADT** — the contract — from the **data structure** that keeps it, and promised that choosing between structures is the engineering. This is the first week you do that choice for real, with two structures you have built yourself.

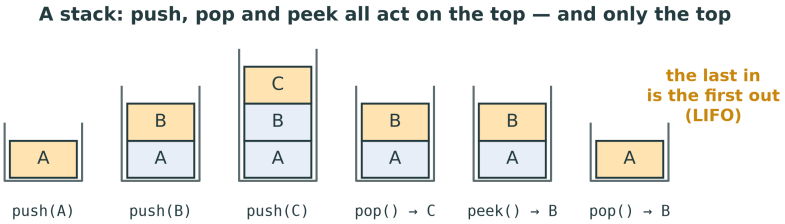
Stack in Arabic: المكد.

Today

1. The Stack ADT: push, pop, peek — LIFO
2. Two honest implementations, and one dishonest one
3. Balanced brackets
4. Postfix: expressions without parentheses
5. Infix to postfix: the shunting-yard algorithm
6. The stack you have used all term: the call stack

The Stack ADT

Last in, first out



Operation	Meaning	Cost
`push(x)`	put x on top	$O(1)$
`pop()`	remove and return the top — `IndexError` if empty	$O(1)$
`peek()`	return the top without removing it — `IndexError` if empty	$O(1)$
`is_empty()`, `len(s)`		$O(1)$

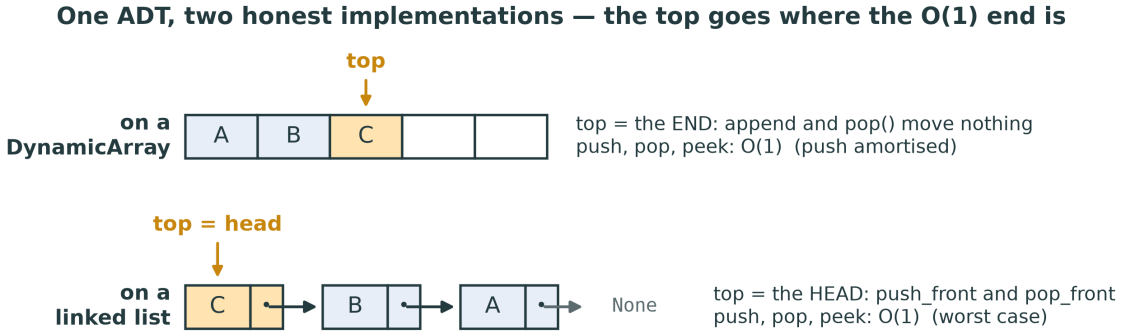
That table **is** the ADT: operations, their meaning, their cost — and nothing about storage. Everything a stack can do happens at one end, the **top**. There is no indexing, no search, no access to the middle, by design: a restriction that turns out to be exactly what a surprising number of problems need.

Everyday stacks: a pile of plates; the *Undo* history of an editor (the last edit is the first undone); the *Back* button of a browser; and, most important for us, the call stack of Lecture 03.

Empty stacks. Popping or peeking an empty stack is an error, and the contract says which one: `IndexError`, like `list.pop()` on an empty list. The tests check it. Returning `None` instead would be worse than an error: a `None` can travel a long way through a program before anyone notices.

Two Honest Implementations

The top goes where the $O(1)$ end is



- On a **dynamic array**: top = the **end**. append and `pop()` — nothing moves.
- On a **linked list**: top = the **head**. `push_front` and `pop_front`.

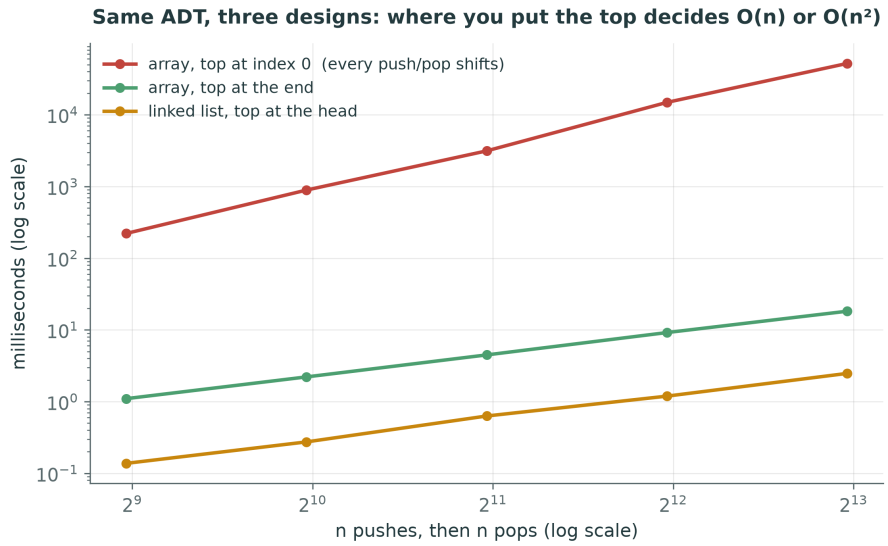
Both are honest stacks: both keep every operation $O(1)$. They differ in the details that Weeks 4 and 5 taught you to see:

	Array, top at end	Linked list, top at head
push	**amortised** $O(1)$ — the occasional resize copies	$O(1)$ worst case
pop, peek	$O(1)$	$O(1)$
memory per item	one reference (+ spare capacity)	a whole node (48 bytes)
memory locality	contiguous	scattered

`dsa/stack.py` uses the first: `self._items` is your own `DynamicArray` from Week 4. That is the storage rule from Lecture 02 at work — a structure built on a structure you built — and it means a bug in last week's `DynamicArray` shows up in this week's `Stack`.

...and one dishonest one

Put the top at **index 0** of an array, and **every** push and pop shifts the whole stack: $O(n)$ each.



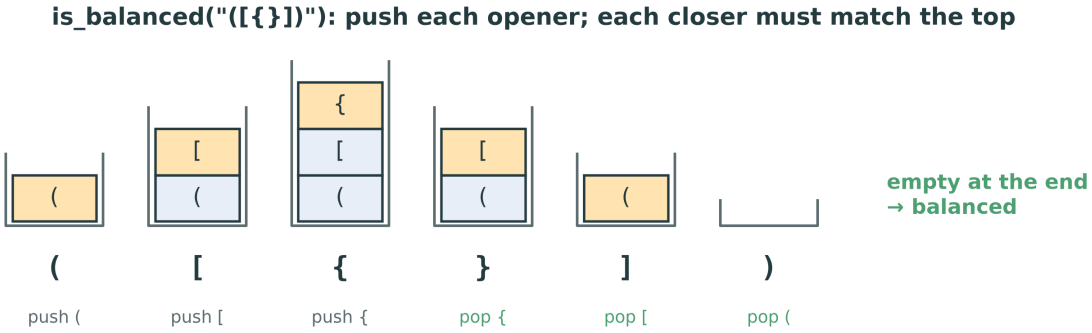
Real timings of n pushes followed by n pops, on reference implementations. The two honest designs grow linearly — one unit of slope on the log–log plot, $O(n)$ for n operations, $O(1)$ each. The top-at-index-0 design has twice the slope: $O(n^2)$ in total. At n = 8,192 that is roughly 50 seconds against a few milliseconds.

The two honest lines are not on top of each other: here the linked version is faster, because every access to the course `Array` goes through a Python-level bounds check (`dsa/array.py`), while a node's `next` is a plain attribute. That is a constant factor — the thing Big-O ignores and measurement shows. Python's own `list`, whose checks run in C, would beat both.

Same ADT, same interface, same tests pass — and one design is quadratic. The tests check *what* your stack does. Only you can check *how*.

Application 1: Balanced Brackets

Every closer must match the most recent opener



- **Opener** `(` `[` `{`: push it.
- **Closer** `)` `]` `}`: the stack must be **non-empty**, and the popped opener must be its **partner**.
- **End of text**: the stack must be **empty**.
- Anything else: ignore.

Why a stack? Because the rule is "the most recently opened bracket must close first" — last in, first out. `([)]` fails exactly because `)` arrives while the most recent opener is `[`.

The three ways to be unbalanced correspond to the three checks:

Input	What happens	Which check
`([)]`	`)` pops `[` — not its partner	the partner check
`)`	a closer arrives with the stack empty	the non-empty check
`(`	the text ends with `(` still on the stack	the empty-at-the-end check

Forgetting any one of them passes most tests and fails one. That is precisely what `tests/test_stack_queue.py::test_is_balanced` is built to catch — read its cases before you write the function.

The cost: every character is looked at once, and each push and pop is $O(1)$: $O(n)$ time, and $O(n)$ space in the worst case `((((`...).

This is how every compiler and editor checks brackets, and how an HTML or XML parser checks that tags nest.

Application 2: Postfix

Expressions with no parentheses

Infix (usual)	Postfix (reverse Polish)
`3 + 4`	`3 4 +`
`3 + 4 * 2`	`3 4 2 * +`
`(3 + 4) * 2`	`3 4 + 2 *`
`8 - 3 - 2`	`8 3 - 2 -`

In **postfix**, each operator comes **after** its operands. No parentheses, no precedence rules — the order says everything.

Postfix notation was introduced by the Australian philosopher and computer scientist Charles Hamblin in the 1950s, building on the prefix ("Polish") notation of the logician Jan Łukasiewicz — hence *reverse Polish notation*, RPN. Hewlett-Packard calculators used it for decades, and the Java and Python virtual machines are essentially postfix machines: they run a stack of operands.

Compare the second and third rows. In infix, the parentheses change the meaning. In postfix, the *order* of the tokens carries that meaning instead, so no parentheses are needed at all.

Evaluating postfix with one stack

- **Number:** push it.
- **Operator:** pop **two** values, apply, push the result.
- **End:** exactly one value is left — the answer.

Token	Action	Stack (top on the right)
3	push	3
4	push	3 4
2	push	3 4 2
*	pop 2 and 4, push 8	3 8
+	pop 8 and 3, push 11	11

The operand order trap. For `8 3 -`, the first value popped is 3 and the second is 8, and the answer is `8 - 3` — **second popped minus first popped**. Get it the wrong way round and addition and multiplication still work (they commute), while subtraction and division silently give wrong answers. A test with `-` or `/` is the only thing that catches it.

Malformed input. If an operator finds fewer than two values, or more than one value is left at the end, the input was not valid postfix. Raise `ValueError`.

This is `evaluate_postfix` in `dsa/translation.py` — an exercise for Week 15, where the whole calculator comes together. You can write it now: it needs only this week's stack. $O(n)$ time.

Application 3: Infix to Postfix

The shunting-yard algorithm

Read the infix tokens left to right, with an **output list** and an **operator stack**:

1. **Number** → output.
2. **Operator** → first pop to the output every operator on the stack with **higher or equal precedence**, stopping at a `(`; then push this one.
3. **`(`** → push.
4. **`)`** → pop to the output until the `(`; discard the `(`.
5. **End** → pop everything left to the output.

Precedence: `*` `/` above `+` `-`.

Edsger Dijkstra described this algorithm in 1961, and named it after a railway shunting yard: numbers go straight through the junction, while operators wait on a siding — the stack — until it is their turn.

The idea: an operator must wait until its right operand has been output. It can leave the siding only when a *later* operator of lower or equal precedence arrives (which means its operand is complete), or a `)` closes its group, or the input ends.

Equal precedence pops too — that is what makes `-` and `/` **left associative**: `8 - 3 - 2` means `(8 - 3) - 2`. When the second `-` arrives, the first `-` (equal precedence) is popped to the output first, giving `8 3 - 2 -`. (An operator like `**`, which is right associative, pops only on strictly higher precedence. The exercise does not include it.)

Trace: `3 + 4 * 2`

Token	Output	Stack
3	3	
+	3	+
4	3 4	+
*	3 4	+ * ← + is lower, so it stays
2	3 4 2	+ *
end	**3 4 2 * +**	

Trace: `( 3 + 4 ) * 2`

Token	Output	Stack
(		(
3, +, 4	3 4	(+
)	3 4 +	← pop to (
*, 2	3 4 + 2	*
end	**3 4 + 2** *	

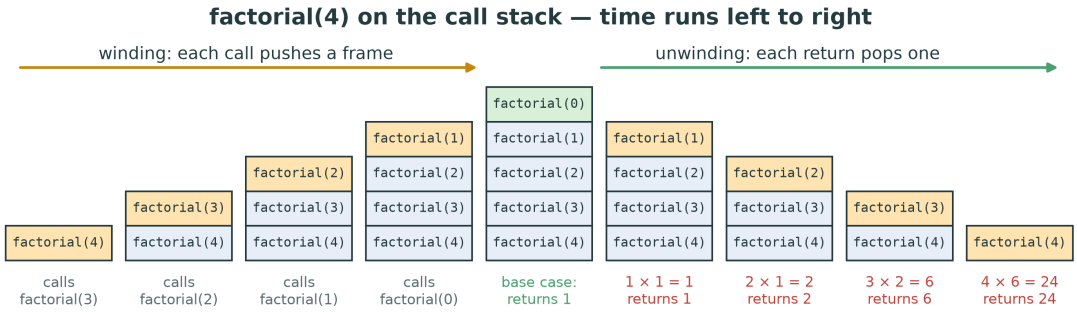
These are exactly the two cases in `tests/test_stack_queue.py::test_infix_to_postfix`. Trace `8 - 3 - 2` and `2 * 3 + 4` yourself before writing the function: one checks the "equal

precedence pops" rule, the other checks that a lower-precedence operator empties the higher ones first.

The whole calculator. Infix → (shunting-yard) → postfix → (evaluation) → a value: two passes, two stacks, $O(n)$, no tree and no recursion. Week 15 builds the other route — a recursive parser and an expression tree — and compares them. Declared in the 2013/2014 bylaws as "the principles of language translation", and this is where it starts.

Application 4: The Call Stack

You have been using a stack since Week 3



Each call **pushes** a frame; each `return` **pops** one. Recursion **is** a stack.

Lecture 03's figure, seen again: the call stack is a stack of frames, managed by Python for you. That is why any recursive algorithm can be rewritten with a loop and an explicit stack — the stack you now know how to build. Week 14 does exactly that for depth-first search: `dfs` recursive, and `dfs_iterative` with your own `Stack`, visiting the nodes in the same order.

An example you can write now: summing a nested list without recursion.

```
from dsa.stack import Stack

def nested_sum(nested):
    total, pending = 0, Stack([nested])
    while not pending.is_empty():
        item = pending.pop()
        if isinstance(item, list):
            for inner in item:
                pending.push(inner)        # "call" on every inner item
        else:
            total += item
    return total
```

Every `push` is a recursive call that has not happened yet; every `pop` is one being made. And unlike the call stack, this one has no 1,000-frame limit.

Undo and redo use two stacks: doing an action pushes it on the undo stack; *Undo* pops it and

pushes it on the redo stack; *Redo* moves it back; any new action clears the redo stack.

This Week

Exercises: `dsa/stack.py`

Function	Target	The trap
`Stack.push(value)`	$O(1)$ amortised	top at the **end** of `self._items`
`Stack.pop()`	$O(1)$	`IndexError` when empty
`Stack.peek()`	$O(1)$	`IndexError` when empty; do not remove
`is_balanced(text)`	$O(n)$	three checks: partner, non-empty, empty at the end
`infix_to_postfix(tokens)`	$O(n)$	pop on **higher or equal** precedence; `(` and `)`

```
pytest tests/test_stack_queue.py -v -k "lifo or peek or empty_behaviour or
↪  balanced or infix"
```

Your `Stack` stands on your `DynamicArray`: `tests/test_dynamic_array.py` must pass first.

`__init__`, `is_empty`, `__len__` and `__repr__` are given. `is_balanced` and `infix_to_postfix` should use your `Stack` — not a Python list as a stack, which is what the storage rule forbids for working storage when a structure of yours is available. Their **result** — the list of output tokens from `infix_to_postfix` — is a Python list, because that is the interface the tests use.

Both algorithms are $O(n)$: each token is pushed at most once and popped at most once.

Homework 6 — before Lecture 07

1. **Implement** `dsa/stack.py` until the stack tests pass.
2. **Trace** `infix_to_postfix` on `8 - 3 - 2`, `2 * 3 + 4` and `( 1 + 2 ) * ( 3 - 4 ) / 5`, as tables. Then evaluate each result by hand with the postfix algorithm.
3. **Write** `evaluate_postfix` in `dsa/translation.py` (it only needs your `Stack`), and check it against your traces.
4. **Design.** Sketch an undo/redo system for a text editor with two stacks. What happens to the redo stack when the user types after undoing?

For item 3, `pytest tests/test_translation.py -v -k evaluate_postfix` runs just the postfix tests; the rest of that file belongs to Week 15.

Summary

Seven things to keep

1. A stack is **LIFO**: push, pop and peek act on the **top** only, all $O(1)$.
2. Empty pop and peek raise `IndexError`.
3. On an array the top is the **end**; on a linked list the **head**. Both honest.
4. Top at index 0 of an array: $O(n)$ per operation — same tests pass, wrong design.
5. **Balanced brackets**: push openers; each closer must match the top; empty at the end.
6. **Postfix**: push numbers; an operator pops two — **second popped is the left operand**.
7. **Shunting-yard**: operators wait on a stack and leave on **higher or equal** precedence, or at).

Next

Week 7 — Queues. First in, first out: the ADT that needs **both** ends — and the week your `DynamicArray` finally meets an operation it is bad at.

Sources and further reading

- **E. W. Dijkstra.** "Algol 60 translation: An Algol 60 translator for the X1 and making a translator for Algol 60", Mathematisch Centrum report MR 35, 1961 — where the shunting-yard algorithm was first described.
- **C. L. Hamblin.** "Translation to and from Polish notation", *The Computer Journal* 5(3), 1962 — reverse Polish notation and its evaluation with a stack.
- **M. T. Goodrich, R. Tamassia and M. H. Goldwasser.** *Data Structures and Algorithms in Python*, Wiley, 2013, chapter 6, "Stacks, Queues, and Deques" — array-based stacks, and bracket and HTML tag matching.
- **T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein.** *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, section 10.1, "Stacks and queues".

Every figure in this lecture is generated by `tools/figures_l06.py`; the call stack figure is Lecture 03's. The timing figure is real measurement and will differ slightly on your machine.

Questions for Week 6

> ***The answers are at the back of the book,*** *in* Answers — Week 6. *Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify.* `Stack` *is the class of* `dsa/stack.py`*:* `push`*,* `pop`*,* `peek`*,* `is_empty`*,* `len`*, with* `pop` *and* `peek` *raising* `IndexError` *on an empty stack. Operators in expressions are* `+ - * /` *with the usual precedence, all left associative.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W6-M01 – W6-M22
B	Short answer and essay	W6-E1 – W6-E5
C	Trace the code and the algorithms	W6-T1 – W6-T5
D	Find and fix the bug	W6-B1 – W6-B4

Part	Type	Questions
E	Write the code — checked by `pytest`	W6-C1 – W6-C5

Part A — Multiple choice

W6-M01 [what] "LIFO" means:

- **a)** the last item pushed is the first one popped
- **b)** the first item pushed is the first one popped
- **c)** items are popped in sorted order
- **d)** any item can be popped

W6-M02 [how] `push(1)`, `push(2)`, `push(3)`, `pop()`, `push(4)`, `pop()`. What does the **next** `pop()` return?

- **a)** 4
- **b)** 3
- **c)** 2
- **d)** 1

W6-M03 [why] A stack built on a dynamic array should keep its top at:

- **a)** index 0
- **b)** the end — the last used slot
- **c)** the middle
- **d)** it makes no difference

W6-M04 [why] A stack built on a singly linked list should keep its top at:

- **a)** the last node
- **b)** the middle node
- **c)** a separate array
- **d)** the head

W6-M05 [what] `Stack().pop()` on an empty stack:

- **a)** raises `IndexError`
- **b)** returns `None`
- **c)** returns 0
- **d)** blocks until something is pushed

W6-M06 [why] A stack on an array with its top at **index 0**. `push` costs:

- **a)** $O(1)$
- **b)** amortised $O(1)$
- **c)** $O(n)$ — every element shifts
- **d)** $O(\log n)$

W6-M07 [how] Why is `"([)]"` not balanced?

- **a)** It has an odd number of brackets
- **b)** `)` arrives when the most recent opener is `[`
- **c)** The stack is not empty at the end
- **d)** A closer arrives when the stack is empty

W6-M08 [how] Which check makes `is_balanced(")")` return `False`?

- **a)** the partner check
- **b)** the empty-at-the-end check
- **c)** a length check
- **d)** a closer arrives when the stack is empty

W6-M09 [how] The postfix form of `3 + 4 * 2` is:

- **a)** `3 4 2 * +`
- **b)** `3 4 + 2 *`
- **c)** `+ 3 * 4 2`
- **d)** `3 + 4 2 *`

W6-M10 [how] The postfix form of `( 3 + 4 ) * 2` is:

- **a)** `3 4 2 * +`
- **b)** `3 + 4 * 2`
- **c)** `3 4 + 2 *`
- **d)** `* + 3 4 2`

W6-M11 [how] Evaluating the postfix `8 3 -` gives:

- **a)** −5
- **b)** 5
- **c)** 11
- **d)** an error

W6-M12 [how] Evaluating the postfix `2 3 4 * +` gives:

- **a)** 20
- **b)** 9
- **c)** 24
- **d)** 14

W6-M13 [what] In the shunting-yard algorithm, when an operator arrives, it first pops to the output every stacked operator with:

- **a)** higher or equal precedence
- **b)** lower precedence
- **c)** any precedence
- **d)** strictly higher precedence only

W6-M14 [why] Why does an operator of **equal** precedence also leave the stack?

- **a)** It is faster
- **b)** To keep the stack small
- **c)** So that `-` and `/` are left associative: `8 - 3 - 2` is `(8 - 3) - 2`

- **d)** It makes no difference to the result

W6-M15 [how] The postfix form of `8 - 3 - 2` is:

- **a)** `8 3 2 - -`
- **b)** `8 3 - 2 -`
- **c)** `- - 8 3 2`
- **d)** `8 - 3 2 -`

W6-M16 [how] The worst-case extra space of `is_balanced` on a text of length n is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n^2)$
- **d)** $O(n)$

W6-M17 [why] To turn a recursive algorithm into a loop, in general you need:

- **a)** an explicit stack
- **b)** a queue
- **c)** a sorted array
- **d)** nothing — every recursion is already a loop

W6-M18 [what] In an undo/redo system with two stacks, what happens to the redo stack when the user performs a **new** action?

- **a)** the new action is pushed on it
- **b)** it is cleared
- **c)** its top is undone
- **d)** nothing

W6-M19 [how] n pushes onto the `Stack` of `dsa/stack.py` (built on a `DynamicArray`) cost in total:

- **a)** $O(n^2)$
- **b)** $O(n \log n)$
- **c)** $O(n)$ — amortised $O(1)$ each
- **d)** $O(1)$

W6-M20 [what] Which of these is **not** a natural stack application?

- **a)** checking balanced brackets
- **b)** evaluating postfix expressions
- **c)** undo in an editor
- **d)** serving customers in the order they arrived

W6-M21 [how] While `is_balanced("(((())))")` runs, the largest number of items on the stack at once is:

- **a)** 4
- **b)** 8
- **c)** 1
- **d)** 2

W6-M22 [what] `peek()` differs from `pop()` in that it:

- **a)** is slower
- **b)** returns the top without removing it
- **c)** returns the bottom
- **d)** never raises an error

Part B — Short answer and essay

W6-E1 [why] *(4 marks)* State the Stack ADT with the cost of each operation. Describe two honest implementations, compare them, and describe one design that passes every test but is not honest.

W6-E2 [how] *(4 marks)* Describe the stack algorithm for balanced brackets. Give an example of each of the three ways a text can be unbalanced, and the check that catches it.

W6-E3 [how] *(3 marks)* Explain how a stack evaluates a postfix expression. Why does the order in which the two operands are popped matter, and for which operators?

W6-E4 [why] *(4 marks)* Explain the shunting-yard algorithm, and trace it on `2 * ( 3 + 4 ) - 5`. Why must an operator of equal precedence be popped?

W6-E5 [why] *(3 marks)* "Recursion is a stack." Explain, and describe how you would rewrite a recursive algorithm without recursion. When is that worth doing in Python?

Part C — Trace the code and the algorithms

W6-T1 [how] What is printed?

```
s = Stack()
out = []
s.push(1); s.push(2)
out.append(s.pop())
s.push(3); s.push(4)
out.append(s.peek())
out.append(s.pop())
out.append(s.pop())
out.append(len(s))
print(out)
```

W6-T2 [how] Evaluate the postfix expression `5 1 2 + 4 * + 3 -`, showing the stack after every token.

W6-T3 [how] Convert `2 * ( 3 + 4 ) - 5` to postfix with the shunting-yard algorithm, showing the output and the operator stack after every token.

W6-T4 [how] Run `is_balanced("{[()()]}")` by hand. Show the stack after every character. What is the maximum stack size?

W6-T5 [why] In what order are the numbers printed? Why that order?

```
pending = Stack([[1, [2, 3], 4]])
while not pending.is_empty():
    item = pending.pop()
    if isinstance(item, list):
        for inner in item:
            pending.push(inner)
    else:
        print(item, end=" ")
```

Part D — Find and fix the bug

W6-B1 [how]

```
def pop(self):
    if len(self._items) == 0:
        return None
    return self._items.pop()
```

W6-B2 [how]

```
def is_balanced(text):
    pairs = {")": "(", "]": "[", "}": "{"}
    s = Stack()
    for ch in text:
        if ch in "([{":
            s.push(ch)
        elif ch in pairs:
            if s.is_empty() or s.pop() != pairs[ch]:
                return False
    return True
```

W6-B3 [how] Part of `evaluate_postfix`:

```
if token in "+-*/":
    a = stack.pop()
    b = stack.pop()
    stack.push(apply(token, a, b))      # apply(op, left, right)
```

W6-B4 [why] Part of `infix_to_postfix`:

```
while not ops.is_empty() and ops.peek() in prec and prec[ops.peek()] >
 ↪  prec[token]:
    output.append(ops.pop())
ops.push(token)
```

Part E — Write the code

In `practice/week06.py`; check with `pytest tests/test_practice_week06.py -v`. Use your `Stack` as working storage.

W6-C1 [how] `reverse_words(sentence)`.

W6-C2 [why] `MinStack` — `push`, `pop`, `peek` and `get_min`, all $O(1)$. Why is keeping a single "current minimum" variable not enough?

W6-C3 [why] `next_greater(values)` in $O(n)$. Why is the total work linear, even though there is a loop inside a loop?

W6-C4 [how] `tags_balanced(html)` — the brackets algorithm, for HTML tags.

W6-C5 [how] `decode(encoded)` — expand `k[text]`, which may nest.

Week 7 — Queues

> ***How to read this document.*** *This is the handout for Lecture 07. It holds everything on the slides, plus what I said out loud. A queue looks like a stack with one word changed, and that one word,* first *instead of* last*, is enough to break the obvious design. This week you measure that break yourself, then fix it with the ring buffer, the most useful trick in the course that fits in one line of arithmetic.*
>
> *Slides: `DSA27-L07-slides.pdf` · Code: `dsa/queue.py` · Tests: `tests/test_stack_queue.py` (the queue half)*

Where We Are

One end was easy. Now we need both.

Week 6: the **stack** touches **one** end. Put the top at the cheap end, done.

Today: the **queue** adds at one end and removes at the **other**.

- A dynamic array is cheap at the **end** only.
- A singly linked list is cheap at the **head** (and, with a tail, for adding at the end).

So which end goes where, and what happens to the expensive end?

Lecture 06 closed with a promise: this is the week your `DynamicArray` finally meets an operation it is bad at. A stack could hide the array's weak end simply by never using it. A queue cannot: whatever you do, one of its two operations lands on the front of the array.

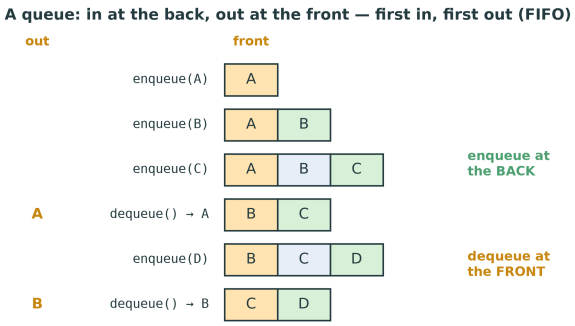
Queue in Arabic: الطابو.

Today

1. The Queue ADT: enqueue, dequeue — FIFO
2. The obvious design, `SlowQueue`, and its $O(n)$ dequeue
3. The ring buffer: `CircularQueue`, every operation $O(1)$
4. Full or empty? — and growing a ring without scrambling it
5. The linked queue, and a queue made of two stacks
6. Measured: the gap is the lesson
7. Where queues are used — and the deque

The Queue ADT

First in, first out



Operation	Meaning	Cost
`enqueue(x)`	add x at the **back**	$O(1)$
`dequeue()`	remove and return the **front** (`IndexError` if empty)	$O(1)$
`is_empty()`, `len(q)`		$O(1)$

That table is the ADT: operations, meaning, cost, and nothing about storage — exactly as with the stack. Many textbooks add a `peek` (also called `front` or `first`) that returns the front without removing it; the exercise does not need it, but it is one line once the rest works.

Everyday queues: a line at a bank or a bakery, where the first to arrive is the first served; a printer's job list; the messages waiting for a WhatsApp server to deliver them; the keys you press while a program is busy, which arrive in the order you pressed them. The rule is **fairness in order of arrival** — and that is exactly what a stack does not give you (W6-M20).

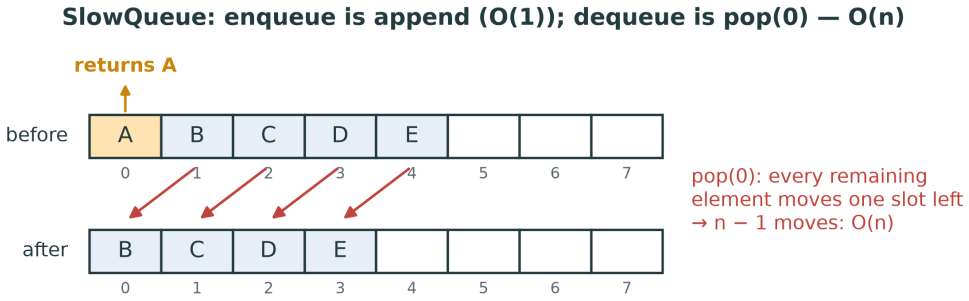
Empty queues. Dequeuing an empty queue raises `IndexError`, like popping an empty stack. `tests/test_stack_queue.py::test_dequeue_on_empty_raises` checks it for both classes.

Stack against queue, in one line each:

	Adds at	Removes at	Order out
Stack	the top	the **same** end	reversed (LIFO)
Queue	the back	the **other** end	preserved (FIFO)

The Obvious Design

`SlowQueue`: a `DynamicArray`, as it is



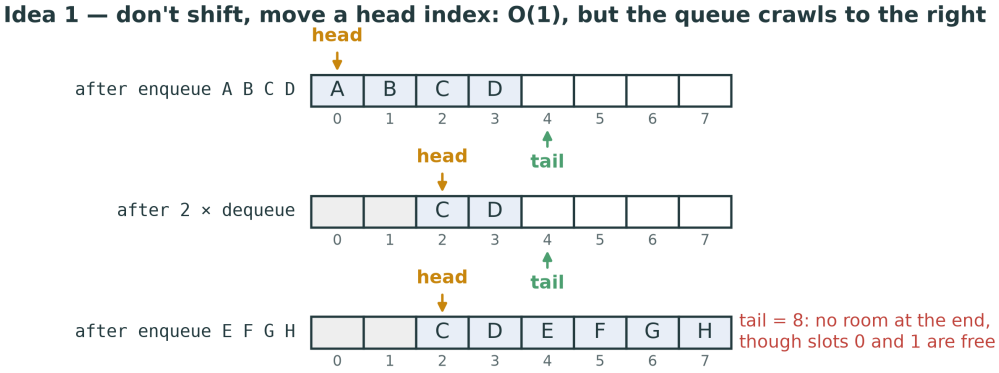
- enqueue = append at the end: amortised $O(1)$.
- dequeue = `pop(0)`: every remaining element shifts left — $O(n)$.

This is `SlowQueue` in `dsa/queue.py`, and it is a correct queue: it passes every test. The class is deliberately named for its flaw, because you are going to measure it.

Could we swap the ends? Put the back at index 0 and the front at the end: then dequeue is `pop()` at the end, $O(1)$, but enqueue becomes `insert_at(0, x)`, which shifts everything — $O(n)$ again. With a plain array, **whichever** end is the front, one of the two operations costs $O(n)$. That is the difference from the stack, which could simply put all its work at the cheap end.

Python's own `list` has the same problem: `list.pop(0)` is $O(n)$. That is why Lab 03 told you to use `collections.deque` for a queue, and promised you would measure the difference this week.

Idea 1: don't shift — move the front



Keep a `head` index. Dequeue reads `block[head]` and adds 1 to `head`: $O(1)$.

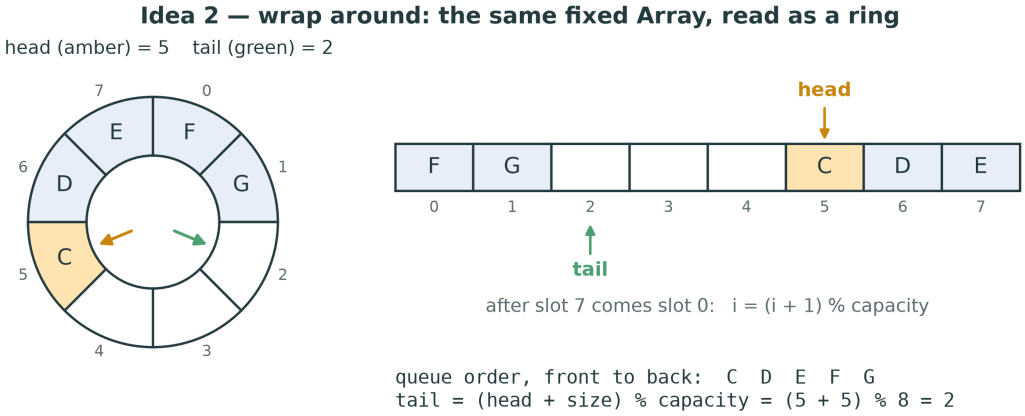
But the used part crawls right, and the slots it leaves behind are never used again.

Dequeue is now $O(1)$ — nothing moves. The price is that dequeued slots at the front are dead. In the figure, eight enqueues and two dequeues leave only six items in an eight-slot block, yet there is no room for a ninth at the end. A long-running queue with this design grows its block for ever, even if it never holds more than three items at a time — a memory leak by design.

You could compact now and then — shift everything back to index 0 when the dead space gets large — and that is a legitimate design with an amortised argument like Lecture 04's. But there is a neater answer that never moves anything.

The Ring Buffer

Idea 2: wrap around



After the last slot comes slot 0 again:

- **front** at head; **back** at `(head + size) % capacity`
- every step forward is `(i + 1) % capacity`

The block is still a plain, fixed `Array`. What changes is how we **read** it: index arithmetic *modulo the capacity* makes the end of the block join up with the beginning, so the slots freed at the front are reused as soon as the back reaches them. This is called a **ring buffer** or **circular buffer**, and it is everywhere in systems programming: keyboard buffers, network cards, audio streaming, logging the last N events. Wherever data arrives at one rate and leaves at another, there is probably a ring buffer between them.

`dsa/queue.py` stores three things besides the block: `_capacity`, `_head` (the index of the front item) and `_size` (how many items are in the queue). It **does not** store the tail; the tail is computed:

$$\text{tail} = (\text{head} + \text{size}) \bmod \text{capacity}$$

In the figure, head = 5 and size = 5 in a block of 8, so the tail — the slot the next enqueue writes to — is (5 + 5) mod 8 = 2. The queue, front to back, is C D E F G: slots 5, 6, 7, then round to 0 and 1. `__iter__` in `dsa/queue.py` is given to you and walks exactly this way: `(head + offset) % capacity` for each offset from 0 to size − 1.

The two operations

enqueue(x)

1. full? — see the next slides

2. write x at `(head + size) % capacity`
3. `size += 1`

dequeue()

1. empty? — `IndexError`
2. read the value at `head`; clear that slot
3. `head = (head + 1) % capacity`; `size -= 1`

Nothing moves. Every step is a little arithmetic: $O(1)$.

Those six steps are the whole exercise, apart from the challenge; turning them into Python is your job. Two details are worth saying out loud.

Why clear the slot? Setting the dequeued slot back to `None` is not needed for the queue to be correct: that slot is outside the live part of the ring and will be overwritten later. But until then the block would still hold a reference to the dequeued object, and Python's garbage collector could not free it. Clearing it is the same care as `DynamicArray.pop` setting the vacated slot to `None`.

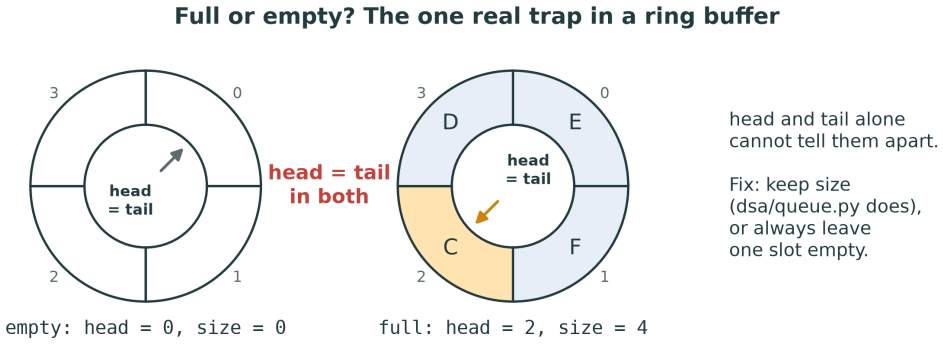
The modulo is the whole trick. Without it, `head + 1` would walk off the end of the `Array` and raise `IndexError`. `test_circular_queue_wraps_around` fills a ring of capacity 3, dequeues once and enqueues again: the new value must land in the slot at index 0, which only the `%` makes possible.

Trace: capacity 4

Operation	Block	head	size	Returns
start	_ _ _ _	0	0	
enqueue A, B, C	A B C _	0	3	
dequeue	_ B C _	1	2	A
dequeue	_ _ C _	2	1	B
enqueue D	_ _ C D	2	2	
enqueue E	E _ C D	2	3	← wrapped to slot 0
enqueue F	E F C D	2	4	← full
dequeue	E F _ D	3	3	C

Read the block as a ring from `head`: after "enqueue F" it is C D E F — slots 2, 3, 0, 1. E and F were written into the slots that A and B vacated; no element ever moved. This trace was produced by running a reference implementation, and every row can be checked with `tail = (head + size) % 4`: before "enqueue E", tail = (2 + 2) % 4 = 0.

Full or empty?



When `head == tail`, the ring is **either** empty **or** full. Head and tail alone cannot tell which.

Look at the start of the trace (head 0, size 0, tail 0) and after "enqueue F" (head 2, size 4, tail (2 + 4) % 4 = 2). In both, the next write would go to the slot `head` points at. A ring that stores only `head` and `tail` cannot tell these two situations apart. There are three standard fixes:

1. **Keep a count** — `_size`. Empty is `size == 0`, full is `size == capacity`. This is what `dsa/queue.py` does, and why `is_empty` and `is_full` are given to you as one-liners.
2. **Leave one slot unused.** Full is defined as `(tail + 1) % capacity == head`, so `head == tail` can only mean empty. Costs one slot; common in hardware and in C, where a count shared between two threads is awkward.
3. **A flag** set on the enqueue that makes `head == tail`, cleared on the next dequeue.

Every one of them works. The bug is to have none, which is a classic exam question (W7-B3).

When the ring is full

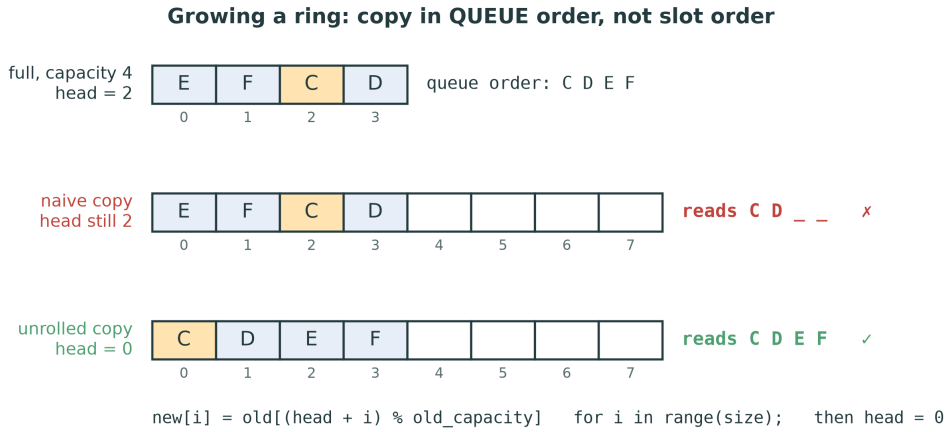
Two honest choices:

- **Refuse:** raise an error. A *bounded* queue — fixed memory, like a real buffer.
- **Grow** (the challenge): allocate a bigger `Array` and copy — as `DynamicArray` does, doubling so that enqueue stays amortised $O(1)$.

The docstring of `CircularQueue.enqueue` calls growing a **challenge**. The tests never overfill a ring, so raising `IndexError("queue is full")` is enough to pass them. But you should grow: Week 11's `level_order` and Week 14's `bfs` use your `CircularQueue`, and neither knows in advance how many items it will hold.

Both choices are used in practice. An audio driver's buffer is bounded on purpose: if the consumer falls behind, old data is dropped or the producer waits, but memory never grows. A general-purpose queue, like Python's deque, grows.

Growing: the trap



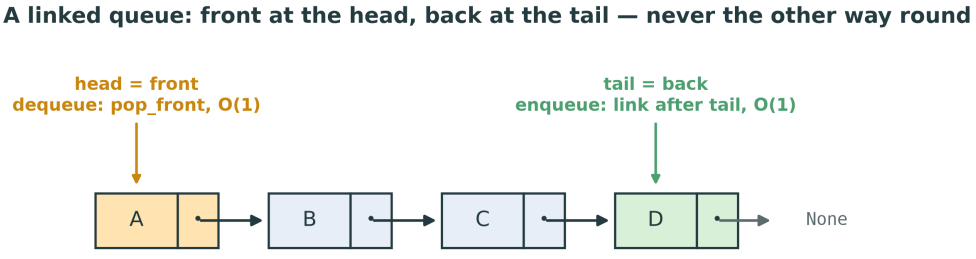
Copying slot by slot keeps the wrapped order — so the queue reads **wrong**. Copy in **queue order**, and restart head at 0.

This is the trap the docstring warns about ("a naive copy scrambles the order once the data has wrapped around"). In the figure, the full ring of capacity 4 holds C D E F starting at slot 2. Copying `new[i] = old[i]` puts E and F at slots 0 and 1 of the new block and C D at 2 and 3; with head still 2 and a capacity of 8, the queue now reads C, D, then two empty slots — E and F are lost behind the front. If the ring has not wrapped yet (head = 0), the naive copy happens to work, which is exactly why this bug survives casual testing.

The fix **unrolls** the ring while copying: the i-th item of the queue, which lives at `(head + i) % old_capacity`, goes to slot i of the new block, for i from 0 to size − 1. Then `head` becomes 0, and `capacity` the new capacity (the last is easy to forget, and then every `%` still wraps at the old size). The front goes to slot 0, and the rest follow in order. Doubling the capacity each time gives the same amortised $O(1)$ enqueue as Lecture 04: over n enqueues, the copies total less than 2n.

Two More Honest Queues

The linked queue



Front at the **head**, back at the **tail**: both operations $O(1)$ worst case.

A singly linked list with a `tail` reference can add at either end in $O(1)$, but can remove only at the head in $O(1)$: removing the last node needs the node **before** it, which takes a walk (Lecture

05, W5-K3). So the queue's front must be the head (where we remove) and its back the tail (where we add). The other way round, every dequeue is $O(n)$.

- **enqueue:** make a node. If the queue is empty, it becomes both `head` and `tail`; otherwise link it after `tail`, and it becomes the new `tail`.
- **dequeue:** `IndexError` if empty; otherwise take the head's value and move `head` to the next node.

The one trap is the end of dequeue: when the queue becomes empty, `tail` must be reset to `None` too, or it keeps pointing at a node that is no longer in the queue and the next enqueue links onto it (W7-B4).

A queue from two stacks

- `inbox`: enqueue pushes here.
- `outbox`: dequeue pops here. When `outbox` is **empty**, first pour **all** of `inbox` into it.

Pouring reverses the order — and LIFO reversed is FIFO.

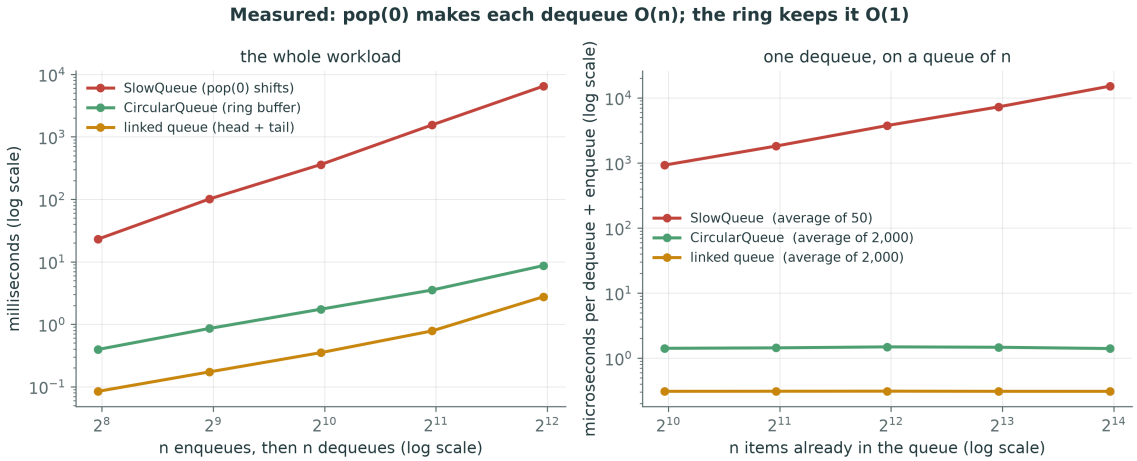
One `dequeue` may pour n items, so a single operation can cost $O(n)$. But each item is pushed onto `inbox` once, moved to `outbox` once, and popped from `outbox` once: three $O(1)$ steps over its whole life. So n operations cost $O(n)$ in total, and each is **amortised** $O(1)$ — the aggregate argument of Lecture 04 again, and of `next_greater` in W6-C3.

The rule "pour only when `outbox` is empty" is essential. Pouring on every dequeue is still correct, but each dequeue then moves everything twice, $O(n)$ each (W7-K3). It is W7-C1 in the question bank.

Why would anyone build a queue this way? In languages where the standard structures are **immutable**, as in Haskell or Erlang, a stack is a linked list that can be shared safely, and two of them make the standard functional queue. It is also a favourite interview question, and a good test of whether you understand amortised cost.

Measured

The gap is the lesson



Real timings of `dsa/queue.py` — the instructor's reference solution, built on the course `Array` exactly as yours will be — against a simple linked queue. The ring is created with room for all n items, so growing plays no part. Left: n enqueues followed by n dequeues. The two honest queues rise with slope 1 on the log–log plot, $O(n)$ for n operations; the `SlowQueue` rises with slope 2, $O(n^2)$. At n = 4,000 that was about 6.5 seconds against 9 milliseconds for the ring: some 700 times slower, and the gap doubles every time n doubles.

Right: the cost of **one** dequeue (paired with one enqueue, to keep the size at n) on a queue already holding n items. The ring and the linked queue are flat lines: $O(1)$, however long the queue. The `SlowQueue` line is a straight line of slope 1: $O(n)$ per dequeue, about 15 milliseconds at n = 16,000 against roughly 1.4 microseconds for the ring. (The $O(1)$ lines are averages over 2,000 operations, because one operation is too quick to time on its own.)

Constant factors. The linked queue beats the ring by four to five times here, because every ring operation does a `%` and goes through the Python-level bounds check of `dsa/array.py`, while a node's `next` is a plain attribute. The same effect as in Lecture 06. Big-O says both are $O(1)$; measurement says how big the 1 is.

This is the demonstration the course plan promised in Week 7: the same tests pass for both classes, and one of them is quadratic. Plot your own with the `07-queues` notebook once `dsa/queue.py` works.

Four queues, compared

	enqueue	dequeue	memory
`SlowQueue` — array, `pop(0)`	amortised $O(1)$	$O(n)$	compact
Array, head index, no wrap	amortised $O(1)$	$O(1)$	**dead slots** grow for ever
`CircularQueue` — ring buffer	$O(1)$ (amortised if it grows)	$O(1)$	compact, reuses slots
Linked, head + tail	$O(1)$	$O(1)$	a node per item

	enqueue	dequeue	memory
Two stacks	amortised $O(1)$	amortised $O(1)$	two arrays

The ring buffer is the usual choice when the maximum size is known or growth is rare: it is contiguous, it allocates nothing per item, and it never moves data except while growing. The linked queue has the best worst case, one allocation per item. Python's `collections.deque` combines the two: a doubly linked list of fixed blocks of 64 slots, so it allocates rarely, like an array, and never shifts, like a list of nodes.

Where Queues Are Used

FIFO is fairness — and breadth

- **Scheduling:** print jobs, requests to a web server, processes sharing a CPU (round-robin).
- **Buffers:** keyboard input, network packets, audio — ring buffers between a producer and a consumer; "the last k values" for a moving average.
- **Breadth-first search** (Week 14): visit everything 1 step away, then 2 steps, … — the queue keeps the frontier in order.
- **Level-order traversal** of a tree (Week 11): the same idea on a tree.

Round-robin. An operating system gives each ready process a short time slice. A process whose slice runs out goes to the back of the ready queue; the next one is dequeued from the front. Everyone gets a turn, in order. The same dequeue-then-enqueue step solves the Josephus problem — people in a circle, every k-th one leaves — which is W7-C2: put everyone in a queue, then move k − 1 people from the front to the back and remove the k-th, until one is left.

Breadth-first order. A queue processes things in the order they were *discovered*. Start from one node, enqueue its neighbours, then theirs: everything one step away is dequeued before anything two steps away. That is why BFS finds shortest paths (by number of edges), and why `level_order` in `dsa/tree.py` visits a tree row by row. Replace the queue with a stack and the same loop becomes depth-first — Week 14 puts them side by side.

A small example you can run now: the binary numbers 1 to n, in order, generated breadth-first (W7-C5). Start with "1"; each time you dequeue a string s, enqueue s + "0" and s + "1".

The last k values. A `CircularQueue(k)` that dequeues the oldest value whenever a new one arrives always holds the latest k: the window of a moving average, the last k log lines, the last k readings of a sensor. Keep a running total beside it and each new average costs $O(1)$ instead of $O(k)$ — W7-C3.

The deque: both ends, both ways

A **deque** (double-ended queue, say "deck"): add and remove at **both** ends, all $O(1)$.

- only one end used → a **stack**
- add at one end, remove at the other → a **queue**

In Python: `collections.deque` — `append`, `appendleft`, `pop`, `popleft`.

The name *deque* was coined by E. J. Schweppe, according to Knuth (*The Art of Computer Programming*, vol. 1, §2.2.1). A ring buffer makes a deque with no new ideas: adding at the front means moving head **back** one slot, `head = (head - 1) % capacity`. Python's `%` always returns a non-negative result for a positive capacity, so `(0 - 1) % 8` is 7: the step back from slot 0 wraps round to the last slot, exactly as needed. (In C and Java, `-1 % 8` is −1, and you must write `(head - 1 + capacity) % capacity` instead.) W7-S3 draws a deque on a ring, step by step.

Where next. Week 12 builds a **priority queue**: a queue that serves the most **urgent** item rather than the oldest one. The ADT has the same two operations, but a completely different structure underneath, the heap. Same name, different contract.

This Week

Exercises: `dsa/queue.py`

Method	Target	The trap
`SlowQueue.enqueue`	amortised $O(1)$	`append` on `self._items`
`SlowQueue.dequeue`	$O(n)$ — on purpose	`IndexError` when empty; `pop(0)`
`CircularQueue.enqueue`	$O(1)$	tail = `(head + size) % capacity`; full?
`CircularQueue.dequeue`	$O(1)$	`IndexError` when empty; head wraps
challenge: grow when full	amortised $O(1)$	copy in queue order; head = 0

```
pytest tests/test_stack_queue.py -v -k "fifo or dequeue or circular"
```

`SlowQueue` stands on your `DynamicArray`, so `tests/test_dynamic_array.py` must pass first. `CircularQueue` stands directly on the course `Array`: its `__init__`, `is_empty`, `is_full`, `__len__`, `__iter__` and `__repr__` are given to you — read `__iter__` first, since it already contains the ring arithmetic.

Four tests cover the queues: FIFO order and the empty `IndexError` for both classes, a wrap-around at capacity 3, and ten interleaved operations that take a capacity-4 ring round twice.

Homework 7 — before Lecture 08

1. **Implement** `dsa/queue.py` until the queue tests pass — then the challenge: grow instead of raising, and test it on a ring that has wrapped.
2. **Measure** `SlowQueue` against `CircularQueue` in `notebooks/07-queues.ipynb` and plot both. Estimate how long the `SlowQueue` would take for n = 10^6.
3. **Trace** a `CircularQueue(capacity=3)` through: enqueue 1, 2, 3; dequeue; enqueue 4; dequeue; dequeue; enqueue 5, 6. Show the block, head and size after each step.
4. **Design.** A queue from two stacks: write it, and explain why it is amortised $O(1)$ (W7-C1).

For item 2, a sensible range is n = 500 to 8,000; beyond that the `SlowQueue` takes minutes. Fit the slope of your log–log line and extrapolate: slope 2 means 10 times more items cost 100 times more time.

For item 3, check your final answer with the given `__iter__`: `list(q)` should read the queue front to back.

Summary

Seven things to keep

1. A queue is **FIFO**: enqueue at the **back**, dequeue at the **front**, both $O(1)$.
2. Empty dequeue raises **`IndexError`**.
3. On a plain array, one end is always the front: `pop(0)` or `insert(0)` — $O(n)$.
4. The **ring buffer**: head moves, nothing shifts; tail = `(head + size) % capacity`.
5. `head == tail` means **full or empty**: keep `size` (or waste one slot).
6. Growing a ring: copy in **queue order**, then `head = 0`.
7. Linked queue: front at the **head**, back at the **tail**. Two stacks: **amortised** $O(1)$.

Next

Week 8 — Searching *(midterm window)*. Seven ways to find a value, and what a **sorted** array buys you: from $O(n)$ to $O(\log n)$.

Sources and further reading

- **D. E. Knuth.** *The Art of Computer Programming*, vol. 1, *Fundamental Algorithms*, 3rd ed., Addison-Wesley, 1997, §2.2.1 "Stacks, queues, and deques" and §2.2.2 "Sequential allocation" — queues in a circular block, and the origin of the word *deque*.
- **M. T. Goodrich, R. Tamassia and M. H. Goldwasser.** *Data Structures and Algorithms in Python*, Wiley, 2013, chapter 6, "Stacks, Queues, and Deques" — an array-based circular queue that grows, written in Python.
- **T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein.** *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, section 10.1, "Stacks and queues", and exercise 10.1-6, a queue from two stacks.
- **CPython source.** `Modules/_collectionsmodule.c` — deque as a doubly linked list of 64-slot blocks.

Every figure in this lecture is generated by `tools/figures_l07.py`. The timing figure is real measurement and will differ slightly on your machine.

Questions for Week 7

> ***The answers are at the back of the book,*** *in* Answers — Week 7. *Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify.* `SlowQueue` *and* `CircularQueue` *are the classes of*

> *`dsa/queue.py`: `enqueue`, `dequeue` (raising `IndexError` when empty) and `len`; `CircularQueue` also has `is_empty`, `is_full` and iteration front to back. A `CircularQueue` stores its items in a fixed `Array` with `head`, `size` and `capacity`, and the next enqueue writes to slot `(head + size) % capacity`. `Stack` is Week 6's.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W7-M01 – W7-M22
B	Short answer and essay	W7-E1 – W7-E5
C	Trace the code	W7-T1 – W7-T5
D	Ring state — draw every step	W7-S1 – W7-S3
E	Complexity analysis	W7-K1 – W7-K3
F	Find and fix the bug	W7-B1 – W7-B4
G	Write the code — checked by `pytest`	W7-C1 – W7-C5

Part A — Multiple choice

W7-M01 [what] "FIFO" means:

- **a)** the last item added is the first one removed
- **b)** the first item added is the first one removed
- **c)** items are removed in sorted order
- **d)** any item can be removed

W7-M02 [how] `enqueue(1)`, `enqueue(2)`, `enqueue(3)`, `dequeue()`, `enqueue(4)`, `dequeue()`. What does the **next** `dequeue()` return?

- **a)** 4
- **b)** 1
- **c)** 2
- **d)** 3

W7-M03 [why] In `SlowQueue`, `dequeue` is `pop(0)` on a `DynamicArray`. Its cost on a queue of n items is:

- **a)** $O(n)$ — every remaining element shifts one slot left
- **b)** $O(1)$
- **c)** amortised $O(1)$
- **d)** $O(\log n)$

W7-M04 [why] An array-based queue that enqueues with `insert_at(0, x)` and dequeues with `pop()` at the end has costs:

- **a)** both $O(1)$
- **b)** enqueue $O(n)$, dequeue $O(1)$
- **c)** enqueue $O(1)$, dequeue $O(n)$
- **d)** both $O(n)$

W7-M05 [how] A `CircularQueue` with capacity 8 has head = 6 and size = 3. The next enqueue writes to slot:

- **a)** 9
- **b)** 0
- **c)** 1
- **d)** 7

W7-M06 [how] A `CircularQueue` with capacity 5 has head = 4 and size = 2. After one dequeue, head is:

- **a)** 5
- **b)** 3
- **c)** 1
- **d)** 0

W7-M07 [what] A ring buffer that stores only `head` and `tail` (no size) finds `head == tail`. The queue is:

- **a)** full
- **b)** empty
- **c)** either full or empty — it cannot tell which
- **d)** corrupted

W7-M08 [why] `CircularQueue` keeps a `_size` field because:

- **a)** it tells a full ring from an empty one, which look the same by head and tail
- **b)** `len` would otherwise be $O(n)$ with no other benefit
- **c)** the `Array` does not know its own length
- **d)** Python requires it for `__iter__`

W7-M09 [how] A full ring of capacity 4 holds `[E, F, C, D]` with head = 2. It grows to capacity 8 by copying `new[i] = old[i]` and keeping head = 2. Read front to back, the queue is now:

- **a)** C D E F
- **b)** C D None None
- **c)** E F C D
- **d)** D C F E

W7-M10 [what] A queue on a singly linked list with `head` and `tail` references should put the front of the queue at:

- **a)** the tail
- **b)** the middle node
- **c)** either end — it makes no difference
- **d)** the head

W7-M11 [why] Why not dequeue at the **tail** of a singly linked list?

- **a)** Removing the last node needs the node before it, which takes an $O(n)$ walk
- **b)** The tail node has no value
- **c)** It would make the queue LIFO

- **d)** Python forbids it

W7-M12 [how] A queue built from two stacks (`inbox` and `outbox`, pouring only when `outbox` is empty): `enqueue(1)`, `enqueue(2)`, `enqueue(3)`, `dequeue()`. How many items are now in `outbox`?

- **a)** 0
- **b)** 1
- **c)** 2
- **d)** 3

W7-M13 [why] The cost of `dequeue` on that two-stack queue is:

- **a)** $O(n)$ on average
- **b)** $O(\log n)$
- **c)** $O(1)$ in the worst case
- **d)** amortised $O(1)$ — each item is poured from `inbox` to `outbox` at most once

W7-M14 [what] Which of these needs a **queue**?

- **a)** checking balanced brackets
- **b)** undo in an editor
- **c)** breadth-first search of a graph
- **d)** evaluating a postfix expression

W7-M15 [how] n enqueues followed by n dequeues on a `SlowQueue` cost in total:

- **a)** $\Theta(n^2)$
- **b)** $\Theta(n)$
- **c)** $\Theta(n \log n)$
- **d)** $\Theta(1)$

W7-M16 [how] In Python, `(0 - 1) % 8` is:

- **a)** −1
- **b)** 7
- **c)** 0
- **d)** an error

W7-M17 [what] A **deque** is a structure that:

- **a)** serves the most urgent item first
- **b)** can only be used as a stack
- **c)** removes items in sorted order
- **d)** adds and removes at both ends

W7-M18 [why] Measured, a ring-buffer dequeue and a linked-queue dequeue are both flat lines as n grows, but the ring is about five times slower. The reason:

- **a)** a constant factor: the `%` and the bounds-checked `Array` access on every operation
- **b)** the ring is really $O(\log n)$
- **c)** the ring copies the queue on every dequeue
- **d)** the linked queue is $O(1)$ and the ring is $O(n)$

W7-M19 [how] Five people A, B, C, D, E stand in a circle; counting from A, every **second** person leaves. Who is left last?

- **a)** A
- **b)** E
- **c)** C
- **d)** D

W7-M20 [how] An array queue with a head index but **no** wrap-around, 8 slots: 8 enqueues, then 5 dequeues. How many more enqueues fit without growing the block?

- **a)** 5
- **b)** 0
- **c)** 3
- **d)** 8

W7-M21 [why] A `CircularQueue` that doubles its capacity when full has an enqueue cost of:

- **a)** $O(n)$ every time
- **b)** $O(\log n)$
- **c)** $O(1)$ in the worst case
- **d)** amortised $O(1)$

W7-M22 [what] A **priority queue** (Week 12) differs from a queue because it dequeues:

- **a)** the most urgent item, not the oldest
- **b)** the newest item
- **c)** a random item
- **d)** from both ends

Part B — Short answer and essay

W7-E1 [why] *(4 marks)* State the Queue ADT with the cost of each operation. Explain why an array-based queue that shifts is $O(n)$ per dequeue, and why swapping which end is the front does not help. How does a ring buffer fix it?

W7-E2 [how] *(4 marks)* In a ring buffer, `head == tail` can mean two different things. Explain which two, with an example of each, and describe two ways of telling them apart. Which one does `dsa/queue.py` use?

W7-E3 [why] *(4 marks)* A full `CircularQueue` must grow. Explain, with an example, why copying the block slot by slot gives a wrong queue once the data has wrapped, give the correct copy, and explain why doubling keeps enqueue amortised $O(1)$.

W7-E4 [why] *(3 marks)* Describe a queue built from two stacks. Show that it is amortised $O(1)$ per operation, and explain why pouring must happen only when the `outbox` is empty.

W7-E5 [why] *(3 marks)* Compare the ring buffer and the linked queue (with head and tail): cost per operation, memory, and when you would choose each. Where does Python's `collections.deque` fit?

Part C — Trace the code

W7-T1 [how] What is printed? Then say which slot of the block holds each value at the end.

```
q = CircularQueue(capacity=4)
out = []
for v in (1, 2, 3):
    q.enqueue(v)
out.append(q.dequeue())
q.enqueue(4)
q.enqueue(5)
out.append(q.dequeue())
q.enqueue(6)
out.append(q.is_full())
out.append(list(q))
print(out)
```

W7-T2 [how] A two-stack queue (W7-C1) runs: `enqueue(1)`, `enqueue(2)`, `enqueue(3)`, `dequeue()`, `enqueue(4)`, `dequeue()`, `dequeue()`, `enqueue(5)`, `dequeue()`, `dequeue()`. Show both stacks (bottom to top) after every operation, and what each dequeue returns.

W7-T3 [how] Six people A–F stand in a circle; counting from A, every **third** person leaves (W7-C2). Show the queue, front to back, after each person leaves. In what order do they leave, and who is last?

W7-T4 [how] `binary_numbers(6)` (W7-C5) starts with the queue `["1"]`. Show the queue after each dequeue (and its two enqueues). What is returned?

W7-T5 [why] What is printed? Why are the two halves of the output in different orders?

```
def mystery(values):
    q, s = CircularQueue(len(values)), Stack()
    for v in values:
        q.enqueue(v)
    while len(q) > 0:
        v = q.dequeue()
        if v % 2 == 0:
            s.push(v)
        else:
            print(v, end=" ")
    while not s.is_empty():
        print(s.pop(), end=" ")

mystery([1, 2, 3, 4, 5, 6])
```

Part D — Ring state

W7-S1 [how] Draw the block, `head` and `size` after each operation on `q = CircularQueue(capacity=5)`. What is `list(q)` at the end, and is the queue full?

```
enqueue 10, enqueue 20, enqueue 30, enqueue 40, dequeue, dequeue,
enqueue 50, enqueue 60, enqueue 70, dequeue
```

W7-S2 [why] A student computes the tail as `self._head + self._size`, without `% self._capacity`. Draw what happens in `test_circular_queue_wraps_around`: capacity 3, enqueue 1, 2, 3, dequeue, enqueue 4. Why did the tests of earlier operations not catch it?

W7-S3 [how] A deque on a ring buffer, like `CircularQueue`, keeps `head` and `size`. `add_front` first steps `head` back one slot, `(head - 1) % capacity`, then writes there; `add_back` writes at `(head + size) % capacity`; `remove_back` takes the slot `(head + size - 1) % capacity`. Starting from an empty deque of capacity 4, draw the block, `head` and `size` after each operation, and the deque front to back:

```
add_back 1, add_back 2, add_front 0, add_front 9, remove_back,
add_back 5, remove_front, add_front 7, add_back 8
```

The last operation finds the deque full: it doubles the capacity first.

Part E — Complexity analysis

W7-K1 [how] Give Θ and justify:

```python
q = SlowQueue()
for i in range(n):
    q.enqueue(i)
while len(q) > 0:
    q.dequeue()
```

What is it with a `CircularQueue(n)` instead?

W7-K2 [how] Give Θ and justify:

```python
q = CircularQueue(n)
for i in range(n):
    q.enqueue(i)
    print(sum(q))                # sum iterates over the whole queue
```

W7-K3 [why] A student's two-stack queue pours `inbox` into `outbox` on **every** dequeue, and pours it back afterwards. Give the cost of n enqueues followed by n dequeues, and compare with the correct version.

Part F — Find and fix the bug

W7-B1 [how] `SlowQueue`:

```
def dequeue(self):
    if len(self._items) == 0:
        raise IndexError("dequeue from empty queue")
    return self._items.pop()
```

W7-B2 [why] `CircularQueue` — this passes `test_circular_queue_wraps_around`:

```
def dequeue(self):
    if self._size == 0:
        raise IndexError("dequeue from empty queue")
    value = self._block[self._head]
    self._block[self._head] = None
    self._head += 1
    self._size -= 1
    return value
```

W7-B3 [how] `CircularQueue`:

```
def enqueue(self, value):
    tail = (self._head + self._size) % self._capacity
    if tail == self._head:
        raise IndexError("queue is full")
    self._block[tail] = value
    self._size += 1
```

W7-B4 [why] A linked queue with head and `tail`:

```
def dequeue(self):
    if self.head is None:
        raise IndexError("dequeue from empty queue")
    value = self.head.value
    self.head = self.head.next
    return value
```

Part G — Write the code

In `practice/week07.py`; check with `pytest tests/test_practice_week07.py -v`. Use your own `CircularQueue`, `Stack` or the course `Array` as working storage.

W7-C1 [why] `StackQueue` — a queue from two stacks. Why is it amortised $O(1)$?

W7-C2 [how] `josephus(names, k)` — the order in which people leave the circle.

W7-C3 [why] `moving_averages(values, k)` — the average of every window of k values, with

a `CircularQueue` as the window. Why is it $O(n)$ and not $O(nk)$?

W7-C4 [how] `reverse_first_k(values, k)` — with one queue and one stack.

W7-C5 [how] `binary_numbers(n)` — the binary strings 1 to n, generated breadth-first.

Week 8 — Searching

> ***How to read this document.*** *This is the handout for Lecture 08. It holds everything on the slides, plus what I said out loud. Eight functions, one idea: an array that is kept* ***sorted*** *can be searched without looking at most of it. This week is also the midterm window, so the handout ends with a map of what weeks 1–7 expect of you.*
>
> *Slides:* `DSA27-L08-slides.pdf` *· Code:* `dsa/searching.py` *· Tests:* `tests/test_searching.py`

Where We Are

From structures to algorithms

Weeks 4–7 built **structures**: dynamic array, linked list, stack, queue.

Weeks 8–10 are about **algorithms** on the simplest of them, the array:

- **Week 8:** find a value — **searching**
- **Weeks 9–10:** put the values in order — **sorting**

Today's question: **what does it buy you to keep an array sorted?**

Every structure so far had a `find` or an `in`, and every one of them was $O(n)$: walk until you see the value. This week shows that the walk is unavoidable only when you know **nothing** about where the value might be. An array whose values are in increasing order tells you a great deal, and each search below uses that knowledge in a different way.

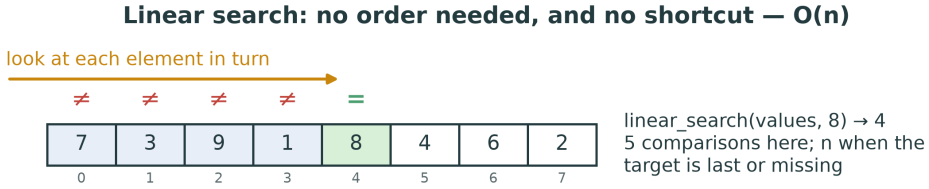
Search in Arabic: ﷺ. *Binary search*: البحث الثنائي.

Today

1. Linear search: the baseline
2. Binary search: halving — and why it is so easy to get wrong
3. `lower_bound` and `upper_bound`: positions, duplicates, insertion
4. Jump, exponential and interpolation search
5. Measured: eight functions, one picture
6. When is sorting first worth it?

Linear Search

Look at everything



- Works on **any** sequence, sorted or not.
- Found at index i: i + 1 comparisons. Missing: **n**. Worst case $O(n)$.

`linear_search` is the one function in `dsa/searching.py` that must work on **unsorted** data (`test_linear_search_works_on_unsorted_data`). Return the index of the **first** match, or –1 when there is none.

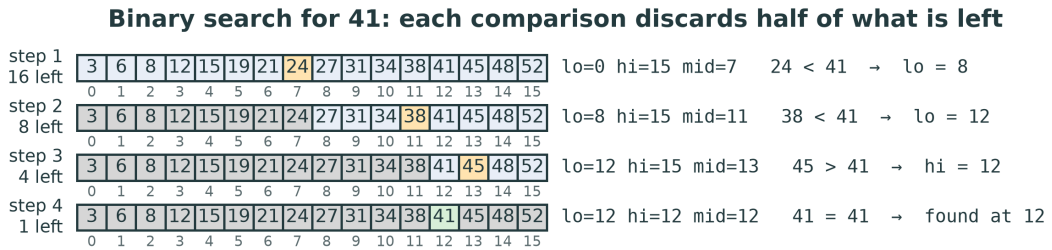
Best, worst, average. Best case: the target is first, 1 comparison. Worst case: it is last or missing, n comparisons. Average, if the target is present and equally likely to be at any position: $(1+2+\cdots+n)/n = (n+1)/2$ comparisons — still $\Theta(n)$. Halving the constant does not change the class (Lecture 02).

On sorted data a linear search can stop early, as soon as it passes a value larger than the target: a miss then costs, on average, half the array instead of all of it. Still $O(n)$. To do fundamentally better, you must stop looking at the elements **in order**.

Linear search is not a bad algorithm. For a few dozen elements it is often the fastest search there is; it needs no sorting, no extra memory, and it is what Python's `in` and `list.index` do.

Binary Search

Halve what is left



Compare the target with the **middle** element. Equal: found. Smaller: it can only be on the left. Larger: only on the right.

This is how you look up a word in a paper dictionary: open it in the middle, see whether your word comes before or after, and throw away the half it cannot be in. Each comparison halves the part still in play. In the figure, 16 values become 8, then 4, then 1: **four** comparisons find 41, where a linear search would need 13.

It works **only** because the values are sorted. On unsorted data the comparison with the middle tells you nothing about either half, and binary search returns wrong answers without any error. Nothing checks that the input is sorted — checking would itself cost $O(n)$ and defeat the purpose. The sorted order is a **precondition**: the caller's promise.

The algorithm, and the promise it keeps

Keep two indices, `lo` and `hi`: the part still in play is `values[lo..hi]`, both ends included.

1. `lo = 0`, `hi = n - 1`
2. while `lo <= hi`: `mid = (lo + hi) // 2`
   - `values[mid] == target` → return `mid`
   - `values[mid] < target` → `lo = mid + 1`
   - otherwise → `hi = mid - 1`
3. return −1

Invariant: if the target is anywhere, it is in `values[lo..hi]`.

An **invariant** is a statement that is true before the loop, stays true after every step, and so is still true when the loop ends. Here: "if the target is in the array at all, it is between `lo` and `hi`". At the start the range is the whole array, so it is true. Each step discards only elements that cannot be the target — everything up to `mid` when `values[mid]` is too small, everything from `mid` when it is too large — so it stays true. When `lo > hi`, the range is empty, and the invariant tells you the target is nowhere: return −1.

This is the whole of `binary_search`. The steps are written out here in words on purpose: turning them into Python, and getting every `+ 1` and `<=` right, is the exercise.

A miss, traced. Searching the same 16 values for 20:

lo	hi	mid	values[mid]	Decision
0	15	7	24	24 > 20 → hi = 6
0	6	3	12	12 < 20 → lo = 4
4	6	5	19	19 < 20 → lo = 6
6	6	6	21	21 > 20 → hi = 5
6	5			lo > hi: return −1

Four comparisons, and the loop ends with `lo` = 6: the position where 20 **would be** inserted. Remember that; it is the idea behind `lower_bound`.

How many comparisons?

Each comparison halves the range: after k comparisons at most $n/2^k$ remain.

n	at most $\lfloor \log_2 n \rfloor + 1$ comparisons
1,000	10
1,000,000	20

n	at most $\lfloor \log_2 n \rfloor + 1$ comparisons
1,000,000,000	30

Binary search is $O(\log n)$ — a thousand times more data costs 10 more comparisons.

Why $\lfloor \log_2 n \rfloor + 1$: the range starts with n elements, and a comparison that does not find the target leaves at most half of them (rounded down). After $\lfloor \log_2 n \rfloor$ such halvings one element is left, and one more comparison settles it. For the 16 values above, $\log_2 16 = 4$, so at most 5: searching for 52, the last value, takes exactly 5.

The recurrence, in the language of Lecture 03: $T(n) = T(n/2) + O(1)$, which solves to $O(\log n)$. Space: $O(1)$ for the loop — just `lo`, `hi` and `mid`.

`test_binary_search_does_not_scan_linearly` searches a list of a million integers. A correct binary search reads 20 of them. An accidental linear scan somewhere in your function reads a million and the test becomes slow — which is the point of the test.

Easy to state, hard to get right

- `while lo < hi` with an inclusive `hi` → **misses** the last candidate.
- `hi = mid` with an inclusive `hi` → can **loop for ever**.
- `lo = mid` → can **loop for ever** when `hi = lo + 1`.
- `mid = (lo + hi) / 2` in Java or C → **overflows** when `lo + hi` exceeds the largest `int`.

Binary search has a famous history of being wrong. Knuth records that the first binary search was published in 1946, and the first that works correctly for **every** n not until 1962 (*TAOCP* vol. 3, §6.2.1). Jon Bentley gave the problem to professional programmers in courses at Bell Labs and IBM, with two hours to write it: about ninety percent of the programs had bugs (*Programming Pearls*, column 4). And in 2006 Joshua Bloch reported that the binary search in Java's own library — `java.util.Arrays.binarySearch`, which he had written himself — failed on arrays of more than about a billion elements, because `(low + high) / 2` overflowed a 32-bit `int`. The fix is `low + (high - low) / 2`. The bug had been in the library for nine years.

Python integers do not overflow, so the last bug cannot happen here — but you will write binary search in C or Java one day, so name it now.

The other three bugs have one cure: **decide what `lo` and `hi` mean, and keep every line faithful to it.** With `hi` inclusive (the range is `values[lo..hi]`), the loop runs while the range is non-empty, `lo <= hi`, and both updates must move past `mid`, because `mid` has just been ruled out. The infinite loops happen when an update does not move: with `lo = 5`, `hi = 6`, `mid` is 5, and `lo = mid` leaves everything as it was. Test the sizes 0, 1 and 2 by hand: nearly every binary-search bug shows up on one of them. The tests check the empty list and one element (`test_handles_empty_and_single_element`), and the first and last positions (`test_finds_first_and_last_element`).

Recursive binary search

Same idea: search the half that can hold the target, **by a call**.

- $O(\log n)$ time, but also $O(\log n)$ **stack** — the loop is $O(1)$ space.
- Pass `lo` and `hi` down. **Never slice**: `values[mid + 1:]` copies — $O(n)$ per call.

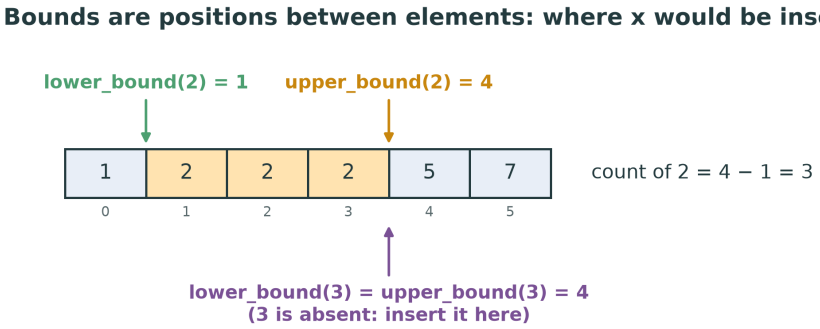
`binary_search_recursive(values, target, lo=0, hi=None)` keeps the same contract. The base case is the empty range, `lo > hi`: return −1. `hi=None` is the usual trick for a default that depends on another argument: the first call replaces it with `len(values) - 1`.

The slicing trap. A recursive search that calls itself on `values[:mid]` or `values[mid + 1:]` looks elegant and is wrong twice. It is slow: each slice copies up to n/2 elements, so the total is $n/2 + n/4 + \cdots = O(n)$ — as slow as a linear search, while looking like a binary one. And it loses the position: the index returned by a call on a slice is an index into the slice, not into the original list, so every call on a right half must add an offset. Passing `lo` and `hi` avoids both problems. (Lecture 03 made the same point about `list_max` on slices.)

The depth is at most about $\log_2 n$ frames — 20 for a million elements — so Python's 1,000-frame limit is no concern here, unlike the linear recursions of Lecture 03. The loop is still the better tool: same speed, no frames at all.

Bounds: Searching for Positions

`lower_bound` and `upper_bound`



- `lower_bound(v, x)`: the first index with `v[i] >= x`
- `upper_bound(v, x)`: the first index with `v[i] > x`
- Both may return **`len(v)`**. Neither returns −1.

`binary_search` answers "is it there, and where?", and with duplicates the "where" is **any** of the matching positions. The bounds answer more useful questions, and always with a position:

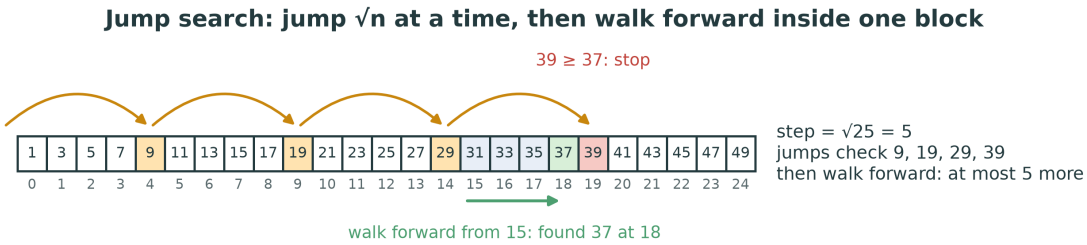
- **Where would x go?** `lower_bound(v, x)` is the index at which inserting x keeps the array sorted — the insertion point.
- **How many copies of x?** `upper_bound(v, x) - lower_bound(v, x)`. In the figure, 4 − 1 = 3 copies of 2, and 4 − 4 = 0 copies of 3 (`test_bounds_count_duplicates`).
- **The first occurrence?** `i = lower_bound(v, x)`; it is x if `i < len(v) and v[i] == x`.
- **How many values lie in [a, b)?** `lower_bound(v, b) - lower_bound(v, a)`. A range query in $O(\log n)$ — which no hash table can answer (Week 13).

They are both binary searches that **never stop early**: they keep halving until the range is empty and return where it closed. The usual way to write them uses a **half-open** range `[lo, hi)` — `lo` included, `hi` excluded — starting from `lo = 0, hi = len(v)`, so that `len(v)` is a possible answer. The loop runs while `lo < hi`, and the only difference between the two functions is which comparison sends the search right.

Python has them already, in the standard library: `bisect.bisect_left` is `lower_bound`, `bisect.bisect_right` is `upper_bound`, and `bisect.insort` inserts into a sorted list. C++ calls them `std::lower_bound` and `std::upper_bound`, which is where the names in `dsa/searching.py` come from. Write your own first; then use the library.

Three More Searches

Jump search: $O(\sqrt{n})$



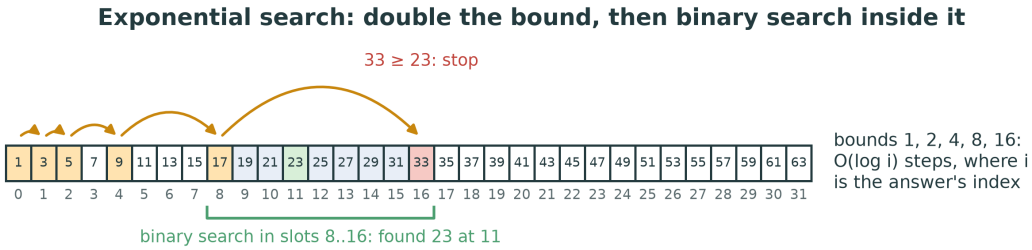
Jump ahead **√n** at a time until you pass the target; then walk forward inside that one block.

With blocks of size m, there are at most n/m jumps and then at most m steps inside a block: about $n/m + m$ reads. That sum is smallest when $m = \sqrt{n}$, giving $2\sqrt{n}$ — for a million elements, about 2,000 reads against 20 for binary search. Slower. So why bother?

Because jump search only ever moves **forward**, in big steps and then small ones. On storage where going back is expensive — a tape, or data arriving over a network in order — that matters more than the number of comparisons. It is also simple enough to be hard to get wrong.

In the figure, n = 25 and the step is $\sqrt{25} = 5$. The jumps check slots 4, 9, 14 and 19 — the last element of each block — and stop at 39, the first block end that is not smaller than 37. The walk goes forward from slot 15 and finds 37 at slot 18. (`math.isqrt(n)` gives the integer square root.)

Exponential search: $O(\log i)$



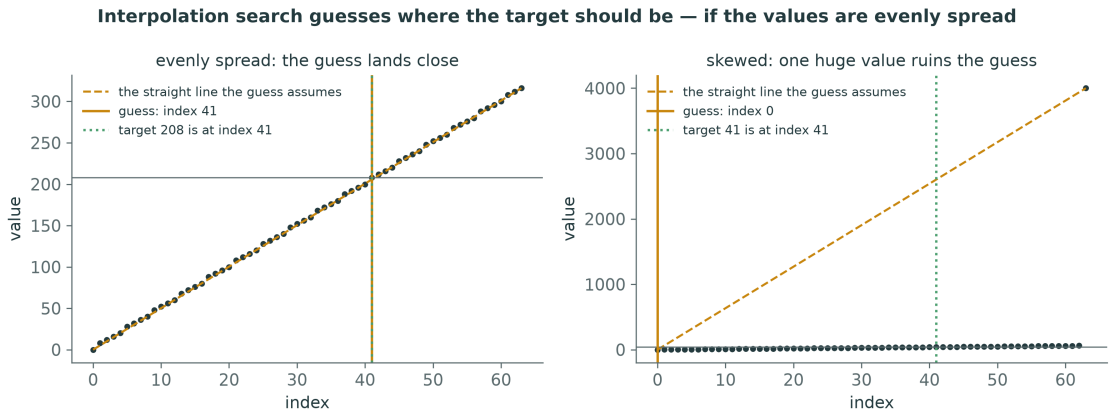
Check slots 1, 2, 4, 8, … until one passes the target; then binary search between the last two bounds.

If the answer is at index i, the doubling stops after about $\log_2 i$ steps, and the binary search that follows runs on a range of about i elements: $O(\log i)$ in total. When the target is near the **front**, that beats a binary search over the whole array; when it is near the end, it costs about twice as much. In the figure, 23 is at index 11: the bounds 1, 2, 4 and 8 are all smaller, 16 is not, so binary search runs on slots 8 to 16.

Its real use is on sequences whose **length you do not know** — a stream, an unbounded function, a huge sorted file: you cannot pick a middle without an end, but you can keep doubling until you have one. Bentley and Yao published it in 1976 as "an almost optimal algorithm for unbounded searching". The doubling is the same trick as the `DynamicArray`'s growth in Lecture 04: double until big enough, and the total work stays proportional to the final size.

Watch the edge cases: check slot 0 first (the doubling starts at 1), and do not let the bound run past `len(values) - 1`.

Interpolation search: guess, don't halve



$$\text{pos} = \text{lo} + \frac{(\text{target} - v[\text{lo}])\,(\text{hi} - \text{lo})}{v[\text{hi}] - v[\text{lo}]}$$

Evenly spread values: $O(\log \log n)$ on average. Skewed: $O(n)$.

Looking up "Zaki" in a phone book, you do not open it in the middle: you open it near the end, because Z is near the end of the alphabet. Interpolation search does exactly that. It assumes the values rise in a straight line from `v[lo]` to `v[hi]` and computes where the target would sit on that line. On evenly spread values the guess is very close, and the range shrinks so fast that the expected number of steps is $O(\log \log n)$ — about 4 or 5 for a million elements, against 20 for binary search. (The idea is W. W. Peterson's, 1957; the $\log \log n$ analysis is by Perl, Itai and Avni, 1978.)

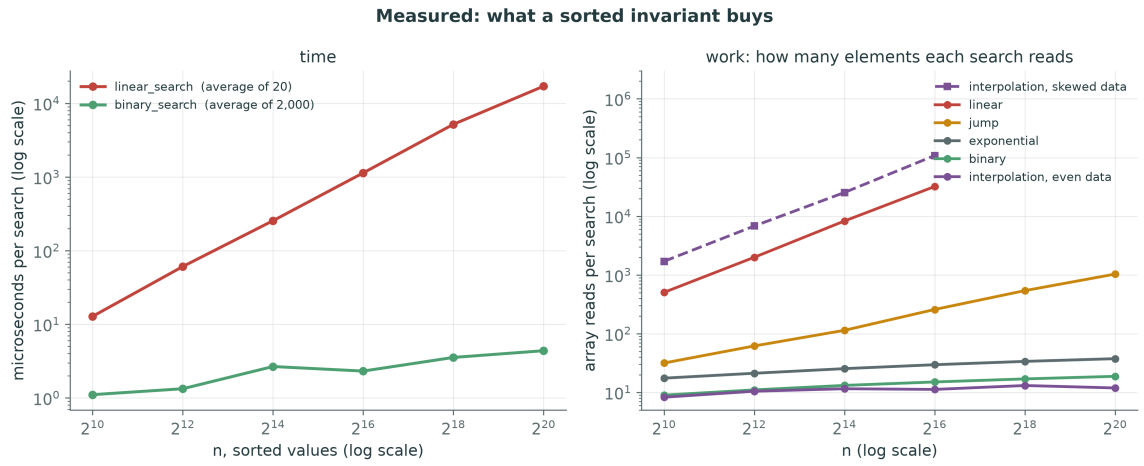
The price is the **worst case**. In the right-hand panel, one huge value at the end makes the straight line useless: every guess lands at the far left, the range shrinks by one element per step, and the search becomes a slow linear scan — $O(n)$. That is the docstring's warning: an average case that hides a bad worst case.

Three traps in the code, which the tests on one element and on random data catch:

- **Division by zero** when `v[hi] == v[lo]`: handle that range separately.
- **A guess outside the range** when the target is smaller than `v[lo]` or larger than `v[hi]`: stop at once — it is not there.
- **Integer division**: the position must be an `int` index. Use `//`.

Measured

Eight functions, one picture



Left: time. On a million sorted integers, a linear search took about 17 milliseconds and a binary search about 4 **microseconds** — some four thousand times faster, and the gap doubles every time n doubles. The binary line is not perfectly smooth: at these tiny times, memory caches and timer noise show. Its rise over the whole range, from 1 to about 4 µs while n grew a thousandfold, is the logarithm.

Right: work — how many elements each search reads, on average, counted exactly by passing the searches a sequence that counts every `values[i]`. The counts come from the reference implementation, and yours may differ by a small constant factor.

- **Linear:** about n/2 reads — slope 1 on the log–log plot.

- **Jump:** about √n reads — slope ½. 1,035 reads at n = 2^{20}, where √n = 1,024.
- **Binary:** 18.8 reads at n = 2^{20}, where $\log_2 n = 20$.
- **Exponential:** about twice binary, since it doubles up to the target and then binary searches a range as big again — 37.5 at n = 2^{20}.
- **Interpolation, even data:** flat at about 12 reads — about four steps of three reads each (v[lo], v[hi] and the guess), at every size. $\log \log n$ is as close to constant as a growing function gets.
- **Interpolation, skewed data:** more than 100,000 reads at n = 65,536 — **three times worse than linear search**, because each of its roughly n/2 one-slot steps reads three values.

The four searches on sorted data all pass the same tests. Measurement is what tells them apart.

When is sorting first worth it?

Binary search needs sorted data, and sorting costs $O(n \log n)$ (Weeks 9–10).

k searches on n values	Cost
linear search, k times	$O(kn)$
sort once, then binary search k times	$O(n \log n + k \log n)$

Sorting wins once **k is larger than about** $\log n$ — for a million values, after a few dozen searches.

For n = 1,000,000: one linear search reads up to a million values; a good sort does about $n \log_2 n \approx 20$ million comparisons. So after roughly twenty searches the sort has paid for itself, and every search after that costs 20 reads instead of a million. That is the reasoning behind every database index: pay once to keep data ordered, then answer questions fast for ever.

It is also why **keeping** an array sorted as it changes matters: inserting at `lower_bound` keeps it sorted, but the insertion itself shifts elements — $O(n)$ (Lecture 02). A sorted array is fast to search and slow to change. Weeks 11 and 13 give two answers: the binary search **tree** (fast to search and to change, while it stays balanced) and the **hash table** ($O(1)$ average lookup, but no order — no bounds, no ranges).

The eight functions

Function	Needs	Time	Note
`linear_search`	nothing	$O(n)$	first match
`binary_search`	sorted	$O(\log n)$	loop, $O(1)$ space
`binary_search_recursive`	sorted	$O(\log n)$	$O(\log n)$ stack; never slice
`lower_bound`	sorted	$O(\log n)$	first `v[i] >= x`; may be `len`
`upper_bound`	sorted	$O(\log n)$	first `v[i] > x`; may be `len`
`jump_search`	sorted	$O(\sqrt{n})$	forward only

Function	Needs	Time	Note
`exponential_search`	sorted	$O(\log i)$	unknown length; near the front
`interpolation_search`	sorted, even	$O(\log \log n)$ avg, $O(n)$ worst	guesses

This Week

Exercises: `dsa/searching.py`

Function	The trap
`linear_search`	unsorted input; return the **first** match
`binary_search`	`lo <= hi`; `mid ± 1`; –1 when missing
`binary_search_recursive`	pass `lo`, `hi`; **no slices**
`lower_bound`, `upper_bound`	half-open `[lo, hi)`; `>=` against `>`
`jump_search`	`math.isqrt`; the last, partial block
`exponential_search`	slot 0 first; bound capped at `len - 1`
`interpolation_search`	equal ends; target outside `[v[lo], v[hi]]`

```
pytest tests/test_searching.py -v
```

Forty tests. The four searches on sorted data are run against the same cases — found, missing, empty, one element, first, last — and against linear search on 30 random sorted lists (`test_agrees_with_linear_search_on_random_data`, with a fixed seed so a failure is repeatable). The values in those tests are distinct, so "any matching index" and "the first matching index" agree.

The functions take Python lists as **input**, which the storage rule allows: they only read them and store nothing.

Homework 8 — before Lecture 09

1. **Implement** `dsa/searching.py` until all forty tests pass.
2. **Trace** `binary_search` on `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]` for 23 and for 60, as tables of `lo`, `hi`, `mid`.
3. **Measure** linear against binary search in `notebooks/08-searching.ipynb`, and count reads with a counting sequence, as in the figure.
4. **Break it.** Change `lo <= hi` to `lo < hi` in your `binary_search`. Which tests fail, and why exactly those?

For item 3, the counting sequence is a class with `__len__` and a `__getitem__` that adds 1 to a counter before returning `self.values[i]`; your searches cannot tell it from a list. `tools/figures_l08.py` has one, `CountingReads`.

For item 4, predict before you run: which **positions** can a `lo < hi` search never examine?

The midterm window

Weeks 1–7 are examinable. For each week, can you:

- **1–2:** define an ADT; give Θ of a loop; prove a Big-O bound?
- **3:** write a recurrence and trace a call stack?
- **4:** explain amortised $O(1)$ append?
- **5–7:** draw a linked list, stack, queue or ring **after every operation**?

The mock exam for weeks 1–7 is in the question bank.

The exact date and format of the midterm will be announced in the lecture and on the WhatsApp channel. Until then, the safest preparation is the one the final rewards: the weekly question banks, done **before** reading their answers, and the mock exam under exam conditions — in one sitting, closed book, no computer. Searching is not on the weeks 1–7 mock; it is on the final.

Summary

Seven things to keep

1. Linear search needs no order and costs $O(n)$.
2. Binary search needs **sorted** data and costs $O(\log n)$: 20 comparisons for a million.
3. Its invariant: if the target is anywhere, it is in `values[lo..hi]`.
4. Get the boundaries right: `lo <= hi`, `mid ± 1` — test sizes 0, 1, 2.
5. Recursive: pass `lo` and `hi`; a slice turns $O(\log n)$ into $O(n)$.
6. `lower_bound`/`upper_bound` return **positions**: insertion points and counts of duplicates.
7. Jump $O(\sqrt{n})$, exponential $O(\log i)$, interpolation $O(\log \log n)$ average but $O(n)$ **worst**.

Next

Week 9 — Basic sorting. Bubble, selection and insertion sort — and counting sort, which beats $O(n \log n)$ by never comparing at all.

Sources and further reading

- **D. E. Knuth.** *The Art of Computer Programming*, vol. 3, *Sorting and Searching*, 2nd ed., Addison-Wesley, 1998, §6.1 "Sequential searching" and §6.2.1 "Searching an ordered table" — binary search, its history, and interpolation search.
- **J. Bentley.** *Programming Pearls*, 2nd ed., Addison-Wesley, 1999, column 4, "Writing correct programs" — binary search, invariants, and the programmers who got it wrong.
- **J. Bloch.** "Extra, Extra — Read All About It: Nearly All Binary Searches and Mergesorts are Broken", Google Research Blog, 2 June 2006.
- **J. L. Bentley and A. C. Yao.** "An almost optimal algorithm for unbounded searching", *Information Processing Letters* 5(3), 1976 — exponential search.
- **Y. Perl, A. Itai and H. Avni.** "Interpolation search — a log log N search", *Communications of the ACM* 21(7), 1978.

- **Python documentation.** The `bisect` module — `bisect_left`, `bisect_right` and `insort`.

Every figure in this lecture is generated by `tools/figures_l08.py`. The diagrams are fixed traces; the measured figure times a working `dsa/searching.py` and will differ slightly on your machine.

Questions for Week 8

> ***The answers are at the back of the book,*** *in* Answers — Week 8. *Levels:* ***[what]*** *recall · **[how]** apply · **[why]** explain and justify. The functions are those of `dsa/searching.py`. `binary_search` uses an inclusive range `values[lo..hi]` and `mid = (lo + hi) // 2`; `lower_bound` and `upper_bound` use a half-open range `[lo, hi)` starting at `lo = 0`, `hi = len(values)`. Unless a question says otherwise,* ***V*** *is the sorted list*
>
> `V = [3, 6, 8, 12, 15, 19, 21, 24, 27, 31, 34, 38, 41, 45, 48, 52]`
>
> *of 16 values, at indices 0 to 15.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W8-M01 – W8-M22
B	Short answer and essay	W8-E1 – W8-E5
C	Trace the algorithms	W8-T1 – W8-T5
D	Complexity analysis	W8-K1 – W8-K3
E	Find and fix the bug	W8-B1 – W8-B4
F	Write the code — checked by `pytest`	W8-C1 – W8-C5

Part A — Multiple choice

W8-M01 [what] Which search works on an **unsorted** list?

- **a)** linear search
- **b)** binary search
- **c)** jump search
- **d)** interpolation search

W8-M02 [how] The largest number of comparisons binary search makes on a sorted list of 1,000 elements is:

- **a)** 1,000
- **b)** 500
- **c)** 10
- **d)** 32

W8-M03 [how] …and on 1,000,000 elements:

- **a)** 1,000

- **b)** 20
- **c)** 1,000,000
- **d)** 6

W8-M04 [why] Binary search is called on a list that is **not** sorted. It:

- **a)** raises an error
- **b)** sorts the list first
- **c)** always returns −1
- **d)** may return a wrong answer, with no error

W8-M05 [how] `binary_search(V, 20)`: the index of the **first** element compared is:

- **a)** 7
- **b)** 8
- **c)** 3
- **d)** 15

W8-M06 [how] …and the index of the **second**:

- **a)** 11
- **b)** 5
- **c)** 3
- **d)** 6

W8-M07 [what] The loop invariant of binary search is:

- **a)** `lo` is always smaller than `mid`
- **b)** if the target is anywhere in the list, it is in `values[lo..hi]`
- **c)** `values[mid]` is always the target
- **d)** the range halves exactly at every step

W8-M08 [why] A binary search with an inclusive `hi` loops `while lo < hi`. Searching `[42]` for 42, it returns:

- **a)** 0
- **b)** an `IndexError`
- **c)** nothing — it loops for ever
- **d)** −1

W8-M09 [why] In Java, `mid = (lo + hi) / 2` is a bug because:

- **a)** `lo + hi` can overflow a 32-bit `int` on very large arrays
- **b)** Java has no integer division
- **c)** it rounds the wrong way
- **d)** it is slower than `lo + (hi - lo) / 2`

W8-M10 [why] A recursive binary search calls itself on `values[:mid]` or `values[mid + 1:]`. Its total time on n elements is:

- **a)** $O(\log n)$
- **b)** $O(1)$
- **c)** $O(n)$ — the slices copy

- **d)** $O(n \log n)$

W8-M11 [what] The extra space of `binary_search_recursive` (passing `lo` and `hi`, no slices) is:

- **a)** $O(1)$
- **b)** $O(n)$
- **c)** $O(n \log n)$
- **d)** $O(\log n)$ — the call stack

W8-M12 [how] `lower_bound([1, 2, 2, 2, 5, 7], 2)` is:

- **a)** 0
- **b)** 1
- **c)** 3
- **d)** 4

W8-M13 [how] `upper_bound([1, 2, 2, 2, 5, 7], 2)` is:

- **a)** 4
- **b)** 3
- **c)** 1
- **d)** 5

W8-M14 [how] `lower_bound([1, 2, 2, 2, 5, 7], 8)` is:

- **a)** –1
- **b)** 5
- **c)** an error
- **d)** 6

W8-M15 [how] How many times does 3 occur in `[1, 3, 3, 3, 3, 9]`, computed as `upper_bound(v, 3) - lower_bound(v, 3)`?

- **a)** 3
- **b)** 5
- **c)** 4
- **d)** 1

W8-M16 [what] Python's `bisect.bisect_left(values, x)` is the same as:

- **a)** `binary_search`
- **b)** `lower_bound`
- **c)** `upper_bound`
- **d)** `linear_search`

W8-M17 [how] Jump search on 100 sorted elements jumps by:

- **a)** 50
- **b)** 2
- **c)** 7
- **d)** 10

W8-M18 [why] Jump search uses blocks of size $\sqrt{n}$ because:

- **a)** it minimises the total $n/m + m$ of jumps plus steps inside a block
- **b)** $\sqrt{n}$ is always an integer
- **c)** smaller blocks would be incorrect
- **d)** it makes jump search $O(\log n)$

W8-M19 [how] Exponential search for a target at index 11 of a 32-element list checks the bounds:

- **a)** 1, 3, 7, 15
- **b)** 11 only
- **c)** 1, 2, 4, 8, 16
- **d)** 16, 8, 4, 2, 1

W8-M20 [why] Exponential search is most useful when:

- **a)** the list is unsorted
- **b)** the length is unknown, or the target is near the front
- **c)** the values are evenly spread
- **d)** the list has duplicates

W8-M21 [why] The worst case of interpolation search is:

- **a)** $O(\log \log n)$
- **b)** $O(\log n)$
- **c)** $O(\sqrt{n})$
- **d)** $O(n)$ — on skewed values

W8-M22 [how] Interpolation search on `[10, 20, 30, …, 100]` (10 values) for 70: the first index it probes is:

- **a)** 6
- **b)** 4
- **c)** 7
- **d)** 5

Part B — Short answer and essay

W8-E1 [why] *(4 marks)* Describe binary search. State its loop invariant and use it to explain why the algorithm is correct, and show that it makes at most $\lfloor \log_2 n \rfloor + 1$ comparisons. What must the caller guarantee?

W8-E2 [how] *(4 marks)* Give four classic bugs in binary search. For each, give an input on which it fails, what happens, and the fix.

W8-E3 [why] *(4 marks)* Define `lower_bound` and `upper_bound`. Give three uses of them, and explain why they never return −1.

W8-E4 [why] *(3 marks)* Compare jump search, exponential search and interpolation search: cost, and a situation in which each is the right choice.

W8-E5 [why] *(3 marks)* A program searches an unsorted list of n values k times. When is it worth sorting the list first? Compare the costs, and give a concrete example with n = 1,000,000.

Part C — Trace the algorithms

W8-T1 [how] Trace `binary_search` on `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]` for 23, and then for 60, as tables of `lo`, `hi`, `mid` and `values[mid]`. Where does `lo` end up when 60 is not found, and what does that position mean?

W8-T2 [how] Trace `lower_bound` and `upper_bound` on `[1, 2, 2, 2, 5, 7]` for 2, as tables of `lo`, `hi`, `mid` and the direction taken.

W8-T3 [how] Draw the calls of `binary_search_recursive(V, 45)` and `binary_search_recursive(V, 7)`: the `lo` and `hi` of each call, and what it returns. What is the depth of the deeper one?

W8-T4 [how] Trace `jump_search` on the 25 odd numbers `[1, 3, 5, …, 49]` for 30. Which elements are read, and what is returned?

W8-T5 [why] Trace `interpolation_search` for 70 and for 75 on `[10, 20, …, 100]`, and for 9 on `[1, 2, 3, 4, 5, 6, 7, 8, 9, 1000]`. Count the probes. What does the last case show?

Part D — Complexity analysis

W8-K1 [how] Give Θ and justify, for a sorted list a of n values and a list b of m values:

```
found = 0
for x in b:
    if binary_search(a, x) != -1:
        found += 1
```

What is it with `linear_search`? And if a were not sorted?

W8-K2 [how] Write the recurrence and give Θ for a recursive binary search that passes **slices** (`values[mid + 1:]`) instead of `lo` and `hi`.

W8-K3 [why] A student counts the copies of x in a sorted list: find **any** copy with `binary_search`, then walk left and right while the neighbours equal x. Give its cost in terms of n and the number of copies c, and its worst case. What is the $O(\log n)$ way?

Part E — Find and fix the bug

W8-B1 [how]

```
def binary_search(values, target):
    lo, hi = 0, len(values) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if values[mid] == target:
            return mid
```

```
        if values[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

W8-B2 [why]

```
def binary_search(values, target):
    lo, hi = 0, len(values) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if values[mid] == target:
            return mid
        if values[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return -1
```

W8-B3 [why]

```
def binary_search_recursive(values, target):
    if not values:
        return -1
    mid = len(values) // 2
    if values[mid] == target:
        return mid
    if values[mid] < target:
        return binary_search_recursive(values[mid + 1:], target)
    return binary_search_recursive(values[:mid], target)
```

W8-B4 [how] Part of `interpolation_search`:

```
lo, hi = 0, len(values) - 1
while lo <= hi and values[lo] <= target <= values[hi]:
    pos = lo + (target - values[lo]) * (hi - lo) // (values[hi] - values[lo])
    ...
```

Part F — Write the code

In `practice/week08.py`; check with `pytest tests/test_practice_week08.py -v`. Each problem must be $O(\log n)$ (C5: $O(n \log S)$): the tests count your reads. Do not use `bisect`, `in`, `index` or `sorted`.

W8-C1 [why] `search_rotated(values, target)` — binary search on a sorted list that has

been rotated. Why is one half always sorted?

W8-C2 [how] `integer_sqrt(n)` — the largest r with $r^2 \le n$, by binary search on the answer.

W8-C3 [why] `find_peak(values)` — an element not smaller than its neighbours, in $O(\log n)$, on an **unsorted** list. Why is it safe to discard half?

W8-C4 [how] `closest_value(values, target)` — the nearest value, ties to the smaller.

W8-C5 [why] `min_capacity(weights, days)` — the smallest ship capacity that delivers the packages in time. Why may you binary search on the capacity?

Week 9 — Basic Sorting

> ***How to read this document.*** *This is the handout for Lecture 09. It holds everything on the slides, plus what I said out loud. Three simple sorts, each written once as a* ***generator*** *so that you can watch it work, one sort that never compares at all, and a proof that no comparison sort can beat $n \log n$. Every count and timing in it was measured on the course's own code.*
>
> *Slides: `DSA27-L09-slides.pdf` · Code: `dsa/sorting.py` · Tests: `tests/test_sorting.py`*

Where We Are

Sorting buys searching

Week 8: on **sorted** data, a search reads 20 values out of a million.

k searches on n values	Cost
linear search, k times	$O(kn)$
sort once, then binary search k times	sort $+O(k \log n)$

Today: **where the sorted data comes from — and what it costs.**

Lecture 08 ended with a promise it could not keep by itself: binary search, `lower_bound`, jump and interpolation search all assume that the input is sorted, and nothing in them makes it so. This week and next are about paying that bill. The answer to "is sorting first worth it?" depends entirely on what the sort costs. With the $O(n \log n)$ sorts of Week 10, sorting a million values costs about 20 million comparisons, and pays for itself after about twenty searches. With today's $O(n^2)$ sorts it costs about 500 **billion** comparisons, and pays for itself after half a million searches. The same idea — sort, then search — is either obviously right or obviously wrong, depending on which sort you use.

Sorting in Arabic: ﷺالترتي. *Sorting algorithm*: ﷺخوارزمية ترتي.

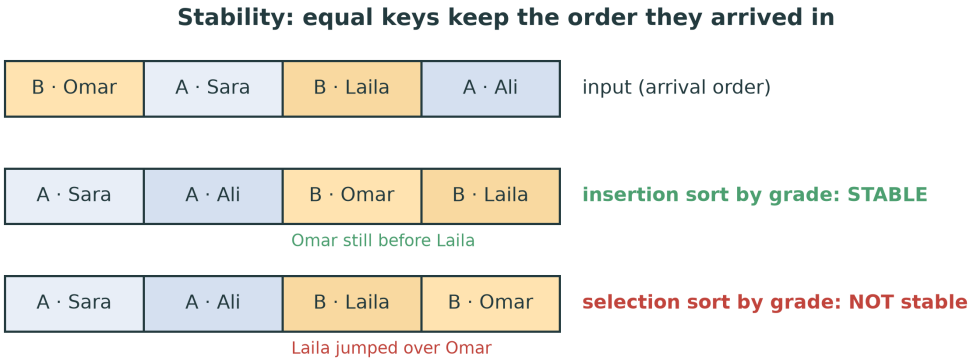
Today

1. Two words first: **in place** and **stable**
2. Bubble, selection and insertion sort — $O(n^2)$ each, but not the same
3. Watching an algorithm: the `_steps` generator — and its price
4. Best, worst, average: comparisons and writes, **measured**
5. Counting sort: sorting without comparing
6. Why no comparison sort can beat $n \log n$

Two Words Before Any Algorithm

In place, and stable

- **In place:** only $O(1)$ extra memory besides the array being sorted.
- **Stable:** elements with **equal keys** keep the order they arrived in.



In place: في المكا. *Stable*: مستق. *Key*: المفتا — the part of an element the sort compares.

Both words describe **how** a sort arrives at the sorted order, not the order itself. A sort of plain integers cannot show you whether it is stable: two 3s look the same whichever came first. Stability only becomes visible when the elements carry more than their key — records sorted by one field.

In the figure, four students are sorted by grade. Omar arrived before Laila and both have a B. A stable sort promises Omar still comes before Laila; the unstable one here swapped Laila over him. Both outputs are **sorted by grade**. Only one kept the promise.

`dsa/sorting.py` works on a copy: the list you pass is copied into a course `Array`, the sort rearranges the `Array`, and a new list comes back (`test_does_not_mutate_the_input`). The rearranging itself uses $O(1)$ extra slots in all three sorts of today — so we call them in place, even though the function as a whole returns a new list. Python makes the same distinction: `values.sort()` sorts in place and returns `None`; `sorted(values)` returns a new list.

Why stability matters

Sort a class list **by name**, then **by grade** with a stable sort:

- the result is by grade, and **by name within each grade**, for free.

This is how spreadsheets sort by two columns, and how every database orders `ORDER BY grade, name`.

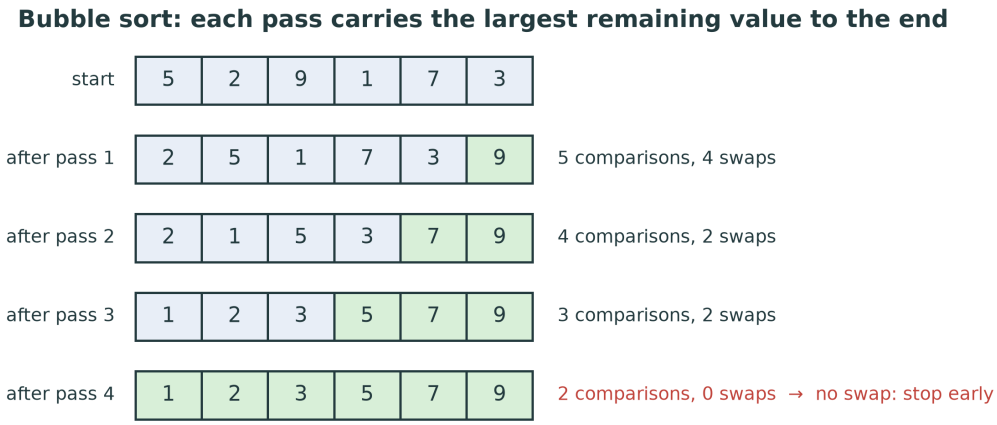
Sorting by several keys with a stable sort works from the **least** important key to the most important: first by name, then by grade. The second sort reorders only students with different grades; students with the same grade compare equal, so a stable sort leaves them in the name order the first sort gave them. An unstable sort would scramble them.

Python's own `sorted` and `list.sort` are guaranteed stable, for exactly this reason, and the

documentation says so. Of today's three sorts, bubble and insertion sort are stable and selection sort is not — the figure's unstable output is what selection sort really does to those four records.

Three O(n^2) Sorts

Bubble sort



Compare **neighbours**; swap them if they are out of order. Each pass carries the largest remaining value to the end. A pass with **no swap** means sorted: stop.

Bubble sort: ترتيب الفقاعات — the large values rise to the end like bubbles.

On the running example `[5, 2, 9, 1, 7, 3]`, pass 1 walks the whole array. The 9 is larger than everything it meets, so it is swapped at every step until it reaches the end: after one pass, the largest value is **in its final place** (green). Pass 2 does the same for the 7 — and need not look at the last slot, because it already holds the largest value. Pass k needs only $n - k$ comparisons.

The **early exit** is what makes bubble sort worth studying at all. If a whole pass makes no swap, every neighbour pair is in order, which means the whole array is sorted: stop. Here pass 4 finds nothing to swap and the sort ends after 14 comparisons instead of $5 + 4 + 3 + 2 + 1 = 15$. On input that is already sorted, the first pass swaps nothing and the sort ends after $n-1$ comparisons: the best case is $O(n)$.

The worst case is reversed input: every comparison swaps, and all $n-1$ passes run — $n(n-1)/2$ comparisons and as many swaps. $O(n^2)$.

Stable, because it swaps only when `a[j] > a[j + 1]` — strictly greater. Two equal neighbours are never swapped, so equal keys never pass each other.

The code, written as a generator

```
def bubble_sort_steps(values, snapshot=list):
    a = _copy(values)
    n = len(a)
    yield snapshot(a), ()
```

```
    for end in range(n - 1, 0, -1):          # a[end+1:] is already in place
        swapped = False
        for j in range(end):
            yield snapshot(a), (j, j + 1)      # about to compare
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            break
    yield snapshot(a), ()


def bubble_sort(values):
    return _finish(bubble_sort_steps(values, snapshot=_live))
```

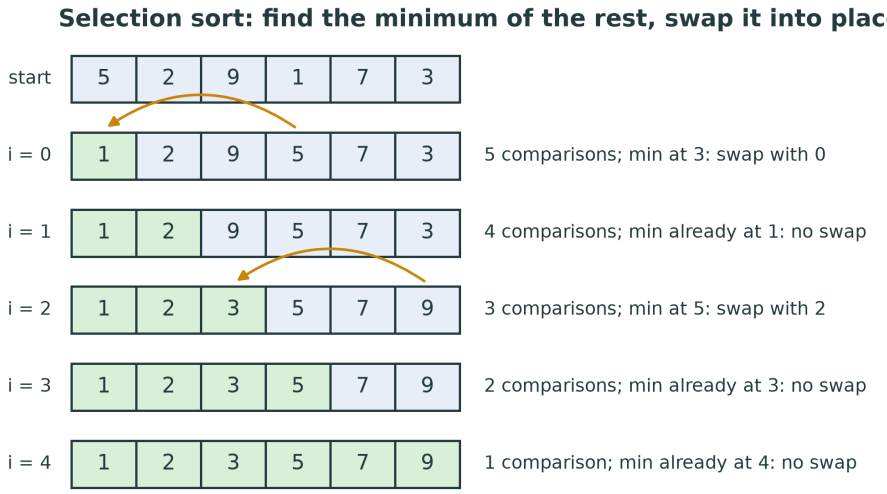
This is the reference solution, exactly (`solutions/dsa/sorting.py`). Read it as two things at once.

The algorithm is the loops and the `if`. `end` is the last slot the current pass still has to look at; after the pass, `a[end]` holds the largest value of `a[0..end]`, so the next pass stops one earlier. `swapped` records whether this pass changed anything. `_copy(values)` copies the input into a fresh course `Array`, so the caller's list is never touched.

The film is the `yield` lines. Before each comparison, the generator hands out a picture of the array and the pair it is about to compare, `(j, j + 1)`, and pauses. Whoever is iterating decides what to do with each picture: draw it, count it, or ignore it. A generator (Lab 03, "Generators") runs only as far as it is asked to — which is what makes a step-by-step animation possible.

The plain sort `bubble_sort` is not a second implementation. `_finish` runs the generator to its end and returns the last state as a list. There is one piece of code to keep correct, and the animation shows exactly the code the tests grade. The `snapshot` parameter, and `_live`, are the subject of "The price of watching" below.

Selection sort



Find the **minimum** of the unsorted part; **swap** it into the next slot. Always $n(n-1)/2$ comparisons — but at most $n-1$ swaps.

Selection sort: ترتيب بالاختيا.

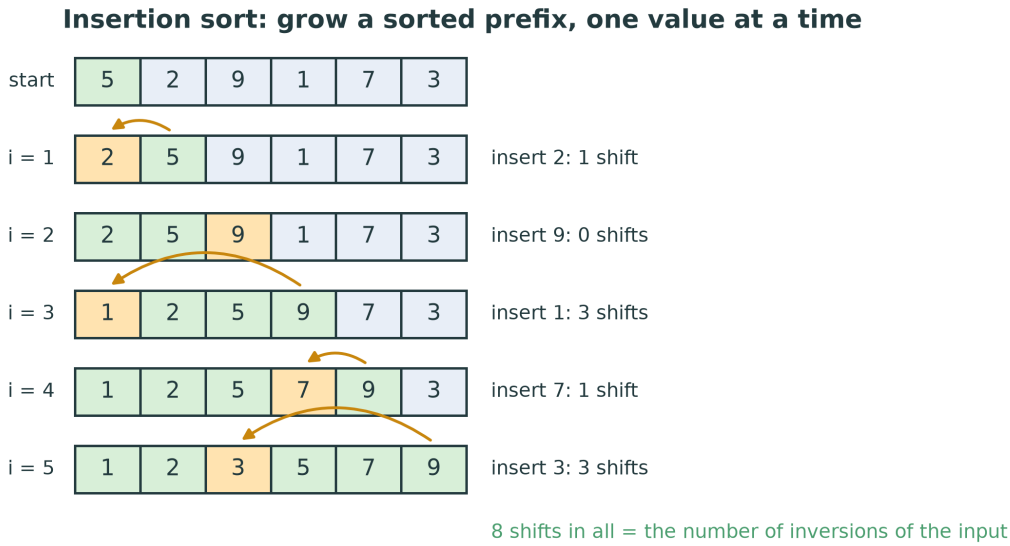
Round i scans `a[i..n-1]` for its smallest value and swaps it into slot i. After round i, `a[0..i]` holds the i + 1 smallest values, in order, in their final places. The scan in round i makes $n-1-i$ comparisons whatever the data, so the total is always $(n-1)+(n-2)+\cdots+1 = n(n-1)/2$: selection sort has **no** best case. Sorted input costs as much as reversed input.

Its virtue is the other count. There is at most one swap per round — none at all when the minimum is already in place, as in rounds i = 1, 3 and 4 of the figure — so at most $n-1$ swaps in total. That is the fewest of any of today's sorts, and it matters when a write is much more expensive than a read: moving large records, or writing to flash memory, which wears out with every write.

Not stable. The swap moves `a[i]` a long way to the right, possibly over an element equal to it. In the stability figure, round i = 1 swaps Omar (B) with Ali (A), the minimum of the rest: Omar lands at the back, behind Laila, who had arrived after him.

Two details in the reference: the scan compares with `a[smallest]`, the best seen **so far** — not with `a[i]` — and the swap is skipped when `smallest == i`.

Insertion sort



Grow a **sorted prefix**. Take the next value; shift every larger value in the prefix one slot right; drop the value into the hole.

Insertion sort: الترتيب بالإدرا.

This is how most people sort a hand of cards. At round i, `a[0..i-1]` is sorted (green); `current = a[i]` is the next card. Walk left from `j = i - 1` while `a[j] > current`, shifting each of those values one slot right; when the walk stops — at a value not greater than `current`, or off the front of the array — put `current` in the gap. The highlighted value in each row of the figure is the one just inserted.

Lab 08's bridge did the same thing with `lower_bound` and `list.insert`. Here the shifting is written out, and it is the shifting that costs: finding the place faster would not help, because the elements still have to move.

Stable, because the walk stops at a value **equal** to `current` — the loop condition is `a[j] > current`, strictly greater — so `current` never passes an equal key. Write `>=` there and the sort still sorts, but loses stability; `test_insertion_sort_is_stable` catches it.

Watch the order of the condition: `j >= 0 and a[j] > current`. The other way round, `a[j]` is read when `j` is -1. On a Python list that silently reads the **last** element; the course `Array` refuses negative indices and raises `IndexError` — one more bug the `Array` catches for you.

The reference's `insertion_sort_steps` writes `current` into the hole after every shift, not only at the end, so that every frame of the animation is a real permutation of the input — no value duplicated, none missing. That costs a second write per shift; the textbook version writes `current` once, after the loop. The comparisons are the same.

Insertion sort counts inversions

An **inversion** is a pair $i < j$ with $a[i] > a[j]$ — a pair in the wrong order.

- Each shift fixes **exactly one** inversion.
- So insertion sort does **one shift per inversion**: 8 for the example.
- Sorted: 0 inversions. Reversed: $n(n-1)/2$. Random: about $n(n-1)/4$.

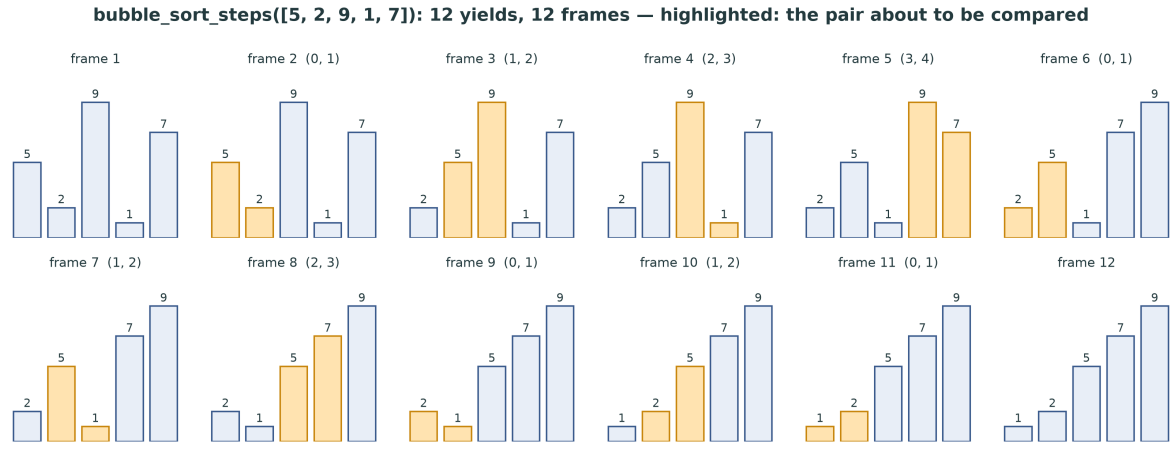
Inversion: انعكا.

The inversions of `[5, 2, 9, 1, 7, 3]` are (5,2), (5,1), (5,3), (2,1), (9,1), (9,7), (9,3) and (7,3): eight, and the figure counts eight shifts. When `current` moves past a larger value, that pair was inverted and now is not; no other pair changes its order. So the number of shifts is exactly the number of inversions, and insertion sort runs in $O(n + I)$ time, where I is the number of inversions.

That one formula explains its best, worst and average case. Sorted input has no inversions: $n-1$ comparisons and no shifts — $O(n)$. Reversed input has every pair inverted: $n(n-1)/2$ shifts — $O(n^2)$. In a random permutation each pair is inverted with probability ½, so the average is $n(n-1)/4$ — still $\Theta(n^2)$, but half the worst case. Bubble sort's swaps obey the same law: each swap of neighbours also fixes exactly one inversion, so bubble sort makes exactly I swaps too.

Watching an Algorithm

One yield, one frame




```
from viz.animate import step_slider, animate_bars
step_slider(bubble_sort_steps([5, 2, 9, 1, 7]))     # scrub by hand
animate_bars(bubble_sort_steps([5, 2, 9, 1, 7]))    # play it
```

Each `yield` of `bubble_sort_steps` becomes one frame: the values as bars, the pair about to be compared in amber. Twelve yields for five values — the first state, ten comparisons, the last state — and twelve frames. `viz/animate.py` does not know which algorithm it is showing; it only needs a generator that yields `(values, highlight)` pairs. That is the contract of every `_steps` function in `dsa/sorting.py`, and it is what lets you watch **your own** code in the notebook, not a video of someone else's.

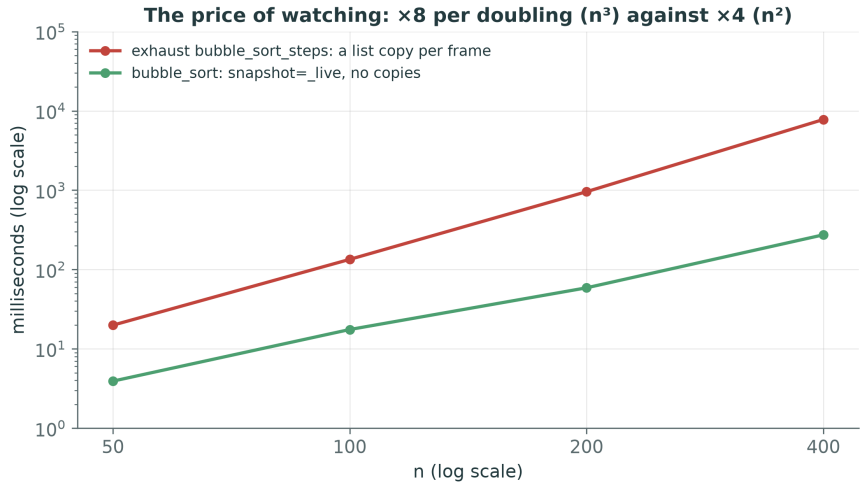
`step_slider` is the better tool for learning: you go at your pace, you can go back, and you can stop on the frame you do not understand. In a notebook, display an animation with

`HTML(animate_bars(...).to_jshtml())`.

Two rules make the frames right, and the tests check both:

- **Each frame must be its own list.** Yield `list(a)`, a fresh copy, not the working `Array`. The animation keeps every frame; if they were all the same object, they would all show its final state (`test_step_frames_are_independent_snapshots`).
- **The last frame must be sorted.** Yield once more after the loops (`test_step_form_is_a_generator_er`

The price of watching



A copy per frame is $O(n)$. Copy before each of $n^2/2$ comparisons: $O(n^3)$.

The plain sort passes `snapshot=_live` — no copies — and is $O(n^2)$ again.

The first reference version of `dsa/sorting.py` built `bubble_sort` by running `bubble_sort_steps` to the end — with a `list(a)` copy at every one of its $n(n-1)/2$ yields. Each copy costs n, so the plain sort cost $\Theta(n^3)$. The tests could not notice: their largest list has 40 values. Measurement did: the red line rises **eight** times per doubling of n, the signature of n^3 (Lecture 02), and sorting 800 values took over a minute.

The fix keeps one implementation. `snapshot` is the function that turns the working `Array` into the frame to yield. The default, `list`, makes a copy — right for an animation. The plain sort passes `_live`:

```
def _live(a):
    """The working Array itself — no copy. For the plain sorts only."""
    return a
```

Now each yield costs $O(1)$, and `_finish` makes one copy at the very end. The green line rises four times per doubling: $O(n^2)$, as promised. The lesson is not about sorting. **Instrumenting code changes its cost**, and only measuring tells you by how much. A print statement inside a loop, a log line, an assertion that checks `is_sorted` at every step: each one is an $O(n)$ that can hide inside your $O(n^2)$.

Best, Worst, Average

Counting comparisons and writes

Time depends on your machine; **counts** do not. The figure counts, for n = 1,000 values, every comparison — by sorting objects whose < and > add 1 to a counter — and every write into the course `Array`, by counting calls to its `__setitem__`. `tools/figures_l09.py` does both; the lab asks you to write the comparison counter yourself.

What to read off it:

- **Selection sort compares 499,500 times on every input** — $n(n-1)/2$ — even on sorted data. But it writes at most 1,998 times: two writes per swap, at most $n-1$ swaps. On reversed input it swaps only 500 times: each swap fixes both ends.
- **Bubble and insertion sort** do $n-1=999$ comparisons on sorted input and about 1,000–2,000 on nearly sorted input (1% of neighbours swapped): **the early exit and the stopping walk pay off.**
- **Writes follow the inversions.** On the random input, with 248,621 inversions, bubble sort writes 2 × 248,621 = 497,242 times — two writes per swap. Insertion sort writes 498,241: two per shift in the reference (see "Insertion sort"), plus one final write per value.
- **Random input:** insertion sort makes about half the comparisons of bubble sort (249,614 against 491,874). Bubble sort's early exit rarely fires on random data, so it does nearly all $n(n-1)/2$.

All three are $O(n^2)$ in the worst case. The counts show that the same class can hide a factor of two, or of a thousand, in the cases you actually meet.

The table

	best	average	worst	writes	stable	in place
bubble (early exit)	n	n^2	n^2	I swaps	yes	yes
selection	n^2	n^2	n^2	$\leq n-1$ swaps	**no**	yes

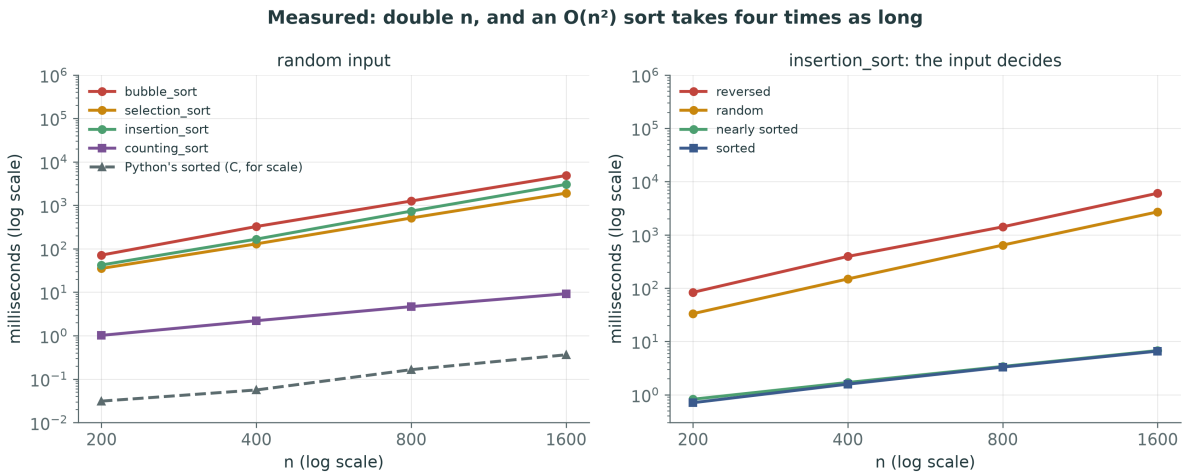
	best	average	worst	writes	stable	in place
insertion	n	n^2	n^2	I shifts	yes	yes
counting	$n+k$	$n+k$	$n+k$	$2n$	—	**no**, $O(n+k)$

I = the number of inversions: 0 when sorted, $n(n-1)/2$ when reversed.

Best, worst and average are about **inputs of the same size**: the best case is the kindest input of size n, the worst case the cruellest, and the average case the expected cost over all $n!$ orders, each equally likely. They are three different functions of n, and each can be given its own O — which is why "insertion sort is $O(n^2)$" and "insertion sort is $O(n)$ on sorted input" are both true.

Exact comparison counts: bubble sort with the early exit makes $n-1$ at best and $n(n-1)/2$ at worst; selection sort exactly $n(n-1)/2$ always; insertion sort between $n-1$ and $n(n-1)/2$, and $I+n-1$ at most. The "stable" column for counting sort is a dash because `counting_sort` rebuilds integers from counts: equal integers are indistinguishable, so there is nothing to keep in order. The stable version, which moves whole records, is a practice problem (W9-C5).

Measured time



Left: random input. Each doubling of n multiplies the time of the three $O(n^2)$ sorts by about four: bubble sort went from 72 ms at n = 200 to 4.9 s at n = 1,600 — 8 times the data, 68 times the time. Selection sort is the fastest of the three here, although it compares the most: in Python, a comparison is cheap next to the reads and writes of a swap, and selection sort writes almost nothing. Counting sort is a straight line of slope 1 — $O(n)$, 9 ms at n = 1,600. The dashed line is Python's built-in `sorted`, written in C: 0.36 ms. That gap is the language, not the algorithm — Week 10's sorts, in Python, will not reach it either.

Right: the same insertion sort on four inputs of the same size. At n = 1,600, reversed input took 6.0 s, random 2.7 s — half, as the inversion count predicts — and nearly sorted input **6.7**

ms, about 400 times faster than random, and no slower than sorted input. Nearly sorted is its best case in practice, not only in theory.

Your times will differ from these; the ratios will not.

Insertion sort on nearly sorted input

$O(n + I)$: if every value is at most **k** places from home, $I \leq nk$ — **linear** for small k.

Real sorts use it:

- **Timsort** (Python, Java): insertion sort on short runs.
- **Introsort** (C++): quicksort, then insertion sort on small pieces.

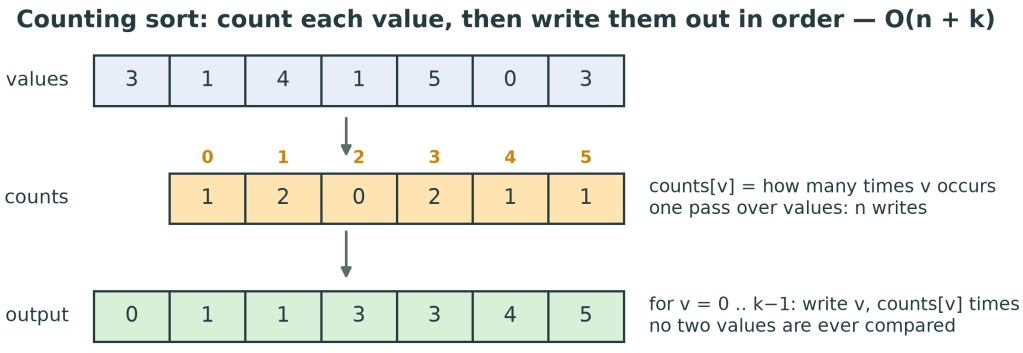
Data in real programs is often nearly sorted: a sorted list with a few new records appended, a log whose timestamps are a little out of order, last week's ranking with a few changes. Insertion sort handles all of these in close to linear time, with no extra memory, and it is stable. It is also fastest of all sorts on **tiny** arrays, because its loop does so little per step.

That is why the $O(n \log n)$ sorts you will meet next week are rarely used alone. Python's `sorted` is **Timsort** (Tim Peters, 2002), a merge sort that first finds the runs already in order and extends short ones with (binary) insertion sort. C++'s `std::sort` is usually **introsort** (David Musser, 1997): quicksort that switches to heap sort if the recursion goes too deep, and finishes small pieces with insertion sort. The "slow" sort of today is inside the fast sorts of the real world.

If each value is at most k places from its sorted position, each has at most k larger values before it, so $I \leq nk$ and insertion sort is $O(nk)$ — a practice problem (W9-C4).

Beyond Comparisons

Counting sort



Values are small non-negative integers, below k: **count** them, then write each value out as many times as it was counted. $O(n + k)$. **No comparisons.**

Counting sort: الترتيب بالع.

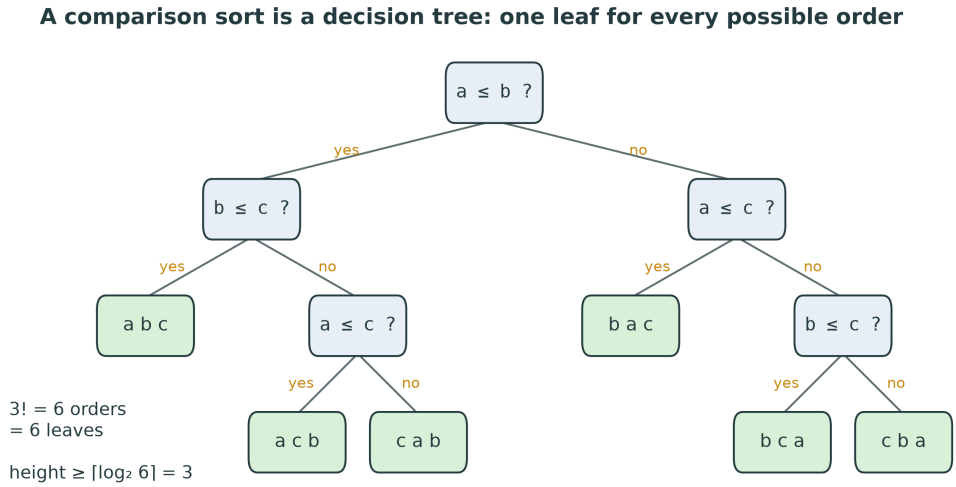
`counting_sort(values, max_value=None)` works in three steps:

1. **k.** With no `max_value`, find the largest value; `k = max + 1` slots, one per possible value 0 .. max. An empty list is sorted already — return `[]` before `max` sees it.
2. **Count.** A course `Array` of k zeros; for each value v, add 1 to `counts[v]`. The value **is** the index — that is the whole trick.
3. **Write out.** For v from 0 to k - 1, write v into the output `counts[v]` times.

Time: one pass over the n values, one over the k counts, and n writes: $O(n + k)$. Extra space: $O(n + k)$ — not in place.

The catch is k. Sorting ten exam marks out of 100 is quick; sorting ten phone numbers means an `Array` of ten billion counters. Counting sort is fast exactly when the **range** of the values is not much larger than their **number**. It also needs keys that can be used as indices: non-negative integers, or things that map to them (letters, grades, ages). The reference raises `ValueError` for anything else.

No comparison sort can beat $n \log n$



n values have **n!** possible orders; the sort must tell them all apart. Each comparison has 2 outcomes. So the worst case needs at least $\log_2 n! \approx n \log_2 n$ comparisons.

Lower bound: الحد الأدن. *Decision tree*: شجرة القرا.

The argument, intuitively. Take any sort that learns about its input **only** by comparing two elements. Draw every comparison it might make as a node, with two branches for the two answers: that is its **decision tree**. The figure is insertion sort's tree for three values a, b and c. A run of the sort on one input is a path from the root to a leaf, and the number of comparisons is the length of the path.

Each leaf must correspond to **one** order of the input. If two different orders reached the same leaf, the sort would do the same moves on both and get one of them wrong. There are $n!$ orders, so the tree needs at least $n!$ leaves. A binary tree of height h has at most 2^h leaves, so $2^h \geq n!$, that is $h \geq \log_2 n!$. The height is the worst-case number of comparisons.

How big is $\log_2 n!$? For 3 values, $\log_2 6 \approx 2.6$: at least **3** comparisons in the worst case, and the tree in the figure achieves it. For 10 values, $\log_2 3{,}628{,}800 \approx 21.8$: at least 22. For a thousand,

about 8,530 — against 499,500 for today's sorts. In general $\log_2 n! = \Theta(n \log n)$: half the factors of $n!$ are at least $n/2$, so $n! \geq (n/2)^{n/2}$, and $\log_2 n! \geq \frac{n}{2} \log_2 \frac{n}{2}$.

So $n \log n$ is a floor for **every** comparison sort, including ones nobody has invented yet. Week 10's merge sort and heap sort reach it; nothing that only compares can go below it. (The same argument also bounds the average case — a stronger result, and beyond this course.)

How counting sort escapes

The lower bound assumes the sort learns **only by comparing**.

Counting sort never compares: `counts[v] += 1` uses the value **as an address** — one step with k possible outcomes, not 2.

This is the honest reading of the lower bound: it is not a law about sorting, it is a law about **comparison** sorting. Counting sort gets more information per step because it assumes more about the data — that the keys are small integers — and uses the key to choose one of k slots at once. The price is the $O(k)$ memory and the restriction on the keys.

Radix sort (not examined in this course) pushes the same idea further: it sorts large integers digit by digit, with a **stable** counting sort per digit, in $O(d(n + b))$ time for d digits in base b. That is why the stable version of counting sort matters, and why W9-C5 asks you to write it.

This Week

Exercises: `dsa/sorting.py`

Function	The trap
`bubble_sort_steps`	yield `list(a)`, not a; `range(end)`; reset `swapped` each pass
`selection_sort_steps`	compare with `a[smallest]`, not `a[i]`; one swap per round
`insertion_sort_steps`	`j >= 0` **first**; `>` not `>=` (stability)
`bubble_sort`, `selection_sort`, `insertion_sort`	exhaust the generator; copy nothing per step
`counting_sort`	`k = max + 1`; the empty list; the value is the index

```
pytest tests/test_sorting.py -v `
    -k "bubble or selection or insertion or counting or is_sorted"
```

Thirty-nine tests (`is_sorted` is given, and its one test already passes). Every sort is held to the same contract by parametrised tests: eight fixed cases (empty, one value, two, duplicates, sorted, reversed, negatives), twenty random lists, "does not mutate the input", and the two tests on the `_steps` form. Merge, quick and heap sort share the file and are Week 10's — the `-k` filter leaves them out.

Store the working copy in a course `Array` (`Array.from_values(values)`), as the storage rule requires; return a list. Do not call `sorted`, `list.sort` or `min` inside `dsa/sorting.py` — the point is to write the sort, not to call one.

Homework 9 — before Lecture 10

1. **Implement** the four sorts until the 39 tests pass.
2. **Trace** each of the three sorts on `[4, 1, 3, 9, 7, 2]`, one row per pass or round, counting comparisons and swaps or shifts.
3. **Count.** Write a class whose < and > count, sort 1,000 of them sorted, reversed and random, and compare with the figure.
4. **Break it.** Change > to >= in your insertion sort. Which tests fail? Now sort `[(1, 'a'), (1, 'b')]` — why can a list of tuples not show the bug?

For item 3, the class needs `__init__`, `__lt__` and `__gt__` (Lab 03, "Special methods"), and a counter they share — a class attribute. The lab walks you through it.

For item 4, look at how tuples compare: `(1, 'a') < (1, 'b')` is `True`. Two tuples that are "equal by their first item" are not equal to Python, so the sort orders them by their second item — which happens to be their arrival order. `test_insertion_sort_is_stable` uses cards that compare by key **only** for exactly this reason.

Summary

Seven things to keep

1. Sorting is what makes Week 8's searches possible — **and its cost decides** whether sort-then-search is worth it.
2. **Stable:** equal keys keep their order. Bubble and insertion are; selection is not.
3. Bubble and insertion sort: $O(n)$ **best,** $O(n^2)$ **worst**. Selection sort: $n(n-1)/2$ comparisons always, but at most $n-1$ swaps.
4. Insertion sort does one shift per **inversion**: $O(n+I)$ — near-linear on nearly sorted data, which is why real sorts use it.
5. A `_steps` generator lets you watch your own code — and a copy per frame turns $O(n^2)$ into $O(n^3)$.
6. Counting sort is $O(n+k)$ with **no comparisons** — fast only when k is small.
7. Every comparison sort needs $\log_2 n! = \Theta(n \log n)$ comparisons in the worst case.

Next

Week 10 — Advanced sorting. Merge sort, quicksort and heap sort: three ways to reach the $n \log n$ floor — and why quicksort's pivot decides between $n \log n$ and n^2.

Sources and further reading

- **D. E. Knuth.** *The Art of Computer Programming*, vol. 3, *Sorting and Searching*, 2nd ed., Addison-Wesley, 1998, §5.1.1 (inversions), §5.2.1 (insertion sort), §5.2.2 (bubble sort), §5.2.3 (selection sort), §5.3.1 (the comparison lower bound).
- **T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein.** *Introduction to Algorithms*, 4th ed., MIT Press, 2022, §2.1 (insertion sort), §8.1 (lower bounds for sorting), §8.2 (counting sort).
- **O. Astrachan.** "Bubble Sort: An Archaeological Algorithmic Analysis", *SIGCSE Bulletin* 35(1), 2003 — where the name and the algorithm came from, and why it survives in teaching.
- **T. Peters.** "listsort.txt", in the CPython source tree — the design of Timsort, including its use of binary insertion sort on short runs.
- **D. R. Musser.** "Introspective Sorting and Selection Algorithms", *Software: Practice and Experience* 27(8), 1997.
- **Python documentation.** "Sorting Techniques" (the Sorting HOW TO) — stability and sorting by several keys.

Every figure in this lecture is generated by `tools/figures_l09.py`. The diagrams are traces computed and checked by the script; the counts and timings are measured on the reference `dsa/sorting.py`, and your timings will differ.

Questions for Week 9

> ***The answers are at the back of the book,*** *in* Answers — Week 9. *Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify. The sorts are those of* `dsa/sorting.py`*:* ***bubble sort*** *compares neighbours and stops after a pass with no swap;* ***selection sort*** *swaps the minimum of* `a[i..n-1]` *into slot i, skipping the swap when it is already there;* ***insertion sort*** *shifts every larger value right and drops* `current` *into the hole, walking left* `while j >= 0 and a[j] > current`*;* `counting_sort` *counts the values 0 .. max. A* ***pass*** *of bubble sort and a* ***round*** *of the other two are one turn of their outer loop. Unless a question says otherwise,* ***A*** *is the lecture's list*
>
> `A = [5, 2, 9, 1, 7, 3]`

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W9-M01 – W9-M22
B	Short answer and essay	W9-E1 – W9-E5
C	Trace the algorithms	W9-T1 – W9-T5
D	Array state — draw every step	W9-S1 – W9-S3
E	Complexity analysis	W9-K1 – W9-K3
F	Find and fix the bug	W9-B1 – W9-B4
G	Write the code — checked by `pytest`	W9-C1 – W9-C5

Part A — Multiple choice

W9-M01 [what] A sorting algorithm is **stable** when:

- **a)** it uses no memory besides the array
- **b)** it takes the same time on every input of size n
- **c)** elements with equal keys keep their original relative order
- **d)** it makes at most n - 1 swaps

W9-M02 [what] A sort is **in place** when:

- **a)** it needs only $O(1)$ extra memory besides the array it sorts
- **b)** it returns the same list object it was given
- **c)** it is stable
- **d)** it runs in $O(n)$ time

W9-M03 [what] Which of these sorts is **not** stable, as written in the course?

- **a)** bubble sort
- **b)** insertion sort
- **c)** merge sort that takes from the left half on a tie
- **d)** selection sort

W9-M04 [how] Bubble sort, with the early exit, on `[1, 2, 3, 4, 5, 6]` makes how many comparisons?

- **a)** 5
- **b)** 15
- **c)** 6
- **d)** 36

W9-M05 [how] Selection sort on `[1, 2, 3, 4, 5, 6]` makes how many comparisons?

- **a)** 5
- **b)** 6
- **c)** 15
- **d)** 0

W9-M06 [how] The array after the **first pass** of bubble sort on A is:

- **a)** `[1, 2, 9, 5, 7, 3]`
- **b)** `[2, 5, 9, 1, 7, 3]`
- **c)** `[2, 1, 5, 3, 7, 9]`
- **d)** `[2, 5, 1, 7, 3, 9]`

W9-M07 [how] The array after the **first round** of selection sort on A is:

- **a)** `[2, 5, 1, 7, 3, 9]`
- **b)** `[1, 2, 9, 5, 7, 3]`
- **c)** `[1, 5, 2, 9, 7, 3]`
- **d)** `[1, 2, 3, 5, 7, 9]`

W9-M08 [how] Insertion sort on A. The array after the round that inserts `a[3]` (the value 1) is:

- **a)** `[2, 5, 9, 1, 7, 3]`
- **b)** `[1, 2, 9, 5, 7, 3]`
- **c)** `[2, 1, 5, 9, 7, 3]`
- **d)** `[1, 2, 5, 9, 7, 3]`

W9-M09 [how] How many **inversions** does `[3, 1, 2]` have?

- **a)** 1
- **b)** 3
- **c)** 2
- **d)** 0

W9-M10 [why] The number of shifts insertion sort makes is always equal to:

- **a)** n - 1
- **b)** the number of inversions of the input
- **c)** $n(n-1)/2$
- **d)** $\lceil \log_2 n! \rceil$

W9-M11 [what] The best case of insertion sort is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n \log n)$
- **d)** $O(n)$, on sorted input

W9-M12 [why] Selection sort is the best of the three basic sorts when:

- **a)** the data is nearly sorted
- **b)** writing an element costs much more than comparing two
- **c)** the sort must be stable
- **d)** the values are small integers

W9-M13 [how] The largest number of swaps selection sort makes on 10 values is:

- **a)** 9
- **b)** 45
- **c)** 10
- **d)** 100

W9-M14 [how] `counting_sort([4, 0, 4, 1])` makes a `counts` array of how many slots?

- **a)** 4
- **b)** 5
- **c)** 3
- **d)** 2

W9-M15 [what] Counting sort on n values from the range 0 .. k - 1 takes:

- **a)** $O(n \log n)$
- **b)** $O(n^2)$
- **c)** $O(n + k)$
- **d)** $O(k \log n)$

W9-M16 [why] Counting sort is a **bad** choice when:

- **a)** the values are small integers
- **b)** n is large
- **c)** the range of the values is much larger than n
- **d)** the input is already sorted

W9-M17 [what] In the worst case, every comparison sort of n values needs at least:

- **a)** $\log_2 n$ comparisons
- **b)** $\log_2 n!$ comparisons
- **c)** n^2 comparisons
- **d)** $n/2$ comparisons

W9-M18 [how] The fewest comparisons that can sort **any** 4 values in the worst case, by comparing, is at least:

- **a)** 4
- **b)** 6
- **c)** 3
- **d)** 5

W9-M19 [why] The decision-tree argument for the lower bound uses the fact that:

- **a)** the tree needs at least $n!$ leaves, and a binary tree of height h has at most 2^h leaves
- **b)** every sort is recursive
- **c)** every comparison halves the array
- **d)** the tree is always balanced

W9-M20 [why] A `_steps` generator yields its working **list itself** (not a copy) before each comparison. Then `frames = list(generator)` holds:

- **a)** the final, sorted state in every frame
- **b)** a `TypeError`
- **c)** the right frames
- **d)** only the first frame

W9-M21 [why] A plain sort runs its `_steps` generator to the end, and the generator copies the array before each of its about $n^2/2$ comparisons. The plain sort costs:

- **a)** $O(n^2)$
- **b)** $O(n^3)$
- **c)** $O(n \log n)$
- **d)** $O(n)$

W9-M22 [how] Bubble sort on `[6, 5, 4, 3, 2, 1]` makes how many swaps?

- **a)** 5
- **b)** 6
- **c)** 15
- **d)** 36

Part B — Short answer and essay

W9-E1 [why] *(4 marks)* Define **stable** and **in place**. Say which of bubble, selection and insertion sort are stable, and why, from their code. Give a small input on which selection sort is not stable, and explain why stability matters when you sort by two keys.

W9-E2 [how] *(4 marks)* Compare bubble, selection and insertion sort: the comparisons they make at best and at worst, and the swaps or shifts. For each, describe a situation in which it is the best of the three.

W9-E3 [why] *(4 marks)* Define an **inversion**. Prove that insertion sort makes exactly one shift per inversion, and use that to give its best, worst and average case. Why do library sorts such as Timsort use insertion sort inside?

W9-E4 [why] *(4 marks)* Explain the `_steps` generator pattern of `dsa/sorting.py`: what it yields and when, how the plain sort reuses it, the two rules its frames must obey, and what "the price of watching" is and how it is fixed.

W9-E5 [why] *(4 marks)* State the lower bound for comparison sorting and give the decision-tree argument for it. Explain why counting sort does not contradict it, and when counting sort is the right choice.

Part C — Trace the algorithms

W9-T1 [how] Trace bubble sort (with the early exit) on `[3, 8, 1, 6, 2]`: the array after each pass, and the comparisons and swaps of each pass.

W9-T2 [how] Trace selection sort on `[29, 10, 14, 37, 13]`: for each round, the index of the minimum, whether it swaps, and the array after the round.

W9-T3 [how] Trace insertion sort on `[8, 4, 6, 2, 9, 1]`: the array after each round, with its shifts and comparisons. Check the total shifts against the number of inversions.

W9-T4 [why] How many frames does each reference `_steps` generator yield on `[5, 4, 3, 2, 1]` — bubble, selection and insertion? Explain each count from where the `yield` statements are.

W9-T5 [how] Trace `counting_sort([2, 5, 3, 0, 2, 3, 0, 3])`: k, the `counts` array, and the output. Then give the **starting positions** that the stable version (W9-C5) computes from the counts, and say where the first 3 and the last 3 of the input are placed.

Part D — Array state

W9-S1 [how] Draw the array after every pass of bubble sort on `[6, 5, 3, 1, 8, 7, 2, 4]`, shading the values that are in their final place. How many passes run, and how many comparisons and swaps in total?

W9-S2 [how] Draw every frame that the reference `insertion_sort_steps` yields on `[5, 1, 4, 2, 8]`, with its highlight. Why does each frame show a permutation of the input — no value twice?

W9-S3 [how] The Dutch national flag partition (W9-C3) keeps three indices: `values[:lo]` are 0, `values[lo:mid]` are 1, `values[hi+1:]` are 2, and `values[mid..hi]` is not yet seen. At each step it looks at `values[mid]`: a 0 is swapped to `lo` (both advance), a 1 is left (`mid` advances), a 2 is swapped to `hi` (`hi` retreats, `mid` stays). Starting from `lo = mid = 0`, `hi = 5`, draw the array and the three indices after each step on `[2, 0, 2, 1, 1, 0]`.

Part E — Complexity analysis

W9-K1 [how] Give the best and worst case, in Θ, and justify:

```
def has_duplicates(values):
    s = insertion_sort(values)
    for i in range(len(s) - 1):
        if s[i] == s[i + 1]:
            return True
    return False
```

What would it be with `counting_sort`, for values in 0 .. k - 1? And with two nested loops comparing every pair?

W9-K2 [why] Derive the exact number of comparisons selection sort makes on n values, and the largest number of swaps. Measured on n = 1,000 random values, selection sort made 499,500 comparisons and 1,986 writes, and insertion sort 249,614 comparisons and 498,241 writes. If a write costs 100 times as much as a comparison, which is cheaper, and by how much?

W9-K3 [why] Every value of a list is at most k places from its sorted position. Show that insertion sort sorts it in $O(nk)$. What does that give for k = 1, and for k = n?

Part F — Find and fix the bug

W9-B1 [how]

```
def bubble_sort_steps(values):
    a = Array.from_values(values)
    n = len(a)
    yield list(a), ()
    for end in range(n - 1, 0, -1):
        swapped = False
        for j in range(end + 1):
            yield list(a), (j, j + 1)
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            break
```

```
    yield list(a), ()
```

W9-B2 [why] It sorts, and it passes all twelve selection tests. What is wrong with it? Count its swaps on `[6, 5, 4, 3, 2, 1]`.

```
def selection_sort(values):
    a = Array.from_values(values)
    n = len(a)
    for i in range(n - 1):
        for j in range(i + 1, n):
            if a[j] < a[i]:
                a[i], a[j] = a[j], a[i]
    return list(a)
```

W9-B3 [why] It passes every test except one. Which one, and why?

```
def insertion_sort(values):
    a = Array.from_values(values)
    for i in range(1, len(a)):
        current = a[i]
        j = i - 1
        while j >= 0 and a[j] >= current:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = current
    return list(a)
```

W9-B4 [how]

```
def counting_sort(values):
    k = max(values)
    counts = Array(k, fill=0)
    for v in values:
        counts[v] += 1
    out, i = Array(len(values)), 0
    for v in range(k):
        for _ in range(counts[v]):
            out[i] = v
            i += 1
    return list(out)
```

Part G — Write the code

In `practice/week09.py`; check with `pytest tests/test_practice_week09.py -v`. Do not call `sorted`, `list.sort`, `min` or `max` on the values. The tests count comparisons (C4) and reads

and writes (C3), so the stated cost is checked.

W9-C1 [how] `count_inversions(values)` — the number of pairs in the wrong order, in $O(n^2)$. Why does it equal the number of swaps bubble sort makes?

W9-C2 [why] `sort_by_key(items, key)` — a stable insertion sort that compares only `key(item)`. Why must it not compare the items themselves?

W9-C3 [why] `dutch_flag(values)` — sort 0s, 1s and 2s in place in one pass. Why does `mid` not advance after swapping a 2 to the back?

W9-C4 [how] `sort_k_sorted(values, k)` — every value is at most k places from home; sort in $O(nk)$.

W9-C5 [why] `counting_sort_by_key(items, key, k)` — a **stable** counting sort of records, $O(n+k)$, with no comparisons. Why must the placing loop go through the items in input order?

Week 10 — Advanced Sorting

> ***How to read this document.*** *This is the handout for Lecture 10. It holds everything on the slides, plus what I said out loud. Last week's three sorts all cost $O(n^2)$. This week, three sorts that cost $O(n \log n)$ — merge sort, quicksort and heap sort — and one idea behind the first two:* ***divide and conquer****. Every trace in this handout was produced by running the reference code, and every number in the measured section was counted, not estimated.*
>
> *Slides:* `DSA27-L10-slides.pdf` *· Code:* `dsa/sorting.py` *(the second half) · Tests:* `tests/test_sorting.py -k "merge or quick or heap"`

Where We Are

From n^2 to $n \log n$

Week 9: bubble, selection, insertion — $O(n^2)$ comparisons.

n	$n^2/2$	$n \log_2 n$
1,000	500,000	10,000
1,000,000	500,000,000,000	20,000,000

Today: three sorts on the right-hand column — and the price each one pays.

At a million values, an $O(n^2)$ sort does about 25,000 times more work than an $O(n \log n)$ one. Nothing you can do to the constant closes a gap like that; only a different algorithm does. Week 9's sorts all move an element by **one** place per comparison, or compare every pair; this week's sorts each find a way to let one comparison settle the order of many elements at once.

Sorting in Arabic: ﷺالترتي﷾. *Merge sort*: ﷺالترتيب بالدم﷾. *Quicksort*: ﷺالترتيب السري﷾. *Heap sort*: ﷺالترتيب بالكوم﷾.

Today

1. Divide and conquer
2. Merge sort: $T(n) = 2T(n/2) + n$, stable, $O(n)$ extra space
3. Quicksort: partition, pivots, and a worst case you will **measure**
4. Heap sort: the array is the tree — Week 12 arriving early
5. Measured: all six sorts, one picture
6. What Python's `sorted()` actually does
7. Stability, and the table to remember

Divide and Conquer

Three steps

1. **Divide** the problem into smaller problems of the same kind.
2. **Conquer** each one — recursively, until it is trivial.
3. **Combine** their answers into the answer.

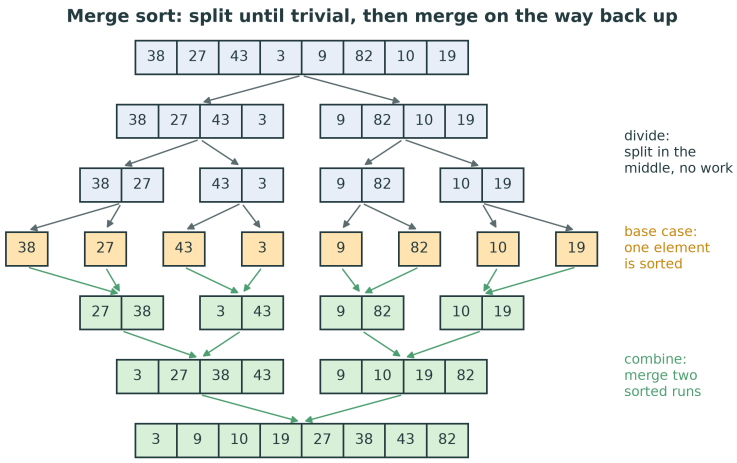
Merge sort: divide is free, **combine** does the work. Quicksort: **divide** does the work, combine is free.

Divide and conquer in Arabic: فرق تس. You have met the pattern twice. Lecture 03's recursions split a problem and trusted the recursive call to solve the smaller part; Lecture 08's binary search is divide and conquer with only **one** of the two halves kept, which is why it costs log n and not n. This week both halves are kept, and each has to be sorted.

The two sorts in this lecture are mirror images, and that is worth saying out loud before any code. Merge sort splits the array in the middle without looking at it, and does all its thinking when it **merges** two sorted halves. Quicksort thinks first — it **partitions** the array into "small" and "large" around a pivot — and then there is nothing left to combine: small, pivot, large is already in order.

Merge Sort

Split until trivial, merge on the way up

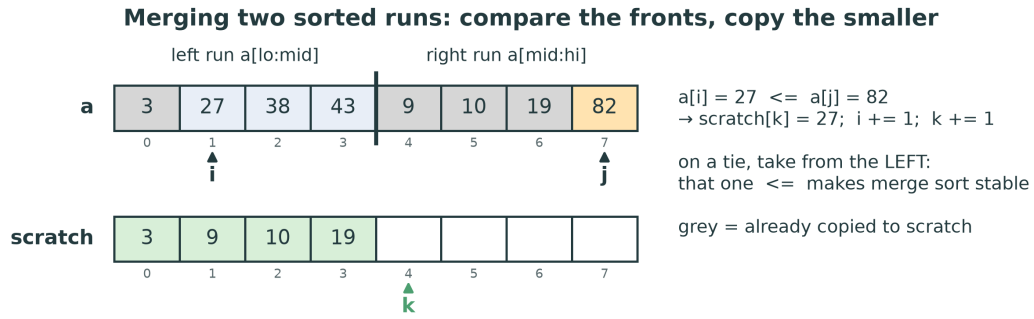


The figure is `merge_sort([38, 27, 43, 3, 9, 82, 10, 19])`. Going down, each range is cut at `mid = (lo + hi) // 2` — no comparisons, no moves. At the bottom, every range holds one element, and **a one-element list is sorted**: that is the base case. Going up, each pair of neighbouring sorted runs is merged into one sorted run, until the whole array is one run. The merged rows are the real states yielded by `merge_sort_steps`, taken from the reference code; the order of the seven merges is

```
a[0:2]  a[2:4]  a[0:4]  a[4:6]  a[6:8]  a[4:8]  a[0:8]
```

— left half first, completely, then the right half, then the two together: the order of a recursive call tree (Lecture 03).

Merging two sorted runs



Compare the two **fronts**; copy the smaller to `scratch`; advance that side. When one run is empty, copy the rest of the other.

You have written this twice already. In Week 2, `merge_sorted(a, b)` in `dsa/array_ops.py` merged two `Array`s into a new one with a loop; in Week 3, `merge_sorted(left, right)` in `dsa/recursion.py` did it recursively, and its docstring promised: *"This is the merge step of merge sort."* (Lecture 03, "Merging: recursion on two inputs"). Lecture 03 also warned that the recursive version is `n + m` frames deep, too deep for Python on long lists — and "a good reason why merge sort's real merge step, in Week 10, is a loop". Here it is.

Two runs of lengths n_1 and n_2 merge in at most $n_1 + n_2 - 1$ comparisons, because each comparison sends exactly one element to `scratch`, and the last element needs no comparison at all. In the figure, the left run is 3 27 38 43 and the right run is 9 10 19 82. Four elements have gone (3, 9, 10, 19); now 27 is compared with 82, and 27 goes.

`_merge`: the code

```
def _merge(a, lo, mid, hi, scratch):
    i, j, k = lo, mid, lo
    while i < mid and j < hi:
        if a[i] <= a[j]:
            scratch[k] = a[i]
            i += 1
        else:
            scratch[k] = a[j]
            j += 1
        k += 1
    while i < mid:
        scratch[k] = a[i]
        i += 1
        k += 1
```

```
    while j < hi:
        scratch[k] = a[j]
        j += 1
        k += 1
    for k in range(lo, hi):
        a[k] = scratch[k]
```

This is `_merge` from the reference solution, exactly (without its docstring). The runs are `a[lo:mid]` and `a[mid:hi]` — half-open ranges, as in Lecture 08's bounds. `i` walks the left run, `j` the right run, and `k` the next free slot of `scratch`. After the main loop, **one** of the two runs is exhausted and the other still has a sorted tail; only one of the two "leftover" loops does anything. The last loop copies the merged range back into `a`, because the next merge up the tree reads from `a`.

The `<=` is the whole of stability. When `a[i]` and `a[j]` are equal, `<=` takes from the **left** run — the element that came first in the input. Write `<` and the sort is still correct, and every test that compares numbers still passes; but equal keys come out in the wrong order. See "Stability" below.

`merge_sort_steps`: divide, conquer, combine

```
a = _copy(values)
scratch = Array(len(a))
yield snapshot(a), ()

def sort(lo, hi):
    if hi - lo <= 1:
        return
    mid = (lo + hi) // 2
    yield from sort(lo, mid)
    yield from sort(mid, hi)
    _merge(a, lo, mid, hi, scratch)
    yield snapshot(a), tuple(range(lo, hi))

yield from sort(0, len(a))
yield snapshot(a), ()
```

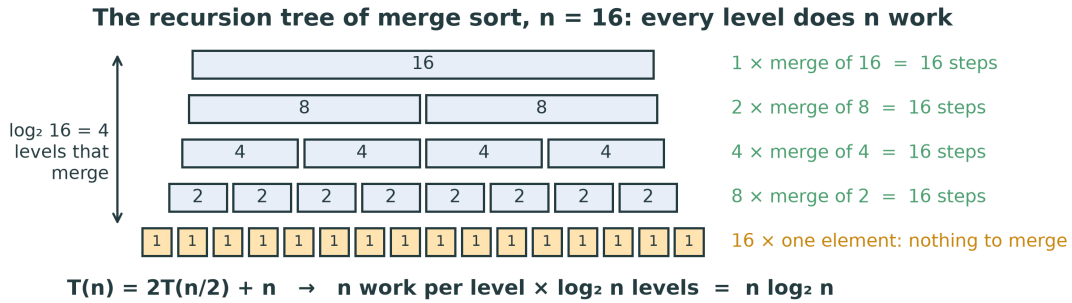
The body of `merge_sort_steps` in the reference. `sort(lo, hi)` sorts `a[lo:hi]`; ranges of 0 or 1 elements are the base case. The three steps of divide and conquer are the three lines in the middle: compute `mid`, sort both halves, merge.

Why `yield from`? `sort` is itself a generator — it yields a snapshot after each merge — so calling `sort(lo, mid)` only **creates** a generator and runs nothing. `yield from` runs it and passes each of its snapshots up to whoever is iterating `merge_sort_steps`. Forget it, write `sort(lo, mid)` on its own line, and nothing below the top level is ever sorted: the sort returns the input barely changed, and the tests fail.

One scratch `Array`, shared by every merge. Allocating a new scratch array in every call would also be correct, but n calls each allocating up to n slots is a lot of memory churn. One `Array(len(a))`, made once, is enough, because a merge of `a[lo:hi]` only ever uses `scratch[lo:hi]`. The extra space is therefore exactly n slots, plus the call stack, which is only $\log_2 n$ frames deep because the ranges halve.

The plain form, `merge_sort(values)`, runs this generator to the end and returns the last state — one implementation for the tests and the animation.

How much work? $T(n) = 2T(n/2) + n$



$\log_2 n$ levels, n work per level: $O(n \log n)$ — best, average and worst case alike.

A **recurrence** (Lecture 03) states the cost of a recursive function in terms of its own cost on smaller inputs. For merge sort on n elements: two sorts of n/2 elements, plus a merge that touches all n. So $T(n) = 2T(n/2) + cn$ for some constant c, with $T(1) = c$.

The recursion tree solves it by picture. The top call merges n elements. Its two children merge n/2 each — n in total. The four grandchildren merge n/4 each — n again. Every level adds up to n, because the ranges on one level partition the array. The ranges halve at each level, so after $\log_2 n$ levels they have one element and the recursion stops. Total: $n \cdot \log_2 n$. *Recursion tree* in Arabic: شجرة الاستدعاءات.

By unrolling, if you prefer algebra: $T(n) = 2T(n/2) + n = 4T(n/4) + 2n = 8T(n/8) + 3n = \cdots = 2^k T(n/2^k) + kn$. Stop when $n/2^k = 1$, that is $k = \log_2 n$: $T(n) = nT(1) + n \log_2 n = O(n \log n)$.

Always $n \log n$: merge sort splits in the middle whatever the values are, so no input is bad for it. Sorted input only saves comparisons inside each merge — the left run empties first, after $n/2$ comparisons instead of about n — measured: 5,120 comparisons for 1,024 sorted values against 8,935 for random ones. The **moves** are the same either way: every level copies all n elements to `scratch` and back.

The price: $O(n)$ extra space

- Merge sort is **not in place**: `scratch` holds n slots.
- The call stack adds $O(\log n)$.
- Fine for n = 10^6 numbers; a real cost on a small device, or for data that barely fits in memory.

Merging two runs in place, without a second array, is possible but complicated and slower in practice; nobody teaches it in a first course, and the standard libraries do not do it. The honest summary is: merge sort buys a guaranteed $O(n \log n)$ and stability with n extra slots. Quicksort and heap sort, next, work inside the array itself.

The same merge is also how you sort data **too big for memory**: sort chunks that fit, write each sorted run to disk, then merge the runs, reading each one front to back. That is *external sorting*, and it is still merge sort — only the runs live in files.

Quicksort

Partition, then recurse

Pick a **pivot**. Rearrange so that

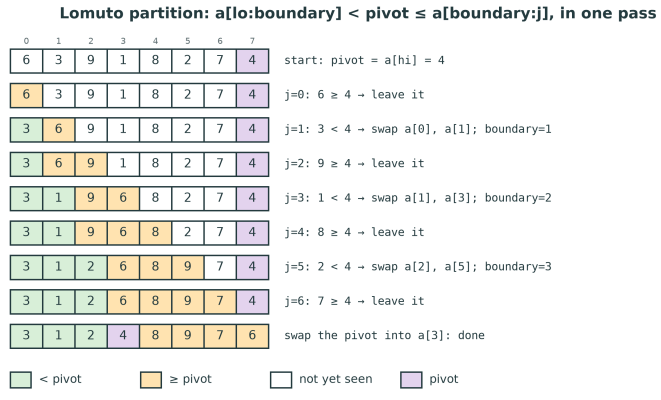
$$\underbrace{< \text{pivot}}_{\text{left part}} \quad \text{pivot} \quad \underbrace{\geq \text{pivot}}_{\text{right part}}$$

The pivot is now in its **final place**. Sort the two parts; nothing to combine.

C. A. R. Hoare invented quicksort in 1959 and published it in 1961 (“Algorithm 64: Quicksort”, *Communications of the ACM* 4(7)). *Pivot* in Arabic: المحو. *Partition*: التقسيم.

After a partition, the pivot never moves again: everything smaller is to its left and everything larger or equal is to its right, and sorting those two parts cannot change that. So each partition fixes **one** element for good and splits the rest into two smaller, independent problems. How much smaller depends entirely on the pivot — and that is the story of quicksort.

Lomuto’s partition



There are two classic partition schemes. Hoare’s original moves two indices towards each other; the one in `dsa/sorting.py` is **Lomuto’s**, popularised by Jon Bentley in *Programming Pearls* because it is the easier of the two to get right. It uses the **last** element as the pivot, one index `j` that scans left to right, and one `boundary`:

- `a[lo:boundary]` holds values **< pivot** (green in the figure);

- `a[boundary:j]` holds values $\geq$ **pivot** (amber);
- `a[j:hi]` has not been looked at yet (white);
- `a[hi]` is the pivot.

Each step looks at `a[j]`. If it is at least the pivot, the amber region simply grows by one: nothing moves. If it is smaller, it is swapped with the first amber value, `a[boundary]`, and `boundary` moves right: the green region grows by one and the amber region shifts along. At the end, one last swap puts the pivot between the two regions, at `boundary`.

The figure traces `Q = [6, 3, 9, 1, 8, 2, 7, 4]`. The pivot is 4; the values 3, 1 and 2 are smaller and end up in `a[0:3]`; the pivot lands at index 3. The partition made 7 comparisons, one per non-pivot element: $n - 1$, always.

`_partition`: the code

```
def _partition(a, lo, hi):
    pivot = a[hi]
    boundary = lo
    for j in range(lo, hi):
        if a[j] < pivot:
            a[boundary], a[j] = a[j], a[boundary]
            boundary += 1
    a[boundary], a[hi] = a[hi], a[boundary]
    return boundary
```

Invariant: `a[lo:boundary] < pivot <= a[boundary:j]`.

Note the ranges: `lo` and `hi` here are **inclusive**, `a[lo..hi]`, with the pivot at `hi` — unlike `_merge`, whose `hi` is excluded. Both are fine; what is not fine is mixing them up within one function. The invariant is true before the loop (both regions are empty), each step keeps it true, and at the end `j == hi`, so the whole range except the pivot has been sorted into the two regions. The final swap moves the first amber value to the end — which keeps it amber — and puts the pivot at `boundary`.

`a[j] < pivot` is strict: values **equal** to the pivot go right. With many equal values that is a weakness — a list of a thousand equal values partitions into 0 and 999, the worst case again. Industrial quicksorts use a three-way partition (<, =, >) for this reason; it is not required here.

Choosing the pivot

```
def _choose_pivot(a, lo, hi, strategy, rng):
    if strategy == "first":
        return lo
    if strategy == "last":
        return hi
    if strategy == "random":
        return rng.randint(lo, hi)
```

```
    if strategy == "median3":
        mid = (lo + hi) // 2
        first, middle, last = a[lo], a[mid], a[hi]
        if first <= middle <= last or last <= middle <= first:
            return mid
        if middle <= first <= last or last <= first <= middle:
            return lo
        return hi
    raise ValueError(...)
```

The chosen pivot is swapped to `a[hi]`; then `_partition` runs as before.

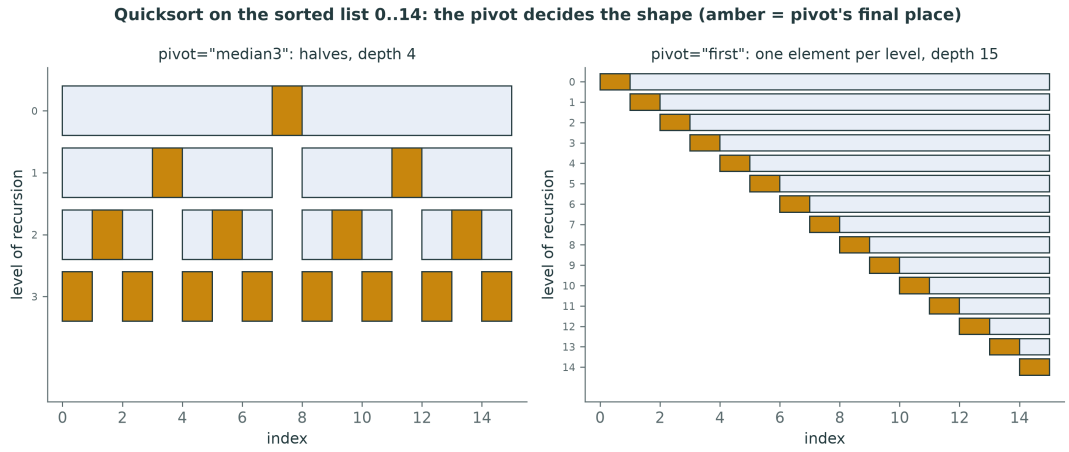
`quick_sort(values, pivot="median3")` takes the strategy by name, and the reference passes it to this function (its `raise` line is shortened here). The index it returns is swapped with `a[hi]`, so that `_partition` never needs to know how the pivot was chosen:

```
p = _choose_pivot(a, lo, hi, pivot, rng)
a[p], a[hi] = a[hi], a[p]
q = _partition(a, lo, hi)
```

- **first / last**: free, and fine on random data. On sorted data, a disaster — next slide.
- **random**: a random index in `lo..hi`. No fixed input is bad for it; a bad run needs bad luck at every level. The reference seeds its generator (`random.Random(27)`) so that the animation is the same every time.
- **median3**: the median of the first, middle and last values — R. C. Singleton's suggestion (1969). Up to three extra comparisons per partition buy a pivot that is never the smallest or largest of those three; on sorted input it is the exact middle.

The median-of-three is written with comparisons, not with `sorted()`: `dsa/sorting.py` does not call the library's sort to write its own.

The worst case, measured



Sorted input, `pivot="first"`: every partition peels off **one** element. $(n-1)+(n-2)+\cdots+1 =$

$n(n-1)/2$ comparisons: $O(n^2)$.

On `[0, 1, ..., 14]` with `pivot="first"`, the pivot is the smallest value in its range every time. The "small" part is empty, the "large" part has all the rest, and the next partition is only one element shorter. Fifteen levels for fifteen elements, against four for `median3`, which picks the exact middle of each sorted range and halves it.

Counted, not guessed. The reference `quick_sort(list(range(n)), pivot="first")` makes exactly $n(n-1)/2$ comparisons: 523,776 for n = 1,024, and 2,096,128 for n = 2,048 — four times as many for twice the input, the signature of n^2. With `pivot="median3"` the same inputs cost 9,228 and 20,493 comparisons. `pivot="last"` is just as bad as "first" on sorted input (and on reversed input, "first" and "last" swap roles).

This is not a curiosity. Sorted and nearly sorted data are the **most common** inputs a real sort receives — a list sorted yesterday with a few new entries today. A quicksort that picks the first element is at its worst on exactly the data it sees most. That is the single best argument for randomisation in this course: a random pivot makes the running time depend on the coin, not on the input.

Why the average is $O(n \log n)$

- A pivot anywhere in the **middle half** splits no worse than 1/4 : 3/4.
- Then the largest part shrinks by 3/4 per level: about $\log_{4/3} n$ levels of $O(n)$ work.
- Half of all pivots are in the middle half. **Random pivots: expected** $O(n \log n)$, about $1.39\, n \log_2 n$ comparisons.

The intuition, not the proof. Call a pivot *good* if it lands in the middle half of its range, so that neither part holds more than three quarters of it. After a good pivot, the larger part has at most $\frac{3}{4}$ of the elements, so a chain of good pivots reaches size 1 after $\log_{4/3} n \approx 2.4 \log_2 n$ steps. A random pivot is good half the time, so on average it takes about two partitions to get one good one: about $2 \times 2.4 \log_2 n$ levels, each doing at most n comparisons. That is still $O(n \log n)$ — a bigger constant, not a bigger class. The exact analysis (Hoare's; see Sedgewick and Wayne, *Algorithms*, §2.3) gives $2n \ln n \approx 1.39\, n \log_2 n$ expected comparisons.

In practice quicksort is often the **fastest** of the three: its inner loop is one comparison and at most one swap, it works in place, and it scans memory in order, which caches love. That is why it is called quick, despite a worst case that is worse than merge sort's.

Stack depth: recurse on the smaller side

```
def sort(lo, hi):
    while lo < hi:
        p = _choose_pivot(a, lo, hi, pivot, rng)
        a[p], a[hi] = a[hi], a[p]
        q = _partition(a, lo, hi)
        yield snapshot(a), (q,)
        if q - lo < hi - q:
            yield from sort(lo, q - 1)
```

```
            lo = q + 1
        else:
            yield from sort(q + 1, hi)
            hi = q - 1
```

The **smaller** part is a call; the **larger** part is the next turn of the loop. Stack depth: $O(\log n)$, even in the worst case.

The inner function of `quick_sort_steps`, exactly. The obvious version — partition, then two recursive calls — has a hidden cost. On sorted input with `pivot="first"` its recursion is n deep, one frame per peeled-off element, and Python stops at 1,000 frames: the obvious quicksort crashes with `RecursionError` on a sorted list of a few thousand elements, which is exactly when it is also at its slowest. `test_quick_sort_survives_its_worst_case_input` sorts `list(range(200))`, but with the default pivot and well under the limit, so it cannot catch this; Lab 10 has you try `pivot="first"` on 1,000 values.

The fix is a classic (it is in Sedgewick's *Algorithms*): make a recursive call only for the **smaller** part, and handle the larger part by updating `lo` or `hi` and going round the `while` loop again — the tail call turned into a loop, as in Lecture 03. Each call is on a range at most half the size of its caller's, so the stack is at most $\log_2 n$ frames deep, whatever the pivots. On sorted input with `pivot="first"` the smaller part is always **empty**, so the stack never grows at all. The time is still $O(n^2)$ on that input; only the space is fixed.

Extra space, then: $O(\log n)$ for the stack, nothing else. Quicksort is in place.

Quicksort is not stable

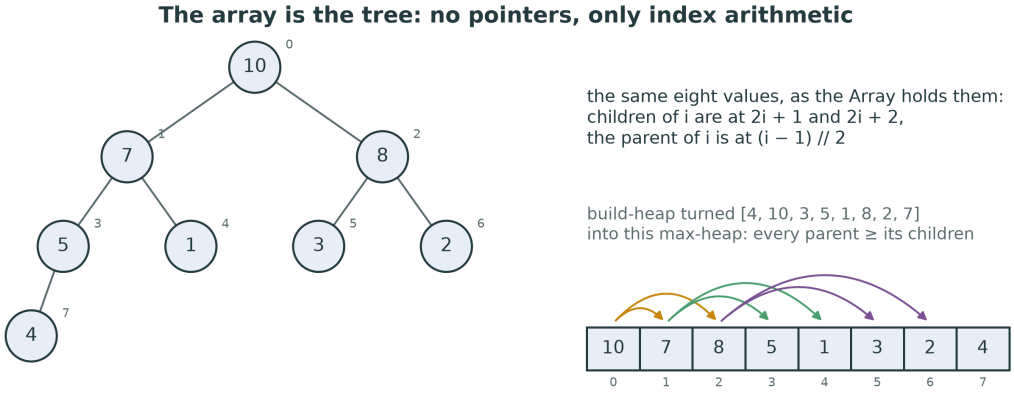
The partition swaps across long distances: an element can jump over its equal.

`quick_sort` on `[3a, 1b, 3c, 2d, 1e, 2f]` (sorted by number) gives `[1b, 1e, 2d, 2f, 3c, 3a]` — **3c before 3a**.

The letters are there only to tell equal numbers apart: the sort compares the numbers alone. The first partition picks the median of 3a, 3c and 2f, which is 3c (the middle one), and swaps it to the end: `[3a, 1b, 2f, 2d, 1e, 3c]`. The four smaller values are swapped to the front one by one, pushing 3a along, and the final swap puts the pivot 3c at index 4 — and 3a at index 5, behind its equal. Nothing in the rest of the algorithm puts them back. The stability figure below shows the whole comparison.

Heap Sort

The array is the tree



A **max-heap**: every parent $\geq$ its children. So the maximum is at `a[0]`.

This is Week 12 arriving early; there it becomes a priority queue. Any array can be **read** as a complete binary tree: `a[0]` is the root, and the children of `a[i]` are `a[2i + 1]` and `a[2i + 2]`. The tree is not stored anywhere — there are no nodes and no pointers, only index arithmetic. `viz.draw.draw_array_as_tree` draws any list this way. *Heap* in Arabic: الكوم.

A **max-heap** is an array whose tree has the heap property: every parent is at least as large as each of its children. That does **not** make it sorted — in the figure, 5 is below 7 but 8 is not below 7 — but it does put the maximum at the root, where it can be read in $O(1)$.

The tree is **complete**: every level full except possibly the last, which fills from the left. So its height is $\lfloor \log_2 n \rfloor$, and anything that walks from the root to a leaf costs $O(\log n)$.

Sift down: repair one parent

```
def _sift_down(a, index, size):
    while True:
        largest = index
        left, right = 2 * index + 1, 2 * index + 2
        if left < size and a[left] > a[largest]:
            largest = left
        if right < size and a[right] > a[largest]:
            largest = right
        if largest == index:
            return
        a[index], a[largest] = a[largest], a[index]
        index = largest
```

`_sift_down(a, index, size)` assumes that both subtrees of `index` are already heaps, and only `a[index]` may be too small. It swaps `a[index]` with its larger child until it is at least as large as both children, or has none. **The larger child**, not just any larger one: otherwise the child that moves up could be smaller than its new sibling. `size` says where the heap ends, so that the

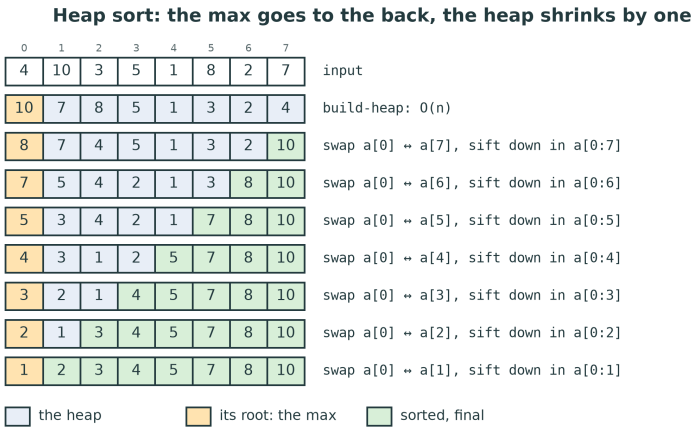
sorted part at the back of the array is never touched. Each swap goes one level down: at most $\log_2 n$ swaps, $O(\log n)$.

Build-heap in $O(n)$, then sort

```
for index in range(n // 2 - 1, -1, -1):        # build the heap: O(n)
    _sift_down(a, index, n)
for end in range(n - 1, 0, -1):
    a[0], a[end] = a[end], a[0]                  # the max goes to the back
    _sift_down(a, 0, end)                       # restore the heap in a[:end]
```

1. **Build** a max-heap: sift down every parent, last to first.
2. **Sort**: swap the max to the back; the heap shrinks by one; sift down.

Heap sort, traced



The two loops of `heap_sort_steps` (the reference also yields a snapshot after the first loop and after each turn of the second).

Build-heap. The leaves — the last $\lceil n/2 \rceil$ slots — are one-element heaps already. Sift down every parent, from the last one, `n // 2 - 1`, back to the root: when a node is sifted, its subtrees are already heaps, which is exactly what `_sift_down` needs. On `H = [4, 10, 3, 5, 1, 8, 2, 7]`, the sifts at indices 3, 2, 1 and 0 give `[10, 7, 8, 5, 1, 3, 2, 4]` — the heap in the previous figure.

Why $O(n)$ and not $O(n \log n)$? n/2 sifts, each up to $\log n$ deep, suggests $O(n \log n)$ — true, but not tight. Most nodes are near the bottom, where a sift is short: about n/4 nodes can sink 1 level, n/8 can sink 2, n/16 can sink 3, and only the root can sink $\log_2 n$. The total is at most $n\,(1/4 + 2/8 + 3/16 + \dots) = n$ swaps. This is R. W. Floyd's improvement (1964) on J. W. J. Williams's original heapsort of the same year, and you will prove it properly in Week 12.

Sort. The maximum is at `a[0]`. Swap it with the last slot of the heap, `a[end]`, where it belongs for good; the heap is now one shorter, and only its new root may break the heap property — one `_sift_down`, $O(\log n)$. Repeat n – 1 times: $O(n \log n)$. The sorted part grows from the back,

like selection sort's — heap sort **is** selection sort, with a heap to find the maximum in $O(\log n)$ instead of $O(n)$.

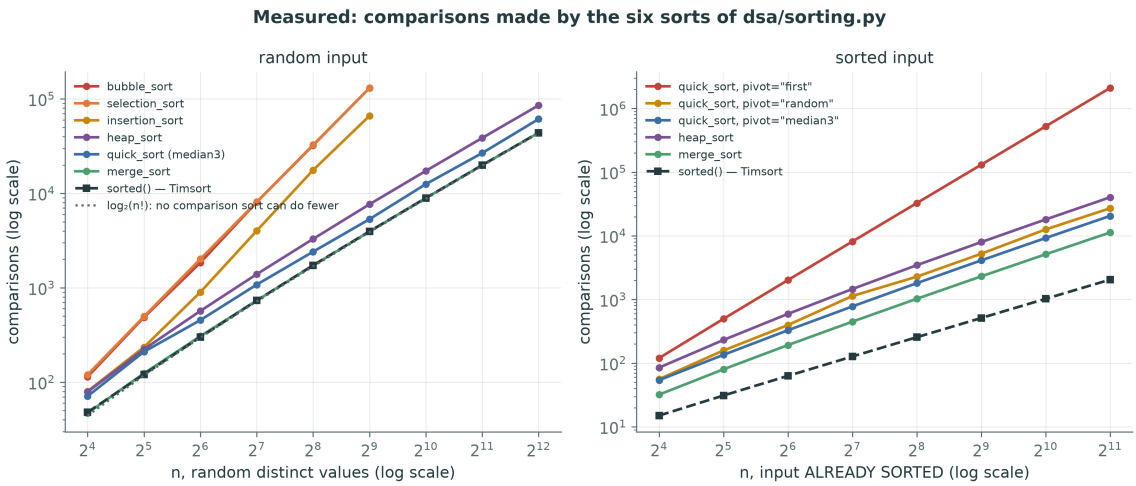
Heap sort: the trade-offs

- $O(n \log n)$ **always** — no bad input, like merge sort.
- $O(1)$ **extra space** — in place, like quicksort; no recursion at all.
- **Not stable**: the swap to the back jumps over equals.
- Slower in practice: it jumps around memory ($i \to 2i + 1$).

On paper heap sort has the best of both: merge sort's guarantee and quicksort's memory. In practice it is usually the slowest of the three, because its sift reads `a[i]` and then `a[2i + 1]`, far away for large i, so the processor's caches help it little; and it makes more comparisons (measured below: about 1.7 n $\log_2$ n on random data). Its real job is as a **safety net**: C++'s `std::sort` is *introsort* (Musser, 1997), a quicksort that switches to heap sort when its recursion gets suspiciously deep, so the worst case stays $O(n \log n)$.

Measured

Six sorts, one picture



Each point is a **count** of comparisons, made by the reference `dsa/sorting.py` on values wrapped in a small class that adds 1 to a counter inside `__lt__`, `__le__`, `__gt__` and `__ge__` — the same trick as `CountingReads` in Lab 08. Counts do not depend on your machine.

Left, random input. The three $O(n^2)$ sorts climb with slope 2 on the log–log plot; the three $O(n \log n)$ sorts climb with slope just over 1. At n = 512, bubble sort made 130,680 comparisons and merge sort 3,945 — 33 times fewer — and the gap doubles each time n doubles. Among the fast three, at n = 4,096: merge sort 43,943, quicksort (median of three) 61,330, heap sort 85,710. The dotted line is $\log_2(n!)$, the fewest comparisons **any** comparison sort can guarantee (see "Can anything beat $n \log n$?" below), and merge sort sits almost on it. So does `sorted()`: 43,939.

Right, sorted input. `pivot="first"` is the red line with slope 2 — $n(n-1)/2$ exactly. `random`

and `median3` stay with the $n \log n$ group. Merge sort needs only $\frac{1}{2} n \log_2 n$ here (5,120 at n = 1,024), and heap sort does not care: it makes a little more than on random data. And Python's `sorted()` makes **n – 1** comparisons — 1,023 at n = 1,024 — because it notices that the input is one sorted run and stops. That is the next section.

Why count instead of time? Because timing the course code can measure something else. `merge_sort` is `merge_sort_steps` run to the end, and a frame made with `list(a)` copies the whole array: after each of the n – 1 merges, n values — $O(n^2)$ copying in total, more than the sort itself. With that version, `merge_sort` took 0.36, 1.29 and 4.7 seconds for 1,000, 2,000 and 4,000 values on this machine: four times as long for twice the input, which is what n^2 looks like. So every `_steps` form in the reference takes a `snapshot` parameter — `list` by default, so the animation gets fresh frames — and the plain form passes a function that returns a itself, uncopied. That is the `snapshot(a)` in the listings above. With it, the same runs took 0.06, 0.12 and 0.27 seconds: a little over twice the time for twice the input, the $n \log n$ shape. Counting comparisons is still the cleaner measure, because a time includes the interpreter and whatever else the machine is doing; Lab 10, Part 8, has you measure both versions. (Python's `sorted()`, written in C, sorts 4,000 values in under a millisecond.)

Can anything beat $n \log n$?

A comparison sort must tell apart all **n!** orders of its input. Each comparison has two outcomes, so it needs at least $\log_2(n!) \approx n \log_2 n - 1.44\, n$ comparisons in the worst case.

Merge sort and heap sort are optimal — up to a constant.

Think of any comparison sort as a game of twenty questions: each comparison is a yes/no question about the input, and the sort must ask enough to know which of the $n!$ possible orders it has been given — otherwise two different inputs get the same treatment and one of them comes out unsorted. k yes/no answers can tell apart at most 2^k cases, so $2^k \geq n!$, i.e. $k \geq \log_2(n!)$. By Stirling's formula that is about $n \log_2 n - 1.44n$. For n = 1,024: $\log_2(1024!) \approx 8{,}769$, and merge sort used 8,935 on random input.

The bound is about **comparison** sorts only. Counting sort (Week 9) never compares two elements — it uses the values as indices — and runs in $O(n+k)$. That is not a contradiction; it is a different game.

What Python Actually Uses

`sorted()` and `list.sort()`: Timsort

Enrichment — not examinable.

1. Find the **runs** already in the data (reversing descending ones).
2. Extend short runs to 32–64 elements with **insertion sort**.
3. **Merge** the runs, like merge sort — stable, $O(n)$ extra space.

Sorted input: one run, **n – 1 comparisons**. Worst case: $O(n \log n)$.

Python's sort is not quicksort. It is **Timsort**, written by Tim Peters for Python 2.3 in 2002 and

described in his notes `listsort.txt` in the CPython source. It is a merge sort that takes the data's existing order seriously:

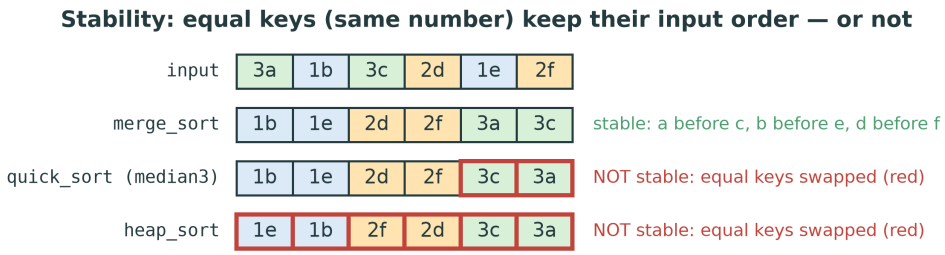
- It scans for **natural runs** — stretches that are already ascending, or strictly descending (which it reverses in place). Real data is full of them.
- A run shorter than a minimum length (between 32 and 64, chosen from n) is extended with **binary insertion sort** — Week 9's insertion sort, which is the fastest thing there is on short or nearly sorted ranges.
- The runs are **merged** in an order that keeps the merges balanced, using a temporary array — merge sort's price. During a merge, when one run keeps winning, it switches to **galloping**: an exponential search (Lecture 08) for how far that run can be copied at once.

It is **stable** — Python guarantees it — and it is what you measured on the right-hand side of the figure: n – 1 comparisons on sorted input, because the whole list is one run and there is nothing to merge. Since Python 3.11 the order in which runs are merged follows the *powersort* rule of Munro and Wild (2018). Java uses Timsort to sort objects, and a dual-pivot quicksort for primitive types, where stability cannot be observed. Even Timsort had a bug in its run-merging invariant, found in 2015 by a team **formally verifying** it (de Gouw et al.) — the binary-search story of Lecture 08 again: correct-looking code, broken on an input nobody had tried.

What this means for you: in your own programs, use `sorted()` or `list.sort()`, and `key=` for anything but plain values. In `dsa/sorting.py`, the point is that you can write the algorithms it is made of.

Stability

Equal keys, input order



A sort is **stable** if equal keys keep their input order.

Stable in Arabic: مستق. Stability matters as soon as you sort **records by one field**. Sort students by name, then **stably** by grade: within each grade they are still in name order — two stable sorts give a sort by grade, then name. With an unstable second sort the name order within a grade is lost.

The figure sorts six cards compared by their **number only**; the letters are there so you can see the order. Merge sort keeps a before c, b before e and d before f, because its merge takes from the left run on a tie. Quicksort's partition and heap sort's swap-to-the-back both move elements across long distances, and swap equal pairs (red). Heap sort, here, reversed all three.

Testing stability needs the right values. Sorting `(key, tag)` **tuples** proves nothing, because Python compares tuples by the tag too when the keys are equal — any correct sort puts `(1, "a")` before `(1, "c")`. To see stability, compare by the key alone: a small class whose `__lt__`, `__le__`, `__gt__` and `__ge__` look only at the key, as in `tools/figures_l10.py` (`Card`), in `test_insertion_sort_is_stable`, and in Lab 10.

The table to remember

Sort	Best	Average	Worst	Extra space	Stable
bubble	n	n^2	n^2	$O(1)$	yes
selection	n^2	n^2	n^2	$O(1)$	no
insertion	n	n^2	n^2	$O(1)$	yes
merge	$n \log n$	$n \log n$	$n \log n$	$O(n)$	**yes**
quick	$n \log n$	$n \log n$	n^2	$O(\log n)$	no
heap	$n \log n$	$n \log n$	$n \log n$	$O(1)$	no
counting	$n + k$	$n + k$	$n + k$	$O(n + k)$	—
Timsort	n	$n \log n$	$n \log n$	$O(n)$	yes

Times are in comparisons (counting sort: steps), all $O(\cdot)$. Notes on the rows:

- **bubble** best case n: with the early exit of the reference — one pass with no swap on sorted input.
- **quick** extra space $O(\log n)$: with the smaller-side recursion. The naive version needs $O(n)$ stack in the worst case. Its worst case is n^2 for every fixed pivot rule; with `random`, it is n^2 only with vanishing probability.
- **heap** best case: with distinct keys; if every key is equal each sift stops at once and it is $O(n)$.
- **counting**: k is the largest value plus one. The course's `counting_sort` rebuilds the integers from their counts, so stability does not apply to it; the textbook version that moves whole records is stable.

Which to use? **Guarantee and stability:** merge sort (or Timsort). **Speed in place, on average:** quicksort with a random or median-of-three pivot. **Worst case and no extra memory:** heap sort. **Tiny or nearly sorted arrays:** insertion sort — which is why Timsort and introsort both use it at the bottom.

This Week

Exercises: `dsa/sorting.py`, second half

Function	The trap
`merge_sort_steps`	`yield from` the recursive calls; `<=` in the merge; one scratch `Array`
`quick_sort_steps`	Lomuto's invariant; swap the pivot to `hi`; smaller side first

Function	The trap
`heap_sort_steps`	build from `n // 2 - 1` **down** to 0; sift with `size = end`

```
pytest tests/test_sorting.py -v -k "merge or quick or heap"
```

37 tests: each sort on the eight fixed cases (empty, one, two, five mixed values, duplicates, sorted, reversed, negatives), 20 random lists, "does not mutate the input", the two generator tests, and quicksort on sorted input. The plain forms are one line each once the `_steps` forms work. Write private helpers — `_merge`, `_partition`, `_choose_pivot`, `_sift_down` — as the lecture does; the tests only call the public names.

Two things the tests do **not** check, and the lab does: stability (sort cards compared by key only) and the pivot strategies (`pivot="first"` against `"median3"` on sorted input, counted).

The storage rule holds: copy the input into an `Array`, sort the `Array`, and return a list. No `sorted()`, `list.sort()` or `heapq` inside `dsa/sorting.py`.

Homework 10 — before Lecture 11

1. **Implement** merge, quick and heap sort until the 37 tests pass.
2. **Trace** `_partition` on `[5, 8, 1, 9, 3, 7, 2, 6]` as a table of `j`, `a[j]`, `boundary` and the array.
3. **Draw** the merge tree of `merge_sort([6, 5, 3, 1, 8, 7, 2, 4])`.
4. **Count** comparisons for `quick_sort(list(range(n)), pivot)` with `"first"` and `"median3"`, n = 100, 200, 400. Predict first.
5. **Break it:** change <= to < in `_merge`. Which tests fail? What else changes?

For item 4, reuse the idea of Lab 08's `CountingReads`: a class whose comparison methods add to a counter (Lab 10, Part 6). Before you run it: $n(n-1)/2$ for `"first"`; for `"median3"`, a little over $n \log_2 n$.

For item 5, predict before you run. (A hint: which of the tests compares values that are equal **but distinguishable**?)

Summary

Seven things to keep

1. **Divide and conquer:** split, solve the parts recursively, combine.
2. **Merge sort:** $T(n) = 2T(n/2) + n = O(n \log n)$ always; $O(n)$ extra; stable because of one <=.
3. **Quicksort:** partition around a pivot; the pivot lands in its final place.
4. `pivot="first"` on sorted input: $n(n-1)/2$ comparisons — **measured**. Random or median-of-three: $O(n \log n)$ on average.
5. Recurse on the **smaller** part: $O(\log n)$ stack, always.

6. **Heap sort:** the array is the tree; build-heap $O(n)$; $O(n \log n)$ always, $O(1)$ extra, not stable.
7. No comparison sort beats $\log_2(n!) \approx n \log_2 n$. Python's `sorted()` is **Timsort**: merge sort that exploits existing runs.

Next

Week 11 — Trees. Binary search trees: the sorted array's fast search, without its $O(n)$ insert. And an in-order traversal gives the values **sorted** — one more sort, for free.

Keep the pivot story in mind. A binary search tree built by inserting sorted values is exactly quicksort with `pivot="first"`: every new value goes to the right of all the others, and the "tree" becomes a linked list, n deep. The same bad input, the same cure (balance), in a different structure.

Sources and further reading

- **C. A. R. Hoare.** "Algorithm 64: Quicksort", *Communications of the ACM* 4(7), 1961; and "Quicksort", *The Computer Journal* 5(1), 1962.
- **J. W. J. Williams.** "Algorithm 232: Heapsort", *Communications of the ACM* 7(6), 1964; **R. W. Floyd.** "Algorithm 245: Treesort 3", *Communications of the ACM* 7(12), 1964 — build-heap in linear time.
- **R. C. Singleton.** "Algorithm 347: An efficient algorithm for sorting with minimal storage", *Communications of the ACM* 12(3), 1969 — median of three.
- **D. E. Knuth.** *The Art of Computer Programming*, vol. 3, *Sorting and Searching*, 2nd ed., Addison-Wesley, 1998, §5.2 and §5.3.1 — merge sort (which Knuth traces to John von Neumann, 1945), quicksort, heapsort, and the $\log_2(n!)$ lower bound.
- **J. Bentley.** *Programming Pearls*, 2nd ed., Addison-Wesley, 1999, column 11, "Sorting" — Lomuto's partition.
- **R. Sedgewick and K. Wayne.** *Algorithms*, 4th ed., Addison-Wesley, 2011, §2.2–2.4 — mergesort, quicksort (with the average-case analysis), heapsort.
- **D. R. Musser.** "Introspective sorting and selection algorithms", *Software: Practice and Experience* 27(8), 1997 — introsort.
- **T. Peters.** `Objects/listsort.txt` in the CPython source — Timsort, explained by its author.
- **J. I. Munro and S. Wild.** "Nearly-optimal mergesorts: fast, practical sorting methods that optimally adapt to existing runs", ESA 2018 — powersort.
- **S. de Gouw, J. Rot, F. S. de Boer, R. Bubel and R. Hähnle.** "OpenJDK's java.utils.Collection.sort() is broken: the good, the bad and the worst case", CAV 2015.

Every figure in this lecture is generated by `tools/figures_l10.py` from the states and counts of a working `dsa/sorting.py` (run it with `python tools/with_solutions.py tools/figures_l10.py` to use the reference).

Questions for Week 10

> ***The answers are at the back of the book,*** *in* Answers — Week 10. *Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify. The code is that of* `dsa/sorting.py` *as Lecture 10 shows it:* `_merge(a, lo, mid, hi, scratch)` *merges the half-open runs* `a[lo:mid]` *and* `a[mid:hi]`*, taking from the left on a tie (<=);* `_partition(a, lo, hi)` *is Lomuto's partition of the inclusive range* `a[lo..hi]` *around* `a[hi]`*, with the test* `a[j] < pivot`*;* `quick_sort` *swaps the chosen pivot to* `a[hi]` *first, and recurses into the smaller part;* `_sift_down(a, index, size)` *sifts in the max-heap* `a[0:size]`*, whose children of* `i` *are* `2i + 1` *and* `2i + 2`*.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W10-M01 – W10-M22
B	Short answer and essay	W10-E1 – W10-E5
C	Trace the algorithms	W10-T1 – W10-T5
D	Array state — draw every step	W10-S1 – W10-S3
E	Complexity analysis	W10-K1 – W10-K3
F	Find and fix the bug	W10-B1 – W10-B4
G	Write the code — checked by `pytest`	W10-C1 – W10-C5

Part A — Multiple choice

W10-M01 [what] Which of these is **not** one of the three steps of divide and conquer?

- **a)** divide the problem into smaller problems of the same kind
- **b)** solve the smaller problems recursively
- **c)** combine their answers
- **d)** compare every pair of elements

W10-M02 [how] The recurrence for the number of steps of merge sort on n elements is:

- **a)** $T(n) = T(n/2) + 1$
- **b)** $T(n) = 2T(n/2) + n$
- **c)** $T(n) = T(n-1) + n$
- **d)** $T(n) = 2T(n/2) + 1$

W10-M03 [how] `_merge` merges two sorted runs of 4 elements each. The largest number of comparisons it can make is:

- **a)** 7
- **b)** 8
- **c)** 4
- **d)** 16

W10-M04 [why] Merge sort is stable because:

- **a)** it splits the array exactly in the middle

- **b)** it is recursive
- **c)** on a tie, `_merge` takes the element from the left run
- **d)** it uses a scratch array

W10-M05 [what] The extra space merge sort needs, beyond the input, is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n \log n)$
- **d)** $O(n)$ — the scratch array

W10-M06 [how] How many calls of `_merge` does `merge_sort` make on 8 elements?

- **a)** 3
- **b)** 8
- **c)** 7
- **d)** 4

W10-M07 [how] `merge_sort(list(range(1024)))` — already sorted — makes how many comparisons?

- **a)** 1,023
- **b)** 5,120
- **c)** 10,240
- **d)** 523,776

W10-M08 [what] During Lomuto's partition, with `j` the next index to look at, the invariant is:

- **a)** `a[lo:boundary] < pivot <= a[boundary:j]`
- **b)** `a[lo:j]` is sorted
- **c)** `a[boundary] == pivot`
- **d)** `a[j:hi] >= pivot`

W10-M09 [how] `_partition(a, 0, 7)` on `a = [6, 3, 9, 1, 8, 2, 7, 4]` returns:

- **a)** 4
- **b)** 3
- **c)** 0
- **d)** 7

W10-M10 [how] How many comparisons with the pivot does one Lomuto partition of a range of 8 elements make?

- **a)** 8
- **b)** 3
- **c)** 7
- **d)** 28

W10-M11 [why] `quick_sort(list(range(n)), pivot="first")` makes:

- **a)** about $n \log_2 n$ comparisons
- **b)** $n - 1$ comparisons
- **c)** $n(n-1)/2$ comparisons

- **d)** about $\log_2 n$ comparisons

W10-M12 [why] On an already sorted range, `pivot="median3"` avoids the worst case because:

- **a)** it makes no comparisons
- **b)** it picks a random element
- **c)** it sorts the range first
- **d)** the median of the first, middle and last values is the middle value, which splits the range in half

W10-M13 [why] Quicksort recurses into the **smaller** part and loops on the larger. This bounds:

- **a)** the call stack at $O(\log n)$ frames
- **b)** the running time at $O(n \log n)$
- **c)** the number of swaps at n
- **d)** the stack at $O(1)$ on every input

W10-M14 [what] Quicksort is not stable because:

- **a)** it is recursive
- **b)** its partition swaps elements across long distances
- **c)** it may use a random pivot
- **d)** it needs a scratch array

W10-M15 [what] In the array layout of a heap, the children of index i are at:

- **a)** i + 1 and i + 2
- **b)** 2i and 2i + 1
- **c)** 2i + 1 and 2i + 2
- **d)** i // 2 and i // 2 + 1

W10-M16 [how] Build-heap on `[4, 10, 3, 5, 1, 8, 2, 7]` gives:

- **a)** `[10, 8, 7, 5, 4, 3, 2, 1]`
- **b)** `[1, 2, 3, 4, 5, 7, 8, 10]`
- **c)** `[10, 4, 8, 5, 1, 3, 2, 7]`
- **d)** `[10, 7, 8, 5, 1, 3, 2, 4]`

W10-M17 [why] Build-heap is $O(n)$, not $O(n \log n)$, because:

- **a)** most nodes are near the bottom, where a sift-down is short
- **b)** it never swaps
- **c)** only the root is sifted
- **d)** the input is already a heap

W10-M18 [how] For n = 10, heap sort's build loop sifts down the indices:

- **a)** 9, 8, ..., 0
- **b)** 5, 4, 3, 2, 1, 0
- **c)** 4, 3, 2, 1, 0
- **d)** 0, 1, 2, 3, 4

W10-M19 [what] Which sort is $O(n \log n)$ in the **worst** case **and** needs only $O(1)$ extra space?

- **a)** merge sort
- **b)** heap sort
- **c)** quicksort
- **d)** insertion sort

W10-M20 [why] The lower bound of about $n \log_2 n$ comparisons applies to:

- **a)** every sorting algorithm
- **b)** comparison sorts only
- **c)** merge sort only
- **d)** counting sort

W10-M21 [what] Python's `sorted()` and `list.sort()` use:

- **a)** quicksort
- **b)** heap sort
- **c)** introsort
- **d)** Timsort

W10-M22 [how] `sorted(list(range(1000)))` — already sorted — makes how many comparisons?

- **a)** 999
- **b)** about 10,000
- **c)** 499,500
- **d)** 0

Part B — Short answer and essay

W10-E1 [why] *(5 marks)* Describe merge sort as divide and conquer. Write its recurrence and solve it with a recursion tree. Give its extra space, and explain what makes it stable.

W10-E2 [why] *(4 marks)* Describe Lomuto's partition: what it does, its invariant, and why the pivot is in its final place afterwards. Why is there nothing to "combine" in quicksort?

W10-E3 [why] *(4 marks)* Compare the pivot strategies `"first"`, `"random"` and `"median3"`: the cost of choosing, and the inputs on which each is at its worst. Why is sorted input the case that matters?

W10-E4 [why] *(4 marks)* Explain how heap sort uses an array as a tree. Describe build-heap and the sort phase, and argue that build-heap is $O(n)$. Why is heap sort often slower in practice than quicksort, though its worst case is better?

W10-E5 [why] *(3 marks)* Define a **stable** sort. Give a practical reason to want one, say which of the six sorts in `dsa/sorting.py` are stable, and explain why sorting `(key, tag)` tuples does **not** test stability.

Part C — Trace the algorithms

W10-T1 [how] Trace `_merge(a, 0, 4, 7, scratch)` on `a = [3, 7, 8, 12, 1, 7, 10]`: for each comparison, give `i`, `j`, k, the two values compared and which one is copied. Which leftover loop runs? Which of the two 7s goes first?

W10-T2 [how] Trace `_partition(a, 0, 7)` on `a = [5, 8, 1, 9, 3, 7, 2, 6]` as a table of `j`, `a[j]`, whether it is swapped, the array, and `boundary`. What does it return?

W10-T3 [how] Trace `quick_sort([6, 3, 9, 1, 8, 2, 7, 4], pivot="last")`: for each partition give the range `lo..hi`, the pivot value, where it lands (q), and the array afterwards. In which order are the ranges partitioned, and why?

W10-T4 [how] Trace `heap_sort([4, 10, 3, 5, 1, 8, 2, 7])`: the array after each sift of build-heap, and after each of the first three steps of the sort phase.

W10-T5 [why] How many comparisons does `merge_sort` make on each of these? Explain the differences.

```
[1, 2, 3, 4, 5, 6, 7, 8]      [5, 6, 7, 8, 1, 2, 3, 4]
[1, 3, 5, 7, 2, 4, 6, 8]      [8, 7, 6, 5, 4, 3, 2, 1]
```

Part D — Array state

W10-S1 [how] Draw the complete merge tree of `merge_sort([6, 5, 3, 1, 8, 7, 2, 4])`: every split, down to single elements, and every merge back up, with the contents of each run. Number the merges in the order they happen.

W10-S2 [how] Draw every partition of `quick_sort([9, 1, 8, 2, 7, 3, 6, 4, 5], pivot="median3")`: the range, the three candidates and the chosen pivot, the array after the partition, and which part is recursed into and which is looped on.

W10-S3 [how] For `heap_sort([2, 9, 4, 7, 1, 8, 5, 3])`, draw the heap **as a tree and as an array** after build-heap, and after each of the first three steps of the sort phase. Mark the sorted tail.

Part E — Complexity analysis

W10-K1 [how] Solve each recurrence (give Θ), and name the algorithm and the case it describes:

- (a) $T(n) = 2T(n/2) + n$
- (b) $T(n) = T(n-1) + (n-1), T(1) = 0$
- (c) $T(n) = T(n/2) + n$

W10-K2 [how] Write the recurrence for the number of comparisons of `quick_sort(list(range(n)), pivot="first")`, and solve it exactly. What is its stack depth with the smaller-side recursion, and without it?

W10-K3 [why] Three ways to find the k largest of n values: (1) sort everything and take the last k; (2) k passes, each finding and removing the largest remaining; (3) build a max-heap and take the root k times. Give the cost of each, and say which is best when k = 1, k = 10 and k = n / 2, for n = 1,000,000.

Part F — Find and fix the bug

W10-B1 [why] Part of a student's `_merge`:

```
while i < mid and j < hi:
    if a[i] < a[j]:
        scratch[k] = a[i]
        i += 1
    else:
        scratch[k] = a[j]
        j += 1
    k += 1
```

All 37 tests pass. What is wrong, and on which input does it show?

W10-B2 [why] A student's build-heap:

```
for index in range(n // 2):
    _sift_down(a, index, n)
```

Find an input of 5 elements on which the heap sort that uses it returns a wrong answer — without running it. Explain, and fix.

W10-B3 [how] A student's quicksort, inside `quick_sort_steps`:

```
def sort(lo, hi):
    if lo < hi:
        p = _choose_pivot(a, lo, hi, pivot, rng)
        a[p], a[hi] = a[hi], a[p]
        q = _partition(a, lo, hi)
        yield list(a), (q,)
        yield from sort(lo, q - 1)
        yield from sort(q + 1, hi)
```

It passes all 13 quicksort tests. What goes wrong on `quick_sort(list(range(1000)), pivot="first")`, and why do the tests not see it? Fix it.

W10-B4 [how]

```
def sort(lo, hi):
    if hi - lo <= 1:
        return
    mid = (lo + hi) // 2
```

```
    sort(lo, mid)
    sort(mid, hi)
    _merge(a, lo, mid, hi, scratch)
    yield list(a), tuple(range(lo, hi))
```

What does `merge_sort([5, 2, 9, 1, 7])` return, and why?

Part G — Write the code

In `practice/week10.py`; check with `pytest tests/test_practice_week10.py -v`. The tests count comparisons (or reads and writes), so a solution that sorts everything fails where the question asks for less. Do not use `sorted`, `list.sort`, `heapq` or `bisect`.

W10-C1 [how] `merge_k_sorted(lists)` — merge k sorted lists in $O(N \log k)$ comparisons. Why is merging them one after another $O(Nk)$?

W10-C2 [why] `count_inversions(values)` — the number of pairs out of order, in $O(n \log n)$. Why does taking an element from the right run count `len(left) - i` inversions at once?

W10-C3 [why] `kth_smallest(values, k)` — quickselect. Why is its expected cost $O(n)$ when quicksort's is $O(n \log n)$?

W10-C4 [how] `sort_colours(values)` — a list of 0s, 1s and 2s, sorted in place in one pass. What is the invariant of the three regions?

W10-C5 [how] `merge_sort_bottom_up(values)` — merge sort with loops and no recursion: merge runs of width 1, 2, 4, … How many passes, and what happens to the last run of a pass when n is not a power of two?

Week 11 — Trees

> ***How to read this document.*** *This is the handout for Lecture 11. It holds everything on the slides, plus what I said out loud. Until now every structure was a* ***line****: each element had one next element. A tree lets each element have* ***two****. That one change gives you a structure that is fast to search* and *fast to change — the thing Lecture 08 said a sorted array could not be — provided it stays short. This week you build the binary search tree, walk it four ways, and measure what happens when it stops being short.*
>
> *Slides:* `DSA27-L11-slides.pdf` *·* *Code:* `dsa/tree.py` *·* *Tests:* `tests/test_tree.py`

Where We Are

From lines to branches

Every structure so far was **linear**: array, linked list, stack, queue.

- A sorted array: search $O(\log n)$, but insert $O(n)$ — everything shifts.
- A linked list: insert $O(1)$ once you are there, but search $O(n)$.

Today: a structure where search **and** insert cost $O(\log n)$ — **while it stays balanced**.

Lecture 08 ended with a trade-off. A sorted array is wonderful to search — twenty comparisons for a million values — and miserable to change, because an insertion in the middle shifts every element after it. A linked list is the opposite: splicing in a node is two assignments, but finding the place is a walk. The **binary search tree** combines the two ideas. It is a linked structure, so an insertion changes a couple of references and moves nothing. And its links are arranged so that every comparison discards a whole branch, the way binary search discards half an array.

The small print, which is the second half of this lecture: the tree is only fast while it is **short**. Feed it sorted data and it grows into a linked list, and every operation is $O(n)$ again. You will measure exactly that.

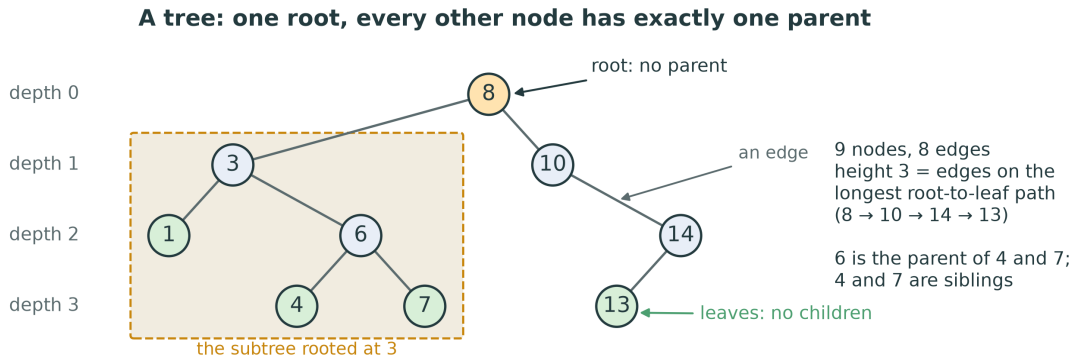
Tree in Arabic: شجر. *Binary search tree*: شجرة البحث الثنائي.

Today

1. Vocabulary: root, leaf, depth, height, subtree
2. Binary trees, and the `TreeNode`
3. The BST property — and why checking parents is not enough
4. Search, insert, min and max: one path down
5. Delete: three cases, and the in-order successor
6. The four traversals — and what each one is **for**
7. When the tree is a linked list: measured

The Vocabulary

A tree, and its words



A **tree** is a set of **nodes** joined by **edges**, with one special node, the **root**, such that every other node has **exactly one parent** and there is exactly one path from the root to it. *Node* in Arabic: عقدة, plural عُقَد. *Edge*: حافة, plural حواف. Computer scientists draw trees upside down: the root at the top, the leaves at the bottom.

Term	Meaning	Arabic
root	the one node with no parent	الجذر
parent, child	an edge goes from a parent down to a child	الأب، الابن
siblings	children of the same parent	الإخوة
leaf	a node with **no** children	ورقة
internal node	a node with at least one child	عقدة داخلية
subtree of x	x and everything below it	شجرة فرعية
ancestor, descendant	anything on the path above x; anything below x	سلف، خَلَف

The figure's tree has 9 nodes and 8 edges. That is always so: every node except the root has exactly one edge coming down into it, so **n nodes have $n-1$ edges**. The **subtree** idea is what makes trees recursive: the left child of the root is itself the root of a tree, and so is its left child, and so on, down to the empty tree. Every algorithm this week uses that fact.

Depth and height

- **Depth** of a node: edges from the **root** down to it. The root has depth 0.
- **Height** of a node: edges on the longest path from it **down** to a leaf. A leaf has height 0.
- **Height of the tree** = height of the root. Empty tree: -1.

	depth — العمق	height — الارتفاع
measured from	the root, down	the leaves, up

	depth — العم؟	height — الارتفا؟
the root	0	the tree's height
a leaf	its level	0

In the figure, 6 has depth 2 (8 → 3 → 6) and height 1 (6 → 4). The tree's height is 3, along 8 → 10 → 14 → 13. The nodes of the same depth form a **level**, in Arabic مستو؟. Level 0 is `8`, level 1 is `3, 10`, level 2 is `1, 6, 14`.

Why -1 for the empty tree? Because the recursive definition then works without a special case: the height of a node is $1 + \max(\text{height of left}, \text{height of right})$, and a leaf, whose two subtrees are empty, gets $1 + \max(-1, -1) = 0$. `dsa/tree.py` uses this convention — its docstring says "height is always *how many edges*, never *how many nodes*" — and `tests/test_tree.py` checks both `height() == -1` for an empty tree and `0` for a single node. Some books count nodes instead, so the same tree is one taller there. In an exam, say which you mean.

Height matters more than anything else this week, because it is the cost of a search: a search walks one path from the root down, and no path is longer than the height.

Trees are everywhere

- your **file system**: folders inside folders
- a web page: the HTML **DOM**
- an **expression** — `(3 + 4) * 2` — as your compiler sees it
- a **decision** tree; a tournament bracket; a family tree
- the **recursion tree** of `fib(n)` (Lecture 03)

All of these are hierarchies: each thing belongs to exactly one parent thing. A folder is inside one folder; an HTML element is inside one element; in `(3 + 4) * 2` the + is an operand of the *. When a structure lets an element belong to **several** parents, or lets paths go round in a circle, it is no longer a tree but a **graph** — Week 14. A tree is the special graph with no cycles and one root.

You have already drawn trees without calling them that: the call tree of `fib(5)` in Lecture 03 is one, with a call at each node and its recursive calls as its children. That is no accident. Recursion and trees are two views of the same shape, which is why every tree algorithm below is a short recursive function.

Binary Trees

At most two children

A **binary tree**: every node has at most two children, a **left** and a **right**. In Arabic: شجرة ثنائي؟.

```
class TreeNode:
    __slots__ = ("value", "left", "right")

    def __init__(self, value, left=None, right=None):
```

```
        self.value = value
        self.left = left
        self.right = right
```

Lecture 05's `Node` had one link, `next`. This one has two.

`TreeNode` is **given** in `dsa/tree.py`, exactly as above: the docstring says "the structure is the lesson, not the box". An empty subtree is simply `None`, as the end of a linked list was. The tree object itself keeps one reference, `self.root`, as `LinkedList` kept `self.head`. Nothing else is stored — not a size, not a list of values. That is the storage rule of Lecture 02: a structure keeps its data in node objects, and a Python list may appear only as input or output.

Left and right are different positions. A node with only a left child and a node with only a right child are different binary trees — and for a binary search tree the difference is the whole point.

`dsa/tree.py` also gives you `to_edges(node)`, which collects the `(parent, child)` pairs that `viz.draw.draw_tree` needs, so you can **see** your tree before any traversal works:

```
from viz.draw import draw_tree
from dsa.tree import BinarySearchTree, to_edges
bst = BinarySearchTree([8, 3, 10, 1, 6, 14, 4, 7, 13])
draw_tree(to_edges(bst.root), highlight={"6"})
```

How tall must a tree be?

- Level d holds **at most** 2^d nodes: 1, 2, 4, 8, ...
- A tree of height h holds **at most** $2^{h+1} - 1$ nodes.
- So n nodes need height **at least** $\lfloor \log_2 n \rfloor$ — and may have height up to $n - 1$.

n	shortest possible height	tallest possible height
7	2	6
1,000	9	999
1,000,000	19	999,999

Each node has at most two children, so each level can hold at most twice as many nodes as the one above it: $2^0, 2^1, 2^2, \ldots$ Summing the levels 0 to h gives $1 + 2 + \cdots + 2^h = 2^{h+1} - 1$, the same doubling sum as `DynamicArray` growth in Lecture 04. Turn it round: to hold n nodes, you need $2^{h+1} - 1 \geq n$, so $h \geq \log_2(n+1) - 1$, which for whole numbers is $h \geq \lfloor \log_2 n \rfloor$.

That lower bound is the promise: a million values can live in a tree of height 19, and then a search needs at most 20 comparisons — exactly binary search. The upper bound is the threat: the same million values can also form a single chain of height 999,999, where every node has only one child. Both are legal binary trees. Which one you get depends — for the tree you build this week — on the **order in which the values arrive**.

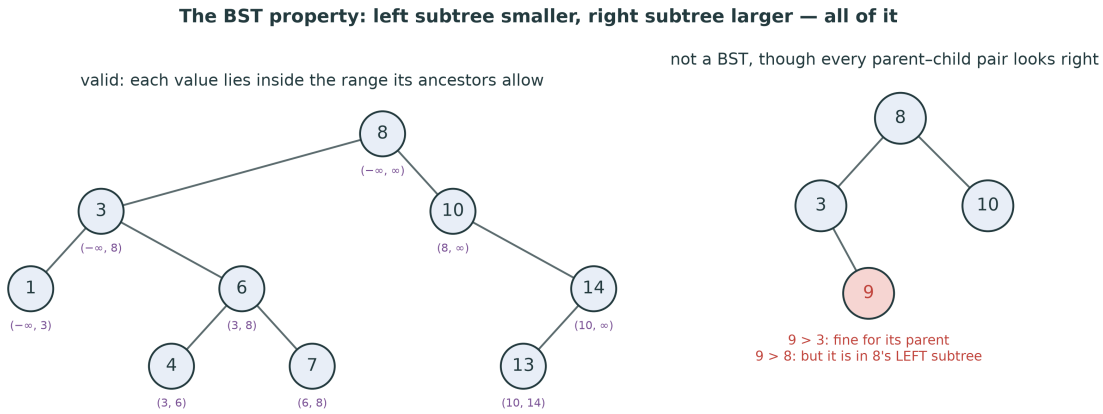
Three shapes have names, and the next lecture needs one of them:

- **full**: every node has 0 or 2 children;

- **perfect**: every level is completely filled — $2^{h+1} - 1$ nodes;
- **complete**: every level is full except possibly the last, which is filled from the left. The **heap** of Week 12 is a complete binary tree, and that is what lets it live in a plain `Array` with no links at all.

Binary Search Trees

The BST property



For **every** node x: everything in x's **left subtree** is smaller than x, and everything in its **right subtree** is larger.

That is the invariant of a **binary search tree** (BST). Note the words *subtree* and *everything*. It is **not** enough that the left child is smaller and the right child larger. In the right-hand tree of the figure, every parent–child pair looks fine: 3 < 8, 10 > 8, and 9 > 3. But 9 sits in 8's **left** subtree, where nothing may be larger than 8. Search that tree for 9 and you go right at 8 — and never find it. That tree is `test_is_valid_catches_a_hand_built_broken_tree`.

The purple ranges in the left-hand tree show the correct way to think about it. Each node inherits an allowed range from its ancestors: going left from 8 promises "less than 8"; going right from 3 then adds "greater than 3". So 6 must lie in (3, 8), and 7, below it on the right, in (6, 8). A value is legal if it lies inside the range of **all** its ancestors — and the tightest range is always set by the nearest ancestor on each side.

Duplicates. `dsa/tree.py` keeps values unique: inserting a value already in the tree leaves the tree unchanged (`test_duplicates_are_ignored`). Other designs send equal values to the right, or store a count in each node. Pick one, and state it — "smaller" and "larger" must be strict for the searches to be exact.

`is_valid`: carry the range down

The wrong check, at each node: `left.value < node.value < right.value`.

The right check: pass an allowed range **(low, high)** down the recursion.

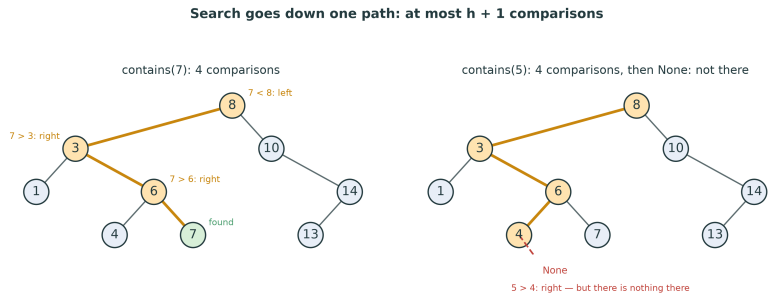
- going **left** from x: the new high is x

- going **right** from x: the new low is x
- the empty tree is valid

`is_valid` is the exercise that proves you understand the invariant. The recursive helper takes a node and the range its ancestors allow, starting with no limits at all at the root (`None` for "no bound" works well). At each node: if the value is outside the range, return `False`; otherwise both subtrees must be valid, the left one with the range narrowed from above by the node's value, the right one narrowed from below. Each node is checked once: $O(n)$.

An alternative is to use the in-order traversal of later in this lecture: a tree is a BST exactly when its in-order walk is strictly increasing. That is also $O(n)$, and a good check of your understanding — but the range version is the one that shows **why**.

Search: one path down



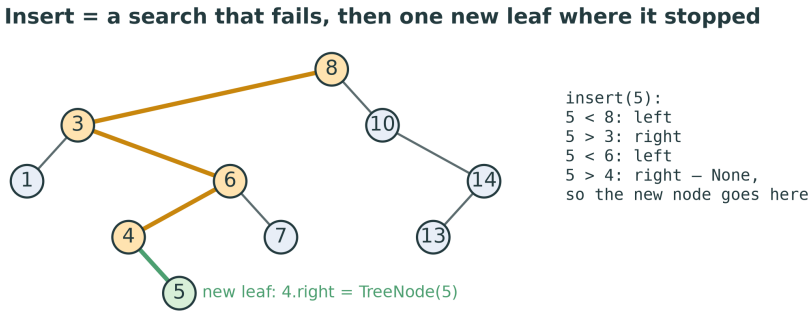

```
node = self.root
while node is not None:
    if value == node.value:
        return True
    node = node.left if value < node.value else node.right
return False
```

That is the body of `contains` from the reference solution. It is the linked list's traversal loop from Lecture 05 with one change: at each node there are two ways on, and the comparison chooses. Every comparison rules out a whole subtree — the one not taken — the way each comparison of binary search rules out half the array. The invariant makes it safe: if 7 is anywhere, and 7 < 8, it can only be in 8's left subtree.

The loop ends in one of two ways: it finds the value, or it steps onto `None`, the empty subtree where the value **would** have been. In the right-hand panel, the search for 5 goes 8, 3, 6, 4, and then to the right of 4, which is empty. That place matters: it is exactly where `insert(5)` will put the new node.

Cost. One comparison per level, at most h + 1 levels: $O(h)$. No recursion is needed, so no call stack either — $O(1)$ extra space. You can write it recursively too; it is a nice two-line exercise, but on a tall tree it can hit Python's recursion limit (see the end of this lecture), so the loop is the form in `dsa/tree.py`.

Insert: a search that fails



1. Empty tree $\rightarrow$ the new node **is** the root.
2. Otherwise walk down as in search. Equal $\rightarrow$ already there: do nothing.
3. When the way on is `None`, hang the new node **there**.

A new value always becomes a **leaf**. Nothing moves: $O(h)$.

Compare with Lecture 02's sorted array: finding the place was $O(\log n)$, but making room shifted everything after it, $O(n)$. Here the place is found with the same halving idea, and making room costs nothing at all — one reference, `parent.left` or `parent.right`, changes from `None` to the new node.

The one trap: you must stop **at the parent**, while you can still set its `left` or `right`. If you walk until `node is None`, you have lost the parent, and assigning `node = TreeNode(value)` only rebinds a local variable — the tree never sees it. Either look one step ahead (is the child I am about to take `None`?) or keep a `parent` variable as you walk. Both are fine.

`test_structure_is_actually_a_bst` checks the **shape** after inserting 8, 3, 10: `root.value == 8`, `root.left.value == 3`, `root.right.value == 10`. An implementation that kept a sorted Python list and pretended to be a tree would pass `in_order` and fail this.

Min and max: no comparisons at all

- **min**: from the root, go **left** until there is no left child.
- **max**: from the root, go **right** until there is no right child.
- Empty tree $\rightarrow$ `ValueError`.

$O(h)$, and not a single comparison of values.

Everything smaller than a node is to its left, so the smallest value is the node you reach by going left for as long as you can. The last node on that path has no left child — nothing is smaller — so it is the minimum. The invariant has already done the comparing, when the values were inserted. In the sample tree, min is 8 $\rightarrow$ 3 $\rightarrow$ 1, and max is 8 $\rightarrow$ 10 $\rightarrow$ 14.

That same walk, started not at the root but at a node's **right child**, finds the smallest value larger than that node: its **in-order successor**. Delete needs exactly that, in a moment.

Delete: Three Cases

Find it, then look at its children

Find the node, **remembering its parent**. Then:

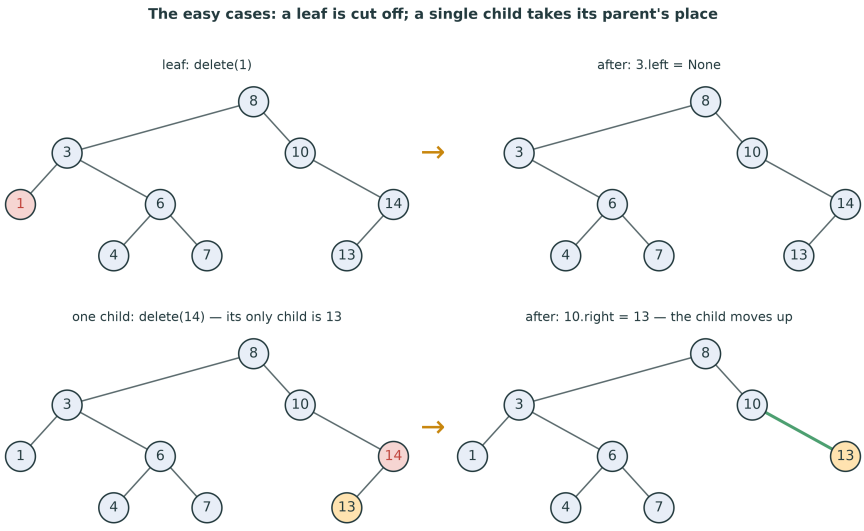
The node has…	Do this
no children (a **leaf**)	the parent's link becomes `None`
one child	the parent's link points at that child
two children	copy in the **in-order successor**, then delete the successor

Deleting a value that is not there does nothing.

Deleting is the hard operation of the week, because removing a node must leave a tree that is still a BST — and a node with two children cannot simply vanish, because two subtrees would be left hanging from one link.

Like insert, delete must hold on to the **parent** of the node it removes: it is the parent's `left` or `right` that changes. And the parent may not exist, when the node to delete is the root; then `self.root` itself changes. That root case is the one most often forgotten, which is why the tests delete the root (`test_delete_the_root`) and delete every node in turn until the tree is empty (`test_delete_everything`).

The easy cases

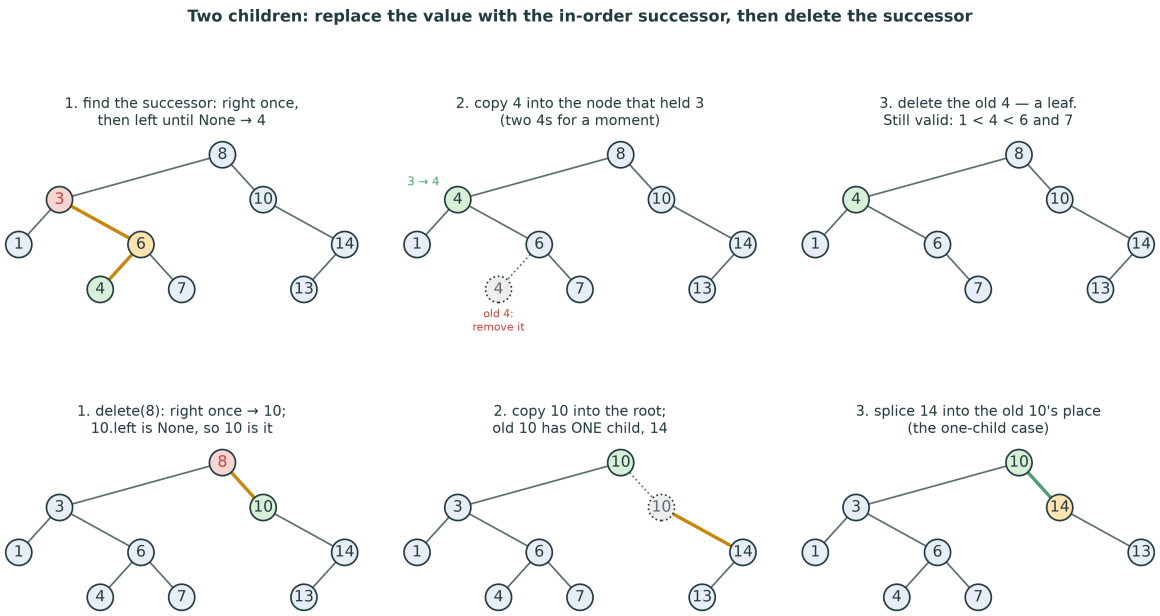


A leaf (delete 1): 1 is the left child of 3, so `3.left = None`. Nothing else changes.

One child (delete 14): 14's only child is 13. Point 14's parent, 10, straight at 13: `10.right = 13`. The whole subtree below 14 moves up one level, still in order — everything in it was larger than 10 before (it was in 10's right subtree), and it still is.

Notice that the leaf case is the one-child case with a child of `None`: "point the parent at the child" works for both. The reference solution uses that to handle the two with one assignment.

Two children: the in-order successor



A node with two children cannot be removed, so we remove a **different** node instead, and move its value up. Which value can take x's place and keep the BST property? One that is larger than everything left of x and smaller than everything right of x. There are exactly two candidates:

- the **in-order successor**: the **smallest** value in x's right subtree — go right once, then left until there is no left child;
- or the **in-order predecessor**: the largest value in x's left subtree.

`dsa/tree.py` uses the successor, the convention of Hibbard's 1962 paper and of most textbooks. The three steps, top row of the figure, deleting 3:

1. **Find the successor.** Right once, to 6; then left for as long as possible, to 4. 4 is the smallest value larger than 3.
2. **Copy** 4 into the node that held 3. For a moment the tree holds two 4s.
3. **Delete the old 4**, the node where the successor was.

Why is the result a BST? The new value 4 is larger than everything in the left subtree (those were smaller than 3, and 3 < 4), and smaller than everything left in the right subtree (4 was the smallest of them). Check the figure: 1 < 4 < 6 and 7.

Why step 3 is always easy. The successor was reached by going left until there was no left child — so it **has no left child**. It is either a leaf (the top row: 4) or has one child, on the right. Step 3 is always one of the two easy cases, and never recurses into a third.

The bottom row is the other shape, and the one students trip on. Deleting the root 8: go right once, to 10; 10 has no left child, so **10 itself** is the successor — the "go left" loop runs zero times. Copy 10 into the root. The old 10 has a right child, 14, so step 3 is the one-child case: 14 moves up into the old 10's place. The parent whose link changes in step 3 is then the node being deleted itself (its `right`), not a node further down (whose `left` changed in the top row). A delete that always writes `successor_parent.left = ...` passes the top row and corrupts the bottom

one — `test_delete_the_root` is exactly this case.

Everything costs one walk down: to find x, then on down to the successor. $O(h)$ in total.

The three cases collapse into two

1. Find the node and its parent. Not found → return.
2. **Two children?** Walk to the successor (keeping *its* parent), copy its value up — and from now on, delete **the successor node** instead.
3. Now the node has **at most one** child, `child` (possibly `None`):
   - no parent → `self.root = child`
   - node was its parent's left → `parent.left = child`
   - otherwise → `parent.right = child`

That is the shape of `delete` in the reference solution: the two-children case does its copy and then **hands over** to the easy case, by moving the "node to remove" and "its parent" variables down to the successor. Step 3 is written once and serves all three cases.

To decide which link of the parent to change, compare **identities**: `parent.left is node`. Comparing values would work too, but `is` says what you mean — the question is "which reference points at this node?", not "which value is smaller".

Writing it recursively is the other classic form: a helper `_delete(node, value)` that returns the new root of the subtree, so that the caller writes `node.left = self._delete(node.left, value)`. It is elegant and it is correct — and it recurses h deep, which the loop does not. Either is accepted in the lab.

The Four Traversals

Recursion on trees

A **traversal** visits **every** node once — in Arabic, اجتياز الشجر. For a binary tree, every recursive walk has the same skeleton:

- **base case:** the empty tree — do nothing
- **recursive case:** do something with the node, and walk the left and right subtrees

Where you put "do something" gives three different orders.

This is the structure of every recursive function on trees, and it is worth saying why it is correct, in the language of Lecture 03. The base case, `None`, needs no recursion. Each recursive call is on a **subtree**, which is strictly smaller than the tree — so the recursion always reaches the base case. And if the calls on the two subtrees do their job, adding the node itself completes the job for the whole tree. That is induction on the size of the tree.

The three recursive traversals differ only in the position of one line:

Name	Order	Mnemonic
pre-order	node, left, right	node **before** its subtrees
in-order	left, node, right	node **in between**

Name	Order	Mnemonic
post-order	left, right, node	node **after** its subtrees

The fourth, **level-order**, is a different kind of walk and needs a queue.

In-order: the code

```
def in_order(self):
    out = []
    self._in_order(self.root, out)
    return out

def _in_order(self, node, out):
    if node is not None:
        self._in_order(node.left, out)
        out.append(node.value)
        self._in_order(node.right, out)
```

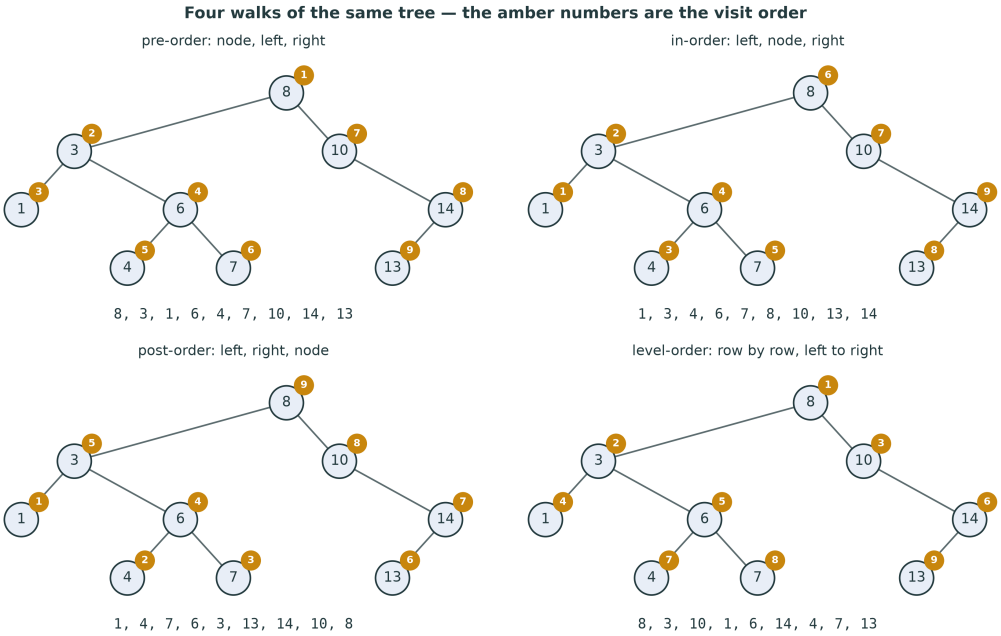
Move `out.append` **up** one line → pre-order. **Down** one line → post-order.

This is the reference `in_order` exactly. The public method makes the output list and starts the recursion at the root; the private helper does the walk. The list `out` is passed down so that every call appends to the **same** list — one list for the whole walk, $O(n)$ appends in total.

Two design points. First, the traversals **return a new list**: the list is output, created for the caller; the tree never keeps it (`test_traversals_return_a_fresh_list` clears the list it was given and checks that the next walk still works). Second, the helper builds one list rather than each call returning its own list and the caller concatenating them: `left_list + [node.value] + right_list` copies at every level, and on a tall tree that is $O(n^2)$ — the slicing trap of Lectures 03 and 08 in a new form.

The helper's name starts with `_`, the Python convention for "private: not part of the interface" (Lab 03).

Four walks of one tree



Check each walk against its rule by hand before you trust the picture. Pre-order starts at the root, then does the **whole** left subtree (3, 1, 6, 4, 7) before touching the right. Post-order finishes both subtrees of a node before the node: the root comes **last**. In-order comes out **sorted**: 1, 3, 4, 6, 7, 8, 10, 13, 14. Level-order reads the drawing row by row. These four lists are the four traversal tests in `tests/test_tree.py`.

Every traversal visits each node once and does $O(1)$ work there: $O(n)$ **time**. The recursive three use the call stack, one frame per level of the current path: $O(h)$ **space** — about $\log_2 n$ frames on a short tree, n on a chain.

What each one is FOR

Walk	Gives you	Used for
in-order	a BST's values **sorted**	listing, range queries, `is_valid`
pre-order	parents before children	**copying** / saving a tree: re-inserting it **rebuilds** the same shape
post-order	children before parents	**freeing** a tree; **evaluating** an expression; sizes and heights
level-order	nearest to the root first	printing by level; shortest distances (BFS, Week 14)

In-order is sorted — for a BST, and only for a BST. Everything left of a node is smaller and is written first; everything right is larger and is written after. This is the single most useful property a BST has: a sorted listing in $O(n)$ with no sorting at all. (Insert n values, then walk in-order,

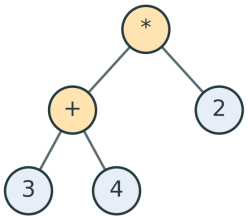
and you have sorted them: that is **tree sort**, $O(n \log n)$ when the tree stays short — and $O(n^2)$ when it does not.)

Pre-order rebuilds. Insert the pre-order list of a BST into an empty BST, and you get the same shape back: each node arrives after its parent, so it follows the same path and lands in the same place (`test_pre_order_rebuilds_the_same_tree`). That makes pre-order the natural way to **save** a tree to a file. The in-order list would not do — it is sorted, and inserting sorted values builds a chain.

Post-order frees and evaluates. A node is visited only after both of its subtrees are done. In C, where you free memory yourself, that is the only safe order: free the children, then the parent — free the parent first and you have lost the links to its children. It is also how anything that **depends on the children** is computed: the size of a tree is the sizes of its subtrees plus one; its height is one more than the taller subtree. `size` and `height` in `dsa/tree.py` are post-order walks.

Post-order evaluates: expression trees

An expression tree for (3 + 4) * 2: the shape holds the precedence



`pre-order`	`* + 3 4 2`	prefix (Polish) notation
`in-order`	`3 + 4 * 2`	infix — but the brackets are lost!
`post-order`	`3 4 + 2 *`	postfix: Lecture 06's stack evaluates it → 14

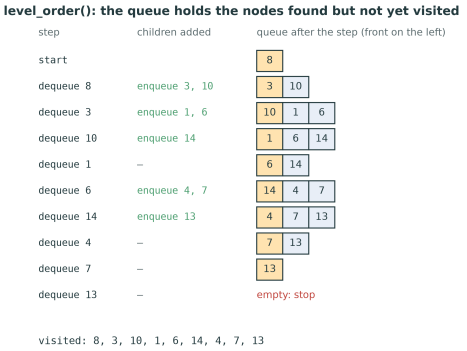
An **expression tree** has an operator at each internal node and a number at each leaf. The tree for `(3 + 4) * 2` has * at the root, because * is done **last**: the shape records the precedence, and the tree needs no brackets.

Walk it three ways and you get the three notations of Lecture 06:

- **post-order**, `3 4 + 2 *`, is **postfix** — exactly what `evaluate_postfix` evaluated with a stack. Evaluating the tree itself is also post-order: evaluate the left operand, then the right, then apply the operator — 7, then 14.
- **pre-order**, `* + 3 4 2`, is **prefix** (Polish) notation.
- **in-order**, `3 + 4 * 2`, is the familiar **infix** — and it is **wrong**: it reads as 11. In-order loses the brackets; to print infix correctly you must add a pair around each operator node.

Week 15 builds these trees from text with a recursive parser, in `dsa/translation.py`, and evaluates them in post-order.

Level-order needs a queue



1. Enqueue the root.
2. While the queue is not empty: **dequeue** a node, visit it, **enqueue** its children (left, then right).

Level-order (also called **breadth-first**) is the odd one out. It cannot be written with the simple recursive skeleton, because it does not finish one subtree before starting the other: after visiting 3, it must visit 10 — in the **other** subtree — before 3's children. Recursion follows the call stack, which is **last in, first out**: it always goes deeper into the most recent node. To visit in order of **distance from the root**, you need the nodes in the order they were **found**: **first in, first out**. A queue.

The queue holds the nodes that have been discovered but not yet visited — the "frontier". The figure shows it after every step. Children are enqueued behind everything already waiting, so all of level 1 is dequeued before any of level 2.

In `dsa/tree.py` the queue must be **your** `CircularQueue` from Week 7, not a Python list: the course rule, and the reason Lecture 07 asked you to make the ring **grow**. A tree's widest level can hold about half its nodes — a perfect tree of 31 nodes has 16 leaves — and the default capacity is 8. `test_level_order_on_a_wide_tree` is that tree. A list with `pop(0)` would also give the right answer — in $O(n)$ per dequeue, and $O(n^2)$ for the walk: Lecture 07's `SlowQueue` again.

Cost. Each node is enqueued once and dequeued once: $O(n)$ time. The queue holds at most one level and a bit of the next: $O(w)$ space, where w is the widest level — up to about n/2 for a short tree, but only 1 for a chain. The recursive walks are the opposite: $O(h)$ space, small for a short tree, n for a chain.

Replace the queue by a **stack** in this same loop, and the walk becomes depth-first — pre-order, if you push the right child before the left. Week 14 puts the two side by side on graphs, as BFS and DFS.

Height and size: post-order in one line

```
def _height(self, node):
    if node is None:
        return -1
    return 1 + max(self._height(node.left), self._height(node.right))
```

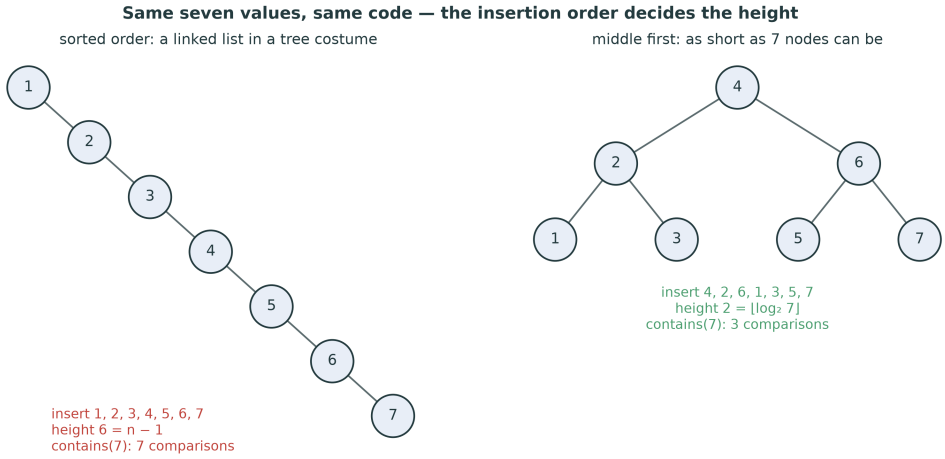
`size` is the same shape: 0 for `None`, else $1+$ left size $+$ right size.

This is the reference `_height`, called by `height()` on the root. Both calls must finish before the `max` can be taken — the node is handled **after** its children, which makes it a post-order walk, $O(n)$. The base case returns -1, as the slide on depth and height promised, so a leaf comes out as 0 without a special case.

`size` is not stored in `dsa/tree.py`; it is counted, $O(n)$. You could keep a `_size` counter up to date in `insert` and `delete`, as `LinkedList` did, and make it $O(1)$ — but then `delete` must be careful to decrement only when it really removed something. The skeleton keeps it simple.

When the Tree Is a Linked List

Same values, same code, different height



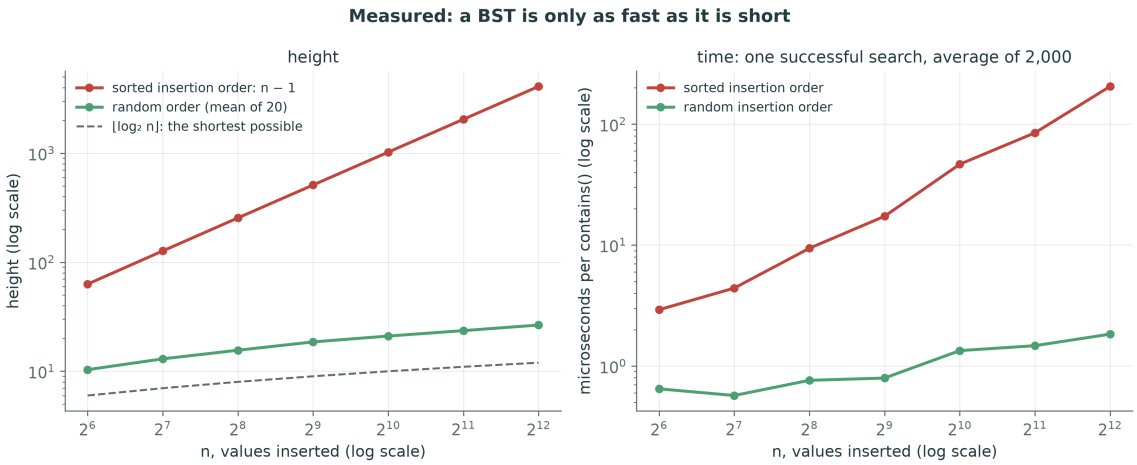
Insert **sorted** values and each new one is the largest so far: it always goes **right**. Height $n - 1$. Every operation: $O(n)$.

This is the catch in the module docstring: "insert 1, 2, 3, 4, 5 in order and you build a linked list wearing a tree costume". Nothing in the code is wrong; the BST property holds (`test_degenerate_tree_is_a_linked_list` checks height 4 and an empty left side). But the tree has one node per level, and a search for the largest value visits every node — a linear search through a linked list, with extra comparisons.

Sorted input is not an exotic worst case. Data often arrives sorted, or nearly: dates in a log, student IDs in registration order, a file that was saved sorted last time. The same values inserted middle-first — 4, then 2 and 6, then the rest — give the shortest tree 7 nodes can make.

Random order is kind. If the values arrive in a random order, the tree is short **on average**: the average depth of a node is about $2 \ln n \approx 1.39 \log_2 n$ (Hibbard, 1962; Knuth vol. 3, §6.2.2). But "random" is a promise about your data that you usually cannot make.

Measured



Real measurement of the reference `dsa/tree.py`, for n = 64 to 4,096 values (`tools/figures_l11.py`).

Left: height. Sorted insertion gives exactly $n - 1$: 4,095 for 4,096 values. Random insertion order (the mean over 20 shuffles) gives 26.6 at n = 4,096 — more than the 12 of a perfect tree, but a **logarithm**: it grows by 2 to 3 each time n doubles, while the sorted line doubles. The tallest of the twenty random trees at that size was 32.

Right: time for one successful `contains`, averaged over 2,000 searches. At n = 4,096: about 205 microseconds on the chain against 1.8 on the random tree — more than a hundred times slower, and the gap doubles every time n doubles. The chain is a straight line of slope 1: $O(n)$ per search. The random tree's line is almost flat: $O(\log n)$.

A second failure: recursion. Your recursive `height`, `size` and traversals go one call deeper per level. On the 4,096-node chain that is 4,096 frames, and Python's default limit is 1,000: `BinarySearchTree(range(1000)).height()` raises `RecursionError`. (The figure script raises the limit with `sys.setrecursionlimit` to measure the chain at all.) The loops — `contains`, `insert`, `delete`, `min`, `max` — and the queue-based `level_order` do not care. A tall tree is not only slow; it breaks recursive code.

The cure: keep it balanced — beyond this course

Balanced trees re-shape themselves on every insert and delete, so that h stays $O(\log n)$ whatever the input order:

- **AVL trees** (Adelson-Velsky and Landis, 1962)
- **red-black trees** (Guibas and Sedgewick, 1978)

Both are **beyond the bylaw**: named here, not examined.

A balanced search tree — in Arabic, شجرة متوازن — keeps a little extra information in each node — a height, or a colour — and after an insertion or deletion repairs the shape with **rotations**, local rearrangements of two or three nodes that keep the BST property and change the heights. An AVL tree keeps the two subtrees of every node within one level of each other; a red-black tree allows a little more slack in exchange for fewer repairs. Both guarantee height $O(\log n)$, so

search, insert and delete are $O(\log n)$ in the **worst** case.

None of the three bylaws that define this course asks for them (`docs/course/02-coverage.md`), so they are not in `dsa/tree.py` and not in the exam. You will meet them in the courses that follow, and in libraries: Java's `TreeMap` and C++'s `std::map` are red-black trees. Python has no balanced tree in its standard library at all; its `dict` and `set` are **hash tables** — Week 13 — which are faster on average but keep no order.

What you **can** do with this week's tree: if you have all the values up front, sort them and insert the middle first, then the middles of each half — the "middle first" tree of the figure, of height $\lfloor \log_2 n \rfloor$. That is W11-C4 in the question bank.

What each operation costs

Operation	Cost	Short tree	Chain
`contains`, `insert`, `delete`	$O(h)$	$O(\log n)$	$O(n)$
`min`, `max`	$O(h)$	$O(\log n)$	$O(n)$
`height`, `size`, `is_valid`	$O(n)$		
the four traversals	$O(n)$		
extra space, recursive walks	$O(h)$	$O(\log n)$	$O(n)$
extra space, `level_order`	$O(w)$	up to $O(n)$	$O(1)$

h is the height and w the width of the widest level. Against the structures of the earlier weeks, a **short** BST is the first to give $O(\log n)$ for search **and** insert **and** delete, and it keeps the values in order — so min, max, the sorted listing and range queries come for free. A hash table (Week 13) will beat it on plain lookups, $O(1)$ on average, but cannot list its keys in order or answer "all values between 20 and 40".

This Week

Exercises: `dsa/tree.py`

Method	Target	The trap
`insert`	$O(h)$	stop **at the parent**; ignore duplicates
`contains`, `min`, `max`	$O(h)$	a loop; `ValueError` on empty
`delete`	$O(h)$	keep the parent; the **root**; successor = right child
`height`, `size`	$O(n)$	empty tree is -1 and 0
`is_valid`	$O(n)$	carry **(low, high)** down, not just the parent
the four traversals	$O(n)$	return a **new list**; level-order uses **your** queue

```
pytest tests/test_tree.py -v
```

Twenty-six tests. `TreeNode`, `to_edges`, `__init__` and the dunders (`len`, `in`, `iter`, `repr`) are given; the dunders call your methods, so they start working as soon as those do. Write `insert` and `contains` first — every other test builds its tree with `insert`. Then draw your tree with `to_edges` and `draw_tree` before going further.

`level_order` imports `CircularQueue` from `dsa/queue.py`: your Week 7 queue, and it must grow when full (`tests/test_stack_queue.py` must pass, and your growth challenge must work on a wrapped ring).

Do not filter with `-k tree`: `-k` matches the file name `test_tree.py`, so it selects all 26. Filter by the method instead: `-k delete`, `-k order`.

Homework 11 — before Lecture 12

1. **Implement** `dsa/tree.py` until all 26 tests pass.
2. **Draw** the BST built by inserting 50, 30, 70, 20, 40, 60, 80, 35, 45, 65, then delete 20, 30 and 50 — the tree after each delete.
3. **Walk** your tree of item 2 (before the deletes) in all four orders, by hand. Check with your code.
4. **Measure** the height of a BST built from `range(n)` and from a shuffled `range(n)`, for n up to 900, in `notebooks/11-trees.ipynb`. Why stop at 900?

For item 2, the three deletes are one of each case: 20 is a leaf, 30 then has one child, and 50 is the root with two children. Draw the successor search for 50 before you move anything.

For item 4, the answer to "why 900" is the recursion limit — try 1,000 and read the error. Then explain why `contains` would still work on that tree.

Summary

Seven things to keep

1. A tree: one root; every other node has **one parent**; n nodes, $n - 1$ edges.
2. **Height** counts edges down to the deepest leaf; empty tree -1, leaf 0.
3. BST: **left subtree smaller, right subtree larger** — for the whole subtree.
4. Search, insert, min, max: **one path down**, $O(h)$.
5. Delete with two children: copy the **in-order successor**, delete it instead.
6. In-order is **sorted**; pre-order **rebuilds**; post-order **frees and evaluates**; level-order needs a **queue**.
7. Sorted input gives height $n - 1$: $O(n)$. Balanced trees fix it — beyond this course.

Next

Week 12 — Heaps and priority queues. A tree with no links at all: a **complete** binary tree stored in an `Array`, where the children of slot i are slots 2i + 1 and 2i + 2.

A BST keeps **everything** in order, and pays for it with a height that the input order controls. A heap keeps only a weaker order — every parent is smaller than its children — and in exchange its shape is always complete, so its height is always $\lfloor \log_2 n \rfloor$, whatever the input. It cannot search for an arbitrary value quickly, but it always knows its minimum: exactly what a priority queue needs.

Sources and further reading

- **D. E. Knuth.** *The Art of Computer Programming*, vol. 1, *Fundamental Algorithms*, 3rd ed., Addison-Wesley, 1997, §2.3 "Trees" — the vocabulary, and traversals; vol. 3, *Sorting and Searching*, 2nd ed., 1998, §6.2.2 "Binary tree searching" — insertion, deletion, and the average depth of a random BST.
- **T. N. Hibbard.** "Some combinatorial properties of certain trees with applications to searching and sorting", *Journal of the ACM* 9(1), 1962 — deletion by the successor, and the analysis of random BSTs.
- **T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein.** *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, chapter 12, "Binary search trees".
- **M. T. Goodrich, R. Tamassia and M. H. Goldwasser.** *Data Structures and Algorithms in Python*, Wiley, 2013, chapters 8 "Trees" and 11 "Search trees".
- **G. M. Adelson-Velsky and E. M. Landis.** "An algorithm for the organization of information", *Soviet Mathematics Doklady* 3, 1962 — AVL trees.
- **L. J. Guibas and R. Sedgewick.** "A dichromatic framework for balanced trees", *Proceedings of the 19th FOCS*, 1978 — red-black trees.

Every figure in this lecture is generated by `tools/figures_l11.py`. The diagrams are fixed trees, checked against the reference solution; the measured figure times a working `dsa/tree.py` and will differ slightly on your machine.

Questions for Week 11

> ***The answers are at the back of the book,*** *in* Answers — Week 11. *Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify.* `BinarySearchTree` *and* `TreeNode` *are the classes of* `dsa/tree.py`*: values are unique (inserting a duplicate changes nothing),* ***height*** *counts* ***edges*** *(empty tree* -1*, one node 0), and* `delete` *of a node with two children uses the* ***in-order successor****, the smallest value in its right subtree. The four traversals return lists. Unless a question says otherwise,* ***Q*** *is the BST built by inserting, in this order, into an empty tree:*
>
> `Q = [30, 15, 45, 8, 20, 40, 60, 17, 25, 50]`

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W11-M01 – W11-M22
B	Short answer and essay	W11-E1 – W11-E5
C	Trace the code	W11-T1 – W11-T5
D	Tree state — draw every step	W11-S1 – W11-S3

Part	Type	Questions
E	Complexity analysis	W11-K1 – W11-K3
F	Find and fix the bug	W11-B1 – W11-B4
G	Write the code — checked by `pytest`	W11-C1 – W11-C5

Part A — Multiple choice

W11-M01 [what] A **leaf** is a node that:

- **a)** has no parent
- **b)** has no children
- **c)** has exactly one child
- **d)** is at depth 1

W11-M02 [what] With height counted in edges, the height of a tree with exactly one node is:

- **a)** -1
- **b)** 1
- **c)** 0
- **d)** undefined

W11-M03 [how] The largest number of nodes a binary tree of height 3 can hold is:

- **a)** 7
- **b)** 8
- **c)** 16
- **d)** 15

W11-M04 [how] The smallest possible height of a binary tree with 100 nodes is:

- **a)** 6
- **b)** 7
- **c)** 10
- **d)** 99

W11-M05 [what] A binary tree is a binary **search** tree when, for every node x:

- **a)** x's left child is smaller than x and its right child is larger
- **b)** every value in x's left subtree is smaller than x, and every value in its right subtree is larger
- **c)** x is smaller than both of its children
- **d)** the values read level by level are sorted

W11-M06 [how] In the tree Q, the **right child of the root's left child** is:

- **a)** 20
- **b)** 25
- **c)** 17
- **d)** 8

W11-M07 [how] `contains(25)` on Q compares 25 with how many nodes?

- **a)** 3
- **b)** 5
- **c)** 4
- **d)** 10

W11-M08 [why] The worst-case cost of `contains` on a BST of n nodes, built by `dsa/tree.py` from values in an unknown order, is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n \log n)$
- **d)** $O(n)$

W11-M09 [how] Which insertion order of 1 to 7 builds the **tallest** BST?

- **a)** 4, 2, 6, 1, 3, 5, 7
- **b)** 4, 6, 5, 7, 2, 3, 1
- **c)** 3, 1, 2, 6, 5, 7, 4
- **d)** 7, 1, 6, 2, 5, 3, 4

W11-M10 [how] The **pre-order** walk of Q is:

- **a)** 8, 15, 17, 20, 25, 30, 40, 45, 50, 60
- **b)** 30, 15, 8, 20, 17, 25, 45, 40, 60, 50
- **c)** 30, 15, 45, 8, 20, 40, 60, 17, 25, 50
- **d)** 8, 17, 25, 20, 15, 40, 50, 60, 45, 30

W11-M11 [how] The **post-order** walk of Q is:

- **a)** 8, 17, 25, 20, 15, 40, 50, 60, 45, 30
- **b)** 8, 15, 17, 20, 25, 30, 40, 45, 50, 60
- **c)** 30, 45, 60, 50, 40, 15, 20, 25, 17, 8
- **d)** 17, 25, 8, 20, 15, 50, 40, 60, 45, 30

W11-M12 [what] Which walk of **any** BST lists its values in increasing order?

- **a)** pre-order
- **b)** post-order
- **c)** in-order
- **d)** level-order

W11-M13 [why] You save a BST to a file as a list of values, and later rebuild it by inserting the list into an empty tree. To get back **the same shape**, the list should be the tree's:

- **a)** in-order walk
- **b)** pre-order walk
- **c)** post-order walk
- **d)** in-order walk, reversed

W11-M14 [why] Which walk frees a tree safely in a language where you release memory yourself, and evaluates an expression tree?

- **a)** post-order
- **b)** pre-order
- **c)** in-order
- **d)** level-order

W11-M15 [why] `level_order` uses a queue rather than the call stack because:

- **a)** a queue uses less memory than recursion
- **b)** it must visit nodes in order of distance from the root, and the call stack always goes deeper into the most recent node first
- **c)** recursion cannot visit every node of a tree
- **d)** the tree stores its nodes in a queue

W11-M16 [how] During `level_order` on Q, the largest number of nodes waiting in the queue at one time is:

- **a)** 2
- **b)** 3
- **c)** 4
- **d)** 10

W11-M17 [what] Deleting a node with **two** children, `dsa/tree.py` replaces its value with:

- **a)** the largest value in its right subtree
- **b)** the value of its right child
- **c)** the value of its parent
- **d)** the smallest value in its right subtree

W11-M18 [how] `delete(30)` on Q. Which value ends up at the root?

- **a)** 40
- **b)** 25
- **c)** 45
- **d)** 17

W11-M19 [why] The in-order successor of a node with two children never has a left child. Why does that matter to `delete`?

- **a)** it makes the successor the largest value in the tree
- **b)** removing the successor is then always a leaf or one-child case, never a second two-children case
- **c)** it means the successor is always a leaf
- **d)** it lets `delete` skip the search

W11-M20 [why] A student's `is_valid` checks, at every node, only that the left child is smaller and the right child larger. It:

- **a)** accepts some trees that are not BSTs
- **b)** rejects some valid BSTs
- **c)** is correct, but $O(n^2)$
- **d)** is correct and $O(n)$

W11-M21 [how] The expression tree for `(2 + 3) * 4` is walked in post-order. The result is:

- **a)** `* + 2 3 4`
- **b)** `2 + 3 * 4`
- **c)** `2 3 + 4 *`
- **d)** `2 3 4 + *`

W11-M22 [what] AVL trees and red-black trees:

- **a)** are the BSTs of `dsa/tree.py`, under other names
- **b)** store their nodes in an array, like a heap
- **c)** are faster than hash tables for every operation
- **d)** re-shape themselves on insert and delete so that the height stays $O(\log n)$

Part B — Short answer and essay

W11-E1 [what] *(3 marks)* Define root, leaf, parent, subtree, depth and height, using the tree Q as the example for each. Why does a tree of n nodes always have exactly $n - 1$ edges?

W11-E2 [why] *(4 marks)* State the BST property. Give a tree in which every parent–child pair is in the right order but which is **not** a BST, and explain how `is_valid` must be designed to reject it. What is its cost?

W11-E3 [why] *(4 marks)* Describe the three cases of BST deletion. For the two-children case, explain why the in-order successor may take the deleted value's place, and why removing the successor is always easy.

W11-E4 [why] *(4 marks)* Name the four traversals of a binary tree, give the order of each, and for each say one thing it is **used for** and why it suits that use.

W11-E5 [why] *(4 marks)* "A BST gives $O(\log n)$ search." When is that true, and when is it false? Explain with the height, give an insertion order that makes it false, and name the cure. Compare a BST with a sorted array for search, insert and listing in order.

Part C — Trace the code

W11-T1 [how] Trace `contains(25)` and `contains(42)` on Q: list each node compared, the decision taken, and the result. Then say where `insert(42)` would put the new node.

W11-T2 [how] Give the four walks — pre-order, in-order, post-order and level-order — of the BST built from `[12, 5, 18, 2, 9, 15, 20, 7, 16]`.

W11-T3 [why] (a) The **pre-order** walk of a BST is `[20, 10, 5, 15, 12, 30, 25, 40]`. Draw the tree and give its post-order walk. (b) A binary tree that is **not** a BST has pre-order `A B D E C F` and in-order `D B E A C F`. Draw it and give its post-order walk. Why does (b) need two walks when (a) needed one?

W11-T4 [how] What do these functions return for the root of Q?

```
def f(node):
    if node is None:
        return 0
    return node.value + max(f(node.left), f(node.right))

def g(node, k):
    if node is None:
        return 0
    if k == 0:
        return 1
    return g(node.left, k - 1) + g(node.right, k - 1)
```

Give `f(root)`, and `g(root, k)` for k = 0, 1, 2, 3, 4. Say in words what each function computes.

W11-T5 [why] Trace `_is_valid(node, low, high)` of `dsa/tree.py` on the hand-built tree below: for each call, the node, `low` and `high`, and what it returns. Which call first returns `False`?

```
root = TreeNode(20, TreeNode(10, TreeNode(5), TreeNode(25)), TreeNode(30))
```

Part D — Tree state — draw every step

W11-S1 [how] Insert `45, 20, 70, 10, 30, 60, 90, 25, 35, 65` into an empty BST, drawing the tree after each insert and giving its height. Then, on the final tree, perform `delete(90)`, `delete(70)`, `delete(45)` and `delete(20)`, in that order, drawing the tree after each. For each delete, name the case, and for the two-children cases name the successor.

W11-S2 [how] Draw the BST built by each insertion order of the values 1 to 7, and give its height:

(a) `1, 2, 3, 4, 5, 6, 7`
(b) `4, 2, 6, 1, 3, 5, 7`
(c) `3, 1, 2, 6, 5, 7, 4`
(d) `4, 6, 5, 7, 2, 3, 1`

Which two give the same tree? Give their pre-order walks, and explain why they must match.

W11-S3 [how] Draw the queue of `level_order` (front on the left) after every dequeue, for the tree Q. Then do the same walk with a **stack** in place of the queue, pushing the **right** child before the left, and give the order in which the values come out. Which of the named walks is it?

Part E — Complexity analysis

W11-K1 [how] Give Θ and justify:

(a) building a BST by inserting the values 1, 2, …, n in order;
(b) the same n values inserted in a random order (on average);
(c) **tree sort**: insert all n values, then walk in-order — in the best and in the worst case.

W11-K2 [why] A student writes `height` like this:

```
def height(node):
    if node is None:
        return -1
    if height(node.left) > height(node.right):
        return 1 + height(node.left)
    return 1 + height(node.right)
```

It returns the right answer. Write a recurrence for the number of calls it makes on a chain of n nodes that all hang to the **right**, and on a perfect tree of height h. Solve both. How many calls does it make on a right chain of 20 nodes? What is the fix?

W11-K3 [why] Compare the **extra memory** of the recursive in-order walk and of `level_order`: on a perfect tree of n nodes, and on a chain of n nodes. Which is better on which tree, and why?

Part F — Find and fix the bug

W11-B1 [how]

```
def insert(self, value):
    node = self.root
    while node is not None:
        if value == node.value:
            return
        node = node.left if value < node.value else node.right
    node = TreeNode(value)
```

W11-B2 [why]

```
def is_valid(self):
    return self._ok(self.root)

def _ok(self, node):
    if node is None:
        return True
    if node.left is not None and node.left.value >= node.value:
        return False
    if node.right is not None and node.right.value <= node.value:
        return False
    return self._ok(node.left) and self._ok(node.right)
```

W11-B3 [why] The two-children part of a `delete`, after `node` (with parent `parent`) has been found:

```
if node.left is not None and node.right is not None:
    succ_parent, succ = node, node.right
```

```
    while succ.left is not None:
        succ_parent, succ = succ, succ.left
    node.value = succ.value
    succ_parent.left = succ.right
    return
```

It passes `test_delete_node_with_two_children`. Find an input on which it breaks the tree, and fix it.

W11-B4 [how]

```
def in_order(self):
    out = []
    self._in_order(self.root, out)
    return out

def _in_order(self, node, out):
    out = []
    if node is not None:
        self._in_order(node.left, out)
        out.append(node.value)
        self._in_order(node.right, out)
```

Part G — Write the code

In `practice/week11.py`; check with `pytest tests/test_practice_week11.py -v`. Each function takes a tree by its **root** — a `TreeNode`, or `None` — not a `BinarySearchTree`. The tests hand you nodes that count how often their `value`, `left` and `right` are read, so an answer that visits more of the tree than it needs fails. Do not copy the tree into a list and work on the list.

W11-C1 [how] `count_leaves(node)` — the number of leaves.

W11-C2 [why] `is_balanced(node)` — `True` when at **every** node the two subtrees' heights differ by at most 1, in **one** $O(n)$ walk. Why is calling a separate `height` at every node not good enough?

W11-C3 [why] `range_values(node, low, high)` — the values v with `low <= v <= high`, in sorted order, visiting only the part of the tree that can hold them. Which subtrees may you skip, and why?

W11-C4 [how] `build_balanced(values)` — from a sorted list of distinct values, a BST of the smallest possible height. No slicing.

W11-C5 [why] `lowest_common_ancestor(node, a, b)` — in a BST, the lowest node with both a and b in its subtree, in $O(h)$ with a loop. Why is the first node where a and b "split" the answer?

Week 12 — Heaps and Priority Queues

> ***How to read this document.*** *This is the handout for Lecture 12. It holds everything on the slides, plus what I said out loud. Last week's tree was made of nodes and references. This week's tree has neither: it lives in a flat array, and the links between parent and child are two lines of arithmetic. On top of it sits the structure behind every "most urgent first" system you will ever use — the priority queue.*
>
> *Slides:* `DSA27-L12-slides.pdf` · *Code:* `dsa/heap.py` · *Tests:* `tests/test_heap.py`

Where We Are

A queue that does not wait its turn

Week 7's queue is **fair**: first in, first out.

An emergency department is not fair, on purpose:

- a patient with chest pain is seen **before** one with a sprained ankle
- who arrived first matters only between equally urgent patients

We need a queue that serves the **most urgent** item: a **priority queue**.

Lecture 07 ended with this promise: same two operations, *add* and *remove*, but a different rule for which item leaves. A FIFO queue answers "who has waited longest?". A priority queue answers "who matters most right now?" — and that small change of contract needs a completely different structure underneath.

Priority queue in Arabic: طابور الأولويات. *Heap*: الكوم. *Complete binary tree*: شجرة ثنائية كامل.

A word of warning about the name. The **heap** of this lecture has nothing to do with "the heap" in memory management, where `new` objects and Python's objects live. The two words collided in the 1960s and have been confusing students ever since. Here, a heap is a tree with an order.

Today

1. The priority queue ADT, and three ways to build it badly
2. The heap: a **shape** and an **order**
3. The array **is** the tree: `2i + 1`, `2i + 2`, `(i - 1) // 2`
4. Sift up (`push`) and sift down (`pop`): $O(\log n)$
5. Build-heap in $O(n)$ — the summation, and a measurement
6. The priority queue on top; top-k; heap sort revisited

The Priority Queue ADT

Serve the most urgent

Operation	Meaning
`enqueue(item, priority)`	add an item with a priority
`dequeue()`	remove and return the item with the **lowest** priority number
`peek()`	look at that item without removing it
`is_empty()`, `len(pq)`	

"Priority 1" means "most urgent" — as in triage, and in `dsa/heap.py`.

As with the stack and the queue, this table is the ADT: operations and meaning, nothing about storage. Two conventions exist and both are common. In `dsa/heap.py`, as in hospital triage and in Python's `heapq`, the **smallest** number is served first. Some books use the largest; the structure is the same with the comparison reversed — which is exactly the difference between the `MinHeap` and `MaxHeap` classes below.

Empty behaves as it did for the stack and the queue: `dequeue()` and `peek()` on an empty priority queue raise `IndexError` (`test_priority_queue_empty`).

Three obvious designs, and what each costs

Storage	enqueue	dequeue	Why
unsorted array	$O(1)$	$O(n)$	append; then **search** for the minimum
sorted array	$O(n)$	$O(1)$	**shift** to insert in order; take from the end
sorted linked list	$O(n)$	$O(1)$	**walk** to the insertion point
binary heap	$O(\log n)$	$O(\log n)$	today

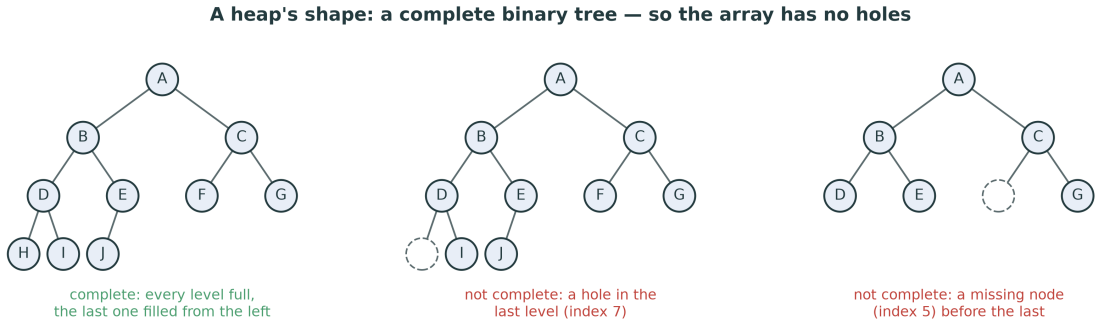
Each naive design makes one operation cheap by making the other expensive. For a workload of n enqueues and n dequeues — which is what every real priority queue sees — all three cost $O(n^2)$ in total. The heap makes **both** operations $O(\log n)$, so the same workload costs $O(n \log n)$. For a million items, that is the difference between about 10^{12} steps and about 2×10^7.

The sorted array is worth a second look. It keeps the items in **complete** order, when the priority queue only ever asks for the **first** item. That wasted order is what we pay for with $O(n)$ insertion. The heap's trick is to keep **just enough** order to know the first item, and not a comparison more.

A balanced binary search tree (Week 11) would also give $O(\log n)$ for both — but it needs nodes, references, and a balancing scheme this course does not build. The heap needs none of that.

The Heap

Shape: a complete binary tree



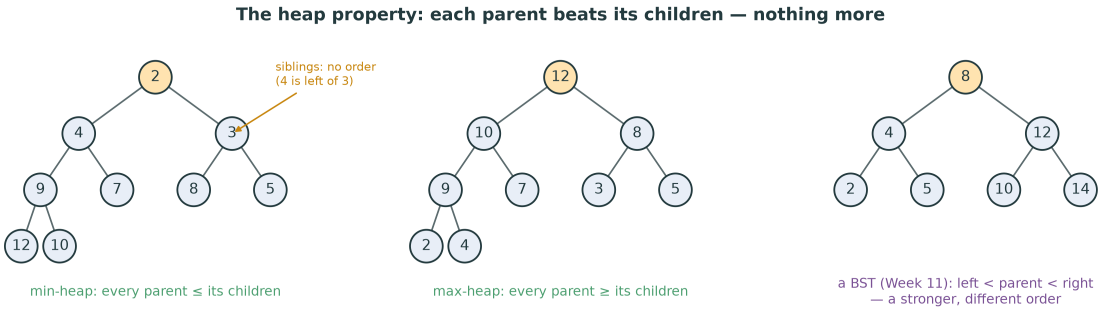
Complete: every level is full, except perhaps the last, which is filled **from the left**, with no gaps.

A binary heap has two rules. The first is about **shape** and says nothing about values: the tree is **complete**. Read the tree level by level, left to right, and you meet no gap until the very end. The left-hand tree in the figure is complete. The middle one is not — there is a hole where H should be. Neither is the right-hand one: F is missing while G is present.

Two things follow from the shape, and both matter.

- **The height is $\lfloor \log_2 n \rfloor$.** A complete tree with n nodes is as short as a binary tree can be: level k holds up to 2^k nodes, so ten levels hold over a thousand nodes and twenty levels over a million. Every operation below walks one path from the root to a leaf, or back, so every operation is $O(\log n)$. Compare Week 11's binary search tree, which can degrade into a linked list of height n: a heap never can, because its shape is **forced**.
- **The tree fits in an array with no holes.** That is the next slide, and the reason the whole structure is two pages of code.

Order: the heap property



Min-heap: every parent $\leq$ its children. **Max-heap:** every parent $\geq$ its children.

So the best item is at the **root**. Siblings are **not** ordered.

The second rule is about **order**. In a min-heap, each parent is no larger than its children. Follow any path downwards and the values never decrease; so the smallest value in the whole heap is

at the root, which is all a priority queue needs.

What the property does **not** say is just as important. In the left tree, 4 is the left child and 3 the right: the property says nothing about siblings, or about cousins in different subtrees — 9 sits on level 2 while 5, smaller, sits on the same level elsewhere. A heap is **not sorted**. Reading its array from left to right gives `2, 4, 3, 9, 7, 8, 5, 12, 10`, which is not in order, and that is correct.

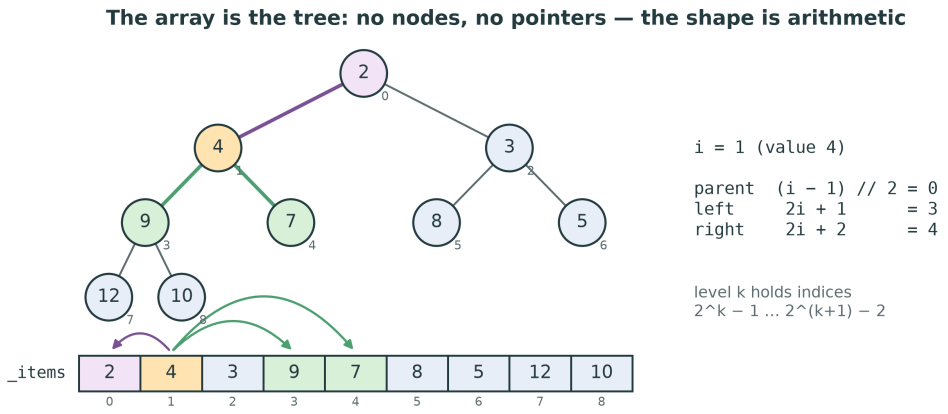
Compare the binary search tree of Week 11 (right). Its order is **stronger** — left subtree < node < right subtree — which is why it can **search** for any value in $O(\log n)$. A heap cannot search; finding an arbitrary value in a heap is $O(n)$. What it can do is keep its **best** value at the top, cheaply, while values come and go. Weaker order is cheaper to maintain.

In `dsa/heap.py`, the rule is one method:

```
def _beats(self, child, parent):
    """True when `child` must move above `parent`."""
    return child < parent
```

`MaxHeap` overrides only this method, with `>`. Everything else — sifting, `push`, `pop`, `heapify`, `is_valid` — is written once, in `MinHeap`, in terms of `_beats`. Write a single `<` anywhere else in your class and `MaxHeap` breaks while `MinHeap` passes: `test_max_heap_differs_from_min_heap` is there to catch it.

The array is the tree



Node `i`: children at **`2*i + 1`** and **`2*i + 2`**; parent at **`(i - 1) // 2`**.

Number the nodes level by level, left to right, starting at 0 — the order in which you would read them. Store node i in slot i of an array. Because the tree is complete, the slots are filled from 0 to n – 1 **with no gaps**, and the numbering gives every node its family by arithmetic:

- level k starts at index $2^k - 1$ (0, 1, 3, 7, 15, …);
- the children of i are **2i + 1** and **2i + 2**;
- the parent of i is **(i – 1) // 2**, for i > 0.

Check it on the figure: node 1 (value 4) has children 3 and 4 (values 9 and 7) and parent 0. Node

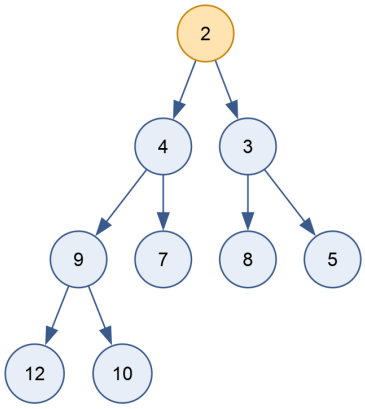
4 (value 7) has children 9 and 10 — both past the end of a nine-element array, so it is a leaf. In general, i is a leaf exactly when `2 * i + 1 >= n`, so the leaves are the second half of the array, from index `n // 2` on, and the **last parent** is at `n // 2 - 1`. That index will start `heapify`.

No `TreeNode`, no `left` or `right` attributes, no `None` checks: the shape is entirely in the index. This is why `dsa/heap.py` stores the heap in your Week 4 `DynamicArray` — `self._items` — and nothing else. `push` appends at the end, `pop` removes from the end, and both are the $O(1)$ operations of the dynamic array. The storage rule is kept with no effort at all.

Why 0-based arithmetic looks slightly odd: many textbooks, including CLRS, start the array at index 1, where the formulas are the prettier `2i`, `2i + 1` and `i // 2`. Python counts from 0, so we shift everything by one.

See it: `draw_array_as_tree`

```
from viz.draw import draw_array_as_tree
draw_array_as_tree([2, 4, 3, 9, 7, 8, 5, 12, 10], highlight=0)
```



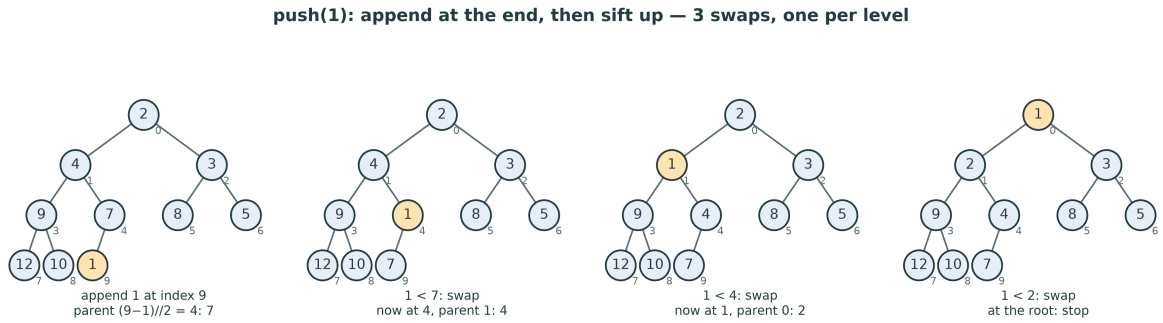
A plain list, drawn as the tree it already is. No conversion.

`viz.draw.draw_array_as_tree` takes any list and draws edges from i to 2i + 1 and 2i + 2. It knows nothing about heaps: it does not check the heap property, and it will happily draw an unsorted list as a tree that is not a heap. That makes it the best debugging tool of the week. When a test fails, draw `list(heap._items)` and look for a parent that is larger than a child.

In the notebook `notebooks/12-heaps.ipynb` it renders inline. In a script, `.render("heap", format="png")` writes a file.

Sift Up: `push`

Append, then sift up



1. **Append** the new value at the end — the shape stays complete.
2. While it beats its **parent**, swap it with the parent.

Adding a value must keep both rules. The **shape** has only one legal new position: the next free slot, index n, which in the tree is the next place on the last level. So `push` begins with `self._items.append(value)`.

Now the **order** may be broken, but only in one place: between the new value and its parent. Everything else was a heap before and is untouched. If the new value beats its parent, swap them. The value is now one level higher, and the only possible violation is again between it and its (new) parent. Repeat until it does not beat its parent, or it reaches the root. This is **sift up** (also called *bubble up*, *swim* or *percolate up*).

Why does a swap never break anything below? When the new value x replaces its parent p, p moves down into x's old place. p was at most every value in its old subtree (the heap property held there), so p is still at most x's old children, which were p's grandchildren. And x, now above p's other child, beats p, which beat that child. Nothing below is disturbed.

Trace: `push(1)`

Step	`1` at	parent index	parent value	Decision
append	9	(9 – 1) // 2 = 4	7	1 < 7 → swap
	4	(4 – 1) // 2 = 1	4	1 < 4 → swap
	1	(1 – 1) // 2 = 0	2	1 < 2 → swap
	0	—	—	root: stop

Result: `[1, 2, 3, 9, 4, 8, 5, 12, 10, 7]`. Three swaps: one per level.

The figure and this table were produced by running the reference `push` on the heap `[2, 4, 3, 9, 7, 8, 5, 12, 10]` — every state is checked against the code by `tools/figures_l12.py`. The new value climbs from index 9 to the root through the indices 9, 4, 1, 0. That path is the chain of parents, and it has at most height + 1 nodes, so **push is** $O(\log n)$ in the worst case — the worst case being a new value smaller than everything, which climbs all the way.

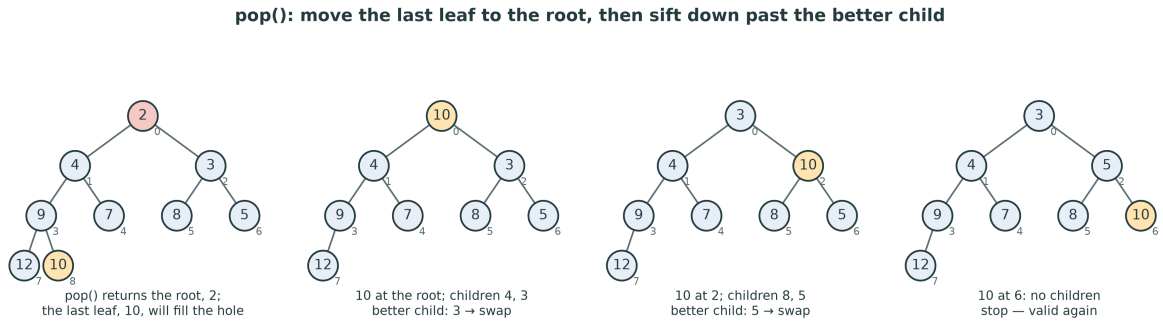
Most pushes stop much earlier. Half the nodes of a heap are leaves, so a random new value is quite likely to belong near the bottom. On random input a push makes, on average, **about two**

comparisons, whatever n is. The measured figure later in this lecture shows both cases: $\log_2 n$ comparisons per push on descending input, and a flat line on random input.

In words, the method you write is: start at the last index; while the index is not 0, compute the parent; if the item does not beat the parent, stop; otherwise swap the two and continue from the parent. Every comparison goes through `self._beats(child, parent)`, with the child first.

Sift Down: pop

Remove the root, fill the hole from the end



1. Keep the root — it is the answer.
2. Move the **last** item into the root; shrink the array by one.
3. While it is beaten by its **better child**, swap with that child.

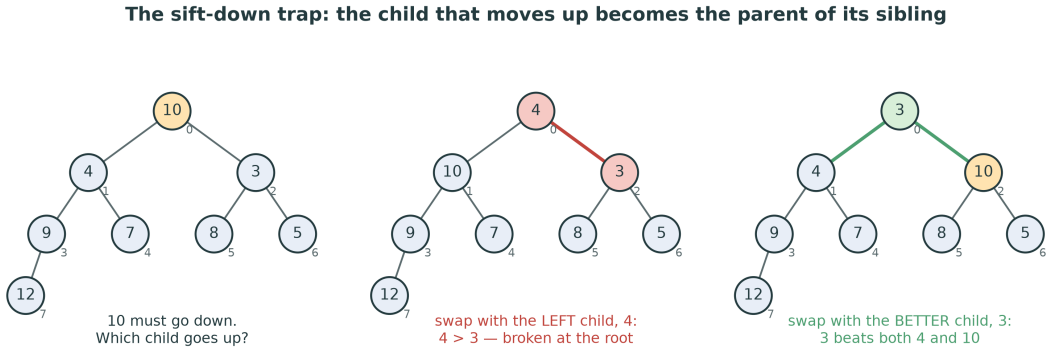
The best item is at index 0, so reading it is easy. Removing it leaves a hole at the root, and the shape must stay complete. The only slot that can disappear without making a gap is the **last** one. So take the last item, remove it from the end of the array ($O(1)$ — `self._items.pop()` with no argument), and put it in the root.

That item was a leaf, so it is probably large, and the root is the wrong place for it. It **sifts down**: compare it with its children; if one of them beats it, swap with the better of the two, and continue from there. Stop when neither child beats it, or when it has no children. In the figure, 10 moves from the root to index 2 and then to index 6, a leaf; the heap is valid again, and pop returns 2.

Two special cases, both in the tests:

- **Empty heap:** pop raises `IndexError` (`test_empty_heap`).
- **One item:** after removing the last item, the heap is empty, and there is no root to write into. Return that item directly (`test_single_element`). A pop that always writes `self._items[0] = last` fails here with an `IndexError` from your own `DynamicArray`.

The trap: swap with the better child



The child that moves up becomes the **parent of its sibling** — so it must beat the sibling too.

This is the bug of the week, and the skeleton's docstring warns about it: "swapping with the wrong one silently breaks the invariant". A sift-down that always swaps with the **left** child, or with the first child it finds smaller than the moving item, works on many inputs and fails on others. In the middle panel, 4 moves up above 3 — and 4 > 3 at the root of a min-heap. Nothing crashes; pop returns a wrong value some time later.

The fix is to compare three items, not two: the item, its left child (if it exists), and its right child (if it exists), and to find the **best of the three**. If the best is the item itself, stop. Otherwise swap with the best child and continue. The two "if it exists" checks are where the other bugs live: a node near the end of the array may have a left child but no right child, so each child index must be compared with `len(self._items)` before it is read.

`test_invariant_holds_after_every_pop` pops a 40-element heap to the end and calls `is_valid()` after every pop, so a wrong-child bug cannot hide there — provided `is_valid` really inspects the whole array (`test_is_valid_rejects_a_broken_array` plants a violation to make sure it does).

Both sifts are $O(\log n)$

Operation	Walks	Comparisons per level	Worst case
push $\rightarrow$ sift up	leaf to root	1	$\lfloor \log_2 n \rfloor$ swaps
pop $\rightarrow$ sift down	root to leaf	2	$2\lfloor \log_2 n \rfloor$ comparisons
peek	—	—	$O(1)$

A heap of a million items: at most **20** levels.

Sift up makes one comparison per level (the item against its parent). Sift down makes up to two (left child against the best so far, then right child). Both walk at most the height of the tree, which is $\lfloor \log_2 n \rfloor$, so both operations are $O(\log n)$. The space is $O(1)$ beyond the array: the loop keeps one index.

Could sift down be done with recursion? Yes — "sift down from the child" is a tail call. Write it as a loop anyway: Python does not remove tail calls (Lecture 03), and the loop is the same length.

Build-Heap in $O(n)$

Two ways to make a heap from n values

Way 1 — n pushes. Start empty; push each value. $n \times O(\log n) = O(n \log n)$.

Way 2 — `heapify`, bottom up. Leave the values where they are. Sift **down** every parent, from the **last** parent back to the root.

Way 2 is $O(n)$.

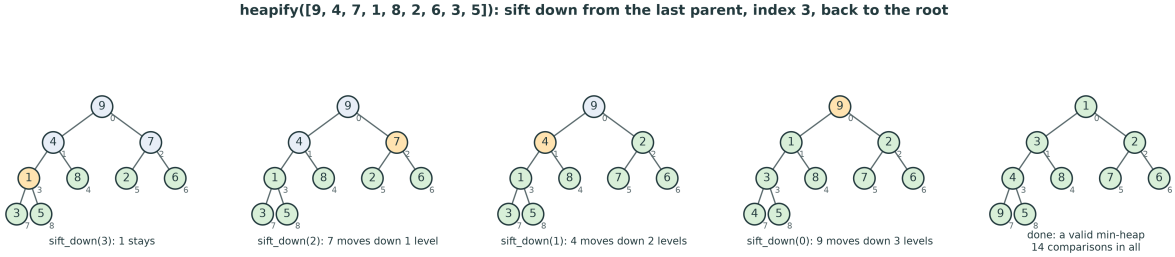
`MinHeap(values)` copies the values into the `DynamicArray` in their given order and calls `heapify`. The array then has the right shape — every array does — and possibly no order at all. `heapify` fixes the order **in place**.

The idea: a leaf is already a heap of one. The last parent, at `n // 2 - 1`, has only leaves below it, so a single sift-down makes its subtree a valid heap. Work backwards: by the time you sift down node i, both of its subtrees are already heaps, so the sift-down of i makes the subtree rooted at i a heap. When you reach index 0, the whole array is a heap. In the language of Lecture 08, the loop invariant is: *every subtree rooted at an index greater than i is a heap.*

Doing it **top down**, from index 0 forwards, does not work: sifting down the root first compares it with children that are not yet heaps, and the result is not a heap. The direction matters.

This bottom-up construction is Robert W. Floyd's, from 1964 ("Algorithm 245: Treesort 3"). The heap itself was introduced earlier the same year by J. W. J. Williams, as the data structure behind heap sort.

Trace: `heapify([9, 4, 7, 1, 8, 2, 6, 3, 5])`



n = 9, last parent = 9 // 2 − 1 = **3**. Sift down 3, 2, 1, 0.

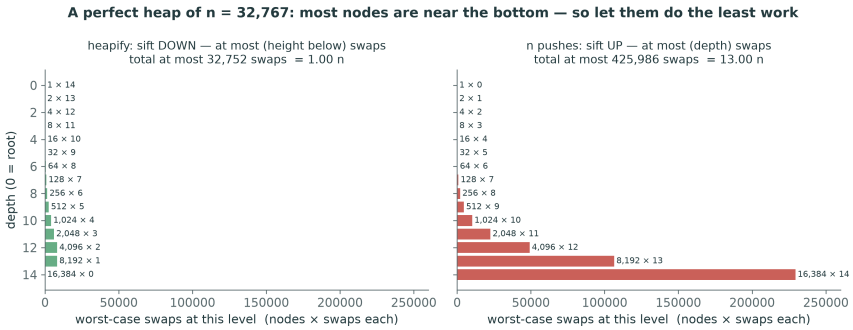
These are the values of `test_heapify_from_arbitrary_order`. Green nodes are subtrees that are already heaps; the amber node is the one being sifted down.

`sift_down(i)`	Before	Moves	After
3 (value 1)	children 3, 5	none — 1 beats both	`[9, 4, 7, 1, 8, 2, 6, 3, 5]`

`sift_down(i)`	Before	Moves	After
2 (value 7)	children 2, 6	swap with 2	`[9, 4, 2, 1, 8, 7, 6, 3, 5]`
1 (value 4)	children 1, 8	swap with 1, then with 3	`[9, 1, 2, 3, 8, 7, 6, 4, 5]`
0 (value 9)	children 1, 2	swap with 1, then 3, then 4	`[1, 3, 2, 4, 8, 7, 6, 9, 5]`

Six swaps and 14 comparisons (counted by running the code). Notice who moved furthest: the item at the **root**, which is the only node that can travel three levels. The five leaves moved on their own account not at all.

Why heapify is $O(n)$: count the work level by level



About $n/2^{h+1}$ nodes sit at **height h**; each sifts down at most h levels:

$$\sum_{h\geq 0} \frac{n}{2^{h+1}} \cdot h \;=\; \frac{n}{2} \sum_{h\geq 0} \frac{h}{2^h} \;=\; \frac{n}{2} \cdot 2 \;=\; n$$

A first, lazy estimate says: n/2 sift-downs, each $O(\log n)$, so $O(n \log n)$. That bound is true but not tight, because it charges every node the full height of the tree, and almost no node has it.

Count properly. In a complete tree, **half** the nodes are leaves (height 0) and do no work. A quarter are just above the leaves (height 1) and move at most one level. An eighth move at most two, and so on; only the root can move $\lfloor \log_2 n \rfloor$ levels. The total number of swaps is at most

$$0 \cdot \frac{n}{2} + 1 \cdot \frac{n}{4} + 2 \cdot \frac{n}{8} + 3 \cdot \frac{n}{16} + \cdots = \frac{n}{2} \left(\frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \ldots \right).$$

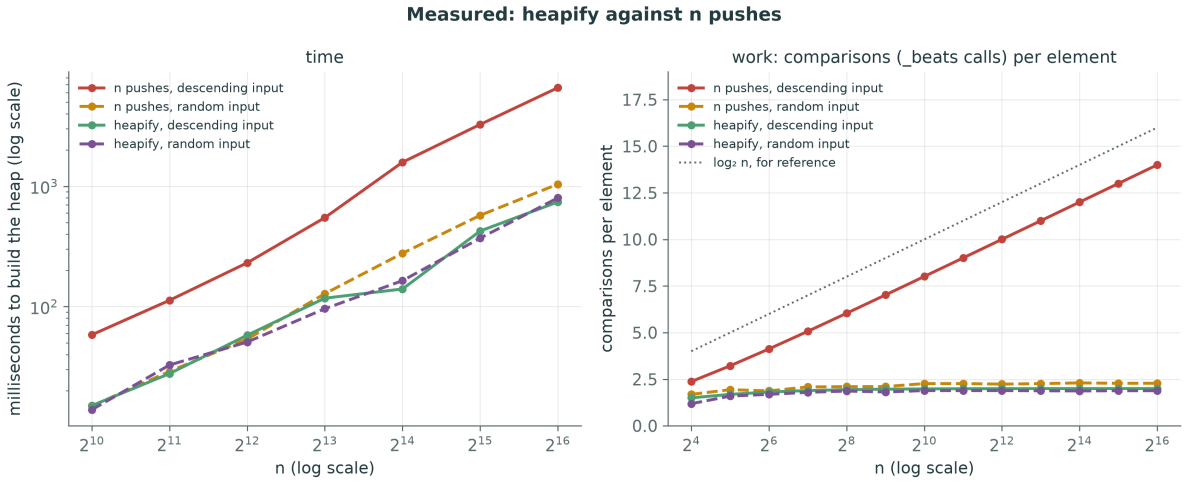
The series $\sum h/2^h = 1/2 + 2/4 + 3/8 + 4/16 + \ldots$ converges to **2** (a standard result; one way to see it is to write each term $h/2^h$ as h copies of $1/2^h$ and sum the geometric series column by column). So the total is at most n swaps, and at most 2n comparisons: $O(n)$.

The figure makes it concrete for a perfect heap of $n = 32{,}767$ (15 levels). Left: heapify. The widest bar, the 16,384 leaves, is zero; the tallest cost per node, 14, belongs to one node. The worst-case total is 32,752 swaps — $n - \log_2(n+1)$, just under n. Right: n pushes, where the

cost is the **depth**, not the height, and so the many nodes at the bottom pay the most. Worst case 425,986 swaps, 13 n. The same nodes, the same tree, and the opposite end of it doing the work.

That asymmetry is the whole trick: sift **down** costs what is **below** a node, and most nodes have little below them; sift **up** costs what is **above**, and most nodes have a lot above them.

Measured: heapify against n pushes



Real timings of the reference `dsa/heap.py`, built on the course `DynamicArray` exactly as yours will be.

Right: work — the number of calls to `_beats` per element, counted exactly. Heapify stays **flat**, at most 2 comparisons per element at every size (1.88 on random input, 2.00 on descending input at n = 2^{16}): $O(n)$ in total, as the summation promised. n pushes of **descending** values — the worst case for a min-heap, since every new value is the smallest so far and climbs to the root — run parallel to the dotted $\log_2 n$ line, about $\log_2 n - 2$: 14 per element at n = 2^{16}: $O(n \log n)$ in total. And n pushes of **random** values are flat too, at about 2.3 per element: the average push stops near the bottom, as the sift-up section said.

That last line is an honest complication. For **random** input, n pushes are also $O(n)$ on average; heapify's advantage is that it is $O(n)$ **in the worst case**, whatever the input. Big-O of the worst case and the typical cost are different questions, and both are worth asking (Lecture 02).

Left: time. On descending input, n pushes are slower than heapify by a factor that grows with n — about 4 times at n = 2^{10} and about 9 times at n = 2^{16} (6.6 seconds against 0.74). On random input the two are close, with heapify slightly ahead at the larger sizes: their comparison counts differ only by 2.3 against 1.9 per element. The kinks in the heapify lines are timer and machine noise — the comparison counts on the right have none. The slopes are what matter; the exact milliseconds will differ on your machine.

The Priority Queue on Top

`PriorityQueue`: a `MinHeap` of triples

`enqueue(item, priority)` pushes the triple

$$(\text{priority},\ \text{counter},\ \text{item})$$

and adds 1 to `counter`. `dequeue()` pops a triple and returns its **item**.

- Python compares tuples **left to right**: priority first.
- `counter` is unique, so the comparison **never reaches** `item`.

The skeleton says: build it **on top of** `MinHeap` rather than re-implementing the sifting. `PriorityQueue.__init__` is given, and creates `self._heap`, a `MinHeap`, and `self._counter = 0`. The heap does not know it is holding tasks; it compares whatever you push with `<`, and Python compares tuples **lexicographically** — the first elements, and only if they are equal, the second, and so on.

Why not the pair (priority, item)? Because two items with the same priority make Python compare the items themselves. For strings that silently orders equal-priority tasks alphabetically. For dictionaries, or most objects, it raises

```
TypeError: '<' not supported between instances of 'dict' and 'dict'
```

which is exactly what `test_priority_queue_handles_unorderable_items` provokes. The counter sits between the priority and the item as a **tie-breaker**. No two entries share a counter, so any two triples are decided by their first two elements, and the item is never compared.

A bonus: fairness among equals. The counter grows with every enqueue, so between two items of equal priority, the one enqueued **first** has the smaller counter and is dequeued first. The contract of `dsa/heap.py` only promises that ties "may break in any order", but a counter gives you **first come, first served among equals** at no cost. A bare heap does not: it is **not stable** (the heap sort of Week 10 is not stable for the same reason).

The tuple is not storage in the sense of the storage rule: it is one **value**, a small record, and the records live in your `DynamicArray`. This is the design of Python's own `heapq` documentation, which recommends exactly this triple.

Trace: triage

Operation	Heap array: `(priority, counter, item)`	Returns
enqueue A, 2	`(2,0,A)`	
enqueue C, 1	`(1,1,C)` `(2,0,A)`	
enqueue K, 3	`(1,1,C)` `(2,0,A)` `(3,2,K)`	
enqueue L, 1	`(1,1,C)` `(1,3,L)` `(3,2,K)` `(2,0,A)`	
dequeue	`(1,3,L)` `(2,0,A)` `(3,2,K)`	C
dequeue	`(2,0,A)` `(3,2,K)`	L

A broken arm · C chest pain · K sprained ankle · L allergic reaction

Produced by running the reference `PriorityQueue`. Chest pain and the allergic reaction share priority 1; the chest pain arrived first (counter 1 against 3), so it is served first. The broken arm arrived before both and waits, which is the point of a priority queue. When "allergic reaction" is pushed, it enters at index 3 under `(2,0,arm)` at index 1 and sifts up one level, pushing the arm down: `(1, 3, …) < (2, 0, …)` because 1 < 2, and the counters are not looked at.

Where priority queues are used

- **Scheduling:** an operating system runs the highest-priority ready process; a print server, a build server, a job queue.
- **Triage:** emergency departments; bug trackers ("P1 before P3").
- **Event simulation:** the next event is the one with the smallest time.
- **Top-k:** the k largest of n values, in $O(n \log k)$.
- **Graph algorithms:** Dijkstra's shortest paths, Prim's spanning tree, Huffman coding — *beyond this course's bylaw*.

Simulation. A bank or network simulator keeps future events — "customer 17 arrives at t = 4.2", "server 3 finishes at t = 5.0" — in a priority queue keyed by time. The main loop is: dequeue the earliest event, handle it, enqueue the events it causes. The clock jumps from event to event rather than ticking.

Beyond the bylaw. The course's bylaw lists heaps and priority queues, and graph *searches* (Week 14), but not shortest paths. Still, you should know the names. **Dijkstra's algorithm** (1959) finds shortest paths in a weighted graph by repeatedly taking the unvisited vertex with the smallest known distance — from a priority queue. **Prim's algorithm** grows a minimum spanning tree the same way, and **Huffman coding** (1952), which compresses the files in every ZIP archive, repeatedly merges the two least frequent symbols — two pops and a push. In each of them the priority queue is the engine, and its $O(\log n)$ operations are what make the algorithm fast. Week 14's BFS is Dijkstra with a FIFO queue in place of the priority queue, for graphs whose edges all weigh the same.

In Python the library version is the `heapq` module: functions `heappush`, `heappop` and `heapify` that treat an ordinary list as a min-heap, with exactly the index arithmetic of this lecture. There is no max-heap in `heapq`; the usual trick is to push `-x`. Write your own first; then use the library.

Top-k: keep a small heap, not a big one

The k **largest** of n values:

1. Keep a **min**-heap of at most k values.
2. For each value: if the heap has fewer than k, push it. Otherwise, if the value beats the heap's **smallest** (its root), pop the root and push the value.
3. The heap now holds the k largest.

$O(n \log k)$ time, $O(k)$ space. Sorting everything: $O(n \log n)$ and $O(n)$.

It feels backwards — a **min**-heap to find the **largest** values — until you see what the root is for. The heap holds the best k seen so far, and its root is the **weakest** of them: the one to throw out

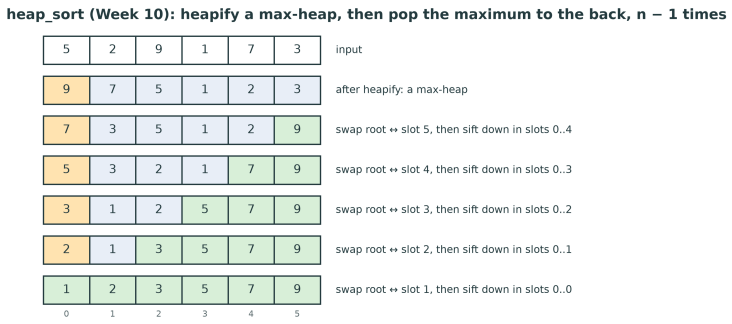
when something better arrives. A new value that does not beat the root cannot be among the top k, and costs one peek, $O(1)$.

For the 10 largest of a million values, the heap never holds more than 10 items: each step is $O(\log 10)$, a handful of comparisons, and the memory is 10 slots — so it works on a stream that does not fit in memory, such as a log file of a billion lines. Sorting the million needs all of them in memory and about 20 million comparisons. This is W12-C2 in the question bank.

The dual problem — the k **smallest** — uses a **max**-heap of size k.

Heap Sort, Revisited

Week 10 was a heap in disguise



1. `heapify` the array as a **max**-heap: $O(n)$.
2. Repeat n – 1 times: swap the root (the maximum) with the **last** item of the heap; the heap shrinks by one; sift the new root down: $O(\log n)$.

$O(n \log n)$ always, $O(1)$ extra space, **not stable**.

The study plan called Week 10's heap sort "week 12 arriving early". Here is the whole of `heap_sort_steps` from `solutions/dsa/sorting.py`, with the snapshots removed:

```
def _sift_down(a, index, size):
    """Max-heap sift-down of a[index] within a[0:size]."""
    while True:
        largest = index
        left, right = 2 * index + 1, 2 * index + 2
        if left < size and a[left] > a[largest]:
            largest = left
        if right < size and a[right] > a[largest]:
            largest = right
        if largest == index:
            return
        a[index], a[largest] = a[largest], a[index]
        index = largest

for index in range(n // 2 - 1, -1, -1):        # build the heap: O(n)
```

```
    _sift_down(a, index, n)
for end in range(n - 1, 0, -1):
    a[0], a[end] = a[end], a[0]                # the max goes to the back
    _sift_down(a, 0, end)                      # restore the heap in a[:end]
```

Recognise every line. The index arithmetic is `2 * index + 1` and `2 * index + 2`. The sift-down finds the **better** of three — here "better" means larger, because it is a max-heap. The first loop is `heapify`, from the last parent `n // 2 - 1` back to 0. The second loop is pop done in place: the root is swapped to the back of the array rather than returned, and the heap is told it is one smaller through the `size` argument rather than by shrinking the array.

Why a max-heap for an ascending sort? Each extraction puts the current maximum at the **end** of the shrinking heap, which is exactly where the largest remaining value belongs in the sorted output. The sorted part grows from the right while the heap shrinks from the right: one array, two regions, no extra memory. (The green cells in the figure are the sorted region.)

Why not stable? The swap of the root with the last item jumps an element across the whole array, over any equal values in between. Week 10 measured it.

The difference between the week-10 function and this week's class is only packaging: the class keeps the heap alive between operations, so items can arrive and leave in any order. That is what a priority queue needs, and what sorting does not.

This Week

Exercises: `dsa/heap.py`

Method	Target	The trap
`_sift_up`	$O(\log n)$	parent is `(i - 1) // 2`; stop at the root
`_sift_down`	$O(\log n)$	the **better** child; a missing right child
`push`, `peek`	$O(\log n)$, $O(1)$	`IndexError` on an empty `peek`
`pop`	$O(\log n)$	last item to the root; the one-item heap
`heapify`	$O(n)$	from `n // 2 - 1` **down** to 0 — not `n` pushes
`is_valid`	$O(n)$	check **every** child against its parent
`PriorityQueue`	$O(\log n)$	the triple; return the **item**

```
pytest tests/test_heap.py -v
```

Twenty-eight tests. Eleven of them run twice, once for `MinHeap` and once for `MaxHeap` — with `[MinHeap]` or `[MaxHeap]` in their names — so a `<` written outside `_beats` shows up as a set of `[MaxHeap]` failures. Five test the priority queue.

Write `heapify` early. Any test that builds a heap from a list — `cls([5, 3, 8, ...])` — goes through `__init__`, which calls `heapify`. So does the `heapify`-free-looking `test_pop_returns_sorted_order`. The lab orders the methods so that each part's tests can pass.

Use `_beats` for every comparison, and nothing else. Not `<`, not `min`.

Homework 12 — before Lecture 13

1. **Implement** `dsa/heap.py` until all 28 tests pass.
2. **Trace** by hand, as trees and as arrays: push 6, 2, 8, 1, 5, 3 into an empty `MinHeap`; then pop twice.
3. **Heapify** `[3, 9, 2, 1, 4, 5, 8, 7, 6, 0]` by hand. Count the swaps.
4. **Measure** heapify against n pushes on descending input in `notebooks/12-heaps.ipynb`, and count `_beats` calls with a subclass.

For item 2, check yourself with the real object: `list(h._items)` after each step, and `draw_array_as_tree` to see it. For item 3, predict the last parent first. For item 4, a subclass of `MinHeap` whose `_beats` adds 1 to a counter and then returns `super()._beats(child, parent)` counts every comparison your code makes — the method of the measured figure. Lab 12 walks you through it.

Summary

Seven things to keep

1. A priority queue serves the **most urgent** item; a heap gives $O(\log n)$ enqueue **and** dequeue.
2. **Shape:** a complete binary tree — height $\lfloor \log_2 n \rfloor$.
3. **Order:** every parent beats its children; siblings are unordered.
4. The array **is** the tree: children `2i + 1`, `2i + 2`; parent `(i - 1) // 2`.
5. `push` = append + **sift up**; pop = last to root + **sift down** past the **better** child.
6. `heapify` sifts down from `n // 2 - 1` to 0: $O(n)$, because most nodes are near the bottom.
7. `PriorityQueue` pushes `(priority, counter, item)`: the counter breaks ties — first come, first served.

Next

Week 13 — Hash tables. From $O(\log n)$ to $O(1)$ — **on average**. What "average" promises, and what breaks it.

A heap finds the **best** item fast, and nothing else; a binary search tree finds **any** item in $O(\log n)$ while it stays balanced. Week 13 asks for less than both — no order at all — and in exchange finds any item in $O(1)$ on average, by computing where it lives instead of searching for it.

Sources and further reading

- **J. W. J. Williams.** "Algorithm 232: Heapsort", *Communications of the ACM* 7(6), 1964 — the binary heap, introduced for sorting.
- **R. W. Floyd.** "Algorithm 245: Treesort 3", *Communications of the ACM* 7(12), 1964 — building the heap bottom up, in linear time.
- **T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein.** *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, chapter 6, "Heapsort" — heaps, `BUILD-MAX-HEAP` and its $O(n)$ bound, and priority queues (1-based indices).
- **R. Sedgewick and K. Wayne.** *Algorithms*, 4th ed., Addison-Wesley, 2011, §2.4 "Priority Queues" — *swim* and *sink*, and top-k on a stream.
- **D. E. Knuth.** *The Art of Computer Programming*, vol. 3, *Sorting and Searching*, 2nd ed., Addison-Wesley, 1998, §5.2.3 — heapsort and its analysis.
- **Python documentation.** The `heapq` module — including the `(priority, count, task)` recipe for a priority queue.

Every figure in this lecture is generated by `tools/figures_l12.py`. The trace figures are drawn from states recorded by running the reference solution; the measured figure is real measurement and will differ slightly on your machine.

Questions for Week 12

> ***The answers are at the back of the book,*** *in* Answers — Week 12. *Levels:* ***[what]*** *recall · **[how]** apply · **[why]** explain and justify.* `MinHeap`*,* `MaxHeap` *and* `PriorityQueue` *are the classes of* `dsa/heap.py`*. A heap is stored in an array* `_items`*, indexed from 0: the children of index* `i` *are* `2i + 1` *and* `2i + 2`*, its parent is* `(i - 1) // 2`*.* `push` *appends and sifts up;* `pop` *moves the last item to the root and sifts down, swapping with the* ***better*** *child;* `heapify` *sifts down from index* `n // 2 - 1` *back to 0.* `PriorityQueue.enqueue(item, priority)` *pushes the triple* `(priority, counter, item)`*; a lower number is served first. Unless a question says otherwise,* ***Q*** *is the min-heap*
>
> `Q = [1, 5, 3, 8, 6, 4, 9, 11, 10]`
>
> *of 9 values, at indices 0 to 8.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W12-M01 – W12-M22
B	Short answer and essay	W12-E1 – W12-E5
C	Trace the code	W12-T1 – W12-T5
D	Heap state — draw every step	W12-S1 – W12-S3
E	Complexity analysis	W12-K1 – W12-K3
F	Find and fix the bug	W12-B1 – W12-B4
G	Write the code — checked by `pytest`	W12-C1 – W12-C5

Part A — Multiple choice

W12-M01 [what] In a **min**-heap, which statement is always true?

- **a)** the array is sorted in ascending order
- **b)** every parent is less than or equal to its children
- **c)** every left child is less than or equal to its sibling
- **d)** the largest value is at the last index

W12-M02 [what] With 0-based indices, the **left** child of index i is at:

- **a)** 2i
- **b)** i + 1
- **c)** 2i + 1
- **d)** 2i + 2

W12-M03 [how] The parent of index 10 is at index:

- **a)** 4
- **b)** 5
- **c)** 3
- **d)** 9

W12-M04 [how] The height of a heap holding 1,000 values (a single node has height 0) is:

- **a)** 10
- **b)** 500
- **c)** 31
- **d)** 9

W12-M05 [how] Which array is a valid **min**-heap?

- **a)** `[1, 3, 2, 7, 4]`
- **b)** `[1, 3, 2, 2, 4]`
- **c)** `[2, 1, 3]`
- **d)** `[1, 4, 2, 3]`

W12-M06 [how] `Q.push(2)`: how many swaps does the sift-up make?

- **a)** 0
- **b)** 1
- **c)** 2
- **d)** 3

W12-M07 [how] `Q.pop()` returns 1. Which value is at the root afterwards?

- **a)** 5
- **b)** 3
- **c)** 10
- **d)** 4

W12-M08 [why] Sift-down swaps an item with the **better** of its two children, not just any child that beats it, because:

- **a)** it is faster to compare with the better child
- **b)** it keeps the tree complete
- **c)** the left child might not exist
- **d)** the child that moves up becomes the parent of its sibling, so it must beat the sibling too

W12-M09 [what] `heapify` on an array of n values sifts down first at index:

- **a)** 0
- **b)** n – 1
- **c)** n // 2 – 1
- **d)** n // 2 + 1

W12-M10 [what] The worst-case cost of `heapify` on n values is:

- **a)** $O(n)$
- **b)** $O(n \log n)$
- **c)** $O(\log n)$
- **d)** $O(n^2)$

W12-M11 [why] `heapify` costs less than n pushes because:

- **a)** it never looks at half of the values
- **b)** most nodes are near the bottom, and a sift-down costs only the levels **below** a node
- **c)** it uses sift-up, which is cheaper than sift-down
- **d)** the input is usually almost sorted

W12-M12 [how] Pushing the values n, n – 1, ..., 2, 1 (in that order) into an empty **min**-heap costs, in total:

- **a)** $O(n)$
- **b)** $O(\log n)$
- **c)** $O(n^2)$
- **d)** $O(n \log n)$

W12-M13 [what] `peek()` on a heap of n values costs:

- **a)** $O(\log n)$
- **b)** $O(1)$
- **c)** $O(n)$
- **d)** $O(n \log n)$

W12-M14 [what] Checking whether an arbitrary value x is in a heap of n values costs, in the worst case:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n \log n)$

W12-M15 [why] A heap is stored in an array, with no node objects, because:

- **a)** Python lists are faster than objects
- **b)** a heap is always sorted

- **c)** arrays can grow and nodes cannot
- **d)** its shape is complete, so there are no gaps, and index arithmetic replaces the child and parent references

W12-M16 [how] `enqueue("a", 2)`, `enqueue("b", 1)`, `enqueue("c", 1)`, then three `dequeue()`s return:

- **a)** a, b, c
- **b)** b, c, a
- **c)** c, b, a
- **d)** b, a, c

W12-M17 [why] `PriorityQueue` pushes `(priority, counter, item)` rather than `(priority, item)` because:

- **a)** it makes dequeue faster
- **b)** it counts the items in the queue
- **c)** when two priorities tie, Python would compare the items, which may not be comparable
- **d)** it turns the min-heap into a max-heap

W12-M18 [how] The k largest of n values, found with a min-heap of at most k values, cost:

- **a)** $O(n \log k)$
- **b)** $O(k \log n)$
- **c)** $O(n \log n)$
- **d)** $O(nk)$

W12-M19 [why] To find the k **largest** values you keep a **min**-heap of size k because:

- **a)** a max-heap cannot hold k values
- **b)** a min-heap is faster than a max-heap
- **c)** the smallest values must be found first
- **d)** its root is the weakest of the best k so far — the one to throw out when a better value arrives

W12-M20 [what] The extra space of heap sort (Week 10) is:

- **a)** $O(n)$
- **b)** $O(\log n)$
- **c)** $O(1)$
- **d)** $O(n \log n)$

W12-M21 [why] Heap sort sorts into **ascending** order with a **max**-heap because:

- **a)** each extraction swaps the maximum to the end of the heap, which is where it belongs in the sorted array
- **b)** a max-heap is stable
- **c)** a min-heap cannot be built in place
- **d)** a max-heap needs fewer comparisons

W12-M22 [how] After `MaxHeap([3, 1, 4, 1, 5, 9, 2, 6])`, the value at `_items[1]` is:

- **a)** 5

- **b)** 1
- **c)** 4
- **d)** 6

Part B — Short answer and essay

W12-E1 [why] *(4 marks)* Define a binary heap by its two properties. Explain why the **shape** property matters: what it gives the height, and what it gives the storage.

W12-E2 [how] *(4 marks)* Describe `push` and `pop` on a min-heap, including sift-up and sift-down, the cost of each, and the special cases the code must handle.

W12-E3 [why] *(4 marks)* Explain how `heapify` builds a heap bottom-up, why it must go from the last parent **backwards**, and show that it is $O(n)$. Why are n pushes $O(n \log n)$ in the worst case?

W12-E4 [why] *(3 marks)* Compare four ways to implement a priority queue — unsorted array, sorted array, binary heap, balanced binary search tree — by the cost of enqueue, dequeue and peek. Which would you choose, and why?

W12-E5 [why] *(3 marks)* `PriorityQueue` stores `(priority, counter, item)`. What goes wrong with `(priority, item)`? What does the counter add besides avoiding that error?

Part C — Trace the code

W12-T1 [how] Starting from an empty `MinHeap`, push 6, 2, 8, 1, 5, 3, then pop twice. Give the array after every operation, and each value returned.

W12-T2 [how] Trace `MinHeap([3, 9, 2, 1, 4, 5, 8, 7, 6, 0])`: for every `sift_down(i)` that `heapify` calls, give the array afterwards. How many swaps are made in all?

W12-T3 [how] Pop from Q three times. Give each value returned and the array after each pop.

W12-T4 [how] Trace this `PriorityQueue`. Give the heap array (as triples) after each operation, and each value returned.

```
pq = PriorityQueue()
pq.enqueue("print report", 3)
pq.enqueue("fix login bug", 1)
pq.enqueue("reply to email", 2)
pq.dequeue()
pq.enqueue("restart server", 1)
pq.enqueue("update docs", 3)
pq.dequeue(); pq.dequeue(); pq.dequeue()
```

W12-T5 [how] Trace `heap_sort` (Week 10) on `[4, 10, 3, 5, 1]`: the array after the build phase, and after each swap-and-sift-down.

Part D — Heap state — draw every step

W12-S1 [how] Starting from an empty `MaxHeap`, push 5, 12, 7, 20, 3, 15. Draw the tree **and** the array after each push, and mark each swap.

W12-S2 [how] Find the 3 largest of `[5, 1, 9, 3, 7, 2, 8]` with a min-heap of at most 3 values, as in Lecture 12's top-k. For each value, say whether it is pushed, replaces the root, or is skipped, and draw the heap array afterwards.

W12-S3 [why] Draw each array as a tree and say whether it is a valid **min**-heap. For each that is not, mark every parent–child pair that breaks the property.

- (a) `[1, 2, 3, 4, 5, 6]`
- (b) `[1, 3, 2, 5, 4, 1]`
- (c) `[2, 2, 2, 3, 2]`
- (d) `[4, 5, 6, 7, 8, 9, 3]`

Part E — Complexity analysis

W12-K1 [how] Give Θ and justify, for a list `values` of n numbers:

```
h = MinHeap()
for x in values:
    h.push(x)
result = [h.pop() for _ in range(len(values))]
```

What does `result` contain? How does it compare with heap sort in time and space?

W12-K2 [how] Give the cost of top-k (the k largest of n values) with a min-heap of size k, and compare it with sorting everything and taking k. Which is better for k = 10 and n = 10^6? For k = n?

W12-K3 [why] A student writes `heapify` **forwards**:

```
def heapify(self):
    for index in range(len(self._items) // 2):
        self._sift_down(index)
```

Is it correct? Give an input on which it fails, or prove that it works. What does a forward loop of `self._sift_up(index)` over **every** index cost, and is that one correct?

Part F — Find and fix the bug

W12-B1 [how]

```
def _sift_up(self, index):
    items = self._items
    while index > 0:
        parent = index // 2
        if not self._beats(items[index], items[parent]):
            return
        items[index], items[parent] = items[parent], items[index]
        index = parent
```

W12-B2 [why]

```
def _sift_down(self, index):
    items = self._items
    size = len(items)
    while True:
        left, right = 2 * index + 1, 2 * index + 2
        if left < size and self._beats(items[left], items[index]):
            child = left
        elif right < size and self._beats(items[right], items[index]):
            child = right
        else:
            return
        items[index], items[child] = items[child], items[index]
        index = child
```

W12-B3 [how]

```
def pop(self):
    if len(self._items) == 0:
        raise IndexError("pop from an empty heap")
    last = self._items.pop()
    best = self._items[0]
    self._items[0] = last
    self._sift_down(0)
    return best
```

W12-B4 [how]

```
def heapify(self):
    for index in range(len(self._items) // 2 - 1, 0, -1):
        self._sift_down(index)
```

Part G — Write the code

In `practice/week12.py`; check with `pytest tests/test_practice_week12.py -v`. Use your own `MinHeap` and `MaxHeap` as the working storage — not `sorted`, `list.sort` or `heapq`. Some tests count your comparisons.

W12-C1 [how] `is_min_heap(values)` — is a plain list a valid min-heap? $O(n)$.

W12-C2 [why] `top_k(values, k)` — the k largest, largest first, in $O(n \log k)$. Why a **min**-heap?

W12-C3 [how] `merge_sorted(lists)` — merge k sorted lists in $O(N \log k)$. What does each heap entry hold?

W12-C4 [why] `join_ropes(lengths)` — the cheapest total cost of joining ropes, two at a time, where a join costs the sum of the two lengths. Why is "always join the two shortest" right?

W12-C5 [why] `running_medians(values)` — the median after each value, with two heaps. Why do the two roots give the median?

Week 13 — Hash Tables

> ***How to read this document.*** *This is the handout for Lecture 13. It holds everything on the slides, plus what I said out loud. Every structure so far finds a value by walking or by halving. This week you build one that finds it by* ***computing where it is*** *— the structure behind Python's* `dict` *and* `set`*, and behind nearly every "look it up by name" in software. You will build it twice, measure it, and then break it on purpose, because its famous* $O(1)$ *is a promise about the* ***average****, and you should know exactly who can break it.*
>
> *Slides:* `DSA27-L13-slides.pdf` *· Code:* `dsa/hashmap.py` *· Tests:* `tests/test_hashmap.py`

Where We Are

Finding a value by its key

Structure	Find a key	Needs
unsorted array, linked list	$O(n)$	nothing
sorted array (Week 8)	$O(\log n)$	sorted order
binary search tree (Week 11)	$O(\log n)$ if balanced	order, balance
hash table (today)	$O(1)$ **on average**	a good hash function

Today's question: **can we find a key without comparing it to other keys at all?**

Every search in this course so far worked by **comparison**: is this the key? Is the key smaller or larger? Comparison-based search cannot beat $O(\log n)$ in the worst case — each comparison gives at most one bit of information, and telling n positions apart takes about $\log_2 n$ bits. A hash table escapes that bound the same way counting sort (Week 9) escaped $O(n \log n)$: it does not compare keys, it **computes** with them.

Hash table in Arabic: جدول التجزئة. *Hash function*: دالة التجزئة. *Key*: المفتاح.

Today

1. The dream: the key **is** the index
2. Hash functions: what `hash()` does, and what is and is not reproducible
3. Collisions, and the birthday paradox
4. Chaining: a linked list in every bucket
5. Open addressing: linear probing, clustering, **tombstones**
6. Load factor and resizing — amortised again
7. Measured: probes against load factor; and the keys that break it
8. Python's `dict` and `set`: the real thing, which you now understand

The Dream

Direct addressing: the key is the index

Keys are student numbers 0 to 999? Use an `Array(1000)`:

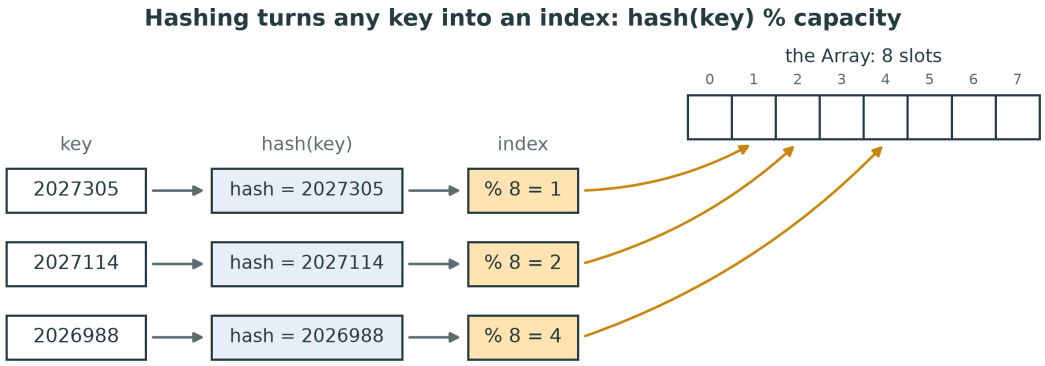
- `put(k, v)`: `a[k] = v` — $O(1)$
- `get(k)`: `a[k]` — $O(1)$

No search at all. **But:** the space is the size of the **key range**, not the number of keys.

This is the best possible dictionary, and you have had it since Lecture 02: indexing an `Array` is $O(1)$, so if the key **is** an index, there is nothing to search. Counting sort used exactly this in Week 9 — one counter per possible value.

It fails as soon as the keys are not small integers. Mansoura student numbers have seven digits: an `Array` indexed by them needs ten million slots to hold a class of two hundred. A name, a URL or a tuple is not an index at all. We need a function that squeezes **any** key into a **small** range of indices — and we must then live with what that squeezing costs.

Hashing: squeeze the key into the table



$$\text{index} = \text{hash}(\text{key}) \bmod \text{capacity}$$

Two steps, and it pays to keep them apart:

1. **`hash(key)`** turns the key into an integer. That is Python's job: every hashable object has a `__hash__`, and `hash()` calls it. You do not write a hash function this week; the docstring of `_index` says so.
2. **`% capacity`** folds that integer into the table: a number from 0 to capacity - 1, an index into the `Array`. That is the table's job, and it is one line of `dsa/hashmap.py`, given to you:

```
def _index(self, key):
    return hash(key) % len(self._buckets)
```

Python's `%` always returns a non-negative result for a positive capacity, even for a negative hash: `-5 % 8` is 3. (In C and Java, `-5 % 8` is -5, and a negative index is a crash — the same trap as the

ring buffer's step back in Lecture 07.)

The figure uses three seven-digit student numbers. For an `int`, `hash(n)` is just `n`, so the index is simply the last three bits of the number: 2027305, 2027114 and 2026988 land in slots 1, 2 and 4 of an 8-slot table. Ten million possible keys, eight slots. Something has to give — and it is the next section.

Hash Functions

Python's `hash()`

Expression	Value	Every run?
`hash(42)`	`42`	yes
`hash(-1)`	`-2`	yes
`hash(1.0)`, `hash(True)`	`1`	yes
`hash((1, 2))`	`-3550055125485641917`	yes
`hash("abc")`	a different number **each run**	**no**
`hash([1, 2])`	`TypeError: unhashable type: 'list'`	—

Every value in that table was printed by Python 3.14 on a 64-bit machine; the course's 3.13 gives the same. Three of them deserve a sentence.

- **`hash(n)` is `n`** for every integer you will meet in this course. (The exact rule is `n` reduced modulo the prime $2^{61} - 1$, so `hash(2**61 - 1)` is 0.) The one oddity is **`hash(-1)` is `-2`**: inside CPython, -1 is the return value that means "an error happened", so no object may hash to it.
- **`hash(1.0) == hash(1) == hash(True)`.** Because `1 == 1.0 == True` in Python, and the one rule every hash function must keep is: **equal keys must have equal hashes.** Otherwise `d[1.0]` would look in a different bucket from `d[1]` and miss a key that is, by `==`, the same one.
- **A list is unhashable.** A list can change after it is put in a table; its hash would change with it, and the table would look for it in the wrong bucket. So Python refuses. Tuples of hashable things, strings, numbers and frozensets are hashable; lists, dicts and sets are not. Your `put` inherits this for free: `hash(key)` raises the same `TypeError`.

Same run, same hash. Next run — maybe not.

```
python -c "print(hash('abc'))"      # -60919065288970757
python -c "print(hash('abc'))"      # 2138548942786055565
python -c "print(hash(12345))"      # 12345, every time
```

- **Deterministic within one run:** always — a table depends on it.
- **Across runs:** `int`, `float`, tuples of them — yes. **`str` and `bytes` — no**: randomised on purpose.
- `PYTHONHASHSEED=0` fixes the seed, for reproducing a bug.

Since Python 3.3, the hash of a `str` (and of `bytes`) is **randomised**: it is computed with a secret key chosen when the interpreter starts. Since Python 3.4 the function is SipHash, a keyed hash designed for exactly this job (PEP 456), and since 3.11 its faster SipHash-1-3 variant (`sys.hash_info.algorithm` prints `siphash13`). So:

- **Inside one run**, `hash("abc")` is the same every time you ask. A table could not work otherwise.
- **In the next run** it is almost certainly different. So the order in which your `ChainingHashMap` iterates string keys can change from one run to the next, and so can which bucket a string key lands in. A bug that depends on where keys land — the lab has one — can pass on one run and fail on the next.
- **Integers are not randomised.** Nor are floats, or tuples made of ints and floats. That is why every trace in this lecture, the lab and the question bank uses **integer keys**: `hash(n) = n`, and anyone can check them by hand.

Why randomise at all? Section 7 answers that: an attacker who can predict your hashes can choose keys that all collide. To make string hashes reproducible — to chase a bug that only shows on some runs — fix the seed before Python starts: `$env:PYTHONHASHSEED = "0"` in PowerShell, `PYTHONHASHSEED=0` in a Linux shell. Unset it afterwards.

(Objects of your own classes hash by **identity** unless you define `__hash__` — based on where the object lives in memory, so that is not reproducible across runs either.)

What makes a good hash function

1. **Consistent:** `a == b` ⇒ `hash(a) == hash(b)`. Non-negotiable.
2. **Spreads keys evenly** over the indices — similar keys, different slots.
3. **Uses the whole key**, not just part of it.
4. **Fast** — it runs on every single operation.

A bad one: **sum of the character codes**. `"listen"`, `"silent"` and `"enlist"` all hash to 655.

Rule 1 is about correctness; the other three are about speed. A hash function that returns 0 for everything is **correct** — every table built on it still finds every key — it is just $O(n)$, because every key lands in one bucket. Hold on to that: a bad hash never makes a table *wrong*, only *slow*. (W13-E2 asks you to argue it.)

The sum of character codes breaks rule 3 in a subtle way: it uses every character but ignores their **order**, so every anagram collides. Here is a toy hash you can compute by hand, with the fix:

```python
def sum_hash(s):                  # bad: order-blind
    return sum(ord(c) for c in s)

def poly_hash(s):                 # better: position matters
    h = 0
    for c in s:
        h = 31 * h + ord(c)
    return h
```

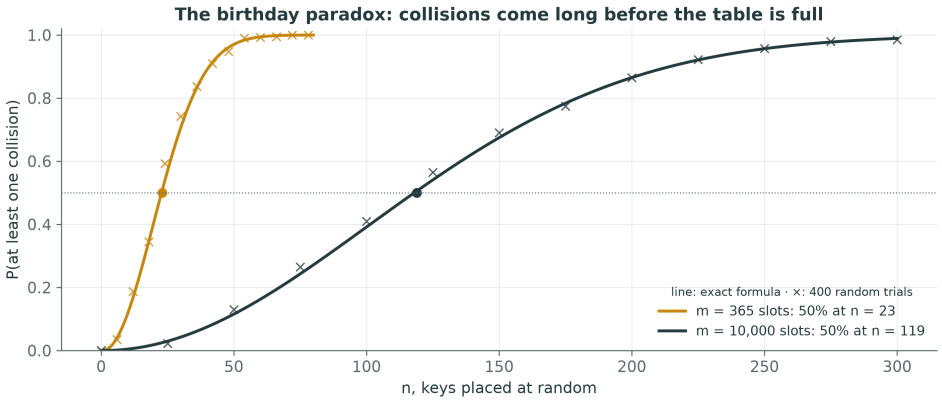
`sum_hash` gives 655 for all three anagrams; `poly_hash` gives `"listen"` 3192458695, `"silent"` 3392640085 and `"enlist"` 2996453575 (these numbers are deterministic — no randomisation here, you can check them). `poly_hash` is the scheme Java's `String.hashCode` uses (Java then keeps only 32 bits). It is a polynomial in 31, evaluated by Horner's rule: each character is multiplied by a different power of 31 according to its position.

Why % with a power of two is risky. `dsa/hashmap.py` starts at capacity 8 and doubles: the capacity is always a power of two, and `h % 2**k` keeps only the **last k bits** of the hash. If the keys differ only in their high bits — multiples of 1,024, say — they all land in slot 0. Choosing a **prime** capacity spreads such keys; CPython and Java keep powers of two (so % is a fast bit mask) and instead **mix the high bits in**: Java's `HashMap` uses `h ^ (h >>> 16)`, and CPython's probe sequence folds in the high bits as it goes. Section 7 measures what happens when nobody does.

Collisions

Collisions are certain

- **Pigeonhole:** more possible keys than slots ⇒ some keys **must** share.
- **Birthday paradox:** it happens **early**.



A **collision** is two different keys with the same index. They are not a sign of a bad hash function — with ten million student numbers and eight slots, they are forced. The question is only how early they come.

Much earlier than intuition says. With 23 people in a room, the chance that two share a birthday is 50.7%, although there are 365 days. The exact probability that n keys placed at random in m slots all miss each other is

$$P(\text{no collision}) = \frac{m}{m} \cdot \frac{m-1}{m} \cdots \frac{m-n+1}{m}$$

and it drops below one half at about $n \approx 1.18\sqrt{m}$. For m = 10,000 slots, that is n = 119: a table **one percent full** has probably already had a collision. The crosses in the figure are 400 random trials at each n, and they sit on the formula.

Collision in Arabic: تصادم.

The lesson for the design: **every hash table needs a collision strategy**, not as a rare fallback but as a normal path that runs all the time. There are two families.

Two ways to live with collisions

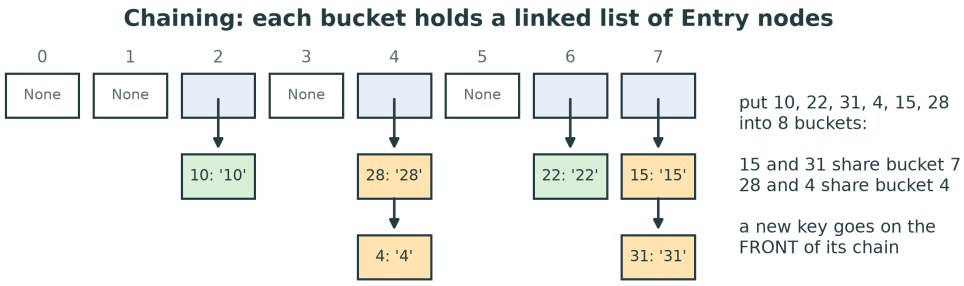
	Chaining	Open addressing
a slot holds	a **list** of entries	**one** entry
on collision	add to the list	**probe** another slot
load factor	may exceed 1	must stay **< 1**
delete	unlink a node	leave a **tombstone**
used by	Java `HashMap`	Python `dict`, `set`

Chaining in Arabic: "التسلسل". *Open addressing*: "العنونة المفتوحة".

You build both. `ChainingHashMap` reuses what you learnt in Week 5 — every bucket is a small linked list of `Entry` nodes. `OpenAddressingHashMap` keeps everything in the `Array` itself and needs one genuinely new idea, the tombstone. Their tests are the same eight, run on each class, plus one test that only open addressing needs.

Chaining

A linked list in every bucket



`self._buckets` is an `Array`; each slot is `None` or the **first** `Entry` of a chain.

`Entry` is given to you, and it is a Week 5 node with one extra field:

```
class Entry:
    __slots__ = ("key", "value", "next")

    def __init__(self, key, value, next=None):
        self.key = key
        self.value = value
        self.next = next
```

The figure is the real state of a `ChainingHashMap(capacity=8)` after `put(10, '10')`, `put(22, '22')`, `put(31, '31')`, `put(4, '4')`, `put(15, '15')` and `put(28, '28')`, read back out of

its buckets by `tools/figures_l13.py`. The indices are just the keys mod 8: 10 → 2, 22 → 6, 31 → 7, 4 → 4, 15 → 7, 28 → 4. Two collisions, two chains of two.

Why the front? A new key is linked in at the **head** of its chain — `push_front`, $O(1)$, with no walk to the tail. But `put` has to walk the chain anyway, to find out whether the key is already there (then it overwrites rather than adding a duplicate). Adding at the front is simply the shortest code once the walk has found nothing.

The storage rule holds: the buckets are an `Array`, the chains are node objects. No Python `list` or `dict` anywhere — which would, after all, be cheating on a hash table.

The three operations, in words

get(key): walk the chain at `_index(key)`; return the value of the entry whose key `==` `key`; at the end of the chain: `default`, or **`KeyError`**.

put(key, value): walk the same chain; if the key is there, **overwrite** and return. Otherwise push a new `Entry` on the front; `size += 1`; if `size / capacity > max_load`, **resize** to twice the capacity.

delete(key): walk with **two** references, `prev` and `entry`; unlink the match — the head of the chain is the special case; `size -= 1`. Not found: **`KeyError`**.

Those are the exercises; turning them into Python is your job. Three details.

- **Compare keys with `==`, not `is`.** Two equal strings may be two different objects. `hash` found the bucket; `==` finds the entry in it.
- **`get` has a sentinel default.** `get(key)` with no default raises `KeyError`; `get(key, "fallback")` returns the fallback — and so must `get(key, None)`. The skeleton uses a private object, `_MISSING`, as the "no default given" marker, because `None` is a value a caller may legitimately pass (and store). The given `__contains__` calls `get(key)` and catches `KeyError` for the same reason: a key whose value is `None` is still **in** the table.
- **`delete` is Week 5's `remove`**, on a list whose head lives in an `Array` slot instead of a `head` field. When the match is the first entry, the bucket slot itself must change; otherwise `prev.next` skips over the match.

Trace: chaining, capacity 8, `max_load` 0.75

Operation	Index	Buckets (non-empty)	size	load
put 10, 22, 31, 4	2, 6, 7, 4	2:10 · 4:4 · 6:22 · 7:31	4	0.5
put 15	7	… 7: **15** → **31**	5	0.625
put 28	4	4: **28** → **4** …	6	0.75
put 7	7	7 > 0.75·8: **resize to 16** 4:4 · 6:22 · 7:7 · 10:10 · 12:28 · 15: 31 → 15	7	0.4375
delete 31	15	15: 15	6	0.375

Produced by running the reference solution and printing its buckets. Three things to see.

- **0.75 is allowed; above it is not.** After `put 28` the load is exactly 0.75, and the test is `>`: no resize. `put 7` makes it 7/8 = 0.875, and the table doubles.
- **After the resize, every key has a new index**, because the index is `key % 16` now: 10 moves from bucket 2 to bucket 10, 28 from 4 to 12, 31 from 7 to 15. 15 and 31 still collide — they differ by 16. The chain order came out reversed (31 → 15), because `_resize` pushes each moved entry on the front of its new chain; order within a chain means nothing.
- **Delete unlinks the head.** 31 was first in bucket 15, so the bucket slot itself now points at 15.

Cost: $1 + \alpha$

Load factor: $\alpha = n/\text{capacity}$ — the average chain length.

- successful `get`: about $1 + \alpha/2$ entries examined
- unsuccessful `get`: about α entries — the whole chain
- keep $\alpha \leq 0.75 \Rightarrow O(1)$ **on average**

Worst case: every key in **one** chain — $O(n)$.

Load factor in Arabic: معامل التحمي.

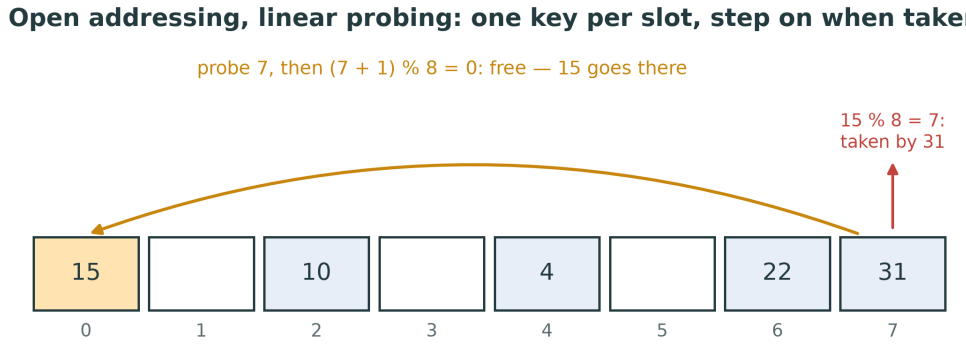
Assume the hash spreads keys as if at random (the "simple uniform hashing" assumption of the textbooks). Then a chain has, on average, α entries. An unsuccessful search walks the whole chain of its bucket: α entries, plus the $O(1)$ cost of hashing. A successful one stops, on average, halfway along the chain of a key that is there: $1 + \alpha/2$. (CLRS, theorems 11.1 and 11.2.) Section 7 shows both formulas holding in measurement, within a percent.

So a hash table is $O(1)$ **only while** α **is bounded by a constant.** That is the whole reason for the resize: without it, $\alpha = n/8$ and every operation is $O(n)$ — a linked list with an array of eight heads in front. `test_load_factor_stays_bounded` puts 500 keys and checks that the load factor is not above `max_load + 0.05`; a table that never resizes shows 62.5.

The worst case does not go away: $O(n)$ happens when many keys share a bucket, and **no** resize fixes it, because keys with equal hashes collide at every capacity. That is Section 7.

Open Addressing

Linear probing: one key per slot



Two `Arrays`, `_keys` and `_values`. Start at `hash(key) % capacity`; if that slot is taken by another key, try the **next**: `(i + 1) % capacity`.

No nodes, no chains: the table **is** the array. A collision is resolved by walking forward to the next free slot, wrapping round the end with `%` exactly like the ring buffer of Lecture 07. The sequence of slots a key tries is its **probe sequence**; with linear probing it is

$$h,\ h+1,\ h+2,\ \ldots \pmod{\text{capacity}}, \qquad h = \text{hash}(\text{key}) \bmod \text{capacity}$$

In `dsa/hashmap.py`, `_probe(key)` is a **generator** (Lab 03) that yields those indices, capacity of them at most — every slot once. `put`, `get` and `delete` then all loop `for i in self._probe(key):`, and only the loop body differs. If you later want quadratic probing (the docstring's challenge), you change `_probe` and nothing else.

The figure is the real state after `put` 10, 22, 31, 4 and 15 into 8 slots. 15 hashes to 7, which 31 holds; the probe moves to `(7 + 1) % 8 = 0`, which is free. Note what that means for **`get(15)`**: it must follow the **same** probe sequence, and it cannot stop at slot 7 just because slot 7 holds something else.

When does a search stop? At the key (found), or at a **never-used** slot, `None` — because `put` would have put the key there. `get(23)` on this table: 23 % 8 = 7 holds 31, slot 0 holds 15, slot 1 is `None` — 23 is not in the table, after 3 probes.

Trace: open addressing, capacity 8, `max_load` 0.66

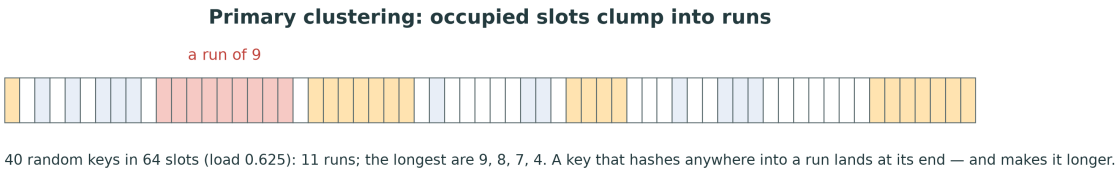
Operation	Home	Probes	Slots 0–7	size
put 10, 22, 31, 4	2, 6, 7, 4	1 each	`_ _ 10 _ 4 _ 22 31`	4
put 15	7	7, **0**	`15 _ 10 _ 4 _ 22 31`	5
delete 31	7	7	`15 _ 10 _ 4 _ 22 †`	4
get 15	7	7 (†, go on), **0**	unchanged	4
put 23	7	*load check: rebuild first*	`23 _ 10 _ 4 _ 22 15`	5

† is the tombstone.

Produced by the reference solution. The first two rows are the figure. The interesting rows are the last three.

- **delete 31** does not empty slot 7. It leaves a **tombstone** there — the module's `TOMBSTONE` object — and the reason is the next row.
- **get 15** starts at 7, as it must. Slot 7 holds a tombstone: "something was here once; keys may have probed past it". So the search goes on, to slot 0, and finds 15. Had delete emptied slot 7, `get(15)` would have stopped there and said "not in the table" — wrong.
- **put 23** runs the load check first, and **counts the tombstone as used**: (4 live + 1 tombstone + 1 new) / 8 = 0.75 > 0.66. But the live keys alone, (4 + 1) / 8 = 0.625, are within the limit, so the table does not grow: it **rebuilds at the same capacity**, dropping the tombstone. 15 moves back to its home slot 7, and 23 probes 7, then 0. Why count tombstones at all? See the next slide but one.

Primary clustering



A key that hashes **anywhere** into a run lands at its **end** — and the run grows. Long runs get longer faster.

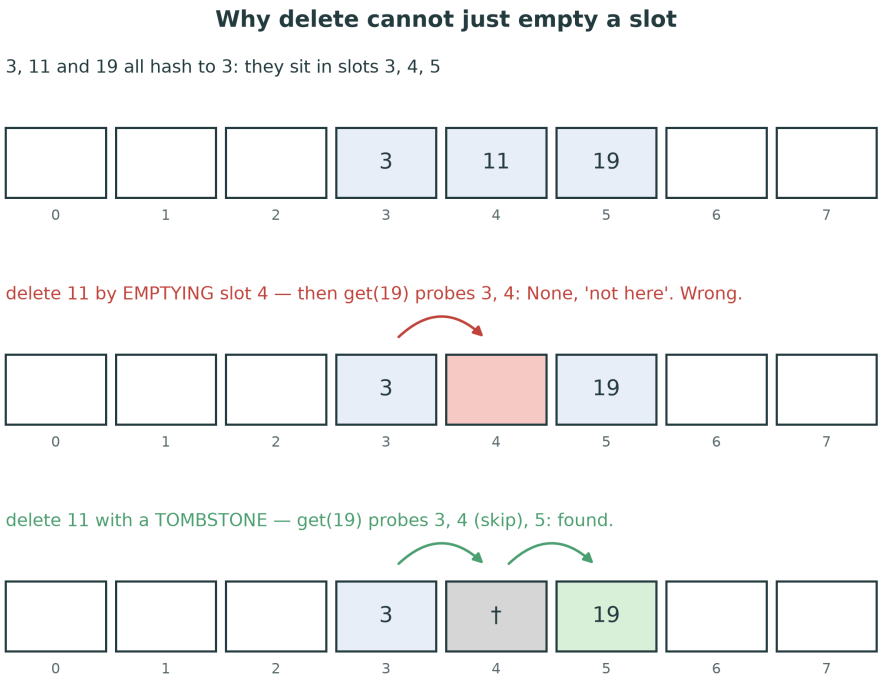
This is the price of linear probing's simplicity, and it has a name: **primary clustering**. The figure is a real `OpenAddressingHashMap` with 64 slots and 40 random integer keys: at a load of only 0.625, the occupied slots are not spread out but clumped into runs, the longest 9 slots long (and the table is a ring, so the run that ends at slot 63 carries on at slot 0).

Why runs feed themselves: a run of length L is hit by any key whose home is in it, or just before it — L + 1 homes out of the capacity. So a long run is a bigger target than a short one, and every hit makes it longer still. A search that starts in a run must walk to its end. Knuth's analysis of linear probing (1963, and *TAOCP* vol. 3, §6.4) gives, for a random hash:

$$\text{hit} \approx \tfrac{1}{2}\left(1 + \tfrac{1}{1-\alpha}\right), \qquad \text{miss} \approx \tfrac{1}{2}\left(1 + \tfrac{1}{(1-\alpha)^2}\right)$$

At $\alpha = 0.5$: 1.5 and 2.5 probes. At $\alpha = 0.9$: 5.5 and 50.5. The square in the miss formula is clustering. **Quadratic probing** ($h, h+1, h+4, h+9, \ldots$) and **double hashing** (a step size taken from a second hash) break up the runs; they are the docstring's challenge. And yet linear probing is widely used, because consecutive slots are next to each other in memory, and on real hardware a few extra probes in the same cache line cost less than one jump elsewhere.

Delete: why not just empty the slot?



This is the one subtle idea of the week, and the one the tests check on its own (`test_open_addressing_delete`). Keys 3, 11 and 19 all hash to 3, so they fill slots 3, 4, 5. A probe for 19 **passes through** slots 3 and 4. Empty slot 4 when 11 is deleted, and 19 becomes unreachable: `get(19)` stops at the `None` in slot 4, because `None` means "no key ever probed past here". The table has not lost 19 — it is still in slot 5 — it has lost the **path** to it.

A tombstone — "deleted, but something lay here" — keeps the path open:

Operation	Meets `None`	Meets `TOMBSTONE`
`get`, `delete`	stop: not found	**skip it**, keep probing
`put`	stop: insert here — or at the first tombstone passed	remember the first one; **keep probing** in case the key is further on
`_resize`	—	**drop it**: only live keys are re-inserted

The `put` row hides a bug worth naming. A put may **reuse** a tombstone's slot — that is the point of reusing space — but it must not stop at the first tombstone, because the key may already be further along the probe sequence. Insert at the first tombstone without looking further, and the table holds the same key **twice** (W13-B3).

Why count tombstones in the load check. Tombstones are not keys, so `len()` and `load_factor` ignore them. But they are not `None` either, and only `None` stops a probe. A table that sees many puts and deletes fills up with tombstones; if the load check counts only live keys, the table never rebuilds, and eventually there is **no** `None` left: every unsuccessful search walks the

whole table, $O(n)$, and a probe loop that only stops at `None` never stops at all. So `put` checks (size + tombstones + 1) / capacity. When the live keys alone are within `max_load`, it rebuilds at the **same** capacity — a sweep that throws the tombstones away; otherwise it doubles.

Tombstone in Arabic: شاهد القبر — literally, a gravestone.

The alternative, without tombstones. For linear probing only, a delete can instead **shift back** the keys after the hole that would otherwise be cut off ("backward-shift deletion"; Knuth's Algorithm R, §6.4). It leaves no tombstones, at the price of a trickier loop. It is an honest option; the course uses tombstones because they work for every probe sequence.

Load Factor and Resizing

Resize: every key moves

$$\alpha = \frac{\text{size}}{\text{capacity}} \qquad \text{chaining: } \alpha \leq 0.75 \qquad \text{open addressing: } \alpha \leq 0.66$$

When a `put` would pass the limit: allocate **twice** the buckets and **re-insert every key**.

You **cannot copy the buckets across**: the index is `hash % capacity`, and the capacity just changed.

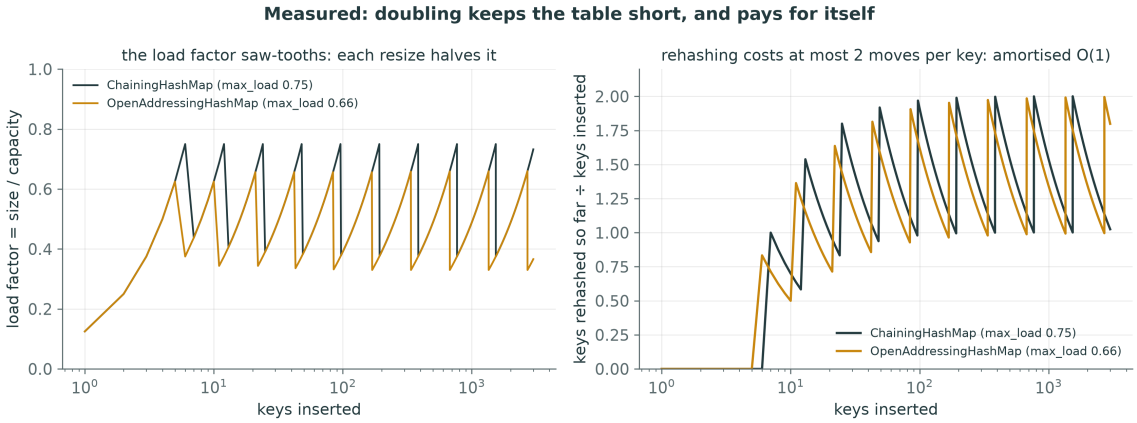
This is `_resize(capacity)`, the last method of each class. Its docstring says it in one line: "Every key must be rehashed". A key in bucket 2 of 8 — say 10 — belongs in bucket 10 of 16. Copy `old[i]` to `new[i]` and the table *looks* fine: all 7 keys are there, `len` is right. But `get(10)` now looks in bucket 10, finds nothing, and raises `KeyError`. The lab makes you watch it happen.

Resizing a chaining table **re-links the existing `Entry` nodes** into the new buckets instead of creating new ones: walk each old chain, and for each entry compute its new index and push it on the front of that chain. Save `entry.next` **before** you relink the entry — relinking overwrites it, and the rest of the old chain is lost (W13-B2 in the question bank).

For open addressing, `_resize` makes two new `Array`s, resets `size` and the tombstone count to 0, and re-inserts every **live** key — skipping `None` and `TOMBSTONE`. In the new table there are no tombstones and no duplicates, so each key can simply go into the first `None` of its probe sequence.

Why different limits? A chain can be longer than 1, so a chaining table still works at $\alpha = 1$ or 2 — only slower. An open-addressing table cannot exceed $\alpha = 1$ at all, and by the formulas above its misses cost 5 probes at 0.7 and 50 at 0.9. Java's `HashMap` resizes at 0.75; CPython's `dict` at about two-thirds.

Amortised $O(1)$ — Week 4 again



Left: the load factor of each table after every one of 3,000 puts, measured. It climbs to the limit, halves at the resize, and climbs again — a saw-tooth that never leaves the band between about 0.33 and 0.75.

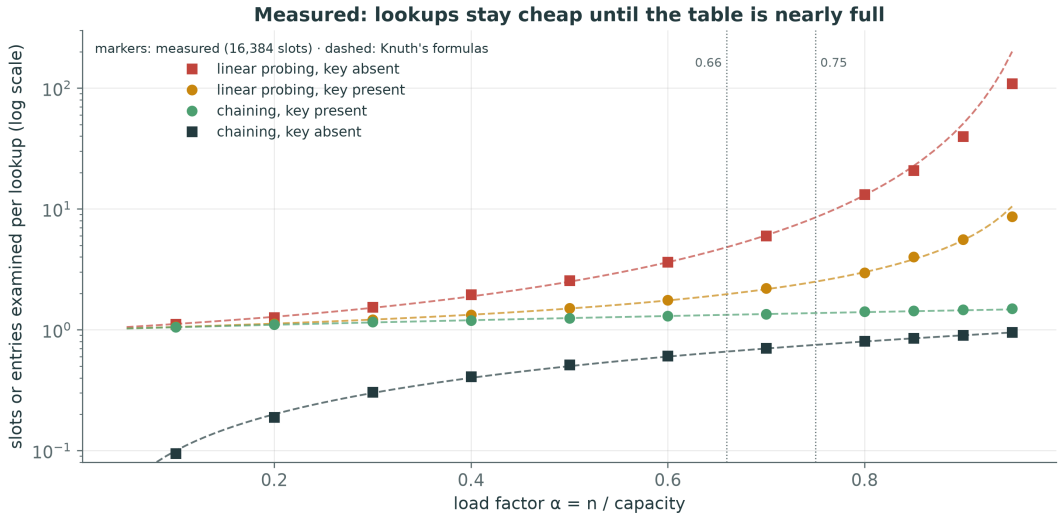
Right: the total number of keys rehashed so far, divided by the number of keys inserted. One resize is expensive — the chaining table's last one moved 1,537 keys at once — but it never rises above 2 per key. The argument is Lecture 04's exactly: resizes happen at sizes s, s/2, s/4, ..., so the total moved is at most $s + s/2 + s/4 + \cdots < 2s$. Over n puts, the rehashing costs $O(n)$ in total: **amortised $O(1)$ per put**. Over the 3,000 puts, the chaining table rehashed 3,075 keys in all and the open-addressing table 5,396.

The `DynamicArray` copied **values**; a hash table must **rehash** them — call `hash()` again and recompute every index. CPython's `dict` avoids the repeated `hash()` by storing each key's hash beside it; `Entry` here does not, which keeps it simple and is fine for `int` keys, whose hash is free. Python's `str` caches its own hash inside the string object for the same reason.

Should a table also **shrink** when keys are deleted? Ours never does. Shrinking at a quarter full keeps the memory proportional to the size with the same amortised bound — the same answer as for a `DynamicArray` that shrinks (Lecture 04). CPython's `dict` does not shrink on delete either.

Measured

Probes against load factor

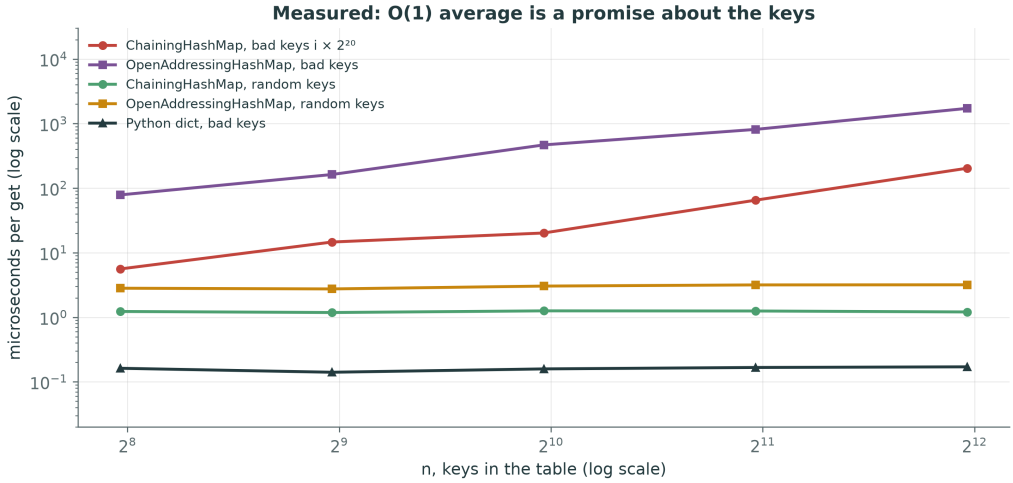


Every marker is measured on the reference `dsa/hashmap.py`: tables of 16,384 slots, filled with random integer keys to each load factor without resizing, then searched for keys that are there and for keys that are not. For open addressing the count is the number of slots `_probe` yielded; for chaining, the number of entries compared. The dashed lines are the formulas of the previous sections. They agree to within a few percent everywhere.

α	linear probing, hit	linear probing, miss	chaining, hit	chaining, miss
0.5	1.51	2.56	1.25	0.52
0.7	2.21	6.01	1.35	0.70
0.9	5.58	39.74	1.47	0.90
0.95	8.67	108.83	1.49	0.96

Read two things off it. First, **below about 0.7 everything is a handful of probes** — that is what $O(1)$ looks like, and why the limits sit where they do. Second, **open addressing falls off a cliff** as $\alpha \to 1$, while chaining degrades gently: the miss curve of linear probing grows like $1/(1-\alpha)^2$. A chaining miss can cost less than one comparison, because many buckets are empty.

What breaks $O(1)$



Keys `i * 2**20` all end in twenty zero bits: in **every** table of $2^k \le 2^{20}$ slots they share **one** bucket. Each `get` becomes $O(n)$.

"O(1) on average" hides an assumption: the keys spread over the buckets. The figure times one `get` on tables of n keys, for two kinds of keys.

- **Random keys** (the flat lines): about 1.3 µs per `get` for chaining and 3 µs for open addressing, whatever n is — $O(1)$. (Open addressing is slower here only by a constant: a generator call per probe and two `Array`s to read.)
- **Keys** $i \times 2^{20}$ (the rising lines): every one is a multiple of 2^{20}, so `key % capacity` is 0 for any power-of-two capacity up to 2^{20}, and resizing never helps. Every key lands in one chain, or one cluster, and each `get` costs $O(n)$: about 190 µs per `get` at n = 4,000 for chaining, 150 times slower than random keys, and the gap doubles with n. Building the table is $O(n^2)$.
- **Python's `dict` with the same bad keys** stays at about 0.15 µs. Its probe sequence mixes the high bits of the hash in (next slide), so keys that agree in their low bits do not stay together.

Adversarial keys. A bad hash is an accident; an attacker is not. In 2003 Crosby and Wallach showed that anyone who can predict a program's hash function can send it keys that all collide — a denial of service from a few kilobytes of input. In December 2011 Klink and Wälde showed it working against the web frameworks of the day: one HTTP POST with thousands of colliding form-field names kept a server's CPU busy for minutes, because the framework stored the fields in a hash table. The fix in Python was the **hash randomisation** of Section 2 — optional from 3.2.3, on by default since 3.3, with SipHash since 3.4 (PEP 456): if the attacker cannot predict `hash("...")`, they cannot choose colliding strings. Integers were left unrandomised, as you have just seen, and Java took another route: since Java 8, a `HashMap` chain longer than 8 turns into a balanced tree, so even a total collision costs $O(\log n)$, not $O(n)$.

So when someone says a hash table is $O(1)$, the honest version is: $O(1)$ **expected, if the hash function spreads the keys;** $O(n)$ **worst case, and the worst case can be forced by whoever chooses the keys.**

The Real Thing

Python's `dict` and `set` — which you now understand

- **Open addressing**, capacity a power of two, **8** slots to start.
- Grows at about **⅔** full (your `max_load = 0.66`).
- A deleted key leaves a **dummy** — a tombstone.
- Probes `i = (5·i + 1 + perturb) mod 2^k`, with `perturb` feeding in the high bits of the hash.
- Since Python 3.7, a `dict` keeps **insertion order**. A `set` does not.

Everything on that slide is in `Objects/dictobject.c` of CPython, and every idea in it is one you have now built.

- **Compact dict (3.6).** A `dict` keeps two arrays: a small sparse **index table** — the hash table proper, probed exactly like yours — and a dense **entries array** of (hash, key, value) in insertion order. The index table holds only positions in the entries array. Iterating a `dict` walks the entries array, so it comes out in insertion order, and the language has guaranteed that since 3.7. Your `ChainingHashMap` iterates in bucket order, which depends on the hashes — for string keys, a different order every run.
- **Perturbation.** Instead of `i + 1`, CPython steps with `i = (5*i + 1 + perturb) % 2**k` and shifts `perturb` right by 5 bits each time; `perturb` starts as the full hash. The first probes use the high bits that `%` threw away, which is why the bad keys above did not hurt it — and once `perturb` reaches 0, `5*i + 1` still visits every slot.
- **Dummy entries** are CPython's tombstones, for the same reason as yours; and a resize drops them, as yours does.
- **Each entry stores its hash**, so a resize never calls `hash()` again, and two keys are compared with `==` only when their full hashes are equal.
- A **`set`** is a separate hash table of keys only, with the same ideas; it probes a few neighbouring slots linearly before perturbing, and it has no insertion order.

`in` on a `list` is $O(n)$; `in` on a `set` or a `dict` is $O(1)$ on average. Lab 03 told you that `set` membership is "O(1) average"; now you know where the "average" comes from.

Hash table, sorted array or tree?

	Hash table	Sorted array	Balanced BST
find by key	$O(1)$ avg	$O(\log n)$	$O(\log n)$
insert, delete	$O(1)$ avg, amortised	$O(n)$	$O(\log n)$
min, max, in order	$O(n)$ — **no order**	$O(1)$, $O(n)$	$O(\log n)$, $O(n)$
range `[a, b)`, next larger	**no**	$O(\log n)$	$O(\log n)$
worst case	$O(n)$	guaranteed	guaranteed

The hash table wins at exactly one thing — "is this key here, and what is its value?" — and that one thing is the most common question in software. It loses everything that depends on **order**.

`lower_bound` (Week 8) can answer "how many values lie in [a, b)?" in $O(\log n)$; a hash table must look at every key, because hashing deliberately destroys order: 2027114 and 2027115 land far apart on purpose. So a database keeps both: a hash index for equality lookups, a tree (a B-tree) for ranges and sorting.

Choose by the questions you will ask. "Have I seen this before?", "count each word", "the value for this ID": hash table. "Everything between these two dates", "the next appointment after 10:00", "the smallest": sorted array or tree.

This Week

Exercises: `dsa/hashmap.py`

Class	Method	The trap
Chaining	`get`	`==` not `is`; `KeyError` only without a default
Chaining	`put`	overwrite, never duplicate; resize above 0.75
Chaining	`delete`	the head of a chain; `size -= 1`
Chaining	`_resize`	**rehash** every key; save `next` first
Open addr.	`_probe`	`% capacity`; every slot once
Open addr.	`put`, `get`	stop at `None`, **not** at a tombstone
Open addr.	`delete`	leave `TOMBSTONE`
Open addr.	`_resize`	drop tombstones; reset both counts

```
pytest tests/test_hashmap.py -v
```

Every method is $O(1)$ on average except the two `_resize`s, which are $O(n)$ and make `put` amortised $O(1)$. Seventeen tests: eight run on each class (`-k Chaining` or `-k OpenAddressing` selects them), and one, `test_open_addressing_delete_preserves_probe_chain`, only for open addressing. `-k` matches case-insensitively, but the underscore matters: `-k OpenAddressing` does **not** select the tombstone test; use `-k "OpenAddressing or open_addressing"` for all nine.

Given to you, and worth reading first: `Entry`, both `__init__`s, `_index`, `load_factor`, `__len__`, `__contains__` and `__iter__`. `__iter__` of the chaining table walks every chain, the loop you will write in `_resize`.

Next week needs this. Week 14's `Graph` stores its adjacency list in **your** `ChainingHashMap` (node → neighbours), and BFS and DFS keep their visited set in one. A hash map that loses a key after a resize breaks graph search too.

Homework 13 — before Lecture 14

1. **Implement** `dsa/hashmap.py` until all 17 tests pass.
2. **Trace** a `ChainingHashMap(capacity=4)` through put 5, 9, 13, 2, then delete 9: the chains after each step, and when it resizes.

3. **Trace** an `OpenAddressingHashMap(capacity=8)` through put 3, 11, 19, delete 11, get 19, put 27. Show every probe.
4. **Break it.** Replace `TOMBSTONE` with `None` in your `delete`. Which test fails, and exactly why?
5. **Measure** probes against load factor in `notebooks/13-hash-tables.ipynb`.

For item 2: `max_load` is 0.75, so the fourth put makes 4/4 = 1.0 and the table doubles to 8 — then every index changes.

For item 5, count probes by subclassing: a class that inherits from `OpenAddressingHashMap`, overrides `_probe`, and adds 1 to a counter for every index it passes on from `super()._probe(key)`. `tools/figures_l13.py` does exactly that, in `figure_probes`.

Summary

Seven things to keep

1. **index = hash(key) % capacity** — computed, not searched.
2. Equal keys **must** hash equal. `str` hashes change **between runs**; `int` hashes do not.
3. Collisions are certain and early: $\sim 1.18\sqrt{m}$ keys (birthday paradox).
4. **Chaining:** a linked list per bucket; cost about $1 + \alpha$.
5. **Linear probing:** next free slot; clustering; delete leaves a **tombstone**.
6. Keep α bounded: **double and rehash every key** — amortised $O(1)$.
7. $O(1)$ is the **average** for well-spread keys; the worst case is $O(n)$, and an adversary can force it.

Next

Week 14 — Graphs. The most general structure in the course — stored in the hash map you just built — and the two searches that walk it: breadth-first and depth-first.

Sources and further reading

- **D. E. Knuth.** *The Art of Computer Programming*, vol. 3, *Sorting and Searching*, 2nd ed., Addison-Wesley, 1998, §6.4 "Hashing" — chaining, open addressing, the analysis of linear probing, and deletion without tombstones (Algorithm R).
- **T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein.** *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, chapter 11, "Hash Tables" — theorems 11.1–11.2 (chaining) and 11.6–11.8 (open addressing).
- **S. A. Crosby and D. S. Wallach.** "Denial of Service via Algorithmic Complexity Attacks", *12th USENIX Security Symposium*, 2003.
- **A. Klink and J. Wälde.** "Efficient Denial of Service Attacks on Web Application Platforms", 28th Chaos Communication Congress (28C3), 2011.
- **C. Heimes.** PEP 456, "Secure and interchangeable hash algorithm", 2013 — SipHash for `str` and `bytes` in Python 3.4.

- **CPython source.** `Objects/dictobject.c` — the long comment at the top explains the compact layout, the perturbation probe and the dummy entries; `Objects/setobject.c` for `set`.

Every figure in this lecture is generated by `tools/figures_l13.py`. The tables drawn are real states of a working `dsa/hashmap.py`, read back from its slots; the three measured figures will differ slightly on your machine.

Questions for Week 13

> ***The answers are at the back of the book,*** *in* Answers — Week 13. *Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify.* `ChainingHashMap` *and* `OpenAddressingHashMap` *are the classes of* `dsa/hashmap.py`*. Both start at* ***capacity 8*** *unless a question says otherwise, and double when they resize. The index of a key is* `hash(key) % capacity`*, and every key in these questions is an* ***integer****, so* `hash(k)` *is* `k`*.*
>
> - `ChainingHashMap` (`max_load` 0.75)*: a new key is pushed on the* ***front*** *of its chain; after the insert, if* `size / capacity > 0.75`*, the table resizes to twice the capacity, re-linking every entry onto the front of its new chain, old buckets in order 0, 1, 2, …*
> - `OpenAddressingHashMap` (`max_load` 0.66)*: linear probing,* `h, h+1, h+2, …` *modulo the capacity;* `delete` *leaves a* ***tombstone*** *(†).* ***Before*** *an insert, if* `(size + tombstones + 1) / capacity > 0.66`*, the table is rebuilt without tombstones — at twice the capacity if* `(size + 1) / capacity > 0.66`*, otherwise at the same capacity. A* `put` *that passes a tombstone remembers the first one, probes on to* `None`*, and inserts at that tombstone.*
>
> *A* ***probe*** *is one slot examined. Slots are written left to right, 0 to 7, with _ for a never-used slot.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W13-M01 – W13-M22
B	Short answer and essay	W13-E1 – W13-E5
C	Trace the code	W13-T1 – W13-T5
D	Table state — draw every step	W13-S1 – W13-S3
E	Complexity analysis	W13-K1 – W13-K3
F	Find and fix the bug	W13-B1 – W13-B4
G	Write the code — checked by `pytest`	W13-C1 – W13-C5

Part A — Multiple choice

W13-M01 [what] A hash table finds the slot of a key by computing:

- **a)** `key // capacity`
- **b)** `hash(key) % capacity`
- **c)** a binary search over the slots
- **d)** `hash(key) * capacity`

W13-M02 [how] In a table of capacity 8, the index of the key 45 is:

- **a)** 5
- **b)** 4
- **c)** 3
- **d)** 45

W13-M03 [what] The one rule every hash function **must** obey is:

- **a)** different keys have different hashes
- **b)** the hash is a prime number
- **c)** equal keys have equal hashes
- **d)** the hash is smaller than the capacity

W13-M04 [why] `python -c "print(hash('abc'))"` prints a different number each time you run it because:

- **a)** it is a bug in Python
- **b)** the hash of a string is its address in memory
- **c)** the capacity of the table changed
- **d)** string hashing uses a random secret key chosen at start-up, on purpose

W13-M05 [how] Which of these is the same in every run of Python?

- **a)** `hash(12345)`
- **b)** `hash("12345")`
- **c)** `hash(b"12345")`
- **d)** the order in which a `set` of strings is printed

W13-M06 [why] `m.put([1, 2], "x")` raises `TypeError: unhashable type: 'list'` because:

- **a)** a list can change after it is stored, and then it would be in the wrong bucket
- **b)** lists are too long to hash
- **c)** `put` only accepts integers
- **d)** the table is full

W13-M07 [how] 23 keys are placed in 365 slots at random. The probability that at least two share a slot is about:

- **a)** 6%
- **b)** 50%
- **c)** 23/365
- **d)** 100%

W13-M08 [what] The load factor of a hash table is:

- **a)** capacity / size
- **b)** the number of collisions so far
- **c)** size / capacity
- **d)** the length of the longest chain

W13-M09 [how] A `ChainingHashMap()` receives `put` for 5, 13 and 21. Bucket 5 is:

- **a)** 5 → 13 → 21
- **b)** 21 → 13 → 5
- **c)** 13 → 5 → 21
- **d)** 5 only; 13 and 21 are in other buckets

W13-M10 [how] In the table of W13-M09, `get(29)` compares 29 with how many entries before raising `KeyError`?

- **a)** 1
- **b)** 0
- **c)** 3
- **d)** 4

W13-M11 [how] A `ChainingHashMap()` receives distinct keys one at a time. The first resize happens during the put of the:

- **a)** 6th key
- **b)** 8th key
- **c)** 4th key
- **d)** 7th key

W13-M12 [why] `_resize` cannot copy `old[i]` into `new[i]` because:

- **a)** an `Array` cannot be copied
- **b)** the index depends on the capacity, so keys would sit where `get` no longer looks
- **c)** the chains would come out reversed
- **d)** the load factor would be wrong

W13-M13 [how] An `OpenAddressingHashMap()` receives `put` for 6, 14 and 22. Key 22 is in slot:

- **a)** 0
- **b)** 6
- **c)** 7
- **d)** it is not stored: slot 6 is taken

W13-M14 [how] In the table of W13-M13, `get(30, None)` makes how many probes?

- **a)** 1
- **b)** 3
- **c)** 5
- **d)** 4

W13-M15 [why] In open addressing, `delete` leaves a tombstone instead of `None` because:

- **a)** a tombstone uses less memory
- **b)** writing `None` is slower
- **c)** `None` would cut the probe sequences that pass through the slot, and keys beyond it would become unreachable
- **d)** an `Array` slot cannot hold `None`

W13-M16 [how] `get` meets a tombstone while probing. It:

- **a)** keeps probing

- **b)** stops: the key is not in the table
- **c)** returns the tombstone
- **d)** raises `KeyError`

W13-M17 [why] `put` counts tombstones in its load check because:

- **a)** tombstones are keys
- **b)** otherwise tombstones can take every `None`, and a probe that stops only at `None` scans the whole table — or never stops
- **c)** `len()` would be wrong otherwise
- **d)** Python requires it

W13-M18 [what] Primary clustering is:

- **a)** keys stored in sorted groups
- **b)** chains growing long in a chaining table
- **c)** runs of occupied slots in linear probing, which grow faster the longer they are
- **d)** several hash tables sharing one `Array`

W13-M19 [how] For linear probing with a good hash, Knuth's formula for an **unsuccessful** search is $\frac{1}{2}(1 + 1/(1-\alpha)^2)$. At $\alpha = 0.5$ it gives:

- **a)** 1.5
- **b)** 2.5
- **c)** 4
- **d)** 50.5

W13-M20 [why] The keys 0, 1024, 2048, 3072, … (n of them) are put into a `ChainingHashMap`, whose capacity is always a power of two. While the capacity is at most 1,024, one `get` costs:

- **a)** $O(1)$ — the table resizes
- **b)** $O(\log n)$
- **c)** $O(n \log n)$
- **d)** $O(n)$ — every key is in bucket 0

W13-M21 [what] Python's `dict` is:

- **a)** open addressing, with a probe sequence that mixes in the high bits of the hash; it grows at about two-thirds full
- **b)** chaining, with a linked list in every bucket
- **c)** a balanced binary search tree
- **d)** a sorted array searched with `bisect`

W13-M22 [why] Which question does a sorted array answer in $O(\log n)$ but a hash table cannot answer faster than $O(n)$?

- **a)** is key k present?
- **b)** what is the value stored for key k?
- **c)** add a new key k
- **d)** how many keys lie between a and b?

Part B — Short answer and essay

W13-E1 [why] *(4 marks)* Explain how a hash table finds a key in $O(1)$ time on average. State the assumption behind "on average", give the worst case, and name three things that can push a real table towards it.

W13-E2 [why] *(3 marks)* A student's class defines `__hash__` to return 0 for every object. Is a hash table of these objects **wrong**, or only **slow**? Answer for chaining and for open addressing, and give the cost of `get` and of building a table of n keys.

W13-E3 [how] *(4 marks)* Compare chaining and open addressing: what a slot holds, the load factors each can tolerate, how each deletes, and one practical advantage of each.

W13-E4 [why] *(4 marks)* Explain tombstones. Why can an open-addressing `delete` not simply empty the slot? State how `get`, `put` and `_resize` each treat a tombstone, and why the load check must count them.

W13-E5 [why] *(3 marks)* Show that a hash table that doubles its capacity when the load factor passes a fixed limit does amortised $O(1)$ work per `put` on rehashing. What happens if it grows by a fixed 8 slots instead?

Part C — Trace the code

W13-T1 [how] Give the index of each key in a table of capacity 8 and in a table of capacity 16: 7, 15, 23, -9, 40, 100. Which keys share a slot at capacity 8 but not at capacity 16?

W13-T2 [how] A `ChainingHashMap(capacity=4)` receives `put(5, 5)`, `put(9, 9)`, `put(13, 13)`, `put(2, 2)` and then `delete(9)`. Show the chains, the size and the load factor after each operation, and say where it resizes.

W13-T3 [how] An `OpenAddressingHashMap()` receives `put` for 3, 11 and 19, then `delete(11)`, `get(19)` and `put(27, 27)`. For each operation list the slots probed, and give the slots at the end.

W13-T4 [how] What does this print?

```
def sum_hash(s):
    return sum(ord(c) for c in s)

def poly_hash(s):
    h = 0
    for c in s:
        h = 31 * h + ord(c)
    return h

for w in ["cat", "act", "tac", "dog", "god"]:
    print(w, sum_hash(w) % 8, poly_hash(w) % 8)
```

(`ord("a")` is 97, and the letters follow in order.) Which pairs collide under each hash, and why is that fair to one of them and not to the other?

W13-T5 [why] An `OpenAddressingHashMap()` receives `put` for 0, 8, 16, 24. Then: `get(24)`, `get(32, None)`, `delete(8)`, `get(24)`, `put(40, 40)`, `get(32, None)`. Give the probes of each operation and the final slots. What do these six operations show about clustering?

Part D — Table state: draw every step

W13-S1 [how] Draw a `ChainingHashMap(capacity=4)` — every bucket, every chain, the size and the capacity — after each of:

`put 10` · `put 3` · `put 14` · `put 7` · `put 18` · `delete 10` · `put 26` · `delete 26`

W13-S2 [how] Draw the 8 slots of an `OpenAddressingHashMap()`, with the size and the tombstone count, after each of:

`put 5` · `put 13` · `put 21` · `put 6` · `delete 13` · `put 29` · `delete 5` · `put 14`

For each `put`, list the slots probed.

W13-S3 [why] Draw the slots of an `OpenAddressingHashMap()` after each of:

`put 1` · `put 2` · `put 3` · `put 4` · `put 5` · `delete 1` · `delete 2` · `put 9` · `put 10` · `put 11`

State the load check of each of the last three puts, and whether it causes a sweep at the same capacity, a doubling, or nothing.

Part E — Complexity analysis

W13-K1 [how] a and b are lists of n and m integers. Give Θ on average and in the worst case, and justify:

```
seen = ChainingHashMap()
for x in a:
    seen.put(x, True)
common = 0
for y in b:
    if y in seen:
        common += 1
```

What is it if `seen` is a Python `list` and `in` is the list's `in`?

W13-K2 [why] n distinct keys are put into a chaining table that **never** resizes, capacity 8. Give the total cost of the n puts. Then give it for a table that doubles at load 0.75, and for one that adds 8 buckets at load 0.75.

W13-K3 [why] Count how many times each word occurs in a text of n words, three ways: (a) for each word, count it by scanning the whole text; (b) sort the words, then count the runs; (c) a hash

map from word to count. Give the time of each, the worst case of (c), and which you would use.

Part F — Find and fix the bug

W13-B1 [why] A student's `ChainingHashMap.__contains__`:

```
def __contains__(self, key):
    return self.get(key, None) is not None
```

W13-B2 [how] Part of a `ChainingHashMap._resize`:

```
old = self._buckets
self._buckets = Array(capacity)
for head in old:
    entry = head
    while entry is not None:
        i = self._index(entry.key)
        entry.next = self._buckets[i]
        self._buckets[i] = entry
        entry = entry.next
```

W13-B3 [why] Part of an `OpenAddressingHashMap.put`, after the load check:

```
for i in self._probe(key):
    slot = self._keys[i]
    if slot is None or slot is TOMBSTONE:
        self._keys[i] = key
        self._values[i] = value
        self._size += 1
        return
    if slot == key:
        self._values[i] = value
        return
```

W13-B4 [how] A `ChainingHashMap.delete`:

```
def delete(self, key):
    i = self._index(key)
    entry = self._buckets[i]
    while entry is not None:
        if entry.key == key:
            self._buckets[i] = entry.next
            self._size -= 1
            return
        entry = entry.next
    raise KeyError(key)
```

Part G — Write the code

In `practice/week13.py`; check with `pytest tests/test_practice_week13.py -v`. Each problem is $O(n)$ with a hash map, and each has a test large enough that an $O(n^2)$ answer is painfully slow. Use your `ChainingHashMap` as the working storage — not a Python `dict` or `set`.

W13-C1 [how] `two_sum(values, target)` — two indices whose values add up to the target, in one pass.

W13-C2 [how] `first_repeated(items)` — the first item to appear a second time.

W13-C3 [how] `group_anagrams(words)` — group words that are anagrams of one another, keeping the input order.

W13-C4 [why] `longest_distinct_run(text)` — the length of the longest stretch with no repeated character. Why is it $O(n)$ although the window moves back and forth?

W13-C5 [why] `count_subarrays_with_sum(values, k)` — how many contiguous runs add up to k, with negative values allowed. Why does a sliding window not work here, and why do prefix sums?

Week 14 — Graphs and Graph Searches

> ***How to read this document.*** *This is the handout for Lecture 14. It holds everything on the slides, plus what I said out loud. A graph is the most general structure in the course: every structure you have built so far is a special case of it. This week you store one in two ways and measure the difference, then walk it in the two orders every other graph algorithm is built from — breadth first, with your queue, and depth first, with your stack.*
>
> *Slides: `DSA27-L14-slides.pdf` · Code: `dsa/graph.py` · Tests: `tests/test_graph.py`*

Where We Are

The last structure

Every structure so far was a graph with rules:

- a **linked list**: each node points to at most one next node;
- a **tree**: one root, one parent per node, no cycles;
- a **graph**: nodes and edges, **no rules at all**.

Today: store one, then **search** it — breadth first and depth first.

A linked list is a graph in which every node has one outgoing edge (or none). A tree (Week 11) is a graph in which every node except the root has exactly one parent and no path loops back. Drop those rules and you have a **graph**: any set of things, and any set of connections between them. That freedom is why graphs model so much — roads between cities, friendships, links between web pages, wires on a chip, dependencies between software packages, and the prerequisites between the courses of your own degree.

It is also why this week comes last among the structures: a graph needs everything before it. `dsa/graph.py` stores its edges in your `DynamicArray` (Week 4) inside your `ChainingHashMap` (Week 13); breadth-first search runs on your `CircularQueue` (Week 7); depth-first search runs on the call stack (Week 3) or on your own `Stack` (Week 6). If one of those is broken, this week finds it.

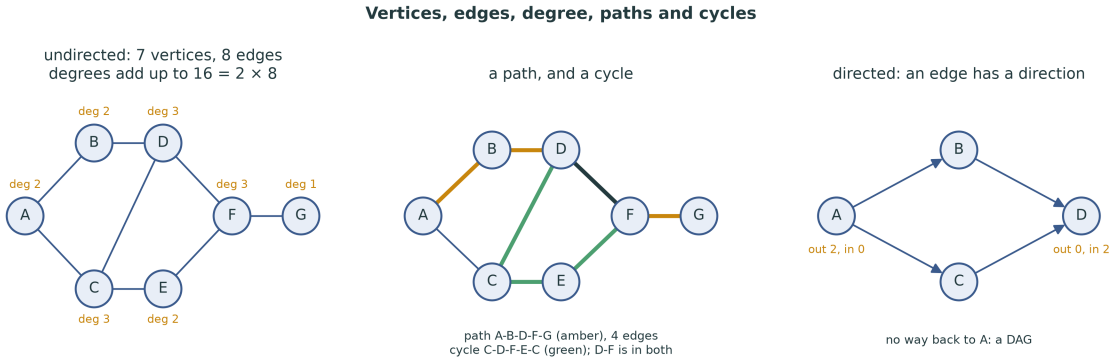
Graph in Arabic: رسم المخطط, also رسم البياني. *Graph search*: البحث في المخطط.

Today

1. The words: vertex, edge, degree, path, cycle, component, DAG
2. Two representations: adjacency list and adjacency matrix — measured
3. Breadth-first search, with a queue — and shortest paths
4. Depth-first search, recursive and with your own stack
5. Components, cycles, topological sort
6. A maze, and your own degree's prerequisites

The Vocabulary

Vertices and edges



- **Vertex** (node) — الرأس. **Edge** — الحافة. V vertices, E edges.
- **Undirected**: an edge goes both ways. **Directed**: it has a direction.
- **Degree** — الدرجة: the number of edges at a vertex.

A graph is a pair: a set of **vertices** (also called **nodes**; the course code says *node*) and a set of **edges**, each joining two vertices. The two ends of an edge are **adjacent**, متجاوران, and each is a **neighbour** of the other. We write V for the number of vertices and E for the number of edges; all costs this week are stated in V and E.

Directed or undirected, موجّهة and غير موجّهة. In an undirected graph an edge A–B can be walked both ways: a friendship, a two-way road. In a directed graph an edge A → B goes one way only: "follows" on social media, a one-way street, "is a prerequisite of". `Graph(directed=True)` records only A → B; `Graph()` records the edge in both directions.

Degree. In an undirected graph, the degree of a vertex is the number of edges touching it. In the left panel, D has degree 3. Every edge has two ends, so adding up all the degrees counts every edge twice:

$$\sum_{v} \deg(v) = 2E$$

— here $2+2+3+3+2+3+1 = 16 = 2\times 8$. This is sometimes called the *handshake lemma*: at a party, the handshakes counted per person add up to twice the number of handshakes. In a directed graph each vertex has an **out-degree** (edges leaving) and an **in-degree** (edges arriving); in the right panel A has out-degree 2 and in-degree 0. `degree(node)` in `dsa/graph.py` returns the number of edges **leaving** the node, which is the out-degree for a directed graph and the plain degree for an undirected one.

Sparse and dense. A simple undirected graph has at most $V(V-1)/2$ edges, so E is somewhere between 0 and about $V^2/2$. A graph with E close to V (a road map, where a junction joins three or four roads) is **sparse**; one with E close to V^2 is **dense**. Most real graphs are sparse — a social network of a billion people is not a billion friends each. This single fact decides the representation, as the measured slide will show.

Paths, cycles and components

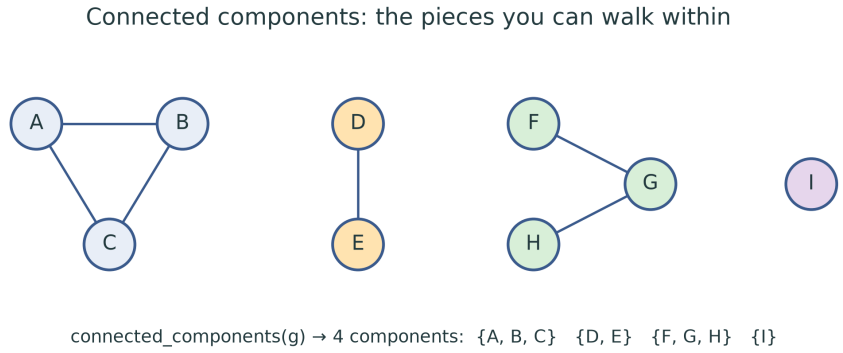
- **Path** — المسار: a sequence of vertices, each joined to the next by an edge. Its **length** is its number of edges.
- **Cycle** — الدور: a path that returns to where it started.
- **Connected** — every vertex can reach every other. A **connected component**, المركّبة المتصل, is a largest connected piece.
- **DAG**, مخطط موجَّه لا دوري: a directed graph with **no** cycle.

In the middle panel of the figure, A–B–D–F–G is a path of length 4, and C–D–F–E–C is a cycle of length 4. A path is usually required to be **simple** — no vertex twice — and a cycle to have at least three vertices in an undirected graph: going A–B–A along the same edge does not count. That detail is exactly the trap of undirected cycle detection later.

A **tree** is a connected undirected graph with no cycle. A tree with V vertices always has exactly $V-1$ edges: one fewer and it falls apart, one more and it has a cycle. (Week 11's trees are rooted trees, which pick one vertex as the root and point every edge away from it.)

A **DAG** — a *directed acyclic graph* — is the shape of every dependency problem: prerequisites, build steps, spreadsheet cells, package installs. The diamond on the right of the figure is a DAG: you can reach D in two ways, but you can never get back to A. DAGs are the graphs that have a **topological order**, which ends the lecture.

Connected components



One search from a vertex visits **exactly its component**. To find all components: start a new search from every vertex not yet seen.

This graph has nine vertices, six edges and four components; I, with no edges at all, is a component on its own. The number of components answers practical questions: is this road network in one piece? after this cable is cut, can every office still reach the server? how many separate friend groups are there? The algorithm is on a later slide, once we have a search to run.

"Connected" is defined here for undirected graphs, and `connected_components` expects one. In a directed graph, "A can reach B" does not mean "B can reach A", and the idea splits into *weakly* and *strongly* connected components; the strongly connected ones need a cleverer algorithm (Tarjan's, 1972), beyond this course.

Graphs are everywhere

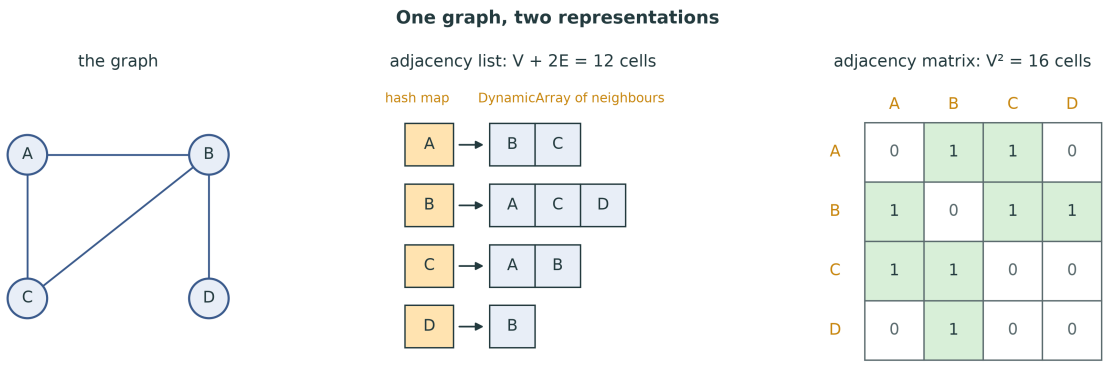
Problem	Vertices	Edges	Kind
Maze, game map	open cells	neighbouring cells	undirected
Road map	junctions	roads	(un)directed
Web	pages	links	directed
Your degree	courses	“is a prerequisite of”	DAG
Build, installs	files, packages	“is needed by”	DAG

The same two searches answer questions on all of them: is there a way through the maze, and what is the shortest one? which pages can be reached from this one? in what order can I take these courses, and is that even possible? The lecture ends with two of these rows worked in full — the maze and the prerequisites of the Software Engineering programme.

Graph theory began with a walk. In 1736 Leonhard Euler asked whether a walk through Königsberg could cross each of its seven bridges exactly once. He answered it by drawing the city as four land masses (vertices) joined by bridges (edges) and throwing away everything else: the first time a problem was solved by recognising that only the connections matter.

Two Representations

Adjacency list and adjacency matrix



- **List**: for each vertex, the vertices it reaches.
- **Matrix**: a V × V grid; cell [i][j] is 1 when there is an edge i → j.

Both panels describe the test file’s sample graph: A–B, A–C, B–C, B–D.

The adjacency list, قائمة التجاو�, keeps, for every vertex, a list of its neighbours. In `dsa/graph.py`, `Graph._adjacent` is a `ChainingHashMap` from each node to a `DynamicArray` of its neighbours, in the order the edges were added. The hash map is what lets the nodes be anything hashable — strings, numbers, `(row, column)` tuples for a maze — and still be found in $O(1)$ on average. An undirected edge is stored **twice**, once in each end’s list, so the list holds $V + 2E$ cells: 4 + 2 × 4 = 12 here.

A hash map hands its keys back in bucket order, which for strings changes from one run of Python to the next (string hashing is randomised for security). So `Graph` also keeps `_order`, a `DynamicArray` of the nodes in the order they were added, and `nodes()` returns that. Without it, any algorithm that loops over all the nodes — components, cycle detection, topological sort — would give a different, equally correct answer on every run, and no trace in this handout could be checked.

The adjacency matrix, مصفوفة التجاو, numbers the vertices 0 to V − 1 and keeps a V × V grid of booleans. In `MatrixGraph`, `_index` maps a node to its number, `_order` maps a number back to its node, and `_matrix` is a `DynamicArray` of rows, each a `DynamicArray` of `True`/`False`. An undirected graph gives a **symmetric** matrix: cell [A][B] and cell [B][A] are both set. It always holds V^2 cells, however few edges there are: 16 here.

What `add_node` must not do. Adding a node that is already there must leave its edges alone. The careless version, `self._adjacent.put(node, DynamicArray())` with no check, replaces the node's neighbour list with an empty one and silently drops its edges — and since `add_edge` calls `add_node` on both ends, every new edge would wipe the old edges of its endpoints. `test_adding_a_node_twice_keeps_its_edges` is there for that bug.

The trade-off

	Adjacency list	Adjacency matrix
Memory	$O(V + E)$	$O(V^2)$
`has_edge(u, v)`	$O(\deg u)$	$O(1)$
`neighbours(u)`	$O(\deg u)$	$O(V)$ — scan the row
`add_node`	$O(1)$	$O(V)$ — a new column
One BFS or DFS	$O(V + E)$	$O(V^2)$

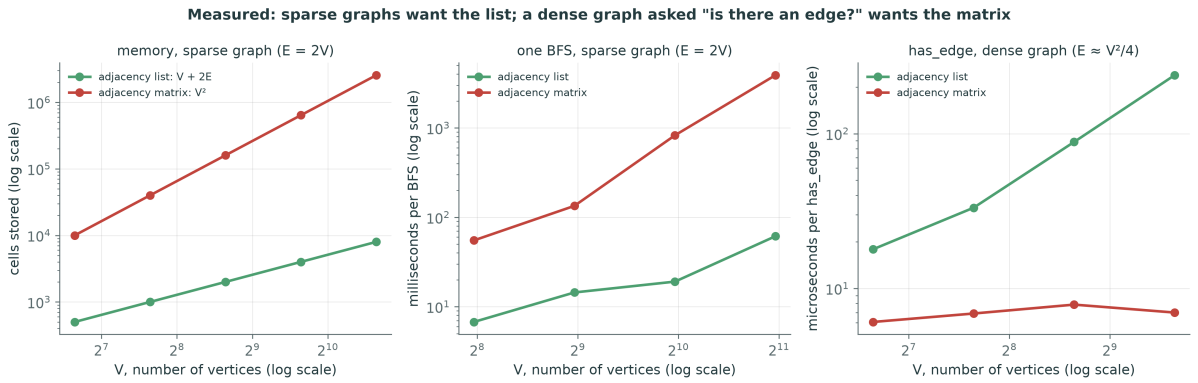
Why a search costs $O(V + E)$ on the list. A search visits each vertex once and, at each vertex, looks at its neighbours once. The total work on the neighbour lists is the sum of all the degrees, which is $2E$ (undirected) or E (directed). Add $O(1)$ per vertex for taking it off the queue or stack, and the total is $O(V + E)$ — proportional to the size of the graph, which is the best any algorithm that must look at the whole graph can do.

Why it costs $O(V^2)$ on the matrix. The search does the same thing, but finding the neighbours of a vertex means scanning its whole row of V cells, whether they are 0 or 1. V vertices times V cells is V^2, even for a graph with almost no edges.

The matrix's prize is `has_edge`: one cell, $O(1)$. On the list, asking "is there an edge from u to v?" means scanning u's neighbours, $O(\deg u)$ — up to $O(V)$ in a dense graph.

The rule of thumb. Sparse graph, or you walk it: the list. Dense graph, and you mostly ask "is there an edge?": the matrix. Because most real graphs are sparse, the adjacency list is the default almost everywhere; `networkx` uses one (a dictionary of dictionaries).

Measured



Real measurements of the reference `dsa/graph.py`, on random graphs built with a fixed seed. Both classes pass the same 63 tests; they differ only in cost.

Left: memory, counted as the cells each structure actually stores, on a sparse graph with E = 2V (average degree 4). At V = 1,600 the list stores 8,000 cells and the matrix 2,560,000 — 320 times more. The list's line has slope 1 on the log–log plot, the matrix's slope 2.

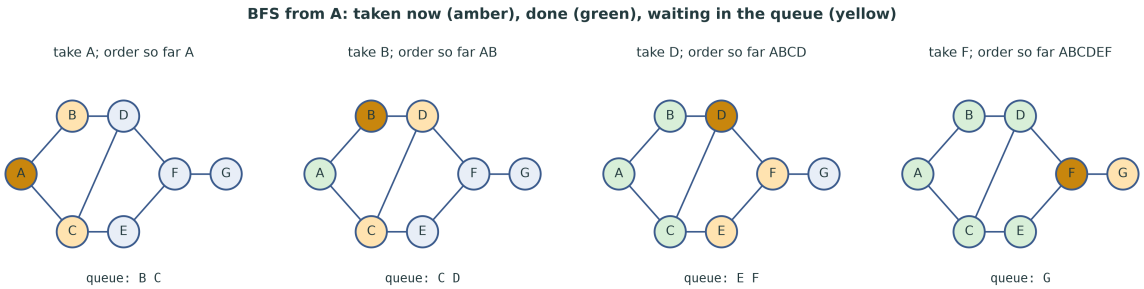
Middle: one BFS on the same kind of sparse graph. The matrix line has slope 2: each doubling of V made the BFS between 2.4 and 6 times slower — about four on average, as $O(V^2)$ predicts — and at V = 2,000 it took about 3.9 seconds, against about 60 milliseconds on the list. The list's line is close to slope 1, with visible noise — these runs shared the machine with other work, and at these small times that shows. The shapes are the point: $O(V^2)$ against $O(V + E)$.

Right: has_edge on a dense graph, every vertex joined to about half of the others. The matrix answers in about 6–8 microseconds at every size — flat, $O(1)$. The list's time grows with the degree, which grows with V: about 240 microseconds at V = 800, some 30 times slower. On a dense graph that is asked this question constantly, the matrix is worth its memory.

Your own numbers will differ; the slopes will not. Draw your own with the `14-graphs` notebook once `dsa/graph.py` works.

Breadth-First Search

Rings round the start



Visit the start, then **everything 1 edge away**, then everything 2 away, …

A **queue** keeps the frontier in order: first discovered, first expanded.

Breadth-first search, BFS — البحث بالعرض أولا — explores a graph like a ripple on water: the start first, then all its neighbours, then all *their* unvisited neighbours, and so on outwards. Lecture 07 promised that a queue would do this, and here it is. A vertex waits in the queue from the moment it is **discovered** until it is **taken** off the front; because the queue is FIFO, everything discovered from layer 1 is taken before anything discovered from layer 2.

In the figure, the running example — seven vertices, eight edges — is searched from A. After A is taken, B and C wait. B is taken and adds D; C is taken and adds E (its neighbour D is already waiting); D adds F; E adds nothing (F is already waiting); F adds G. The visit order is A B C D E F G.

The algorithm

```
bfs(graph, start)
    order   = empty list
    visited = empty hash map;  visited[start] = True
    queue   = empty CircularQueue;  enqueue start
    while the queue is not empty:
        node = dequeue
        append node to order
        for each neighbour of node, in graph.neighbours(node) order:
            if neighbour is not in visited:
                visited[neighbour] = True      # mark when queued
                enqueue neighbour
    return order
```

Each vertex is enqueued once and dequeued once: $O(V + E)$ on a list.

This is the structure of `bfs` in the reference solution. Three details decide whether yours is right.

Mark a vertex when you enqueue it, not when you dequeue it. If you only mark on dequeue, a vertex can be discovered twice before it is taken — in the example, D is a neighbour of both B and C — and it goes into the queue twice. The visit order then has D twice unless you also check on dequeue, and on a dense graph the queue can hold $O(E)$ entries instead of $O(V)$. Marking on enqueue means "already in the queue or already done", which is exactly the set you must not add again.

The visited set is your hash map, `ChainingHashMap`, with `True` as the value: `neighbour in visited` is an average $O(1)$ lookup. A Python `set` would do the same job, and the storage rule forbids it inside `dsa/`. Without any visited set at all, the search goes round the cycle A–B–C–A for ever: `test_bfs_visits_each_node_once` checks for that.

The queue is your `CircularQueue`, and it must grow: BFS does not know in advance how many vertices will wait at once. That is why Lecture 07 made growing the challenge. A `SlowQueue` would also work, but every dequeue would shift the whole queue: $O(V)$ per dequeue, $O(V^2)$ for the search.

`bfs` returns only the vertices **reachable** from the start. On a graph with several components, the

others are simply never discovered — `test_search_from_isolated_node` starts from a vertex with no edges and expects just `["Z"]`.

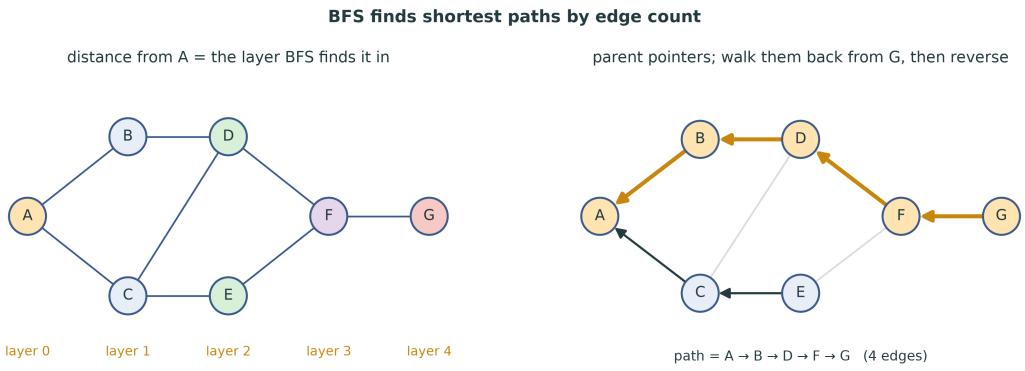
Trace: `bfs(G, "A")`

Take	Discover (enqueue)	Queue afterwards	Order so far
A	B, C	B C	A
B	D (A is visited)	C D	A B
C	E (A, D are visited)	D E	A B C
D	F	E F	A B C D
E	— (C, F are visited)	F	A B C D E
F	G	G	A B C D E F
G	—	empty	A B C D E F G

The neighbour lists, in the order the edges were added, are A: B C · B: A D · C: A D E · D: B C F · E: C F · F: D E G · G: F. The neighbour order decides the tie-breaks: A lists B before C, so B is taken first. Change the order of the edges and you get a different — equally correct — BFS order, but the **layers** never change: {A}, {B, C}, {D, E}, {F}, {G}. `test_bfs_visits_by_distance` checks only what does not depend on the tie-breaks: D, two edges from A, comes after B.

This table was produced by running the reference solution on `G_EDGES` in `tools/figures_l14.py`.

Shortest paths, by edge count



The first time BFS reaches a vertex, it has come by a **shortest** path.

Remember **who discovered whom** (a parent map), then walk back from the goal.

Why the first arrival is by a shortest path. The queue holds vertices in non-decreasing order of distance: first the start (distance 0), then everything at distance 1, then everything at distance 2, and so on, because each vertex at distance d discovers only vertices at distance d + 1. So a vertex at distance d is discovered while BFS is taking the vertices of layer d — 1, before any longer route can reach it. "Shortest" here means **fewest edges** — every edge counts as 1. (This argument was first written down for mazes by E. F. Moore in 1959.)

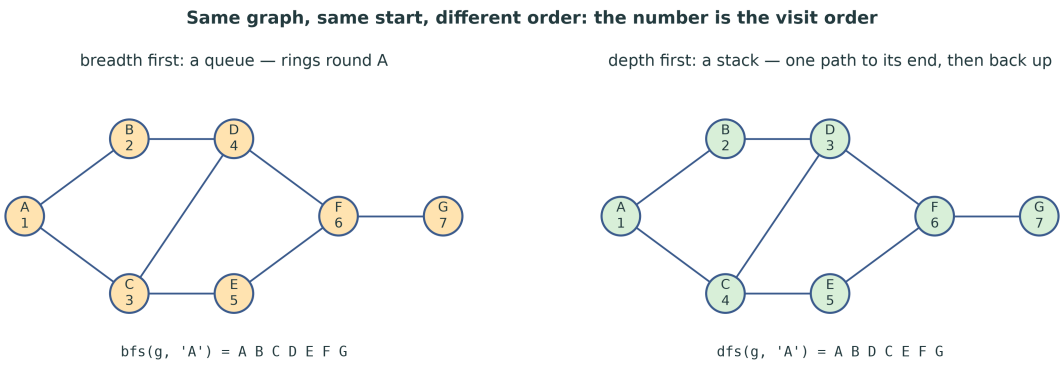
`shortest_path_unweighted(graph, start, goal)` is BFS with one change: the visited map stores, for each vertex, the vertex it was discovered **from** — its *parent* — with `None` for the start. When the goal is taken off the queue, follow the parents back: G ← F ← D ← B ← A. That list runs goal-first, so reverse it: `["A", "B", "D", "F", "G"]`, four edges. When the queue empties without reaching the goal, it is unreachable: return `None`. When start and goal are the same, the answer is `[start]` — a path of length 0.

The parent pointers form a tree — the **BFS tree** — whose paths from A are all shortest paths. There are two shortest paths from A to F here (A–B–D–F and A–C–E–F); BFS records one of them, the one through whichever parent discovered F first. `test_shortest_path_is_shortest_not_merely_a_pa` builds a graph with a long way round (A–X–Y–Z) and a short one (A–B–Z): DFS might find the long one; BFS cannot.

The parent map is a `ChainingHashMap` again, and it doubles as the visited set: a vertex is visited exactly when it has a parent entry. That needs a hash map whose `in` works when the stored value is `None` — the start's parent. Week 13's `__contains__` catches the `KeyError` from `get` rather than comparing the value with `None`, so it does.

Depth-First Search

One path to its end, then back up



DFS goes **deep** first: follow a neighbour, then its neighbour, … until stuck; then **back up** to the last vertex with an unvisited neighbour.

Depth-first search, DFS — البحث بالعمق أولا — is how you would explore a maze with a ball of string: keep walking into unexplored corridors, and when you reach a dead end, wind the string back to the last junction with a corridor you have not tried. From A it goes to B (A's first neighbour), then to D (B's first unvisited neighbour), then to C (D's first unvisited neighbour), then to E, F and G. The order is A B D C E F G, against BFS's A B C D E F G.

`test_dfs_goes_deep_and_bfs_goes_wide` tells them apart on the smallest graph that can: A–B, B–C, A–D. DFS goes A, B, C before it touches D; BFS takes D before C.

DFS does **not** find shortest paths: it reached C through B and D, three edges, when C is a neighbour of A. What it gives instead is structure — the order in which vertices are finished, which detects cycles and gives topological orders.

Recursive DFS: the call stack does the work

```
dfs(graph, start)
    order = empty list;  visited = empty hash map
    visit(start)
    return order

visit(node)
    visited[node] = True
    append node to order
    for each neighbour of node, in graph.neighbours(node) order:
        if neighbour is not in visited:
            visit(neighbour)
```

$O(V + E)$ on a list — and $O(V)$ **stack** in the worst case.

This is `dfs` and its helper `_dfs_visit` in the reference solution. Each call is one vertex; the loop inside it is that vertex's neighbours; returning from a call is "backing up". Everything DFS has to remember — which vertex it is in, and how far through that vertex's neighbours it has got — lives in the call stack of Lecture 03, one frame per vertex on the current path.

That is also its weakness. The deepest path can be as long as V − 1 edges: on a graph that is one long chain, recursive DFS makes V nested calls. Python stops at 1,000 frames by default: the reference `dfs` on a chain of 1,000 edges raises `RecursionError`, while on a chain of 900 it works. Raising the limit with `sys.setrecursionlimit` only moves the wall. The real answer is the next slide.

DFS with your own stack

```
dfs_iterative(graph, start)
    order = empty list;  visited = empty hash map
    stack = empty Stack;  push start
    while the stack is not empty:
        node = pop
        if node is in visited:  continue    # a second copy
        visited[node] = True                # mark when taken
        append node to order
        for each neighbour, in REVERSE order:
            if it is not in visited:  push it
    return order
```

Same order as `dfs`, and no recursion limit.

Replace BFS's queue by a stack and the search becomes depth-first: the most recently discovered vertex is expanded next. Two details make the order match the recursive `dfs` exactly, which `test_dfs_and_dfs_iterative_agree` demands.

Push the neighbours in reverse. A stack gives back the last thing pushed. To explore B before

C (A's list is B, C), push C first and B last.

Mark when you pop, not when you push. The recursive version marks a vertex when it *enters* it, and it can reach a vertex by a later, deeper route before the earlier call gets round to it. The stack must allow the same: a vertex may be pushed more than once, and the first time it is popped wins; later copies are skipped. So the stack can hold up to $O(E)$ entries, not $O(V)$.

Trace on G, stack written bottom → top:

Pop	Action	Push	Stack afterwards	Order
A	visit	C, B	C B	A
B	visit	D	C D	A B
D	visit	F, C	C F C	A B D
C	visit	E	C F E	A B D C
E	visit	F	C F F	A B D C E
F	visit	G	C F G	A B D C E F
G	visit	—	C F	A B D C E F G
F	already visited: skip		C	
C	already visited: skip		empty	

C was pushed twice — by A and by D — and D's copy, the deeper route, was taken first, exactly as recursion would. This trace was produced by running the reference solution.

Why the two orders can differ

"Mark when pushed", BFS-style, with a stack:

Variant	Order from A
recursive `dfs`	A B D C E F G
stack, mark on pop, push reversed	A B D C E F G
stack, mark on **push**, push reversed	A B D **F E G C**
stack, mark on push, push in order	A C E F G D B

The last two rows are the tempting shortcut: copy BFS, swap the queue for a stack. They are real depth-first searches — each follows a path as far as it can before backing up — and they visit every reachable vertex once in $O(V + E)$. For reachability, or for counting components, they are fine.

But they are **not** the same order as recursive DFS. Marking C when A pushes it means that when D later looks at C, C is "already visited" and is not pushed again; so C waits at the bottom of the stack under A's copy and is visited last, instead of being entered from D. The recursive DFS would have gone A → B → D → C. All four rows were produced by running code on the same graph.

The order matters when an algorithm depends on it — cycle detection and topological sort use exactly the recursive order, in which a vertex is "on the path" from the moment it is entered until

every vertex below it is finished. So: mark on pop, and push neighbours in reverse, when you want the recursive order. `test_dfs_and_dfs_iterative_agree` fails for the shortcut.

Why not always write the iterative version? The recursive one is shorter and reads like its definition; the iterative one survives a graph deeper than Python's recursion limit. In languages with larger stacks, recursive DFS is the common choice; in Python, on big graphs, write your own stack.

BFS and DFS side by side

	BFS	DFS
Container	**queue** (FIFO)	**stack** (LIFO) — or the call stack
Order	rings, by distance	one path to its end, then back up
Shortest path by edges	**yes**	no
Time, adjacency list	$O(V + E)$	$O(V + E)$
Time, adjacency matrix	$O(V^2)$	$O(V^2)$
Extra memory	$O(V)$	$O(V)$ recursive; up to $O(E)$ own stack
Good for	distances, "nearest", levels	cycles, topological order, mazes, backtracking

The two searches are one loop with one choice: which waiting vertex to expand next. The oldest gives BFS; the newest gives DFS. (Let a *priority queue* — Week 12 — choose the nearest by weighted distance, and you have Dijkstra's algorithm; see the last section.)

Both need a visited set, and both work on either representation unchanged, because they only ever call `graph.neighbours(node)`. That is the adjacency list and matrix's shared contract paying off: the tests run every search against both classes.

Using the Searches

Connected components

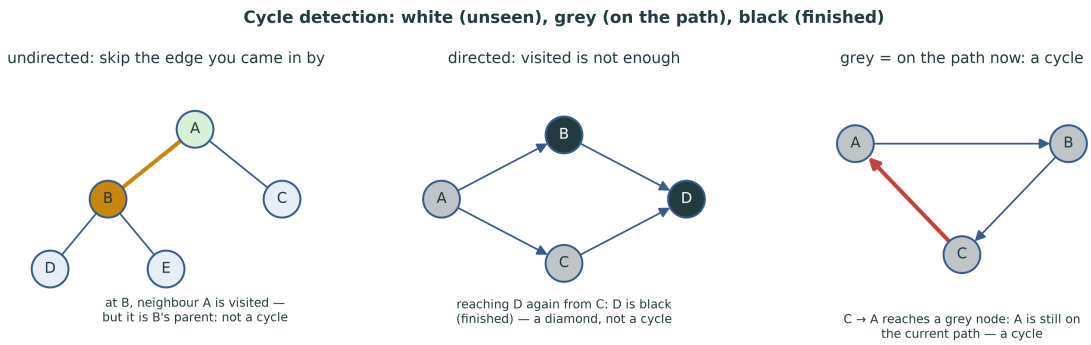
```
connected_components(graph)
    seen = empty hash map;  components = empty list
    for each node in graph.nodes():
        if node is not in seen:
            component = bfs(graph, node)
            mark every member of component as seen
            append component to components
    return components
```

$O(V + E)$ in total — each vertex is searched from once.

Each call of `bfs` visits exactly the component of its start (it can never cross to another component — there is no edge to cross), and the loop starts a new search only from a vertex that no earlier search reached. So every vertex is in exactly one component, and the total work is still $O(V+E)$: every vertex and every edge belongs to exactly one of the searches.

On the components figure, the loop starts searches at A, D, F and I: `[['A', 'B', 'C'], ['D', 'E'], ['F', 'G', 'H'], ['I']]`. Any search would do in place of `bfs` — `dfs` gives the same groups, listed in a different order.

Cycles: undirected and directed are different



- **Undirected:** a visited neighbour that is **not your parent** closes a cycle.
- **Directed:** a neighbour that is **grey** — still on the current path.

Undirected: the parent check. Run DFS, remembering for each vertex the vertex it was entered from. When a vertex has a neighbour that is already visited, you have found a second route to it — a cycle — **unless** that neighbour is simply the vertex you came from. In an undirected graph every edge is stored in both directions, so when DFS enters B from A, B's neighbour list contains A, and A is visited. That is not a cycle; it is the same edge seen from the other end. Forget the parent check and **every** undirected graph with an edge looks cyclic: `test_undirected_tree_has_no_cycle` is that trap.

```
cycle_from(node, parent)                # undirected
    visited[node] = True
    for each neighbour of node:
        if neighbour is not visited:
            if cycle_from(neighbour, node):  return True
        else if neighbour != parent:  return True
    return False
```

Directed: three colours. In a directed graph "already visited" is not enough. In the diamond A → B → D and A → C → D, DFS reaches D from B, finishes it, backs up to A, goes to C, and meets D again. D is visited — but there is no cycle: no path leads from D back to C. What matters is whether the vertex is **on the current path**, the chain of calls still running. So each vertex has one of three colours:

- **white** — not yet seen (in the code: not in the map);
- **grey** — entered, and its call is still running: on the current path;

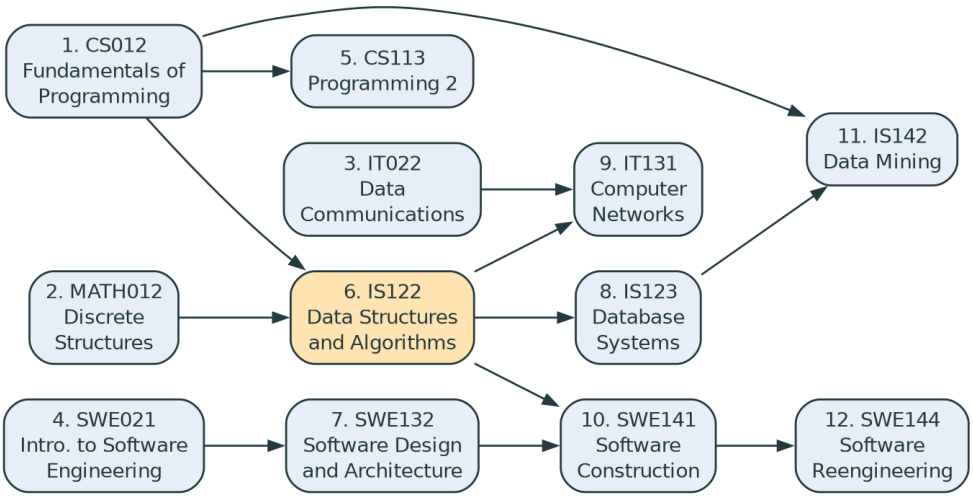
- **black** — finished: every vertex below it is done.

An edge to a **grey** vertex points back into the current path: a cycle. An edge to a **black** vertex is only a second route to finished work, like C → D in the diamond (`test_directed_diamond_is_not_a_cycle`). In A → B → C → A, when DFS at C looks at A, A is grey — its call is still waiting for B's, which is waiting for C's — so C → A closes a cycle.

```
cycle_from(node)                          # directed
    colour[node] = GREY
    for each neighbour of node:
        if colour[neighbour] is GREY:  return True
        if colour[neighbour] is WHITE and cycle_from(neighbour):  return
↪   True
    colour[node] = BLACK
    return False
```

In both cases `has_cycle` runs the helper from every vertex not yet visited (white), because a cycle can sit in any component. That is $O(V + E)$ in total. The undirected parent check is wrong for directed graphs (A → B, B → A **is** a cycle, and the parent check would skip it), and the three colours would be wrong for undirected ones (every edge would be seen back into a grey parent). The docstring's warning — "the two cases genuinely differ" — is the point.

Topological sort



Software Engineering bylaw (2013): an arrow X → Y means X is a prerequisite of Y.
The number is the position in topological_sort(g).

An order of the vertices in which **every edge points forwards**. It exists **exactly when the graph is a DAG**.

A topological order, الترتيب الطوبولوجي, of a directed graph lists every vertex so that for every edge X → Y, X comes before Y. For prerequisites, it is an order in which you could take every course without ever taking one before its prerequisites. For a build, it is an order that compiles every file after the files it depends on.

If the graph has a cycle, no such order exists: in A → B → C → A, A must come before B,

B before C, and C before A — impossible. If it has no cycle, one always exists, and usually many: MATH012 and CS012 can come in either order. The tests accept any valid order; `test_topological_sort_respects_every_edge` checks every edge.

The figure is real: twelve courses from the specifications in the **Software Engineering bylaw (2013)**, with an arrow for each prerequisite it lists — for example, IS122 (this course) needs CS012 and MATH012, and is itself needed by IS123 Database Systems, IT131 Computer Networks and SWE141 Software Construction. It is only part of the programme; the numbers are the order the reference `topological_sort` returned.

Kahn's algorithm: take what nothing waits on

1. Count each vertex's **in-degree** — its unmet prerequisites.
2. Queue every vertex with in-degree 0.
3. Repeat: dequeue X, output it; for each X $\rightarrow$ Y, subtract 1 from Y's count; if it reaches 0, enqueue Y.
4. Output shorter than V? The rest wait on each other: **a cycle** — return `None`.

$O(V + E)$: each vertex queued once, each edge crossed once.

This is `topological_sort` in the reference solution (A. B. Kahn, 1962). The in-degrees live in a `ChainingHashMap` from vertex to count, the ready vertices in a `CircularQueue`. It is how you would plan a degree by hand: take the courses with no prerequisites; crossing them off frees other courses; take those next.

Trace on the prerequisites graph (vertices in the order they were added: CS012, CS113, IS122, MATH012, IS123, IS142, IT022, IT131, SWE021, SWE132, SWE141, SWE144):

Take	Counts that reach 0	Ready queue afterwards
(start)	CS012, MATH012, IT022, SWE021 have none	CS012 MATH012 IT022 SWE021
CS012	CS113 (IS122 and IS142 still wait)	MATH012 IT022 SWE021 CS113
MATH012	IS122	IT022 SWE021 CS113 IS122
IT022	— (IT131 still waits on IS122)	SWE021 CS113 IS122
SWE021	SWE132	CS113 IS122 SWE132
CS113	—	IS122 SWE132
IS122	IS123, IT131 (SWE141 waits on SWE132)	SWE132 IS123 IT131
SWE132	SWE141	IS123 IT131 SWE141
IS123	IS142	IT131 SWE141 IS142
IT131	—	SWE141 IS142
SWE141	SWE144	IS142 SWE144
IS142, SWE144	—	empty

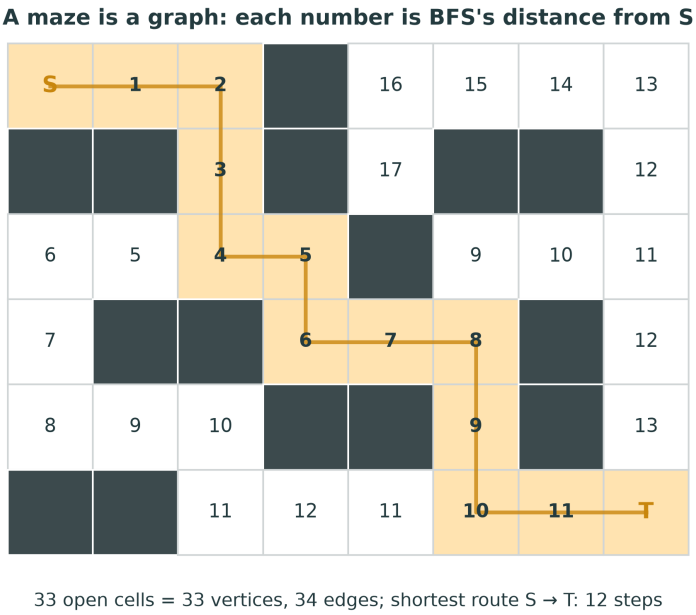
Twelve vertices out: a valid order. Add a made-up edge SWE144 $\rightarrow$ SWE021 and the reference returns `None`: SWE021, SWE132, SWE141 and SWE144 then wait on each other for ever, so only

eight vertices come out. A real bylaw with such a cycle could not be completed — which is exactly what the check detects.

The other classic method is DFS: run the three-colour DFS from every white vertex and record each vertex when it turns **black**. A vertex finishes only after everything it points to has finished, so the reverse of the finishing order is a topological order — and meeting a grey vertex means there is none. It is the same cost, and W14-C3 in the question bank asks you to write it.

Two Real Graphs

A maze is a graph



33 open cells = 33 vertices, 34 edges; shortest route S → T: 12 steps

Open cells are vertices; neighbouring open cells are joined. BFS from S gives every cell's distance — and the shortest route to T.

Nothing about the maze needs a special algorithm. `maze_graph` in `tools/figures_l14.py` turns each open cell into a vertex named by its `(row, column)` tuple — a tuple is hashable, so your hash map takes it — and joins it to its open neighbours on the right and below; the graph is undirected, so that covers left and up too. The 33 open cells give 33 vertices and 34 edges. `shortest_path_unweighted` returns the 12-step route highlighted, and the numbers are the BFS layers: every cell with a 7 is exactly 7 steps from S.

The dead ends show BFS's thoroughness and its cost: to be sure the route is shortest, it explored the whole maze up to distance 12, including the corridor at the top right that leads nowhere near T. A DFS might have found *a* route faster, but not a guaranteed shortest one. This is how game characters find their way on a grid, and how circuit-board routers laid wires in the 1960s (C. Y. Lee's algorithm, 1961, is BFS on a grid).

Your degree is a DAG

- **Topological sort** → a valid study plan.
- **Cycle check** → can the plan be completed at all?
- **BFS from IS122** → everything this course unlocks.
- **Layers** of Kahn's algorithm → the fewest semesters, if you could take anything that is ready.

Every question a registrar asks about a study plan is a graph question. Which courses can I take now? The ones whose in-degree has dropped to 0. What does failing IS122 delay? Everything reachable from IS122: IS123, IT131, SWE141, and through them IS142 and SWE144. How many semesters, at the least? The number of layers when Kahn's algorithm takes all the ready courses at once — here four: {CS012, MATH012, IT022, SWE021}, {CS113, IS122, SWE132}, {IS123, IT131, SWE141}, {IS142, SWE144}. No plan can do better, because the chains CS012 → IS122 → IS123 → IS142 and SWE021 → SWE132 → SWE141 → SWE144 are four courses long. (Real plans are also limited by the credit hours per semester.)

Your own programme's full prerequisite list is in its bylaw (`docs/course/regulations/`), in the course specifications. Typing it into a `Graph(directed=True)` is a good hour with this week's code: W14-C4 in the question bank does exactly that with the layers.

Beyond the Bylaw

Weighted graphs: named, not examined

- **Weighted edges**: a road has a length, a link a cost.
- **Dijkstra's algorithm**: shortest paths by total weight — BFS with a **priority queue** (Week 12) instead of a queue.
- **Minimum spanning tree** (Prim, Kruskal): the cheapest set of edges that keeps a graph connected.

Neither is declared by the bylaw for this course.

BFS's "shortest" counts edges. When edges have different lengths — a 2 km road and a 40 km road — the path with the fewest edges is not the shortest. Edsger Dijkstra's algorithm (1959) keeps the same loop but takes from a priority queue the waiting vertex with the smallest distance so far, and it works as long as no weight is negative. The same 1959 paper also solved the minimum spanning tree problem. `viz.draw.draw_graph` already draws edge weights when a networkx graph has them, if you want to experiment.

These, and balanced trees and NP-completeness, are what this course does **not** reach (`docs/course/02-coverage.md`, "Deliberately outside the declared content"). AI students meet all of them in AI3001 Analysis and Design of AI Algorithms; for SWE and Medical Informatics students, this week is the last graph content in the degree, which is one more reason to finish the exercises. Not examinable.

This Week

Exercises: `dsa/graph.py`

Exercise	Target	The trap
`add_node`, `add_edge`	$O(1)$	adding twice keeps the edges
`neighbours`, `edges`, …	$O(\deg)$, $O(V+E)$	undirected edges reported **once**
`MatrixGraph`	see the table	a new row **and** a new column
`bfs`, `shortest_path_unweighted`	$O(V+E)$	mark on enqueue; parents
`dfs`, `dfs_iterative`	$O(V+E)$	mark on pop; push reversed
`has_cycle`, `topological_sort`	$O(V+E)$	parent against grey; `None`

```
pytest tests/test_graph.py -v
```

The table leaves out `nodes`, `has_edge`, `degree` and `connected_components`, which have no trap beyond those listed (one search per unseen node, for the last). 63 tests: most run twice, once on `Graph` and once on `MatrixGraph`, so a failure shows which representation is wrong. `dsa/graph.py` imports your `ChainingHashMap` and `DynamicArray`, and the searches need your `CircularQueue` (growing, Lecture 07's challenge) and your `Stack`: if `tests/test_hashmap.py` or `tests/test_stack_queue.py` fail, fix them first. `to_networkx` is given to you, and so are `__len__`, `__contains__` and `__repr__` of both classes.

Homework 14 — before Lecture 15

1. **Implement** `dsa/graph.py` until all 63 tests pass.
2. **Trace** `bfs` and `dfs_iterative` from A on the test file's sample graph, with the queue or stack after every step.
3. **Measure** a BFS on `Graph` against `MatrixGraph` in `notebooks/14-graphs.ipynb`, for V = 250 to 2,000.
4. **Your degree.** Type ten prerequisites of your own programme into a directed `Graph`, and print a topological order.

For item 3, build the graphs before timing — `MatrixGraph` spends $O(V^2)$ just on `add_node` — and use a sparse graph, E about 2V. Expect the matrix line to rise with slope 2 on a log–log plot.

For item 4, the specifications are in `docs/course/regulations/`; the lab shows how to draw the result with `viz.draw.draw_graph`.

Summary

Seven things to keep

1. A graph is vertices and edges; lists and trees are graphs with rules.
2. **List**: $O(V+E)$ memory, neighbours in $O(\deg)$. **Matrix**: V^2 memory, `has_edge` in $O(1)$.
3. **BFS** uses a queue, visits in layers, finds shortest paths **by edge count**. Mark on enqueue.
4. **DFS** uses a stack — the call stack or yours. Own stack: mark on pop, push reversed.

5. Both are $O(V + E)$ on a list, $O(V^2)$ on a matrix.
6. Cycles: undirected — not the parent; directed — **grey**, not merely visited.
7. A topological order exists **exactly** for a DAG: Kahn, or reversed DFS finish order.

Next

Week 15 — Principles of language translation. A tokenizer, a recursive-descent parser and an evaluator: a **stack**, **recursion** and a **tree**, together — and an expression's parse tree is one more graph.

Sources and further reading

- **T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein.** *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, chapter 22, "Elementary Graph Algorithms" — representations, BFS, DFS with white/grey/black colours, topological sort.
- **M. T. Goodrich, R. Tamassia and M. H. Goldwasser.** *Data Structures and Algorithms in Python*, Wiley, 2013, chapter 14, "Graph Algorithms" — the same material in Python.
- **R. Sedgewick and K. Wayne.** *Algorithms*, 4th ed., Addison-Wesley, 2011, sections 4.1 "Undirected Graphs" and 4.2 "Directed Graphs".
- **E. F. Moore.** "The shortest path through a maze", *Proceedings of an International Symposium on the Theory of Switching*, Harvard University Press, 1959 — breadth-first search.
- **A. B. Kahn.** "Topological sorting of large networks", *Communications of the ACM* 5(11), 1962.
- **R. Tarjan.** "Depth-first search and linear graph algorithms", *SIAM Journal on Computing* 1(2), 1972.
- **E. W. Dijkstra.** "A note on two problems in connexion with graphs", *Numerische Mathematik* 1, 1959 — shortest paths and spanning trees, beyond this course.
- **Faculty of Computers and Information Sciences, Mansoura University.** Software Engineering programme bylaw, 2013, course specifications — the prerequisites in the topological-sort figure.

Every figure in this lecture is generated by `tools/figures_l14.py` from what the reference `dsa/graph.py` returns. The measured figure is real timing and will differ on your machine.

Questions for Week 14

> ***The answers are at the back of the book,*** *in* Answers — Week 14. *Levels:* ***[what]*** *recall ·* ***[how]*** *apply ·* ***[why]*** *explain and justify. The functions are those of* `dsa/graph.py`. `Graph` *is an adjacency list that keeps each node's neighbours in the order the edges were added, and returns* `nodes()` *in the order the nodes were added;* `MatrixGraph` *is an adjacency matrix.* `bfs` *marks a node when it is* ***enqueued****;* `dfs` *is recursive;* `dfs_iterative` *marks a node when it is* ***popped*** *and pushes neighbours in reverse;* `topological_sort` *is Kahn's algorithm with a queue. Unless a question says otherwise,* ***H*** *is the undirected graph built by*
>
> `A-B, A-D, B-C, B-E, D-E, C-F, E-F, F-G`

> *(added in that order), and **P** is the directed graph built by*
>
> *`A->C, B->C, B->D, C->E, D->E, E->F`.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W14-M01 – W14-M22
B	Short answer and essay	W14-E1 – W14-E5
C	Trace the algorithms	W14-T1 – W14-T5
D	Graph state — draw every step	W14-S1 – W14-S3
E	Complexity analysis	W14-K1 – W14-K3
F	Find and fix the bug	W14-B1 – W14-B4
G	Write the code — checked by `pytest`	W14-C1 – W14-C5

Part A — Multiple choice

W14-M01 [what] An adjacency matrix for a graph with V vertices and E edges uses memory:

- **a)** $O(V + E)$
- **b)** $O(E)$
- **c)** $O(V^2)$
- **d)** $O(V \log V)$

W14-M02 [what] Breadth-first search keeps the vertices waiting to be expanded in a:

- **a)** queue
- **b)** stack
- **c)** priority queue
- **d)** sorted array

W14-M03 [how] An undirected graph has 7 vertices with degrees 2, 3, 2, 2, 3, 3 and 1. How many edges does it have?

- **a)** 16
- **b)** 7
- **c)** 14
- **d)** 8

W14-M04 [why] BFS from s always reaches a vertex first along a path with the fewest edges because:

- **a)** the queue releases vertices in order of non-decreasing distance from s
- **b)** it marks vertices when they are dequeued
- **c)** it tries every path and keeps the shortest
- **d)** the adjacency lists are sorted

W14-M05 [how] `bfs(H, "A")` returns:

- **a)** A B C D E F G

- **b)** A B D C E F G
- **c)** A B C F E D G
- **d)** A D B E C F G

W14-M06 [how] `dfs(H, "A")` returns:

- **a)** A B D C E F G
- **b)** A B C E F D G
- **c)** A B C F E D G
- **d)** A D E B C F G

W14-M07 [how] `shortest_path_unweighted(H, "D", "C")` returns:

- **a)** `["D", "E", "F", "C"]`
- **b)** `["D", "A", "B", "C"]`
- **c)** `["D", "E", "B", "C"]`
- **d)** `None`

W14-M08 [what] `has_edge(u, v)` on an adjacency matrix costs:

- **a)** $O(V)$
- **b)** $O(\deg u)$
- **c)** $O(E)$
- **d)** $O(1)$

W14-M09 [what] `neighbours(u)` on an adjacency matrix costs:

- **a)** $O(V)$
- **b)** $O(1)$
- **c)** $O(\deg u)$
- **d)** $O(E)$

W14-M10 [why] A BFS on a `MatrixGraph` costs $O(V^2)$ even when the graph has very few edges, because:

- **a)** the queue has to grow
- **b)** finding each vertex's neighbours scans its whole row of V cells
- **c)** every cell of the matrix is hashed
- **d)** the visited set is a matrix too

W14-M11 [how] An undirected graph with E edges is stored as an adjacency list. How many neighbour entries do the lists hold in total?

- **a)** E
- **b)** 2E
- **c)** E^2
- **d)** V^2

W14-M12 [why] BFS marks a vertex as visited when it is **enqueued**, not when it is dequeued, because otherwise:

- **a)** a vertex can be put in the queue more than once
- **b)** BFS would miss some reachable vertices

- **c)** the queue would raise `IndexError`
- **d)** the search would no longer find shortest paths

W14-M13 [why] `dfs_iterative` pushes each vertex's neighbours in **reverse** order because:

- **a)** a stack is first in, first out
- **b)** the first neighbour must come off the stack first, to match `dfs`
- **c)** otherwise the search never ends on a cycle
- **d)** it saves memory

W14-M14 [why] The recursive `dfs` is run on a graph that is one path of 5,000 vertices. In Python, with the default settings, it:

- **a)** returns all 5,000 vertices
- **b)** runs out of memory
- **c)** raises `RecursionError`
- **d)** loops for ever

W14-M15 [what] A DAG is:

- **a)** a connected undirected graph with no cycle
- **b)** a graph whose every vertex has the same degree
- **c)** a graph stored as a matrix
- **d)** a directed graph with no cycle

W14-M16 [how] `topological_sort` on the directed graph A $\to$ B, B $\to$ C, C $\to$ A returns:

- **a)** `["A", "B", "C"]`
- **b)** `[]`
- **c)** `None`
- **d)** it raises `ValueError`

W14-M17 [why] Undirected cycle detection ignores a visited neighbour that is the vertex's parent, because:

- **a)** every undirected edge is stored in both lists, so the vertex you came from always looks visited
- **b)** the parent is always grey
- **c)** a parent cannot be part of a cycle
- **d)** it makes the search $O(V + E)$

W14-M18 [why] In the directed diamond A $\to$ B, A $\to$ C, B $\to$ D, C $\to$ D, a DFS from A meets D a second time, from C. That is not a cycle, because at that moment D is:

- **a)** white
- **b)** grey
- **c)** black
- **d)** the start vertex

W14-M19 [how] A graph has vertices A to I and the undirected edges A–B, B–C, D–E, F–G. How many connected components does it have?

- **a)** 4

- **b)** 3
- **c)** 9
- **d)** 5

W14-M20 [how] `topological_sort(P)` returns:

- **a)** A B C D E F
- **b)** B D A C E F
- **c)** A C B D E F
- **d)** F E D C B A

W14-M21 [why] Which of these can BFS on its own **not** answer correctly?

- **a)** the path with the fewest edges from s to t
- **b)** whether t can be reached from s
- **c)** the shortest route by kilometres on a road map with different road lengths
- **d)** the connected components of an undirected graph

W14-M22 [how] One BFS over an adjacency list costs:

- **a)** $O(V \cdot E)$
- **b)** $O(V^2)$
- **c)** $O(E \log V)$
- **d)** $O(V + E)$

Part B — Short answer and essay

W14-E1 [why] *(4 marks)* Compare the adjacency list and the adjacency matrix: memory, `has_edge`, `neighbours`, and the cost of one BFS. Which would you choose for a road map of 100,000 junctions, and which for a graph of 500 people in which almost everyone knows almost everyone and you ask "do these two know each other?" millions of times? Justify with numbers.

W14-E2 [why] *(4 marks)* Describe breadth-first search. Explain why it finds shortest paths by number of edges, how `shortest_path_unweighted` rebuilds the path, and why the answer can be wrong on a road map with different road lengths.

W14-E3 [why] *(4 marks)* Recursive DFS and DFS with your own stack: what does the call stack remember for you? Why must `dfs_iterative` mark a vertex when it is **popped** and push neighbours in **reverse** to give the same order as `dfs`? Why write the iterative version at all in Python?

W14-E4 [why] *(4 marks)* Explain cycle detection in an undirected graph and in a directed graph. Why does each rule give wrong answers when used on the other kind of graph? Give a small example of each failure.

W14-E5 [why] *(3 marks)* Define a topological order. When does one exist? Describe Kahn's algorithm, how it detects that no order exists, and one real application.

Part C — Trace the algorithms

W14-T1 [how] Trace `bfs(H, "A")`: after each vertex is taken off the queue, give the vertices it adds and the queue. What is the distance from A to each vertex?

W14-T2 [how] Trace the recursive `dfs(H, "A")`: show each call, indented by depth, and each neighbour that is skipped because it is already visited. What is the deepest the call stack gets?

W14-T3 [how] Trace `dfs_iterative(H, "A")`: after each pop, the action and the stack (bottom → top). Which vertices are pushed twice?

W14-T4 [how] Trace `shortest_path_unweighted(H, "A", "G")`: give the parent map as BFS builds it, and the walk back from G. Then give `shortest_path_unweighted(H, "D", "C")`, and explain why a different path of the same length is not returned.

W14-T5 [how] Trace `topological_sort(P)`: the in-degrees at the start, and after each vertex is taken, the in-degrees that change and the queue. Is B D A C E F also a valid topological order of P?

Part D — Graph state

W14-S1 [how] Starting from an empty undirected `Graph` and an empty `MatrixGraph`, apply to both:

```
add_edge A B,  add_edge B C,  add_node D,  add_edge C A,  add_edge D B
```

Draw the adjacency lists after each operation, and the matrix at the end, with rows and columns in node order. Then give `nodes()`, `edges()` and `degree("B")`.

W14-S2 [how] The directed graph Q is built by

```
A->B, B->C, A->D, D->C, D->E, E->F, F->D
```

Run `has_cycle(Q)`. After every vertex is entered or finished, give the colour of every vertex seen so far (white, grey, black). Which edge proves the cycle? Which edge reaches a black vertex, and why is that not a cycle? What would `topological_sort` return for Q, and for Q without the edge F → D?

W14-S3 [how] The undirected graph U is built by `A-B, A-C, C-D, D-E, E-C`. Trace `has_cycle(U)` with the parent check: for each call, the vertex, its parent, and what happens to each neighbour. At which moment is the cycle found?

Part E — Complexity analysis

W14-K1 [how] Give Θ for this code on a `Graph` with V vertices and E edges, and on a `MatrixGraph`, and say what it computes for an undirected graph:

```
total = 0
for node in graph.nodes():
    for other in graph.neighbours(node):
        total += 1
```

W14-K2 [why] A student writes connected components without the `seen` check:

```
def components(graph):
    return [bfs(graph, node) for node in graph.nodes()]
```

What is wrong with the result, and what is its cost? What is the cost of the correct version, and why?

W14-K3 [why] To count the edges of an undirected graph, a student asks about every pair:

```
count = 0
for u in graph.nodes():
    for v in graph.nodes():
        if graph.has_edge(u, v):
            count += 1
count //= 2
```

Give Θ on a `Graph` and on a `MatrixGraph`. What does `len(graph.edges())` cost on each?

Part F — Find and fix the bug

W14-B1 [how]

```
def bfs(graph, start):
    order = []
    visited = ChainingHashMap()
    queue = CircularQueue()
    queue.enqueue(start)
    while not queue.is_empty():
        node = queue.dequeue()
        visited.put(node, True)
        order.append(node)
        for neighbour in graph.neighbours(node):
            if neighbour not in visited:
                queue.enqueue(neighbour)
    return order
```

W14-B2 [why] In `Graph`:

```
def add_node(self, node):
    self._adjacent.put(node, DynamicArray())
    self._order.append(node)
```

W14-B3 [why] Undirected cycle detection:

```
def has_cycle(graph):
    visited = ChainingHashMap()

    def visit(node):
        visited.put(node, True)
        for neighbour in graph.neighbours(node):
            if neighbour in visited:
                return True
            if visit(neighbour):
                return True
        return False

    for node in graph.nodes():
        if node not in visited and visit(node):
            return True
    return False
```

W14-B4 [why] The same function as W14-B3, used as the **directed** cycle check. Find an acyclic directed graph on which it returns `True`, and fix it.

Part G — Write the code

In `practice/week14.py`; check with `pytest tests/test_practice_week14.py -v`. Use your own `Graph`, `ChainingHashMap`, `CircularQueue` or `Stack` as working storage — not a Python dict, set or deque.

W14-C1 [how] `maze_distance(maze)` — the fewest steps from S to T in a grid maze, or $-$1.

W14-C2 [why] `is_bipartite(graph)` — can the vertices be coloured with two colours so that every edge joins different colours? Why does an odd cycle make it impossible?

W14-C3 [how] `topological_order_dfs(graph)` — a topological order from reversed DFS finishing order, or `None` for a cycle.

W14-C4 [why] `semester_plan(prerequisites)` — the courses grouped into the fewest semesters. Why is the number of semesters the length of the longest prerequisite chain?

W14-C5 [how] `count_islands(grid)` — connected components of the land cells in a grid.

Week 15 — The Principles of Language Translation

> ***How to read this document.*** *This is the handout for Lecture 15, the last of the course. It holds everything on the slides, plus what I said out loud. This week you build a small but complete translator: it reads a line of arithmetic as text and computes its value, in three stages that every compiler and interpreter has. It needs a stack, recursion and a tree at once, which is why it closes the course. The handout ends with a look back over the fifteen weeks and a map for the final exam.*
>
> *Slides: `DSA27-L15-slides.pdf` · Code: `dsa/translation.py` · Tests: `tests/test_translation.py`*

Where We Are

Fourteen weeks of parts. One machine.

You have built: the `Array`, recursion, the dynamic array, linked lists, **stacks**, queues, searches, sorts, **trees**, heaps, hash tables, graphs.

Today they meet in one program:

```
>>> calculate("2 * (3 + (4 - 1))")
12.0
```

Text in, a number out. How does a machine **read**?

Every program you have written this term was text until something read it. When you press *Run*, CPython reads your file character by character, groups the characters into words, works out how the words fit together, and only then does anything. That reading is **language translation**, and the bylaws of 2013 and 2014 ask this course for "an introduction to the principles of language translation" (`docs/course/02-coverage.md`, row 19).

We will not build a Python. We will build the smallest language that still has every hard part: arithmetic with `+ - * /`, parentheses and a unary minus. Small enough to finish in a week; big enough that precedence, associativity, nesting and error messages all have to be got right.

Language translation in Arabic: ترجمة اللغات.

Today

1. What a translator does: compiler, interpreter, three stages
2. Stage 1 — the **lexer**: text → tokens
3. Grammars: precedence and associativity as **shape**

4. Stage 2 — **recursive descent**: tokens → tree
5. Stage 3 — **evaluation**: a post-order walk
6. Two routes to one answer: the Week 6 stack against the tree
7. Measured; where this grows up; the course, looking back

What a Translator Does

Compiler or interpreter?

	Reads the source …	… and produces	Example
Compiler	once, ahead of time	another program (machine code, bytecode)	`gcc`, `javac`
Interpreter	as it runs	the **result** directly	a shell, a calculator
Both	compiles to bytecode, then interprets it		CPython, the JVM

Our `calculate` is an interpreter. The front half is the same for both.

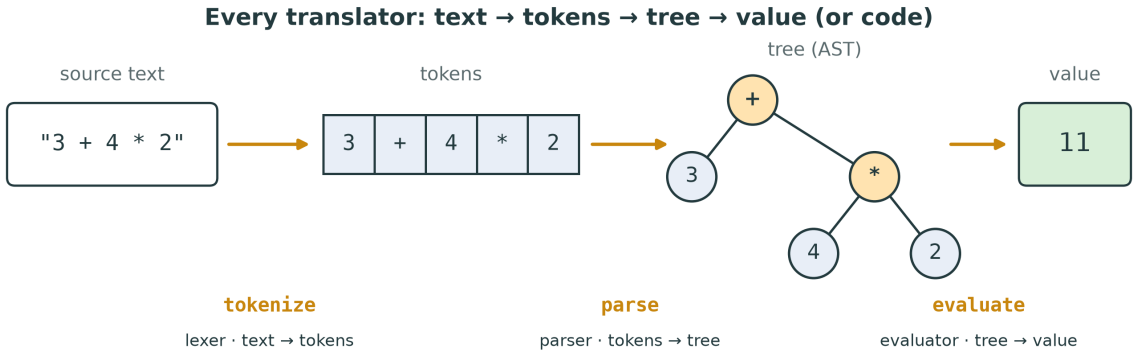
A **translator** turns text written in one language into something else: another language, or an action. A **compiler** translates the whole program first and hands you the result to run later: C and C++ compile to machine code, Java to bytecode for the JVM. An **interpreter** executes as it reads, and gives you the answer rather than a program: a calculator, a shell, a spreadsheet formula.

Python is both, which surprises most people. When you run a file, CPython first **compiles** it to bytecode (the `.pyc` files in `__pycache__`), then its **interpreter loop** runs the bytecode. You will see that bytecode at the end of the lecture, and it will look familiar.

The distinction matters less than it seems, because the first stages are identical: both must read the text and understand its structure before they can do anything with it. That shared front end is this week's subject.

Translator in Arabic: المترج. *Compiler*: المُصرّف. *Interpreter*: المفسّ.

Three stages




```
calculate(text) = evaluate(Parser(tokenize(text)).parse())
```

Lexer, parser, evaluator: each stage has one job, and each can say no.

Stage	Input → output	This week	Structure it uses
Lexer	text → tokens	`tokenize`	a scan (Week 2)
Parser	tokens → tree	`Parser`	recursion (Week 3), the `Array`
Evaluator	tree → value	`evaluate`	a tree walk (Week 11)

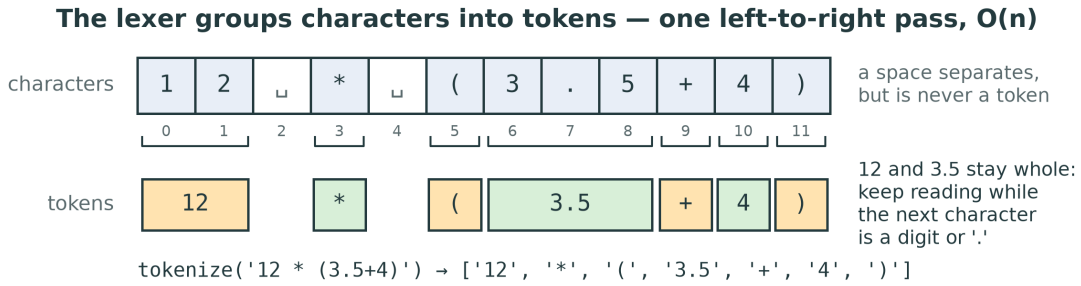
- The **lexer**, the *lexical analyser*, reads characters and groups them into **tokens**: numbers, operators, parentheses. It knows nothing about structure; `3 + + 4` tokenizes happily.
- The **parser**, the *syntax analyser*, reads the tokens and decides how they fit together, according to a **grammar**. It produces a **tree** — the *abstract syntax tree*, or **AST** — and rejects anything the grammar does not allow.
- The **evaluator** walks the tree and computes the value. In a compiler, this stage is replaced by *code generation*: the same walk, but it writes instructions instead of computing numbers.

Each stage checks one kind of mistake. `3 $ 4` is a **lexical** error: `$` is not a token of our language. `3 + * 4` is a **syntax** error: every token is fine, but they are in an order the grammar forbids. `1 / 0` is a **runtime** error: a perfectly formed expression whose value does not exist. Python reports the same three kinds, and now you will know which stage raised each.

Parser in Arabic: المحلل النحوي. *Abstract syntax tree*: شجرة الإعراب المجرد. *Evaluator*: المُقيِّ.

Stage 1: The Lexer

Characters into tokens



`tokenize("12 * (3.5+4)")` → `['12', '*', '(', '3.5', '+', '4', ')']`

- Whitespace **separates**, but is never a token.
- A number **keeps going** while the next character is a digit or `.`.
- Anything else is an error: `ValueError`.

A token is the smallest piece that means something. In `12 * (3.5+4)` there are 12 characters but only 7 tokens: `12` is one number, not a 1 followed by a 2, and the spaces mean nothing except "here one token ends". Notice the lexer does not need spaces at all: `3+4` and `3 + 4` give the same tokens, because + can never be part of a number.

The lexer is a single left-to-right scan with an index `i`, the loop shape you have written since Lab 01. At each position it looks at one character and decides:

1. a space — skip it;
2. one of `+ - * / ( )` — it is a whole token by itself;
3. a digit or a decimal point — the start of a number: keep moving `i` while the next character is a digit or `.`, then cut the number out of the text in one piece;
4. anything else — raise `ValueError`, naming the character and its position.

Each character is looked at a constant number of times, so `tokenize` is $O(n)$ in the length of the text. The docstring forbids regular expressions: real lexers are often generated from them, but scanning by hand is how you learn what a regular expression is doing for you.

Two traps. The inner loop that reads a number must check `i < len(text)` **before** reading `text[i]`: a number at the very end of the text is the normal case, not an edge case, and forgetting the check raises `IndexError` on nearly every test. And a number must not contain two points: `1..2` is a lexical error, not the number 1.

Token in Arabic: وحدة لفظي. *Lexer*: المحلل اللفظي.

Grammars

Rules that say what is allowed

```
expr   := term (('+' | '-') term)*
term   := factor (('*' | '/') factor)*
```

```
factor := NUMBER | '(' expr ')' | '-' factor
```

Read `:=` as "is", `|` as "or", `( … )*` as "zero or more times".

A **grammar** is a finite set of rules that describes an infinite set of sentences. The notation is **EBNF** — Extended Backus–Naur Form. John Backus and Peter Naur used the plain form, BNF, to define Algol 60 in 1960 (Naur edited the report), and every programming language since has been specified this way; the Python Language Reference defines Python's syntax in a grammar of the same kind.

Read the three rules as three sentences:

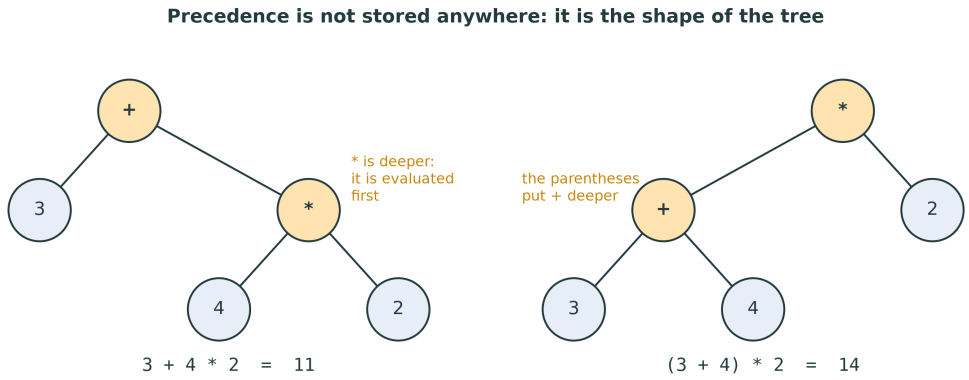
- an **expression** is a term, followed by any number of `+ term` or `- term`;
- a **term** is a factor, followed by any number of `* factor` or `/ factor`;
- a **factor** is a number, or a whole expression in parentheses, or a minus sign followed by a factor.

The names are traditional: a *term* is something you add, a *factor* something you multiply. Plain BNF has no * for repetition, so the same grammar is written with recursion instead — `expr := expr '+' term | term` — and that version hides a trap we will fall into on purpose later.

Notice the third rule mentions `expr`, the first. The grammar is **recursive**: an expression can contain a factor that contains an expression. That is what lets `((((7))))` be as deep as you like, and it is why the parser will be recursive too.

Grammar in Arabic: القواعد النحوي.

Precedence is the shape of the grammar



`expr` is built from `terms`, so every * is grouped **before** any + sees it. No table of numbers — the **layering** of the rules does it.

In Week 6 the shunting-yard algorithm needed a precedence table: `{"+": 1, "-": 1, "*": 2, "/": 2}`. The grammar has no table anywhere, yet it gets `3 + 4 * 2` right. How?

An `expr` is a sum of `terms`, and a `term` is a product of `factors`. So to read `3 + 4 * 2` as an `expr`, the parser must split it into terms at the + signs: the terms are `3` and `4 * 2`. The * is swallowed

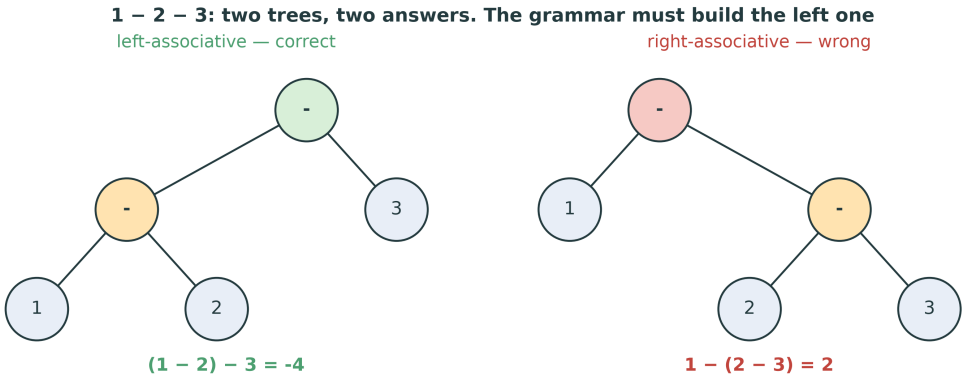
inside the second term before the + is ever applied. In the tree, that means * sits **deeper** than +, and deeper nodes are evaluated first.

Precedence is therefore not a property of the operators; it is a property of **which rule each operator lives in**. The lower the rule — the closer to `factor` — the tighter the operator binds. To add a new level, such as ^ for powers, you add a new rule between `term` and `factor`; that is problem W15-C5.

Parentheses work because `factor := '(' expr ')'`: a parenthesised expression, however loose its operators, is a **factor** — the tightest thing there is. In the right-hand tree, `3 + 4` is pushed below the *.

Precedence in Arabic: أولوية العمليات.

Associativity: which way the tree leans



`1 - 2 - 3` means `(1 - 2) - 3 = -4`. Operators of **equal** precedence group from the **left**: the tree must lean left.

Precedence decides between *different* levels. **Associativity** decides between operators on the *same* level. For + and * the order hardly matters mathematically, but for - and / it changes the answer: `8 / 4 / 2` is `(8 / 4) / 2 = 1`, not `8 / (4 / 2) = 4`.

All four of our operators are **left-associative**, as in mathematics and in Python. In the tree, left-associative means that each new operator takes the tree built so far as its **left** child. The right-hand tree in the figure is what a plausible-looking but wrong parser builds — and it passes every test that does not contain two - or two / in a row (W15-B1).

Exponentiation is the classic **right**-associative operator: `2 ** 3 ** 2` is `2 ** 9 = 512` in Python, not `8 ** 2 = 64`. In Week 6 you saw associativity as "pop on higher **or equal** precedence"; here you will see it as "loop, not recursion" — the same fact in two costumes.

Associativity in Arabic: خاصية التجمي.

Stage 2: Recursive Descent

One method per rule

Grammar	Code
a rule	a method: `expr`, `term`, `factor`
a rule named inside another	a **call**
`( … )*`	a `while` loop
`a | b | c`	`if` / `elif` on the next token
a token in quotes	check it, then `advance()`

The methods call each other **down** the grammar — and back **up** through the parentheses. Hence *recursive descent*.

Recursive descent is the most direct way ever found to turn a grammar into a program: you translate it line by line using the table above, and the parser is finished. It is not a toy. The front ends of GCC and Clang for C and C++ are hand-written recursive-descent parsers, and so were many famous compilers before them.

`Parser` in `dsa/translation.py` holds the tokens in the course `Array` and one integer, `position`: the index of the next token to read. Two helpers are given:

- `peek()` returns the current token **without** consuming it, or `None` at the end;
- `advance()` returns the current token **and** moves past it.

Everything the parser does is a decision made by **peeking** at one token: "is the next thing a `+`?", "is it a `(`?". A parser that decides by looking one token ahead, never back, is called *predictive*, and our grammar was written so that one token is always enough.

Recursive descent in Arabic: التحليل التنازلي العودي.

`expr`, in full

```
def expr(self):
    node = self.term()
    while self.peek() in ("+", "-"):
        op = self.advance()
        node = BinOp(op, node, self.term())  # the old tree goes LEFT
    return node
```

`term` is the same shape, one level down: `factor` for `term`, `*` and `/` for `+` and `-`.

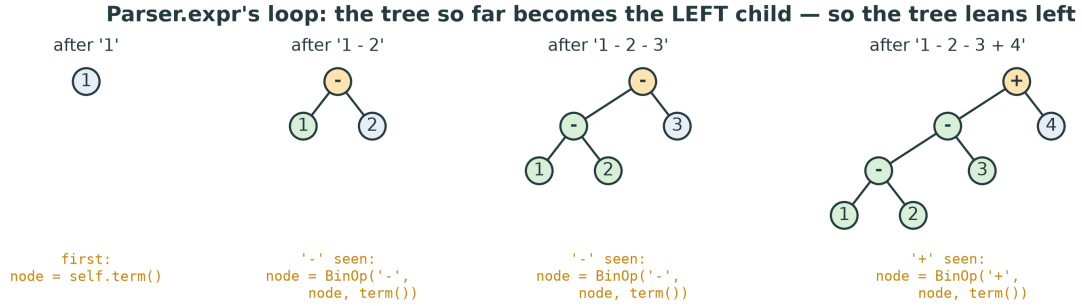
This is the reference solution's `expr`, without its docstring (and shown without the class's indentation), and it is the template for `term`. Read it against the rule `expr := term (('+' | '-') term)*`: first a `term`; then a loop that runs for as long as the next token is `+` or `-`; inside, consume the operator and parse one more `term`.

The line that matters is the last one in the loop. The tree built so far, `node`, becomes the **left** child of a new `BinOp`, and the new term becomes the right. Do this three times and the tree leans left — exactly the associativity we need, built by a loop with no special effort.

`self.peek() in ("+", "-")` is safe at the end of the input: `peek()` returns `None`, and `None`

`in ("+", "-")` is simply `False`. The loop ends; `expr` returns what it has; whoever called it decides whether stopping there is allowed.

The loop builds a left-leaning tree



The figure is `expr` on `1 - 2 - 3 + 4`, one snapshot per turn of the loop. Green is the old `node`, now hanging on the left; amber is the new root. After three turns the root is the **last** operator, `+`, and the first subtraction is deepest — so it is evaluated first. That is `((1 - 2) - 3) + 4 = 0`, which is what `1 - 2 - 3 + 4` means.

`term` builds `*` and `/` chains the same way, and each of its operands is a `factor`, so a `term` inside an `expr` is a finished subtree before `expr` places it. That is how the two loops combine into one tree with precedence and associativity both right.

`factor`: where the recursion closes

Look at the next token:

- **a number** → consume it; return `Num(float(token))`.
- `(` → consume it; `node = self.expr()`; the next token **must** be `)` — consume it; return `node`.
- `-` → consume it; return `BinOp("-", Num(0.0), self.factor())`.
- **anything else**, or the end → `ValueError`.

`factor` calls `expr`: the grammar's recursion, and the parser's.

This is the method with the most cases and the fewest lines per case. Three points need care.

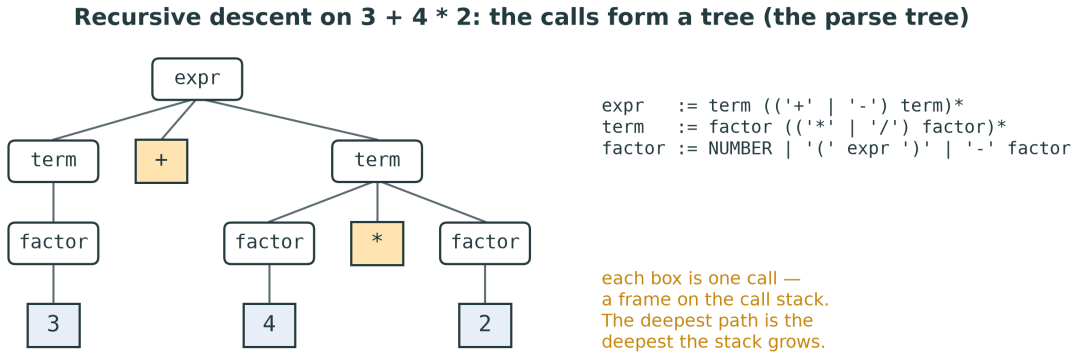
The parentheses. After `self.expr()` returns, the parser has consumed everything up to — but not including — the token that stopped the inner expression. If that token is `)`, good: consume it, or the outer parser will meet it and complain about an unexpected `)`. If it is anything else, the parentheses are unbalanced: `"(3 + 4"` must raise `ValueError`, not quietly return 7.

Unary minus. `-5` has no left operand, so we invent one: it becomes the tree for `0 - 5`. The operand is a `factor`, not an `expr`, so the minus binds tightly: `-5 + 2` is `(0 - 5) + 2 = -3`. With `self.expr()` there instead, the minus would swallow the whole rest of the line, and `-5 + 2` would become `-(5 + 2) = -7` (W15-B3). Because `factor` calls itself, `- - 3` is 3, and `2 - -3` is 5.

Running out. `"3 +"` reaches `factor` after the `+` with nothing left: `peek()` is `None`. Check for `None` first; an index into `None` raises `TypeError`, the wrong exception with an unhelpful message.

Then `parse`, the entry point: refuse an empty token list, call `expr`, and afterwards insist that **every** token has been used. `"3 + 4 )"` parses `3 + 4` and stops at the `)`; without that last check, it would be accepted as 7.

The calls are the parse tree



Each call is a frame on the call stack (Lecture 03). The **deepest** chain of calls is the **deepest** the stack grows.

The figure was drawn by recording every call and return of the reference parser on `3 + 4 * 2`. Each rounded box is one call; its children are the calls it made and the tokens it consumed, left to right. This tree of calls is called the **parse tree** (or *concrete syntax tree*): it records every rule used, including the ones that did nothing but pass a value up, like the `term` above 3.

The AST is the parse tree with the scaffolding removed. Collapse every box that has a single child into that child, and put each operator in the place of the box that consumed it: what is left is the tree + over 3 and *, with 4 and 2 under the * — the AST of the pipeline figure. That is why it is called *abstract*: it keeps the structure and drops the grammar's bookkeeping.

At any moment during parsing, the frames on Python's call stack are exactly one path from the root of this tree down to the box being worked on. You never built a stack for the parser, because Python's call stack **is** the parser's stack. Week 6 said recursion is a stack; this is recursion using that stack to remember "I am inside a `term`, inside an `expr`, inside a parenthesis, inside an `expr`".

Two grammars that look right and are not

Right recursion — `expr := term ('+' expr)?`

```
node = self.term()
if self.peek() in ("+", "-"):
    return BinOp(self.advance(), node, self.expr())
```

`1 - 2 - 3` → **2**. Wrong, silently.

Left recursion — `expr := expr '+' term | term`

```
left = self.expr()      # before reading any token
```

`RecursionError`, on every input.

Both come from writing the recursive BNF form directly as code.

The **right-recursive** version is the most common recursive-descent bug in student code. It parses every correct expression without an error, and gets precedence right, because `term` still does its job. Only associativity is wrong: the recursive call parses the whole **rest** of the line and puts it on the right. On the Week 15 tests it fails exactly 3 of 55: the tree test for `1 - 2 - 3`, `calculate("1 - 2 - 3")` (2 instead of -4), and the test that compares the two routes. `8 / 4 / 2` survives only because `term` was written with a loop — the same bug in `term` would fail it too.

The **left-recursive** version is the grammar that textbooks write for left-associative operators, and it is correct *as a grammar*. As a recursive-descent program it never terminates: `expr` calls `expr` with `position` unchanged, which calls `expr` with `position` unchanged, until Python stops it at 1,000 frames. A recursion must make progress towards its base case (Lecture 03) — this one consumes nothing before recursing. That is why our grammar is written with `( … )*`: the repetition becomes a loop, and the loop builds the left-leaning tree that the left recursion described.

Errors: say no, and say where

Input	Stage	Raised
`3 $ 4`	lexer	`ValueError`
(empty text), `3 +`, `* 3`, `()`	parser	`ValueError`
`(3 + 4`, `3 + 4 )`, `3 4`	parser	`ValueError`
`1 / 0`	evaluator	`ZeroDivisionError`

The reference solution's messages:

- `3 $ 4` — `unexpected character '$' at position 2` (the lexer);
- empty text — `empty expression` (`parse`);
- `3 +` — `unexpected end of input` (`factor`, finding nothing after the +);
- `* 3` and `()` — `unexpected '*'` and `unexpected ')'` (`factor`, finding a token that cannot start a factor);
- `(3 + 4` — `missing ')'` (`factor`, after the inner `expr`);
- `3 + 4 )` — `unexpected ')' after the expression` (`parse`, with a token left over); `3 4` gives `unexpected '4' after the expression` for the same reason: `expr` parses the 3, finds no + or -, and returns;
- `1 / 0` — `ZeroDivisionError: division by zero`, raised by Python's own `/` inside `evaluate`.

The tests check only the exception **type**, so yours may word them differently.

A parser's job is as much to **reject** as to accept. A calculator that answered 7 to `3 + 4 )` would be hiding a mistake from its user, and a compiler that did the same would turn a typo into a bug. Good translators also say **where** the problem is — a position, a line and column — because a

message without a location sends the programmer hunting. Real parsers go further and **recover**: they skip to a likely restart point and keep parsing, so that one run reports every error, not just the first. That is beyond this course.

Stage 3: Walking the Tree

The AST is a Week 11 tree

- `Num(value)` — a **leaf**. `BinOp(op, left, right)` — a node with two children.
- Height = the depth of nesting. Size = the number of tokens, minus the parentheses.

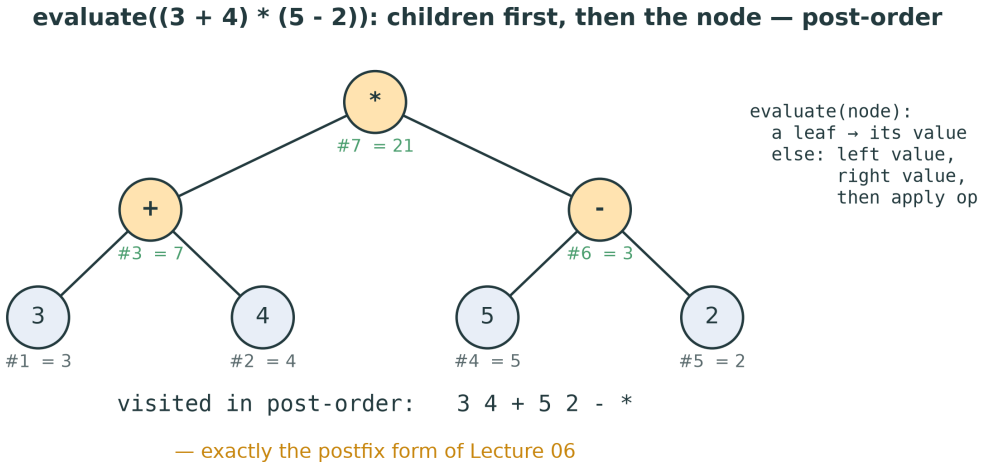
```
from viz.draw import draw_tree
from dsa.translation import Parser, tokenize, to_edges
draw_tree(to_edges(Parser(tokenize("3 + 4 * 2")).parse()))
```

`Num` and `BinOp` are node classes, given to you, exactly like `TreeNode` in `dsa/tree.py` — the storage rule's "node objects". An expression tree is a **binary tree** but not a binary **search** tree: its order means "which operand is which", not "smaller on the left".

`to_edges` (given) turns a tree into the list of (parent, child) pairs that `viz.draw.draw_tree` renders. It names each node `label#n`, because one tree can hold the same number twice: in `2 + 2` the two leaves are different nodes that happen to print alike, and without the `#n` Graphviz would draw them as one (`test_to_edges_keeps_duplicate_values_apart`).

Every tree in this lecture is the tree the reference parser actually built; `tools/figures_l15.py` parses each expression and draws the result.

`evaluate` is a post-order walk



A leaf: its value. Otherwise: evaluate the **left**, evaluate the **right**, **then** apply the operator. Children first — post-order (Lecture 11).

`evaluate` is recursion on a tree, and it could not be shorter: a base case for `Num`, and for `BinOp` two recursive calls followed by one arithmetic step chosen by `node.op`. An operator that is none

of the four raises `ValueError`; dividing by zero raises `ZeroDivisionError` on its own, because Python's `/` does.

Each node is visited once and does $O(1)$ work, so evaluation is $O(n)$ in the number of nodes. The recursion is as deep as the tree is **high**: for a balanced tree that is about $\log_2 n$, but for `1 + 1 + ... + 1` the tree leans left all the way down, its height is n - 1, and a thousand-term sum reaches Python's recursion limit. The measured figure below shows this.

Now read the order in which the figure visits the nodes: `3 4 + 5 2 - *`. That is the **postfix** form of `(3 + 4) * (5 - 2)`, from Lecture 06. It is not a coincidence. Evaluating postfix with a stack processes the operands before their operator; evaluating a tree in post-order does the same. The postfix list **is** the post-order traversal of the expression tree, written out flat (W15-C1).

`calculate`: the three stages, in three lines

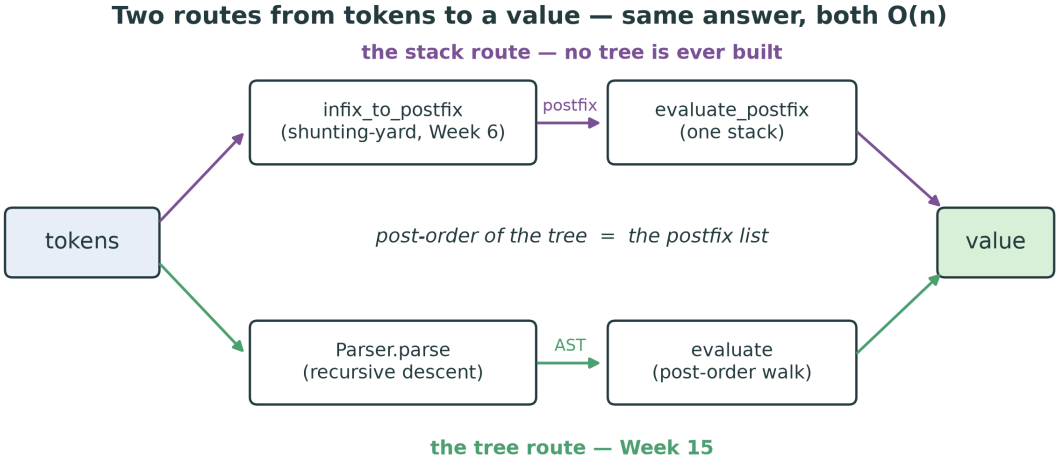
```
tokens = tokenize(text)
tree = Parser(tokens).parse()
return evaluate(tree)
```

The body of the reference `calculate`, exactly. It is the least interesting code of the week and the most satisfying to write: the moment the three stages stop being three exercises and become one program. Each stage can fail with its own exception, and `calculate` lets every one of them through to the caller.

Keeping the stages separate is the point of the design. You can test the lexer without a parser, and the evaluator on a tree you built by hand (`test_evaluate_a_hand_built_tree`) — which is how compilers are built and tested too.

Two Routes, One Answer

The stack route and the tree route



`test_both_routes_agree`: for `3 + 4 * 2`, `(3 + 4) * 2`, `1 - 2 - 3` and `8 / 4 / 2`, both routes give the same value.

Week 6 already had a complete calculator: `infix_to_postfix` (Dijkstra's shunting-yard, with an operator stack) followed by `evaluate_postfix` (with a value stack). No tree and no recursion, and it is $O(n)$ too. So why build the tree route at all?

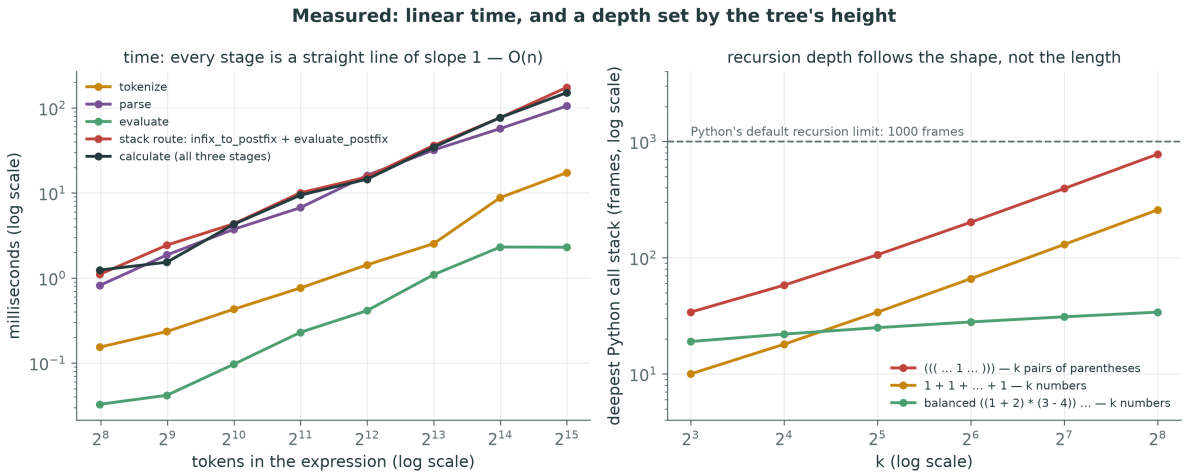
- **The tree can be used more than once.** Evaluate it, print it back with minimal parentheses (W15-C3), simplify it, differentiate it, or generate code from it. The stack route produces a value and forgets the structure; a compiler needs the structure.
- **The grammar grows; the table does not.** Unary minus is one line in `factor`. The Week 6 `infix_to_postfix` has no idea what to do with `-5`: it treats the minus as binary, produces `5 -`, and `evaluate_postfix` rejects that with `ValueError`. Supporting it needs special cases in the algorithm. Function calls, variables and `if` are routine in a grammar and awkward in shunting-yard.
- **Errors are easier to explain** when a rule is being parsed: "missing) after the expression that started at position 4".

And why keep the stack route? It needs no recursion, so no depth limit; and postfix is a perfect **target**. The stack is where the tree route ends up: the post-order walk of the tree *is* the postfix list. Python itself does exactly this — shown at the end of the lecture.

The shunting-yard algorithm is, in a sense, the post-order walk of a tree it never builds; recursive descent builds the tree and then walks it.

Measured

Linear time; depth set by the shape



Left: time. Every stage, timed with the reference solution on balanced expressions from 253 to 32,765 tokens. On a log–log plot every line has slope about 1: doubling the input roughly doubles the time, $O(n)$, for the lexer, the parser, the evaluator and the Week 6 stack route alike. The lines wobble a little because the machine was busy with other work while measuring; the slope is what to read, not any single point.

Constant factors. Parsing dominates: in this run it was 5 to 10 times slower than tokenizing and 30 to 50 times slower than evaluating, because every token passes through several method calls (`expr`, `term`, `factor`, `peek`, `advance`) and each of those goes through the bounds check of the course `Array`. The stack route costs about the same as the tree route: both pass each token through a few Python-level function calls.

Right: depth. The deepest Python call stack reached while running `calculate`, counted exactly, for three shapes of input with k numbers or k pairs of parentheses:

- **Nested parentheses**, `((( … 1 … )))`: three frames per pair — `expr`, `term`, `factor` — so the depth is about 3k: 778 frames at k = 256. With Python's default limit of 1,000 frames, about 330 pairs of parentheses raise `RecursionError`.
- **A flat chain**, `1 + 1 + … + 1`: the parser handles it with a loop, but the tree it builds leans left all the way down, and `evaluate` recurses once per level: about k frames, 258 at k = 256. A chain of about a thousand numbers fails — in `evaluate`, not in the parser.
- **Balanced** input: 34 frames at k = 256. The depth grows with $\log_2 k$, because the height of a balanced tree does (Lecture 11).

So: the time of a translator depends on the **length** of its input, but its recursion depth depends on the **shape** — the height of the tree. This is the balanced-against-degenerate lesson of Week 11, met again from the other side. Real compilers raise their stack limits, or evaluate deep chains with a loop and an explicit stack, for exactly this reason.

Where This Grows Up

From calculator to compiler

```
>>> import ast
>>> ast.dump(ast.parse("3 + 4 * 2", mode="eval"))
Expression(body=BinOp(left=Constant(value=3),
  op=Add(), right=BinOp(left=Constant(value=4),
  op=Mult(), right=Constant(value=2))))
>>> import dis
>>> dis.dis(compile("a + b * c", "<s>", "eval"))
LOAD_NAME a;  LOAD_NAME b;  LOAD_NAME c
BINARY_OP *;  BINARY_OP +
```

Python's own tree is our tree. Its bytecode is our **postfix**.

The first line is CPython's own parser, from its standard `ast` module, on the same expression: a `BinOp` for + whose right child is a `BinOp` for * — the shape our parser builds, with longer names. (The real output is one line, in quotes; it is broken here to fit.) Parse `x - y - z` and you get the left-leaning tree of the associativity figure.

The second is CPython's compiler: the bytecode for `a + b * c` (Python 3.14), condensed here to the instruction names, without the line numbers and the `RESUME` and `RETURN_VALUE` instructions around them. Push a, push b, push c, multiply the top two, add the top two: **postfix**, run on a stack machine. Every Python program you have written this term was turned into a post-order

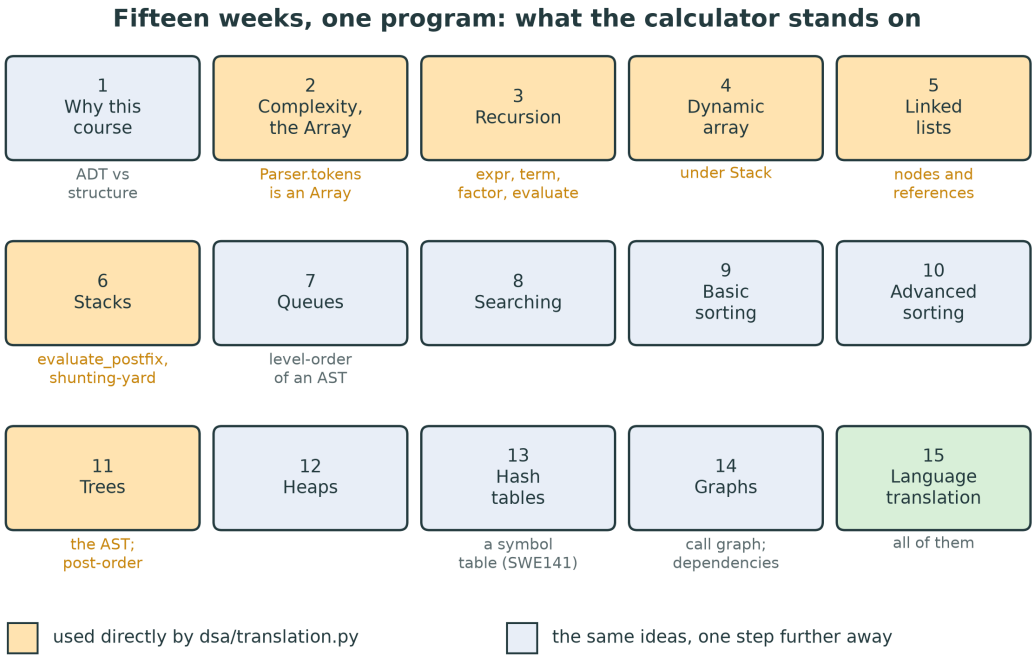
walk of its syntax tree and executed with a stack. The two routes of this lecture are not rivals; a real compiler uses both, one after the other.

What a real translator adds. Variables need a **symbol table** — a hash table from names to values or types (Week 13). Control flow — `if`, loops, functions — becomes a **graph** of basic blocks that the optimiser walks (Week 14). Types are checked by walking the AST before anything runs. And the evaluator is replaced by a **code generator**: the same post-order walk, emitting instructions instead of computing numbers.

Where you meet it again. For Software Engineering students (2013 bylaw), this course is a prerequisite of **SWE141 Software Construction**, which covers "BNF and basic theory of grammars and parsing… formal languages" (2013, p. 42). There the grammar becomes a formal object, and parsers are generated from it by tools rather than written by hand. Whatever your program, you now know what happens between pressing *Run* and the first line executing.

The Course, Looking Back

Fifteen weeks, one program



`dsa/translation.py` stands on more of the course than any other module:

- **The `Array`** (Week 2) holds the parser's tokens: `Parser.__init__` calls `Array.from_values`.
- **Recursion** (Week 3) *is* the parser, and the evaluator; the base cases are a number and a leaf.
- **The dynamic array** (Week 4) is inside your `Stack`, which `evaluate_postfix` and `infix_to_postfix` use (Week 6).
- **Node objects and references** (Week 5) are what `Num` and `BinOp` are.
- **The tree** and its **post-order** walk (Week 11) are the AST and `evaluate`.

- **Hash tables** (Week 13) and **graphs** (Week 14) are one step away: the symbol table and the control-flow graph of a real compiler.

Queues, searching, sorting and heaps do not appear in the calculator, and it would be dishonest to pretend otherwise: one small program cannot need everything. You could walk an AST level by level with your `CircularQueue`, as `level_order` walks a search tree, but the calculator has no reason to. What the capstone shows is not that every structure is used every time, but that a real program is **assembled** from them — and that you can now read one and name its parts.

What to carry out of the course

Week	The idea
1–2	An ADT is a contract; a structure keeps it at a **cost**. Measure, don't guess.
3	Recursion = a base case + progress. It **is** a stack.
4–5	Contiguous against linked: amortised $O(1)$ append vs $O(1)$ head.
6–7	Restrict access to one end or two, and the right algorithm appears.
8–10	An **invariant** (sorted) buys speed; sorting pays for it: $O(n \log n)$.
11–12	Trees: $O(h)$; balanced h = $\log n$. A heap **is** an array.
13–14	Hashing: $O(1)$ **average**. Graphs: BFS with a queue, DFS with a stack.
15	A grammar's shape **is** its precedence; a translator uses them all.

If you remember one sentence from each row a year from now, the course has done its job. The single idea that ran through all of them: **the same behaviour can be kept by different structures at very different costs**, and the engineer's work is to choose — with a measurement, not a guess.

This Week

Exercises: `dsa/translation.py`

Function	Target	The trap
`tokenize`	$O(n)$	numbers stay whole; `i < len(text)` in the inner loop; unknown character

Function	Target	The trap
`Parser.expr`, `term`	$O(n)$ overall	a **loop**; the old tree goes on the **left**
`Parser.factor`		consume the `)`; unary minus calls `factor`; `None` at the end
`Parser.parse`		empty input; tokens left over
`evaluate`	$O(n)$	children **first**; unknown operator
`calculate`	$O(n)$	three lines

```
pytest tests/test_translation.py -v
```

Fifty-five tests: 13 for the lexer, 12 for postfix (your Week 6 work, plus one that pairs it with `tokenize`), 14 for the parser, 4 for `evaluate`, 10 end to end with `calculate` and the two-routes test, and 2 for `to_edges`. `evaluate_postfix` should already pass; if it does not, the stack must be fixed first, because two of the end-to-end tests use it.

`Parser.__init__`, `peek`, `advance`, `Num`, `BinOp` and `to_edges` are given. The `Parser` keeps its tokens in the course `Array`; `tokenize` returns a Python list, because a list is the interface the tests use — the storage rule allows lists for data passed in and out.

Homework 15 — before the final exam

1. **Implement** `dsa/translation.py` until all 55 tests pass.
2. **Draw** the AST, by hand, of `8 / 4 / 2`, `2 * (3 + 4) - 5` and `-(1 - 2) * 3`. Then check with `draw_tree(to_edges(...))`.
3. **Trace** the calls of `expr`, `term` and `factor` on `(1 + 2) * 3`, as an indented list — one line per call, with the tokens it consumes.
4. **Break it.** Replace the `while` in `expr` with the right-recursive version of this lecture. Which tests fail, and why only those?

For item 3, a quick way to check your trace is to add a `print` with an indent of `"  " * depth` at the start of each method, and a `depth` counter that goes up on entry and down on return. Remove it before you run the tests.

For item 4, predict first: which of the test expressions contain two `+`/`-` operators at the same level, one after the other?

Preparing for the final

- **Format:** 30 marks multiple choice, 30 marks written — traces, drawings, complexity, code on paper.
- **Pass rule:** $\geq$ 60% overall **and** $\geq$ 30% of the final.
- **Every week** has a question bank with worked answers. Do them **closed**, then mark yourself.
- **Draw** every structure after every operation. The exam rewards pictures.

The written part of the final can ask any of the question types of the bank: traces, drawings of a structure after each operation, complexity with a justification, find-and-fix, and short code. Everything declared for the course is examinable; dynamic programming is not (`docs/course/01-study-plan.md`, "Two notes on scope").

A plan for the last weeks: take the mock papers already in the bank under exam conditions, in one sitting, closed book, with no computer. Mark them with the answer files. For each question you lose, go back to that week's handout, redo the question bank for that week, and write the relevant `dsa/` function again from memory on paper. What you cannot write on paper you do not yet know.

The exact date and rules of the final exam will be announced by the faculty; the course guide (`docs/course/00-course-guide.md`) has the grading rules and their sources.

Summary

Seven things to keep

1. A translator has three stages: **lexer** (text $\rightarrow$ tokens), **parser** (tokens $\rightarrow$ tree), **evaluator** or code generator (tree $\rightarrow$ value or code).
2. A grammar in EBNF: `expr`, `term`, `factor` — one rule per precedence level.
3. **Precedence is the shape of the grammar**: the lower the rule, the tighter the operator.
4. **Recursive descent**: one method per rule; `( … )*` is a loop; parentheses recurse back to `expr`.
5. Left-associative = the old tree becomes the **left** child. Left recursion never terminates; right recursion gets `1 - 2 - 3` wrong.
6. Evaluation is a **post-order** walk, and post-order of the AST **is** postfix.
7. Time is $O(n)$; recursion depth is the tree's **height**.

Next

The final exam. Everything from Week 1 to Week 15.

Then, depending on your program: **SWE141 Software Construction** (grammars and parsing, grown up), **AI3001** (algorithm design), and every course that asks you to choose a structure and defend the choice.

Sources and further reading

- **A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman.** *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, 2006 — chapter 2, "A Simple Syntax-Directed Translator", builds a translator for exactly this grammar; chapters 3–4 are lexing and parsing in depth. The "dragon book".
- **R. Nystrom.** *Crafting Interpreters*, Genever Benning, 2021 — chapters 4–7 write a scanner, a recursive-descent parser and a tree-walking evaluator in the same style as this lecture; free to read at craftinginterpreters.com.

- **P. Naur (ed.).** "Report on the Algorithmic Language ALGOL 60", *Communications of the ACM* 3(5), 1960 — the first language defined by a grammar in Backus–Naur Form.
- **N. Wirth.** *Compiler Construction*, Addison-Wesley, 1996 — a whole compiler built by recursive descent, one procedure per rule.
- **E. W. Dijkstra.** "Algol 60 translation", Mathematisch Centrum report MR 35, 1961 — the shunting-yard algorithm of Lecture 06.
- **Python documentation.** The `ast` and `dis` modules, and "Full Grammar specification" in the Python Language Reference.

Every figure in this lecture is generated by `tools/figures_l15.py`. The trees are the ones the reference parser builds; the measured figure is real measurement and will differ slightly on your machine.

Questions for Week 15

> ***The answers are at the back of the book,*** *in* Answers — Week 15. *Levels:* ***[what]*** *recall · **[how]** apply · **[why]** explain and justify. The functions are those of `dsa/translation.py`: `tokenize`, `Parser` (with `parse`, `expr`, `term`, `factor`, `peek` and `advance`), `evaluate` and `calculate`, with the tree nodes `Num(value)` and `BinOp(op, left, right)`. Unary minus `-x` is parsed as `BinOp("-", Num(0.0), x)`. `infix_to_postfix` and `evaluate_postfix` are Week 6's. Unless a question says otherwise, the grammar is*
>
> ```
> expr   := term (('+' | '-') term)*
> term   := factor (('*' | '/') factor)*
> factor := NUMBER | '(' expr ')' | '-' factor
> ```

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W15-M01 – W15-M22
B	Short answer and essay	W15-E1 – W15-E5
C	Trace the code	W15-T1 – W15-T5
D	Trees and stacks — draw every step	W15-S1 – W15-S3
E	Complexity analysis	W15-K1 – W15-K3
F	Find and fix the bug	W15-B1 – W15-B4
G	Write the code — checked by `pytest`	W15-C1 – W15-C5

Part A — Multiple choice

W15-M01 [what] The job of the **lexer** is to:

- **a)** compute the value of the expression
- **b)** group the characters of the text into tokens
- **c)** check that the parentheses are balanced
- **d)** build the syntax tree

W15-M02 [how] `tokenize("12+3")` returns a list of how many tokens?

- **a)** 4
- **b)** 2
- **c)** 3
- **d)** 5

W15-M03 [what] `calculate("3 + * 4")` fails. Which stage rejects it?

- **a)** the parser
- **b)** the lexer
- **c)** the evaluator
- **d)** none — it returns 7

W15-M04 [what] "AST" stands for:

- **a)** array of sorted tokens
- **b)** automatic syntax translator
- **c)** algebraic stack tree
- **d)** abstract syntax tree

W15-M05 [why] In the grammar, * binds more tightly than + because:

- **a)** the parser keeps a table in which * has the higher number
- **b)** * comes before + in ASCII
- **c)** * is handled in `term`, a rule that `expr` is built from
- **d)** * is always evaluated from right to left

W15-M06 [how] `calculate("2 + 3 * 4 - 1")` returns:

- **a)** 19.0
- **b)** 13.0
- **c)** 11.0
- **d)** 15.0

W15-M07 [how] The **root** of the AST of `2 * 3 + 4` is:

- **a)** +
- **b)** *
- **c)** 2
- **d)** 4

W15-M08 [how] The root of the AST of `(1 + 2) * (3 - 4)` is:

- **a)** +
- **b)** -
- **c)** (
- **d)** *

W15-M09 [why] `1 - 2 - 3` evaluates to -4 because subtraction is:

- **a)** commutative
- **b)** left-associative: `(1 - 2) - 3`
- **c)** right-associative: `1 - (2 - 3)`
- **d)** of higher precedence than itself

W15-M10 [why] A student writes `expr` right-recursively: parse a `term`; **if** the next token is `+` or `-`, return `BinOp(op, term, self.expr())`. What does it give for `8 - 4 - 2`?

- **a)** 2
- **b)** an error
- **c)** 6
- **d)** 14

W15-M11 [why] The rule `expr := expr '+' term | term` is turned directly into a method whose first line is `left = self.expr()`. On the input `3` it:

- **a)** raises `RecursionError` — it recurses without consuming a token
- **b)** returns `Num(3.0)`
- **c)** raises `ValueError: empty expression`
- **d)** loops for ever without using any memory

W15-M12 [what] In recursive descent, the EBNF repetition `( … )*` becomes:

- **a)** a recursive call
- **b)** a stack
- **c)** an `if`
- **d)** a `while` loop

W15-M13 [how] Parsing `1 + 2 * 3`, how many times is `factor` called?

- **a)** 1
- **b)** 2
- **c)** 3
- **d)** 5

W15-M14 [how] The tree our parser builds for `-5 + 2` is:

- **a)** `BinOp("-", Num(0.0), BinOp("+", Num(5.0), Num(2.0)))`
- **b)** `BinOp("+", BinOp("-", Num(0.0), Num(5.0)), Num(2.0))`
- **c)** `BinOp("+", Num(-5.0), Num(2.0))`
- **d)** `BinOp("-", Num(5.0), Num(2.0))`

W15-M15 [what] `evaluate` visits the nodes of the tree in:

- **a)** post-order — both children before the node
- **b)** pre-order
- **c)** in-order
- **d)** level-order

W15-M16 [how] The post-order sequence of the AST of `(3 + 4) * 2` is:

- **a)** `* + 3 4 2`
- **b)** `3 + 4 * 2`
- **c)** `2 3 4 + *`
- **d)** `3 4 + 2 *`

W15-M17 [why] `parse` raises `ValueError` if tokens are left over after `expr` returns. Without that check:

- **a)** `"3 +"` would return 3
- **b)** `"3 + 4 )"` would return 7
- **c)** `"(3 + 4"` would return 7
- **d)** nothing would change

W15-M18 [how] Each pair of parentheses costs the parser about three Python frames. With Python's default limit of 1,000 frames, roughly how deeply can `calculate` nest parentheses?

- **a)** 1,000
- **b)** 100
- **c)** 330
- **d)** without limit

W15-M19 [how] The time of `calculate` on an expression of n tokens is:

- **a)** $O(n)$
- **b)** $O(n \log n)$
- **c)** $O(n^2)$
- **d)** $O(\log n)$

W15-M20 [why] `calculate("1 + 1 + … + 1")` with n numbers: how deep does the recursion of `evaluate` go?

- **a)** 1 — the parser used a loop
- **b)** about $\log_2 n$
- **c)** 3 per number
- **d)** about n — the tree leans left all the way down

W15-M21 [what] When you run a `.py` file, CPython:

- **a)** interprets the text line by line, with no compilation
- **b)** compiles it to machine code, like a C compiler
- **c)** compiles it to bytecode, then interprets the bytecode
- **d)** translates it to Java

W15-M22 [why] Compared with the Week 6 route (shunting-yard, then `evaluate_postfix`), the main advantage of building a tree is that:

- **a)** it is asymptotically faster
- **b)** the tree can be reused — printed, simplified, compiled — and new grammar rules such as unary minus are easy to add
- **c)** it needs no recursion
- **d)** it needs no memory beyond the tokens

Part B — Short answer and essay

W15-E1 [what] *(4 marks)* Name the three stages of a translator, with the input and output of each. For each, give an input that it — and only it — rejects, and the exception our calculator raises. How does a compiler differ from an interpreter?

W15-E2 [why] *(4 marks)* "Precedence is the shape of the grammar." Explain this sentence using the three rules of our grammar and the tree of `3 + 4 * 2`. How do parentheses override precedence? Where would you add a rule for a tighter operator such as `^`?

W15-E3 [why] *(4 marks)* What is associativity? Explain why `expr` must use a **loop** that makes the tree so far the **left** child. Describe what goes wrong with a right-recursive `expr` and with a left-recursive one, with an input for each.

W15-E4 [why] *(3 marks)* Compare the two routes from infix tokens to a value: shunting-yard plus `evaluate_postfix`, and `Parser` plus `evaluate`. Give their costs, one advantage of each, and the connection between the postfix list and the tree.

W15-E5 [why] *(3 marks)* "In a recursive-descent parser, the call stack is the parser's stack." Explain. What decides how deep it grows, and what happens when it grows too deep?

Part C — Trace the code

W15-T1 [how] Give `tokenize("  4.5*(10-3) ")`. For each token, give the index in the text at which it starts, and say what the lexer does with the spaces.

W15-T2 [how] Trace `Parser(tokenize("2 * (3 - 1) + 4")).parse()` as an indented list of calls of `expr`, `term` and `factor`, one line per call, showing the token each call starts at. Give the tree and its value.

W15-T3 [how] For `(8 - 2) / (1 + 2) * 3`, draw the tree, then list the order in which `evaluate` **finishes** its nodes, with each node's value. What is the result?

W15-T4 [how] Evaluate `2 * (3 - 1) + 4` by the **stack** route: `infix_to_postfix`, then `evaluate_postfix`. Give the postfix list and the value stack after each token. Compare with the tree of W15-T2.

W15-T5 [how] What does `calculate` return, or raise (the exception type and the stage), for each of: `"2 * -3"`, `"--4"`, `"4 -"`, `"(2 + 3) (4)"`, `"6 / (3 - 3)"`, `" "`, `"2 ^ 3"`, `"((2)"`?

Part D — Trees and stacks: draw every step

W15-S1 [how] Draw the AST our parser builds for each expression, and give its value: (a) `4 + 2 * 3`; (b) `(4 + 2) * 3`; (c) `9 - 3 - 2`; (d) `-3 * -2`; (e) `12 / 2 / 3 * 4`. Then draw the tree a **right-recursive** `expr` would build for (c), and its value.

W15-S2 [how] `expr` parses `10 - 2 + 3 - 1`. Draw the tree held in `node` after `node = self.term()` and after **each** turn of the `while` loop. What is the final value?

W15-S3 [how] `(1 + 2) * (3 - 4)`: give its postfix list, and draw the value stack of `evaluate_postfix` after each token. Then draw the AST and number its nodes in post-order. What do you notice?

Part E — Complexity analysis

W15-K1 [how] Our `tokenize` scans with an index `i` and cuts each number out with one slice. A student instead writes the loop as `while text: ch = text[0]; …; text = text[1:]`. Give Θ of both versions for a text of n characters, and justify.

W15-K2 [why] Give the depth of the Python call stack — as a function of k — reached by (a) the **parser** and (b) `evaluate`, for three inputs: k nested pairs of parentheses around `1`; the flat chain `1 + 1 + … + 1` with k numbers; and a balanced, fully parenthesised expression with k numbers. Which of these can raise `RecursionError` for k = 1,000, and in which function?

W15-K3 [how] Two ways to write `to_postfix(node)` (W15-C1):

```
def concat(node):
    if isinstance(node, Num):
        return [number_text(node.value)]
    return concat(node.left) + concat(node.right) + [node.op]
```

and a version that appends to one shared output list during a post-order walk. Give Θ of each for a tree of n nodes, in the worst case. Which tree shape is the worst case for the first?

Part F — Find and fix the bug

W15-B1 [why]

```
def expr(self):
    node = self.term()
    if self.peek() in ("+", "-"):
        op = self.advance()
        return BinOp(op, node, self.expr())
    return node
```

W15-B2 [how]

```
def factor(self):
    token = self.peek()
    if token is None:
        raise ValueError("unexpected end of input")
    if token == "(":
        self.advance()
        node = self.expr()
        if self.peek() != ")":
            raise ValueError("missing ')'")
        return node
    if token == "-":
        self.advance()
        return BinOp("-", Num(0.0), self.factor())
```

```
    if token[0] in "0123456789.":
        self.advance()
        return Num(float(token))
    raise ValueError(f"unexpected {token!r}")
```

W15-B3 [why] Part of `factor`:

```
    if token == "-":
        self.advance()
        return BinOp("-", Num(0.0), self.expr())
```

W15-B4 [how] Part of `tokenize`:

```
        elif ch in DIGITS or ch == ".":
            start = i
            while text[i] in DIGITS or text[i] == ".":
                i += 1
            tokens.append(text[start:i])
```

Part G — Write the code

In `practice/week15.py`; check with `pytest tests/test_practice_week15.py -v`. The trees are the `Num` and `BinOp` of `dsa/translation.py`. Do not use `eval`.

W15-C1 [how] `to_postfix(node)` — the postfix token list of a tree, by a post-order walk. Why is it the same list as `infix_to_postfix` gives?

W15-C2 [how] `to_infix(node)` — fully parenthesised infix: `"(3 + (4 * 2))"`.

W15-C3 [why] `to_infix_minimal(node)` — infix with only the parentheses the grammar needs, so that the result parses back to the **same** tree. When does a child need parentheses?

W15-C4 [how] `postfix_to_tree(tokens)` — build the tree from a postfix list with a stack of subtrees; `ValueError` on malformed input.

W15-C5 [why] `calculate_with_power(tokens)` — add a right-associative `^` that binds tighter than unary minus on its left, as `**` does in Python: `2 ^ 3 ^ 2` is 512 and `-2 ^ 2` is -4. Why does `^` use recursion where `+` uses a loop?

Answers

Answers — Week 1

> ***The questions*** *are at the end of the* Week 1 *chapter. Read an answer only after you have committed to your own. For multiple choice, the explanation of why each wrong option is wrong is the part worth studying: the wrong options are the mistakes students actually make.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M07	b	M13	b	M19	c
M02	d	M08	a	M14	c	M20	a
M03	c	M09	c	M15	d	M21	d
M04	c	M10	b	M16	a	M22	b
M05	a	M11	d	M17	c	M23	c
M06	d	M12	a	M18	b	M24	d

W1-M01 — b. Turing-completeness is about *what can be computed*: the same set of problems. It says nothing about speed (a), is not the same claim as "only the syntax differs" (c) — languages differ in cost to write, read and run — and programs are certainly not portable between languages (d).

W1-M02 — d. *Dynamic*: types are checked at run time and a name can be rebound to a different type. *Strong*: Python refuses to mix incompatible types silently (see M03). "Static" (a, b) describes C, Java, Rust; "weak" (b, c) describes C's implicit conversions or JavaScript's `"3" + 5`.

W1-M03 — c. `TypeError: can only concatenate str (not "int") to str`. `"35"` (a) is JavaScript's answer — weak typing. `8` (b) would need an implicit conversion Python refuses to make. It is valid syntax, so not (d).

W1-M04 — c. Ownership and borrowing are checked at compile time. Manual `free` (a) is C/C++; a garbage collector (b) is Python, Java, Go; (d) describes no mainstream language.

W1-M05 — a. Ritchie, Bell Labs, early 1970s, grown out of Thompson's B, in order to write Unix (rewritten in C in 1973). Stroustrup (b) created C++; Knuth (c) created TeX; van Rossum (d) created Python.

W1-M06 — d. The ADT is the *contract*. Memory layout (a) is exactly what it leaves out — that belongs to the data structure. The language (b) and class hierarchy (c) are implementation choices.

W1-M07 — b. A generator expression inside `sum` states the result wanted. (a) and (d) are procedural — steps and state; (c) is functional — composing `map` and `filter`.

W1-M08 — a. TIOBE counts search-engine hits. It measures how much people write about a language, not quality (b); GitHub activity is Octoverse (c); self-reported usage is the Stack Overflow survey (d).

W1-M09 — c. Know-what is (a) and (d) — recall. Know-how is (b) — building it. Know-why is choosing and justifying by cost, which is what the exam weighs.

W1-M10 — b. Both thresholds, from the bylaws: ≥ 60% overall **and** ≥ 30% of the final. (c) is the common misreading — a strong coursework mark cannot rescue a final below 30%. And attendance below 75% bars you from the final entirely.

W1-M11 — d. `//` is *floor* division: it rounds **down**. −3.5 rounded down is −4. `-3` (a) is rounding towards zero, which is C's behaviour; `-3.5` (b) is what `/` gives.

W1-M12 — a. Python keeps `a == (a // b) * b + a % b`. Here `-7 // 3` is −3, and −3 × 3 + 2 = −7, so the remainder is 2. With a positive divisor, `%` is never negative in Python.

W1-M13 — b. `**` groups right to left: `2 ** (3 ** 2)` = 2^9 = 512. 64 (a) is `(2 ** 3) ** 2`.

W1-M14 — c. `/` always returns a `float`, even when the division is exact: `8 / 4` is `2.0`. Use `//` for an integer result.

W1-M15 — d. Indices 1, 2, 3 — up to but **not including** 4: `"yth"`. `"ytho"` (b) includes index 4.

W1-M16 — a. Step −2 from the end: indices 7, 5, 3, 1 → a, u, s, a. `"aruosnaM"` (b) is `[::-1]`; `"Mnor"` (c) is `[::2]`.

W1-M17 — c. `b = a` copies nothing: both names refer to the **same** list, so appending through b changes what a sees.

W1-M18 — b. `a[:]` makes a new list with equal contents: a different object (`is` is `False`) with the same values (`==` is `True`).

W1-M19 — c. `[[0] * 2] * 2` puts **two references to one inner list** into the outer list. Changing it through `grid[0]` shows through `grid[1]`. The correct construction is `[[0] * 2 for _ in range(2)]`.

W1-M20 — a. Python rounds exact halves to the nearest **even** number: `round(2.5)` is 2 and `round(3.5)` is 4 — total 6.

W1-M21 — d. A float is a binary fraction; 0.1 has no exact binary representation, so `0.1 + 0.2` is `0.30000000000000004`. Not a bug (a) — every language with IEEE floats does this. Compare floats with a tolerance.

W1-M22 — b. Strings are immutable. (c) is how you *should* do it — `s = "z" + s[1:]` — but item assignment itself is refused.

W1-M23 — c. A function with no `return` returns `None`. The `12` appears on the screen, but it is not *returned*. Tests see only what is returned.

W1-M24 — d. `int()` of a string accepts only an integer literal. Use `int(float("4.5"))` if truncation is really what you want.

Part B — Short answer and essay

Model answers. The **marking points** are what an examiner looks for; each is roughly one mark.

W1-E1 *(4)*

- **True part:** general-purpose languages are Turing-complete, so they can all compute the same set of functions.
- **What is wrong:** equivalence of *what* can be computed says nothing about the **cost** — to write, to read and maintain, to run — nor about which mistakes the language lets survive into production.
- A language is a **set of design decisions** with authors and reasons (language → design → philosophy).
- **Example**, any one well explained: C trusts the programmer and does no bounds checking, so `a[10]` on a 5-element array silently corrupts memory, while Python raises `IndexError`; or Python is strongly typed (`"3" + 5` is an error) while JavaScript is weak (`"35"`); or Rust's ownership versus C's manual `free`.

W1-E2 *(4)*

- **Static vs dynamic** is *when* types are checked — at compile time, or at run time.
- **Strong vs weak** is *how strictly* types are kept apart — refusing, or silently converting.
- Python is **dynamic and strong**; JavaScript is **dynamic and weak**.
- `"3" + 5`: Python raises `TypeError`; JavaScript returns `"35"`. Neither is a bug — they are different decisions about loud failure versus convenient conversion.

W1-E3 *(3)*

- **Manual** (C, C++): the programmer allocates and frees — fast, and the source of many security bugs. **Garbage collected** (Python, Java, Go): the runtime frees unreachable objects — safe, less control. **Ownership** (Rust): the compiler proves each allocation is freed exactly once — safe and fast, harder to satisfy.
- Python uses **garbage collection** (reference counting plus a cycle collector).
- Consequence: removing a node from a linked list is just "stop referring to it" — no `free`, no visible pointer — so the structure is invisible unless you use the debugger to watch references change.

W1-E4 *(4)*

- **ADT**: the contract — operations, their meaning, their cost — with nothing about storage.
- **Data structure**: a concrete arrangement of data in memory that fulfils the contract.
- **Stack ADT**: `push` adds, `pop` removes the most recent, `peek` looks; all O(1).
- **Two structures**: a dynamic array with the top at the end; a linked list with the top at the head. Both honest stacks; they differ in memory overhead, cache behaviour and whether one operation can occasionally be slow. Choosing between them is the engineering.

W1-E5 *(3)*

- **For**, any two: the field (research, data, AI/ML) runs on Python; it binds to fast C/C++/CUDA underneath; it reads close to pseudocode, so the algorithm survives onto the page; students meet it again in every program.

- **Against:** Python already has the structures as built-ins — `list`, `dict`, `deque`, `sorted`, `heapq` — which hides exactly what the course studies.
- **Remedy:** the course reimplements them, and (from Lecture 02) structures may store data only in the course `Array`, in nodes, or in structures students built — never in `list`, `dict` or `set`.

W1-E6 *(3)*

- `a = [1, 2, 3]` creates a list object and binds the name `a` to it; `b = a` binds a **second name to the same object** — no copy.
- Diagram: two names, `a` and `b`, with arrows to one list object.
- Consequence: `b.append(4)` changes what `a` shows. To get an independent list, copy it: `a[:]`, `a.copy()` or `list(a)` — and that copy is **shallow**.

Part C — Trace the code

W1-T1

```
3 2 -4 3
(5, 3) 1024 3.5
```

`-17 // 5` is −3.4 rounded **down**, −4; and −4 × 5 + 3 = −17, so `-17 % 5` is 3.

W1-T2

```
D s Stru DATA
['Data', 'Structures'] 15 3
```

`s[5:9]` is indices 5, 6, 7, 8 — `S t r u`. `split()` with no argument splits on runs of whitespace. The three `t`s: one in *Data*, two in *Structures*.

W1-T3

```
[99, 2, 3, 4] [99, 2, 3, 4] [1, 2, 3, 5]
[99, 2, 3, 4, 6]
```

`a` and `b` are one list, so `append` and `b[0] = 99` both show in both. `c` was a copy taken *before* those changes; `c + [5]` builds a new list. `a += [6]` extends the list **in place**, so `b` sees it — unlike `a = a + [6]`, which would have made a new list for `a` only.

W1-T4

```
1 6
```

1000 → 333 → 111 → 37 → 12 → 4 → 1: six divisions by 3. Dividing by 3 until 1 takes about $\log_3 n$ steps — the Week 2 idea of logarithmic cost.

W1-T5

```
  Omar|87.5|   87.46|004
```

`>6` right-aligns in 6 characters; `.1f` rounds to one decimal; `8.2f` is 8 characters wide with two decimals; `03d` pads the integer to 3 digits with zeros.

W1-T6

```
[0, 1, 1, 2, 3, 5, 8, 13]
[5, 8, 13] [0, 2, 8]
```

`[::3]` takes indices 0, 3, 6.

Part D — Find and fix the bug

W1-B1. `split(" ")` splits on **each single space**, so two spaces in a row, or a leading or trailing space, produce empty strings `""`, and `""[0]` raises `IndexError`. Input that fails: `"Ada Lovelace"` or `" Ada"`. **Fix:** use `split()` with no argument, which splits on runs of whitespace and drops empty pieces.

W1-B2. `i` is never incremented, so the loop never ends for any non-empty list — the program hangs (Ctrl+C to stop it). **Fix:** add `i += 1` inside the loop — or, better, `for v in values: total += v`, which cannot forget.

W1-B3. `result = values` does **not** copy; append then changes the caller's list, which the specification forbids. Test: after `with_item(original, 9)`, `original` has changed. **Fix:** `result = values[:]` (or `values.copy()`, or simply `return values + [item]`).

Part E — Write the code

Worked solutions. Yours may differ and still be correct — the tests decide.

W1-C1

```
def count_vowels(text):
    count = 0
    for ch in text.lower():
        if ch in "aeiou":
            count += 1
    return count
```

W1-C2 — one pass: keep the largest and the second largest seen so far.

```
def second_largest(values):
    largest = second = None
    for v in values:
```

```
        if largest is None or v > largest:
            if largest is not None:
                second = largest          # the old maximum is now second
            largest = v
        elif v != largest and (second is None or v > second):
            second = v
    if second is None:
        raise ValueError("need at least two distinct values")
    return second
```

The `v != largest` check is what makes `[9, 9, 7]` give 7, not 9. Sorting would work too but is O(n log n); this is O(n).

W1-C3 — find where each run ends.

```
def compress(text):
    result = ""
    i = 0
    while i < len(text):
        j = i
        while j < len(text) and text[j] == text[i]:
            j += 1                        # j stops at the end of the run
        result += text[i] + str(j - i)
        i = j
    return result
```

Each character is looked at a constant number of times, so this is O(n) steps (ignoring the cost of building the string).

W1-C4

```
def is_anagram(first, second):
    a = sorted(first.replace(" ", "").lower())
    b = sorted(second.replace(" ", "").lower())
    return a == b
```

O(n log n) because of sorting. Counting letters in a dictionary (Lab 03) makes it O(n).

Answers — Week 2

> ***The questions*** *are at the end of the* Week 2 *chapter. Commit to your own answer before reading one here.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M08	a	M15	d	M22	c
M02	c	M09	b	M16	b	M23	d
M03	a	M10	c	M17	a	M24	a
M04	d	M11	d	M18	c	M25	b
M05	a	M12	a	M19	d	M26	c
M06	c	M13	b	M20	a		
M07	d	M14	c	M21	b		

W2-M01 — b. Keep the fastest-growing term, drop its constant: $3n^2 \to n^2$. It is also $O(n^3)$, but not $\Theta(n^3)$ (c) — Θ is a tight bound.

W2-M02 — c. $n \le 1 \cdot n^2$ for all $n \ge 1$. O is an *upper* bound, so a slower-growing function is O of a faster one. (a) and (d) put a faster function under a slower one; (b) claims n grows at least as fast as n^2.

W2-M03 — a. Constants **you choose**, and only from some n_0 on. (b) has no constant and no n_0; (c) is equality, not a bound; (d) is the definition of Ω.

W2-M04 — d. Worst case: the target is last or absent, so every element is compared once.

W2-M05 — a. $(1 + 2 + \cdots + n)/n = (n + 1)/2$. Still $O(n)$ — half is a constant.

W2-M06 — c. Halving until 1 takes about $\log_2 n$ steps. $\Theta(n/2)$ (d) would be subtracting, not dividing — and it is just $\Theta(n)$ anyway.

W2-M07 — d. $0 + 1 + \cdots + (n - 1) = n(n - 1)/2$. The answer must be a function of n only, so (c) is not an answer.

W2-M08 — a. The inner loop is a constant 5: $5n = \Theta(n)$. Nested loops multiply their *iteration counts*, not their nesting depth.

W2-M09 — b. A list is an array; `in` checks items one by one. Sets and dicts are the hash tables (a).

W2-M10 — c. append writes at the end — nothing moves — and the occasional resize averages out (Week 4). The other three shift or search: $O(n)$.

W2-M11 — d. Address arithmetic. It has nothing to do with order (a) or caching (b).

W2-M12 — a. Slots 2, 3, 4, 5 move right: size − index = 6 − 2 = 4.

W2-M13 — b. Moving right, start at the right, or you overwrite values before you move them (see W2-S2).

W2-M14 — c. The course `Array` has no negative indices, like C.

W2-M15 — d. `len` is the capacity, fixed at creation. Tracking how many slots are in use is your job.

W2-M16 — b. $r \times \text{cols} + c = 2 \times 4 + 1 = 9$.

W2-M17 — a. Two indices and one temporary for the swap, whatever n is.

W2-M18 — c. A structure may not store its data in a Python `list`. (a) and (b) build on the student's own structures — allowed; (d) is a list used as a return value — the interface, allowed.

W2-M19 — d. A block of references plus a size and a capacity; it grows by copying into a bigger block.

W2-M20 — a. Sequence adds: $O(n + n^2) = O(n^2)$.

W2-M21 — b. Each iteration hides **two** $O(n)$ operations: the slice copies up to n items and `in` searches them. n iterations of $O(n)$ work.

W2-M22 — c. Think of it as "no break".

W2-M23 — d. The default list is created **once**, when `def` runs, and shared by every call that uses it.

W2-M24 — a. `and` binds tighter: `0 or ('x' and '')`. `'x' and ''` gives `''` (the last value looked at); `0 or ''` gives `''`.

W2-M25 — b. Start 2, step 3, stop **before** 11: 2, 5, 8.

W2-M26 — c. `return 1` is decided, then `finally` runs (printing `bye`) before the function actually returns; then `print` shows `1`.

Part B — Short answer and essay

W2-E1 *(4)*

- **Lower-order terms** stop mattering as n grows: in $\frac{1}{2}n^2 - \frac{1}{2}n$, the n^2 term is 99.9% of the total by n = 1,000.
- **Constant factors** are not properties of the algorithm: they change with the machine, the language and how you count steps. What stays the same everywhere is the **shape** of growth.
- **Misleading when n is small**, or constants are huge: a $1000n$ algorithm is worse than an n^2 one for every n < 1,000; linear search beats binary search on tiny arrays.

- **Measuring** (`viz.complexity.measure`) shows the constants and the crossover point that Big-O hides; counting shows the shape that one machine's timings cannot prove. You need both.

W2-E2 *(4)*

- $f = O(g)$: there exist $c > 0$, n_0 with $f(n) \leq c\,g(n)$ for all $n \geq n_0$ — an upper bound.
- $f = \Omega(g)$: there exist $c > 0$, n_0 with $f(n) \geq c\,g(n)$ for all $n \geq n_0$ — a lower bound.
- $f = \Theta(g)$: both — $c_1 g(n) \leq f(n) \leq c_2 g(n)$ eventually — a tight bound.
- For $3n^2 + 2n$: it is $O(n^2)$, $\Omega(n^2)$, $\Theta(n^2)$; it is also $\Omega(n)$. **True but uninformative:** $3n^2 + 2n = O(n^3)$ (or $O(2^n)$).

W2-E3 *(3)*

- **Best:** the target is the first element — 1 comparison, $O(1)$.
- **Worst:** last or absent — n comparisons, $O(n)$.
- **Average** (present, uniformly placed): $(n + 1)/2$, still $O(n)$ — and the average needs a stated assumption about the inputs.
- "Linear search is O(n)" means the **worst case**: it is a guarantee that holds for every input of size n, needs no assumption, and is what you plan capacity around.

W2-E4 *(4)*

- An array is contiguous, equal-sized slots, so the address of `a[i]` is base $+\ i \times$ size: one multiplication and one addition, independent of n → $O(1)$.
- Inserting at index i in an array of size n needs every element from i to the end moved one place right: $n - i$ moves, n in the worst case → $O(n)$.
- **Fixed size:** the memory after the block may belong to something else, so the array cannot grow in place. When it is full, you must allocate a **new, larger** array and **copy** every element across: $O(n)$ for that operation.
- (Doubling the capacity each time makes appends $O(1)$ amortised — Week 4.)

W2-E5 *(4)*

- **Rule:** a data structure in `dsa/` stores its data only in the course `Array`, in node objects, or in another structure the student has built — never in a Python `list`, `dict`, `set` or `deque`.
- **Reason:** those built-ins hide exactly the costs the course teaches — one short call like `insert(0, x)`, `pop(0)` or `x in values` is an $O(n)$ algorithm someone else wrote. Building on a bare array makes every cost visible and yours.
- **Still allowed:** lists as function inputs and return values; lists in tests; plain local variables and tuples.
- **Examples**, any two: `Stack` on the student's `DynamicArray`; `CircularQueue` on an `Array`; `ChainingHashMap` as an `Array` of `Entry` chains; `Graph` on the student's hash map and `DynamicArrays`; `MinHeap` on a `DynamicArray`.

W2-E6 *(3)*

	Grows?	Holds	Hides the cost?
`list`	yes (dynamic array)	references to any objects	yes

	Grows?	Holds	Hides the cost?
`array.array`	yes	packed numbers of one type	yes
`numpy.ndarray`	no	packed numbers of one type	yes — whole-array operations run in C
`ctypes` array	no	C values or references	no — index and assign only

A `ctypes` array of object references is a genuine fixed-size C array that can hold any Python object. The course wraps it, adds bounds checking, and offers nothing else — so every other operation must be written, and priced, by the student.

Part C — Complexity analysis

W2-K1 — $\Theta(n^2)$. The nested loops do $n \times n$ iterations of O(1) work; the following loop adds n. Sequence adds: $n^2 + n = \Theta(n^2)$.

W2-K2 — $\Theta(\log n)$. i takes the values 1, 2, 4, 8, ...; it reaches n after about $\log_2 n$ doublings. Doubling up to n is the mirror of halving down from n.

W2-K3 — $\Theta(n \log n)$. The outer loop runs n times; each time the inner loop halves j from n down to 1, about $\log_2 n$ iterations. Loops multiply. (Counting: for n = 1,024 the inner body runs exactly 10,240 times.)

W2-K4 — best $\Theta(1)$, worst $\Theta(n^2)$. Best: `arr[0] == arr[1]`, found on the first comparison. Worst: no duplicates, so every pair is compared: $n(n-1)/2$ comparisons.

W2-K5 — $\Theta(n^2)$. The loop body looks like one step, but `values[i:]` **copies** $n - i$ items. Total copying: $n + (n-1) + \cdots + 1 = n(n+1)/2$.

W2-K6 — $\Theta(n^2)$. The middle loop is a constant 100. Total $100 \times (0 + 1 + \cdots + (n-1)) = 50\,n(n-1)$. A large constant is still a constant. (For n = 10, `count` is 4,500.)

Part D — Proofs

W2-P1. Choose $c = 3$, $n_0 = 10$. For every $n \geq 10$, $10 \leq n$, so $2n + 10 \leq 2n + n = 3n$. Hence $2n + 10 \leq 3 \cdot n$ for all $n \geq 10$, and $2n + 10 = O(n)$. ■ *(Other valid choices exist, e.g. $c = 12$, $n_0 = 1$.)*

W2-P2. Suppose, for contradiction, that $n^2 = O(n)$: there are $c > 0$ and n_0 with $n^2 \leq c\,n$ for all $n \geq n_0$. Dividing by $n > 0$ gives $n \leq c$ for all $n \geq n_0$. But take $n = \max(n_0, \lceil c \rceil + 1)$: then $n \geq n_0$ and $n > c$ — a contradiction. So no such c exists, and n^2 is not $O(n)$. ■

W2-P3. Need $c_1, c_2 > 0$ and n_0 with $c_1 n^2 \leq 4n^2 + n \leq c_2 n^2$ for all $n \geq n_0$.

- **Lower bound:** $4n^2 + n \geq 4n^2$ for all $n \geq 0$, so $c_1 = 4$ works: $4n^2 + n = \Omega(n^2)$.
- **Upper bound:** for $n \geq 1$, $n \leq n^2$, so $4n^2 + n \leq 5n^2$; $c_2 = 5$: $4n^2 + n = O(n^2)$.
- With $n_0 = 1$ both hold, so $4n^2 + n = \Theta(n^2)$. ■

Part E — Array state

W2-S1.

(a) Shift 50, 40, 30, 20 one place right (4 moves, starting from 50), then write 15 at index 1:

```
[10, 15, 20, 30, 40, 50, _]      size 6
```

(b) Remove index 3 (the value 30): shift 40 and 50 one place left (2 moves), then clear the slot that is no longer used:

```
[10, 15, 20, 40, 50, _, _]       size 5, returned 30
```

W2-S2.

```
start           [a, b, c, d, _]
i = 1: arr[2] = arr[1]   ->  [a, b, b, d, _]     c is overwritten before it
 ↪  moved
i = 2: arr[3] = arr[2]   ->  [a, b, b, b, _]
i = 3: arr[4] = arr[3]   ->  [a, b, b, b, b]
arr[1] = "X"             ->  [a, X, b, b, b]
```

Copying forwards overwrites each value before it has been moved, so one value (b) is smeared across the rest and c and d are lost. **Fix:** move from the end backwards:

```
for i in range(size, index, -1):
    arr[i] = arr[i - 1]
arr[index] = value
```

W2-S3.

```
start                           [1, 2, 3, 4, 5, 6]
reverse the first 2 (0..1)      [2, 1, 3, 4, 5, 6]
reverse the rest (2..5)         [2, 1, 6, 5, 4, 3]
reverse the whole (0..5)        [3, 4, 5, 6, 1, 2]
```

Each reversal is a two-pointer swap: O(n) time in total, O(1) extra space.

W2-S4.

(a) Value **7**, at flat index $1 \times 4 + 3 = 7$ (the values are the indices, because they were stored row by row).
(b) Row $10 // 4 = 2$, column 10 mod $4 = 2$: **(2, 2)**.
(c) Transposing moves (1, 3) to (3, 1) in a 4 × 3 matrix: flat index $3 ×3 + 1 = $ **10**.

Part F — Trace the code

W2-T1

```
11 13 done
```

The `else` belongs to the inner `for`: it prints n only when no divisor was found, i.e. n is prime. 10, 12, 14 and 15 hit `break`.

W2-T2

```
5 -4 0 3 ?
```

`"sum"` matches `["sum", *rest]` with `rest = []`, so it returns 0. `"add 1"` has only two words, so it fails the `add` pattern (three words) and falls to `_`.

W2-T3

```
A B D E 5
A C E 0
```

`finally` always runs, **before** the returned value reaches `print`. With 0, the division raises, so `B` is skipped and `except` prints `C`; `else` runs only when the `try` did not raise.

W2-T4

```
[1, 2] 2 20
```

`values.append(n)` mutates the caller's list. `n = n * 10` rebinds a local name — k is untouched. `values = [0]` rebinds the local name too — `data` keeps `[1, 2]`.

Part G — Find and fix the bug

W2-B1. `range(2, int(n ** 0.5))` stops **before** the square root, so a perfect square of a prime is never tested against its root: `is_prime(9)`, `is_prime(25)` and `is_prime(49)` return `True`. **Fix:** `range(2, int(n ** 0.5) + 1)` — or, avoiding floats, `d = 2; while d * d <= n:`.

W2-B2. The `else: return -1` is inside the loop, so the function gives up after checking **only** `arr[0]`. `find([4, 7], 7)` returns −1. **Fix:** move `return -1` after the loop, and delete the `else`.

W2-B3. The loop runs one step too far: for `i = size - 1` it reads `arr[size]`, one past the used part — an `IndexError` when the array is full (`size == len(arr)`), and a silent read of a free slot otherwise. The slot that is no longer used is also never cleared. **Fix:**

```
for i in range(index, size - 1):
    arr[i] = arr[i + 1]
arr[size - 1] = None
```

Part H — Write the code

W2-C1

```
def count_occurrences(arr, target, size=None):
    n = len(arr) if size is None else size
    count = 0
    for i in range(n):
        if arr[i] == target:
            count += 1
    return count
```

W2-C2 — a *read* index and a *write* index.

```
def remove_all(arr, size, target):
    write = 0                              # next slot to keep a value in
    for read in range(size):
        if arr[read] != target:
            arr[write] = arr[read]
            write += 1
    for i in range(write, size):           # clear the freed tail
        arr[i] = None
    return write
```

Each element is read once and written at most once: O(n), O(1) extra. Calling `remove_at` for each occurrence shifts up to n elements **per removal**; with k occurrences that is O(kn), which is $O(n^2)$ when k is proportional to n.

W2-C3 — two indices moving towards each other.

```
def two_sum_sorted(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        total = arr[lo] + arr[hi]
        if total == target:
            return (lo, hi)
        if total < target:
            lo += 1                        # need a bigger sum
        else:
            hi -= 1                        # need a smaller sum
    return None
```

Why it is correct: if `arr[lo] + arr[hi]` is too small, then `arr[lo]` plus *any* element at or before `hi` is also too small (the array is sorted), so `arr[lo]` can be in no solution — discard it. Symmetrically for too big. Each step discards one element, so at most n − 1 steps: O(n).

W2-C4

```
def prefix_sums(arr):
    result = Array(len(arr))
    running = 0
    for i in range(len(arr)):
        running += arr[i]
        result[i] = running
    return result
```

Then sum$(arr[i..j]) = p[j] - p[i-1]$ (or $p[j]$ when $i = 0$): one subtraction, O(1), after one O(n) pass. Paying once up front to answer many questions cheaply is a trade you will see again and again.

Answers — Week 3

> ***The questions*** *are at the end of the* Week 3 *chapter. Commit to your own answer before reading one here.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	a	M08	c	M15	d	M22	d
M02	d	M09	b	M16	c	M23	b
M03	b	M10	a	M17	a	M24	c
M04	c	M11	d	M18	b	M25	b
M05	a	M12	c	M19	d	M26	d
M06	d	M13	b	M20	c		
M07	b	M14	a	M21	a		

W3-M01 — a. A base case that answers directly, and progress towards it. A loop (b) is not needed; one recursive call (c) is enough; a global counter (d) is a debugging aid, not a requirement.

W3-M02 — d. −1, −2, −3, … moves *away* from the base case 0, so the calls never stop until Python's depth limit. `ValueError` (c) is what the exercise version should raise — by checking `n < 0` explicitly.

W3-M03 — b. `sys.getrecursionlimit()` is 1,000 by default.

W3-M04 — c. n + 1 frames are on the stack at the deepest point (n down to 0). Space is decided by the depth, not by the answer's size (d).

W3-M05 — a. Unroll: c is added once per halving, $\log_2 n$ times.

W3-M06 — d. The number of calls doubles at every level: $1 + 2 + 4 + \cdots + 2^{n-1}$.

W3-M07 — b. $cn + c(n-1) + \cdots + c = c\,n(n+1)/2$.

W3-M08 — c. $2^5 - 1 = 31$.

W3-M09 — b. Each item is either in or out: $2^4 = 16$. (24 (c) is 4!, the number of permutations.)

W3-M10 — a. $4! = 4 \times 3 \times 2 \times 1 = 24$.

W3-M11 — d. fib(5) calls fib(4) and fib(3), and so on: the recursion tree has 15 nodes. (In general $2\,\text{fib}(n+1) - 1$.) 8 (b) is the *value* fib(6).

W3-M12 — c. Two calls on n/2: $T(n) = 2T(n/2) + c = O(n)$. Compute the half power **once**, store it, and square the variable: $O(\log n)$.

W3-M13 — b. Each call copies the rest of the list: $(n-1) + (n-2) + \cdots = O(n^2)$ — the hidden loop from Lecture 02.

W3-M14 — a. Solve each subproblem once, store its answer (for Fibonacci, in an `Array` indexed by n), and look it up afterwards.

W3-M15 — d. Frames are popped as calls return; the deepest chain is hanoi(20) → hanoi(19) → … → hanoi(1): about 20 frames. The *total* number of calls is exponential; the *depth* is linear.

W3-M16 — c. Each call prints n on the way **down**, then again on the way **back up**, after the inner call has returned: `3 2 1` then `1 2 3`.

W3-M17 — a. The empty set has exactly one subset — itself. Returning `[]` (b) means "no subsets", and every result built from it is then empty too.

W3-M18 — b. Python deliberately keeps every frame (it keeps tracebacks complete). Deep recursion should become a loop.

W3-M19 — d. `sort()` sorts in place and returns `None`. `sorted(marks)` returns the new list.

W3-M20 — c. `{}` is an empty dictionary; an empty set is `set()`.

W3-M21 — a. A set is a hash table: average $O(1)$ membership. A list would be $O(n)$.

W3-M22 — d. `tricks` is a **class** variable: one list shared by every instance. Create per-object data in `__init__`: `self.tricks = []`.

W3-M23 — b. The body only starts running when `next()` is called — by a `for` loop, `list()` and so on.

W3-M24 — c. Every remaining item shifts one place left. Use `collections.deque` in plain Python — or, in this course, your own `CircularQueue` (Week 7).

W3-M25 — b. `get(w, 0)` supplies 0 for a missing key, so no `KeyError`.

W3-M26 — d. Each call spawns two subtrees almost as large as itself; the call count grows by a factor of about the golden ratio, 1.618, per step of n.

Part B — Short answer and essay

W3-E1 *(4)* — writing f for `factorial`, the stack at each step, bottom → top (Lecture 03's figure draws the same thing for `factorial(4)`):

Step	Stack	What happens
1	f(3)	calls f(2)
2	f(3) f(2)	calls f(1)
3	f(3) f(2) f(1)	calls f(0)
4	f(3) f(2) f(1) f(0)	**base case**: f(0) returns 1

Step	Stack	What happens
5	f(3) f(2) f(1)	1 × 1 = 1; f(1) returns 1
6	f(3) f(2)	2 × 1 = 2; f(2) returns 2
7	f(3)	3 × 2 = 6; f(3) returns 6

Marking points: each call pushes a frame with its own n; nothing is multiplied during winding; the base case stops the growth; each return pops a frame and the frame below finishes its multiplication; maximum depth 4 = n + 1.

W3-E2 *(4)*

- `fib(n)` calls `fib(n-1)` **and** `fib(n-2)`; the recursion tree roughly doubles with each increase in n, because each node has two children almost as large as itself.
- The same subproblems are recomputed many times: in fib(5), fib(2) is computed 3 times, fib(3) twice. Total calls $2\,\text{fib}(n+1) - 1$ — exponential, about 1.618^n; fib(30) makes 2,692,537 calls.
- **Memoisation:** store each fib(k) in a memo `Array` the first time it is computed; later calls look it up. Each value is computed once.
- New cost: $O(n)$ time (2n − 1 calls — 59 for n = 30), $O(n)$ space for the memo plus $O(n)$ stack depth.

W3-E3 *(3)*

- **Recursion** when the data or the problem is recursive: trees, nested lists, grammars; divide and conquer (binary search, merge sort); branching searches (subsets, Hanoi, backtracking) — where it is clearer and the depth is small.
- **A loop** when the recursion would be one call per element of flat data, or the depth could be large.
- **Python concerns**, any two: the ~1,000-frame recursion limit (`RecursionError`); no tail-call optimisation, so every call costs a frame; a function call is relatively slow in CPython; slicing inside recursion silently adds $O(n)$ per call.

W3-E4 *(4)*

- **Strategy** for n disks from A to C: move n − 1 disks A → B (recursively, using C as spare); move the largest disk A → C; move the n − 1 disks B → C (recursively, using A as spare). Base case: one disk, one move.
- **Recurrence:** $T(1) = 1, T(n) = 2T(n-1) + 1$.
- **Solution:** $T(n) = 2^n - 1$ (unroll: 1, 3, 7, 15, ...; or the induction proof in W3-R5).
- **Minimum:** the largest disk must move at least once. At that moment all the other n − 1 disks must be on the spare peg — which takes at least $T(n-1)$ moves — and afterwards they must all be moved back on top, at least $T(n-1)$ more. So any solution needs at least $2T(n-1) + 1$ moves: the exponential cost is in the problem, not the algorithm.

W3-E5 *(3)*

- When writing the recursive case, **assume** the recursive call correctly solves the smaller problem, and only ask how to build this answer from that one. Do not trace every call.

- This is **induction**: the base case is the induction base; "if the smaller call is right then this call is right" is the inductive step. Together they prove the function correct for every input the measure reaches.
- It only works if the base case is right and the recursive case really makes progress towards it.

W3-E6 *(4)*

- (a) **list** — ordered, grows by append in amortised $O(1)$, allows duplicates (two students can have the same mark).
- (b) **set** — "have I seen this ID?" is $O(1)$ on average against $O(n)$ for a list; duplicates are exactly what it prevents.
- (c) **dict** — ID → name, $O(1)$ average lookup by key.
- (d) **tuple** — a fixed record of related values of different types; immutable, therefore hashable, so it can be a dictionary key or set member (a list cannot).

Part C — Recurrences

W3-R1. One call on a number with one digit fewer, O(1) work: $T(1) = c, T(n) = T(n-1) + c$ → $T(n) = O(n)$ in the number of digits, i.e. $O(\log x)$ in the value x.

W3-R2. Two calls on n – 1: $T(1) = c, T(n) = 2T(n-1) + c$ → $O(2^n)$. It returns 2^{n-1} (for n ≥ 1): 1, 2, 4, 8, … $O(n)$ **version** — call once, double the result:

```
def mystery(n):
    if n <= 1:
        return 1
    return 2 * mystery(n - 1)
```

$O(1)$ **version** (arithmetic on small numbers counted as O(1)): `return 2 ** (n - 1)` for n ≥ 1, else 1. Recognising what a recursion *computes* can remove it entirely.

W3-R3. Two calls on halves, O(1) work: $T(1) = c, T(n) = 2T(n/2) + c$. The recursion tree has 1 + 2 + 4 + … + n ≈ 2n nodes, each O(1): $O(n)$ — the same as a loop, not $O(n \log n)$ (that needs $O(n)$ work per call, as in merge sort). The stack depth is the height of the tree: $\log_2 n$, so $O(\log n)$ space — much less than a one-element-at-a-time recursion, which would need depth n.

W3-R4. `text[1:]` copies $n - 1$ characters and the concatenation builds a string of length n: $O(n)$ work outside one call on $n - 1$. $T(0) = c, T(n) = T(n-1) + cn$ → $O(n^2)$.

W3-R5. Claim: $T(n) = 2^n - 1$ for all $n \geq 1$.

- **Base:** $T(1) = 1 = 2^1 - 1$, true.
- **Step:** assume $T(k) = 2^k - 1$ for some $k \geq 1$. Then $T(k+1) = 2T(k) + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 1$, as required.
- By induction the claim holds for every $n \geq 1$. ■

Part D — Trace the code and the call stack

W3-T1

```
19 7
```

g adds the digits: 4 + 0 + 9 + 6 = 19. `g(7)` is the base case.

W3-T2

```
1101 1 1000
```

The binary representations of 13, 1 and 8. The leading digits are built by the inner calls; each call appends its own last bit on the way back.

W3-T3

Call	a % b	Returns
gcd(48, 18)	48 % 18 = 12	6
gcd(18, 12)	18 % 12 = 6	6
gcd(12, 6)	12 % 6 = 0	6
gcd(6, 0)	base case	6

Maximum depth: **4** frames. Every frame returns the same value unchanged — a tail call, which is why gcd converts to a loop so easily.

W3-T4

```
8 25
```

fib(6) = 8; the call count is $2\,\text{fib}(7) - 1 = 2 \times 13 - 1 = 25$.

W3-T5. `hanoi(2, "A", "C", "B")`: **A→B, A→C, B→C**. The moves of `hanoi(3, "A", "C", "B")` are A→C, A→B, C→B, **A→C**, B→A, B→C, A→C: the 4th move is **A→C** — the largest disk, exactly in the middle.

W3-T6

```
[2, 1, 0, 1, 2] 11
```

Each level adds n on both sides of the inner result: length $2n + 1$, so 11 for n = 5.

W3-T7

```
{'the': 2, 'cat': 1, 'hat': 1}
the {3}
```

Dictionaries keep insertion order. Sorting the keys by negative count puts `the` first. The set comprehension removes duplicate lengths: all four words have 3 letters.

W3-T8

```
[3, 2, 1] 6 3
```

`__iter__` is a generator, and each `for`/`list()`/`sum()` calls it again, so the object can be iterated any number of times.

Part E — Find and fix the bug

W3-B1. For even n it makes **two** identical calls on n/2: $T(n) = 2T(n/2) + c$, which is $O(n)$ — the halving saves nothing. **Fix:** call once and reuse the result:

```python
def power(x, n):
    if n == 0:
        return 1
    half = power(x, n // 2)
    result = half * half
    if n % 2 == 1:
        result = result * x
    return result
```

Now $T(n) = T(n/2) + c = O(\log n)$.

W3-B2. The base case returns `[]` — "there are no subsets" — so `rest` is always empty and so is every result: `subsets([1, 2])` returns `[]`. **Fix:** `return [[]]` — the empty set has exactly one subset, the empty one.

W3-B3. Strings have `__iter__`, so `"ab"` is treated as a nested list; every character is itself a one-character string, which again has `__iter__`, and the function recurses into `"a"` for ever: `RecursionError`. **Fix:** descend only into lists: `if isinstance(item, list):`.

W3-B4. The recursive case computes the sum but does not **return** it, so every call except the base case returns `None`. `total([5])` gives `None`; `total([1, 2])` raises `TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'`, one level above the base case. **Fix:** `return values[i] + total(values, i + 1)`.

Part F — Write the code

W3-C1

```python
def array_sum(arr, i=0):
    if i >= len(arr):
        return 0                          # nothing left
    return arr[i] + array_sum(arr, i + 1)
```

W3-C2

```
def count_char(text, ch):
    if text == "":
        return 0
    return (1 if text[0] == ch else 0) + count_char(text[1:], ch)
```

Correct, but $O(n^2)$ because of the slice — pass an index to make it $O(n)$.

W3-C3

```
def is_sorted_rec(arr, i=0):
    if i >= len(arr) - 1:
        return True                          # zero or one element left
    if arr[i] > arr[i + 1]:
        return False
    return is_sorted_rec(arr, i + 1)
```

W3-C4

```
def no_consecutive_ones(n):
    def build(prefix, remaining):
        if remaining == 0:
            return [prefix]
        result = build(prefix + "0", remaining - 1)
        if not prefix.endswith("1"):
            result = result + build(prefix + "1", remaining - 1)
        return result
    return build("", n)
```

Counts for n = 1, 2, 3, 4, 5: **2, 3, 5, 8, 13** — the Fibonacci numbers. A valid string of length n either starts with `0` followed by any valid string of length n − 1, or starts with `10` followed by any valid string of length n − 2, so count(n) = count$(n-1)$ + count$(n-2)$.

W3-C5

```
def pascal_row(n):
    if n == 0:
        return [1]
    previous = pascal_row(n - 1)
    middle = [previous[k - 1] + previous[k] for k in range(1,
↪   len(previous))]
    return [1] + middle + [1]
```

Each entry is the sum of the two above it. One recursive call per row: depth n, and $O(n^2)$ additions in total.

Answers — Week 4

> ***The questions*** *are at the end of the* Week 4 *chapter. Commit to your own answer before reading one here.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	a	M07	c	M13	a	M19	c
M02	c	M08	d	M14	b	M20	d
M03	b	M09	a	M15	c	M21	a
M04	d	M10	b	M16	d	M22	b
M05	a	M11	c	M17	a		
M06	b	M12	d	M18	b		

W4-M01 — a. `len` is the number of elements the user stored. The capacity (b) is internal; showing it would expose the implementation.

W4-M02 — c. Capacity 1 → 2 (2nd append) → 4 (3rd) → 8 (5th).

W4-M03 — b. Resizes happen when the size reaches 1, 2, 4, …, 512 — ten powers of two below 1,000 — so exactly 10 ($2^{10} = 1024$). 999 (d) is growing by one.

W4-M04 — d. Every append copies everything already there: $0+1+\cdots+(n-1) = n(n-1)/2$.

W4-M05 — a. Copies are 100, 200, 300, …: about $n^2/200$ in total — still quadratic, so $O(n)$ per append on average. A constant step only changes the constant. (d) confuses the step size with the cost.

W4-M06 — b. Fewer than 3n units of work for n appends: under 3 each.

W4-M07 — c. The append that finds the array full copies all n elements. The amortised bound does not change that single worst case.

W4-M08 — d. Amortised analysis bounds the total over **any** sequence — a guarantee. Average case (c) averages over an assumed distribution of inputs.

W4-M09 — a. $1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1 < 2n$, since $2^k < n$.

W4-M10 — b. Appends that find the array full are those made when the size is a power of two: the 2nd, 3rd, 5th, 9th, 17th, 33rd, … (one more than a power of two).

W4-M11 — c. A gap between the grow threshold (full) and the shrink threshold (one quarter) means many cheap operations must happen between resizes. Halving at one half (a) leaves no gap.

W4-M12 — d. For example: full at capacity 8, append (grow to 16), pop (shrink to 8), append (grow), … — an $O(n)$ copy on every operation.

W4-M13 — a. Capacities 4, 8, 16, 24, 32, 40, 52, 64, 76, … on Python 3.13.

W4-M14 — b. `sys.getsizeof` returns the header plus one reference (8 bytes) per slot of **capacity**; subtract the empty list's size and divide by 8.

W4-M15 — c. –1 + size = 4: the last *used* slot, not the last slot (a).

W4-M16 — d. All n elements shift right — whether or not a resize is needed.

W4-M17 — a. Nothing moves; clear the slot and decrement the size.

W4-M18 — b. The copies at each resize double, so the last resize copies more than all the earlier ones together, and it copies fewer than n.

W4-M19 — c. Any factor r > 1 gives a geometric series: about $n \cdot \frac{r}{r-1}$ copies — 3n for r = 1.5 against 2n for r = 2 — while the unused space after a resize is about a third rather than a half.

W4-M20 — d. Growing from c to 2c with c + 1 elements leaves c – 1 slots unused: about half.

W4-M21 — a. `Stack` keeps its top at the end of a `DynamicArray`. `CircularQueue` and the open-addressing map use a plain `Array`; the tree uses nodes.

W4-M22 — b. One coin for the append's own write, two saved to pay for copying at the next doubling.

Part B — Short answer and essay

W4-E1 *(5)*

- **Definition:** the amortised cost of an operation is the maximum total cost of any sequence of n operations, divided by n. It is a guarantee over sequences, not an average over inputs.
- **Writes:** each of the n appends writes one element: n.
- **Copies:** resizes happen when the size reaches 1, 2, 4, …, 2^k with $2^k < n$, each copying that many elements.
- **Sum:** $1 + 2 + \cdots + 2^k = 2^{k+1} - 1 < 2 \cdot 2^k < 2n$.
- **Conclusion:** total < 3n = O(n), so the amortised cost of one append is < 3 = O(1). (The accounting argument — 3 coins per append — is equally acceptable.)

W4-E2 *(4)*

- Each resize copies **everything** stored so far.
- **Constant step c:** resizes come every c appends, at sizes about c, 2c, 3c, …; the amounts copied grow **linearly**, and a sum of n/c linearly growing terms is about $n^2/(2c)$ — $\Theta(n^2)$. A larger c only shrinks the constant.
- **Factor r:** resizes come at sizes growing **geometrically**; each one copies r times more than the last, so the whole sum is dominated by its last term, which is less than n: total $\Theta(n)$.
- The difference is linear against geometric spacing of the resizes — not the size of the step.

W4-E3 *(3)*

- **Worst case O(n):** a single append that finds the array full must copy all n elements before writing.
- **Amortised O(1):** such appends are exponentially rare; any n appends cost under 3n in total. Both are true because they measure different things — one operation, against a whole sequence.
- **When the worst case matters:** when one slow operation is itself a failure — real-time systems, a game frame, a latency-critical request. Those reserve capacity in advance.

W4-E4 *(4)*

- **Thrashing:** operations alternating around a single boundary force a resize almost every time, so each costs $O(n)$.
- **Example** (grow at full, shrink at half full): capacity 8, size 8. append → grow to 16 (copy 8); pop → size 8 = half of 16 → shrink to 8 (copy 8); append → grow (copy 8); and so on.
- **Rule:** grow when full, but shrink (halve) only when the size falls to **one quarter** of the capacity.
- **Why it works:** after any resize the array is about half full, so at least about n/4 cheap operations must happen before the next resize, which pays for it: amortised O(1) for any mix of appends and pops.

W4-E5 *(3)*

- A **larger factor** resizes less often (fewer total copies, e.g. under 2n for ×2) but can leave more capacity unused (up to about half).
- A **smaller factor** wastes less memory but copies more (about $n\,r/(r-1)$ in total — around 9n for r = 1.125).
- Any factor > 1 keeps append amortised O(1). CPython chooses a small one because Python programs create huge numbers of lists, most of them small, so the saved memory matters more than the extra copying.

Part C — Trace the growth

W4-T1. Resizes on appends 2, 3, 5 and 9 (sizes 1, 2, 4, 8 were full). Capacity **16**, **4** resizes, **15** elements copied (1 + 2 + 4 + 8).

W4-T2. Resizes on appends **2, 3, 5, 9, 17**. Copies $1+2+4+8+16=31$; writes 20; total **51**; average \$51 / 20 = \$ **2.55** per append — under 3, as the proof promises.

W4-T3. Capacities 1 → 4 → 7 → 10 → 13. Resizes on appends **2, 5, 8, 11**. Final capacity **13**; copies \$1 + 4 + 7 + 10 = \$ **22**. The copies grow linearly — the signature of a constant step.

W4-T4.

Operation	Size	Capacity	Why
append(1)	1	1	room
append(2)	2	2	full → double

Operation	Size	Capacity	Why
append(3)	3	4	full → double
append(4)	4	4	room
append(5)	5	8	full → double
pop() → 5	4	8	4 > 8 // 4 = 2
pop() → 4	3	8	3 > 2
pop() → 3	2	4	2 ≤ 2 → halve
pop() → 2	1	2	1 ≤ 4 // 4 = 1 → halve

W4-T5. Appends **2, 3, 5, 9, 17, 33**: each is one more than a power of two, because the array is full exactly when its size is a power of two. The expensive appends double their spacing each time — which is why their cost averages out.

Part D — Proofs

W4-P1. Resizes happen when the size reaches $1, 4, 16, \dots, 4^k$ with $4^k < n$, and each copies that many elements. The copies total

$$1 + 4 + \cdots + 4^k = \frac{4^{k+1} - 1}{3} < \frac{4}{3} \cdot 4^k < \frac{4n}{3}.$$

Adding the n writes, the total is below $n + \frac{4n}{3} = \frac{7n}{3} = O(n)$, so append is amortised $O(1)$. ■ (The same proof works for any factor $r > 1$: copies $< n \cdot \frac{r}{r-1}$.)

W4-P2. Capacities are $1, 1 + c, 1 + 2c, \dots$. A resize happens when the size equals the capacity, $1 + jc$, for every j with $1 + jc < n$; call the number of resizes m. Then $m \geq (n - 1)/c$, and resize j copies $1 + jc$ elements, so

$$\text{copies} = \sum_{j=0}^{m-1} (1 + jc) = m + c\,\frac{m(m-1)}{2} \;\geq\; \frac{n-1}{c} + \frac{(n-1)(n-1-c)}{2c} = \frac{(n-1)(n+1-c)}{2c}.$$

Expanding, $\frac{(n-1)(n+1-c)}{2c} = \frac{n^2}{2c} - \frac{n}{2} + \frac{c-1}{2c} \geq \frac{n^2}{2c} - n$. For fixed c this is $\Omega(n^2)$: no constant step can make append amortised $O(1)$. ■

Part E — Find and fix the bug

W4-B1. The test should be `==`: with `>`, the array is never grown when it is exactly full, so the write goes one past the end of the `Array` — the second append raises `IndexError: Array index 1 out of range for length 1`. **Fix:** `if self._size == self._capacity:`.

W4-B2. The bound is the **capacity**, so reading an unused slot returns `None` instead of raising: with `[1, 2, 3]` stored (capacity 4), `a[3]` gives `None`. `test_index_out_of_range_raises` fails. **Fix:** `0 <= index < self._size`.

W4-B3. It grows by a **constant** (`growth` is 2, so `+ 2`), not by a factor: 1,000 appends resize about 500 times, and append is amortised $O(n)$. **Fix:** `self._resize(self._capacity * self.growth)`.

W4-B4. After shifting, the slot at the old last position still refers to the last element (and, for a pop from the end, to the removed value). The array keeps that object alive, so the garbage collector cannot reclaim it — a memory leak for large objects. **Fix:** after `self._size -= 1`, add `self._block[self._size] = None`.

Part F — Write the code

W4-C1

```
def capacity_after(n, factor=2):
    capacity, size = 1, 0
    for _ in range(n):
        if size == capacity:
            capacity *= factor
        size += 1
    return capacity
```

W4-C2

```
def copies_for(n, grow):
    capacity, size, copies = 1, 0, 0
    for _ in range(n):
        if size == capacity:
            copies += size              # the resize copies everything stored
            capacity = grow(capacity)
        size += 1
    return copies
```

`copies_for(100_000, lambda c: 2 * c)` is 131,071 and `copies_for(100_000, lambda c: c + 1)` is 4,999,950,000 — Lecture 04's numbers.

W4-C3

```
class ShrinkingArray:
    def __init__(self):
        self._block = Array(1)
        self._size = 0
        self.resize_count = 0
```

```
    @property
    def capacity(self):
        return len(self._block)

    def _resize(self, capacity):
        block = Array(capacity)
        for i in range(self._size):
            block[i] = self._block[i]
        self._block = block
        self.resize_count += 1

    def append(self, value):
        if self._size == self.capacity:
            self._resize(2 * self.capacity)
        self._block[self._size] = value
        self._size += 1

    def pop(self):
        if self._size == 0:
            raise IndexError("pop from empty array")
        self._size -= 1
        value = self._block[self._size]
       self._block[self._size] = None          # let the object be collected
        if self._size <= self.capacity // 4 and self.capacity > 1:
            self._resize(self.capacity // 2)
        return value

    def __len__(self):
        return self._size

    def __getitem__(self, index):
        if not 0 <= index < self._size:
            raise IndexError(index)
        return self._block[index]
```

The thrashing test: it fills the array to size 8, capacity 8, then alternates append and pop. The first append grows to 16 (size 9); the pop leaves size 8, which is above 16 // 4 = 4, so nothing happens — and the same for every later pair: one resize in total. With a half rule, size 8 ≤ 16 // 2 would shrink back to 8, and the next append would grow again: a resize on every operation, and the test fails.

Answers — Week 5

> ***The questions*** *are at the end of the* Week 5 *chapter. Commit to your own answer before reading one here — and for linked lists, draw it.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	a	M07	b	M13	a	M19	c
M02	c	M08	d	M14	b	M20	d
M03	b	M09	a	M15	c	M21	a
M04	d	M10	c	M16	d	M22	b
M05	a	M11	b	M17	a		
M06	c	M12	d	M18	b		

W5-M01 — a. The order is in the links themselves; there are no indices (b) and no separate order (d).

W5-M02 — c. Follow `next` i times from `head`: $O(i)$, $O(n)$ in the worst case.

W5-M03 — b. Two reference assignments, independent of n. "Amortised" (c) describes the dynamic array's append, not this.

W5-M04 — d. It must walk to the last node first. A tail reference makes it $O(1)$.

W5-M05 — a. Front insertion: two references here; every element shifts in an array.

W5-M06 — c. Indexing: address arithmetic in an array, a walk in a list. `len` (d) is $O(1)$ in both — because the list keeps `_size`.

W5-M07 — b. After `self.head = new`, `new.next = self.head` makes the node point at itself, and the old first node is no longer referenced by anything. Python evaluates each assignment's right-hand side first (a), but that is not the reason for the order of two separate statements.

W5-M08 — d. Only the predecessor holds the reference that must be redirected around the removed node.

W5-M09 — a. The head has no predecessor, so it is removed by moving `head` instead.

W5-M10 — c. Each `ll[i]` walks i steps from the head: $0 + 1 + \cdots + (n-1)$. Use `for value in ll:` for one $O(n)$ walk.

W5-M11 — b. A stored count, updated by `push_front`, `append`, `insert_at`, `pop_front` and `remove`. Forget one, and `len` lies.

W5-M12 — d. Three references — `prev`, `node` and the saved `next` — whatever `n` is.

W5-M13 — a. One 8-byte reference per slot; the node is six times larger.

W5-M14 — b. With `prev` in each node, the node itself gives you its predecessor. Indexing (a) and search (c) are still $O(n)$.

W5-M15 — c. With a permanent node before the first real one, removing the first real node looks exactly like removing any other.

W5-M16 — d. After removing the last node, `_tail` must move to its predecessor, and only a walk from the head finds it in a singly linked list.

W5-M17 — a. Blocks give array-like memory efficiency; the links give $O(1)$ at both ends.

W5-M18 — b. It is the standard off-by-one: `node.next is not None` is the right test only when you *want* to stop on the last node (to append after it).

W5-M19 — c. `c.next` is `b`, and `b.next` is `a`, whose value is 1.

W5-M20 — d. Unreachable objects are reclaimed automatically. `del` (b) only removes a *name*.

W5-M21 — a. The node at `i − 1` is the one whose `next` must change. Index 0 is `push_front`.

W5-M22 — b. `ChainingHashMap` (Week 13) stores an `Array` of chains of `Entry` nodes.

Part B — Short answer and essay

W5-E1 *(4)*

Operation	Dynamic array	Singly linked list
read element i	$O(1)$	$O(n)$
insert / remove at the front	$O(n)$	$O(1)$
append at the end	amortised $O(1)$	$O(n)$, or $O(1)$ with a tail
insert next to a node you hold	$O(n)$	$O(1)$

- **Choose the array** when you index by position, iterate a lot, or care about memory — e.g. storing measurements you read by index. It is compact and cache-friendly.
- **Choose the list** when you insert and remove at the front, or splice at positions you already hold — e.g. a stack of undo actions, the chains of a hash table. No copying, ever.

W5-E2 *(4)*

- Step 1: `new.next = self.head` — the new node points at the current first node.
- Step 2: `self.head = new` — the list now starts at the new node. Then `_size += 1`. Two assignments: $O(1)$.
- Diagram: new node → old first node → … → `None`, with `head` moving from the old first node to the new one.

- **Reversed order:** `self.head = new` first, then `new.next = self.head` sets `new.next = new` — a node pointing at itself. Nothing refers to the old first node any more, so the rest of the list is lost, and iterating loops for ever over the one node.

W5-E3 *(3)*

- Array indexing is address arithmetic, $O(1)$. List indexing must follow `next` from `head` i times, $O(i)$ — the same interface hides a different cost.
- In the loop, each `ll[i]` starts again from the head: $0 + 1 + \cdots + (n-1) = O(n^2)$ for what should be one pass.
- Write `for value in ll:` — `__iter__` keeps its place and walks once: $O(n)$.

W5-E4 *(4)*

- **Benefit:** append links after `_tail` and moves `_tail`: $O(1)$ instead of a walk.
- **Cost:** one more reference to keep correct in every method that can change the last node: `push_front` and `append` on an **empty** list (the new node is also the tail), `insert_at` at the end, `pop_front` of the **only** node, `remove` of the last node, and `reverse` (head and tail swap).
- **Removing the last node** is still $O(n)$: `_tail` must move to the node before it, and in a singly linked list only a walk from the head finds that node.

W5-E5 *(3)*

- **Doubly linked:** each node also references its predecessor. Removing a node you hold, and removing from the end, become $O(1)$; the list can be walked backwards. Cost: one more reference per node, and every insertion and removal updates twice as many links.
- **Sentinel:** a permanent node before the first real one, so every real node has a predecessor and there is no special case for the head (or for an empty list). Cost: one extra node, and every method must remember that the first real node is `sentinel.next`.

Part C — Trace the code

W5-T1

```
3 2 1
```

Each value is pushed on the **front**, so the last one pushed comes first — the behaviour of a stack (Week 6).

W5-T2

```
1 3 4 None
```

`a.next = a.next.next` makes node 1 skip node 2 — exactly how `remove` works.

W5-T3. 5 nodes → **2**; 6 nodes → **3**; 1 node → **0**; `None` → **0**. It moves two nodes at a time and counts the moves: it returns n // 2, half the length rounded down, in one pass — the "fast reference" of W5-C1.

W5-T4.

After step	prev	node	Links
start	None	1	1 → 2 → 3 → None
1	1	2	1 → None; 2 → 3 → None
2	2	3	2 → 1 → None; 3 → None
3	3	None	3 → 2 → 1 → None

Then `self.head = prev` = 3.

W5-T5

```
b True
```

`x.next = y` closes a loop: a → b → a → b … — a **cycle**. Iterating it would never end; W5-C2 detects it.

Part D — List state

W5-S1.

```
LinkedList([1, 2, 3])   1 → 2 → 3
push_front(0)           0 → 1 → 2 → 3
insert_at(2, 9)         0 → 1 → 9 → 2 → 3
pop_front()             1 → 9 → 2 → 3          (returns 0)
remove(2)               1 → 9 → 3              (returns True)
append(5)               1 → 9 → 3 → 5
```

`len(ll)` is **4**; `ll.find(3)` is **2**.

W5-S2. head points at the new node 5, and then `5.next = self.head` makes it point at **itself**. Nodes 3 and 7 are unreachable (and collected).

```
head → 5 → 5 → 5 → ...      (5.next is node 5 itself)
       3 → 7 → None         (unreachable: lost)
```

`list(ll)` never finishes: `__iter__` yields 5, 5, 5, … for ever. (Press Ctrl+C.)

W5-S3. 7 is not at the head, so walk with `prev`:

```
start:        head, prev → 3 → 7 → 1 → 9 → None      prev.next.value is 7:
 ↪  match
splice:       prev.next = prev.next.next
result:       head → 3 → 1 → 9 → None                 node 7: unreachable
```

`_size` goes from 4 to 3, and `remove` returns `True`.

Part E — Complexity analysis

W5-K1 — $\Theta(n^2)$. The i-th append walks past the i nodes already there: $0+1+\cdots+(n-1) = n(n-1)/2$. **In $\Theta(n)$:** `push_front` every value ($O(1)$ each) and then `reverse()` once ($O(n)$) — or keep a tail reference.

W5-K2 — $\Theta(n^2)$. The `for` loop is one walk, n iterations; each `find(value)` walks from the head to that value's position: $1 + 2 + \cdots + n$ comparisons in total.

W5-K3. Make the **head** the front (dequeue there) and the **tail** the back (enqueue there). Enqueue: link after `_tail`, $O(1)$. Dequeue: `pop_front`, $O(1)$. The other way round, dequeue would have to remove the **last** node, which is $O(n)$ in a singly linked list (W5-M16).

Part F — Find and fix the bug

W5-B1. `while node.next is not None` stops **on** the last node without checking it: `find` of the last value returns –1. On an empty list, `node` is `None` and `node.next` raises `AttributeError`. **Fix:** `while node is not None:`.

W5-B2. It never checks the **head** itself: `remove` of the first value returns `False` and leaves it in place. **Fix:** handle the head first —

```
if self.head is not None and self.head.value == value:
    self.head = self.head.next
    self._size -= 1
    return True
```

— or use a sentinel.

W5-B3. Two bugs: on an empty list `self.head.value` raises `AttributeError` instead of the promised `IndexError`; and `_size` is never decreased, so `len` becomes wrong. **Fix:** `if self.head is None: raise IndexError(...)` first, and `self._size -= 1`.

W5-B4. `node = node.next` runs **after** `node.next = prev`, so it moves to `prev` — backwards — instead of on. On 1 → 2 → 3 → 4, the first step sets `1.next = None` and then `node = None`: the loop ends, `head` becomes node 1, and the list is just `[1]`; nodes 2, 3, 4 are lost. **Fix:** save the next node first:

```
while node is not None:
    following = node.next
    node.next = prev
    prev = node
    node = following
```

Part G — Write the code

W5-C1 — slow and fast references.

```
def middle_value(head):
    if head is None:
        raise ValueError("empty chain")
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next                  # one step
        fast = fast.next.next             # two steps
    return slow.value
```

When `fast` reaches the end, `slow` has gone half as far.

W5-C2 — Floyd's "tortoise and hare".

```
def has_cycle(head):
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:                  # the fast one lapped the slow one
            return True
    return False
```

Why they meet: without a cycle, `fast` reaches `None`. With one, both references end up inside the loop; each step the distance from `fast` to `slow` along the loop shrinks by exactly one, so it reaches 0 within one lap. Note `is`, not ==: the question is whether they are the **same node**.

W5-C3 — a sentinel makes the head no special case.

```
def merge_sorted_chains(first, second):
    dummy = Node(None)
    tail = dummy
    while first is not None and second is not None:
        if first.value <= second.value:   # <= keeps it stable
            tail.next, first = first, first.next
        else:
            tail.next, second = second, second.next
        tail = tail.next
    tail.next = first if first is not None else second
    return dummy.next
```

The final line links the remaining chain in one step — no loop needed, unlike an array merge. $O(n + m)$ time, $O(1)$ extra space.

W5-C4

```
def remove_duplicates_sorted(head):
    node = head
    while node is not None and node.next is not None:
        if node.next.value == node.value:
            node.next = node.next.next    # skip the duplicate; stay on node
        else:
            node = node.next
    return head
```

Staying on node after a removal matters: the new node.next may be another duplicate.

W5-C5 — two references k apart.

```
def kth_from_end(head, k):
    if k < 1:
        raise IndexError(k)
    lead = head
    for _ in range(k):                    # move lead k nodes ahead
        if lead is None:
            raise IndexError(k)
        lead = lead.next
    trail = head
    while lead is not None:               # move both until lead falls off
     ↪ the end
        lead = lead.next
        trail = trail.next
    return trail.value
```

When lead is None, trail is exactly k nodes before the end.

Answers — Week 6

> ***The questions*** *are at the end of the* Week 6 *chapter. Commit to your own answer before reading one here.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	a	M07	b	M13	a	M19	c
M02	c	M08	d	M14	c	M20	d
M03	b	M09	a	M15	b	M21	a
M04	d	M10	c	M16	d	M22	b
M05	a	M11	b	M17	a		
M06	c	M12	d	M18	b		

W6-M01 — a. First in, first out (b) is a queue — Week 7.

W6-M02 — c. The stack goes [1, 2, 3] → pop 3 → [1, 2] → push 4 → [1, 2, 4] → pop 4 → [1, 2]; the next pop returns 2.

W6-M03 — b. `append` and `pop()` at the end move nothing. Index 0 (a) shifts everything on every operation.

W6-M04 — d. `push_front` and `pop_front` are $O(1)$. At the last node (a), popping would need the node before it — an $O(n)$ walk.

W6-M05 — a. The contract says `IndexError`, as `list.pop()` does. Returning `None` (b) lets an error travel silently.

W6-M06 — c. Inserting at index 0 of an array shifts every element — the "dishonest" design that still passes every test.

W6-M07 — b. The partner check fails. The count is even (a), and the stack is not empty at that point (d).

W6-M08 — d. The closer finds nothing to match. The partner check (a) cannot even run.

W6-M09 — a. `*` binds tighter, so `4 2 *` is computed first, then added to 3.

W6-M10 — c. The parentheses group `3 4 +` first; the result is then multiplied by 2.

W6-M11 — b. The first value popped (3) is the **right** operand, the second (8) the **left**: 8 − 3.

W6-M12 — d. `3 4 *` is 12, then `2 12 +` is 14.

W6-M13 — a. Higher **or equal**. Strictly higher (d) is the rule for right-associative operators such as `**`.

W6-M14 — c. Popping the earlier `-` before pushing the later one computes the left subtraction first.

W6-M15 — b. Left associative: `(8 - 3) - 2`. `8 3 2 - -` (a) is `8 - (3 - 2)`, which is 7, not 3.

W6-M16 — d. A text of n openers, `((((...`, pushes all n.

W6-M17 — a. The call stack does the bookkeeping for a recursion; without it, you keep that stack yourself.

W6-M18 — b. Redo only makes sense for actions just undone; a new action starts a new history.

W6-M19 — c. Each push is an append to the `DynamicArray`: amortised $O(1)$.

W6-M20 — d. First come, first served is FIFO — a queue.

W6-M21 — a. Four openers in a row, then four closers.

W6-M22 — b. And, like pop, it raises `IndexError` on an empty stack — so (d) is wrong.

Part B — Short answer and essay

W6-E1 *(4)*

- **ADT:** `push(x)`, `pop()`, `peek()`, `is_empty()`, `len()` — LIFO, all $O(1)$; pop and peek raise `IndexError` when empty.
- **On a dynamic array**, top at the end: push is amortised $O(1)$ (occasional resize), pop and peek $O(1)$; compact memory, contiguous.
- **On a linked list**, top at the head: all worst-case $O(1)$; a node per item (about six times the memory of an array slot).
- **Dishonest:** top at index 0 of an array — every push and pop shifts all the elements, $O(n)$ each, $O(n^2)$ for n operations — yet the stack behaves correctly and passes every test.

W6-E2 *(4)*

- Scan the text. **Opener**: push. **Closer**: the stack must be non-empty and the popped opener must be its partner. **End**: the stack must be empty. Other characters are ignored.
- **Wrong partner:** `([)]` — `)` pops `[`.
- **Closer with nothing open:** `)` or `())` — the stack is empty when a closer arrives.
- **Opener never closed:** `(` or `(()` — the stack is not empty at the end.
- $O(n)$ time, $O(n)$ space in the worst case.

W6-E3 *(3)*

- Scan the tokens: push numbers; for an operator, pop two values, apply it, push the result; at the end exactly one value must remain (otherwise the input was malformed).
- The **first** value popped is the **right** operand and the **second** the **left**, because the left operand was pushed earlier.

- It matters for the non-commutative operators, `-` and `/`: `8 3 -` must be 8 − 3 = 5, not 3 − 8 = −5.

W6-E4 *(4)*

- Output list and operator stack. Numbers go to the output; `(` is pushed; `)` pops to the output down to the matching `(`; an operator pops every stacked operator of higher **or equal** precedence, then is pushed; at the end the stack is emptied to the output.
- **Trace** of `2 * ( 3 + 4 ) - 5`:

Token	Output	Stack
2	2	
*	2	*
(	2	* (
3	2 3	* (
+	2 3	* (+
4	2 3 4	* (+
)	2 3 4 +	*
-	2 3 4 + *	-
5	2 3 4 + * 5	-
end	**2 3 4 + * 5 -**	

- **Equal precedence pops** so that left-associative operators apply left to right: in `8 - 3 - 2`, the first `-` must be applied before the second.

W6-E5 *(3)*

- Every call pushes a frame (its local variables and where to return) on the call stack, and every return pops one — so a recursion is a computation driven by a stack.
- **Rewrite:** push the initial problem on an explicit `Stack`; loop while it is not empty: pop a problem, solve it directly if it is a base case, otherwise push its sub-problems.
- **Worth it in Python** when the recursion could go deeper than about 1,000 frames (`RecursionError`), since there is no tail-call optimisation — e.g. a depth-first search on a large graph (Week 14).

Part C — Trace the code and the algorithms

W6-T1

```
[2, 4, 4, 3, 1]
```

After pop → 2 the stack is [1]; push 3, 4 → [1, 3, 4]; peek → 4 (not removed); pop → 4; pop → 3; one item, 1, is left.

W6-T2

Token	Stack (top on the right)
5	5
1	5 1
2	5 1 2
+	5 3
4	5 3 4
*	5 12
+	17
3	17 3
-	14

The answer is **14** — the expression `5 + (1 + 2) * 4 - 3`.

W6-T3. The table in W6-E4: the result is **`2 3 4 + * 5 -`**.

W6-T4.

Character	Stack after it
{	{
[	{ [
(	{ [(
)	{ [
(	{ [(
)	{ [
]	{
}	*(empty)*

Balanced — the stack is empty at the end. Maximum size **3**.

W6-T5

```
4 3 2 1
```

The inner items of a list are pushed left to right, so the **last** one is popped first: 4, then the list `[2, 3]`, whose items are pushed 2, 3 and popped 3, 2; then 1. A stack reverses the order things were pushed in. To visit left to right, push the items in reverse — exactly what `dfs_iterative` does in Week 14.

Part D — Find and fix the bug

W6-B1. An empty pop returns `None` instead of raising `IndexError`, breaking the contract (and `test_stack_empty_behaviour`). Worse, `None` can then be pushed, compared or printed far from the real mistake. **Fix:** `raise IndexError("pop from empty stack")`.

W6-B2. It never checks that the stack is **empty at the end**, so unclosed openers pass: `is_balanced("(")` returns `True`. **Fix:** `return s.is_empty()`.

W6-B3. The first value popped is the **right** operand, but it is passed as the left: `8 3 -` gives 3 − 8 = −5, and `8 2 /` gives 0.25. Addition and multiplication hide the bug. **Fix:** `right = stack.pop(); left = stack.pop(); stack.push(apply(token, left, right))`.

W6-B4. Popping only on **strictly** higher precedence makes `-` and `/` right associative: `8 - 3 - 2` becomes `8 3 2 - -`, which evaluates to 7 instead of 3. **Fix:** `>=`.

Part E — Write the code

W6-C1

```
def reverse_words(sentence):
    words = Stack(sentence.split())
    out = []
    while not words.is_empty():
        out.append(words.pop())
    return " ".join(out)
```

Pushing and popping reverses the order — the stack's defining property.

W6-C2

```
class MinStack:
    def __init__(self):
        self._values = Stack()
        self._mins = Stack()            # _mins.peek() is the minimum so far

    def push(self, value):
        self._values.push(value)
        if self._mins.is_empty() or value < self._mins.peek():
            self._mins.push(value)
        else:
            self._mins.push(self._mins.peek())

    def pop(self):
        self._mins.pop()                # IndexError when empty, as required
        return self._values.pop()

    def peek(self):
        return self._values.peek()

    def get_min(self):
        return self._mins.peek()
```

```
    def __len__(self):
        return len(self._values)
```

Why one variable is not enough: when the minimum is popped, you need the minimum of what is left — which you could only find again by searching, $O(n)$. The second stack remembers the minimum at every level, so after any pop the new minimum is simply on top.

W6-C3

```
def next_greater(values):
    result = [-1] * len(values)
    waiting = Stack()                    # indices still waiting for an answer
    for i, value in enumerate(values):
        while not waiting.is_empty() and values[waiting.peek()] < value:
            result[waiting.pop()] = value
        waiting.push(i)
    return result
```

Why linear: the inner `while` does not run up to n times per iteration *every* time. Each index is pushed exactly once and popped at most once, so across the whole run the inner loop body executes at most n times in total: $O(n)$. This is **aggregate** counting — the same argument as amortised analysis in Lecture 04. (The stack always holds values in decreasing order — a *monotonic stack*.)

W6-C4

```
def tags_balanced(html):
    open_tags = Stack()
    i = 0
    while True:
        start = html.find("<", i)
        if start == -1:
            break
        end = html.find(">", start)
        if end == -1:
            return False
        tag = html[start + 1:end]
        if tag.startswith("/"):
            if open_tags.is_empty() or open_tags.pop() != tag[1:]:
                return False
        else:
            open_tags.push(tag)
        i = end + 1
    return open_tags.is_empty()
```

The same three checks as brackets, with tag names instead of bracket characters.

W6-C5

```
def decode(encoded):
    counts, texts = Stack(), Stack()
    current, number = "", 0
    for ch in encoded:
        if ch.isdigit():
            number = number * 10 + int(ch)
        elif ch == "[":
            counts.push(number)
            texts.push(current)           # save what came before this group
            current, number = "", 0
        elif ch == "]":
            current = texts.pop() + current * counts.pop()
        else:
            current += ch
    return current
```

On `[`, the text so far and the repeat count are saved and a fresh group starts; on `]`, the group is repeated and appended to what was saved. Nesting works because the innermost group is always on top — exactly as with brackets.

Answers — Week 7

> ***The questions*** *are at the end of the* Week 7 *chapter. Commit to your own answer before reading one here.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M07	c	M13	d	M19	c
M02	d	M08	a	M14	c	M20	b
M03	a	M09	b	M15	a	M21	d
M04	b	M10	d	M16	b	M22	a
M05	c	M11	a	M17	d		
M06	d	M12	c	M18	a		

W7-M01 — b. Last in, first out (a) is a stack — Week 6.

W7-M02 — d. The queue goes [1, 2, 3] → dequeue 1 → [2, 3] → enqueue 4 → [2, 3, 4] → dequeue 2 → [3, 4]; the next dequeue returns 3. (A stack would have returned 4 — the difference in one question.)

W7-M03 — a. `pop(0)` moves the n − 1 elements behind the front. Amortised $O(1)$ (c) is the cost of `append`, the enqueue.

W7-M04 — b. `pop()` at the end moves nothing, but `insert_at(0, x)` shifts everything. With a plain array, whichever end is the front, one operation is $O(n)$.

W7-M05 — c. (6 + 3) % 8 = 9 % 8 = 1. Slot 9 (a) does not exist; the items are in slots 6, 7 and 0.

W7-M06 — d. (4 + 1) % 5 = 0: the head wraps. Slot 5 (a) is past the end of the `Array`.

W7-M07 — c. Empty: nothing between head and tail. Full: the tail has come all the way round to the head. Both look the same.

W7-M08 — a. With the size, empty is `size == 0` and full is `size == capacity`. The `Array` does know its length (c): `len(block)`.

W7-M09 — b. Slots 2 and 3 of the new block hold C and D, slots 4 and 5 are empty, and E and F sit in slots 0 and 1 — behind the front. The correct copy gives (a) with head = 0.

W7-M10 — d. Dequeue at the head (`pop_front`, $O(1)$), enqueue after the tail ($O(1)$).

W7-M11 — a. The new tail is the node before the old one, and a singly linked list can only find it by walking from the head.

W7-M12 — c. The first dequeue finds `outbox` empty and pours all three: `outbox` holds 3, 2, 1 (1 on top). It pops 1, leaving 2 and 3.

W7-M13 — d. One dequeue can pour n items, so the worst case (c) is $O(n)$, but each item is pushed, poured and popped once in its life.

W7-M14 — c. BFS visits nodes in the order they were discovered: FIFO. The other three are stack applications from Week 6.

W7-M15 — a. Dequeue number i shifts n − i elements: $(n-1)+(n-2)+\cdots+0 = n(n-1)/2$.

W7-M16 — b. Python's `%` takes the sign of the divisor, so the result is in 0..7: stepping back from slot 0 wraps to slot 7. In C or Java the answer would be −1 (a).

W7-M17 — d. A double-ended queue. (a) is a priority queue.

W7-M18 — a. Both are $O(1)$; the ring pays more per operation. Big-O hides constant factors, and measurement shows them.

W7-M19 — c. B, D, A and E leave in that order (W7-C2); C is last.

W7-M20 — b. Tail = 8: there is no slot 8, even though slots 0 to 4 are dead and free. That is why the queue must wrap.

W7-M21 — d. The occasional copy is $O(n)$, but doubling makes the copies total less than 2n over n enqueues — Lecture 04's argument.

W7-M22 — a. Same two operations, a different rule — and a different structure underneath, the heap.

Part B — Short answer and essay

W7-E1 *(4)*

- **ADT:** `enqueue(x)` at the back, `dequeue()` from the front (FIFO), `is_empty`, `len` — all $O(1)$; `dequeue` raises `IndexError` when empty.
- **Shifting:** on a plain array the front is a fixed position, index 0; removing it moves the n − 1 others: $O(n)$.
- **Swapping ends** does not help: the back at index 0 makes **enqueue** shift everything instead. A queue uses both ends, and an array is cheap at only one.
- **Ring buffer:** do not move data, move the indices. `head` marks the front; the back is `(head + size) % capacity`; both wrap round with `%`, so slots freed at the front are reused. Every operation $O(1)$.

W7-E2 *(4)*

- **Empty:** nothing in the ring, e.g. a new queue: head = tail = 0.
- **Full:** the tail has caught up with the head, e.g. capacity 4, head 2, four items: tail = (2 + 4) % 4 = 2.

- **Fixes:** (1) keep a count `size` — empty is `size == 0`, full is `size == capacity`; (2) leave one slot always unused — full is `(tail + 1) % capacity == head`, so `head == tail` means only empty; (3) a flag set when an enqueue makes head equal tail.
- `dsa/queue.py` keeps **`_size`** (and computes the tail from it).

W7-E3 *(4)*

- **Example:** capacity 4, head 2, block `[E, F, C, D]`, queue C D E F. A slot-by-slot copy into 8 slots keeps C, D in slots 2, 3 and E, F in slots 0, 1; with head 2 the queue now reads C, D, then empty slots — E and F are lost behind the front.
- **Correct copy:** in queue order: `new[i] = old[(head + i) % old_capacity]` for i in 0..size − 1, then head = 0 (and the capacity updated).
- **Amortised $O(1)$:** a copy happens only when the ring is full, and doubling means the copies after n enqueues total 1 + 2 + 4 + … < 2n: $O(n)$ over n operations, $O(1)$ each on average.
- It only goes wrong once the data has wrapped, which is why it survives tests that never dequeue before growing.

W7-E4 *(3)*

- `inbox` takes every enqueue; `dequeue` pops `outbox`, and when `outbox` is empty it first pours **all** of `inbox` into it — which reverses them, so the oldest is on top.
- **Amortised:** each item is pushed on `inbox` once, moved to `outbox` once and popped once — three $O(1)$ steps in its life, so n operations cost $O(n)$.
- **Pour only when empty:** pouring into a non-empty `outbox` would put newer items on top of older ones and break FIFO; pouring back and forth on every dequeue keeps FIFO but costs $O(n)$ per dequeue (W7-K3).

W7-E5 *(3)*

- **Ring buffer:** $O(1)$ (amortised if it grows); one contiguous block, no allocation per item, good locality. Choose it when the maximum size is known or bounded — hardware and OS buffers — or for speed in general.
- **Linked queue:** $O(1)$ worst case, never copies; but a node per item (more memory, scattered). Choose it when a worst-case guarantee matters more than memory.
- **`collections.deque`** combines them: a doubly linked list of fixed blocks — $O(1)$ at both ends, few allocations. It is what to use in real Python code.

Part C — Trace the code

W7-T1

```
[1, 2, True, [3, 4, 5, 6]]
```

After 1 is dequeued, head = 1; 4 goes to slot 3 and 5 wraps to slot 0; 2 is dequeued (head = 2); 6 goes to slot (2 + 3) % 4 = 1. The block is **`[5, 6, 3, 4]`**: slot 0 holds 5, slot 1 holds 6, slot 2 holds 3, slot 3 holds 4 — and the queue reads 3 4 5 6 from head = 2. Full: size 4 = capacity.

W7-T2

Operation	inbox	outbox	Returns
enqueue 1	1		
enqueue 2	1 2		
enqueue 3	1 2 3		
dequeue		3 2	1 *(poured 3 items, then popped 1)*
enqueue 4	4	3 2	
dequeue	4	3	2
dequeue	4		3
enqueue 5	4 5		
dequeue		5	4 *(poured 2 items)*
dequeue			5

Stacks are shown bottom to top. The values come out 1, 2, 3, 4, 5: FIFO. Only two pours happened, one per time `outbox` ran dry.

W7-T3

Leaves	Queue after, front to back
C	D E F A B
F	A B D E
D	E A B
B	E A
E	A
A	

Order: **C, F, D, B, E, A** — A is last. Each round moves two people from the front to the back, then removes the third.

W7-T4

Dequeued	Queue after its two enqueues
1	10 11
10	11 100 101
11	100 101 110 111
100	101 110 111 1000 1001
101	110 111 1000 1001 1010 1011
110	111 1000 1001 1010 1011 1100 1101

Returned: `["1", "10", "11", "100", "101", "110"]`. Every string of length k is dequeued before any of length k + 1: the queue works level by level, exactly as breadth-first search does.

W7-T5

```
1 3 5 6 4 2
```

The odd numbers are printed as they leave the queue, in their original order (FIFO). The even numbers wait on the stack and come off in reverse (LIFO). One loop, two structures, two orders.

Part D — Ring state

W7-S1.

```
operation    block                    head  size
enqueue 10   [10, _,  _,  _,  _ ]      0     1
enqueue 20   [10, 20, _,  _,  _ ]      0     2
enqueue 30   [10, 20, 30, _,  _ ]      0     3
enqueue 40   [10, 20, 30, 40, _ ]      0     4
dequeue      [_,  20, 30, 40, _ ]      1     3     returns 10
dequeue      [_,  _,  30, 40, _ ]      2     2     returns 20
enqueue 50   [_,  _,  30, 40, 50]      2     3
enqueue 60   [60, _,  30, 40, 50]      2     4     wrapped to slot 0
enqueue 70   [60, 70, 30, 40, 50]      2     5     full
dequeue      [60, 70, _,  40, 50]      3     4     returns 30
```

`list(q)` is **`[40, 50, 60, 70]`**; the queue is **not** full (size 4 of 5).

W7-S2.

```
enqueue 1, 2, 3   [1, 2, 3]   head 0, size 3     tails were 0, 1, 2: all fine
dequeue            [_, 2, 3]   head 1, size 2      returns 1
enqueue 4          tail = 1 + 2 = 3  ->  block[3]: IndexError (no slot 3)
```

Without %, the tail never wraps: slot 0, freed by the dequeue, is never reused. Every earlier enqueue happened while head was 0, where `head + size` is always less than the capacity, so it was correct. The bug only appears once the queue has both dequeued and filled up again — exactly the case this test was written for.

W7-S3.

```
operation        block            head  size   front to back
add_back 1       [1, _, _, _]      0     1     1
add_back 2       [1, 2, _, _]      0     2     1 2
add_front 0      [1, 2, _, 0]      3     3     0 1 2          head (0 - 1) %
 ↪  4 = 3
add_front 9      [1, 2, 9, 0]      2     4     9 0 1 2         full
remove_back      [1, _, 9, 0]      2     3     9 0 1           returns 2
add_back 5       [1, 5, 9, 0]      2     4     9 0 1 5        slot (2 + 3) %
 ↪  4 = 1
remove_front     [1, 5, _, 0]      3     3     0 1 5           returns 9
```

```
add_front 7      [1, 5, 7, 0]      2     4     7 0 1 5        full
add_back 8       [7, 0, 1, 5, 8, _, _, _]   0   5   7 0 1 5 8
```

Python's % gives a result from 0 to capacity − 1 even for −1, so stepping back from slot 0 lands on slot 3 (in C or Java, write `(head - 1 + capacity) % capacity`). `remove_back` needs no change to head: shrinking `size` moves the back. The last step grows first: the four items are copied **in deque order** into slots 0–3 of an 8-slot block (head becomes 0), then 8 goes to slot (0 + 4) % 8 = 4.

Part E — Complexity analysis

W7-K1 — $\Theta(n^2)$. The n enqueues are amortised $O(1)$ each: $\Theta(n)$. The dequeue that finds k items shifts k − 1 of them: $(n-1)+(n-2)+\cdots+0 = n(n-1)/2$. **With `CircularQueue(n)`:** every operation is $O(1)$ and no growth is needed, so $\Theta(n)$ in total.

W7-K2 — $\Theta(n^2)$. After the i-th enqueue the queue holds i items, and `sum(q)` iterates over all of them: $1 + 2 + \cdots + n = n(n+1)/2$. The queue operations are not the problem; the full walk inside the loop is. Keeping a running total instead makes it $\Theta(n)$.

W7-K3 — $\Theta(n^2)$ against $\Theta(n)$. When k items are in the queue, each dequeue moves all k to `outbox` and the k − 1 left back to `inbox`: about 2k moves, so the n dequeues cost about $2(n + (n-1) + \cdots + 1) \approx n^2$. The correct version pours once — all n items — and every later dequeue is a single pop: $\Theta(n)$ in total. (Measured with this course's `Stack`: 6,000 enqueues mixed with 2,000 dequeues took about 16 seconds with the pouring-back version; the correct version runs the W7-C1 test — 200,000 enqueues and 66,666 dequeues — in under a second.)

Part F — Find and fix the bug

W7-B1. `pop()` removes the **last** item — the newest — so this is a stack, not a queue: enqueue 1, 2, 3 and dequeue returns 3. `test_queue_is_fifo` fails. **Fix:** `return self._items.pop(0)` — $O(n)$, which is the point of `SlowQueue`.

W7-B2. `head` never wraps. `test_circular_queue_wraps_around` still passes because `__iter__` applies `% capacity` itself, but in `test_circular_queue_interleaved_operations` (capacity 4) the fifth dequeue reads `block[4]`: `IndexError`, although the queue holds items. **Fix:** `self._head = (self._head + 1) % self._capacity`.

W7-B3. It tests fullness by `tail == head`, which is also true when the queue is **empty**: the very first enqueue on a new queue raises "queue is full", and the queue can never hold anything. **Fix:** `if self._size == self._capacity:` — the size is what tells full from empty.

W7-B4. When the last node is removed, `head` becomes `None` but `tail` still points at the removed node. The next enqueue sees a non-empty `tail`, links the new node after the dead one, and

leaves head at None: enqueue(1), dequeue(), enqueue(2), dequeue() raises "dequeue from empty queue". **Fix:** after moving head, `if self.head is None: self.tail = None`.

Part G — Write the code

W7-C1

```
class StackQueue:
    def __init__(self):
        self._inbox = Stack()              # enqueue pushes here
        self._outbox = Stack()             # dequeue pops here; oldest on top

    def enqueue(self, value):
        self._inbox.push(value)

    def dequeue(self):
        if self._outbox.is_empty():        # pour only when the outbox is
        ↪  empty
            while not self._inbox.is_empty():
                self._outbox.push(self._inbox.pop())
        return self._outbox.pop()          # IndexError when both are empty

    def __len__(self):
        return len(self._inbox) + len(self._outbox)
```

Why amortised $O(1)$: each value is pushed on inbox once, moved to outbox once, and popped once — three $O(1)$ steps over its lifetime, whatever the order of operations. A single dequeue can cost $O(n)$, but only after n cheap enqueues have paid for it.

W7-C2

```
def josephus(names, k):
    circle = CircularQueue(max(1, len(names)))
    for name in names:
        circle.enqueue(name)
    order = []
    while len(circle) > 0:
        for _ in range(k - 1):
            circle.enqueue(circle.dequeue())   # passed by: front to back
        order.append(circle.dequeue())          # the k-th leaves
    return order
```

The queue never holds more than it started with, so it never needs to grow. Cost: $O(nk)$ — each of the n removals first moves k − 1 people.

W7-C3

```
def moving_averages(values, k):
    if k < 1:
        raise ValueError("k must be at least 1")
    window = CircularQueue(k)
    total, result = 0, []
    for value in values:
        if len(window) == k:
            total -= window.dequeue()           # the oldest value leaves
        window.enqueue(value)
        total += value
        if len(window) == k:
            result.append(total / k)
    return result
```

Why $O(n)$: each value is enqueued once and dequeued at most once, and the running total changes by two $O(1)$ steps per value. Summing each window afresh would cost k additions per window: $O(nk)$ — 1.6 billion additions in the test with n = 100,000 and k = 20,000. The window never holds more than k values, so a `CircularQueue(k)` never needs to grow: a fixed-size ring buffer holding "the last k events" is exactly what ring buffers are for.

W7-C4

```
def reverse_first_k(values, k):
    if not 0 <= k <= len(values):
        raise ValueError(f"k must be between 0 and {len(values)}")
    queue = CircularQueue(max(1, len(values)))
    for value in values:
        queue.enqueue(value)
    stack = Stack()
    for _ in range(k):                          # the first k onto the stack...
        stack.push(queue.dequeue())
    result = []
    while not stack.is_empty():                 # ...come off reversed
        result.append(stack.pop())
    while len(queue) > 0:                       # the rest keep their order
        result.append(queue.dequeue())
    return result
```

The stack reverses what passes through it; the queue preserves it. $O(n)$.

W7-C5

```
def binary_numbers(n):
    result = []
    pending = CircularQueue(2 * n + 1)          # never needs to grow
    pending.enqueue("1")
    while len(result) < n:
```

```
        s = pending.dequeue()
        result.append(s)
        pending.enqueue(s + "0")
        pending.enqueue(s + "1")
    return result
```

Each dequeue adds two strings, so after n dequeues the queue holds n + 1: a capacity of 2n + 1 is more than enough. The strings come out in order because the queue finishes every length before starting the next — breadth-first, the idea behind Week 14's BFS. $O(n)$ queue operations (building each string also costs its length, $O(\log n)$).

Answers — Week 8

> ***The questions*** *are at the end of the* Week 8 *chapter. Commit to your own answer before reading one here.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	a	M07	b	M13	a	M19	c
M02	c	M08	d	M14	d	M20	b
M03	b	M09	a	M15	c	M21	d
M04	d	M10	c	M16	b	M22	a
M05	a	M11	d	M17	d		
M06	c	M12	b	M18	a		

W8-M01 — a. The other three all rely on the order to skip elements.

W8-M02 — c. $\lfloor \log_2 1000 \rfloor + 1 = 9 + 1 = 10$. 1,000 (a) is linear search's worst case.

W8-M03 — b. $\lfloor \log_2 10^6 \rfloor + 1 = 19 + 1 = 20$.

W8-M04 — d. Nothing checks the order — checking would cost $O(n)$. The comparison with the middle then discards the wrong half, and a value that is present can be reported as −1.

W8-M05 — a. `mid = (0 + 15) // 2 = 7`, `V[7] = 24`.

W8-M06 — c. 24 > 20, so `hi = 6`; `mid = (0 + 6) // 2 = 3`, `V[3] = 12`.

W8-M07 — b. It is what makes "the range is empty" mean "the target is not there".

W8-M08 — d. With `lo = hi = 0` the loop never runs, so the only candidate is never compared (W8-B1).

W8-M09 — a. Joshua Bloch's 2006 bug report: arrays of more than about 2^{30} elements. Fix: `lo + (hi - lo) / 2`. Python integers do not overflow.

W8-M10 — c. The slices copy $n/2 + n/4 + \cdots < n$ elements in total (W8-K2).

W8-M11 — d. One frame per halving: about $\log_2 n$ frames. The loop form is $O(1)$ (a).

W8-M12 — b. The first index with `v[i] >= 2`.

W8-M13 — a. The first index with `v[i] > 2`: the 5 at index 4.

W8-M14 — d. No element is ≥ 8, so the answer is `len(v)` = 6 — the insertion point at the end. The bounds never return −1 (a).

W8-M15 — c. upper_bound = 5 and `lower_bound` = 1: 5 − 1 = 4.

W8-M16 — b. And `bisect_right` is upper_bound.

W8-M17 — d. $\sqrt{100} = 10$: at most 10 jumps and 10 steps inside a block.

W8-M18 — a. $n/m + m$ is smallest at $m = \sqrt{n}$, giving $2\sqrt{n}$. Jump search is $O(\sqrt{n})$, not $O(\log n)$ (d).

W8-M19 — c. It checks slot 0, then doubles: `v[16]` is the first bound not smaller than the target, and binary search runs on slots 8 to 16.

W8-M20 — b. Doubling needs no end, and it costs $O(\log i)$ for a target at index i.

W8-M21 — d. One huge value makes every guess land at the left end, and the range shrinks by one per step (W8-T5).

W8-M22 — a. $0 + (70 - 10)(9 - 0)/(100 - 10) = 540/90 = 6$, and `v[6] = 70`: found in one probe.

Part B — Short answer and essay

W8-E1 *(4)*

- **Algorithm:** keep `lo = 0`, `hi = n - 1`; while `lo <= hi`, compare the target with `values[mid]`, `mid = (lo + hi) // 2`: equal → return `mid`; smaller middle → `lo = mid + 1`; larger → `hi = mid - 1`. Return −1 when the range is empty.
- **Invariant:** if the target is in the list, it is in `values[lo..hi]`. True at the start (whole list); each step removes only elements that the sorted order proves cannot be the target; so when the range is empty, the target is absent — the −1 is correct.
- **Count:** each unsuccessful comparison leaves at most half the range; after $\lfloor \log_2 n \rfloor$ halvings one element is left, and one more comparison settles it: $O(\log n)$.
- **Caller's guarantee:** the list is **sorted** (a precondition — not checked).

W8-E2 *(4)*

- `while lo < hi` (inclusive `hi`): the last candidate is never compared — `[42]` searched for 42 gives −1. **Fix:** `lo <= hi`.
- `hi = mid`: when the range is one element and it is too big, nothing changes — `[1, 3]` searched for 0 loops for ever. **Fix:** `hi = mid - 1`.
- `lo = mid`: with `lo = 0, hi = 1` and `values[0]` too small, `mid` stays 0 — loops for ever. **Fix:** `lo = mid + 1`.
- `mid = (lo + hi) / 2` in Java/C: overflows on arrays over about 2^{30} elements, giving a negative index. **Fix:** `lo + (hi - lo) / 2`.
- (Also accepted: recursing on slices — returns an index into the slice, W8-B3.)

W8-E3 *(4)*

- `lower_bound(v, x)`: the first index i with `v[i] >= x`; `upper_bound(v, x)`: the first with `v[i] > x`. Both in $O(\log n)$, on a sorted list.

- **Uses:** the insertion point that keeps the list sorted (`lower_bound`); the number of copies of x (`upper_bound - lower_bound`); the first occurrence (`lower_bound`, then check `v[i] == x`); the count of values in `[a, b)`.
- **Never −1:** they answer "where", not "whether" — every x has a position in the order, from 0 (before everything) to `len(v)` (after everything). That is why the search range is half-open, `[0, len(v))`: `len(v)` must be a possible answer.
- Python: `bisect_left` and `bisect_right`.

W8-E4 *(3)*

- **Jump**, $O(\sqrt{n})$: moves only forward, in blocks of $\sqrt{n}$ — for storage where going back is expensive (tape, a stream read in order).
- **Exponential**, $O(\log i)$ for a target at index i: doubles a bound, then binary searches — for sequences of unknown length, or targets near the front.
- **Interpolation**, $O(\log \log n)$ on average for evenly spread values but $O(n)$ in the worst case: guesses the position from the values — for large, uniformly distributed keys (numeric IDs, timestamps) where you can trust the distribution.

W8-E5 *(3)*

- Linear search k times: $O(kn)$. Sort once and binary search k times: $O(n \log n + k \log n)$.
- Sorting pays once $kn > n \log n$, i.e. once **k exceeds about** $\log_2 n$.
- **Example:** n = 1,000,000. One linear search: up to 10^6 reads. Sorting: about $n \log_2 n \approx 2 \times 10^7$ comparisons, then 20 per search. After about 20 searches the sort has paid for itself.
- If the data also changes often, a sorted array's $O(n)$ insertion is the catch: a balanced search tree (Week 11) or a hash table (Week 13) may fit better.

Part C — Trace the algorithms

W8-T1

Target	lo	hi	mid	values[mid]	Decision
23	0	9	4	16	16 < 23 → lo = 5
	5	9	7	56	56 > 23 → hi = 6
	5	6	5	23	**found: 5**
60	0	9	4	16	16 < 60 → lo = 5
	5	9	7	56	56 < 60 → lo = 8
	8	9	8	72	72 > 60 → hi = 7
	8	7			lo > hi: **−1**

For 60 the loop ends with `lo = 8`: the index where 60 **would be inserted** to keep the list sorted (between 56 and 72) — exactly `lower_bound(values, 60)`.

W8-T2

Function	lo	hi	mid	v[mid]	Direction
`lower_bound(v, 2)`	0	6	3	2	2 < 2? no → left: hi = 3
	0	3	1	2	no → left: hi = 1
	0	1	0	1	1 < 2 → right: lo = 1
	1	1			empty: **return 1**
`upper_bound(v, 2)`	0	6	3	2	2 ≤ 2 → right: lo = 4
	4	6	5	7	7 ≤ 2? no → left: hi = 5
	4	5	4	5	no → left: hi = 4
	4	4			empty: **return 4**

The two differ only in the comparison that sends the search right: `<` for `lower_bound`, `<=` for `upper_bound`. Neither stops when it meets a 2.

W8-T3

```
binary_search_recursive(V, 45)
  lo=0  hi=15  mid=7   V[7]=24 < 45
    lo=8  hi=15  mid=11  V[11]=38 < 45
      lo=12 hi=15  mid=13  V[13]=45: return 13      (3 calls)

binary_search_recursive(V, 7)
  lo=0  hi=15  mid=7   V[7]=24 > 7
    lo=0  hi=6   mid=3   V[3]=12 > 7
      lo=0  hi=2   mid=1   V[1]=6 < 7
        lo=2  hi=2   mid=2   V[2]=8 > 7
          lo=2  hi=1: empty, return -1              (5 calls)
```

The search for 7 is 5 frames deep — about $\log_2 16 + 1$. Each call returns the value of the call it made, unchanged, so the −1 or the index travels back up the stack.

W8-T4. Step $\sqrt{25} = 5$. The jumps read the last element of each block: `v[4] = 9`, `v[9] = 19`, `v[14] = 29` — all smaller than 30 — then `v[19] = 39`, not smaller: stop. The walk starts at index 15: `v[15] = 31` is already larger than 30, so 30 is not there. **Returns −1** after reading 5 elements.

W8-T5

Search	lo	hi	pos	v[pos]	Result
70 in `[10, …, 100]`	0	9	0 + 60·9 // 90 = 6	70	found, **1 probe**
75 in `[10, …, 100]`	0	9	0 + 65·9 // 90 = 6	70	70 < 75 → lo = 7
	7	9			75 < v[7] = 80: outside the range → **−1**, 1 probe
9 in `[1, …, 9, 1000]`	0	9	0 + 8·9 // 999 = 0	1	lo = 1
	1	9	1 + 7·8 // 998 = 1	2	lo = 2
	…		each guess is lo		…

Search	lo	hi	pos	v[pos]	Result
	8	9	8 + 0 = 8	9	found, **9 probes**

On evenly spread values the guess is exact. With one huge value, the straight line from `v[lo]` to 1000 puts every target at the far left, so the range shrinks by **one** element per probe: interpolation search has become a linear search — its $O(n)$ worst case. Binary search would need at most 4 probes here.

Part D — Complexity analysis

W8-K1 — $\Theta(m \log n)$. m searches of $O(\log n)$ each (worst case). With `linear_search`: $\Theta(mn)$. **If a were unsorted,** binary search would give wrong answers; sort it first: $O(n \log n + m \log n)$ — worth it once m exceeds about $\log n$ (W8-E5).

W8-K2 — $\Theta(n)$. The slice copies about half the list, so $T(n) = T(n/2) + \Theta(n)$. Unrolled: $n/2 + n/4 + \cdots + 1 < n$ — $\Theta(n)$, no better than linear search. (And the index it returns is wrong — W8-B3.)

W8-K3 — $\Theta(\log n + c)$. The binary search is $O(\log n)$; the walk visits all c copies. **Worst case:** a list that is all x — $\Theta(n)$. **In $O(\log n)$:** `upper_bound(v, x) - lower_bound(v, x)`, two binary searches that never walk.

Part E — Find and fix the bug

W8-B1. `while lo < hi` with an inclusive `hi` stops while one candidate, `values[lo]`, is still unchecked. `binary_search([42], 42)` returns –1, and so do `binary_search([1, 3], 3)` and `binary_search([1, 3, 5, 7, 9], 9)` — the tests for one element and for the last element fail. **Fix:** `while lo <= hi`.

W8-B2. `hi = mid` does not move past `mid`, which has just been ruled out. When the range is a single too-large element, `lo = hi = mid` and nothing changes: `binary_search([1, 3], 0)` and `binary_search([1, 3, 5], 4)` loop for ever. (It still finds values that are present, which is why it can survive a few tests.) **Fix:** `hi = mid - 1`.

W8-B3. Two bugs. The index returned from a call on `values[mid + 1:]` is an index into the **slice**: on `[1, 3, 5, 7, 9]` it returns 0 for 7 and 1 for 9 (right answers 3 and 4), while 1, 3 and 5 happen to come out right. And every slice copies: $\Theta(n)$ in total (W8-K2). **Fix:** pass `lo` and `hi` and index the original list — or add `mid + 1` to a non-negative result from the right half, which fixes the index but not the cost.

W8-B4. When `values[hi] == values[lo]` — for example `[5, 5, 5]`, or any range that has shrunk to equal values — the division is by zero: `ZeroDivisionError`. **Fix:** before computing pos, if `values[hi] == values[lo]`, return `lo` if it equals the target, else –1.

Part F — Write the code

W8-C1

```
def search_rotated(values, target):
    lo, hi = 0, len(values) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        value = values[mid]
        if value == target:
            return mid
        first = values[lo]
        if first <= value:                    # left half lo..mid is sorted
            if first <= target < value:
                hi = mid - 1
            else:
                lo = mid + 1
        else:                                 # right half mid..hi is sorted
            if value < target <= values[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1
```

Why one half is sorted: the rotation creates a single "drop" (from the largest value to the smallest). Cutting at `mid` puts that drop in at most one of the two halves, so the other is in order. For the sorted half, two comparisons tell whether the target lies inside it; if not, it can only be in the other half. Still one halving per step: $O(\log n)$.

W8-C2

```
def integer_sqrt(n):
    if n < 0:
        raise ValueError("no integer square root of a negative number")
    lo, hi = 0, n                             # the answer is in 0..n
    while lo < hi:
        mid = (lo + hi + 1) // 2              # round up, or lo = mid never
↪   moves
        if mid * mid <= n:
            lo = mid                          # mid is possible; the answer
↪   is >= mid
        else:
            hi = mid - 1
    return lo
```

The test "$r^2 \le n$" is true for every r up to the answer and false after it, and that is all binary search needs — a **monotonic** yes/no question, not a sorted list. Note the `+ 1` in `mid`: with `lo = mid` as an update, rounding down would loop for ever on `lo = 0, hi = 1` (the bug of W8-E2). $O(\log n)$ multiplications; for n = 10^{18} about 60.

W8-C3

```
def find_peak(values):
    lo, hi = 0, len(values) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if values[mid] < values[mid + 1]:       # rising: a peak lies to the
        ↪  right
            lo = mid + 1
        else:                                   # falling or flat: mid or left
            hi = mid
    return lo
```

Why half can go: if `values[mid] < values[mid + 1]`, walk right from `mid + 1`: the values either keep rising to the end — then the last element is a peak, because the outside counts as −∞ — or they fall somewhere, and the top before the fall is a peak. Either way a peak lies in `mid + 1..hi`. The same argument the other way keeps `lo..mid`. The list is not sorted, but the question "is there a peak on this side?" still has a guaranteed answer, and that is enough to halve.

W8-C4

```
def closest_value(values, target):
    lo, hi = 0, len(values)                     # lower_bound, half-open
    while lo < hi:
        mid = (lo + hi) // 2
        if values[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    if lo == len(values):
        return values[lo - 1]
    if lo == 0:
        return values[0]
    before, after = values[lo - 1], values[lo]
    return before if target - before <= after - target else after
```

`lo` is where the target would be inserted: every value before it is smaller, every value from it on is at least as large. So the closest value is one of the two neighbours of that position — or the only one, at either end. <= breaks a tie toward the smaller value.

W8-C5

```
def min_capacity(weights, days):
    def fits(capacity):
```

```
        needed, load = 1, 0
        for w in weights:
            if load + w > capacity:
                needed += 1
                load = 0
            load += w
        return needed <= days

    lo, hi = max(weights), sum(weights)
    while lo < hi:
        mid = (lo + hi) // 2
        if fits(mid):
            hi = mid                                # mid works; maybe smaller
↪   does too
        else:
            lo = mid + 1
    return lo
```

Why binary search: if a capacity works, every larger capacity works too — the answer to "does capacity c fit?" is *no, no, …, no, yes, yes, …*. The smallest "yes" is a `lower_bound` on that yes/no sequence, even though the sequence is never stored. The capacity lies between `max(weights)` (the heaviest package must fit) and `sum(weights)` (everything in one day). Each check is one greedy pass, $O(n)$, and there are $O(\log S)$ checks: $O(n \log S)$. Trying every capacity from the maximum upward would take $O(nS)$ — hopeless in the test, where S is 2×10^{10}.

Answers — Week 9

> ***The questions*** *are at the end of the* Week 9 *chapter. Commit to your own answer before reading one here. Every trace and count below was produced by running the reference code.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	c	M07	b	M13	a	M19	a
M02	a	M08	d	M14	b	M20	a
M03	d	M09	c	M15	c	M21	b
M04	a	M10	b	M16	c	M22	c
M05	c	M11	d	M17	b		
M06	d	M12	b	M18	d		

W9-M01 — c. Stability is about equal keys and their arrival order. (a) is "in place"; (d) is a property of selection sort.

W9-M02 — a. The course sorts return a new list, but they rearrange their working `Array` with $O(1)$ extra slots — so they are in place in this sense. (b) is what Python's `list.sort` does; it is not the definition.

W9-M03 — d. Its long-distance swap can jump an element over an equal one. Bubble and insertion sort move a value only past **strictly** greater ones; a merge that takes from the left on a tie is stable (Week 10).

W9-M04 — a. The first pass compares the 5 neighbour pairs, swaps nothing, and the early exit stops the sort: $n - 1 = 5$.

W9-M05 — c. $5 + 4 + 3 + 2 + 1 = 15 = n(n-1)/2$. Selection sort scans the whole rest in every round, sorted or not.

W9-M06 — d. The 9 is carried to the end; the pass swaps 5 and 2, then 9 and 1, 9 and 7, 9 and 3. (a) is the first round of **selection** sort; (c) is after the second pass.

W9-M07 — b. The minimum, 1 at index 3, is swapped with the 5 at index 0. (a) is bubble sort's first pass.

W9-M08 — d. Before that round the prefix is `[2, 5, 9]`; the 1 shifts all three right and lands at index 0.

W9-M09 — c. (3, 1) and (3, 2). The pair (1, 2) is in order.

W9-M10 — b. Each shift moves `current` past one larger value before it — one inverted pair — and changes the order of no other pair (W9-E3).

W9-M11 — d. One comparison per round and no shifts: $n - 1$ comparisons.

W9-M12 — b. At most $n - 1$ swaps, whatever the input. It is not stable (c), gains nothing from sorted input (a), and (d) is counting sort.

W9-M13 — a. At most one swap per round, and there are $n - 1$ rounds. 45 (b) is the number of comparisons.

W9-M14 — b. k = max + 1 = 5: one counter for each value 0, 1, 2, 3, 4.

W9-M15 — c. One pass to count (n), one over the counters (k), n writes.

W9-M16 — c. Sorting two values up to a million needs a million counters and a loop over all of them.

W9-M17 — b. A decision tree with at least $n!$ leaves has height at least $\log_2 n!$, which is $\Theta(n \log n)$.

W9-M18 — d. $4! = 24$ orders, and $2^4 = 16 < 24 \leq 32 = 2^5$: at least $\lceil \log_2 24 \rceil = 5$ comparisons in the worst case.

W9-M19 — a. Two different input orders must end at different leaves, and h yes/no answers can tell apart at most 2^h cases.

W9-M20 — a. Every frame holds a reference to the **same** list; by the time you look, it is sorted. `step_slider` hides the bug by copying each frame as it arrives; `list(generator)` does not.

W9-M21 — b. $n^2/2$ copies of n values each: $\Theta(n^3)$. The reference fixes it with `snapshot=_live` in the plain sort (Lecture 09, "The price of watching").

W9-M22 — c. Reversed input has every pair inverted: $6 \times 5/2 = 15$ inversions, and bubble sort makes one swap per inversion.

Part B — Short answer and essay

W9-E1 *(4)*

- **Stable:** elements with equal keys come out in the order they went in. **In place:** only $O(1)$ memory besides the array being sorted.
- **Bubble sort** swaps neighbours only when `a[j] > a[j + 1]` — strictly — so two equal values are never swapped and never pass each other: **stable**. **Insertion sort** shifts only values `> current`, so `current` stops at an equal key: **stable**. **Selection sort** swaps `a[i]` with a minimum far to its right, which can jump `a[i]` over an equal value: **not stable**. All three are in place.
- **Example:** cards `2a, 2b, 1c` compared by number. Round 0 swaps `1c` with `2a`: `1c, 2b, 2a` — `2a` and `2b` have changed order.

- **Two keys:** to sort by grade, and by name within a grade, sort by name first, then **stably** by grade. The second sort leaves students with equal grades in name order. An unstable second sort scrambles them.

W9-E2 *(4)*

	comparisons, best	comparisons, worst	swaps or shifts
bubble (early exit)	$n-1$ (sorted)	$n(n-1)/2$	I swaps (inversions)
selection	$n(n-1)/2$	$n(n-1)/2$	at most $n-1$ swaps
insertion	$n-1$ (sorted)	$n(n-1)/2$	I shifts

- **Insertion sort** for small or nearly sorted data, or when stability is needed: it adapts to the input ($O(n+I)$) and does about half the comparisons of bubble sort on random data.
- **Selection sort** when writes are expensive (large records, flash memory): it writes $O(n)$ times whatever the input.
- **Bubble sort** has no real niche — insertion sort is at least as good in every case. Its one merit is teaching: with the early exit it detects sorted input in one pass, and "neighbours swap" is the easiest sort to explain and to animate.

W9-E3 *(4)*

- **Inversion:** a pair of positions $i < j$ with $a[i] > a[j]$.
- **One shift per inversion:** when `current` is inserted, it moves left past exactly the values before it that are larger — each such pair was an inversion, and after the move it is not. The relative order of every other pair is unchanged (the shifted values keep their order among themselves). So each shift removes exactly one inversion, and the sort ends when none are left: shifts = I, and comparisons $\le I + n - 1$.
- **Cases:** sorted, I = 0: $\Theta(n)$. Reversed, $I = n(n-1)/2$: $\Theta(n^2)$. Random: each pair inverted with probability ½, so the expected I is $n(n-1)/4$ — $\Theta(n^2)$, half the worst.
- **Libraries:** real data is often nearly sorted, and short pieces are common inside divide-and-conquer sorts. On both, insertion sort is near-linear, has a tiny constant, needs no memory and is stable. Timsort (Python, Java) extends short runs with binary insertion sort; introsort (C++) finishes small pieces with insertion sort.

W9-E4 *(4)*

- **What it yields:** `(values, highlight)` — a snapshot of the array and the indices the next step concerns. Bubble sort yields once at the start, before every comparison, and once at the end.
- **Reuse:** the plain sort runs the generator to the end and keeps the last state (`_finish`). One implementation: the animation shows exactly the code the tests grade.
- **Rules:** each frame must be its **own** list (`list(a)`), or the frames all show the final state (`test_step_frames_are_independent_snapshots`); and the last frame must be sorted (`test_step_form_is_a_generator_ending_sorted`).
- **The price of watching:** a copy costs $O(n)$; made before each of $n^2/2$ comparisons, it makes the plain sort $\Theta(n^3)$ — measured, the time grows about 8× per doubling instead of 4×. **Fix:** a `snapshot` parameter, `list` by default for animations; the plain sort passes

`_live`, which returns the working `Array` without copying, and `_finish` makes one copy at the end.

W9-E5 *(4)*

- **Bound:** every sort that learns about its input **only by comparing** two elements needs at least $\lceil \log_2 n! \rceil$ comparisons in the worst case, and $\log_2 n! = \Theta(n \log n)$.
- **Argument:** draw the sort's comparisons as a binary decision tree; one run is a root-to-leaf path. Different input orders need different rearrangements, so they must reach different leaves: at least $n!$ leaves. A binary tree of height h has at most 2^h leaves, so $h \geq \log_2 n!$. Since $n! \geq (n/2)^{n/2}$, $\log_2 n! \geq \frac{n}{2} \log_2 \frac{n}{2}$. For n = 10: at least 22 comparisons.
- **Counting sort:** does not compare. `counts[v] += 1` uses the value as an address — one step with k outcomes, not 2 — so the bound, which is about comparison sorts only, does not apply. It pays with $O(k)$ memory and with keys restricted to small non-negative integers.
- **When:** integer keys whose range k is at most about n — marks, ages, letters, grades. Not for phone numbers, floats or strings.

Part C — Trace the algorithms

W9-T1

```
start          3  8  1  6  2
after pass 1   3  1  6  2  8    4 comparisons, 3 swaps
after pass 2   1  3  2  6  8    3 comparisons, 2 swaps
after pass 3   1  2  3  6  8    2 comparisons, 1 swap
after pass 4   1  2  3  6  8    1 comparison,  0 swaps: stop
```

10 comparisons and 6 swaps; the input has 6 inversions. Pass 4 is the last possible pass anyway, so the early exit saves nothing here.

W9-T2

Round	Minimum of the rest	Swap?	Array after
i = 0	10 at index 1	29 with 10	`[10, 29, 14, 37, 13]`
i = 1	13 at index 4	29 with 13	`[10, 13, 14, 37, 29]`
i = 2	14 at index 2	no	`[10, 13, 14, 37, 29]`
i = 3	29 at index 4	37 with 29	`[10, 13, 14, 29, 37]`

4 + 3 + 2 + 1 = 10 comparisons, 3 swaps.

W9-T3

```
start      8  4  6  2  9  1
i = 1      4  8  6  2  9  1    1 shift,  1 comparison
i = 2      4  6  8  2  9  1    1 shift,  2 comparisons
```

```
i = 3       2  4  6  8  9  1    3 shifts, 3 comparisons
i = 4       2  4  6  8  9  1    0 shifts, 1 comparison
i = 5       1  2  4  6  8  9    5 shifts, 5 comparisons
```

10 shifts and 12 comparisons. The inversions: 8 is before 4, 6, 2 and 1 (4); 4 before 2 and 1 (2); 6 before 2 and 1 (2); 2 before 1 (1); 9 before 1 (1) — **10**, the number of shifts. Rounds 1, 3 and 5 walk off the front, so they make as many comparisons as shifts; the others make one more, the comparison that stops the walk.

W9-T4

- **Bubble: 12.** One yield at the start, one before each comparison, one at the end. Reversed input swaps in every pass, so all four passes run: $4 + 3 + 2 + 1 = 10$ comparisons. $1 + 10 + 1 = 12$.
- **Selection: 14.** One at the start, one before each of the $4+3+2+1 = 10$ comparisons, one after each swap, one at the end. Round 0 swaps 5 with 1 (`[1, 4, 3, 2, 5]`), round 1 swaps 4 with 2 (`[1, 2, 3, 4, 5]`), rounds 2 and 3 swap nothing: 2 swaps. $1+10+2+1 = 14$.
- **Insertion: 12.** One at the start, one after each **shift**, one at the end. The shifts are the inversions: 10 in a reversed list of 5. $1 + 10 + 1 = 12$.

(Checked with `len(list(sorting.bubble_sort_steps([5, 4, 3, 2, 1])))` and its two siblings, on the reference solution.)

W9-T5

- k = 5 + 1 = **6**. `counts` = `[2, 0, 2, 3, 0, 1]` (indices 0 to 5): two 0s, no 1, two 2s, three 3s, no 4, one 5.
- Output: `[0, 0, 2, 2, 3, 3, 3, 5]`.
- **Starting positions** (a running sum of the counts before each value): 0 starts at 0, 1 at 2, 2 at 2, 3 at 4, 4 at 7, 5 at 7.
- The three 3s are placed in input order at positions 4, 5 and 6: the **first** 3 of the input (index 2) goes to **4**, the last (index 7) to **6**. Equal keys keep their order — that is the stability.

Part D — Array state

W9-S1 (shaded values in brackets)

```
start          6  5  3  1  8  7  2  4
after pass 1   5  3  1  6  7  2  4 [8]     7 comparisons, 6 swaps
after pass 2   3  1  5  6  2  4 [7  8]     6 comparisons, 4 swaps
after pass 3   1  3  5  2  4 [6  7  8]     5 comparisons, 3 swaps
after pass 4   1  3  2  4 [5  6  7  8]     4 comparisons, 2 swaps
after pass 5   1  2  3 [4  5  6  7  8]     3 comparisons, 1 swap
after pass 6  [1  2  3  4  5  6  7  8]     2 comparisons, 0 swaps: stop
```

6 passes, 27 comparisons, 16 swaps — the input has 16 inversions. The early exit saves the

seventh pass (1 comparison). After pass 5 the array is already sorted, but the sort cannot know that until a pass makes no swap.

W9-S2

```
frame 1   [5, 1, 4, 2, 8]   ()        the start
frame 2   [1, 5, 4, 2, 8]   (0, 1)    i = 1: 1 shifts past 5
frame 3   [1, 4, 5, 2, 8]   (1, 2)    i = 2: 4 shifts past 5
frame 4   [1, 4, 2, 5, 8]   (2, 3)    i = 3: 2 shifts past 5
frame 5   [1, 2, 4, 5, 8]   (1, 2)    i = 3: 2 shifts past 4
frame 6   [1, 2, 4, 5, 8]   ()        the end (i = 4: 8 does not move)
```

Six frames: the start, one per shift (4 shifts = 4 inversions), the end. Each frame is a permutation because the reference writes `current` into the hole after **every** shift (`a[j + 1] = current` inside the loop). A textbook insertion sort writes `current` once, after the walk, and a frame taken in between would show the shifted value twice — `[5, 5, 4, 2, 8]` — and the 1 missing.

W9-S3

Step	`values[mid]`	Action	Array after	lo	mid	hi
start			`[2, 0, 2, 1, 1, 0]`	0	0	5
1	2	swap with `hi`; `hi` - 1	`[0, 0, 2, 1, 1, 2]`	0	0	4
2	0	swap with `lo`; both + 1	`[0, 0, 2, 1, 1, 2]`	1	1	4
3	0	swap with `lo`; both + 1	`[0, 0, 2, 1, 1, 2]`	2	2	4
4	2	swap with `hi`; `hi` - 1	`[0, 0, 1, 1, 2, 2]`	2	2	3
5	1	`mid` + 1	`[0, 0, 1, 1, 2, 2]`	2	3	3
6	1	`mid` + 1	`[0, 0, 1, 1, 2, 2]`	2	4	3

`mid > hi`: the unseen region is empty — stop. Six steps for six values; each step shrinks the unseen region `values[mid..hi]` by one. In step 1, the value that arrives at `mid` from the back (a 0) has not been looked at, which is why `mid` does not advance.

Part E — Complexity analysis

W9-K1

- **Worst case $\Theta(n^2)$:** insertion sort on reversed (or random) input; the scan of neighbours is only $\Theta(n)$ after it.
- **Best case $\Theta(n)$:** sorted input with no duplicates — insertion sort makes $n-1$ comparisons, and the scan finds nothing in $n - 1$. (With a duplicate early in a sorted list, the scan can stop sooner, but the sort already cost $\Theta(n)$.)
- **With `counting_sort`**, values in 0 .. k - 1: $\Theta(n + k)$ — linear when k is $O(n)$. (A duplicate is then simply a count of 2 or more; the sort is not even needed.)
- **Two nested loops** over all pairs: $\Theta(n^2)$ in the worst case (no duplicates) — no better than the sort in the worst case, and it has no $\Theta(n)$ best case on duplicate-free input.

W9-K2

- Round i compares `a[smallest]` with each of `a[i+1..n-1]`: $n-1-i$ comparisons. Total $\sum_{i=0}^{n-2}(n-1-i) = (n-1)+\cdots+1 = n(n-1)/2$, **for every input**. Swaps: at most one per round — at most $n-1$.
- **Cost with a write = 100 comparisons:** selection $499{,}500 + 1{,}986 \times 100 = 698{,}100$; insertion $249{,}614 + 498{,}241 \times 100 = 50{,}073{,}714$. Selection sort is about **72 times cheaper**, although it makes twice the comparisons. Which sort is "faster" depends on what an operation costs.

W9-K3

- Let x be at index i, with sorted position p, $|i-p| \le k$. Let L be the number of larger values before x, and S the number of smaller values after it. Then $p = i - L + S$.
- If $S = 0$: $L = i - p \le k$.
- If $S > 0$: every smaller value after x has sorted position below p, so it sits at an index at most $p-1+k$ — all S of them in the indices $i+1..p+k-1$, so $S \le p+k-1-i$. Then $L = i - p + S \le i - p + (p+k-1-i) = k-1$.
- So every value has **at most k** larger values before it, shifts at most k times, and the sort makes $I \le nk$ shifts and at most $n(k+1)$ comparisons: $O(nk)$.
- **k = 1:** $O(n)$ — only neighbours are out of order. **k = n:** $O(n^2)$ — no information, the general worst case.

Part F — Find and fix the bug

W9-B1. `range(end + 1)` lets `j` reach `end`, and then `a[j + 1]` is `a[end + 1]` — past the end on the first pass, where `end = n - 1`: `IndexError: Array index 5 out of range for length 5` on `[5, 2, 9, 1, 7]`. It fails 10 of the 12 bubble tests; only `[]` and `[1]` pass, because their loops never run. **Fix:** `for j in range(end):` — the last pair of the pass is `(end - 1, end)`.

W9-B2. It swaps **every** time it meets a value smaller than the current `a[i]`, instead of remembering the index of the minimum and swapping once after the scan. It still sorts — after round i, `a[i]` holds the minimum of the rest — so the tests pass, but it throws away selection sort's only virtue. On `[6, 5, 4, 3, 2, 1]` it makes **15 swaps**; the real selection sort makes **3** (6 with 1, 5 with 2, 4 with 3). **Fix:** keep `smallest`, compare with `a[smallest]`, and swap `a[i]` with `a[smallest]` once, after the inner loop, only if they differ.

W9-B3. `a[j] >= current` walks `current` past values **equal** to it, so equal keys swap order: **not stable**. On plain numbers nobody can see it, so every test passes except `test_insertion_sort_is_stable`: `assert ['0d', '0b', '1c', '1a'] == ['0b', '0d', '1a', '1c']`. It also does extra shifts on every run of equal values. **Fix:** `a[j] > current`.

W9-B4. Two bugs.

- `k = max(values)` makes the counters 0 .. max - 1, so `counts[max]` does not exist:

`counting_sort([3, 1, 4, 1, 5, 0])` raises `IndexError: Array index 5 out of range for length 5`. (And the write-out loop, `range(k)`, would never write the largest value.) **Fix:** `k = max(values) + 1`.
- The empty list: `max([])` raises `ValueError: max() iterable argument is empty`. **Fix:** return `[]` first.

The reference also checks that every value is a non-negative `int`, and accepts `max_value` to skip the `max` pass.

Part G — Write the code

W9-C1

```
def count_inversions(values):
    n = len(values)
    count = 0
    for i in range(n):
        for j in range(i + 1, n):
            if values[i] > values[j]:
                count += 1
    return count
```

Every pair once: $n(n-1)/2$ comparisons, $\Theta(n^2)$. **Why it equals bubble sort's swaps:** a swap of two neighbours changes the order of exactly one pair — the two neighbours — and bubble sort swaps only when that pair is inverted. So each swap removes one inversion, and the sort ends with none: swaps = I. Week 10 counts inversions in $O(n \log n)$, inside merge sort.

W9-C2

```
def sort_by_key(items, key):
    out = list(items)
    for i in range(1, len(out)):
        current = out[i]
        k = key(current)
        j = i - 1
        while j >= 0 and key(out[j]) > k:  # strictly greater: stable
            out[j + 1] = out[j]
            j -= 1
        out[j + 1] = current
    return out
```

Why keys only: the items may not be comparable at all — the test sorts `Record` objects with no `<` — and even when they are, comparing whole items would sort by something other than the key: two tuples with the same key would be ordered by their second field, and the stability test could not tell. `>` (not `>=`) is what keeps it stable. It calls `key` once per comparison; a faster version computes each key once and keeps it beside its item.

W9-C3

```
def dutch_flag(values):
    lo, mid, hi = 0, 0, len(values) - 1
    # values[:lo] are 0, values[lo:mid] are 1, values[hi+1:] are 2
    while mid <= hi:
        v = values[mid]
        if v == 0:
            values[lo], values[mid] = values[mid], values[lo]
            lo += 1
            mid += 1
        elif v == 1:
            mid += 1
        else:
            values[mid], values[hi] = values[hi], values[mid]
            hi -= 1  # do not advance mid: new value unseen
```

Why `mid` stays: the value swapped in from `hi` comes from the unseen region; it could be a 0, a 1 or a 2, and must be looked at next. The value swapped in from `lo`, by contrast, is a 1 (or `lo == mid`), already seen — so both can advance. Each step shrinks `values[mid..hi]` by one: at most n steps, $O(n)$, and $O(1)$ extra space. (Counting the 0s, 1s and 2s and writing them back is also $O(n)$ and passes the tests, but takes two passes — and would not work if the 0s, 1s and 2s were keys of records with other data attached; the Dutch flag moves the records themselves.)

W9-C4

```
def sort_k_sorted(values, k):
    out = list(values)
    for i in range(1, len(out)):
        current = out[i]
        j = i - 1
        while j >= 0 and out[j] > current:
            out[j + 1] = out[j]
            j -= 1
        out[j + 1] = current
    return out
```

It is plain insertion sort: k is not used by the code, only by the analysis. Each value has at most k larger values before it (W9-K3), so each round makes at most k shifts and k + 1 comparisons: at most $n(k+1)$ comparisons — exactly the bound the test checks, on 2,000 values with k = 3 (the reference makes 3,456; an $O(n^2)$ sort would make about a million).

W9-C5

```
def counting_sort_by_key(items, key, k):
    counts = Array(k, fill=0)
    for item in items:
        kk = key(item)
```

```
        if not 0 <= kk < k:
            raise ValueError(f"key {kk!r} is outside 0..{k - 1}")
        counts[kk] += 1
    start = 0  # counts[v] becomes where v starts
    for v in range(k):
        counts[v], start = start, start + counts[v]
    out = Array(len(items))
    for item in items:  # input order: this is the stability
        kk = key(item)
        out[counts[kk]] = item
        counts[kk] += 1
    return list(out)
```

Three passes: count the keys; turn each count into the **starting position** of its key (a running sum — for banana, a starts at 0, b at 3, n at 4); then place each item at the next free position of its key. $O(n+k)$ time, $O(n+k)$ extra space, and no item is ever compared with another. **Why input order:** items with the same key are placed at consecutive positions in the order the loop meets them. Going through the input front to back puts the first one first — stable. Going back to front would reverse every group of equal keys. This stable counting sort is the step that radix sort repeats once per digit.

Answers — Week 10

> ***The questions*** *are at the end of the* Week 10 *chapter. Commit to your own answer before reading one here. Every trace and count below was produced by running the reference* `dsa/sorting.py`*, and comparisons were counted with a value class whose* `__lt__`*,* `__le__`*,* `__gt__` *and* `__ge__` *add to a counter.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	d	M07	b	M13	a	M19	b
M02	b	M08	a	M14	b	M20	b
M03	a	M09	b	M15	c	M21	d
M04	c	M10	c	M16	d	M22	a
M05	d	M11	c	M17	a		
M06	c	M12	d	M18	c		

W10-M01 — d. Comparing every pair is what $O(n^2)$ sorts do; divide and conquer is exactly the way to avoid it.

W10-M02 — b. Two sorts of half the size, plus a merge that touches all n. (a) is binary search; (c) is quicksort's worst case; (d) would be a divide and conquer whose combine step is $O(1)$.

W10-M03 — a. Each comparison sends one element to `scratch`, and the last element needs none: at most $4 + 4 - 1 = 7$, when the two runs interleave to the end (for example `[1, 3, 5, 7]` and `[2, 4, 6, 8]`).

W10-M04 — c. The `<=` in `if a[i] <= a[j]`. With `<` the sort is still correct but no longer stable (W10-B1). Where it splits (a) has nothing to do with it.

W10-M05 — d. One scratch `Array` of n slots, shared by every merge, plus an $O(\log n)$ call stack — $O(n)$ in total.

W10-M06 — c. Every merge joins two runs into one; starting from 8 runs of one element, 7 merges leave one run. In general $n - 1$.

W10-M07 — b. On sorted input each merge of two runs of m elements empties the left run after m comparisons: $\frac{n}{2}\log_2 n = 512 \times 10 = 5{,}120$ (measured). 523,776 (d) is quicksort with `pivot="first"`; 1,023 (a) is Timsort.

W10-M08 — a. The smaller-than-pivot values sit in `a[lo:boundary]`, the others in `a[boundary:j]`; `a[j:hi]` is still unknown, so (d) claims too much.

W10-M09 — b. The pivot is 4; only 3, 1 and 2 are smaller, so it lands at index 3: `[3, 1, 2, 4, 8, 9, 7, 6]`.

W10-M10 — c. One comparison per non-pivot element, $n - 1 = 7$, whatever the order.

W10-M11 — c. Every pivot is the smallest of its range, so each partition peels off one element: $(n-1) + (n-2) + \cdots + 1$. For n = 1,024 that is 523,776 comparisons, measured.

W10-M12 — d. Of the first, middle and last values of a sorted range, the middle one is the median — the exact middle of the range. Median-of-three does cost comparisons (a), and it is not random (b).

W10-M13 — a. Each recursive call is on at most half of its caller's range, so there are at most $\log_2 n$ frames. The **time** is still $O(n^2)$ in the worst case (b). On sorted input with `"first"` the stack is $O(1)$, but not on every input (d).

W10-M14 — b. A swap in the partition can move an element past its equal, and nothing puts them back. Recursion (a) and in-place work do not by themselves break stability.

W10-M15 — c. With the root at index 0. (b) is the layout with the root at index 1, used in some textbooks.

W10-M16 — d. The sifts at indices 3, 2, 1, 0 give `[4, 10, 3, 7, 1, 8, 2, 5]`, `[4, 10, 8, 7, 1, 3, 2, 5]`, unchanged, and `[10, 7, 8, 5, 1, 3, 2, 4]` (W10-T4). A heap is not sorted (a, b).

W10-M17 — a. About n/4 nodes can sink 1 level, n/8 can sink 2, … — the sum $n(1/4 + 2/8 + 3/16 + \dots)$ is at most n. Measured: about 1.9 n comparisons, the same ratio from n = 1,000 to 8,000.

W10-M18 — c. From the last parent, `n // 2 - 1 = 4`, down to 0. Indices 5 to 9 are leaves. Going up (d) is W10-B2's bug.

W10-M19 — b. Merge sort (a) needs $O(n)$ extra; quicksort (c) is $O(n^2)$ in the worst case; insertion sort (d) is $O(n^2)$.

W10-M20 — b. The argument counts yes/no answers to comparisons. Counting sort (d) never compares and runs in $O(n + k)$.

W10-M21 — d. Timsort, by Tim Peters (2002): natural runs, insertion sort on short runs, stable merging. Introsort (c) is C++'s `std::sort`.

W10-M22 — a. Timsort finds one ascending run of 1,000 in 999 comparisons and has nothing to merge (measured: 999).

Part B — Short answer and essay

W10-E1 *(5)*

- **Divide:** split `a[lo:hi]` at `mid = (lo + hi) // 2` — no comparisons. **Conquer:** sort both halves recursively; a range of 0 or 1 elements is sorted (base case). **Combine:** merge

the two sorted runs with `_merge`.
- **Recurrence:** $T(n) = 2T(n/2) + cn$, $T(1) = c$.
- **Recursion tree:** the root merges n elements; its two children n/2 each; the four grandchildren n/4 each — every level totals n. The sizes halve, so there are $\log_2 n$ levels that merge: $T(n) = O(n \log n)$, in every case, because the split never depends on the values.
- **Space:** one scratch `Array` of n slots, plus $\log_2 n$ stack frames: $O(n)$. Not in place.
- **Stable:** on a tie, `_merge` takes from the **left** run (`<=`), and the left run holds the elements that came first in the input.

W10-E2 *(4)*

- Lomuto's partition takes the pivot `a[hi]` and one scan `j = lo..hi-1`. Values smaller than the pivot are swapped to the front, into `a[boundary]`, and `boundary` grows. A final swap puts the pivot at `a[boundary]`; it returns `boundary`.
- **Invariant:** `a[lo:boundary] < pivot <= a[boundary:j]`. True at the start (both empty), kept by each step, and at the end `j = hi`, so every value is on the correct side.
- **Final place:** every value smaller than the pivot is to its left, every other value to its right; sorting either side cannot move any value across, so the pivot's position is its position in the sorted array.
- **No combine:** after both parts are sorted, *left part, pivot, right part* is already in order — the partition did the combining in advance. Merge sort is the mirror image: free divide, expensive combine.

W10-E3 *(4)*

- `"first"` and `"last"`: $O(1)$ to choose. Worst on **sorted** and reverse-sorted input: the pivot is the extreme of its range every time, one element per level, $n(n-1)/2$ comparisons.
- `"random"`: $O(1)$ plus a random number. **No fixed input** is bad for it: the worst case needs bad luck at almost every level, which is vanishingly unlikely; the expected cost is about $1.39\, n \log_2 n$.
- `"median3"`: up to three extra comparisons per partition. Exact middle on sorted and reversed input. Its worst case exists — specially built "median-of-three killer" inputs (Musser, 1997) — but it is not an input that occurs by accident.
- **Why sorted input matters:** it is the most common input a real sort sees — data sorted before, with a few changes. A pivot rule that is worst on sorted input is worst exactly when it matters.

W10-E4 *(4)*

- **Array as tree:** `a[0]` is the root; the children of `a[i]` are `a[2i + 1]` and `a[2i + 2]`; the parent is `a[(i - 1) // 2]`. The tree is complete, height $\lfloor \log_2 n \rfloor$; there are no pointers.
- **Build-heap:** sift down every parent from `n // 2 - 1` down to 0; each sift may assume both subtrees are already heaps. **Sort:** swap the root (the max) with `a[end]`, shrink the heap to `a[0:end]`, sift the new root down; repeat for `end = n - 1` … `1`. $O(n \log n)$.
- **Build-heap is $O(n)$:** a node at height h sinks at most h levels, and there are at most $n/2^{h+1}$ such nodes: $\sum_h h\, n/2^{h+1} \le n$. Most nodes are leaves or near them.
- **Slower in practice:** the sift jumps from `i` to `2i + 1` — far apart in memory, so caches help little — and it makes more comparisons (measured on random data: about 1.7 n $\log_2$ n,

against about 1.2 for median-of-three quicksort). Its strength is the guarantee: $O(n \log n)$ worst case with $O(1)$ extra space.

W10-E5 *(3)*

- **Stable:** elements with equal keys come out in the order they went in.
- **Why:** sorting records by one field after another. Sort students by name, then **stably** by grade: within a grade they stay in name order.
- **In `dsa/sorting.py`:** bubble, insertion and merge sort are stable; selection, quick and heap sort are not.
- **Tuples do not test it:** Python compares `(key, tag)` tuples by the tag when the keys are equal, so every correct sort — stable or not — puts `(1, "a")` before `(1, "c")`. A stability test needs values that compare by key **only**, such as a class whose comparison methods look at the key alone.

Part C — Trace the algorithms

W10-T1

i	j	k	compare	copied to scratch[k]
0	4	0	3 > 1	1 (right)
0	5	1	3 <= 7	3 (left)
1	5	2	7 <= 7	**7 from the left**
2	5	3	8 > 7	7 (right)
2	6	4	8 <= 10	8 (left)
3	6	5	12 > 10	10 (right)

Now `j = 7 = hi`: the right run is empty. The **left** leftover loop copies 12 into `scratch[6]`; the right one does nothing. Six comparisons; the result is `[1, 3, 7, 7, 8, 10, 12]`. The 7 from the **left** run (index 1) goes first, because `7 <= 7` is true: that is stability.

W10-T2

j	a[j]	< 6?	array after	boundary
0	5	yes: swap a[0], a[0]	`[5, 8, 1, 9, 3, 7, 2, 6]`	1
1	8	no	`[5, 8, 1, 9, 3, 7, 2, 6]`	1
2	1	yes: swap a[1], a[2]	`[5, 1, 8, 9, 3, 7, 2, 6]`	2
3	9	no	`[5, 1, 8, 9, 3, 7, 2, 6]`	2
4	3	yes: swap a[2], a[4]	`[5, 1, 3, 9, 8, 7, 2, 6]`	3
5	7	no	`[5, 1, 3, 9, 8, 7, 2, 6]`	3
6	2	yes: swap a[3], a[6]	`[5, 1, 3, 2, 8, 7, 9, 6]`	4
end		swap a[4], a[7]	`[5, 1, 3, 2, 6, 7, 9, 8]`	returns **4**

Seven comparisons, four swaps plus the final one. The pivot 6 is at index 4: four smaller values to its left, three larger to its right.

W10-T3

#	range	pivot	q	array after
1	0..7	4	3	`[3, 1, 2, 4, 8, 9, 7, 6]`
2	0..2	2	1	`[1, 2, 3, 4, 8, 9, 7, 6]`
3	4..7	6	4	`[1, 2, 3, 4, 6, 9, 7, 8]`
4	5..7	8	6	`[1, 2, 3, 4, 6, 7, 8, 9]`

After partition 1 the left part `0..2` has 3 elements and the right part `4..7` has 4, so the left — the **smaller** — is a recursive call (partition 2, whose parts `0..0` and `2..2` have one element each and need nothing), and the right is the next turn of the loop (partition 3). There, the left part is empty, so the loop continues with `5..7` (partition 4), whose parts `5..5` and `7..7` are single elements. Four partitions; the reference yields frames highlighting 3, 1, 4 and 6.

W10-T4

step	array
input	`[4, 10, 3, 5, 1, 8, 2, 7]`
sift index 3 (5; child 7)	`[4, 10, 3, 7, 1, 8, 2, 5]`
sift index 2 (3; children 8, 2)	`[4, 10, 8, 7, 1, 3, 2, 5]`
sift index 1 (10; children 7, 1)	`[4, 10, 8, 7, 1, 3, 2, 5]` — no change
sift index 0 (4 sinks: 10, then 7, then 5)	`[10, 7, 8, 5, 1, 3, 2, 4]` — the heap
swap a[0], a[7]; sift in a[0:7]	`[8, 7, 4, 5, 1, 3, 2, 10]`
swap a[0], a[6]; sift in a[0:6]	`[7, 5, 4, 2, 1, 3, 8, 10]`
swap a[0], a[5]; sift in a[0:5]	`[5, 3, 4, 2, 1, 7, 8, 10]`

After three steps the three largest, 7, 8 and 10, are in their final places at the back, and `a[0:5]` is a max-heap of the rest.

W10-T5 — **12, 12, 15 and 12** comparisons (measured).

- **Sorted** `[1 … 8]`: in every merge the whole left run is smaller, so it empties after as many comparisons as it has elements, and the right run is copied free: $4 \times 1 + 2 \times 2 + 1 \times 4 = 12$.
- `[5, 6, 7, 8, 1, 2, 3, 4]`: both halves are sorted inside (4 + 4 comparisons), and in the last merge the **right** run is smaller throughout, so it empties after 4: 12.
- **Reversed**: the mirror image of sorted — in every merge the right run empties first: 12.
- `[1, 3, 5, 7, 2, 4, 6, 8]`: the last merge alternates between the runs to the very end, $4 + 4 - 1 = 7$ comparisons, on top of 4 + 4: 15.

Merge sort's **moves** are the same for all four; only the comparisons depend on how the runs interleave. The worst case for n = 8 is $4 \times 1 + 2 \times 3 + 1 \times 7 = 17$.

Part D — Array state

W10-S1

```
                  [6 5 3 1 8 7 2 4]
          [6 5 3 1]                 [8 7 2 4]
      [6 5]         [3 1]       [8 7]         [2 4]
    [6]   [5]     [3]   [1]   [8]   [7]     [2]   [4]
      (1)[5 6]     (2)[1 3]     (4)[7 8]     (5)[2 4]
           (3)[1 3 5 6]               (6)[2 4 7 8]
                    (7)[1 2 3 4 5 6 7 8]
```

The merges happen in the order (1) to (7): the left half is sorted completely before the right half is touched. The array after each merge, as `merge_sort_steps` yields it:

merge	range	array	
1	a[0:2]	`[5, 6, 3, 1, 8, 7, 2, 4]`	
2	a[2:4]	`[5, 6, 1, 3, 8, 7, 2, 4]`	
3	a[0:4]	`[1, 3, 5, 6, 8, 7, 2, 4]`	
4	a[4:6]	`[1, 3, 5, 6, 7, 8, 2, 4]`	
5	a[6:8]	`[1, 3, 5, 6, 7, 8, 2, 4]`	— already in order
6	a[4:8]	`[1, 3, 5, 6, 2, 4, 7, 8]`	
7	a[0:8]	`[1, 2, 3, 4, 5, 6, 7, 8]`	

14 comparisons in all.

W10-S2

#	range	candidates (first, middle, last)	pivot	array after	then
1	0..8	9, 7, 5	7	`[1, 2, 5, 3, 6, 4, 7, 9, 8]`	right `7..8` (2) is smaller: recurse; loop on `0..5`
2	7..8	9, 9, 8	9 (index 7)	`[1, 2, 5, 3, 6, 4, 7, 8, 9]`	both parts trivial
3	0..5	1, 5, 4	4	`[1, 2, 3, 4, 6, 5, 7, 8, 9]`	right `4..5` (2) is smaller: recurse; loop on `0..2`
4	4..5	6, 6, 5	6 (index 4)	`[1, 2, 3, 4, 5, 6, 7, 8, 9]`	both parts trivial

#	range	candidates (first, middle, last)	pivot	array after	then
5	0..2	1, 2, 3	2	`[1, 2, 3, 4, 5, 6, 7, 8, 9]`	both parts one element: done

In partition 1, the pivot 7 (index 4) is swapped to the end first, giving `[9, 1, 8, 2, 5, 3, 6, 4, 7]`; the scan then moves 1, 2, 5, 3, 6 and 4 to the front. In 2 and 4, the first and middle candidates are the same slot, and the tie picks the middle index. The reference yields frames highlighting 6, 8, 3, 5 and 1 — the order above.

W10-S3

```
after build-heap          [9, 7, 8, 3, 1, 4, 5, 2]
            9
         7     8
        3 1   4 5
       2

step 1: swap 9 and 2, sift  [8, 7, 5, 3, 1, 4, 2 | 9]
            8
         7     5
        3 1   4 2

step 2: swap 8 and 2, sift  [7, 3, 5, 2, 1, 4 | 8, 9]
            7
         3     5
        2 1   4

step 3: swap 7 and 4, sift  [5, 3, 4, 2, 1 | 7, 8, 9]
            5
         3     4
        2 1
```

Build-heap: index 3 ($7 \geq 3$) and index 1 ($9 \geq 7, 1$) do not move; index 2 swaps 4 with 8; index 0 sinks 2 past 9, 7 and 3. In each sort step the root is swapped with the last slot of the heap (after the bar, sorted and final), and the new root sinks to the larger child each time.

Part E — Complexity analysis

W10-K1

- (a) $\Theta(n \log n)$ — merge sort, every case; quicksort's best case (perfect halves). $\log_2 n$

levels of n work.

- (b) $\Theta(n^2)$: $T(n) = (n-1) + (n-2) + \cdots + 1 = n(n-1)/2$. Quicksort's worst case — every pivot the extreme of its range.
- (c) $\Theta(n)$: $n + n/2 + n/4 + \cdots < 2n$. Quickselect (W10-C3) when every pivot halves the range: only **one** side is kept, so the work shrinks geometrically instead of staying n per level.

W10-K2 — $T(n) = T(n-1) + (n-1)$, $T(1) = T(0) = 0$: the partition of n elements costs $n-1$ comparisons and leaves an empty left part and $n-1$ elements on the right. So $T(n) = \sum_{m=1}^{n-1} m = n(n-1)/2$ exactly — 4,950 for n = 100, 523,776 for n = 1,024 (measured). **Stack depth:** with the smaller-side recursion, the smaller part is always the empty left one, so each call returns at once: $O(1)$ extra frames. Without it — two recursive calls — the depth is n – 1: `list(range(1000))` raises `RecursionError` (W10-B3).

W10-K3 — with n = 1,000,000, $\log_2 n \approx 20$:

Method	Cost	k = 1	k = 10	k = n/2
(1) sort, take k	$O(n \log n)$	2×10^7	2×10^7	2×10^7
(2) k selection passes	$O(nk)$	10^6	10^7	5×10^{11}
(3) heap, pop k	$O(n + k \log n)$	2×10^6	$\approx 2 \times 10^6$	$\approx 2.2 \times 10^7$

k = 1: one pass (2) — the maximum, $n-1$ comparisons. **k = 10:** the heap (3) — a tenth of the cost of (2), and ten times cheaper than sorting. **k = n/2:** sorting (1) and the heap are the same order, $O(n \log n)$; the $O(nk)$ method is hopeless. (A fourth way, quickselect for the k-th largest and then sorting the k above it, is $O(n + k \log k)$.)

Part F — Find and fix the bug

W10-B1. `<` in place of `<=`: on a tie the element from the **right** run is taken first, so equal keys come out in the wrong order — merge sort is no longer stable. The numbers are still sorted, which is why all 37 tests pass: they sort plain numbers, where two equal values cannot be told apart. With cards compared by key only, `[2a, 1b, 2c, 1d]` sorts to `[1d, 1b, 2c, 2a]` instead of `[1b, 1d, 2a, 2c]` (measured). **Fix:** `if a[i] <= a[j]:`.

W10-B2. Going **up** from index 0 sifts a parent before its subtrees are heaps, so a large value deep in the tree cannot reach the root. On `[1, 2, 3, 4, 5]` (n = 5): sifting index 0 swaps 1 with 3 → `[3, 2, 1, 4, 5]`; sifting index 1 swaps 2 with 5 → `[3, 5, 1, 4, 2]`. The root 3 is smaller than its child 5: not a heap. The sort phase then puts 3 at the back as if it were the maximum; the reference with this bug returns `[1, 2, 4, 5, 3]`. (Only 2 of the 37 tests catch it.) **Fix:** `for index in range(n // 2 - 1, -1, -1):`.

W10-B3. Both parts are recursive calls. On sorted input with `pivot="first"`, every partition leaves an empty left part and all the rest on the right, so the recursion is n deep — one generator

frame per element — and `quick_sort(list(range(1000)), pivot="first")` raises `RecursionError: maximum recursion depth exceeded` (measured: 500 still works). The tests do not see it: their worst-case test uses 200 elements and the default `"median3"`. **Fix:** recurse into the smaller part only, and loop on the larger:

```
def sort(lo, hi):
    while lo < hi:
        p = _choose_pivot(a, lo, hi, pivot, rng)
        a[p], a[hi] = a[hi], a[p]
        q = _partition(a, lo, hi)
        yield snapshot(a), (q,)
        if q - lo < hi - q:
            yield from sort(lo, q - 1)
            lo = q + 1
        else:
            yield from sort(q + 1, hi)
            hi = q - 1
```

W10-B4. It returns `[5, 2, 9, 1, 7]` — unchanged. `sort` is a generator (it contains `yield`), so `sort(lo, mid)` only creates a generator object and throws it away: the halves are never sorted. Only the top-level call runs, and it merges `[5, 2]` with `[9, 1, 7]`, which are not sorted runs: 5 and 2 are taken first (both below 9), then the rest is copied as it is. **Fix:** `yield from sort(lo, mid)` and `yield from sort(mid, hi)`.

Part G — Write the code

W10-C1

```
def merge_k_sorted(lists):
    if not lists:
        return []
    runs = [list(run) for run in lists]
    while len(runs) > 1:
        merged = []
        for i in range(0, len(runs) - 1, 2):
            merged.append(_merge_two(runs[i], runs[i + 1]))
        if len(runs) % 2 == 1:
            merged.append(runs[-1])                # the odd one out waits a
 ↪  round
        runs = merged
    return runs[0]


def _merge_two(left, right):
    out = []
```

```
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:                    # a tie takes from the left
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
    out.extend(left[i:])
    out.extend(right[j:])
    return out
```

Why pairs: merging one list after another into a growing result makes the i-th merge cost about the size of everything so far — $O(Nk)$ in all (measured on 64 lists of 64: about 131,000 comparisons). Merging in **pairs**, round after round, is merge sort's recursion tree seen bottom-up: each round touches every value once and halves the number of lists, so there are $\lceil \log_2 k \rceil$ rounds of $O(N)$ — 24,452 comparisons on the same input. Ties take from the earlier list, so it is stable.

W10-C2

```
def count_inversions(values):
    def sort(items):
        if len(items) <= 1:
            return list(items), 0
        mid = len(items) // 2
        left, inside_left = sort(items[:mid])
        right, inside_right = sort(items[mid:])
        merged, i, j, across = [], 0, 0, 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
                j += 1
                across += len(left) - i            # right[j] jumps over all
↪   of these
        merged.extend(left[i:])
        merged.extend(right[j:])
        return merged, inside_left + inside_right + across

    return sort(list(values))[1]
```

Why `len(left) - i`: when `right[j]` is taken, it is smaller than `left[i]` — and, since `left` is sorted, smaller than every value still waiting in `left[i:]`. All of those came **before** it in the input, so each is one inversion, counted at once. Inversions inside each half are counted by the recursive

calls; the merge counts the ones across the middle. $O(n \log n)$; the reversed list of 4,096 values, with 8,386,560 inversions, takes 24,576 comparisons. (The slices copy, but only $O(n)$ per level — the order is unchanged.)

W10-C3

```
def kth_smallest(values, k):
    if not 1 <= k <= len(values):
        raise ValueError("k must be between 1 and len(values)")
    a = list(values)
    rng = random.Random(10)
    lo, hi, target = 0, len(a) - 1, k - 1
    while True:
        p = rng.randint(lo, hi)
        a[p], a[hi] = a[hi], a[p]
        pivot, boundary = a[hi], lo
        for j in range(lo, hi):                    # Lomuto, as in quick sort
            if a[j] < pivot:
                a[boundary], a[j] = a[j], a[boundary]
                boundary += 1
        a[boundary], a[hi] = a[hi], a[boundary]
        if boundary == target:
            return a[boundary]
        if boundary < target:
            lo = boundary + 1                      # the answer is on the right
        else:
            hi = boundary - 1                      # the answer is on the left
```

Why $O(n)$: after a partition, the answer's position `k - 1` is on **one** side, and only that side is searched further. With good pivots the work is $n + n/2 + n/4 + \cdots < 2n$ (W10-K1 (c)); with random pivots the expected total is at most about $3.4n$ for the median — measured: 11,326 comparisons for n = 4,096, about 2.8n. Quicksort must sort **both** sides, so its levels each cost n. The worst case is still $O(n^2)$, and many equal values push towards it, because the strict `<` sends every copy of the pivot to one side; a three-way partition (W10-C4) fixes that.

W10-C4

```
def sort_colours(values):
    low, mid, high = 0, 0, len(values) - 1
    while mid <= high:                             # values[mid..high] is
    ↪ unknown
        colour = values[mid]
        if colour == 0:
            values[low], values[mid] = values[mid], values[low]
            low += 1
            mid += 1
        elif colour == 1:
            mid += 1
```

```
        else:
            values[mid], values[high] = values[high], values[mid]
            high -= 1                                  # do not advance mid:
↪   re-check
```

Invariant: `values[0:low]` are 0s, `values[low:mid]` are 1s, `values[high+1:]` are 2s, and `values[mid..high]` is still unknown. A 0 is swapped to the end of the 0s — the value it swaps with is a 1 (or itself), so `mid` may advance; a 2 is swapped to the front of the 2s, and the value that comes back is unknown, so `mid` stays and it is checked next. Each step shrinks the unknown region by one: one pass, $O(n)$, $O(1)$ extra — at most n swaps. This is Dijkstra's "Dutch national flag" problem, and it is the three-way partition that fixes quicksort on many equal keys.

W10-C5

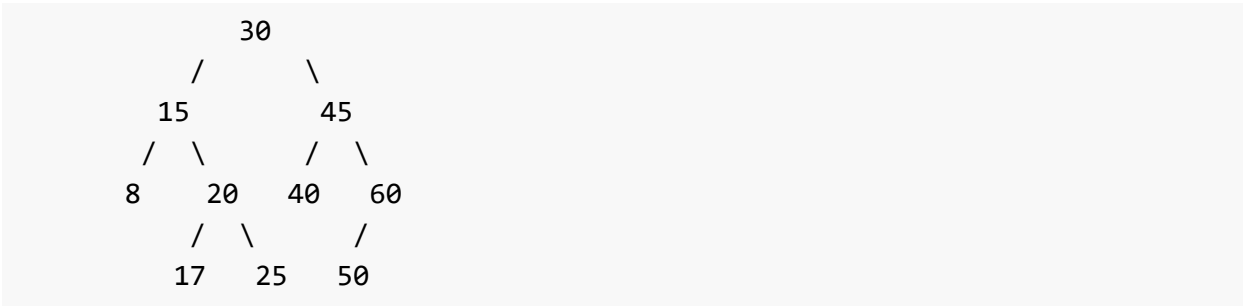
```
def merge_sort_bottom_up(values):
    a = list(values)
    n = len(a)
    scratch = [None] * n
    width = 1
    while width < n:
        for lo in range(0, n, 2 * width):           # merge a[lo:mid] and
        ↪  a[mid:hi]
            mid = min(lo + width, n)
            hi = min(lo + 2 * width, n)
            i, j = lo, mid
            for k in range(lo, hi):
                if j >= hi or (i < mid and a[i] <= a[j]):
                    scratch[k] = a[i]               # left run: on a tie too
                    i += 1
                else:
                    scratch[k] = a[j]
                    j += 1
        a, scratch = scratch, a                     # the merged pass becomes
↪   the input
        width *= 2
    return a
```

Passes: width 1, 2, 4, … while `width < n`: $\lceil \log_2 n \rceil$ passes, each reading and writing every element once — $O(n \log n)$, the same recursion tree as Lecture 10, walked from the leaves up instead of from the root down. **Short runs:** `mid` and `hi` are capped at n, so the last pair of a pass may have a short right run, or none at all (`mid == hi`); then the loop simply copies the left run across. For n = 3,000 (12 passes) it made 31,192 comparisons, under $n\lceil \log_2 n \rceil$ = 36,000. The condition `j >= hi or (i < mid and a[i] <= a[j])` takes from the left when the right run is used up, and on a tie — stable. Swapping `a` and `scratch` after each pass avoids copying back: the merged pass simply becomes the next pass's input. No recursion means no call stack at all.

Answers — Week 11

> ***The questions*** *are at the end of the* Week 11 *chapter. Commit to your own answer before reading one here. Every walk, tree and count below was checked by running the reference* `dsa/tree.py`.

The tree **Q**, built from `[30, 15, 45, 8, 20, 40, 60, 17, 25, 50]`:



Height 3; leaves 8, 17, 25, 40 and 50.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M07	c	M13	b	M19	b
M02	c	M08	d	M14	a	M20	a
M03	d	M09	d	M15	b	M21	c
M04	a	M10	b	M16	c	M22	d
M05	b	M11	a	M17	d		
M06	a	M12	c	M18	a		

W11-M01 — b. No children. (a) is the root; a node with one child (c) is internal.

W11-M02 — c. Height counts edges, and a single node has none. -1 (a) is the **empty** tree, which makes $1 + \max(-1, -1) = 0$ work for a leaf.

W11-M03 — d. Levels 0 to 3 hold at most 1 + 2 + 4 + 8 = 15 = $2^{h+1} - 1$. 7 (a) is height 2; 8 (b) is the size of level 3 alone.

W11-M04 — a. $\lfloor \log_2 100 \rfloor = 6$: height 6 holds up to 127 nodes, height 5 only 63. 99 (d) is the **tallest** possible height, a chain.

W11-M05 — b. The property is about whole subtrees. (a) is the weaker parent–child check, which W11-M20 shows is not enough; (c) is the heap property of Week 12.

W11-M06 — a. The root is 30, its left child 15; 20 > 15 went right. 17 and 25 are 20's children; 8 is 15's left child.

W11-M07 — c. 30 (go left), 15 (go right), 20 (go right), 25 (found).

W11-M08 — d. The cost is $O(h)$, and nothing in `dsa/tree.py` keeps h small: sorted input makes h = $n - 1$. $O(\log n)$ (b) is true only while the tree stays short — on average for random input, or always for a balanced tree.

W11-M09 — d. 7 first, then 1, then each value between the last two: every node gets one child, a zigzag chain of height 6. (a) and (b) build the perfect tree of height 2 (W11-S2); (c) has height 3.

W11-M10 — b. Node, then the whole left subtree, then the right. (a) is in-order, (c) level-order, (d) post-order.

W11-M11 — a. Both subtrees before the node, so the root 30 is last. (c) is not any of the walks; (d) visits 8 after 17 and 25, which are in a different subtree.

W11-M12 — c. Left subtree (all smaller), node, right subtree (all larger): sorted, for every BST.

W11-M13 — b. Pre-order lists every node before its descendants, so re-inserting it sends each value down the same path to the same place. The in-order walk (a) is sorted and rebuilds a **chain**; post-order (c) inserts leaves first, so the first value inserted — a leaf — becomes the root.

W11-M14 — a. Post-order finishes a node's children before the node: free the children while you still hold the links to them, and apply an operator only after both operands have values.

W11-M15 — b. Recursion is last in, first out: it finishes the most recent node's subtree before returning. Level-order needs first in, first out — the nodes in the order they were discovered. Memory (a) is not the reason; the queue can be larger than the stack (W11-K3).

W11-M16 — c. After dequeuing 45, and again after dequeuing 20, four nodes wait: 8, 20, 40, 60 and then 40, 60, 17, 25 (W11-S3).

W11-M17 — d. The in-order successor. The largest value of the right subtree (a) is too large: everything else in that subtree is smaller than it, yet would sit on its right.

W11-M18 — a. Right once to 45, then left to 40, which has no left child: 40 is the successor and goes to the root. 25 (b) is the **predecessor** — also a correct choice in general, but not `dsa/tree.py`'s.

W11-M19 — b. With no left child, the successor is a leaf or has one (right) child, so it is removed by the easy rule. It need not be a leaf (c): in Lecture 11's second example the successor 10 has a right child, 14.

W11-M20 — a. It accepts the lecture's tree 8 $\rightarrow$ 3 $\rightarrow$ 9, where 9 is a right child of 3 but sits in 8's left subtree. It never rejects a valid BST, and it is $O(n)$ — but wrong.

W11-M21 — c. Left operand, right operand, operator: `2 3 + 4 *`, the postfix form of Lecture 06. (a) is pre-order; (b) is in-order, which has lost the brackets and means 14, not 20.

W11-M22 — d. Both keep extra data in each node — a height or a colour — and rotate nodes after changes so that the height stays $O(\log n)$. They are beyond the bylaw (named, not examined);

`dsa/tree.py` is **unbalanced** (a).

Part B — Short answer and essay

W11-E1 *(3)*

- **Root:** the one node with no parent — 30. **Leaf:** a node with no children — 8, 17, 25, 40, 50. **Parent:** 20 is the parent of 17 and 25.
- **Subtree:** a node and everything below it — the subtree of 45 is {45, 40, 60, 50}.
- **Depth:** edges from the root down to the node — 17 has depth 3. **Height:** edges on the longest path down to a leaf — 15 has height 2 (15, 20, 17), and the tree's height is 3.
- $n - 1$ **edges:** every node except the root has exactly one parent, so exactly one edge comes into it from above; the root has none. Counting edges by the node they lead into gives $n - 1$.

W11-E2 *(4)*

- **Property:** for every node x, every value in x's left subtree is smaller than x and every value in its right subtree is larger.
- **Counter-example:** root 8, left child 3, right child 10, and 9 as the right child of 3. Each pair is in order (3 < 8, 10 > 8, 9 > 3), but 9 is in 8's left subtree. `contains(9)` goes right at 8 and never finds it.
- **Design:** carry an allowed range (low, high) down the recursion, starting unbounded at the root. Going left from x sets high = x; going right sets low = x. A node outside its range makes the tree invalid; the empty tree is valid. Test "no bound" with `is not None`, not truthiness, or a bound of 0 is lost.
- **Cost:** each node is checked once, $O(1)$ work: $O(n)$ time, $O(h)$ stack.

W11-E3 *(4)*

- **Leaf:** set the parent's link to it to `None`.
- **One child:** point the parent's link at that child; the child's whole subtree moves up one level and stays in order, because it lay on the same side of every ancestor as the deleted node.
- **Two children:** find the in-order successor — right once, then left until there is no left child. Copy its value into the node, then remove the successor's node.
- **Why the successor fits:** it is larger than everything in the left subtree (they are smaller than the deleted value, which is smaller than it), and it is the smallest value of the right subtree, so smaller than all that remain there.
- **Why removal is easy:** it was reached by going left until there was no left child, so it has no left child: it is a leaf or has one right child — one of the easy cases. Watch the case where the successor is the node's own right child: then the link to change is the node's `right` (W11-B3).
- **Always:** keep the parent; if the node is the root, `self.root` changes. $O(h)$.

W11-E4 *(4)*

- **Pre-order** (node, left, right): serialising a tree — re-inserting the list rebuilds the same shape, because every node comes after its ancestors; also prefix notation.
- **In-order** (left, node, right): listing a BST's values in **sorted** order without sorting; checking a BST (strictly increasing); range queries.
- **Post-order** (left, right, node): freeing a tree (children before the parent whose links you still need), evaluating an expression tree (operands before operator), computing sizes and heights (a node's answer needs its children's).
- **Level-order** (by depth, left to right): printing a tree level by level; the nearest nodes first — the idea of breadth-first search. Needs a queue.
- (Marks: one for the orders, one per two correct uses with reasons.)

W11-E5 *(4)*

- **True when the tree is short:** every operation walks one path, $O(h)$, and h can be as small as $\lfloor \log_2 n \rfloor$. On random insertion orders h is $O(\log n)$ on average — measured in Lecture 11, 26.6 for 4,096 values.
- **False when the tree is tall:** inserting sorted values (1, 2, …, n) makes every new value the largest so far, so it always goes right: a chain of height $n - 1$, and search is $O(n)$ — about 205 µs against 1.8 µs at n = 4,096 in the lecture's measurement.
- **Cure:** a balanced BST — AVL or red-black — which re-shapes itself with rotations to keep h = $O(\log n)$ in the worst case (beyond this course); or, with all values known in advance, insert the middle first (W11-C4).
- **Against a sorted array:** search $O(\log n)$ in both (the BST only while short); insert $O(n)$ in the array (shifting) against $O(h)$ in the BST (no moves); listing in order $O(n)$ in both (in-order walk).

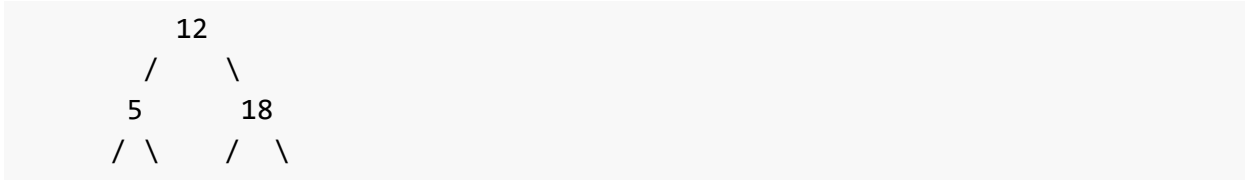
Part C — Trace the code

W11-T1

`contains(25)`	Decision	`contains(42)`	Decision
30	25 < 30: left	30	42 > 30: right
15	25 > 15: right	45	42 < 45: left
20	25 > 20: right	40	42 > 40: right
25	equal: **True**	None	**False**

`contains(42)` stepped off the right of 40, so `insert(42)` makes the new node **40's right child**. Both searches took 4 comparisons at most, h + 1 = 4.

W11-T2 The tree:



```
     2   9  15   20
        /     \
       7       16
```

```
pre-order    12, 5, 2, 9, 7, 18, 15, 16, 20
in-order     2, 5, 7, 9, 12, 15, 16, 18, 20
post-order   2, 7, 9, 5, 16, 15, 20, 18, 12
level-order  12, 5, 18, 2, 9, 15, 20, 7, 16
```

Two checks that need no drawing: the in-order walk is the values sorted; the level-order walk equals the insertion order here, because the values were listed level by level.

W11-T3 (a) Insert the pre-order list into an empty BST (that is exactly what pre-order is for):

```
          20
        /    \
      10      30
     /  \    /  \
    5   15  25   40
        /
       12
```

Post-order: **5, 12, 15, 10, 25, 40, 30, 20**.

(b) Pre-order says A is the root. In the in-order walk, what is left of A (D B E) is A's left subtree and what is right of it (C F) its right subtree. Repeat on each part: B is the left subtree's root (first in pre-order), with D on its left and E on its right; C is the right subtree's root, with nothing on its left and F on its right.

```
        A
      /   \
     B     C
    / \     \
   D   E     F
```

Post-order: **D, E, B, F, C, A**.

Why two walks for (b): in a BST the order of the values tells you, at every node, which side each value belongs to — the in-order walk is always just the sorted values, so pre-order alone is enough. For a general binary tree nothing says whether B is left or right of A; the in-order walk supplies exactly that. Pre-order alone would fit many trees: A with the chain B, D, E, C, F hanging to the left, for instance.

W11-T4 `f(root)` = **185**: the largest sum of values along a root-to-leaf path, 30 + 45 + 60 + 50. `g(root, k)` for k = 0 … 4 = **1, 2, 4, 3, 0**: the number of nodes at depth k — {30}; {15, 45}; {8, 20, 40, 60}; {17, 25, 50}; and none at depth 4. `g(root, k)` is 0 exactly when k is larger than the height.

W11-T5

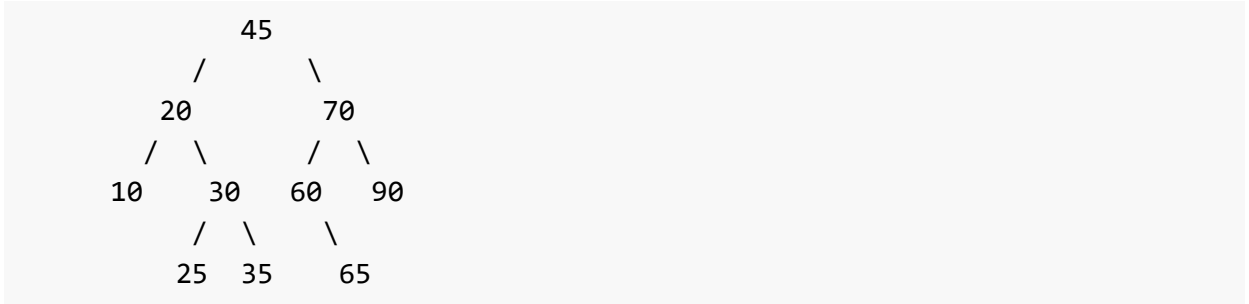
```
_is_valid(20, None, None)
  _is_valid(10, None, 20)
    _is_valid(5, None, 10)
      _is_valid(None, None, 5)  -> True
      _is_valid(None, 5, 10)    -> True
    -> True
    _is_valid(25, 10, 20)       -> False   (25 >= 20)
  -> False
-> False
```

The call on 25 returns `False` first: 25 is a right child of 10, which is fine for its parent, but it is in 20's left subtree, so its range is (10, 20). The `and` stops there, so 30 is never visited, and `is_valid()` returns `False`.

Part D — Tree state — draw every step

W11-S1 The inserts (height after each):

Insert	Where it goes	Height
45	root	0
20	45.left	1
70	45.right	1
10	20.left	2
30	20.right	2
60	70.left	2
90	70.right	2
25	30.left	3
35	30.right	3
65	60.right	3



The deletes:

- **delete(90)** — a **leaf**: `70.right = None`.
- **delete(70)** — now **one child**, 60: `45.right = 60`, and 60 keeps its right child 65.

- **delete(45)** — the root, **two children**. Successor: right once to 60; 60 has no left child, so 60 **itself** is the successor. Copy 60 into the root; the old 60 has one child, 65, which takes its place: the root's `right` becomes 65.
- **delete(20)** — **two children**. Successor: right once to 30, then left to 25, a leaf. Copy 25 into 20's node; `30.left = None`.

```
after delete(90)            after delete(70)
       45                          45
     /    \                      /    \
   20      70                  20      60
  /  \    /                   /  \       \
 10  30  60                  10  30       65
    / \    \                    / \
   25 35    65                 25 35

after delete(45)            after delete(20)
       60                          60
     /    \                      /    \
   20      65                  25      65
  /  \                        /  \
 10   30                     10   30
     /  \                           \
    25   35                          35
```

Every step leaves a valid BST (checked with `is_valid()`); the final pre-order is 60, 25, 10, 30, 35, 65, and the final height is 3.

W11-S2

```
(a) 1, 2, 3, 4, 5, 6, 7          (b) 4, 2, 6, 1, 3, 5, 7
1                                        4
 \                                     /   \
  2                                   2     6
   \                                 / \   / \
    3                               1   3 5   7
     ...
        7
height 6                         height 2

(c) 3, 1, 2, 6, 5, 7, 4          (d) 4, 6, 5, 7, 2, 3, 1
      3                                  4
    /   \                             /   \
   1     6                           2     6
    \   / \                         / \   / \
     2 5   7                       1   3 5   7
      /
```

```
     4
height 3                          height 2
```

(b) and (d) build the **same tree**, and both have pre-order **4, 2, 1, 3, 6, 5, 7**. They must: the shape of a BST depends only on which values arrive before which **on each path**. In both orders 4 comes first, 2 comes before 1 and 3, and 6 before 5 and 7; the relative order of the two subtrees' values does not matter. (Pre-order is also how you check: equal pre-order walks mean equal BSTs.)

W11-S3

Dequeued	Queue after the step
(start)	30
30	15, 45
15	45, 8, 20
45	8, 20, 40, 60
8	20, 40, 60
20	40, 60, 17, 25
40	60, 17, 25
60	17, 25, 50
17	25, 50
25	50
50	(empty)

Level-order: 30, 15, 45, 8, 20, 40, 60, 17, 25, 50.

With a stack, pushing the right child before the left, the left child is always on top, so the walk finishes a node's whole left side before its right: **30, 15, 8, 20, 17, 25, 45, 40, 60, 50** — **pre-order**. Same loop, different container, different order: the difference between BFS and DFS (Week 14).

Part E — Complexity analysis

W11-K1

- (a) $\Theta(n^2)$**.** The i-th value is the largest so far and walks the whole chain of i $-$ 1 nodes: $0 + 1 + \cdots + (n-1) = n(n-1)/2$ comparisons.
- (b) $\Theta(n \log n)$ **on average.** In a randomly built BST the average node depth is about $2 \ln n \approx 1.39 \log_2 n$, so each insert walks $O(\log n)$ nodes.
- (c) **Tree sort:** the in-order walk is $\Theta(n)$, so the cost is the building: $\Theta(n \log n)$ in the best case (a short tree, e.g. middle first), $\Theta(n^2)$ in the worst (sorted or reverse-sorted input — the input that is already sorted is the one it handles worst).

W11-K2 The fault is calling `height` on the taller child **twice**: once in the comparison and again in the `return`.

- **Right chain of n nodes.** Each call on a node makes one call on the empty left side and **two** on the chain of $n-1$ below: $C(n) = 2C(n-1) + 2$, $C(0) = 1$, which solves to $C(n) = 3 \cdot 2^n - 2$: $\Theta(2^n)$. For n = 20: **3,145,726 calls** — counted by running it — where a correct `height` makes 41.
- **Perfect tree of height h.** Three calls on subtrees of height h $-$ 1 (both sides are equal, so the `return` recomputes the right one): $C(h) = 3C(h-1) + 1$, which solves to $C(h) = (3^{h+2} - 1)/2$: $\Theta(3^h) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$. For n = 1,023 (h = 9): 88,573 calls.
- **Fix:** compute each child's height once, into variables, and return `1 + max(left, right)` — two calls per node, $\Theta(n)$. The same trap as `fib` in Lecture 03: a recursion that repeats work multiplies it.

W11-K3

	recursive in-order: $O(h)$ stack	`level_order`: $O(w)$ queue
perfect tree, n nodes	$\lfloor \log_2 n \rfloor + 1$ frames — tiny	up to $(n+1)/2$ nodes (the last level)
chain of n nodes	n frames — and `RecursionError` near 1,000	1 node

On a short, bushy tree the recursive walk is far cheaper in memory; on a chain the queue is. Both are $\Theta(n)$ in time either way. The recursion's memory is the call stack, which Python limits to about 1,000 frames; the queue's is ordinary memory in your `CircularQueue`, limited only by the machine.

Part F — Find and fix the bug

W11-B1. The loop walks until `node` is `None`, and then `node = TreeNode(value)` only rebinds a local variable: no link in the tree changes. Even the first insert is lost — `self.root` is never set, so `BinarySearchTree([8, 3, 10])` is empty, with `root` still `None`. **Fix:** handle the empty tree (`self.root = TreeNode(value)`), and stop **at the parent**: before stepping to a child, if it is `None`, set `node.left` or `node.right` to the new node and return.

W11-B2. It checks each node only against its own children. The tree 8, with 3 and 10 as children and 9 as the right child of 3, passes every check, but 9 is in 8's left subtree: not a BST. **Fix:** carry the range `(low, high)` allowed by all the ancestors down the recursion, as in `dsa/tree.py`'s `_is_valid` (W11-E2, W11-T5).

W11-B3. `succ_parent.left = succ.right` assumes the successor is the **left** child of its parent. That holds when the "go left" loop runs at least once, but when the successor is the node's own **right child** the loop runs zero times, `succ_parent` is the node itself, and the link to change is its `right`. On Q, `delete(20)`: the successor is 25, 20's right child. The code copies 25 into 20's node and then sets that node's **left** to `None` — so 17 vanishes, and the old 25 is still there as the right child:

```
in-order before: 8, 15, 17, 20, 25, 30, 40, 45, 50, 60
in-order after:  8, 15, 25, 25, 30, 40, 45, 50, 60
is_valid():      False
```

The tests' two-children case (deleting 3 from the sample tree) has a deep successor, so it passes; deleting the root 8 (successor 10, the right child) is what fails. **Fix:**

```
if succ_parent is node:
    succ_parent.right = succ.right
else:
    succ_parent.left = succ.right
```

— or, as `dsa/tree.py` does, move `node` and `parent` down to the successor and let the one-child code, which asks `parent.left is node`, do the removal.

W11-B4. `out = []` inside the helper rebinds `out` to a new, local list at every call, so every append goes into a list that is thrown away. The caller's list is never touched: `in_order()` always returns `[]`, and `test_in_order_is_sorted` fails with `assert [] == [1, 3, 4, 6, ...]`. **Fix:** delete that line; the helper must append to the list it was given.

Part G — Write the code

W11-C1

```
def count_leaves(node):
    if node is None:
        return 0
    if node.left is None and node.right is None:
        return 1
    return count_leaves(node.left) + count_leaves(node.right)
```

A leaf answers 1, the empty tree 0, and every other node the sum of its two subtrees' answers. One visit per node: $O(n)$. On Q: 5.

W11-C2

```
def is_balanced(node):
    return _balanced_height(node) != -2


def _balanced_height(node):
    # The height of the subtree, or -2 if it is unbalanced anywhere.
    if node is None:
        return -1
    left = _balanced_height(node.left)
    if left == -2:
```

```
        return -2
    right = _balanced_height(node.right)
    if right == -2 or abs(left - right) > 1:
        return -2
    return 1 + max(left, right)
```

The helper returns the height when the subtree is balanced and -2 — an impossible height — as soon as it finds a node that is not, and every caller passes the -2 straight up without further work. Each node is visited once: $O(n)$. A separate `height` at every node would recompute each subtree's height once for every ancestor: $O(n \log n)$ on a balanced tree, $O(n^2)$ on a chain — and the test, which counts reads on 8,191 nodes, measures 212,992 reads for that approach against a budget of 32,768.

W11-C3

```
def range_values(node, low, high):
    out = []
    _collect(node, low, high, out)
    return out


def _collect(node, low, high, out):
    if node is None:
        return
    value = node.value
    if value > low:                  # answers may lie on the left
        _collect(node.left, low, high, out)
    if low <= value <= high:
        out.append(value)
    if value < high:                 # answers may lie on the right
        _collect(node.right, low, high, out)
```

It is the in-order walk with two `if`s. If `value <= low`, everything in the left subtree is smaller than `value`, hence below `low`: skip it. If `value >= high`, the right subtree is above `high`: skip it. What remains is visited in in-order, so the answers come out sorted. Cost $O(h + k)$ for k answers: the walk follows at most two root-to-leaf boundary paths plus the subtrees that are entirely inside the range.

W11-C4

```
def build_balanced(values):
    return _build(values, 0, len(values) - 1)


def _build(values, lo, hi):
    if lo > hi:
        return None
```

```
    mid = (lo + hi) // 2
    root = TreeNode(values[mid])
    root.left = _build(values, lo, mid - 1)
    root.right = _build(values, mid + 1, hi)
    return root
```

The middle value is the root; the two halves, built the same way, are its subtrees. Each half has at most half the values, so the height is $\lfloor \log_2 n \rfloor$ — the smallest possible. Passing `lo` and `hi` avoids slicing, which would copy $O(n)$ values per level. $O(n)$ in total: one `TreeNode` per value. With all values known and sorted in advance, this is the "middle first" cure of Lecture 11.

W11-C5

```
def lowest_common_ancestor(node, a, b):
    while node is not None:
        value = node.value
        if a < value and b < value:
            node = node.left
        elif a > value and b > value:
            node = node.right
        else:
            return value            # a and b split here
    raise ValueError("a and b must be in the tree")
```

While a and b are both smaller than the node, both are in its left subtree, so the answer is there too; both larger, the right subtree. The first node where they are **not** on the same side — one is smaller and one larger, or one of them **is** the node — has a in one subtree (or is a) and b in the other (or is b), so no lower node can hold both. One path down: $O(h)$, with no recursion.

Answers — Week 12

> ***The questions*** *are at the end of the* Week 12 *chapter. Commit to your own answer before reading one here. Every array, trace and count below was produced by running the reference solution,* `solutions/dsa/heap.py`.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M07	b	M13	b	M19	d
M02	c	M08	d	M14	c	M20	c
M03	a	M09	c	M15	d	M21	a
M04	d	M10	a	M16	b	M22	d
M05	a	M11	b	M17	c		
M06	c	M12	d	M18	a		

W12-M01 — b. That is the heap property, and it is all a heap promises. The array is not sorted (a): Q itself is `[1, 5, 3, …]`. Siblings are unordered (c). The largest value is in some leaf, not necessarily the last one (d).

W12-M02 — c. 2i + 1; the right child is 2i + 2 (d). 2i (a) is the 1-based formula of CLRS.

W12-M03 — a. (10 – 1) // 2 = 4. Check: the children of 4 are 9 and 10.

W12-M04 — d. $\lfloor \log_2 1000 \rfloor = 9$: levels 0 to 9 hold up to 1,023 nodes. 10 (a) is the number of **levels**.

W12-M05 — a. 3's children are 7 and 4; 2 has none. In (b), 2 at index 3 is below its parent 3; in (c), 1 is below 2; in (d), 3 at index 3 is below 4.

W12-M06 — c. 2 is appended at index 9, under 6 (index 4): swap. Now at index 4, under 5 (index 1): swap. Under 1 at the root: stop. The array becomes `[1, 2, 3, 8, 5, 4, 9, 11, 10, 6]`.

W12-M07 — b. 10, the last item, moves to the root; its children are 5 and 3, so it swaps with 3, then with 4. The array is `[3, 5, 4, 8, 6, 10, 9, 11]`. The new root is always the smaller of the old root's two children.

W12-M08 — d. Swapping with a child that beats the item but not its sibling puts a larger value above a smaller one (W12-B2). (b) is wrong: sift-down never changes the shape.

W12-M09 — c. n // 2 – 1 is the last index with a child; everything after it is a leaf, already a heap of one.

W12-M10 — a. At most about n swaps and 2n comparisons (W12-E3).

W12-M11 — b. Half the nodes are leaves (no work), a quarter move at most one level, and so on: $\sum h \cdot n/2^{h+1} \leq n$. It does look at every value (a) — it just does little with most of them.

W12-M12 — d. Each new value is the smallest so far and climbs to the root: about $\log_2 i$ swaps for the i-th push, $\Theta(n \log n)$ in total. On random input the pushes average about two comparisons each.

W12-M13 — b. The best value is at index 0.

W12-M14 — c. The heap property says nothing about where a value that is not the best might be — it could be in any subtree. Only the root is cheap to find.

W12-M15 — d. Complete shape means indices 0 … n – 1 with no holes, and 2i + 1, 2i + 2, (i – 1) // 2 replace `left`, `right` and `parent`.

W12-M16 — b. b and c both have priority 1; b was enqueued first (counter 1 against 2), so it is served first. Then c, then a.

W12-M17 — c. With `(1, item_b)` against `(1, item_c)`, Python compares the items: dictionaries raise `TypeError`, strings silently order alphabetically. The unique counter decides every tie before the item is reached.

W12-M18 — a. n values, each costing at most one peek and a pop + push on a heap of k: $O(n \log k)$. (c) is sorting everything.

W12-M19 — d. The root is the smallest of the k kept, and it is the one a new, larger value must replace.

W12-M20 — c. Heap sort builds the heap inside the input array and moves each maximum into the space the heap gives up.

W12-M21 — a. The sorted region grows from the right as the heap shrinks from the right. Heap sort is not stable (b).

W12-M22 — d. `heapify` gives `[9, 6, 4, 1, 5, 3, 2, 1]`: `_items[1]` is 6.

Part B — Short answer and essay

W12-E1 *(4)*

- **Shape:** a **complete** binary tree — every level full except possibly the last, which is filled from the left.
- **Order (heap property):** every parent beats its children — $\leq$ in a min-heap, $\geq$ in a max-heap. So the best value is at the root. Siblings are unordered.
- **Height:** a complete tree of n nodes has height $\lfloor \log_2 n \rfloor$, so every operation that walks one path is $O(\log n)$. A heap can never degenerate into a list, unlike an unbalanced BST.
- **Storage:** level-by-level numbering fills indices 0 … n – 1 with no gaps, so the tree is an array; children at 2i + 1 and 2i + 2, parent at (i – 1) // 2. No nodes, no references.

W12-E2 *(4)*

- **push(x):** append x at index n (the only place that keeps the shape complete), then **sift up**: while x beats its parent, swap them. At most one swap per level: $O(\log n)$.
- **pop():** raise `IndexError` if empty. Save the root; remove the **last** item; if the heap is now empty, return it (the one-item case); otherwise put it at the root and **sift down**: find the better of the item and its (up to two) children; if a child is better, swap and continue. At most two comparisons per level: $O(\log n)$.
- **Special cases:** empty pop/peek; a single item; a node with a left child but no right child (check each child index against `len` before reading it).
- peek is $O(1)$: the root.

W12-E3 *(4)*

- **Algorithm:** leave the values in place; for i from n `// 2 - 1` **down** to 0, sift down i. Leaves (the back half) are heaps already.
- **Why backwards:** when i is sifted down, both of its subtrees must already be heaps — sift-down only repairs a root sitting on two valid heaps. Going backwards guarantees that; going forwards does not (W12-K3).
- **O(n):** a node of height h moves at most h levels, and at most $\lceil n/2^{h+1} \rceil$ nodes have height h, so the work is at most $\sum_h h \cdot n/2^{h+1} = \frac{n}{2} \sum_h h/2^h = \frac{n}{2} \cdot 2 = n$ swaps.
- **n pushes:** a push costs the node's **depth**, and half the nodes are at the bottom, at depth $\approx \log_2 n$: in the worst case (descending input for a min-heap) the total is $\Theta(n \log n)$. Measured in Lecture 12: 2.0 comparisons per element for heapify against 14 per element for pushes, at n = 2^{16}.

W12-E4 *(3)*

	enqueue	dequeue	peek
unsorted array	$O(1)$ amortised	$O(n)$	$O(n)$
sorted array	$O(n)$	$O(1)$ (from the end)	$O(1)$
binary heap	$O(\log n)$	$O(\log n)$	$O(1)$
balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$, or $O(1)$ with a pointer to the minimum

- **Choose the heap:** a workload of n enqueues and n dequeues costs $O(n \log n)$, against $O(n^2)$ for either array; it lives in one array with no references, and needs no balancing. The BST matches the Big-O but costs nodes, references and a balancing scheme — worth it only if you also need to search for or delete **arbitrary** items.

W12-E5 *(3)*

- With `(priority, item)`, two equal priorities make Python compare the items. Items that do not support `<` (dictionaries, most objects) raise `TypeError`; items that do (strings) are ordered by value, which is not what anyone asked for.
- The counter is unique and sits before the item, so every comparison is settled by priority or counter and never reaches the item.
- It also makes ties **FIFO**: the counter grows with each enqueue, so of two equal priorities the earlier one is served first. A heap on its own is not stable, so this fairness comes only from

the counter.

Part C — Trace the code

W12-T1

Operation	Array afterwards	Returns
push 6	`[6]`	
push 2	`[2, 6]` — 2 swaps with 6	
push 8	`[2, 6, 8]`	
push 1	`[1, 2, 8, 6]` — index 3 → 1 → 0	
push 5	`[1, 2, 8, 6, 5]` — 5 is not below 2	
push 3	`[1, 2, 3, 6, 5, 8]` — 3 swaps with 8	
pop	`[2, 5, 3, 6, 8]` — 8 to the root, then swaps with 2, then with 5	1
pop	`[3, 5, 8, 6]` — 8 to the root, swaps with 3	2

W12-T2. n = 10, so the last parent is 10 // 2 − 1 = **4**.

`sift_down(i)`	Value	Moves	Array afterwards
4	4	swaps with 0 (index 9)	`[3, 9, 2, 1, 0, 5, 8, 7, 6, 4]`
3	1	stays (children 7, 6)	unchanged
2	2	stays (children 5, 8)	unchanged
1	9	swaps with 0 (index 4), then with 4 (index 9)	`[3, 0, 2, 1, 4, 5, 8, 7, 6, 9]`
0	3	swaps with 0 (index 1), then with 1 (index 3)	`[0, 1, 2, 3, 4, 5, 8, 7, 6, 9]`

5 swaps (and 14 comparisons) in all. The result is a valid min-heap; it happens to look almost sorted, but 8 before 7 and 6 shows it is not.

W12-T3

Pop	Returns	Array afterwards	How
1	1	`[3, 5, 4, 8, 6, 10, 9, 11]`	10 to the root; swaps with 3, then 4

Pop	Returns	Array afterwards	How
2	3	`[4, 5, 9, 8, 6, 10, 11]`	11 to the root; swaps with 4, then 9
3	4	`[5, 6, 9, 8, 11, 10]`	11 to the root; swaps with 5, then 6

Each pop returns the next smallest value: 1, 3, 4.

W12-T4

Operation	Heap array afterwards	Returns
enqueue report, 3	`(3,0,report)`	
enqueue login, 1	`(1,1,login)`	
	`(3,0,report)`	
enqueue email, 2	`(1,1,login)`	
	`(3,0,report)`	
	`(2,2,email)`	
dequeue	`(2,2,email)`	fix login bug
	`(3,0,report)`	
enqueue server, 1	`(1,3,server)`	
	`(3,0,report)`	
	`(2,2,email)`	
enqueue docs, 3	`(1,3,server)`	
	`(3,0,report)`	
	`(2,2,email) (3,4,docs)`	
dequeue	`(2,2,email)`	restart server
	`(3,0,report) (3,4,docs)`	
dequeue	`(3,0,report) (3,4,docs)`	reply to email
dequeue	`(3,4,docs)`	print report

“print report” and “update docs” tie at priority 3; the report was enqueued first (counter 0 against 4) and is served first.

W12-T5. `heap_sort_steps` in `solutions/dsa/sorting.py`:

Stage	Array	
input	`[4, 10, 3, 5, 1]`	
build (max-heap)	`[10, 5, 3, 4, 1]`	sift_down(1): 10 stays; sift_down(0): 4 swaps with 10, then with 5
swap 0 ↔ 4, sift down in 0..3	`[5, 4, 3, 1, 10]`	
swap 0 ↔ 3, sift down in 0..2	`[4, 1, 3, 5, 10]`	
swap 0 ↔ 2, sift down in 0..1	`[3, 1, 4, 5, 10]`	
swap 0 ↔ 1	`[1, 3, 4, 5, 10]`	sorted

Stage	Array

Part D — Heap state — draw every step

W12-S1 (max-heap; draw each array as a tree, children of i at 2i + 1 and 2i + 2)

Push	Array afterwards	Swaps
5	`[5]`	—
12	`[12, 5]`	12 ↔ 5
7	`[12, 5, 7]`	none: 7 < 12
20	`[20, 12, 7, 5]`	20 ↔ 5 (index 3 → 1), 20 ↔ 12 (1 → 0)
3	`[20, 12, 7, 5, 3]`	none: 3 < 12
15	`[20, 12, 15, 5, 3, 7]`	15 ↔ 7 (index 5 → 2); 15 < 20, stop

Final tree: 20 at the root; children 12 and 15; 12's children 5 and 3; 15's child 7.

W12-S2 (k = 3)

Value	Action	Heap array afterwards
5	push (fewer than 3)	`[5]`
1	push	`[1, 5]`
9	push	`[1, 5, 9]`
3	3 > root 1: pop 1, push 3	`[3, 9, 5]`
7	7 > root 3: pop 3, push 7	`[5, 9, 7]`
2	2 $\leq$ root 5: skip	`[5, 9, 7]`
8	8 > root 5: pop 5, push 8	`[7, 9, 8]`

The heap holds 7, 8, 9: the 3 largest. Popping them gives 7, 8, 9, so `top_k` reverses that to `[9, 8, 7]`.

W12-S3

- (a) `[1, 2, 3, 4, 5, 6]` — **valid** (a sorted array is always a min-heap; the reverse is false).
- (b) `[1, 3, 2, 5, 4, 1]` — **not valid**: 1 at index 5 is below its parent 2 (index 2). Only that pair; 1 at the root and 1 at index 5 are not parent and child.
- (c) `[2, 2, 2, 3, 2]` — **valid**: equal values do not break $\leq$.
- (d) `[4, 5, 6, 7, 8, 9, 3]` — **not valid**: 3 at index 6 is below 6 (index 2). 3 is also smaller than the root, but the property only compares parents with children; the violation is the pair (6, 3).

Part E — Complexity analysis

W12-K1 — $\Theta(n \log n)$ in the worst case. n pushes cost up to $\log_2 i$ each, and n pops up to $2 \log_2 i$ comparisons each. `result` is `values` **sorted ascending**: this is heap sort, written with a priority queue. Against Week 10's `heap_sort`: the same $O(n \log n)$ time (with a worse constant, and the build is $O(n \log n)$ instead of `heapify`'s $O(n)$), but $O(n)$ **extra space** — the heap and the result list — where the in-place version needs $O(1)$.

W12-K2 — $O(n \log k)$ time, $O(k)$ space, against $O(n \log n)$ time and $O(n)$ space for sorting. For k = 10, n = 10^6: each value costs one `peek` comparison, plus a pop and a push on a heap of height 3 when it gets in — a few million comparisons at most, against about 2×10^7 for the sort — and the heap needs 10 slots, so it also works on a stream too large for memory. For k = n the two are the same, $O(n \log n)$: the heap is then a whole heap sort.

W12-K3. The forward `sift_down` loop is **wrong**. On `[5, 4, 3, 2, 1]` it gives `[3, 1, 5, 2, 4]` — 1 is below 3. On the lecture's `[9, 4, 7, 1, 8, 2, 6, 3, 5]` it gives `[4, 1, 2, 3, 8, 7, 6, 9, 5]`. Sifting down the root first compares it with children that are not yet heaps, so smaller values further down are never brought up to the root. (It happens to work on some inputs, such as `[3, 2, 1]`.)

A forward loop of `self._sift_up(index)` for every index from 1 to n – 1 **is** correct: when index i sifts up, indices 0 … i – 1 already form a heap, so it is exactly n pushes done in place. It costs $\Theta(n \log n)$ in the worst case — correct, but it misses the point of `heapify`.

Part F — Find and fix the bug

W12-B1. `parent = index // 2` is the **1-based** formula. With 0-based indices the parent of i is `(index - 1) // 2`; `index // 2` is right only for odd i, and for even i it names the node **after** the true parent — for index 4, node 2 instead of node 1. The sift-up then compares with the wrong node. `MinHeap([1, 5, 2, 6]).push(3)` appends 3 at index 4, compares it with 2 (index 2), stops, and leaves `[1, 5, 2, 6, 3]` — 3 below 5, an invalid heap. The correct result is `[1, 3, 2, 6, 5]`. **Fix:** `parent = (index - 1) // 2`.

W12-B2. It swaps with the **first** child that beats the item, not the **better** child. When both children beat it and the right is better, the left moves up above the right. Popping `[2, 4, 3, 9, 7, 8, 5, 12, 10]` returns 2 but leaves `[4, 9, 3, 10, 7, 8, 5, 12]` — 4 above 3 — and the next pop returns 4 instead of 3. **Fix:** find the better child first — start `best = index`, replace it by the left child if the left beats `items[best]`, then by the right child if the right beats `items[best]`; stop if `best == index`, otherwise swap with `best`.

W12-B3. No one-item case. When the heap holds a single item, `pop()` on the array empties it, and `self._items[0]` raises `IndexError` — on a heap that was **not** empty. `MinHeap([4, 7])`: the first pop returns 4, the second raises. **Fix:** after removing the last item, `if len(self._items) == 0: return last`.

W12-B4. The `range` stops **before** 0, so the root is never sifted down. `MinHeap([9, 4, 7, 1, 8, 2, 6, 3, 5])` ends as `[9, 1, 2, 3, 8, 7, 6, 4, 5]` and `peek()` returns 9, the **largest** value. Every subtree below the root is a heap, which is why the result looks nearly right. **Fix:** `range(len(self._items) // 2 - 1, -1, -1)` — a stop of –1 includes 0.

Part G — Write the code

W12-C1

```
def is_min_heap(values):
    for child in range(1, len(values)):
        if values[child] < values[(child - 1) // 2]:
            return False
    return True
```

Every element except the root has exactly one parent, so checking each **child** against its parent checks every edge of the tree once: n – 1 comparisons, $O(n)$, and no heap object at all. Starting at index 1 avoids the root, which has no parent. Walking the **parents** instead needs two children per step and two bounds checks — correct, but longer. This is also the body of `MinHeap.is_valid`, with < in place of `_beats`.

W12-C2

```
def top_k(values, k):
    if k < 0:
        raise ValueError("k must be >= 0")
    if k == 0:
        return []
    heap = MinHeap()                    # the best k so far; root = weakest
    for value in values:
        if len(heap) < k:
            heap.push(value)
        elif heap.peek() < value:       # beats the weakest: replace it
            heap.pop()
            heap.push(value)
    result = [heap.pop() for _ in range(len(heap))]   # smallest first
    result.reverse()
    return result
```

The heap holds the best k values seen so far, and its root is the **weakest** of them — exactly the value a better newcomer must replace. A value that does not beat the root costs one comparison and is forgotten. Each value costs at most $O(\log k)$, so the total is $O(n \log k)$ in time and $O(k)$ in space. Popping the heap at the end gives the k values smallest first; one `reverse` makes them largest first. A max-heap of all n values would also work, but it is $O(n)$ space and $O(n + k \log n)$ time, and it cannot run on a stream.

W12-C3

```
def merge_sorted(lists):
    heap = MinHeap()
    for which, values in enumerate(lists):
        if values:
            heap.push((values[0], which, 0))      # (value, list, position)
    merged = []
    while not heap.is_empty():
        value, which, position = heap.pop()
        merged.append(value)
        if position + 1 < len(lists[which]):
            heap.push((lists[which][position + 1], which, position + 1))
    return merged
```

Each heap entry is `(value, list number, position)`: the current front of one list, and where it came from, so that after popping it the **next** value of the same list can be pushed. The heap never holds more than k entries — one per list — so every pop and push is $O(\log k)$, and N values cost $O(N \log k)$. The list number also breaks ties between equal values, so the positions are never compared with anything but integers. The input lists are only read. This is the merge step of an external sort, which merges k sorted runs of a file too large for memory.

W12-C4

```
def join_ropes(lengths):
    heap = MinHeap(lengths)                       # heapify: O(n)
    total = 0
    while len(heap) > 1:
        joined = heap.pop() + heap.pop()          # the two shortest
        total += joined
        heap.push(joined)
    return total
```

Why the two shortest: every rope's length is paid once for each join it takes part in, so a rope that is joined early is paid for many times. The shortest ropes should therefore be the ones buried deepest — joined first. Formally this is Huffman's argument (1952): in an optimal plan the two shortest ropes can always be joined together first, and the rest of the problem is the same problem with one rope fewer. `MinHeap(lengths)` heapifies a **copy** in $O(n)$; then n – 1 rounds of two pops and a push, $O(n \log n)$. Re-sorting a list after every join would be $O(n^2 \log n)$.

W12-C5

```
def running_medians(values):
    low = MaxHeap()                   # smaller half; root = its largest
    high = MinHeap()                  # larger half; root = its smallest
    medians = []
    for value in values:
        if low.is_empty() or value <= low.peek():
```

```
            low.push(value)
        else:
            high.push(value)
        if len(low) > len(high) + 1:  # rebalance: sizes differ by <= 1
            high.push(low.pop())
        elif len(high) > len(low):
            low.push(high.pop())
        if len(low) > len(high):
            medians.append(low.peek())
        else:
            medians.append((low.peek() + high.peek()) / 2)
    return medians
```

`low` holds the smaller half as a **max**-heap, so its root is the largest of the small values; `high` holds the larger half as a **min**-heap, so its root is the smallest of the large values. Every value in `low` is $\leq$ every value in `high`, and the sizes differ by at most one, with `low` never the smaller. So the two roots are the two middle values of everything seen: with an odd count, the median is `low`'s root; with an even count, the mean of the two roots. Each new value costs at most three pushes or pops: $O(\log n)$, and $O(n \log n)$ for the whole list — against $O(n^2)$ for keeping a sorted list and inserting into it.

Answers — Week 13

> ***The questions*** *are at the end of the* Week 13 *chapter. Commit to your own answer before reading one here. Every table state, probe list and printed line below was produced by running the reference* `dsa/hashmap.py` *(*`solutions/dsa/hashmap.py`*), or the code shown, not worked out by hand.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M07	b	M13	a	M19	b
M02	a	M08	c	M14	d	M20	d
M03	c	M09	b	M15	c	M21	a
M04	d	M10	c	M16	a	M22	d
M05	a	M11	d	M17	b		
M06	a	M12	b	M18	c		

W13-M01 — b. `hash` turns the key into an integer; `% capacity` folds it into 0..capacity - 1. (a) only works for integer keys and puts all small keys in slot 0; (c) is searching, which is what hashing avoids.

W13-M02 — a. 45 = 5 × 8 + 5.

W13-M03 — c. Otherwise two equal keys look in different buckets, and a key that is present is reported missing. (a) is impossible — there are more keys than hash values — and not needed: collisions are handled.

W13-M04 — d. Hash randomisation, on by default since Python 3.3, so that an attacker cannot predict which strings collide. Within one run the hash never changes. (b) is how objects **without** a `__hash__` of their own are hashed, not strings.

W13-M05 — a. `hash(n)` is n for every integer of this size. `str` and `bytes` hashes are randomised (b, c), and a set's order follows the hashes (d).

W13-M06 — a. A mutable key could change after `put`; its hash would change with it, and it would sit in a bucket no lookup visits. So Python refuses to hash lists, dicts and sets at all.

W13-M07 — b. $1 - \frac{365}{365} \cdot \frac{364}{365} \cdots \frac{343}{365} \approx 0.507$ — the birthday paradox. 23/365 (c) is about 6% (a), the chance for one given pair, and ignores the other 252 pairs.

W13-M08 — c. For chaining it is also the average chain length.

W13-M09 — b. All three are 5 mod 8, and each new key is pushed on the **front**: after 5, then 13 → 5, then 21 → 13 → 5. Load 3/8: no resize.

W13-M10 — c. 29 % 8 = 5: it compares with 21, 13 and 5, reaches the end of the chain, and raises `KeyError`.

W13-M11 — d. The check runs after the insert, with >: 6/8 = 0.75 is allowed, 7/8 = 0.875 is not. The table goes from 8 to 16 buckets during the 7th put.

W13-M12 — b. A key in bucket 2 of 8 (say 10) belongs in bucket 10 of 16. Copied across, it stays in bucket 2, and `get(10)` looks in bucket 10: `KeyError`. Every key must be rehashed.

W13-M13 — a. 6 $\rightarrow$ slot 6. 14 $\rightarrow$ 6 is taken, slot 7. 22 $\rightarrow$ 6, 7 taken, wrap to slot 0. Slots: `22 _ _ _ _ _ 6 14`.

W13-M14 — d. 30 % 8 = 6: probes 6 (6), 7 (14), 0 (22), 1 (`None`) — four probes, then "not there".

W13-M15 — c. A search stops at `None`, because `put` would have used that slot. Emptying a slot in the middle of a probe sequence makes every key beyond it unreachable (`test_open_addressing_delete_preserves_probe_chain`).

W13-M16 — a. A tombstone means "a key was here once, and others may have probed past it". Only `None` ends a search.

W13-M17 — b. Tombstones are not keys (a, c), but they are not `None` either. Uncounted, a table under many puts and deletes fills with tombstones: every miss probes the whole table, and a loop that waits for `None` never ends.

W13-M18 — c. A run of length L is hit by any key whose home is in it or next to it, and the key lands at its end, making it longer. (b) is not clustering in the linear-probing sense.

W13-M19 — b. $\frac{1}{2}(1 + 1/0.25) = \frac{1}{2} \cdot 5 = 2.5$. (a) is the **successful** search at 0.5; (d) the unsuccessful one at 0.9. Measured in Lecture 13 on 16,384 slots: 2.56.

W13-M20 — d. Each key is a multiple of 1,024, so `key % capacity` is 0 for every power-of-two capacity up to 1,024. Resizing does not help (a): the keys collide at every such capacity. One chain of n entries: $O(n)$ per `get`, $O(n^2)$ to build.

W13-M21 — a. CPython's `Objects/dictobject.c`: open addressing, probe `i = (5*i + 1 + perturb) % 2**k` with `perturb` taken from the hash, a resize at about two-thirds, and "dummy" entries as tombstones. (b) is Java's `HashMap`.

W13-M22 — d. A range query needs order, and hashing destroys order on purpose: 2027114 and 2027115 land far apart. A sorted array answers it with two `lower_bounds` (Week 8). (a)–(c) are $O(1)$ average in a hash table.

Part B — Short answer and essay

W13-E1 *(4)*

- **Mechanism:** the index `hash(key) % capacity` is **computed** in $O(1)$ and the `Array` slot is reached in $O(1)$; only the keys that share that slot are compared — a chain, or a short run of probes.

- **Assumption:** the hash spreads the keys evenly, as if at random, and the load factor α is kept below a constant by resizing. Then a chain averages α entries (a probe run: a few slots), so `get`, `put` and `delete` are $O(1)$ **expected**, and `put` amortised because of resizes.
- **Worst case:** all n keys in one bucket or one cluster: $O(n)$ per operation.
- **What pushes towards it** (any three): a poor hash (sum of characters — anagrams collide); keys sharing their low bits on a power-of-two table (multiples of 1,024); an adversary who can predict the hash (hashDoS — hence string hash randomisation); no resizing, so α grows with n; open addressing near $\alpha = 1$, or clogged with tombstones.

W13-E2 *(3)*

- **Only slow — never wrong.** Every key hashes to 0 and lands in bucket (or home slot) 0; keys are still told apart by `==`, so every `get` finds the right entry or correctly reports it missing.
- **Chaining:** one chain holding all n keys. `get` walks it: $O(n)$. Each `put` walks it too (to check for the key): building n keys is $1 + 2 + \cdots + n = O(n^2)$. Resizing never helps: 0 % capacity is always 0.
- **Open addressing:** all keys form one cluster starting at slot 0; a `get` probes up to n slots, $O(n)$; building is $O(n^2)$ — the same.
- The only hash rule for correctness is "equal keys, equal hashes", and a constant keeps it. Every other quality of a hash function is about speed.

W13-E3 *(4)*

- **A slot holds:** chaining — `None` or the head of a linked list of `Entry` nodes; open addressing — one key (in `_keys`) and its value (in `_values`), or `None`, or a tombstone.
- **Load factor:** chaining still works above 1 (chains simply grow; cost about $1 + \alpha$), and resizes at 0.75 here; open addressing cannot exceed 1 at all and degrades sharply near it (a miss $\approx \frac{1}{2}(1 + 1/(1-\alpha)^2)$), so it resizes earlier — 0.66 here, about 2/3 in CPython.
- **Delete:** chaining unlinks a node (the head is the special case); open addressing must leave a tombstone, which later puts may reuse and a resize drops.
- **Advantages:** chaining — simple deletion, graceful at high load, never "full"; open addressing — no node per key (less memory, no allocation), and consecutive probes are neighbours in memory, which suits the cache. CPython's `dict` and `set` use open addressing; Java's `HashMap` uses chaining.

W13-E4 *(4)*

- **Why not empty the slot:** a search stops at `None`. If keys a and b collide and b was placed beyond a, emptying a's slot cuts b's probe sequence: `get(b)` stops at the new `None` and reports b missing although it is still in the table.
- **`get` / `delete`:** skip a tombstone and keep probing; stop only at the key or at `None`.
- **`put`:** remember the first tombstone passed, but keep probing to `None` — the key may be further on (then overwrite it). If it is not there, insert at the remembered tombstone (reusing the space), else at the `None`.
- **`_resize`:** re-insert only live keys; tombstones are dropped, so the new table has none.
- **Load check:** tombstones are not keys, but they fill slots, and only `None` stops a probe. Counting them (`size + tombstones + 1`) forces a rebuild — at the same capacity if the live keys alone are within the limit — before the table runs out of `None`.

W13-E5 *(3)*

- A resize at size s rehashes s keys. With doubling, the resizes before reaching size n happened at sizes of at most $n, n/2, n/4, \ldots$, so the total rehashing is at most $n + n/2 + n/4 + \cdots < 2n$: $O(n)$ for n puts, **amortised** $O(1)$ each. (Measured in Lecture 13: 3,075 keys rehashed for 3,000 puts in the chaining table, 5,396 in the open-addressing one — both under 2n.)
- Growing by 8 slots: a resize every 6 or so puts, each rehashing all keys so far: $6 + 12 + 18 + \cdots \approx n^2/12$ — $\Theta(n^2)$ in total, $\Theta(n)$ amortised per put. The same argument as the `DynamicArray` with a constant growth step (Lecture 04).

Part C — Trace the code

W13-T1

Key	% 8	% 16
7	7	7
15	7	15
23	7	7
-9	7	7
40	0	8
100	4	4

Python's `%` is non-negative here: -9 = -2 × 8 + 7 = -1 × 16 + 7. At capacity 8, 7, 15, 23 and -9 share slot 7. At 16, 15 moves to 15, but 7, 23 and -9 still share slot 7: they differ by multiples of 16. Doubling the capacity separates only keys that differ in the next bit.

W13-T2

Operation	Chains	size	capacity	load
put 5	1: 5	1	4	0.25
put 9	1: 9 → 5	2	4	0.5
put 13	1: 13 → 9 → 5	3	4	0.75
put 2	1: 9 · 2: 2 · 5: 5 → 13	4	**8**	0.5
delete 9	2: 2 · 5: 5 → 13	3	8	0.375

`put 2` is pushed into bucket 2, making 4/4 = 1.0 > 0.75, so the table resizes to 8. The resize walks old bucket 1 front to back — 13, 9, 5 — pushing each on the front of its new chain: 13 → bucket 5, 9 → bucket 1, 5 → bucket 5 in front of 13. Then old bucket 2: 2 → bucket 2. `delete` 9 removes the only entry of bucket 1, which becomes `None`.

W13-T3

Operation	Probes	Slots after
put 3	3	`_ _ _ 3 _ _ _ _`
put 11	3, 4	`_ _ _ 3 11 _ _ _`
put 19	3, 4, 5	`_ _ _ 3 11 19 _ _`
delete 11	3, 4	`_ _ _ 3 † 19 _ _`
get 19	3, 4 (†: go on), 5 → 19	unchanged
put 27	3, 4 (†: remember), 5, 6 (None) → into slot 4	`_ _ _ 3 27 19 _ _`

`put 27` passes its load check: (2 + 1 + 1) / 8 = 0.5. It must probe on past the tombstone to slot 6 to be sure 27 is not already there, then goes back to the tombstone. The tombstone count returns to 0.

W13-T4

```
cat 0 6
act 0 2
tac 0 6
dog 2 4
god 2 4
```

`sum_hash`: cat, act and tac all sum to 312 (0 mod 8); dog and god to 314 (2 mod 8). They collide **because** they are anagrams, at every capacity — the hash ignores the order of the letters. `poly_hash` gives five different full hashes (98262, 96402, 114582, 99644, 102524), and the collisions left — cat with tac, dog with god — come only from folding them into 8 slots (3 bits). At another capacity they would be different pairs, or none. Collisions in a table of 8 slots are certain; collisions built into the hash function are a flaw.

W13-T5

Operation	Probes	Result	Slots after
put 0, 8, 16, 24	1, 2, 3 and 4 probes		`0 8 16 24 _ _ _ _`
get 24	0, 1, 2, 3	24	
get 32	0, 1, 2, 3, 4 (None)	None	
delete 8	0, 1		`0 † 16 24 _ _ _ _`
get 24	0, 1 (†), 2, 3	24	
put 40	0, 1 (†: remember), 2, 3, 4 (None) → slot 1		`0 40 16 24 _ _ _ _`
get 32	0, 1, 2, 3, 4	None	

`put 40`: load check (3 + 1 + 1) / 8 = 0.625, fine. Every key here has home 0, so they form one run, and the cost of each operation is its position in the run: the fourth key costs 4 probes, a miss 5. The tombstone neither shortens nor lengthens the run — which is exactly its job. With n such keys, a miss costs n + 1 probes: the $O(n)$ worst case, built from only 4 keys.

Part D — Table state: draw every step

W13-S1

Operation	Buckets (non-empty)	size	capacity
put 10	2: 10	1	4
put 3	2: 10 · 3: 3	2	4
put 14	2: 14 → 10 · 3: 3	3	4
put 7	2: 10 · 3: 3 · 6: 14 · 7: 7	4	**8**
put 18	2: 18 → 10 · 3: 3 · 6: 14 · 7: 7	5	8
delete 10	2: 18 · 3: 3 · 6: 14 · 7: 7	4	8
put 26	2: 26 → 18 · 3: 3 · 6: 14 · 7: 7	5	8
delete 26	2: 18 · 3: 3 · 6: 14 · 7: 7	4	8

`put 7` goes into bucket 3 (7 % 4) in front of 3, making 4/4 = 1.0 > 0.75: the table doubles. The resize walks old bucket 2 (14, then 10) and old bucket 3 (7, then 3): 14 → 6, 10 → 2, 7 → 7, 3 → 3. `delete 10` unlinks the **last** entry of a chain (`prev.next = None`); `delete 26` unlinks the **head** (the bucket slot itself changes). Loads after each: 0.25, 0.5, 0.75, 0.5, 0.625, 0.5, 0.625, 0.5.

W13-S2

Operation	Probes	Slots	size	†
put 5	5	`_ _ _ _ _ 5 _ _`	1	0
put 13	5, 6	`_ _ _ _ _ 5 13 _`	2	0
put 21	5, 6, 7	`_ _ _ _ _ 5 13 21`	3	0
put 6	6, 7, 0	`6 _ _ _ _ 5 13 21`	4	0
delete 13	5, 6	`6 _ _ _ _ 5 † 21`	3	1
put 29	5, 6 (†), 7, 0, 1 → slot 6	`6 _ _ _ _ 5 29 21`	4	0
delete 5	5	`6 _ _ _ _ † 29 21`	3	1
put 14	6, 7, 0, 1 → slot 1	`6 14 _ _ _ † 29 21`	4	1

Load checks: `put 6` (3 + 0 + 1) / 8 = 0.5; `put 29` (3 + 1 + 1) / 8 = 0.625; `put 14` (3 + 1 + 1) / 8 = 0.625 — none above 0.66. Note `put 6`: its home slot 6 is taken by 13, a key from **another** home — clustering. And `put 14` never passes the tombstone in slot 5 (its sequence starts at 6), so it cannot reuse it: the tombstone stays until a probe for a home-5 key reuses it or a rebuild drops it.

W13-S3

Operation	Load check (before the insert)	Action	Slots after	size	†
put 1 … put 5	at most (4 + 0 + 1)/8 = 0.625	—	`_ 1 2 3 4 5 _ _`	5	0
delete 1			`_ † 2 3 4 5 _ _`	4	1

Operation	Load check (before the insert)	Action	Slots after	size	†
delete 2			`_ † † 3 4 5 _ _`	3	2
put 9	(3 + 2 + 1)/8 = 0.75 > 0.66; live (3 + 1)/8 = 0.5	**sweep**, capacity 8	`_ 9 _ 3 4 5 _ _`	4	0
put 10	(4 + 0 + 1)/8 = 0.625	nothing	`_ 9 10 3 4 5 _ _`	5	0
put 11	(5 + 0 + 1)/8 = 0.75; live 0.75 > 0.66	**double**, to 16	slots 3, 4, 5, 9, 10, 11 of 16	6	0

`put 9`: the sweep re-inserts 3, 4 and 5 into a clean table of 8, and then 9 (home 1) finds slot 1 empty — the tombstones are gone. `put 10`: home 2, empty. `put 11`: the table doubles to 16, re-inserting 3, 4, 5, 9, 10 at their homes in 16 slots (all distinct), then 11 at slot 11.

Part E — Complexity analysis

W13-K1 — average $\Theta(n+m)$, worst $\Theta(nm)$. n puts and m lookups, each $O(1)$ on average (the puts amortised over the resizes). In the worst case — every key in one chain — a put or lookup walks up to n entries: the puts alone are $\Theta(n^2)$, and the lookups $\Theta(nm)$; in all $\Theta(n^2+nm)$. With a **list**, `in` is a linear scan: $\Theta(nm)$ in the worst case (every lookup a miss), whatever the keys.

W13-K2.

- **Never resizes:** the i-th put walks a chain of about i/8 entries to check for the key: $\sum i/8 \approx n^2/16$ — $\Theta(n^2)$. The table is 8 linked lists.
- **Doubles at 0.75:** chains stay at most about 0.75 long on average, so each put is $O(1)$ plus rehashing that totals less than 2n: $\Theta(n)$.
- **Adds 8 at 0.75:** a resize every 6 puts, each rehashing everything so far: $\Theta(n^2)$ in total — the same trap as the constant growth step of Lecture 04.

W13-K3.

- **(a)** n scans of n words: $\Theta(n^2)$.
- **(b)** sorting $O(n \log n)$ comparisons (each comparing words), then one pass over the runs: $O(n \log n)$ — and the counts come out in alphabetical order, a bonus if you need it.
- **(c)** one pass; each word is one `get` and one `put` on average $O(1)$: $O(n)$ **expected**. Worst case — every word in one bucket — $O(n^2)$.
- **Use (c)**, with the language's own hash table (`dict`, or `collections.Counter`): its string hashes are randomised, so an adversary cannot force the worst case. Use (b) if the output must be sorted anyway.

Part F — Find and fix the bug

W13-B1. It treats a key whose **value** is `None` as absent. After `m.put("k", None)`, `"k" in m` is `False` although `len(m)` counts it — the code ran, and printed `False True 2` for `"k"`, `"z"` (value 0) and `len`. `None` is a legal value, so it cannot also mean "missing". **Fix:** ask the question that is really meant — does `get` raise?

```
def __contains__(self, key):
    try:
        self.get(key)
    except KeyError:
        return False
    return True
```

(This is the version given in `dsa/hashmap.py`. The alternative is a private sentinel object as the default, which no caller can ever store.)

W13-B2. `entry = entry.next` runs **after** `entry.next` has been overwritten with the head of the new chain, so the walk jumps into the new table and the rest of the old chain is lost. On a `ChainingHashMap(capacity=4)` with put 1, 5, 9, 2: bucket 1 holds 9 → 5 → 1 when the 4th put resizes; 9 moves to bucket 1 of 8 with `next = None`, the loop ends, and 5 and 1 are gone. `get(1)` and `get(5)` raise `KeyError`, while `len(m)` still says 4. **Fix:** save the next entry first:

```
    while entry is not None:
        nxt = entry.next
        i = self._index(entry.key)
        entry.next = self._buckets[i]
        self._buckets[i] = entry
        entry = nxt
```

W13-B3. It inserts at the **first** tombstone without checking whether the key is further along its probe sequence. Run: `put(0, "a")`, `put(8, "b")` (slot 1), `delete(0)` (slot 0 becomes †), `put(8, "B")` — the key 8 is written into slot 0, although it is already in slot 1. `len(m)` is 2, `list(m)` is `[8, 8]`, and after `delete(8)` the table still answers `get(8)` with the old `"b"`. It also forgets `_tombstones -= 1` when it reuses one. **Fix:** remember the first tombstone, keep probing until the key or `None`, and only then insert (Lecture 13, the tombstone table; the reference `put`).

W13-B4. `self._buckets[i] = entry.next` is right only when the match is the **head** of its chain. For a match further in, it makes the bucket start after the match, dropping every entry **before** it. With 3, 11, 19 in one chain (19 → 11 → 3), `delete(11)` leaves bucket 3 as just 3: `len(m)` says 2, but 19 is gone — `get(19)` raises `KeyError`. The graded tests only delete from a one-key table, so they pass. **Fix:** walk with `prev`, and unlink with `prev.next = entry.next` unless `prev is None`:

```
def delete(self, key):
    i = self._index(key)
    prev, entry = None, self._buckets[i]
    while entry is not None:
```

```
        if entry.key == key:
            if prev is None:
                self._buckets[i] = entry.next
            else:
                prev.next = entry.next
            self._size -= 1
            return
        prev, entry = entry, entry.next
    raise KeyError(key)
```

Part G — Write the code

Every solution here is the code in `solutions/practice/week13.py`, line for line, and passes `pytest --solutions tests/test_practice_week13.py`. Each uses a `ChainingHashMap` as its working storage.

W13-C1

```
def two_sum(values, target):
    """W13-C1. Indices (i, j), i < j, with values[i] + values[j] ==
    ↪ target."""
    first_seen = ChainingHashMap()              # value -> first index seen
    for j, value in enumerate(values):
        i = first_seen.get(target - value, None)
        if i is not None:
            return (i, j)
        if value not in first_seen:
            first_seen.put(value, j)
    return None
```

One pass. For each value, the partner it needs is `target - value`; if that partner has already been seen, its **first** index is the smallest i for this j, and this j is the smallest j with any partner — so the first pair found is the one the docstring asks for. Store a value only the **first** time it is seen: overwriting it with later indices would make `[3, 3, 5]` with target 8 return `(1, 2)` instead of `(0, 2)`. $O(n)$ expected, against $O(n^2)$ for trying every pair.

W13-C2

```
def first_repeated(items):
    """W13-C2. The first item to appear for the second time, or None."""
    seen = ChainingHashMap()                    # used as a set: item -> True
    for item in items:
        if item in seen:
            return item
        seen.put(item, True)
```

```
    return None
```

The map is used as a **set** — only its keys matter. Returning the moment an item is found again gives the item whose second occurrence is earliest. $O(n)$ expected; the obvious "for each item, look back through the earlier ones" is $O(n^2)$.

W13-C3

```
def group_anagrams(words):
    """W13-C3. Group the words that are anagrams of one another."""
    groups = []
    where = ChainingHashMap()                    # sorted letters -> group
↪   index
    for word in words:
        key = "".join(sorted(word))
        i = where.get(key, None)
        if i is None:
            where.put(key, len(groups))
            groups.append([word])
        else:
            groups[i].append(word)
    return groups
```

Two words are anagrams exactly when their sorted letters are equal, so `"".join(sorted(word))` is the key (a string: hashable, unlike the list `sorted` returns). The map sends the key to the **position** of its group in the result, so the groups come out in order of first appearance, and each group grows in input order. The result list is output, which the storage rule allows. Sorting each word costs $O(k \log k)$: $O(nk \log k)$ in all.

W13-C4

```
def longest_distinct_run(text):
    """W13-C4. The longest stretch of text with no repeated character."""
    last_seen = ChainingHashMap()                # character -> last index
    start = best = 0
    for i, ch in enumerate(text):
        previous = last_seen.get(ch, -1)
       if previous >= start:                   # ch repeats inside the window
            start = previous + 1
        last_seen.put(ch, i)
        best = max(best, i - start + 1)
    return best
```

Why $O(n)$: the window `text[start..i]` never contains a repeat. When `ch` was last seen **inside** the window (at `previous >= start`), the window must start just after that position; `start` only ever moves **forward**, so the two ends of the window each move at most n times. The map remembers each character's last position so the jump is $O(1)$, with no scan. The test `previous`

`>= start` matters: a character last seen **before** the window (in "abba", the second a) must not drag `start` back.

W13-C5

```
def count_subarrays_with_sum(values, k):
    """W13-C5. How many contiguous runs of values add up to k?"""
    seen = ChainingHashMap()                    # prefix sum -> times seen
    seen.put(0, 1)                              # the empty prefix
    total = count = 0
    for value in values:
        total += value
        count += seen.get(total - k, 0)
        seen.put(total, seen.get(total, 0) + 1)
    return count
```

Why not a sliding window: with negative values, extending a window can make its sum smaller, so "too big — shrink it" is no longer a safe rule. **Why prefix sums:** `values[i:j]` adds up to `prefix[j] - prefix[i]`. So at each j, the runs ending there that sum to k are exactly the earlier prefixes equal to `prefix[j] - k`; the map counts how many times each prefix sum has occurred (starting with the empty prefix, 0, once). One pass: $O(n)$ expected. `[0] * n` with k = 0 has $n(n+1)/2$ runs — 200,010,000 for n = 20,000 — so no method that lists them one by one can be fast.

Answers — Week 14

> ***The questions*** *are at the end of the* Week 14 *chapter. Commit to your own answer before reading one here. Every trace and output below was produced by running the reference solution,* `solutions/dsa/graph.py`*.*
>
> *For reference, the neighbour lists of* ***H*** *are A: B D · B: A C E · D: A E · C: B F · E: B D F · F: C E G · G: F, and* `H.nodes()` *is A B D C E F G (the order the nodes were first added).*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	c	M07	b	M13	b	M19	d
M02	a	M08	d	M14	c	M20	a
M03	d	M09	a	M15	d	M21	c
M04	a	M10	b	M16	c	M22	d
M05	b	M11	b	M17	a		
M06	c	M12	a	M18	c		

W14-M01 — c. A V × V grid, whatever E is. $O(V + E)$ (a) is the adjacency list.

W14-M02 — a. First discovered, first expanded. A stack (b) gives DFS; a priority queue (c) gives Dijkstra's algorithm, beyond this course.

W14-M03 — d. The degrees add up to $2 + 3 + 2 + 2 + 3 + 3 + 1 = 16$, and every edge is counted at both ends: $E = 16/2 = 8$. 16 (a) forgets to halve. (These are the degrees of H.)

W14-M04 — a. The queue holds all of distance d before any of distance d + 1, so a vertex is discovered while BFS is expanding the layer just before it. It never tries every path (c) — that would be exponential. Marking on dequeue (b) is a bug (W14-B1).

W14-M05 — b. A adds B and D; B adds C and E; D adds nothing new; C adds F; F adds G (W14-T1). (a) is alphabetical order, which BFS does not use.

W14-M06 — c. A → B → C → F → E → D, back up to F, then G (W14-T2). (a) is what you get if you think of BFS layers; (b) goes from C to E, but C and E are not adjacent.

W14-M07 — b. D's list is A, E, so A is taken first and becomes B's parent; B then discovers C. D–E–B–C (c) and D–E–F–C (a) are also three edges long — equally short — but BFS keeps the first parent it records (W14-T4).

W14-M08 — d. One cell, `matrix[i][j]`, after two $O(1)$ hash-map lookups for i and j. $O(\deg u)$ (b) is the list.

W14-M09 — a. The whole row of V cells must be scanned to find the 1s. That is the matrix's price.

W14-M10 — b. V vertices, each costing a V-cell row scan: V^2. The number of edges does not change the work.

W14-M11 — b. Each undirected edge u–v is stored in u's list and in v's list. The same fact is $\sum \deg(v) = 2E$.

W14-M12 — a. A vertex with two neighbours that are taken before it is enqueued by both, and then appears twice in the order (W14-B1). BFS still finds every vertex (b) — it just repeats some.

W14-M13 — b. A stack is **last** in, first out (not a): pushing D then B from A's list B, D puts B on top, so B comes off first — the same choice the recursive `for` loop makes.

W14-M14 — c. One call frame per vertex on the path; Python's default limit is 1,000 frames. The reference `dfs` already fails on a path of 1,000 edges. `dfs_iterative` handles it.

W14-M15 — d. Directed acyclic graph. (a) describes a tree.

W14-M16 — c. Every vertex keeps in-degree 1, so the ready queue starts empty, nothing is output, and 0 < 3 vertices means a cycle: `None`. The `ValueError` (d) is for an **undirected** graph.

W14-M17 — a. When DFS enters B from A, B's list contains A. Without the check every edge would be reported as a cycle (W14-B3).

W14-M18 — c. D was finished when the search backed up from B; it is no longer on the current path A $\to$ C. Only an edge to a **grey** vertex (b) closes a cycle.

W14-M19 — d. {A, B, C}, {D, E}, {F, G}, and H and I on their own. A vertex with no edges is a component of its own; 4 (a) and 3 (b) forget H and I.

W14-M20 — a. In-degree 0 at the start: A and B, queued in node order. A releases nothing (C still waits on B); B releases C and D; C, then D, then E, then F (W14-T5). (b) is also a valid topological order — it is the DFS one — but it is not what Kahn's algorithm returns. (c) puts C before B, although B $\to$ C.

W14-M21 — c. BFS counts edges; with road lengths the fewest roads is not the fewest kilometres. That needs Dijkstra's algorithm, which this course only names.

W14-M22 — d. Every vertex is enqueued and dequeued once; every list is read once, $\sum \deg = 2E$ entries in total.

Part B — Short answer and essay

W14-E1 *(4)*

- **Memory:** list $O(V + E)$ — $V + 2E$ cells undirected; matrix V^2 cells, however few edges.
- **`has_edge(u, v)`:** list $O(\deg u)$, scan u's neighbours; matrix $O(1)$, one cell. **`neighbours(u)`:** list $O(\deg u)$; matrix $O(V)$, a row scan. **One BFS/DFS:** list $O(V + E)$; matrix $O(V^2)$.

- **Road map**, V = 100,000, about three roads per junction (E about 150,000): the list holds about $100{,}000 + 300{,}000 = 400{,}000$ cells; the matrix 10^{10} cells — about 10 GB even at one byte each. The list, without question; a BFS on it is a few hundred thousand steps.
- **500 people, nearly everyone knows everyone:** E is close to $500 \times 499/2 \approx 125{,}000$ and the matrix is only $250{,}000$ cells — about the same as the list. Each `has_edge` is $O(1)$ on the matrix against a scan of up to about 500 neighbours on the list; asked millions of times, the matrix wins by a large factor.

W14-E2 *(4)*

- **Algorithm:** put s in a queue and mark it; repeatedly dequeue a vertex, output it, and enqueue (and mark) each unmarked neighbour. $O(V + E)$.
- **Why shortest:** the queue holds the vertices in non-decreasing order of distance from s — all of distance d before any of distance d + 1 — so a vertex is first discovered from a vertex one layer nearer, and no later route can be shorter.
- **Rebuilding the path:** record, for each vertex, the vertex it was discovered from (the parent, `None` for s). When the goal is dequeued, follow the parents back to s and reverse the list; if the queue empties first, the goal is unreachable — `None`.
- **Weighted roads:** BFS minimises the **number** of edges. A route of two 40 km roads beats one of three 1 km roads by BFS's measure. Weighted shortest paths need Dijkstra's algorithm (a priority queue), beyond this course.

W14-E3 *(4)*

- **The call stack** holds one frame per vertex on the current path: which vertex it is, and how far its `for` loop over the neighbours has got. Returning from a call is backing up.
- **Mark on pop:** recursion marks a vertex when it **enters** it, and may reach it by a deeper route before an earlier vertex's loop gets to it. Marking on push would fix it to the earlier route. So allow a vertex to be pushed more than once and skip it if it is already visited when popped.
- **Push in reverse:** the stack returns the last push first; to take the first neighbour first, it must be pushed last.
- **Why bother in Python:** the recursive version fails with `RecursionError` once the path is about 1,000 vertices deep; your own `Stack` can grow as large as memory allows. (It also shows exactly what recursion was doing.)

W14-E4 *(4)*

- **Undirected:** DFS remembering each vertex's parent; a visited neighbour that is **not** the parent closes a cycle. The parent must be excluded because every edge is stored at both ends.
- **Directed:** three colours — white (unseen), grey (on the current path), black (finished). An edge to a **grey** vertex closes a cycle; an edge to a black one does not.
- **Parent rule on a directed graph fails:** A $\to$ B, B $\to$ A is a cycle, but at B the neighbour A is its parent and is skipped — "no cycle".
- **Visited-only rule on a directed graph fails:** the diamond A $\to$ B $\to$ D, A $\to$ C $\to$ D, or simply B $\to$ C and A $\to$ C searched from B first: C is visited when reached again, and the check says "cycle" wrongly (W14-B4).

- **Colour rule on an undirected graph fails:** at B, entered from A, the neighbour A is grey, so every edge is reported as a cycle.

W14-E5 *(3)*

- **Definition:** an order of the vertices of a directed graph in which, for every edge X $\rightarrow$ Y, X comes before Y. It exists **exactly** when the graph is a DAG: a cycle would need each of its vertices to come before the next.
- **Kahn:** count in-degrees; queue the vertices with in-degree 0; repeatedly dequeue one, output it, and subtract 1 from the in-degree of each vertex it points to, queueing any that reach 0. $O(V + E)$.
- **No order:** if fewer than V vertices are output, the rest wait on each other — they contain a cycle — so return `None`.
- **Application:** course prerequisites (a valid study plan), build systems (compile after dependencies), package managers, spreadsheet recalculation.

Part C — Trace the algorithms

W14-T1

Take	Adds	Queue afterwards
A	B, D	B D
B	C, E (A is visited)	D C E
D	— (A, E are visited)	C E
C	F	E F
E	— (B, D, F are visited)	F
F	G	G
G	—	empty

Order **A B D C E F G**. Distances from A: A 0; B, D 1; C, E 2; F 3; G 4. E is seen by B first, so when D is taken E is already waiting — that is why D adds nothing.

W14-T2

```
visit A
  visit B                    (A: its first neighbour)
    A already visited
    visit C
      B already visited
      visit F
        C already visited
        visit E
          B already visited
          visit D
            A already visited
```

```
            E already visited
          finish D
          F already visited
        finish E
        visit G
          F already visited
        finish G
      finish F
    finish C
    E already visited
  finish B
  D already visited
finish A
```

Order **A B C F E D G**. The deepest point is 6 frames: A, B, C, F, E, D. G is entered only after D and E have finished — it hangs off F, the first vertex on the way back with an unvisited neighbour.

W14-T3 Stack written bottom → top.

Pop	Action	Push	Stack afterwards
A	visit	D, B	D B
B	visit	E, C	D E C
C	visit	F	D E F
F	visit	G, E	D E G E
E	visit	D	D E G D
D	visit	—	D E G
G	visit	—	D E
E	already visited: skip		D
D	already visited: skip		empty

Order **A B C F E D G** — the same as W14-T2. **D** (by A and by E) and **E** (by B and by F) are pushed twice; in each case the copy pushed later, from deeper in the search, is popped first, exactly as the recursion enters them.

W14-T4 BFS from A records each parent when the vertex is first discovered:

Taken	New parents
A	B ← A, D ← A
B	C ← B, E ← B
D	—
C	F ← C
E	—
F	G ← F
G	the goal: stop

Walk back from G: G ← F ← C ← B ← A, reversed: **`["A", "B", "C", "F", "G"]`**, 4 edges. (A–D–E–F–G is also 4 edges; F's parent is C because C was taken before E.)

`shortest_path_unweighted(H, "D", "C")` = **`["D", "A", "B", "C"]`**. From D the queue is A, E. A is taken first and records B ← A; when E is taken, B already has a parent, so E records only F ← E; then B records C ← B. D–E–B–C and D–E–F–C are also 3 edges long. BFS guarantees *a* shortest path, and the neighbour order decides which one.

W14-T5 In-degrees at the start: A 0, C 2, B 0, D 1, E 2, F 1. Queue: A B.

Take	In-degrees that change	Reach 0	Queue afterwards
A	C: 2 → 1	—	B
B	C: 1 → 0, D: 1 → 0	C, D	C D
C	E: 2 → 1	—	D
D	E: 1 → 0	E	E
E	F: 1 → 0	F	F
F	—	—	empty

Six vertices out: **A B C D E F**. **Yes**, B D A C E F is also valid: check every edge — A before C, B before C and D, C and D before E, E before F. It is the order `topological_order_dfs` (W14-C3) returns. A DAG usually has many topological orders.

Part D — Graph state

W14-S1

After	Adjacency lists
`add_edge A B`	A: B · B: A
`add_edge B C`	A: B · B: A C · C: B
`add_node D`	A: B · B: A C · C: B · D: *(empty)*
`add_edge C A`	A: B C · B: A C · C: B A · D: *(empty)*
`add_edge D B`	A: B C · B: A C D · C: B A · D: B

The matrix at the end (rows and columns A, B, C, D) — symmetric, because the graph is undirected:

```
    A  B  C  D
A   0  1  1  0
B   1  0  1  1
C   1  1  0  0
D   0  1  0  0
```

`nodes()` = A B C D. `edges()` = (A, B), (A, C), (B, C), (B, D) — each once; the same list from both classes. `degree("B")` = 3. `add_node D` added a row **and** a column of zeros to the matrix; it would add nothing more if D were added again.

W14-S2

Step	White	Grey (on the path)	Black
enter A	B C D E F	A	
enter B	C D E F	A B	
enter C	D E F	A B C	
finish C (no out-edges)	D E F	A B	C
finish B	D E F	A	B C
enter D (from A)	E F	A D	B C
D → C: C is **black** — skip	E F	A D	B C
enter E	F	A D E	B C
enter F		A D E F	B C
F → D: D is **grey** — **cycle**			

The edge **F → D** proves the cycle D → E → F → D: D's call is still running. **D → C** reaches a black vertex: C was finished long ago, and nothing leads from C back to D; it is only a second route to C (A → B → C and A → D → C). `topological_sort(Q)` returns **`None`**. Without F → D it returns **A B D C E F**.

W14-S3

```
enter A (parent None)
  enter B (parent A)
    A is the parent: skip
  done B
  enter C (parent A)
    A is the parent: skip
    enter D (parent C)
      C is the parent: skip
      enter E (parent D)
        D is the parent: skip
        C is visited and not the parent: cycle
```

The cycle C–D–E–C is found at E, looking at its neighbour C: C is visited, and E was entered from D, not from C. Every "skip" line is an edge seen from its other end — without the parent check the search would have stopped, wrongly, at the very first one, B's neighbour A.

Part E — Complexity analysis

W14-K1. On a `Graph`: $\Theta(V+E)$ — V iterations of the outer loop, and the inner loops together read every list once, $\sum \deg = 2E$ entries (E for a directed graph). On a `MatrixGraph`: $\Theta(V^2)$ — each `neighbours` call scans a row of V cells, even to return nothing. It computes $\sum_v \deg(v) = 2E$, **twice** the number of edges.

W14-K2. The result has one list per **vertex**, not per component: each component of size k appears k times (in different orders). Cost: every call is a full BFS of that vertex's component, so a connected graph costs V searches of $O(V+E)$ — $\Theta(V(V+E))$. The correct version starts a search only from an unseen vertex, so every vertex and edge belongs to exactly one search: $\Theta(V+E)$ in total.

W14-K3. On a `Graph`: for each u, V calls of `has_edge(u, v)`, each $O(\deg u)$ — $\Theta\left(\sum_u V(1+\deg u)\right) = \Theta(V^2 + VE)$; for a dense graph that is $\Theta(V^3)$. On a `MatrixGraph`: V^2 calls of $O(1)$ — $\Theta(V^2)$. `len(graph.edges())` is $\Theta(V+E)$ on the list and $\Theta(V^2)$ on the matrix: never ask about every pair when you can walk the lists.

Part F — Find and fix the bug

W14-B1. A vertex is marked only when it is dequeued, so while it waits in the queue another vertex can enqueue it again. On H, `bfs(H, "A")` returns `A B D C E E F F F G G G` — E is enqueued by B and by D, F three times, and each copy of F enqueues G again. On the triangle A–B, A–C, B–C it returns `A B C C`. `test_bfs_visits_each_node_once` fails. **Fix:** mark the start before the loop, and mark each neighbour **when it is enqueued**:

```
    visited.put(start, True)
    queue.enqueue(start)
    ...
            if neighbour not in visited:
                visited.put(neighbour, True)
                queue.enqueue(neighbour)
```

W14-B2. Adding a node that already exists replaces its neighbour list with an empty one — and `add_edge` calls `add_node` on both ends every time. After `add_edge("A", "B")` and `add_edge("B", "C")`, B's list is just `[C]`: `has_edge("B", "A")` is `False` while `has_edge("A", "B")` is still `True`, and `nodes()` is `["A", "B", "B", "C"]` (B twice) while `len` is 3. **Fix:** do nothing if the node is already there:

```
def add_node(self, node):
    if node not in self._adjacent:
        self._adjacent.put(node, DynamicArray())
        self._order.append(node)
```

W14-B3. No parent check. In an undirected graph, the vertex you came from is always in your list and always visited, so the function returns `True` for **any** graph with an edge — even the single edge A–B. **Fix:** pass the parent down and ignore it:

```
    def visit(node, parent):
        visited.put(node, True)
        for neighbour in graph.neighbours(node):
            if neighbour not in visited:
                if visit(neighbour, node):
```

```
                    return True
            elif neighbour != parent:
                return True
        return False
    ...
        if node not in visited and visit(node, None):
```

W14-B4. "Visited" includes finished vertices, which are not on the current path. Build B → C and A → C, in that order: the loop visits B, then C (finished, no out-edges); then it starts at A, and A's neighbour C is visited — `True`, though the graph has no cycle. The diamond A → B, A → C, B → D, C → D fails the same way (`test_directed_diamond_is_not_a_cycle`). **Fix:** three colours — grey while the call runs, black when it returns; only grey means a cycle:

```
    def visit(node):
        colour.put(node, GREY)
        for neighbour in graph.neighbours(node):
            state = colour.get(neighbour, WHITE)
            if state == GREY:
                return True
            if state == WHITE and visit(neighbour):
                return True
        colour.put(node, BLACK)
        return False
```

Part G — Write the code

The helper constant used by C1 and C5:

```
STEPS = ((-1, 0), (1, 0), (0, -1), (0, 1))          # up, down, left, right
```

W14-C1

```
def maze_distance(maze):
    rows = len(maze)
    cols = len(maze[0]) if rows else 0
    start = None
    for r in range(rows):
        for c in range(cols):
            if maze[r][c] == "S":
                start = (r, c)
    distance = ChainingHashMap()                    # cell -> steps from S
    distance.put(start, 0)
    queue = CircularQueue()
    queue.enqueue(start)
```

```
    while not queue.is_empty():
        r, c = queue.dequeue()
        if maze[r][c] == "T":
            return distance.get((r, c))
        for dr, dc in STEPS:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc] != "#" \
                    and (nr, nc) not in distance:
                distance.put((nr, nc), distance.get((r, c)) + 1)
                queue.enqueue((nr, nc))
    return -1
```

A BFS over the grid, with the cells as vertices named by `(row, column)` tuples, and the distance map doubling as the visited set. The first time T is dequeued, its distance is the answer; if the queue empties, T cannot be reached. Building a `Graph` of the cells first, as in the lecture's maze, works too — this version just reads the neighbours straight from the grid. Each cell is enqueued at most once and looks at four neighbours: $O(R \cdot C)$.

W14-C2

```
def is_bipartite(graph):
    colour = ChainingHashMap()                      # vertex -> 0 or 1
    for start in graph.nodes():
        if start in colour:
            continue
        colour.put(start, 0)
        queue = CircularQueue()
        queue.enqueue(start)
        while not queue.is_empty():
            node = queue.dequeue()
            for neighbour in graph.neighbours(node):
                if neighbour not in colour:
                    colour.put(neighbour, 1 - colour.get(node))
                    queue.enqueue(neighbour)
                elif colour.get(neighbour) == colour.get(node):
                  return False                      # an edge inside one colour
    return True
```

Give the start of each component colour 0, and every newly discovered vertex the other colour from the vertex that discovered it. An edge whose two ends end up with the same colour makes two-colouring impossible. **Why an odd cycle fails:** walking round a cycle, the colours must alternate 0, 1, 0, 1, …; after an odd number of steps you are back at the start with the opposite colour — a contradiction. (The converse also holds: a graph with no odd cycle is bipartite.) The outer loop matters: the odd cycle may be in any component. $O(V + E)$.

W14-C3

```
WHITE, GREY, BLACK = 0, 1, 2


def topological_order_dfs(graph):
    colour = ChainingHashMap()                          # absent means WHITE
    finished = []

    def visit(node):
        colour.put(node, GREY)
        for neighbour in graph.neighbours(node):
            state = colour.get(neighbour, WHITE)
            if state == GREY:
                return False                            # back into the current path
            if state == WHITE and not visit(neighbour):
                return False
        colour.put(node, BLACK)
        finished.append(node)                           # everything after it is
↪   done
        return True

    for node in graph.nodes():
        if colour.get(node, WHITE) == WHITE and not visit(node):
            return None
    finished.reverse()
    return finished
```

A vertex is appended to `finished` only after every vertex it points to has finished, so in the reversed list it comes before all of them — every edge points forwards. A grey neighbour is an edge back into the current path: a cycle, so no order exists. For P the answer is B D A C E F; for the diamond in the docstring, A C B D. The inner function is recursive, so this version has the same depth limit as `dfs`.

W14-C4

```
def semester_plan(prerequisites):
    graph = Graph(directed=True)
    for before, after in prerequisites:
        graph.add_edge(before, after)
    waiting = ChainingHashMap()                         # course -> unmet
↪   prerequisites
    for course in graph.nodes():
        waiting.put(course, 0)
    for course in graph.nodes():
        for later in graph.neighbours(course):
            waiting.put(later, waiting.get(later) + 1)
    semester = [course for course in graph.nodes() if waiting.get(course)
↪   == 0]
```

```
    plan, placed = [], 0
    while semester:
        semester.sort()
        plan.append(semester)
        placed += len(semester)
        following = []
        for course in semester:
            for later in graph.neighbours(course):
                waiting.put(later, waiting.get(later) - 1)
                if waiting.get(later) == 0:
                    following.append(later)
        semester = following
    if placed < len(graph):                          # the rest wait on each
     ↪  other
        return None
    return plan
```

Kahn's algorithm, but taking a whole layer at a time: the courses whose prerequisites are all done form this semester; finishing them may free the next. **Why the fewest semesters is the longest chain:** along a chain of k courses, each needs the one before, so no plan can finish it in fewer than k semesters. And the layer of a course in this plan is exactly the length of the longest chain ending at it — it is taken the semester after its last prerequisite, never later. So the plan uses as many semesters as the longest chain, and no plan can use fewer. On the lecture's bylaw graph it gives four. (Real plans are also limited by credit hours per semester, which this ignores.)

W14-C5

```
def count_islands(grid):
    rows = len(grid)
    cols = len(grid[0]) if rows else 0
    seen = ChainingHashMap()
    islands = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != "1" or (r, c) in seen:
                continue
            islands += 1                             # a new island: flood it
            seen.put((r, c), True)
            queue = CircularQueue()
            queue.enqueue((r, c))
            while not queue.is_empty():
                cr, cc = queue.dequeue()
                for dr, dc in STEPS:
                    nr, nc = cr + dr, cc + dc
                    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] ==
                     ↪  "1" \
```

```
                            and (nr, nc) not in seen:
                        seen.put((nr, nc), True)
                        queue.enqueue((nr, nc))
    return islands
```

Connected components on a grid: every land cell not yet seen starts a new island, and a BFS "floods" it, marking every land cell joined to it. Each cell is seen once: $O(R \cdot C)$. A recursive flood fill is shorter, but an island 2,000 cells long is deeper than Python's recursion limit — `test_count_islands_on_a_large_grid` checks exactly that.

Answers — Week 15

> ***The questions*** *are at the end of the* Week 15 *chapter. Commit to your own answer before reading one here. Every trace, tree and value below was produced by running the reference solution.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M07	a	M13	c	M19	a
M02	c	M08	d	M14	b	M20	d
M03	a	M09	b	M15	a	M21	c
M04	d	M10	c	M16	d	M22	b
M05	c	M11	a	M17	b		
M06	b	M12	d	M18	c		

W15-M01 — b. Values are the evaluator's job (a), balanced parentheses and the tree the parser's (c, d). The lexer knows nothing about structure.

W15-M02 — c. `['12', '+', '3']`. `12` is one token: the lexer keeps reading while the next character is a digit.

W15-M03 — a. Every token is valid, so the lexer accepts it (b); but after the `+`, `factor` finds `*`, which cannot start a factor: `ValueError: unexpected '*'`. Nothing reaches the evaluator (c).

W15-M04 — d. "Abstract" because it drops the grammar's bookkeeping — the parentheses, and the calls that only passed a value up.

W15-M05 — c. An `expr` is a sequence of `term`s, and every `*` is consumed inside a `term` before `expr` sees the next `+`; so `*` ends up deeper in the tree. There is no table (a) — that was shunting-yard.

W15-M06 — b. `2 + (3 * 4) - 1 = 13`. 19 (a) ignores precedence: `((2 + 3) * 4) - 1`.

W15-M07 — a. `(2 * 3) + 4`: the last operator applied is the root.

W15-M08 — d. Each parenthesised group is a factor of the one `term` `(…) * (…)`. Parentheses (c) never appear in the tree.

W15-M09 — b. Left-associative: the leftmost operation first. `1 - (2 - 3)` (c) would be 2.

W15-M10 — c. It builds `8 - (4 - 2) = 6`: the recursive call parses the **whole rest** of the line and hangs it on the right. The correct value is 2 (a).

W15-M11 — a. `expr` calls `expr` at the same `position`, before reading anything, so no call is ever closer to a base case. Python stops it at its recursion limit, so it does use memory — a frame per call (d is wrong).

W15-M12 — d. A `while` loop that runs while `peek()` is one of the operators. A rule name becomes a call (a); `|` becomes an `if` (c).

W15-M13 — c. One per number: `1`, `2` and `3`. (Measured: 3 calls.)

W15-M14 — b. Unary minus is a `factor`, so it binds only the `5`: `(0 - 5) + 2 = -3`. Option (a) is the tree a parser builds if unary minus calls `expr` instead of `factor` (W15-B3). The lexer never produces negative numbers (c): `-5` is two tokens.

W15-M15 — a. An operator can be applied only once both operand values are known: children first.

W15-M16 — d. Left subtree `3 4 +`, right subtree `2`, then the root `*`. It is also the postfix form of Lecture 06.

W15-M17 — b. `expr` parses `3 + 4` and stops at the `)`, which no rule can use; only the final check notices it. (a) and (c) fail inside `factor` anyway.

W15-M18 — c. 1,000 / 3 is about 333, less a few frames already in use: measured from a script, 329 pairs work and 330 raise `RecursionError`.

W15-M19 — a. Each stage touches each token or node a constant number of times.

W15-M20 — d. The parser uses a loop (a is about the parser), but it builds a tree of height n - 1, and `evaluate` recurses once per level. $\log_2 n$ (b) is the balanced case.

W15-M21 — c. The bytecode is in `__pycache__`; `dis` shows it.

W15-M22 — b. Both routes are $O(n)$ (a). The tree route **needs** recursion (c), and memory for the tree (d).

Part B — Short answer and essay

W15-E1 *(4)*

- **Lexer:** text → tokens. Rejects characters outside the language: `"3 $ 4"` → `ValueError` (unexpected character).
- **Parser:** tokens → tree (AST), following the grammar. Rejects valid tokens in an invalid order: `"3 + * 4"`, `"(3 + 4"`, `"3 + 4 )"` → `ValueError`.
- **Evaluator:** tree → value. Rejects well-formed expressions with no value: `"1 / 0"` → `ZeroDivisionError`.
- **Compiler against interpreter:** a compiler translates the whole program ahead of time into another program (machine code, bytecode) that runs later; an interpreter executes as it goes and produces the result. Both share the lexer and parser; a compiler replaces the evaluator by a code generator. CPython does both: compile to bytecode, then interpret it.

W15-E2 *(4)*

- `expr := term (('+' | '-') term)*` — an expression is a sum of terms; `term := factor (('*' | '/') factor)*` — a term is a product of factors.
- To parse `3 + 4 * 2`, `expr` splits at `+` into the terms `3` and `4 * 2`; the `*` is consumed **inside** the second term. In the tree, `*` is below `+`: `+` over `3` and `*(4, 2)`. Deeper nodes are evaluated first, so `*` "wins" — value 11.
- The lower the rule an operator lives in, the tighter it binds: precedence is **which rule** handles the operator, not a number.
- Parentheses: `factor := '(' expr ')'` makes a whole parenthesised expression a **factor**, the tightest level, so `(3 + 4) * 2` puts `+` below `*`.
- A tighter operator such as `^` gets a **new rule between `term` and `factor`**, so that `term` is built from it and it is built from factors (W15-C5).

W15-E3 *(4)*

- **Associativity** decides how operators of the **same** precedence group: left-associative `1 - 2 - 3 = (1 - 2) - 3 = -4`.
- The loop in `expr` does `node = BinOp(op, node, self.term())`: the tree built so far becomes the **left** child of each new operator, so the first operation ends up deepest and is evaluated first — left associativity.
- **Right recursion** (`expr := term ('+' expr)?`) puts the recursive result — the whole rest of the line — on the right: `1 - 2 - 3` becomes `1 - (2 - 3) = 2`. No error; just a wrong answer, and only for chains of `-` or `/` (W15-B1).
- **Left recursion** (`expr := expr '+' term`) calls `expr` first, with nothing consumed: the recursion makes no progress, and **every** input, even `3`, ends in `RecursionError`.
- The `( … )*` form with a loop gives left associativity **and** terminates.

W15-E4 *(3)*

- **Stack route:** `infix_to_postfix` (operator stack, precedence table) then `evaluate_postfix` (value stack). $O(n)$; no recursion, so no depth limit; builds nothing but a list.
- **Tree route:** `Parser` (recursive descent) then `evaluate` (post-order). $O(n)$ too; the tree can be reused — printed, simplified, compiled — and the grammar is easy to extend (unary minus is one case in `factor`; the Week 6 `infix_to_postfix` produces `5 -` for `-5`, which `evaluate_postfix` rejects). Its recursion depth follows the tree's height.
- **Connection:** the postfix list is exactly the **post-order** traversal of the tree. Shunting-yard emits the post-order of a tree it never builds.

W15-E5 *(3)*

- Each method call pushes a frame holding its own locals — including its `node` so far and where it is in its rule. At any moment the frames on the call stack are one path of the parse tree: "inside a `factor`, inside a `term`, inside an `expr`…". Returning pops a frame and resumes the rule that called it. A hand-written parser would need its own explicit stack for this; recursion gets it for free.
- **Depth** is set by the **shape** of the input, not its length: three frames per level of parentheses (`expr`, `term`, `factor`); a flat chain `1 + 1 + …` is a loop and adds nothing to the parser's depth (but see W15-K2 for `evaluate`).

- **Too deep:** Python's limit (1,000 frames by default) raises `RecursionError` — at about 330 nested pairs of parentheses. Real compilers raise the limit, or use an explicit stack for deep cases.

Part C — Trace the code

W15-T1. `['4.5', '*', '(', '10', '-', '3', ')']`. Start indices: `4.5` at 2, `*` at 5, `(` at 6, `10` at 7, `-` at 9, `3` at 10, `)` at 11. The two leading spaces (indices 0 and 1) and the trailing one (index 12) are skipped: they separate tokens but never become one. `4.5` is read as one token: from index 2, the scan continues over `.` and `5` and stops at `*`.

W15-T2.

```
expr    at 0 ('2')
  term    at 0 ('2')
    factor  at 0 ('2')        -> Num(2)
    factor  at 2 ('(')        (term consumed '*')
      expr    at 3 ('3')
        term    at 3 ('3')
          factor  at 3 ('3')  -> Num(3)
        term    at 5 ('1')    (expr consumed '-')
          factor  at 5 ('1')  -> Num(1); ')' consumed by the outer factor
  term    at 8 ('4')          (expr consumed '+')
    factor  at 8 ('4')        -> Num(4)
```

Tree: `BinOp('+', BinOp('*', Num(2.0), BinOp('-', Num(3.0), Num(1.0))), Num(4.0))` — `+` at the root, `*` on its left with `2` and the `-` below it, `4` on its right. Value **8.0**. Eleven calls; the deepest point has 6 frames of the three methods.

W15-T3. The tree: `*` at the root; its left child `/`, whose children are `-`(8, 2) and `+`(1, 2); its right child `3`.

Order	Node	Value
1	8	8
2	2	2
3	-	6
4	1	1
5	2	2
6	+	3
7	/	2
8	3	3
9	*	**6**

Result **6.0**. The labels in this order, `8 2 - 1 2 + / 3 *`, are the postfix list.

W15-T4. `infix_to_postfix` gives `['2', '3', '1', '-', '*', '4', '+']`.

Token	Value stack (top on the right)
2	2
3	2 3
1	2 3 1
-	2 2
*	4
4	4 4
+	**8**

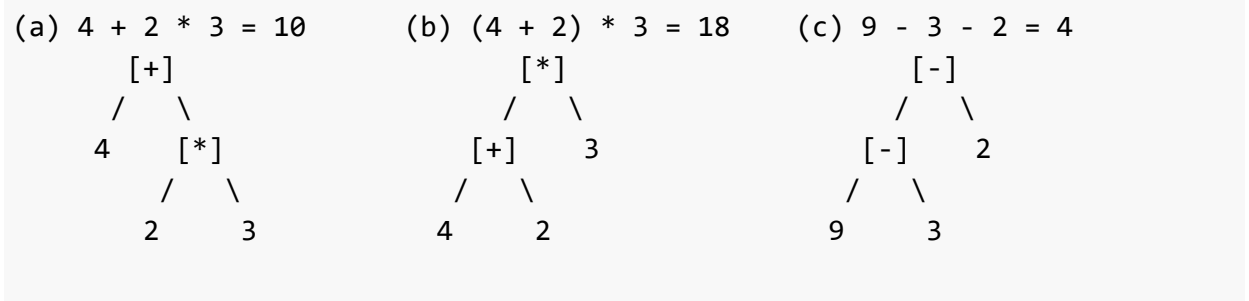
The value is 8 (an `int` here; the tree route gives `8.0`). The postfix list is the post-order of the tree of W15-T2: `2`, then the subtree `3 1 -`, then `*`, then `4`, then the root `+`.

W15-T5.

Input	Result
`"2 * -3"`	`-6.0` — unary minus is a factor, so it may follow *
`"--4"`	`4.0` — `factor` calls itself: `0 - (0 - 4)`
`"4 -"`	`ValueError` from the **parser**: unexpected end of input
`"(2 + 3) (4)"`	`ValueError` from the **parser**: unexpected (after the expression — there is no implicit multiplication
`"6 / (3 - 3)"`	`ZeroDivisionError` from the **evaluator**
`" "`	`ValueError` from the **parser**: the lexer returns `[]`, and `parse` rejects an empty expression
`"2 ^ 3"`	`ValueError` from the **lexer**: unexpected character ^ at position 2
`"((2)"`	`ValueError` from the **parser**: missing)

Part D — Trees and stacks: draw every step

W15-S1. Operators in brackets.



```
(d) -3 * -2 = 6                (e) 12 / 2 / 3 * 4 = 8
          [*]                                [*]
        /     \                            /   \
     [-]       [-]                       [/]    4
    /   \     /   \                     /   \
   0     3   0     2                  [/]    3
                                      /   \
                                    12     2
```

A right-recursive `expr` builds, for (c):

```
    [-]
   /   \
  9    [-]          9 - (3 - 2) = 8, not 4
      /   \
     3     2
```

W15-S2. `10 - 2 + 3 - 1`:

```
after term():   after 1st turn:   after 2nd turn:     after 3rd turn:
                                                            [-]
                                        [+]                /   \
                    [-]                /   \             [+]    1
   10              /   \             [-]    3           /   \
                  10     2          /   \             [-]    3
                                   10     2           /   \
                                                    10     2
```

Each turn makes the old tree the **left** child of the new operator. Final value: `((10 - 2) + 3) - 1 = 10`.

W15-S3. Postfix: `1 2 + 3 4 - *`.

Token	Value stack (top on the right)
1	1
2	1 2
+	3
3	3 3
4	3 3 4
-	3 -1
*	**-3**

The AST: * at the root, +(1, 2) on the left, -(3, 4) on the right. Post-order numbering: 1 → 1, 2 → 2, + → 3, 3 → 4, 4 → 5, - → 6, * → 7. **The post-order sequence is the postfix list**, and the values on the stack after each operator are the values `evaluate` returns at the same node: 3 at +, -1 at -, -3 at *. The stack holds exactly the values of the subtrees finished but not yet used.

Part E — Complexity analysis

W15-K1. Our `tokenize`: $\Theta(n)$. The index moves forward on every step of either loop and never back, so there are at most n steps of $O(1)$ work; each number is cut out once, with a slice as long as the number, so the slices total at most n characters. The `text = text[1:]` version: $\Theta(n^2)$. Each slice copies the rest of the string — n - 1, then n - 2, … characters — and the sum is about $n^2/2$. Strings are immutable in Python, so "removing the first character" always means copying the others. (The same trap as slicing in recursion, Lecture 03 and W8-K2.)

W15-K2.

Input	(a) parser depth	(b) `evaluate` depth
k nested pairs around `1`	about **3k** — `expr`, `term`, `factor` per pair	1 — the tree is a single `Num`
`1 + 1 + … + 1`, k numbers	constant — `expr`'s loop	about **k** — the tree is left-deep, height k - 1
balanced, k numbers	about $3 \log_2 k$	about $\log_2 k$

For k = 1,000, two fail: the nested parentheses in the **parser** (3,000 frames; measured, it already fails at 330 pairs), and the flat chain in **`evaluate`** (about 1,000 frames; measured, the longest chain that works from a script has 997 numbers). The balanced one needs about 40 frames in all. Time is $O(n)$ in every case; depth is set by the **height** of the tree, as in Week 11.

W15-K3.

- **Concatenation:** each + of lists builds a new list, copying both halves. A node whose subtrees hold L and R tokens costs $\Theta(L + R)$. For a **left-deep** tree (a chain like `1 + 1 + … + 1`), the left list grows by 2 at every level and is copied again at every level: $3 + 5 + 7 + \ldots$ — $\Theta(n^2)$. Measured: for 1,000 numbers (1,999 nodes) the concatenations copy 999,999 elements, to produce a list of 1,999.
- **One shared list:** each node appends one token, amortised $O(1)$ (Week 4): $\Theta(n)$.
- On a balanced tree the concatenating version is $\Theta(n \log n)$ — each token is copied once per level above it.

Part F — Find and fix the bug

W15-B1. Right recursion. The recursive call parses the whole rest of the line and hangs it on the **right**, so chains of - group the wrong way: `10 - 4 - 3` gives `10 - (4 - 3) = 9` instead of 3, and `5 - 1 + 2` gives `5 - (1 + 2) = 2` instead of 6. No exception is raised, and chains of + alone still give the right value (`1 + 2 + 3` is 6 either way), which is why the bug survives casual testing. **Fix:** a loop that makes the tree so far the left child:

```
def expr(self):
    node = self.term()
    while self.peek() in ("+", "-"):
        op = self.advance()
        node = BinOp(op, node, self.term())
    return node
```

W15-B2. The `)` is **checked but never consumed**. `factor` returns with `position` still on the `)`; the enclosing `term` and `expr` see a token they cannot use and return, and `parse` rejects it: every parenthesised expression — `(1 + 2) * 3`, `(5)`, `2 * (3)` — raises `ValueError: unexpected ')' after the expression`. Expressions without parentheses still work. **Fix:** `self.advance()` after the check, before `return node`.

W15-B3. The operand of unary minus is parsed with `expr`, so the minus swallows the **whole rest** of the expression: `-2 + 5` becomes `-(2 + 5) = -7` instead of 3, and `2 * -3 + 1` becomes `2 * -(3 + 1) = -8` instead of -5. It happens to be right for `-5` and `-2 * 3` (both -6 either way), which is why only the test for `-5 + 2` catches it. **Fix:** `self.factor()` — unary minus binds as tightly as a number.

W15-B4. The inner loop reads `text[i]` without checking `i < len(text)` first. A number at the very **end** of the text runs the index off the end: `tokenize("3 + 4")` and `tokenize("12")` raise `IndexError: string index out of range`, while `"3 + 4 "` (trailing space) and `"(3)"` work — so the bug looks random. **Fix:** `while i < len(text) and (text[i] in DIGITS or text[i] == "."):` — the bound first, so `and` stops before the read. A second, quieter bug: nothing counts the decimal points, so `"1..2 "` becomes the token `'1..2'` and the error appears later, as a failed `float` conversion in the parser; count the points and raise in the lexer.

Part G — Write the code

W15-C1

```
def to_postfix(node):
    out = []

    def walk(current):
        if isinstance(current, Num):
            out.append(number_text(current.value))
        else:
            walk(current.left)          # left, right ...
            walk(current.right)
            out.append(current.op)      # ... then the node

    walk(node)
    return out
```

The walk visits the left subtree, then the right, then the node, appending as it goes: post-order. `infix_to_postfix` produces the same list because the shunting-yard algorithm outputs an operator exactly when both of its operands are complete — when its subtree is finished — which is when a post-order walk reaches it (`test_to_postfix_agrees_with_the_stack_route` checks five expressions). One shared list, so $\Theta(n)$; building the result by list concatenation would be $\Theta(n^2)$ on a left-deep tree (W15-K3).

W15-C2

```
def to_infix(node):
    if isinstance(node, Num):
        return number_text(node.value)
    return f"({to_infix(node.left)} {node.op} {to_infix(node.right)})"
```

One pair of parentheses per operator, never around a lone number. The output is unambiguous without any knowledge of precedence — which is exactly why it is ugly: `((1 - 2) - 3)`.

W15-C3

```
def to_infix_minimal(node):
    if isinstance(node, Num):
        return number_text(node.value)
    mine = PRECEDENCE[node.op]
    left = to_infix_minimal(node.left)
    right = to_infix_minimal(node.right)
    if isinstance(node.left, BinOp) and PRECEDENCE[node.left.op] < mine:
        left = f"({left})"
    if isinstance(node.right, BinOp) and PRECEDENCE[node.right.op] <= mine:
        right = f"({right})"            # equal too: trees lean left
    return f"{left} {node.op} {right}"
```

When a child needs parentheses: when its operator binds **more loosely** than its parent's (a `+` under a `*`), and — for the **right** child only — also when it binds **equally** (a `-` under a `-`). The grammar builds left-leaning trees, so an unparenthesised `1 - 2 - 3` already means the left child; a right child of the same level can only be expressed with parentheses. That includes `1 + (2 + 3)`: mathematically the parentheses could go, but without them the text parses to a **different tree**, and the round-trip test (`test_to_infix_minimal_round_trips`, 300 random trees) demands the same one. A left child of equal precedence never needs them: `2 * 3 * (4 + 5)`.

W15-C4

```
def postfix_to_tree(tokens):
    trees = Stack()
    for token in tokens:
        if token in PRECEDENCE:
            if len(trees) < 2:
                raise ValueError(f"operator {token!r} needs two operands")
            right = trees.pop()         # popped first: the RIGHT one
```

```
            left = trees.pop()
            trees.push(BinOp(token, left, right))
        else:
            trees.push(Num(float(token)))
    if len(trees) != 1:
        raise ValueError("malformed postfix expression")
    return trees.pop()
```

`evaluate_postfix` with trees in place of numbers: a number becomes a leaf and is pushed; an operator pops **two** subtrees — the first popped is the **right** one — and pushes the tree that joins them. Exactly one tree must be left. Uses your `Stack`, as the storage rule asks. $\Theta(n)$, and `postfix_to_tree(to_postfix(t)) == t` for any tree (tested on 300 random ones).

W15-C5

```
class _PowerParser:

    def __init__(self, tokens):
        self.tokens = Array.from_values(tokens)
        self.position = 0

    def peek(self):
        if self.position < len(self.tokens):
            return self.tokens[self.position]
        return None

    def advance(self):
        token = self.peek()
        self.position += 1
        return token

    def expr(self):
        value = self.term()
        while self.peek() in ("+", "-"):
            if self.advance() == "+":
                value = value + self.term()
            else:
                value = value - self.term()
        return value

    def term(self):
        value = self.unary()
        while self.peek() in ("*", "/"):
            if self.advance() == "*":
                value = value * self.unary()
            else:
```

```
                value = value / self.unary()
        return value

    def unary(self):
        if self.peek() == "-":
            self.advance()
            return -self.unary()
        return self.power()

    def power(self):
        base = self.atom()
        if self.peek() == "^":
            self.advance()
            return base ** self.unary()         # recursion: leans RIGHT
        return base

    def atom(self):
        token = self.advance()
        if token is None:
            raise ValueError("unexpected end of input")
        if token == "(":
            value = self.expr()
            if self.advance() != ")":
                raise ValueError("missing ')'")
            return value
        if token[0] in "0123456789.":
            return float(token)
        raise ValueError(f"unexpected {token!r}")


def calculate_with_power(tokens):
    parser = _PowerParser(tokens)
    if parser.peek() is None:
        raise ValueError("empty expression")
    value = parser.expr()
    if parser.peek() is not None:
        left_over = parser.peek()
        raise ValueError(f"unexpected {left_over!r} after the expression")
    return value
```

Why recursion for ^: power parses one atom and then, if a ^ follows, calls unary for **everything to its right** — that call can itself reach power and take the next ^. So `2 ^ 3 ^ 2` becomes `2 ^ (3 ^ 2)`: the recursive result goes on the right, and the tree leans **right**. This is the "right recursion" of W15-B1 — a bug for -, but exactly the right shape for a right-associative operator. + and * use loops because they lean left.

Precedence: `unary` sits **above** `power`, so in `-2 ^ 2` the minus applies to the whole power: $-(2^2) = -4$. And the right operand of `^` is a `unary`, so `2 ^ -1` is allowed and gives 0.5. Both match Python's `**` (`test_calculate_with_power_matches_python`). This version computes values as it parses rather than building a tree — a translator may do either — and keeps its tokens in the course `Array`.

Mock Exam — Weeks 1 to 3 — practice paper

> ***How to use this paper.*** *Sit it in one go, closed book, with no computer. The questions are taken from the weekly question banks, so the model answers are in* Answers — Week 1, Answers — Week 2 *and* Answers — Week 3 *— look them up* ***only after*** *you have finished and marked yourself. Its format follows the real final:* ***30 marks of multiple choice, 30 marks of written answers.*** *It covers only weeks 1–3; the real exam covers the whole course.*

Section A — Multiple choice (30 marks)

One mark each. Choose one answer out of four. Write the letters on your answer sheet.

#	Question	#	Question	#	Question
1	W1-M02	11	W2-M02	21	W3-M02
2	W1-M06	12	W2-M03	22	W3-M04
3	W1-M10	13	W2-M06	23	W3-M06
4	W1-M11	14	W2-M07	24	W3-M08
5	W1-M15	15	W2-M11	25	W3-M09
6	W1-M17	16	W2-M12	26	W3-M12
7	W1-M19	17	W2-M16	27	W3-M15
8	W1-M21	18	W2-M18	28	W3-M16
9	W1-M23	19	W2-M21	29	W3-M17
10	W1-M24	20	W2-M23	30	W3-M22

The questions are in the *Week 1* questions, the *Week 2* questions and the *Week 3* questions.

Section B — Written answers (30 marks)

Answer **all** questions. Marks are shown in brackets.

#	Question	Topic	Marks
B1	W1-E4	ADT against data structure, with the Stack	4
B2	W2-E4	why array indexing is O(1) and insertion O(n); fixed size	4

#	Question	Topic	Marks
B3	W2-P3	prove $4n^2 + n = \Theta(n^2)$	4
B4	W2-K3 **and** W2-K5	complexity analysis, 2 marks each	4
B5	W2-S2	the forward-shifting insertion bug, traced	3
B6	W3-E2	naive against memoised Fibonacci	4
B7	W3-R3	recurrence, solution and stack depth of `range_sum`	4
B8	W3-T3	trace `gcd(48, 18)` and its call stack	3
		Total	**30**

Marking yourself

- **Section A:** one mark per correct letter. The answer tables are at the top of each weekly answers file.
- **Section B:** use the **marking points** in the answers files. Award a mark for each point your answer makes clearly and correctly — not for length.
- **Then** look at *which* questions you lost marks on, and on which level ([what], [how] or [why]). That tells you what to revise, and how.

Mock Exam — Weeks 1 to 7 — practice paper

> ***How to use this paper.*** *Sit it in one go, closed book, with no computer. It covers weeks 1–7 — everything before the midterm window — and leans on weeks 4–7, since the weeks 1–3 mock already covers the first three. The questions are taken from the weekly question banks, and* ***none*** *of them repeats the weeks 1–3 mock, so you can sit both. The model answers are in the weekly answers files — look them up* ***only after*** *you have finished and marked yourself.*
>
> *The format follows the final:* ***30 marks of multiple choice, 30 marks of written answers.*** *The midterm's own format and date will be announced in the lecture and on the WhatsApp channel.*

Section A — Multiple choice (30 marks)

One mark each. Choose one answer out of four. Write the letters on your answer sheet.

#	Question	#	Question	#	Question
1	W1-M09	11	W3-M11	21	W6-M03
2	W1-M13	12	W3-M19	22	W6-M08
3	W1-M18	13	W4-M03	23	W6-M11
4	W2-M01	14	W4-M07	24	W6-M14
5	W2-M05	15	W4-M10	25	W6-M19
6	W2-M09	16	W4-M14	26	W7-M03
7	W2-M14	17	W4-M18	27	W7-M05
8	W2-M20	18	W5-M04	28	W7-M09
9	W3-M03	19	W5-M09	29	W7-M12
10	W3-M07	20	W5-M17	30	W5-M13

The questions are in the weekly files: week 1 · week 2 · week 3 · week 4 · week 5 · week 6 · week 7.

Section B — Written answers (30 marks)

Answer **all** questions. Marks are shown in brackets.

#	Question	Topic	Marks
B1	W2-E5	the storage rule: what, why, and what remains allowed	4
B2	W2-P2	prove n^2 is not $O(n)$	3

#	Question	Topic	Marks
B3	W3-E4	Towers of Hanoi: strategy, recurrence, optimality	4
B4	W4-E1	amortised cost; n appends with doubling are $O(n)$	5
B5	W5-S1	a linked list drawn after every operation	3
B6	W6-E4	shunting-yard, traced on `2 * ( 3 + 4 ) - 5`	4
B7	W7-S1	a ring buffer drawn after every operation	3
B8	W5-K3	which end of a linked list is the front of a queue	2
B9	W6-B3 **and** W7-B3	two bugs, 1 mark each	2
		Total	**30**

Coverage. Section A has three questions from week 1, five from week 2, four from week 3 and 18 from weeks 4–7; Section B gives 11 marks to weeks 2–3 and 19 to weeks 4–7. Searching (week 8) is **not** on this paper; it is on the final.

Marking yourself

- **Section A:** one mark per correct letter. The answer tables are at the top of each weekly answers file.
- **Section B:** use the **marking points** in the answers files. Award a mark for each point your answer makes clearly and correctly — not for length. For B5 and B7, a mark per correct third of the drawing; for B8, one mark for the choice of ends and one for the costs.
- **Then** look at *which* questions you lost marks on, and on which level ([what], [how] or [why]). That tells you what to revise, and how — and the weekly question bank of that week has more of the same kind.

Mock Exam — Weeks 1 to 15 — practice paper

> ***How to use this paper.*** *Sit it in one go, closed book, with no computer, in the time the final gives you. It covers the whole course and leans on weeks 8–15, since the weeks 1–3 and weeks 1–7 mocks already cover the first half. The questions are taken from the weekly question banks, and* ***none*** *of them repeats either earlier mock, so you can sit all three. The model answers are in the weekly answers files — look them up* ***only after*** *you have finished and marked yourself.*
>
> *The format follows the final:* ***30 marks of multiple choice, 30 marks of written answers.*** *The date and room are announced in the lecture and on the WhatsApp channel.*

Section A — Multiple choice (30 marks)

One mark each. Choose one answer out of four. Write the letters on your answer sheet.

#	Question	#	Question	#	Question
1	W1-M03	11	W9-M10	21	W12-M17
2	W2-M04	12	W9-M17	22	W13-M02
3	W3-M05	13	W10-M02	23	W13-M10
4	W4-M05	14	W10-M10	24	W13-M17
5	W5-M06	15	W10-M17	25	W14-M02
6	W6-M06	16	W11-M02	26	W14-M10
7	W7-M07	17	W11-M10	27	W14-M17
8	W8-M04	18	W11-M17	28	W15-M02
9	W8-M12	19	W12-M02	29	W15-M10
10	W9-M02	20	W12-M10	30	W15-M17

The questions are in the weekly files, the *Week 1* questions to the *Week 15* questions — in the students' book, at the end of each week's chapter.

Section B — Written answers (30 marks)

Answer **all** questions. Marks are shown in brackets.

#	Question	Topic	Marks
B1	W8-E3	`lower_bound` and `upper_bound`: definitions and uses	4
B2	W9-T3	insertion sort traced, the array after every pass	2

#	Question	Topic	Marks
B3	W10-E1	merge sort as divide and conquer; its recurrence	5
B4	W11-S1	a binary search tree drawn after ten insertions	3
B5	W12-T1	a min-heap after every push and pop	3
B6	W13-E4	tombstones in open addressing	4
B7	W14-T1	breadth-first search traced, queue and visited	3
B8	W15-E2	precedence is the shape of the grammar	4
B9	W10-B1 **and** W13-B3	two bugs, 1 mark each	2
		Total	**30**

Coverage. Section A has seven questions from weeks 1–7 and 23 from weeks 8–15; Section B is entirely weeks 8–15, because the two earlier mocks give you written practice on weeks 1–7. The real final is not bound by that split: revise the first half from those two papers.

Marking yourself

- **Section A:** one mark per correct letter. The answer tables are at the top of each weekly answers file.
- **Section B:** use the **marking points** in the answers files. Award a mark for each point your answer makes clearly and correctly — not for length. For the traces and drawings (B2, B4, B5, B7), divide the marks evenly over the steps and award them for the steps you got exactly right.
- **Then** look at *which* questions you lost marks on, and at which level ([what], [how] or [why]). That tells you what to revise, and how — and the weekly question bank of that week has more of the same kind.
- **Remember the rule that fails students:** you need **≥ 30% of the final** as well as ≥ 60% overall. Coursework cannot rescue a final below 30%.